DARA Studio - Complete Code Export

Total files: 145

Table of Contents

src/App.tsx (1 files)
src/main.tsx (1 files)
src/index.css (1 files)
src/App.css (1 files)
src/vite-env.d.ts (1 files)
tailwind.config.ts (1 files)
vite.config.ts (1 files)
index.html (1 files)
components.json (1 files)
supabase/config.toml (1 files)
src/integrations/supabase (1 files)
src/pages/admin (26 files)
src/pages/dashboard (10 files)
src/pages/freelancer (7 files)
src/pages/public (5 files)
src/pages/auth (2 files)
src/components/admin (17 files)
src/components/dashboard (5 files)
src/components/freelancer (2 files)
src/components/home (4 files)
src/components/how-we-work (1 files)
src/components/layout (4 files)
src/components/estimate (15 files)
src/components/orcamento (5 files)
src/components/profile (1 files)
src/hooks/use-mobile.tsx (1 files)
src/hooks/use-toast.ts (1 files)
src/hooks/useAdminRole.tsx (1 files)
src/hooks/useAuth.tsx (1 files)
src/hooks/useFreelancerRole.tsx (1 files)
src/hooks/useLanguage.tsx (1 files)
src/i18n/translations.ts (1 files)
src/modules/clients (1 files)
src/modules/estimates (1 files)
src/modules/freelancers (1 files)
src/modules/payments (1 files)
src/modules/projects (1 files)
src/modules/proposals (1 files)
src/services/estimateService.ts (1 files)
src/services/index.ts (1 files)
src/services/paymentService.ts (1 files)
src/services/projectService.ts (1 files)
src/services/supabase (6 files)
src/constants/projectStages.ts (1 files)
src/utils/countryFlags.ts (1 files)
src/utils/index.ts (1 files)
src/types/index.ts (1 files)
supabase/functions/create-client-account (1 files)
src/components/LanguageSelector.tsx (1 files)
src/components/NavLink.tsx (1 files)

src/App.tsx

src/App.tsx

```
import { Toaster } from "@/components/ui/toaster";
import { Toaster as Sonner } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import { AuthProvider } from "@/hooks/useAuth";
import { LanguageProvider } from "@/hooks/useLanguage";

/* LAYOUT */
import MainLayout from "./components/layout/MainLayout";

/* PUBLIC PAGES */
import Home from "./pages/public/Home";
import HowWeWork from "./pages/public/HowWeWork";
import EstimateRequest from "./pages/public/EstimateRequest";
import ThankYou from "./pages/public/ThankYou";
import NotFound from "./pages/public/NotFound";

/* AUTH */
import Login from "./pages/auth/Login";
import FreelancerLogin from "./pages/auth/FreelancerLogin";
import AdminLogin from "./pages/admin/AdminLogin";

/* DASHBOARD (Client) */
import DashboardLayout from "./components/dashboard/DashboardLayout";
import DashboardHome from "./pages/dashboard/DashboardHome";
import Projects from "./pages/dashboard/Projects";
import ClientProjectDetail from "./pages/dashboard/ClientProjectDetail";
import Invoices from "./pages/dashboard/Invoices";
import Files from "./pages/dashboard/Files";
import Messages from "./pages/dashboard/Messages";
import Profile from "./pages/dashboard/Profile";
import Company from "./pages/dashboard/Company";
import NewQuote from "./pages/dashboard/NewQuote";

/* FREELANCER WORKSPACE */
import FreelancerLayout from "./components/freelancer/FreelancerLayout";
import FreelancerHome from "./pages/freelancer/FreelancerHome";
import FreelancerProjects from "./pages/freelancer/FreelancerProjects";
import FreelancerPayments from "./pages/freelancer/FreelancerPayments";
import FreelancerContracts from "./pages/freelancer/FreelancerContracts";
import FreelancerCalendar from "./pages/freelancer/FreelancerCalendar";
import FreelancerProfile from "./pages/freelancer/FreelancerProfile";

/* ADMIN PANEL */
import AdminLayout from "./components/admin/AdminLayout";
import AdminDashboard from "./pages/admin/AdminDashboard";
import AdminCompanies from "./pages/admin/AdminCompanies";
import AdminProjects from "./pages/admin/AdminProjects";
import AdminProjectDetail from "./pages/admin/AdminProjectDetail";
import AdminPayments from "./pages/admin/AdminPayments";
import AdminMilestones from "./pages/admin/AdminMilestones";
import AdminActivity from "./pages/admin/AdminActivity";
import AdminMap from "./pages/admin/AdminMap";
import AdminGantt from "./pages/admin/AdminGantt";
import AdminFreelancers from "./pages/admin/AdminFreelancers";
import AdminTickets from "./pages/admin/AdminTickets";
import AdminDrawings from "./pages/admin/AdminDrawings";
import AdminLeads from "./pages/admin/AdminLeads";
import AdminProposals from "./pages/admin/AdminProposals";
import AdminContracts from "./pages/admin/AdminContracts";
import AdminFiles from "./pages/admin/AdminFiles";
import AdminPermits from "./pages/admin/AdminPermits";
import AdminAICodeCheck from "./pages/admin/AdminAICodeCheck";
import AdminClientPortal from "./pages/admin/AdminClientPortal";
import AdminAnalytics from "./pages/admin/AdminAnalytics";
import AdminSettings from "./pages/admin/AdminSettings";
import AdminClientProfile from "./pages/admin/AdminClientProfile";
import AdminFinanceOverview from "./pages/admin/AdminFinanceOverview";
import AdminSiteUpdates from "./pages/admin/AdminSiteUpdates";

const queryClient = new QueryClient();

const App = () =&gt; (
  &lt;QueryClientProvider client={queryClient}&gt;
    &lt;LanguageProvider&gt;
      &lt;TooltipProvider&gt;
        &lt;Toaster /&gt;
        &lt;Sonner /&gt;
        &lt;BrowserRouter&gt;
          &lt;AuthProvider&gt;
            &lt;MainLayout&gt;
              &lt;Routes&gt;
                {/* PUBLIC WEBSITE */}
                &lt;Route path="/" element={&lt;Home /&gt;} /&gt;
                &lt;Route path="/how-we-work" element={&lt;HowWeWork /&gt;} /&gt;
                &lt;Route path="/estimate-request" element={&lt;EstimateRequest /&gt;} /&gt;
                &lt;Route path="/orcamento" element={&lt;EstimateRequest /&gt;} /&gt;
                &lt;Route path="/thank-you" element={&lt;ThankYou /&gt;} /&gt;

                {/* AUTH */}
                &lt;Route path="/login" element={&lt;Login /&gt;} /&gt;
                &lt;Route path="/auth" element={&lt;Login /&gt;} /&gt;
```

```
                &lt;Route path="/freelancer-login" element={&lt;FreelancerLogin /&gt;} /&gt;
                &lt;Route path="/freelancer" element={&lt;FreelancerLogin /&gt;} /&gt;

                {/* ADMIN LOGIN */}
                &lt;Route path="/adm/login" element={&lt;AdminLogin /&gt;} /&gt;

                {/* DASHBOARD (Client - Protected) */}
                &lt;Route path="/dashboard" element={&lt;DashboardLayout /&gt;}&gt;
                  &lt;Route index element={&lt;DashboardHome /&gt;} /&gt;
                  &lt;Route path="projects" element={&lt;Projects /&gt;} /&gt;
                  &lt;Route path="projects/:id" element={&lt;ClientProjectDetail /&gt;} /&gt;
                  &lt;Route path="invoices" element={&lt;Invoices /&gt;} /&gt;
                  &lt;Route path="files" element={&lt;Files /&gt;} /&gt;
                  &lt;Route path="messages" element={&lt;Messages /&gt;} /&gt;
                  &lt;Route path="profile" element={&lt;Profile /&gt;} /&gt;
                  &lt;Route path="company" element={&lt;Company /&gt;} /&gt;
                  &lt;Route path="new-quote" element={&lt;NewQuote /&gt;} /&gt;
                &lt;/Route&gt;

                {/* FREELANCER WORKSPACE (Protected) */}
                &lt;Route path="/work" element={&lt;FreelancerLayout /&gt;}&gt;
                  &lt;Route index element={&lt;FreelancerHome /&gt;} /&gt;
                  &lt;Route path="projects" element={&lt;FreelancerProjects /&gt;} /&gt;
                  &lt;Route path="payments" element={&lt;FreelancerPayments /&gt;} /&gt;
                  &lt;Route path="contracts" element={&lt;FreelancerContracts /&gt;} /&gt;
                  &lt;Route path="calendar" element={&lt;FreelancerCalendar /&gt;} /&gt;
                  &lt;Route path="profile" element={&lt;FreelancerProfile /&gt;} /&gt;
                &lt;/Route&gt;

                {/* ADMIN PANEL (Protected) */}
                &lt;Route path="/adm" element={&lt;AdminLayout /&gt;}&gt;
                  &lt;Route index element={&lt;AdminDashboard /&gt;} /&gt;
                  &lt;Route path="companies" element={&lt;AdminCompanies /&gt;} /&gt;
                  &lt;Route path="clients/:id" element={&lt;AdminClientProfile /&gt;} /&gt;
                  &lt;Route path="projects" element={&lt;AdminProjects /&gt;} /&gt;
                  &lt;Route path="projects/:id" element={&lt;AdminProjectDetail /&gt;} /&gt;
                  &lt;Route path="gantt" element={&lt;AdminGantt /&gt;} /&gt;
                  &lt;Route path="map" element={&lt;AdminMap /&gt;} /&gt;
                  &lt;Route path="payments" element={&lt;AdminPayments /&gt;} /&gt;
                  &lt;Route path="finance-overview" element={&lt;AdminFinanceOverview /&gt;} /&gt;
                  &lt;Route path="milestones" element={&lt;AdminMilestones /&gt;} /&gt;
                  &lt;Route path="activity" element={&lt;AdminActivity /&gt;} /&gt;
                  &lt;Route path="freelancers" element={&lt;AdminFreelancers /&gt;} /&gt;
                  &lt;Route path="tickets" element={&lt;AdminTickets /&gt;} /&gt;
                  &lt;Route path="drawings" element={&lt;AdminDrawings /&gt;} /&gt;
                  &lt;Route path="leads" element={&lt;AdminLeads /&gt;} /&gt;
                  &lt;Route path="proposals" element={&lt;AdminProposals /&gt;} /&gt;
                  &lt;Route path="contracts" element={&lt;AdminContracts /&gt;} /&gt;
                  &lt;Route path="files" element={&lt;AdminFiles /&gt;} /&gt;
                  &lt;Route path="permits" element={&lt;AdminPermits /&gt;} /&gt;
                  &lt;Route path="site-updates" element={&lt;AdminSiteUpdates /&gt;} /&gt;
                  &lt;Route path="ai-code-check" element={&lt;AdminAICodeCheck /&gt;} /&gt;
                  &lt;Route path="client-portal" element={&lt;AdminClientPortal /&gt;} /&gt;
                  &lt;Route path="analytics" element={&lt;AdminAnalytics /&gt;} /&gt;
                  &lt;Route path="settings" element={&lt;AdminSettings /&gt;} /&gt;
                &lt;/Route&gt;

                {/* FALLBACK */}
                &lt;Route path="*" element={&lt;NotFound /&gt;} /&gt;
              &lt;/Routes&gt;
            &lt;/MainLayout&gt;
          &lt;/AuthProvider&gt;
        &lt;/BrowserRouter&gt;
      &lt;/TooltipProvider&gt;
    &lt;/LanguageProvider&gt;
  &lt;/QueryClientProvider&gt;
);

export default App;
```

src/main.tsx

src/main.tsx

```
import { createRoot } from "react-dom/client";
import App from "./App.tsx";
import "./index.css";

createRoot(document.getElementById("root")!).render(&lt;App /&gt;);
```

src/index.css

src/index.css

```
@import url("https://fonts.googleapis.com/css2?family=Roboto&amp;display=swap");
@import url("https://fonts.googleapis.com/css2?family=Libre+Caslon+Text&amp;display=swap");
@import url("https://fonts.googleapis.com/css2?family=Roboto+Mono&amp;display=swap");
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Definition of the design system. All colors, gradients, fonts, etc should be defined here.
All colors MUST be HSL.
*/

@layer base {
  :root {
    --background: 0 0% 96%;
    --foreground: 0 0% 9%;

    --card: 0 0% 98%;
    --card-foreground: 0 0% 9%;

    --popover: 0 0% 89%;
    --popover-foreground: 0 0% 9%;

    --primary: 42 55% 44%;
    --primary-foreground: 33 100% 96%;

    --secondary: 0 0% 32%;
    --secondary-foreground: 0 0% 98%;

    --muted: 0 0% 63%;
    --muted-foreground: 0 0% 9%;

    --accent: 47 100% 96%;
    --accent-foreground: 42 55% 44%;

    --destructive: 0 72% 50%;
    --destructive-foreground: 0 85% 97%;

    --border: 0 0% 83%;
    --input: 0 0% 83%;
    --ring: 42 55% 44%;

    --radius: 0.75rem;

    --sidebar-background: 0 0% 98%;

    --sidebar-foreground: 0 0% 9%;

    --sidebar-primary: 20 90% 48%;

    --sidebar-primary-foreground: 33 100% 96%;

    --sidebar-accent: 0 0% 32%;

    --sidebar-accent-foreground: 0 0% 98%;

    --sidebar-border: 0 0% 83%;

    --sidebar-ring: 20 90% 48%;

    --chart-1: 42 55% 44%;

    --chart-2: 43 96% 56%;

    --chart-3: 47 95% 53%;

    --chart-4: 42 55% 44%;

    --chart-5: 0 0% 45%;

    --sidebar: 0 0% 98%;

    --font-sans: 'Roboto', ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Helvetica Neue', Arial, 'Noto Sans', sans-serif;

    --font-serif: 'Libre Caslon Text', ui-serif, Georgia, Cambria, 'Times New Roman', Times, serif;

    --font-mono: 'Roboto Mono', ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, 'Liberation Mono', 'Courier New', monospace;

    --shadow-2xs: 0 1px 3px 0px hsl(0 0% 0% / 0.05);

    --shadow-xs: 0 1px 3px 0px hsl(0 0% 0% / 0.05);

    --shadow-sm: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 1px 2px -1px hsl(0 0% 0% / 0.1);

    --shadow: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 1px 2px -1px hsl(0 0% 0% / 0.1);

    --shadow-md: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 2px 4px -1px hsl(0 0% 0% / 0.1);

    --shadow-lg: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 4px 6px -1px hsl(0 0% 0% / 0.1);

    --shadow-xl: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 8px 10px -1px hsl(0 0% 0% / 0.1);

    --shadow-2xl: 0 1px 3px 0px hsl(0 0% 0% / 0.25);
    --tracking-normal: 0em;
```

```
    --spacing: 0.25rem;
  }

  .dark {
    --background: 0 0% 9%;
    --foreground: 0 0% 98%;

    --card: 0 0% 14%;
    --card-foreground: 0 0% 98%;

    --popover: 0 0% 25%;
    --popover-foreground: 0 0% 98%;

    --primary: 27 95% 60%;
    --primary-foreground: 12 81% 14%;

    --secondary: 0 0% 45%;
    --secondary-foreground: 0 0% 98%;

    --muted: 0 0% 45%;
    --muted-foreground: 0 0% 98%;

    --accent: 20 91% 14%;
    --accent-foreground: 43 96% 56%;

    --destructive: 0 84% 60%;
    --destructive-foreground: 0 85% 97%;

    --border: 0 0% 32%;
    --input: 0 0% 32%;
    --ring: 27 95% 60%;
    --sidebar-background: 240 5.9% 10%;
    --sidebar-foreground: 0 0% 98%;
    --sidebar-primary: 27 95% 60%;
    --sidebar-primary-foreground: 12 81% 14%;
    --sidebar-accent: 43 96% 56%;
    --sidebar-accent-foreground: 20 91% 14%;
    --sidebar-border: 0 0% 32%;
    --sidebar-ring: 27 95% 60%;
    --chart-1: 30 97% 72%;
    --chart-2: 45 96% 64%;
    --chart-3: 48 96% 76%;
    --chart-4: 27 95% 60%;
    --chart-5: 0 0% 45%;
    --sidebar: 0 0% 14%;
    --shadow-2xs: 0 1px 3px 0px hsl(0 0% 0% / 0.05);
    --shadow-xs: 0 1px 3px 0px hsl(0 0% 0% / 0.05);
    --shadow-sm: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 1px 2px -1px hsl(0 0% 0% / 0.1);
    --shadow: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 1px 2px -1px hsl(0 0% 0% / 0.1);
    --shadow-md: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 2px 4px -1px hsl(0 0% 0% / 0.1);
    --shadow-lg: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 4px 6px -1px hsl(0 0% 0% / 0.1);
    --shadow-xl: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 8px 10px -1px hsl(0 0% 0% / 0.1);
    --shadow-2xl: 0 1px 3px 0px hsl(0 0% 0% / 0.25);
  }
}

@layer base {
  * {
    @apply border-border;
  }

  body {
    @apply bg-background text-foreground;
  }
}
```

src/App.css

src/App.css

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}

@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}

.read-the-docs {
  color: #888;
}
```

src/vite-env.d.ts

src/vite-env.d.ts

```
/// &lt;reference types="vite/client" /&gt;
```

tailwind.config.ts

tailwind.config.ts

```
import type { Config } from "tailwindcss";

export default {
  darkMode: ["class"],
  content: ["./pages/**/*.{ts,tsx}", "./components/**/*.{ts,tsx}", "./app/**/*.{ts,tsx}", "./src/**/*.{ts,tsx}"],
  prefix: "",
  theme: {
  container: {
  center: true,
  padding: '2rem',
  screens: {
  '2xl': '1400px'
  }
  },
  extend: {
  colors: {
  border: 'hsl(var(--border))',
  input: 'hsl(var(--input))',
  ring: 'hsl(var(--ring))',
  background: 'hsl(var(--background))',
  foreground: 'hsl(var(--foreground))',
  primary: {
  DEFAULT: 'hsl(var(--primary))',
  foreground: 'hsl(var(--primary-foreground))'
  },
  secondary: {
  DEFAULT: 'hsl(var(--secondary))',
  foreground: 'hsl(var(--secondary-foreground))'
  },
  destructive: {
  DEFAULT: 'hsl(var(--destructive))',
  foreground: 'hsl(var(--destructive-foreground))'
  },
  muted: {
  DEFAULT: 'hsl(var(--muted))',
  foreground: 'hsl(var(--muted-foreground))'
  },
  accent: {
  DEFAULT: 'hsl(var(--accent))',
  foreground: 'hsl(var(--accent-foreground))'
  },
  popover: {
  DEFAULT: 'hsl(var(--popover))',
  foreground: 'hsl(var(--popover-foreground))'
  },
  card: {
  DEFAULT: 'hsl(var(--card))',
  foreground: 'hsl(var(--card-foreground))'
  },
  sidebar: {
  DEFAULT: 'hsl(var(--sidebar-background))',
  foreground: 'hsl(var(--sidebar-foreground))',
  primary: 'hsl(var(--sidebar-primary))',
  'primary-foreground': 'hsl(var(--sidebar-primary-foreground))',
  accent: 'hsl(var(--sidebar-accent))',
  'accent-foreground': 'hsl(var(--sidebar-accent-foreground))',
  border: 'hsl(var(--sidebar-border))',
  ring: 'hsl(var(--sidebar-ring))'
  }
  },
  borderRadius: {
  lg: 'var(--radius)',
  md: 'calc(var(--radius) - 2px)',
  sm: 'calc(var(--radius) - 4px)'
  },
  keyframes: {
  'accordion-down': {
  from: {
  height: '0'
  },
  to: {
  height: 'var(--radix-accordion-content-height)'
  }
  },
  'accordion-up': {
  from: {
  height: 'var(--radix-accordion-content-height)'
  },
  to: {
  height: '0'
  }
  }
  },
  animation: {
  'accordion-down': 'accordion-down 0.2s ease-out',
  'accordion-up': 'accordion-up 0.2s ease-out'
  },
  fontFamily: {
  sans: [
  'Roboto',
  'ui-sans-serif',
  'system-ui',
  'sans-serif',
```

```
  ■■■■'Apple Color Emoji',
  ■■■■'Segoe UI Emoji',
  ■■■■'Segoe UI Symbol',
  ■■■■'Noto Color Emoji'
  ■■■],
  ■■■serif: [
  ■■■■'Libre Caslon Text',
  ■■■■'ui-serif',
  ■■■■'Georgia',
  ■■■■'Cambria',
  ■■■■'Times New Roman',
  ■■■■'Times',
  ■■■■'serif'
  ■■■],
  ■■■mono: [
  ■■■■'Roboto Mono',
  ■■■■'ui-monospace',
  ■■■■'SFMono-Regular',
  ■■■■'Menlo',
  ■■■■'Monaco',
  ■■■■'Consolas',
  ■■■■'Liberation Mono',
  ■■■■'Courier New',
  ■■■■'monospace'
  ■■■]
  ■■}
  ■}
  },
  plugins: [require("tailwindcss-animate")],
} satisfies Config;
```

vite.config.ts

vite.config.ts

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react-swc";
import path from "path";
import { componentTagger } from "lovable-tagger";

// https://vitejs.dev/config/
export default defineConfig(({ mode }) =&gt; ({
  server: {
    host: "::",
    port: 8080,
    hmr: {
      overlay: false,
    },
  },
  plugins: [react(), mode === "development" &amp;&amp; componentTagger()].filter(Boolean),
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
    dedupe: ["react", "react-dom", "react/jsx-runtime"],
  },
}));
```

index.html

index.html

```
&lt;!doctype html&gt;
&lt;html lang="en"&gt;
  &lt;head&gt;
    &lt;meta charset="UTF-8" /&gt;
    &lt;meta name="viewport" content="width=device-width, initial-scale=1.0" /&gt;
    &lt;!-- TODO: Set the document title to the name of your application --&gt;
    &lt;title&gt;DARA Studio | Drafting &amp; 3D Support&lt;/title&gt;
    &lt;meta name="description" content="DARA Studio - Professional Drafting &amp; 3D Support Services" /&gt;
    &lt;meta name="author" content="DARA Studio" /&gt;

    &lt;meta property="og:title" content="DARA Studio | Drafting &amp; 3D Support" /&gt;
    &lt;meta property="og:description" content="Lovable Generated Project" /&gt;
    &lt;meta property="og:type" content="website" /&gt;
    &lt;meta property="og:image" content="https://lovable.dev/opengraph-image-p98pqg.png" /&gt;

    &lt;meta name="twitter:card" content="summary_large_image" /&gt;
    &lt;meta name="twitter:site" content="@Lovable" /&gt;
    &lt;meta name="twitter:image" content="https://lovable.dev/opengraph-image-p98pqg.png" /&gt;
  &lt;/head&gt;

  &lt;body&gt;
    &lt;div id="root"&gt;&lt;/div&gt;
    &lt;script type="module" src="/src/main.tsx"&gt;&lt;/script&gt;
  &lt;/body&gt;
&lt;/html&gt;
```

components.json

components.json

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "default",
  "rsc": false,
  "tsx": true,
  "tailwind": {
    "config": "tailwind.config.ts",
    "css": "src/index.css",
    "baseColor": "slate",
    "cssVariables": true,
    "prefix": ""
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  }
}
```

supabase/config.toml

supabase/config.toml

```
project_id = "tcyfqcfqwjqmlkvalycy"
```

src/integrations/supabase

src/integrations/supabase/types.ts

```
export type Json =
  | string
  | number
  | boolean
  | null
  | { [key: string]: Json | undefined }
  | Json[]

export type Database = {
  // Allows to automatically instantiate createClient with right options
  // instead of createClient&lt;Database, { PostgrestVersion: 'XX' }&gt;(URL, KEY)
  __InternalSupabase: {
    PostgrestVersion: "14.1"
  }
  public: {
    Tables: {
      activity_log: {
        Row: {
          created_at: string
          description: string
          id: string
          log_date: string
          log_type: string
          project_id: string
        }
        Insert: {
          created_at?: string
          description: string
          id?: string
          log_date?: string
          log_type?: string
          project_id: string
        }
        Update: {
          created_at?: string
          description?: string
          id?: string
          log_date?: string
          log_type?: string
          project_id?: string
        }
        Relationships: [
          {
            foreignKeyName: "activity_log_project_id_fkey"
            columns: ["project_id"]
            isOneToOne: false
            referencedRelation: "projects"
            referencedColumns: ["id"]
          },
        ]
      }
      admin_payments: {
        Row: {
          amount_paid: number
          amount_total: number
          created_at: string
          id: string
          installment_type: string
          net_received: number | null
          payment_date: string | null
          payment_method: string | null
          payment_notes: string | null
          price_table: string | null
          project_id: string
          status: string
          stripe_fee_percent: number | null
          third_party_costs: number | null
          updated_at: string
          value_brl: number | null
        }
        Insert: {
          amount_paid?: number
          amount_total?: number
          created_at?: string
          id?: string
          installment_type?: string
          net_received?: number | null
          payment_date?: string | null
          payment_method?: string | null
          payment_notes?: string | null
          price_table?: string | null
          project_id: string
          status?: string
          stripe_fee_percent?: number | null
          third_party_costs?: number | null
          updated_at?: string
          value_brl?: number | null
        }
        Update: {
          amount_paid?: number
          amount_total?: number
          created_at?: string
          id?: string
```

```
      installment_type?: string
      net_received?: number | null
      payment_date?: string | null
      payment_method?: string | null
      payment_notes?: string | null
      price_table?: string | null
      project_id?: string
      status?: string
      stripe_fee_percent?: number | null
      third_party_costs?: number | null
      updated_at?: string
      value_brl?: number | null
    }
    Relationships: [
      {
        foreignKeyName: "admin_payments_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  admin_tasks: {
    Row: {
      completed_at: string | null
      created_at: string
      created_by: string | null
      due_date: string
      id: string
      is_completed: boolean
      priority: string
      project_id: string | null
      title: string
      updated_at: string
    }
    Insert: {
      completed_at?: string | null
      created_at?: string
      created_by?: string | null
      due_date?: string
      id?: string
      is_completed?: boolean
      priority?: string
      project_id?: string | null
      title: string
      updated_at?: string
    }
    Update: {
      completed_at?: string | null
      created_at?: string
      created_by?: string | null
      due_date?: string
      id?: string
      is_completed?: boolean
      priority?: string
      project_id?: string | null
      title?: string
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "admin_tasks_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  client_ticket_messages: {
    Row: {
      content: string
      created_at: string
      id: string
      is_from_admin: boolean
      sender_id: string | null
      ticket_id: string
    }
    Insert: {
      content: string
      created_at?: string
      id?: string
      is_from_admin?: boolean
      sender_id?: string | null
      ticket_id: string
    }
    Update: {
      content?: string
      created_at?: string
      id?: string
      is_from_admin?: boolean
      sender_id?: string | null
      ticket_id?: string
    }
    Relationships: [
      {
        foreignKeyName: "client_ticket_messages_ticket_id_fkey"
        columns: ["ticket_id"]
        isOneToOne: false
        referencedRelation: "client_tickets"
```

```
      referencedColumns: ["id"]
    },
  ]
}
client_tickets: {
  Row: {
    company_id: string
    created_at: string
    created_by: string | null
    id: string
    project_id: string
    status: string
    subject: string
    updated_at: string
  }
  Insert: {
    company_id: string
    created_at?: string
    created_by?: string | null
    id?: string
    project_id: string
    status?: string
    subject: string
    updated_at?: string
  }
  Update: {
    company_id?: string
    created_at?: string
    created_by?: string | null
    id?: string
    project_id?: string
    status?: string
    subject?: string
    updated_at?: string
  }
  Relationships: [
    {
      foreignKeyName: "client_tickets_company_id_fkey"
      columns: ["company_id"]
      isOneToOne: false
      referencedRelation: "companies"
      referencedColumns: ["id"]
    },
    {
      foreignKeyName: "client_tickets_project_id_fkey"
      columns: ["project_id"]
      isOneToOne: false
      referencedRelation: "projects"
      referencedColumns: ["id"]
    },
  ]
}
companies: {
  Row: {
    address: string | null
    avatar_url: string | null
    city: string | null
    company_type: string | null
    country: string | null
    created_at: string
    email: string | null
    id: string
    instagram: string | null
    language: string | null
    linkedin: string | null
    name: string
    neighborhood: string | null
    owner_name: string | null
    personal_email: string | null
    personal_instagram: string | null
    phone: string | null
    state: string | null
    status: string | null
    street_name: string | null
    street_number: string | null
    unit: string | null
    updated_at: string
    website: string | null
    zip_code: string | null
  }
  Insert: {
    address?: string | null
    avatar_url?: string | null
    city?: string | null
    company_type?: string | null
    country?: string | null
    created_at?: string
    email?: string | null
    id?: string
    instagram?: string | null
    language?: string | null
    linkedin?: string | null
    name: string
    neighborhood?: string | null
    owner_name?: string | null
    personal_email?: string | null
    personal_instagram?: string | null
    phone?: string | null
    state?: string | null
    status?: string | null
```

```
      street_name?: string | null
      street_number?: string | null
      unit?: string | null
      updated_at?: string
      website?: string | null
      zip_code?: string | null
    }
    Update: {
      address?: string | null
      avatar_url?: string | null
      city?: string | null
      company_type?: string | null
      country?: string | null
      created_at?: string
      email?: string | null
      id?: string
      instagram?: string | null
      language?: string | null
      linkedin?: string | null
      name?: string
      neighborhood?: string | null
      owner_name?: string | null
      personal_email?: string | null
      personal_instagram?: string | null
      phone?: string | null
      state?: string | null
      status?: string | null
      street_name?: string | null
      street_number?: string | null
      unit?: string | null
      updated_at?: string
      website?: string | null
      zip_code?: string | null
    }
    Relationships: []
  }
  company_employees: {
    Row: {
      company_id: string
      created_at: string
      email: string | null
      id: string
      instagram: string | null
      linkedin: string | null
      name: string
      phone: string | null
    }
    Insert: {
      company_id: string
      created_at?: string
      email?: string | null
      id?: string
      instagram?: string | null
      linkedin?: string | null
      name: string
      phone?: string | null
    }
    Update: {
      company_id?: string
      created_at?: string
      email?: string | null
      id?: string
      instagram?: string | null
      linkedin?: string | null
      name?: string
      phone?: string | null
    }
    Relationships: [
      {
        foreignKeyName: "company_employees_company_id_fkey"
        columns: ["company_id"]
        isOneToOne: false
        referencedRelation: "companies"
        referencedColumns: ["id"]
      },
    ]
  }
  drawing_versions: {
    Row: {
      created_at: string
      drawing_name: string
      file_name: string
      file_url: string
      id: string
      is_current: boolean
      project_id: string
      revision_notes: string | null
      uploaded_by: string | null
      version_number: number
    }
    Insert: {
      created_at?: string
      drawing_name: string
      file_name: string
      file_url: string
      id?: string
      is_current?: boolean
      project_id: string
      revision_notes?: string | null
      uploaded_by?: string | null
      version_number?: number
```

```
    }
    Update: {
      created_at?: string
      drawing_name?: string
      file_name?: string
      file_url?: string
      id?: string
      is_current?: boolean
      project_id?: string
      revision_notes?: string | null
      uploaded_by?: string | null
      version_number?: number
    }
    Relationships: [
      {
        foreignKeyName: "drawing_versions_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  freelancer_assignments: {
    Row: {
      assigned_at: string
      deadline: string | null
      freelancer_id: string
      id: string
      notes: string | null
      project_id: string
    }
    Insert: {
      assigned_at?: string
      deadline?: string | null
      freelancer_id: string
      id?: string
      notes?: string | null
      project_id: string
    }
    Update: {
      assigned_at?: string
      deadline?: string | null
      freelancer_id?: string
      id?: string
      notes?: string | null
      project_id?: string
    }
    Relationships: [
      {
        foreignKeyName: "freelancer_assignments_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  freelancer_contracts: {
    Row: {
      created_at: string
      file_url: string | null
      freelancer_id: string
      id: string
      is_signed: boolean
      project_id: string
      signed_at: string | null
      title: string
    }
    Insert: {
      created_at?: string
      file_url?: string | null
      freelancer_id: string
      id?: string
      is_signed?: boolean
      project_id: string
      signed_at?: string | null
      title: string
    }
    Update: {
      created_at?: string
      file_url?: string | null
      freelancer_id?: string
      id?: string
      is_signed?: boolean
      project_id?: string
      signed_at?: string | null
      title?: string
    }
    Relationships: [
      {
        foreignKeyName: "freelancer_contracts_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  freelancer_deliveries: {
```

```
    Row: {
      created_at: string
      file_name: string
      file_url: string
      freelancer_id: string
      id: string
      notes: string | null
      project_id: string
    }
    Insert: {
      created_at?: string
      file_name: string
      file_url: string
      freelancer_id: string
      id?: string
      notes?: string | null
      project_id: string
    }
    Update: {
      created_at?: string
      file_name?: string
      file_url?: string
      freelancer_id?: string
      id?: string
      notes?: string | null
      project_id?: string
    }
    Relationships: [
      {
        foreignKeyName: "freelancer_deliveries_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  freelancer_payments: {
    Row: {
      agreed_amount: number
      amount: number
      created_at: string
      description: string | null
      due_date: string | null
      freelancer_id: string
      id: string
      paid_amount: number
      payment_method: string | null
      project_id: string
      status: string
      updated_at: string
    }
    Insert: {
      agreed_amount?: number
      amount: number
      created_at?: string
      description?: string | null
      due_date?: string | null
      freelancer_id: string
      id?: string
      paid_amount?: number
      payment_method?: string | null
      project_id: string
      status?: string
      updated_at?: string
    }
    Update: {
      agreed_amount?: number
      amount?: number
      created_at?: string
      description?: string | null
      due_date?: string | null
      freelancer_id?: string
      id?: string
      paid_amount?: number
      payment_method?: string | null
      project_id?: string
      status?: string
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "freelancer_payments_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  messages: {
    Row: {
      content: string
      created_at: string
      id: string
      is_from_admin: boolean
      project_id: string
      sender_id: string | null
    }
    Insert: {
      content: string
```

```
      created_at?: string
      id?: string
      is_from_admin?: boolean
      project_id: string
      sender_id?: string | null
    }
    Update: {
      content?: string
      created_at?: string
      id?: string
      is_from_admin?: boolean
      project_id?: string
      sender_id?: string | null
    }
    Relationships: [
      {
        foreignKeyName: "messages_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  milestones: {
    Row: {
      approved_date: string | null
      created_at: string
      id: string
      name: string
      notes: string | null
      project_id: string
      sent_date: string | null
      status: string
    }
    Insert: {
      approved_date?: string | null
      created_at?: string
      id?: string
      name: string
      notes?: string | null
      project_id: string
      sent_date?: string | null
      status?: string
    }
    Update: {
      approved_date?: string | null
      created_at?: string
      id?: string
      name?: string
      notes?: string | null
      project_id?: string
      sent_date?: string | null
      status?: string
    }
    Relationships: [
      {
        foreignKeyName: "milestones_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  packages: {
    Row: {
      created_at: string
      description: string | null
      id: string
      name: string
      slug: string
    }
    Insert: {
      created_at?: string
      description?: string | null
      id?: string
      name: string
      slug: string
    }
    Update: {
      created_at?: string
      description?: string | null
      id?: string
      name?: string
      slug?: string
    }
    Relationships: []
  }
  payment_milestones: {
    Row: {
      amount: number
      created_at: string
      id: string
      payment_date: string | null
      percentage: number
      project_id: string
      stage: string
      status: string
      updated_at: string
```

```
    }
    Insert: {
      amount?: number
      created_at?: string
      id?: string
      payment_date?: string | null
      percentage?: number
      project_id: string
      stage: string
      status?: string
      updated_at?: string
    }
    Update: {
      amount?: number
      created_at?: string
      id?: string
      payment_date?: string | null
      percentage?: number
      project_id?: string
      stage?: string
      status?: string
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "payment_milestones_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  profiles: {
    Row: {
      avatar_url: string | null
      company_id: string | null
      created_at: string
      email: string | null
      full_name: string | null
      id: string
      instagram: string | null
      phone: string | null
      updated_at: string
    }
    Insert: {
      avatar_url?: string | null
      company_id?: string | null
      created_at?: string
      email?: string | null
      full_name?: string | null
      id: string
      instagram?: string | null
      phone?: string | null
      updated_at?: string
    }
    Update: {
      avatar_url?: string | null
      company_id?: string | null
      created_at?: string
      email?: string | null
      full_name?: string | null
      id?: string
      instagram?: string | null
      phone?: string | null
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "profiles_company_id_fkey"
        columns: ["company_id"]
        isOneToOne: false
        referencedRelation: "companies"
        referencedColumns: ["id"]
      },
    ]
  }
  project_files: {
    Row: {
      created_at: string
      document_category: string
      file_name: string
      file_size: number | null
      file_type: string | null
      file_url: string
      id: string
      project_id: string
      uploaded_by: string | null
    }
    Insert: {
      created_at?: string
      document_category?: string
      file_name: string
      file_size?: number | null
      file_type?: string | null
      file_url: string
      id?: string
      project_id: string
      uploaded_by?: string | null
    }
    Update: {
```

```
    created_at?: string
    document_category?: string
    file_name?: string
    file_size?: number | null
    file_type?: string | null
    file_url?: string
    id?: string
    project_id?: string
    uploaded_by?: string | null
  }
  Relationships: [
    {
      foreignKeyName: "project_files_project_id_fkey"
      columns: ["project_id"]
      isOneToOne: false
      referencedRelation: "projects"
      referencedColumns: ["id"]
    },
  ]
}
project_homeowners: {
  Row: {
    address: string | null
    created_at: string
    email: string | null
    id: string
    name: string
    phone: string | null
    project_id: string
  }
  Insert: {
    address?: string | null
    created_at?: string
    email?: string | null
    id?: string
    name: string
    phone?: string | null
    project_id: string
  }
  Update: {
    address?: string | null
    created_at?: string
    email?: string | null
    id?: string
    name?: string
    phone?: string | null
    project_id?: string
  }
  Relationships: [
    {
      foreignKeyName: "project_homeowners_project_id_fkey"
      columns: ["project_id"]
      isOneToOne: false
      referencedRelation: "projects"
      referencedColumns: ["id"]
    },
  ]
}
project_services: {
  Row: {
    id: string
    project_id: string
    service_id: string
  }
  Insert: {
    id?: string
    project_id: string
    service_id: string
  }
  Update: {
    id?: string
    project_id?: string
    service_id?: string
  }
  Relationships: [
    {
      foreignKeyName: "project_services_project_id_fkey"
      columns: ["project_id"]
      isOneToOne: false
      referencedRelation: "projects"
      referencedColumns: ["id"]
    },
    {
      foreignKeyName: "project_services_service_id_fkey"
      columns: ["service_id"]
      isOneToOne: false
      referencedRelation: "services"
      referencedColumns: ["id"]
    },
  ]
}
project_tasks: {
  Row: {
    assigned_to: string | null
    completed_at: string | null
    created_at: string
    due_date: string | null
    id: string
    is_completed: boolean
    priority: string
```

```
      project_id: string
      requires_client_review: boolean
      sort_order: number
      stage: string
      title: string
      updated_at: string
    }
    Insert: {
      assigned_to?: string | null
      completed_at?: string | null
      created_at?: string
      due_date?: string | null
      id?: string
      is_completed?: boolean
      priority?: string
      project_id: string
      requires_client_review?: boolean
      sort_order?: number
      stage?: string
      title: string
      updated_at?: string
    }
    Update: {
      assigned_to?: string | null
      completed_at?: string | null
      created_at?: string
      due_date?: string | null
      id?: string
      is_completed?: boolean
      priority?: string
      project_id?: string
      requires_client_review?: boolean
      sort_order?: number
      stage?: string
      title?: string
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "project_tasks_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  project_timeline: {
    Row: {
      created_at: string
      event_date: string | null
      event_type: string
      id: string
      notes: string | null
      project_id: string
      revision_number: number
    }
    Insert: {
      created_at?: string
      event_date?: string | null
      event_type: string
      id?: string
      notes?: string | null
      project_id: string
      revision_number?: number
    }
    Update: {
      created_at?: string
      event_date?: string | null
      event_type?: string
      id?: string
      notes?: string | null
      project_id?: string
      revision_number?: number
    }
    Relationships: [
      {
        foreignKeyName: "project_timeline_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  projects: {
    Row: {
      address: string | null
      city: string | null
      company_id: string
      country: string | null
      cover_image: string | null
      created_at: string
      currency: string
      date_notes: Json | null
      delivery_date: string | null
      description: string | null
      display_address: string | null
      entry_payment_percent: number
      final_delivery_date: string | null
      homeowner_address: string | null
```

```
    homeowner_email: string | null
    homeowner_name: string | null
    homeowner_phone: string | null
    id: string
    internal_notes: string | null
    is_portfolio: boolean
    latitude: number | null
    longitude: number | null
    name: string | null
    neighborhood: string | null
    package_id: string | null
    payment_effective_date: string | null
    payment_stage: string
    preview_delivery_date: string | null
    preview_sent_date: string | null
    project_number: string | null
    revision_count: number
    revision_return_date: string | null
    service_type: string
    square_feet: number | null
    stage: string | null
    start_date: string | null
    state: string | null
    status: Database["public"]["Enums"]["project_status"]
    street_name: string | null
    street_number: string | null
    total_value: number | null
    unit: string | null
    updated_at: string
    zip_code: string | null
  }
  Insert: {
    address?: string | null
    city?: string | null
    company_id: string
    country?: string | null
    cover_image?: string | null
    created_at?: string
    currency?: string
    date_notes?: Json | null
    delivery_date?: string | null
    description?: string | null
    display_address?: string | null
    entry_payment_percent?: number
    final_delivery_date?: string | null
    homeowner_address?: string | null
    homeowner_email?: string | null
    homeowner_name?: string | null
    homeowner_phone?: string | null
    id?: string
    internal_notes?: string | null
    is_portfolio?: boolean
    latitude?: number | null
    longitude?: number | null
    name?: string | null
    neighborhood?: string | null
    package_id?: string | null
    payment_effective_date?: string | null
    payment_stage?: string
    preview_delivery_date?: string | null
    preview_sent_date?: string | null
    project_number?: string | null
    revision_count?: number
    revision_return_date?: string | null
    service_type?: string
    square_feet?: number | null
    stage?: string | null
    start_date?: string | null
    state?: string | null
    status?: Database["public"]["Enums"]["project_status"]
    street_name?: string | null
    street_number?: string | null
    total_value?: number | null
    unit?: string | null
    updated_at?: string
    zip_code?: string | null
  }
  Update: {
    address?: string | null
    city?: string | null
    company_id?: string
    country?: string | null
    cover_image?: string | null
    created_at?: string
    currency?: string
    date_notes?: Json | null
    delivery_date?: string | null
    description?: string | null
    display_address?: string | null
    entry_payment_percent?: number
    final_delivery_date?: string | null
    homeowner_address?: string | null
    homeowner_email?: string | null
    homeowner_name?: string | null
    homeowner_phone?: string | null
    id?: string
    internal_notes?: string | null
    is_portfolio?: boolean
    latitude?: number | null
    longitude?: number | null
```

```
      name?: string | null
      neighborhood?: string | null
      package_id?: string | null
      payment_effective_date?: string | null
      payment_stage?: string
      preview_delivery_date?: string | null
      preview_sent_date?: string | null
      project_number?: string | null
      revision_count?: number
      revision_return_date?: string | null
      service_type?: string
      square_feet?: number | null
      stage?: string | null
      start_date?: string | null
      state?: string | null
      status?: Database["public"]["Enums"]["project_status"]
      street_name?: string | null
      street_number?: string | null
      total_value?: number | null
      unit?: string | null
      updated_at?: string
      zip_code?: string | null
    }
    Relationships: [
      {
        foreignKeyName: "projects_company_id_fkey"
        columns: ["company_id"]
        isOneToOne: false
        referencedRelation: "companies"
        referencedColumns: ["id"]
      },
      {
        foreignKeyName: "projects_package_id_fkey"
        columns: ["package_id"]
        isOneToOne: false
        referencedRelation: "packages"
        referencedColumns: ["id"]
      },
    ]
  }
  quote_requests: {
    Row: {
      company_name: string | null
      created_at: string
      description: string | null
      email: string
      file_url: string | null
      full_name: string
      id: string
      phone: string | null
      processed: boolean
      project_address: string | null
      service_type: Database["public"]["Enums"]["service_type"]
    }
    Insert: {
      company_name?: string | null
      created_at?: string
      description?: string | null
      email: string
      file_url?: string | null
      full_name: string
      id?: string
      phone?: string | null
      processed?: boolean
      project_address?: string | null
      service_type: Database["public"]["Enums"]["service_type"]
    }
    Update: {
      company_name?: string | null
      created_at?: string
      description?: string | null
      email?: string
      file_url?: string | null
      full_name?: string
      id?: string
      phone?: string | null
      processed?: boolean
      project_address?: string | null
      service_type?: Database["public"]["Enums"]["service_type"]
    }
    Relationships: []
  }
  quotes: {
    Row: {
      amount: number
      company_id: string
      created_at: string
      description: string | null
      id: string
      payment_link: string | null
      project_id: string
      status: Database["public"]["Enums"]["quote_status"]
      title: string
      updated_at: string
      valid_until: string | null
    }
    Insert: {
      amount: number
      company_id: string
      created_at?: string
      description?: string | null
```

```
      id?: string
      payment_link?: string | null
      project_id: string
      status?: Database["public"]["Enums"]["quote_status"]
      title: string
      updated_at?: string
      valid_until?: string | null
    }
    Update: {
      amount?: number
      company_id?: string
      created_at?: string
      description?: string | null
      id?: string
      payment_link?: string | null
      project_id?: string
      status?: Database["public"]["Enums"]["quote_status"]
      title?: string
      updated_at?: string
      valid_until?: string | null
    }
    Relationships: [
      {
        foreignKeyName: "quotes_company_id_fkey"
        columns: ["company_id"]
        isOneToOne: false
        referencedRelation: "companies"
        referencedColumns: ["id"]
      },
      {
        foreignKeyName: "quotes_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  services: {
    Row: {
      category: string | null
      created_at: string
      id: string
      name: string
      slug: string
    }
    Insert: {
      category?: string | null
      created_at?: string
      id?: string
      name: string
      slug: string
    }
    Update: {
      category?: string | null
      created_at?: string
      id?: string
      name?: string
      slug?: string
    }
    Relationships: []
  }
  site_updates: {
    Row: {
      created_at: string
      description: string | null
      id: string
      media_url: string
      project_id: string
      uploaded_by: string | null
    }
    Insert: {
      created_at?: string
      description?: string | null
      id?: string
      media_url: string
      project_id: string
      uploaded_by?: string | null
    }
    Update: {
      created_at?: string
      description?: string | null
      id?: string
      media_url?: string
      project_id?: string
      uploaded_by?: string | null
    }
    Relationships: [
      {
        foreignKeyName: "site_updates_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  studio_settings: {
    Row: {
      accent_color: string | null
```

```
      contact_email: string | null
      contact_phone: string | null
      custom_domain: string | null
      id: string
      logo_url: string | null
      primary_color: string | null
      studio_name: string
      tagline: string | null
      updated_at: string
    }
    Insert: {
      accent_color?: string | null
      contact_email?: string | null
      contact_phone?: string | null
      custom_domain?: string | null
      id?: string
      logo_url?: string | null
      primary_color?: string | null
      studio_name?: string
      tagline?: string | null
      updated_at?: string
    }
    Update: {
      accent_color?: string | null
      contact_email?: string | null
      contact_phone?: string | null
      custom_domain?: string | null
      id?: string
      logo_url?: string | null
      primary_color?: string | null
      studio_name?: string
      tagline?: string | null
      updated_at?: string
    }
    Relationships: []
  }
  support_messages: {
    Row: {
      created_at: string
      email: string
      id: string
      message: string
      name: string
      subject: string | null
    }
    Insert: {
      created_at?: string
      email: string
      id?: string
      message: string
      name: string
      subject?: string | null
    }
    Update: {
      created_at?: string
      email?: string
      id?: string
      message?: string
      name?: string
      subject?: string | null
    }
    Relationships: []
  }
  user_roles: {
    Row: {
      id: string
      role: Database["public"]["Enums"]["app_role"]
      user_id: string
    }
    Insert: {
      id?: string
      role: Database["public"]["Enums"]["app_role"]
      user_id: string
    }
    Update: {
      id?: string
      role?: Database["public"]["Enums"]["app_role"]
      user_id?: string
    }
    Relationships: []
  }
}
Views: {
  [_ in never]: never
}
Functions: {
  has_role: {
    Args: {
      _role: Database["public"]["Enums"]["app_role"]
      _user_id: string
    }
    Returns: boolean
  }
  user_belongs_to_company: {
    Args: { target_company_id: string; user_id: string }
    Returns: boolean
  }
}
Enums: {
  app_role: "admin" | "moderator" | "user" | "freelancer"
  project_status:
```

```
        | "pending"
        | "in_progress"
        | "under_review"
        | "completed"
        | "cancelled"
      quote_status: "draft" | "sent" | "approved" | "paid"
      service_type:
        | "technical_drafting"
        | "executive_project"
        | "wood_frame_structures"
        | "project_budgeting"
        | "remodeling"
        | "3d_kitchen"
        | "architecture"
        | "pdf_cad"
        | "3d_bath"
    }
    CompositeTypes: {
      [_ in never]: never
    }
  }
}

type DatabaseWithoutInternals = Omit&lt;Database, "__InternalSupabase"&gt;

type DefaultSchema = DatabaseWithoutInternals[Extract&lt;keyof Database, "public"&gt;]

export type Tables&lt;
  DefaultSchemaTableNameOrOptions extends
    | keyof (DefaultSchema["Tables"] &amp; DefaultSchema["Views"])
    | { schema: keyof DatabaseWithoutInternals },
  TableName extends DefaultSchemaTableNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof (DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["Tables"] &amp;
        DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["Views"])
    : never = never,
&gt; = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? (DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["Tables"] &amp;
      DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["Views"])[TableName] extends {
      Row: infer R
    }
    ? R
    : never
  : DefaultSchemaTableNameOrOptions extends keyof (DefaultSchema["Tables"] &amp;
        DefaultSchema["Views"])
    ? (DefaultSchema["Tables"] &amp;
        DefaultSchema["Views"])[DefaultSchemaTableNameOrOptions] extends {
        Row: infer R
      }
      ? R
      : never
    : never

export type TablesInsert&lt;
  DefaultSchemaTableNameOrOptions extends
    | keyof DefaultSchema["Tables"]
    | { schema: keyof DatabaseWithoutInternals },
  TableName extends DefaultSchemaTableNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["Tables"]
    : never = never,
&gt; = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["Tables"][TableName] extends {
      Insert: infer I
    }
    ? I
    : never
  : DefaultSchemaTableNameOrOptions extends keyof DefaultSchema["Tables"]
    ? DefaultSchema["Tables"][DefaultSchemaTableNameOrOptions] extends {
        Insert: infer I
      }
      ? I
      : never
    : never

export type TablesUpdate&lt;
  DefaultSchemaTableNameOrOptions extends
    | keyof DefaultSchema["Tables"]
    | { schema: keyof DatabaseWithoutInternals },
  TableName extends DefaultSchemaTableNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["Tables"]
    : never = never,
&gt; = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["Tables"][TableName] extends {
      Update: infer U
    }
    ? U
    : never
  : DefaultSchemaTableNameOrOptions extends keyof DefaultSchema["Tables"]
    ? DefaultSchema["Tables"][DefaultSchemaTableNameOrOptions] extends {
        Update: infer U
```

```
      }
      ? U
      : never
    : never

export type Enums&lt;
  DefaultSchemaEnumNameOrOptions extends
    | keyof DefaultSchema["Enums"]
    | { schema: keyof DatabaseWithoutInternals },
  EnumName extends DefaultSchemaEnumNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaEnumNameOrOptions["schema"]]["Enums"]
    : never = never,
&gt; = DefaultSchemaEnumNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? DatabaseWithoutInternals[DefaultSchemaEnumNameOrOptions["schema"]]["Enums"][EnumName]
  : DefaultSchemaEnumNameOrOptions extends keyof DefaultSchema["Enums"]
    ? DefaultSchema["Enums"][DefaultSchemaEnumNameOrOptions]
    : never

export type CompositeTypes&lt;
  PublicCompositeTypeNameOrOptions extends
    | keyof DefaultSchema["CompositeTypes"]
    | { schema: keyof DatabaseWithoutInternals },
  CompositeTypeName extends PublicCompositeTypeNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof DatabaseWithoutInternals[PublicCompositeTypeNameOrOptions["schema"]]["CompositeTypes"]
    : never = never,
&gt; = PublicCompositeTypeNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? DatabaseWithoutInternals[PublicCompositeTypeNameOrOptions["schema"]]["CompositeTypes"][CompositeTypeName]
  : PublicCompositeTypeNameOrOptions extends keyof DefaultSchema["CompositeTypes"]
    ? DefaultSchema["CompositeTypes"][PublicCompositeTypeNameOrOptions]
    : never

export const Constants = {
  public: {
    Enums: {
      app_role: ["admin", "moderator", "user", "freelancer"],
      project_status: [
        "pending",
        "in_progress",
        "under_review",
        "completed",
        "cancelled",
      ],
      quote_status: ["draft", "sent", "approved", "paid"],
      service_type: [
        "technical_drafting",
        "executive_project",
        "wood_frame_structures",
        "project_budgeting",
        "remodeling",
        "3d_kitchen",
        "architecture",
        "pdf_cad",
        "3d_bath",
      ],
    },
  },
} as const
```

src/pages/admin

src/pages/admin/AdminAICodeCheck.tsx

```
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { BrainCircuit } from "lucide-react";

const AdminAICodeCheck = () =&gt; {
  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;AI Code Check&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Upload drawings for automated compliance analysis&lt;/p&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="flex flex-col items-center justify-center py-16"&gt;
          &lt;BrainCircuit className="h-12 w-12 text-muted-foreground/30 mb-4" /&gt;
          &lt;h3 className="text-lg font-medium text-foreground mb-2"&gt;AI Drawing Analysis&lt;/h3&gt;
          &lt;p className="text-sm text-muted-foreground text-center max-w-md"&gt;
            Upload architectural drawings for automated building code compliance checks. AI will analyze setbacks, egress, accessibility, and structural requirements.
          &lt;/p&gt;
          &lt;Badge variant="secondary" className="mt-4"&gt;Coming Soon&lt;/Badge&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminAICodeCheck;
```

src/pages/admin/AdminActivity.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Badge } from "@/components/ui/badge";
import { Plus, CreditCard, Package, RotateCcw, StickyNote } from "lucide-react";
import { toast } from "sonner";
import { format } from "date-fns";

const LOG_TYPES = [
  { value: "payment", label: "Pagamento", icon: CreditCard },
  { value: "delivery", label: "Entrega", icon: Package },
  { value: "revision", label: "Revisão", icon: RotateCcw },
  { value: "internal_note", label: "Nota Interna", icon: StickyNote },
];

const AdminActivity = () =&gt; {
  const [logs, setLogs] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [form, setForm] = useState({ project_id: "", log_type: "internal_note", description: "" });

  const fetchData = async () =&gt; {
    const [logRes, projRes] = await Promise.all([
      supabase.from("activity_log").select("*, projects(name)").order("log_date", { ascending: false }).limit(100),
      supabase.from("projects").select("id, name").order("name"),
    ]);
    setLogs(logRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  const handleSave = async () =&gt; {
    if (!form.project_id || !form.description.trim()) { toast.error("Projeto e descrição são obrigatórios"); return; }
    const { error } = await supabase.from("activity_log").insert({
      project_id: form.project_id, log_type: form.log_type, description: form.description,
    });
    if (error) { toast.error(error.message); return; }
    toast.success("Registro adicionado");
    setDialogOpen(false); setForm({ project_id: "", log_type: "internal_note", description: "" }); fetchData();
  };

  const getTypeInfo = (type: string) =&gt; LOG_TYPES.find(t =&gt; t.value === type) || LOG_TYPES[3];

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Log de Atividades&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Histórico de ações dos projetos&lt;/p&gt;
        &lt;/div&gt;
        &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
          &lt;DialogTrigger asChild&gt;&lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;Novo Registro&lt;/Button&gt;&lt;/DialogTrigger&gt;
          &lt;DialogContent&gt;
            &lt;DialogHeader&gt;&lt;DialogTitle&gt;Novo Registro&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
            &lt;div className="space-y-4"&gt;
              &lt;div&gt;&lt;Label&gt;Projeto *&lt;/Label&gt;
                &lt;Select value={form.project_id} onValueChange={v =&gt; setForm({ ...form, project_id: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Selecione" /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{projects.map(p =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{p.name}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Tipo&lt;/Label&gt;
                &lt;Select value={form.log_type} onValueChange={v =&gt; setForm({ ...form, log_type: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{LOG_TYPES.map(t =&gt; &lt;SelectItem key={t.value} value={t.value}&gt;{t.label}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Descrição *&lt;/Label&gt;&lt;Textarea value={form.description} onChange={e =&gt; setForm({ ...form, description: e.target.value })} /&g
              &lt;Button className="w-full" onClick={handleSave}&gt;Registrar&lt;/Button&gt;
            &lt;/div&gt;
          &lt;/DialogContent&gt;
        &lt;/Dialog&gt;
      &lt;/div&gt;

      &lt;div className="space-y-3"&gt;
        {logs.map(log =&gt; {
          const typeInfo = getTypeInfo(log.log_type);
          const Icon = typeInfo.icon;
          return (
            &lt;Card key={log.id}&gt;
              &lt;CardContent className="flex items-start gap-4 py-4"&gt;
                &lt;div className="p-2 rounded-lg bg-muted"&gt;
                  &lt;Icon className="h-4 w-4 text-muted-foreground" /&gt;
                &lt;/div&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;div className="flex items-center gap-2 mb-1"&gt;
                    &lt;span className="font-medium text-foreground text-sm"&gt;{log.projects?.name}&lt;/span&gt;
                    &lt;Badge variant="outline" className="text-xs"&gt;{typeInfo.label}&lt;/Badge&gt;
                  &lt;/div&gt;
                  &lt;p className="text-sm text-muted-foreground"&gt;{log.description}&lt;/p&gt;
                &lt;/div&gt;
```

```
                &lt;span className="text-xs text-muted-foreground whitespace-nowrap"&gt;
                  {format(new Date(log.log_date), "dd/MM/yyyy")}
                &lt;/span&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          );
        })}
        {logs.length === 0 &amp;&amp; (
          &lt;div className="text-center text-muted-foreground py-12"&gt;Nenhum registro de atividade&lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminActivity;
```

src/pages/admin/AdminAnalytics.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { BarChart, Bar, LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, PieChart, Pie, Cell, Legend } from "recharts";
import { BarChart3 } from "lucide-react";

const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
const COLORS = ["hsl(var(--chart-1))", "hsl(var(--chart-2))", "hsl(var(--chart-3))", "hsl(var(--chart-4))", "hsl(var(--chart-5))"];
const tooltipStyle = { backgroundColor: "hsl(var(--card))", border: "1px solid hsl(var(--border))", borderRadius: "8px", fontSize: 12 };

const AdminAnalytics = () =&gt; {
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [payments, setPayments] = useState&lt;any[]&gt;([]);

  useEffect(() =&gt; {
    Promise.all([
      supabase.from("projects").select("*"),
      supabase.from("admin_payments").select("*"),
    ]).then(([p, pay]) =&gt; {
      setProjects(p.data || []);
      setPayments(pay.data || []);
    });
  }, []);

  // Revenue by month
  const revenueByMonth: Record&lt;string, number&gt; = {};
  payments.filter(p =&gt; p.status === "paid").forEach(p =&gt; {
    const m = months[new Date(p.payment_date || p.created_at).getMonth()];
    revenueByMonth[m] = (revenueByMonth[m] || 0) + Number(p.amount_paid || 0);
  });
  const revenueData = months.filter(m =&gt; revenueByMonth[m]).map(m =&gt; ({ month: m, revenue: revenueByMonth[m] }));

  // Projects by stage
  const stageCount: Record&lt;string, number&gt; = {};
  projects.forEach(p =&gt; { stageCount[p.stage || "Unknown"] = (stageCount[p.stage || "Unknown"] || 0) + 1; });
  const stageData = Object.entries(stageCount).map(([name, value]) =&gt; ({ name, value }));

  // Summary
  const totalRevenue = payments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const totalPipeline = projects.filter(p =&gt; p.stage !== "Completed").reduce((s, p) =&gt; s + Number(p.total_value || 0), 0);

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Analytics&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Financial and project performance analytics&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;${totalRevenue.toLocaleString()}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Total Revenue&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;${totalPipeline.toLocaleString()}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Pipeline Value&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;{projects.length}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Total Projects&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;

      &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardHeader className="pb-3"&gt;&lt;CardTitle className="text-base"&gt;Revenue by Month&lt;/CardTitle&gt;&lt;/CardHeader&gt;
          &lt;CardContent&gt;
            {revenueData.length === 0 ? (
              &lt;div className="flex flex-col items-center py-12"&gt;&lt;BarChart3 className="h-8 w-8 text-muted-foreground/30 mb-2" /&gt;&lt;p className="text-sm text-mute
            ) : (
              &lt;ResponsiveContainer width="100%" height={250}&gt;
                &lt;BarChart data={revenueData}&gt;
                  &lt;CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" /&gt;
                  &lt;XAxis dataKey="month" tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" /&gt;
                  &lt;YAxis tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" /&gt;
                  &lt;Tooltip contentStyle={tooltipStyle} /&gt;
                  &lt;Bar dataKey="revenue" fill="hsl(var(--chart-2))" radius={[4, 4, 0, 0]} /&gt;
                &lt;/BarChart&gt;
              &lt;/ResponsiveContainer&gt;
            )}
          &lt;/CardContent&gt;
        &lt;/Card&gt;

        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardHeader className="pb-3"&gt;&lt;CardTitle className="text-base"&gt;Projects by Stage&lt;/CardTitle&gt;&lt;/CardHeader&gt;
          &lt;CardContent&gt;
            {stageData.length === 0 ? (
              &lt;div className="flex flex-col items-center py-12"&gt;&lt;BarChart3 className="h-8 w-8 text-muted-foreground/30 mb-2" /&gt;&lt;p className="text-sm text-mute
            ) : (
              &lt;ResponsiveContainer width="100%" height={250}&gt;
                &lt;PieChart&gt;
                  &lt;Pie data={stageData} dataKey="value" nameKey="name" cx="50%" cy="50%" outerRadius={90} label={(e) =&gt; e.name}&gt;
```

```
                    {stageData.map((_, i) =&gt; &lt;Cell key={i} fill={COLORS[i % COLORS.length]} /&gt;)}
                  &lt;/Pie&gt;
                  &lt;Tooltip contentStyle={tooltipStyle} /&gt;
                  &lt;Legend wrapperStyle={{ fontSize: 11 }} /&gt;
                &lt;/PieChart&gt;
              &lt;/ResponsiveContainer&gt;
            )}
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminAnalytics;
```

src/pages/admin/AdminClientPortal.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Globe, Users, Ticket, Activity } from "lucide-react";
import { format } from "date-fns";

const AdminClientPortal = () =&gt; {
  const [tickets, setTickets] = useState&lt;any[]&gt;([]);
  const [profiles, setProfiles] = useState&lt;any[]&gt;([]);

  useEffect(() =&gt; {
    supabase.from("client_tickets").select("*, companies(name), projects(name)").order("updated_at", { ascending: false }).limit(20)
      .then(({ data }) =&gt; setTickets(data || []));
    supabase.from("profiles").select("*, companies:company_id(name)").order("created_at", { ascending: false }).limit(20)
      .then(({ data }) =&gt; setProfiles(data || []));
  }, []);

  const statusColor: Record&lt;string, string&gt; = {
    open: "bg-blue-500/10 text-blue-600", waiting: "bg-amber-500/10 text-amber-600", resolved: "bg-emerald-500/10 text-emerald-600",
  };

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Client Portal&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Monitor client activity and portal access&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="p-2 rounded-lg bg-blue-50 dark:bg-blue-950/40"&gt;&lt;Users className="h-5 w-5 text-blue-600" /&gt;&lt;/div&gt;
            &lt;div&gt;&lt;p className="text-2xl font-bold"&gt;{profiles.length}&lt;/p&gt;&lt;p className="text-xs text-muted-foreground"&gt;Portal Users&lt;/p&gt;&lt;/div&g
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="p-2 rounded-lg bg-amber-50 dark:bg-amber-950/40"&gt;&lt;Ticket className="h-5 w-5 text-amber-600" /&gt;&lt;/div&gt;
            &lt;div&gt;&lt;p className="text-2xl font-bold"&gt;{tickets.filter(t =&gt; t.status === "open").length}&lt;/p&gt;&lt;p className="text-xs text-muted-foreground"&
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="p-2 rounded-lg bg-emerald-50 dark:bg-emerald-950/40"&gt;&lt;Activity className="h-5 w-5 text-emerald-600" /&gt;&lt;/div&gt;
            &lt;div&gt;&lt;p className="text-2xl font-bold"&gt;{tickets.filter(t =&gt; t.status === "resolved").length}&lt;/p&gt;&lt;p class="text-xs text-muted-foregrou
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;

      &lt;Card class="border-none shadow-sm"&gt;
        &lt;CardHeader class="pb-3"&gt;&lt;CardTitle class="text-base"&gt;Recent Tickets&lt;/CardTitle&gt;&lt;/CardHeader&gt;
        &lt;CardContent class="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Subject&lt;/TableHead&gt;
                &lt;TableHead&gt;Client&lt;/TableHead&gt;
                &lt;TableHead&gt;Project&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Updated&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {tickets.length === 0 &amp;amp;&amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={5} class="text-center text-muted-foreground py-12"&gt;&lt;Globe class="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;N
              )}
              {tickets.map(t =&gt; (
                &lt;TableRow key={t.id}&gt;
                  &lt;TableCell class="font-medium"&gt;{t.subject}&lt;/TableCell&gt;
                  &lt;TableCell class="text-muted-foreground"&gt;{t.companies?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell class="text-muted-foreground"&gt;{t.projects?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell&gt;&lt;Badge variant="outline" class={statusColor[t.status] || ""}&gt;{t.status}&lt;/Badge&gt;&lt;/TableCell&gt;
                  &lt;TableCell class="text-xs text-muted-foreground"&gt;{format(new Date(t.updated_at), "MMM d, yyyy")}&lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminClientPortal;
```

src/pages/admin/AdminClientProfile.tsx

```
import { useEffect, useState } from "react";
import { useParams, useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { ArrowLeft, Building2, Globe, Languages, Mail, Phone, FolderKanban, FileText, MessageSquare, FolderOpen } from "lucide-react";
import { getCountryFlag, getLanguageLabel } from "@/utils/countryFlags";
import { format } from "date-fns";

const AdminClientProfile = () =&gt; {
  const { id } = useParams&lt;{ id: string }&gt;();
  const navigate = useNavigate();
  const [company, setCompany] = useState&lt;any&gt;(null);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [messages, setMessages] = useState&lt;any[]&gt;([]);
  const [employees, setEmployees] = useState&lt;any[]&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    if (!id) return;
    const fetch = async () =&gt; {
      const [compRes, projRes, empRes] = await Promise.all([
        supabase.from("companies").select("*").eq("id", id).single(),
        supabase.from("projects").select("*").eq("company_id", id).order("created_at", { ascending: false }),
        supabase.from("company_employees").select("*").eq("company_id", id),
      ]);
      setCompany(compRes.data);
      setProjects(projRes.data || []);
      setEmployees(empRes.data || []);

      // Fetch files &amp; messages for all projects
      const projectIds = (projRes.data || []).map((p: any) =&gt; p.id);
      if (projectIds.length &gt; 0) {
        const [filesRes, msgsRes] = await Promise.all([
          supabase.from("project_files").select("*, projects(name)").in("project_id", projectIds).order("created_at", { ascending: false }),
          supabase.from("messages").select("*, projects(name)").in("project_id", projectIds).order("created_at", { ascending: false }).limit(50),
        ]);
        setFiles(filesRes.data || []);
        setMessages(msgsRes.data || []);
      }
      setLoading(false);
    };
    fetch();
  }, [id]);

  if (loading) return &lt;div className="p-6 text-muted-foreground"&gt;Loading...&lt;/div&gt;;
  if (!company) return &lt;div className="p-6 text-muted-foreground"&gt;Client not found&lt;/div&gt;;

  const initials = (company.owner_name || company.name || "C")
    .split(" ").map((w: string) =&gt; w[0]).join("").toUpperCase().slice(0, 2);

  return (
    &lt;div className="p-6 space-y-6 max-w-[1200px] mx-auto"&gt;
      {/* Back */}
      &lt;Button variant="ghost" size="sm" onClick={() =&gt; navigate("/adm/companies")}&gt;
        &lt;ArrowLeft className="h-4 w-4 mr-1" /&gt; Back to Clients
      &lt;/Button&gt;

      {/* Header */}
      &lt;div className="flex items-start gap-4"&gt;
        &lt;Avatar className="h-16 w-16"&gt;
          {company.avatar_url &amp;&amp; &lt;AvatarImage src={company.avatar_url} /&gt;}
          &lt;AvatarFallback className="text-lg bg-primary/10 text-primary"&gt;{initials}&lt;/AvatarFallback&gt;
        &lt;/Avatar&gt;
        &lt;div className="flex-1"&gt;
          &lt;h1 className="text-xl font-semibold text-foreground flex items-center gap-2"&gt;
            {company.owner_name || company.name} {getCountryFlag(company.country)}
          &lt;/h1&gt;
          {company.owner_name &amp;&amp; company.name &amp;&amp; (
            &lt;p className="text-sm text-muted-foreground flex items-center gap-1"&gt;
              &lt;Building2 className="h-3.5 w-3.5" /&gt; {company.name}
            &lt;/p&gt;
          )}
          &lt;div className="flex flex-wrap gap-3 mt-2 text-xs text-muted-foreground"&gt;
            {company.language &amp;&amp; (
              &lt;span className="flex items-center gap-1"&gt;
                &lt;Languages className="h-3.5 w-3.5" /&gt; {getLanguageLabel(company.language)}
              &lt;/span&gt;
            )}
            {company.country &amp;&amp; (
              &lt;span className="flex items-center gap-1"&gt;
                &lt;Globe className="h-3.5 w-3.5" /&gt; {company.country}
              &lt;/span&gt;
            )}
            {company.email &amp;&amp; (
              &lt;span className="flex items-center gap-1"&gt;
                &lt;Mail className="h-3.5 w-3.5" /&gt; {company.email}
              &lt;/span&gt;
            )}
            {company.phone &amp;&amp; (
              &lt;span className="flex items-center gap-1"&gt;
                &lt;Phone className="h-3.5 w-3.5" /&gt; {company.phone}
              &lt;/span&gt;
```

```
      )}
    &lt;/div&gt;
    &lt;div className="flex gap-2 mt-2"&gt;
      &lt;Badge variant={company.status === "active" ? "default" : "secondary"}&gt;
        {company.status || "active"}
      &lt;/Badge&gt;
      {company.company_type &amp;&amp; &lt;Badge variant="outline"&gt;{company.company_type}&lt;/Badge&gt;}
    &lt;/div&gt;
  &lt;/div&gt;
&lt;/div&gt;

{/* Tabs */}
&lt;Tabs defaultValue="projects"&gt;
  &lt;TabsList&gt;
    &lt;TabsTrigger value="projects" className="gap-1"&gt;&lt;FolderKanban className="h-3.5 w-3.5" /&gt; Projects ({projects.length})&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="contacts" className="gap-1"&gt;&lt;Building2 className="h-3.5 w-3.5" /&gt; Contacts ({employees.length})&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="files" className="gap-1"&gt;&lt;FolderOpen className="h-3.5 w-3.5" /&gt; Files ({files.length})&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="messages" className="gap-1"&gt;&lt;MessageSquare className="h-3.5 w-3.5" /&gt; Messages ({messages.length})&lt;/TabsTrigger&gt;
  &lt;/TabsList&gt;

  &lt;TabsContent value="projects"&gt;
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead&gt;Project&lt;/TableHead&gt;
              &lt;TableHead&gt;Stage&lt;/TableHead&gt;
              &lt;TableHead&gt;Service&lt;/TableHead&gt;
              &lt;TableHead&gt;Value&lt;/TableHead&gt;
              &lt;TableHead&gt;Created&lt;/TableHead&gt;
            &lt;/TableRow&gt;
          &lt;/TableHeader&gt;
          &lt;TableBody&gt;
            {projects.length === 0 &amp;&amp; (
              &lt;TableRow&gt;&lt;TableCell colSpan={5} className="text-center text-muted-foreground py-8"&gt;No projects&lt;/TableCell&gt;&lt;/TableRow&gt;
            )}
            {projects.map((p) =&gt; (
              &lt;TableRow key={p.id} className="cursor-pointer hover:bg-muted/50" onClick={() =&gt; navigate(`/adm/projects/${p.id}`)}&gt;
                &lt;TableCell className="font-medium"&gt;{p.name || p.display_address || "Untitled"}&lt;/TableCell&gt;
                &lt;TableCell&gt;&lt;Badge variant="outline"&gt;{p.stage}&lt;/Badge&gt;&lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;{p.service_type}&lt;/TableCell&gt;
                &lt;TableCell&gt;{p.currency} {Number(p.total_value || 0).toLocaleString()}&lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;{format(new Date(p.created_at), "MMM d, yyyy")}&lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/TabsContent&gt;

  &lt;TabsContent value="contacts"&gt;
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead&gt;Name&lt;/TableHead&gt;
              &lt;TableHead&gt;Email&lt;/TableHead&gt;
              &lt;TableHead&gt;Phone&lt;/TableHead&gt;
            &lt;/TableRow&gt;
          &lt;/TableHeader&gt;
          &lt;TableBody&gt;
            {employees.length === 0 &amp;&amp; (
              &lt;TableRow&gt;&lt;TableCell colSpan={3} className="text-center text-muted-foreground py-8"&gt;No contacts&lt;/TableCell&gt;&lt;/TableRow&gt;
            )}
            {employees.map((e) =&gt; (
              &lt;TableRow key={e.id}&gt;
                &lt;TableCell className="font-medium"&gt;{e.name}&lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;{e.email || "—"}&lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;{e.phone || "—"}&lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/TabsContent&gt;

  &lt;TabsContent value="files"&gt;
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead&gt;File&lt;/TableHead&gt;
              &lt;TableHead&gt;Project&lt;/TableHead&gt;
              &lt;TableHead&gt;Date&lt;/TableHead&gt;
            &lt;/TableRow&gt;
          &lt;/TableHeader&gt;
          &lt;TableBody&gt;
            {files.length === 0 &amp;&amp; (
              &lt;TableRow&gt;&lt;TableCell colSpan={3} className="text-center text-muted-foreground py-8"&gt;No files&lt;/TableCell&gt;&lt;/TableRow&gt;
            )}
            {files.map((f) =&gt; (
              &lt;TableRow key={f.id}&gt;
                &lt;TableCell&gt;
                  &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer" className="text-primary hover:underline text-sm font-medium"&gt;
                    {f.file_name}
                  &lt;/a&gt;
```

```
                    &lt;/TableCell&gt;
                    &lt;TableCell className="text-sm text-muted-foreground"&gt;{f.projects?.name || "—"}&lt;/TableCell&gt;
                    &lt;TableCell className="text-sm text-muted-foreground"&gt;{format(new Date(f.created_at), "MMM d, yyyy")}&lt;/TableCell&gt;
                  &lt;/TableRow&gt;
                ))}
              &lt;/TableBody&gt;
            &lt;/Table&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/TabsContent&gt;

      &lt;TabsContent value="messages"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4"&gt;
            {messages.length === 0 &amp;&amp; (
              &lt;p className="text-center text-muted-foreground py-8"&gt;No messages&lt;/p&gt;
            )}
            &lt;div className="space-y-3 max-h-[500px] overflow-y-auto"&gt;
              {messages.map((m) =&gt; (
                &lt;div key={m.id} className="flex gap-3 p-2 rounded-md hover:bg-muted/50"&gt;
                  &lt;div className="flex-1"&gt;
                    &lt;p className="text-sm"&gt;{m.content}&lt;/p&gt;
                    &lt;p className="text-xs text-muted-foreground mt-1"&gt;
                      {m.projects?.name} · {format(new Date(m.created_at), "MMM d, h:mm a")}
                      {m.is_from_admin &amp;&amp; &lt;Badge variant="outline" className="ml-2 text-[9px]"&gt;Admin&lt;/Badge&gt;}
                    &lt;/p&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/TabsContent&gt;
    &lt;/Tabs&gt;
  &lt;/div&gt;
);
};

export default AdminClientProfile;
```

src/pages/admin/AdminCompanies.tsx

```
import { useEffect, useState, useRef } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { Plus, Pencil, Trash2, Search, Camera, Building2, Users, UserPlus } from "lucide-react";
import { toast } from "sonner";
import { Separator } from "@/components/ui/separator";
import CompanyEmployees from "@/components/admin/CompanyEmployees";
import { getCountryFlag, getLanguageLabel } from "@/utils/countryFlags";

interface Company {
  id: string;
  name: string;
  owner_name: string | null;
  email: string | null;
  personal_email: string | null;
  phone: string | null;
  company_type: string | null;
  country: string | null;
  status: string | null;
  website: string | null;
  instagram: string | null;
  personal_instagram: string | null;
  linkedin: string | null;
  street_name: string | null;
  street_number: string | null;
  unit: string | null;
  neighborhood: string | null;
  city: string | null;
  state: string | null;
  zip_code: string | null;
  avatar_url: string | null;
  language: string | null;
}

const TYPES = ["Construtora", "Cliente Final", "Real Estate", "Builder", "Contractor", "Client", "Architect", "Engineering"];
const COUNTRIES = ["USA", "Brazil", "Other"];
const STATUSES = ["active", "inactive"];

const LANGUAGES = [
  { value: "en", label: "English ■■" },
  { value: "pt", label: "Portuguese ■■" },
];

const emptyForm = {
  name: "", owner_name: "", email: "", personal_email: "", phone: "",
  company_type: "Client", country: "USA", status: "active", language: "en",
  website: "", instagram: "", personal_instagram: "", linkedin: "",
  street_name: "", street_number: "", unit: "", neighborhood: "",
  city: "", state: "", zip_code: "", suggested_password: "",
};

const AdminCompanies = () =&gt; {
  const [companies, setCompanies] = useState&lt;Company[]&gt;([]);
  const [search, setSearch] = useState("");
  const [filterStatus, setFilterStatus] = useState&lt;string&gt;("all");
  const [filterCountry, setFilterCountry] = useState&lt;string&gt;("all");
  const [groupBy, setGroupBy] = useState&lt;"none" | "country" | "company_type" | "city"&gt;("none");
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editing, setEditing] = useState&lt;Company | null&gt;(null);
  const [form, setForm] = useState(emptyForm);
  const [avatarPreview, setAvatarPreview] = useState&lt;string | null&gt;(null);
  const [avatarFile, setAvatarFile] = useState&lt;File | null&gt;(null);
  const fileInputRef = useRef&lt;HTMLInputElement&gt;(null);
  const [employeesDialogOpen, setEmployeesDialogOpen] = useState(false);
  const [selectedCompany, setSelectedCompany] = useState&lt;Company | null&gt;(null);

  const fetchCompanies = async () =&gt; {
    const { data } = await supabase.from("companies").select("*").order("name");
    setCompanies((data as Company[]) || []);
  };

  useEffect(() =&gt; { fetchCompanies(); }, []);

  const uploadAvatar = async (companyId: string): Promise&lt;string | null&gt; =&gt; {
    if (!avatarFile) return editing?.avatar_url || null;
    const ext = avatarFile.name.split(".").pop();
    const path = `${companyId}.${ext}`;
    const { error } = await supabase.storage.from("company-avatars").upload(path, avatarFile, { upsert: true });
    if (error) { toast.error("Erro ao enviar imagem"); return null; }
    const { data } = supabase.storage.from("company-avatars").getPublicUrl(path);
    return `${data.publicUrl}?t=${Date.now()}`;
  };

  const handleAvatarChange = (e: React.ChangeEvent&lt;HTMLInputElement&gt;) =&gt; {
    const file = e.target.files?.[0];
    if (!file) return;
    setAvatarFile(file);
    setAvatarPreview(URL.createObjectURL(file));
  };
```

```
const handleSave = async () =&gt; {
  const payload: any = {
    name: form.name || null, owner_name: form.owner_name || null,
    email: form.email || null, personal_email: form.personal_email || null,
    phone: form.phone || null,
    company_type: form.company_type, country: form.country, status: form.status, language: form.language,
    website: form.website || null,
    instagram: form.instagram || null, personal_instagram: form.personal_instagram || null,
    linkedin: form.linkedin || null,
    street_name: form.street_name || null, street_number: form.street_number || null,
    unit: form.unit || null, neighborhood: form.neighborhood || null,
    city: form.city || null, state: form.state || null, zip_code: form.zip_code || null,
  };

  let companyId = editing?.id;

  if (editing) {
    const avatarUrl = await uploadAvatar(editing.id);
    if (avatarUrl) payload.avatar_url = avatarUrl;
    const { error } = await supabase.from("companies").update(payload).eq("id", editing.id);
    if (error) { toast.error(error.message); return; }
    companyId = editing.id;
    toast.success("Company updated");
  } else {
    const { data, error } = await supabase.from("companies").insert({ ...payload, name: payload.name || "No name" }).select("id").single();
    if (error) { toast.error(error.message); return; }
    companyId = data.id;
    if (avatarFile) {
      const avatarUrl = await uploadAvatar(companyId);
      if (avatarUrl) {
        await supabase.from("companies").update({ avatar_url: avatarUrl }).eq("id", companyId);
      }
    }
    toast.success("Company created");
  }

  // Create client login account if password is provided
  if (form.suggested_password &amp;amp;&amp;amp; form.email &amp;amp;&amp;amp; companyId) {
    try {
      const { data: fnData, error: fnError } = await supabase.functions.invoke("create-client-account", {
        body: {
          email: form.email,
          password: form.suggested_password,
          full_name: form.owner_name || form.name,
          company_id: companyId,
        },
      });
      if (fnError) {
        toast.error(`Account error: ${fnError.message}`);
      } else if (fnData?.error) {
        toast.error(`Account error: ${fnData.error}`);
      } else {
        toast.success(`Client login created: ${form.email}`);
      }
    } catch (err: any) {
      toast.error(`Failed to create client login: ${err.message}`);
    }
  }

  setDialogOpen(false); setEditing(null); setForm(emptyForm);
  setAvatarFile(null); setAvatarPreview(null);
  fetchCompanies();
};

const handleDelete = async (id: string) =&gt; {
  if (!confirm("Tem certeza que deseja excluir?")) return;
  const { error } = await supabase.from("companies").delete().eq("id", id);
  if (error) { toast.error(error.message); return; }
  toast.success("Empresa excluída"); fetchCompanies();
};

const openEdit = (c: Company) =&gt; {
  setEditing(c);
  setAvatarPreview(c.avatar_url || null);
  setAvatarFile(null);
  setForm({
    name: c.name || "", owner_name: c.owner_name || "", email: c.email || "",
    personal_email: c.personal_email || "",
    phone: c.phone || "",
    company_type: c.company_type || "Client", country: c.country || "USA", status: c.status || "active", language: (c as any).language || "en",
    website: c.website || "",
    instagram: c.instagram || "", personal_instagram: c.personal_instagram || "",
    linkedin: c.linkedin || "",
    street_name: c.street_name || "", street_number: c.street_number || "", unit: c.unit || "",
    neighborhood: c.neighborhood || "", city: c.city || "", state: c.state || "", zip_code: c.zip_code || "",
    suggested_password: "",
  });
  setDialogOpen(true);
};

const openNew = () =&gt; {
  setEditing(null); setForm(emptyForm);
  setAvatarFile(null); setAvatarPreview(null);
  setDialogOpen(true);
};

const openEmployees = (c: Company) =&gt; {
  setSelectedCompany(c);
  setEmployeesDialogOpen(true);
};
```

```
const filtered = companies.filter(c =&gt; {
  const matchSearch = (c.name || "").toLowerCase().includes(search.toLowerCase()) ||
    (c.owner_name || "").toLowerCase().includes(search.toLowerCase()) ||
    (c.city || "").toLowerCase().includes(search.toLowerCase());
  const matchStatus = filterStatus === "all" || c.status === filterStatus;
  const matchCountry = filterCountry === "all" || c.country === filterCountry;
  return matchSearch &amp;&amp; matchStatus &amp;&amp; matchCountry;
});

const grouped = () =&gt; {
  if (groupBy === "none") return { "": filtered };
  const map: Record&lt;string, Company[]&gt; = {};
  filtered.forEach(c =&gt; {
    const key = (c as any)[groupBy] || "Sem definição";
    if (!map[key]) map[key] = [];
    map[key].push(c);
  });
  return Object.fromEntries(Object.entries(map).sort(([a], [b]) =&gt; a.localeCompare(b)));
};

const getInitials = (name: string) =&gt; {
  return name.split(" ").map(w =&gt; w[0]).join("").toUpperCase().slice(0, 2);
};

return (
  &lt;div className="p-6 space-y-6"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Empresas&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Gerencie seus clientes e parceiros&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={dialogOpen} onOpenChange={(o) =&gt; { setDialogOpen(o); if (!o) { setEditing(null); setForm(emptyForm); setAvatarFile(null); setAvatarPreview(null);
        &lt;DialogTrigger asChild&gt;
          &lt;Button onClick={openNew}&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;Nova Empresa&lt;/Button&gt;
        &lt;/DialogTrigger&gt;
        &lt;DialogContent className="max-w-lg max-h-[85vh] overflow-y-auto"&gt;
          &lt;DialogHeader&gt;
            &lt;DialogTitle&gt;{editing ? "Editar Empresa" : "Nova Empresa"}&lt;/DialogTitle&gt;
          &lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            {/* Avatar Upload */}
            &lt;div className="flex flex-col items-center gap-2"&gt;
              &lt;div className="relative cursor-pointer" onClick={{() =&gt; fileInputRef.current?.click()}&gt;
                &lt;Avatar className="h-20 w-20 border-2 border-border"&gt;
                  &lt;AvatarImage src={avatarPreview || undefined} /&gt;
                  &lt;AvatarFallback className="bg-muted text-muted-foreground text-lg"&gt;
                    {form.name ? getInitials(form.name) : &lt;Building2 className="h-8 w-8" /&gt;}
                  &lt;/AvatarFallback&gt;
                &lt;/Avatar&gt;
                &lt;div className="absolute bottom-0 right-0 bg-primary text-primary-foreground rounded-full p-1"&gt;
                  &lt;Camera className="h-3 w-3" /&gt;
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;p className="text-xs text-muted-foreground"&gt;Clique para adicionar foto&lt;/p&gt;
              &lt;input ref={fileInputRef} type="file" accept="image/*" className="hidden" onChange={handleAvatarChange} /&gt;
            &lt;/div&gt;

            {/* Section 1 — Responsável (FIRST) */}
            &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Responsável&lt;/p&gt;
            &lt;div&gt;&lt;Label&gt;Nome do Responsável&lt;/Label&gt;&lt;Input value={form.owner_name} onChange={e =&gt; setForm({ ...form, owner_name: e.target.value })}
            &lt;div className="grid grid-cols-2 gap-4"&gt;
              &lt;div&gt;&lt;Label&gt;Telefone&lt;/Label&gt;&lt;Input value={form.phone} onChange={e =&gt; setForm({ ...form, phone: e.target.value })} /&gt;&lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Email Pessoal&lt;/Label&gt;&lt;Input type="email" value={form.personal_email} onChange={e =&gt; setForm({ ...form, personal_email: e.
            &lt;/div&gt;
            &lt;div&gt;&lt;Label&gt;Instagram Pessoal&lt;/Label&gt;&lt;Input value={form.personal_instagram} onChange={e =&gt; setForm({ ...form, personal_instagram: e.tar

            &lt;Separator /&gt;
            {/* Section 2 — Empresa (AFTER) */}
            &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Empresa&lt;/p&gt;
            &lt;div&gt;&lt;Label&gt;Nome da Empresa&lt;/Label&gt;&lt;Input value={form.name} onChange={e =&gt; setForm({ ...form, name: e.target.value })} /&gt;&lt;/div&gt
            &lt;div class="grid grid-cols-2 gap-4"&gt;
              &lt;div&gt;&lt;Label&gt;Tipo&lt;/Label&gt;
                &lt;Select value={form.company_type} onValueChange={v =&gt; setForm({ ...form, company_type: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{TYPES.map(t =&gt; &lt;SelectItem key={t} value={t}&gt;{t}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Status&lt;/Label&gt;
                &lt;Select value={form.status} onValueChange={v =&gt; setForm({ ...form, status: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{STATUSES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{s === "active" ? "Ativo" : "Inativo"}&lt;/SelectItem&gt;)}&lt;/SelectCont
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Language&lt;/Label&gt;
                &lt;Select value={form.language} onValueChange={v =&gt; setForm({ ...form, language: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{LANGUAGES.map(l =&gt; &lt;SelectItem key={l.value} value={l.value}&gt;{l.label}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div&gt;&lt;Label&gt;Email Profissional&lt;/Label&gt;&lt;Input type="email" value={form.email} onChange={e =&gt; setForm({ ...form, email: e.target.value }
            &lt;div&gt;&lt;Label&gt;Instagram Profissional&lt;/Label&gt;&lt;Input value={form.instagram} onChange={e =&gt; setForm({ ...form, instagram: e.target.value }
            &lt;div&gt;&lt;Label&gt;Website&lt;/Label&gt;&lt;Input value={form.website} onChange={e =&gt; setForm({ ...form, website: e.target.value })} placeholder="https
            &lt;div&gt;&lt;Label&gt;LinkedIn&lt;/Label&gt;&lt;Input value={form.linkedin} onChange={e =&gt; setForm({ ...form, linkedin: e.target.value })} placeholder="UR

            &lt;Separator /&gt;
            {/* Section 3 — Client Area */}
            &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Área do Cliente&lt;/p&gt;
            &lt;div class="p-3 rounded-md bg-muted border border-border space-y-2"&gt;
              &lt;p class="text-xs text-muted-foreground"&gt;Login e senha para acesso à área do cliente&lt;/p&gt;
              &lt;div&gt;&lt;Label&gt;Login (Email)&lt;/Label&gt;&lt;Input value={form.email} disabled class="bg-background" /&gt;&lt;/div&gt;
```

```
          &lt;div&gt;&lt;Label&gt;Senha Sugerida&lt;/Label&gt;
            &lt;div className="flex gap-2"&gt;
              &lt;Input value={form.suggested_password || ""} onChange={e =&gt; setForm({ ...form, suggested_password: e.target.value })} placeholder="Senha padrão" /&
              &lt;Button type="button" variant="outline" size="sm" onClick={() =&gt; {
                const pwd = `Dara${Math.random().toString(36).slice(2, 8)}!`;
                setForm({ ...form, suggested_password: pwd });
              }}&gt;Gerar&lt;/Button&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;Separator /&gt;
        {/* Section 4 — Address */}
        &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Endereço&lt;/p&gt;
        &lt;div className="grid grid-cols-3 gap-4"&gt;
          &lt;div class="col-span-2"&gt;&lt;Label&gt;Rua&lt;/Label&gt;&lt;Input value={form.street_name} onChange={e =&gt; setForm({ ...form, street_name: e.target
          &lt;div&gt;&lt;Label&gt;Número&lt;/Label&gt;&lt;Input value={form.street_number} onChange={e =&gt; setForm({ ...form, street_number: e.target.value })} /&gt;
        &lt;/div&gt;
        &lt;div class="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;Complemento&lt;/Label&gt;&lt;Input value={form.unit} onChange={e =&gt; setForm({ ...form, unit: e.target.value })} /&gt;&lt;/div&gt;
          &lt;div&gt;&lt;Label&gt;Bairro&lt;/Label&gt;&lt;Input value={form.neighborhood} onChange={e =&gt; setForm({ ...form, neighborhood: e.target.value })} /&gt;&l
        &lt;/div&gt;
        &lt;div class="grid grid-cols-3 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;Cidade&lt;/Label&gt;&lt;Input value={form.city} onChange={e =&gt; setForm({ ...form, city: e.target.value })} /&gt;&lt;/div&gt;
          &lt;div&gt;&lt;Label&gt;Estado&lt;/Label&gt;&lt;Input value={form.state} onChange={e =&gt; setForm({ ...form, state: e.target.value })} /&gt;&lt;/div&gt;
          &lt;div&gt;&lt;Label&gt;CEP&lt;/Label&gt;&lt;Input value={form.zip_code} onChange={e =&gt; setForm({ ...form, zip_code: e.target.value })} /&gt;&lt;/div&gt;
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;País&lt;/Label&gt;
          &lt;Select value={form.country} onValueChange={v =&gt; setForm({ ...form, country: v })}&gt;
            &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;{COUNTRIES.map(c =&gt; &lt;SelectItem key={c} value={c}&gt;{c}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
          &lt;/Select&gt;
        &lt;/div&gt;

        &lt;Button className="w-full" onClick={handleSave}&gt;{editing ? "Salvar" : "Criar"}&lt;/Button&gt;
      &lt;/div&gt;
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;

&lt;div className="flex flex-wrap gap-3"&gt;
  &lt;div className="relative max-w-xs"&gt;
    &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
    &lt;Input placeholder="Buscar empresa..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
  &lt;/div&gt;
  &lt;Select value={filterStatus} onValueChange={setFilterStatus}&gt;
    &lt;SelectTrigger className="w-[140px]"&gt;&lt;SelectValue placeholder="Status" /&gt;&lt;/SelectTrigger&gt;
    &lt;SelectContent&gt;
      &lt;SelectItem value="all"&gt;All Status&lt;/SelectItem&gt;
      &lt;SelectItem value="active"&gt;■ Active&lt;/SelectItem&gt;
      &lt;SelectItem value="inactive"&gt;■ Inactive&lt;/SelectItem&gt;
    &lt;/SelectContent&gt;
  &lt;/Select&gt;
  &lt;Select value={filterCountry} onValueChange={setFilterCountry}&gt;
    &lt;SelectTrigger className="w-[140px]"&gt;&lt;SelectValue placeholder="Country" /&gt;&lt;/SelectTrigger&gt;
    &lt;SelectContent&gt;
      &lt;SelectItem value="all"&gt;All Countries&lt;/SelectItem&gt;
      &lt;SelectItem value="USA"&gt;■■ USA&lt;/SelectItem&gt;
      &lt;SelectItem value="Brazil"&gt;■■ Brazil&lt;/SelectItem&gt;
    &lt;/SelectContent&gt;
  &lt;/Select&gt;
  &lt;Select value={groupBy} onValueChange={v =&gt; setGroupBy(v as any)}&gt;
    &lt;SelectTrigger className="w-[180px]"&gt;&lt;SelectValue placeholder="Agrupar por" /&gt;&lt;/SelectTrigger&gt;
    &lt;SelectContent&gt;
      &lt;SelectItem value="none"&gt;Sem agrupamento&lt;/SelectItem&gt;
      &lt;SelectItem value="country"&gt;País&lt;/SelectItem&gt;
      &lt;SelectItem value="company_type"&gt;Tipo&lt;/SelectItem&gt;
      &lt;SelectItem value="city"&gt;Cidade&lt;/SelectItem&gt;
    &lt;/SelectContent&gt;
  &lt;/Select&gt;
&lt;/div&gt;

{Object.entries(grouped()).map(([group, items]) =&gt; (
  &lt;Card key={group}&gt;
    {group &amp;amp;&amp; (
      &lt;div className="px-6 pt-4 pb-2"&gt;
        &lt;h3 className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;{group}&lt;/h3&gt;
      &lt;/div&gt;
    )}
    &lt;CardContent className="p-0"&gt;
      &lt;Table&gt;
        &lt;TableHeader&gt;
          &lt;TableRow&gt;
            &lt;TableHead&gt;Responsável / Empresa&lt;/TableHead&gt;
            &lt;TableHead&gt;Tipo&lt;/TableHead&gt;
            &lt;TableHead&gt;Cidade&lt;/TableHead&gt;
            &lt;TableHead&gt;Status&lt;/TableHead&gt;
            &lt;TableHead class="w-[140px]"&gt;Ações&lt;/TableHead&gt;
          &lt;/TableRow&gt;
        &lt;/TableHeader&gt;
        &lt;TableBody&gt;
          {items.map(c =&gt; (
            &lt;TableRow key={c.id}&gt;
              &lt;TableCell class="font-medium"&gt;
                &lt;div class="flex items-center gap-3"&gt;
                  &lt;Avatar class="h-8 w-8"&gt;
                    &lt;AvatarImage src={c.avatar_url || undefined} /&gt;
                    &lt;AvatarFallback class="bg-muted text-muted-foreground text-xs"&gt;
                      {getInitials(c.name)}
                    &lt;/AvatarFallback&gt;
```

```
                    &lt;/Avatar&gt;
                    &lt;div&gt;
                      &lt;p className="font-medium text-foreground"&gt;
                        {c.owner_name || "—"} {getCountryFlag(c.country)}
                      &lt;/p&gt;
                      &lt;p className="text-xs text-muted-foreground"&gt;{c.name}&lt;/p&gt;
                    &lt;/div&gt;
                  &lt;/div&gt;
                &lt;/TableCell&gt;
                &lt;TableCell&gt;{c.company_type}&lt;/TableCell&gt;
                &lt;TableCell className="text-muted-foreground"&gt;{c.city ? `${c.city}, ${c.state || ""}` : "—"}&lt;/TableCell&gt;
                &lt;TableCell&gt;
                  &lt;Badge variant={c.status === "active" ? "default" : "secondary"}&gt;
                    {c.status === "active" ? "Ativo" : "Inativo"}
                  &lt;/Badge&gt;
                &lt;/TableCell&gt;
                &lt;TableCell&gt;
                  &lt;div className="flex gap-1"&gt;
                    &lt;Button variant="ghost" size="sm" onClick={() =&gt; openEmployees(c)} title="Funcionários"&gt;&lt;Users className="h-4 w-4 mr-1" /&gt;&lt;span cla
                    &lt;Button variant="ghost" size="icon" onClick={() =&gt; openEdit(c)}&gt;&lt;Pencil className="h-4 w-4" /&gt;&lt;/Button&gt;
                    &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleDelete(c.id)}&gt;&lt;Trash2 className="h-4 w-4 text-destructive" /&gt;&lt;/Button&gt;
                  &lt;/div&gt;
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
            {items.length === 0 &amp;&amp; (
              &lt;TableRow&gt;&lt;TableCell colSpan={5} className="text-center text-muted-foreground py-8"&gt;Nenhuma empresa encontrada&lt;/TableCell&gt;&lt;/TableRow&g
            )}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  ))}

  {/* Employees Dialog */}
  &lt;Dialog open={employeesDialogOpen} onOpenChange={setEmployeesDialogOpen}&gt;
    &lt;DialogContent className="max-w-lg max-h-[85vh] overflow-y-auto"&gt;
      &lt;DialogHeader&gt;
        &lt;DialogTitle&gt;Funcionários — {selectedCompany?.name}&lt;/DialogTitle&gt;
      &lt;/DialogHeader&gt;
      {selectedCompany &amp;&amp; &lt;CompanyEmployees companyId={selectedCompany.id} /&gt;}
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;
);
};

export default AdminCompanies;
```

src/pages/admin/AdminContracts.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Search, FileSignature, ExternalLink } from "lucide-react";
import { Button } from "@/components/ui/button";
import { format } from "date-fns";

const AdminContracts = () =&gt; {
  const [contracts, setContracts] = useState&lt;any[]&gt;([]);
  const [search, setSearch] = useState("");

  useEffect(() =&gt; {
    supabase.from("freelancer_contracts").select("*, projects(name, companies(name))").order("created_at", { ascending: false })
      .then(({ data }) =&gt; setContracts(data || []));
  }, []);

  const filtered = contracts.filter(c =&gt;
    (c.title || "").toLowerCase().includes(search.toLowerCase()) ||
    (c.projects?.name || "").toLowerCase().includes(search.toLowerCase())
  );

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Contracts&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Track contracts and agreements&lt;/p&gt;
        &lt;/div&gt;
        &lt;Badge variant="secondary"&gt;{contracts.length} total&lt;/Badge&gt;
      &lt;/div&gt;
      &lt;div className="relative max-w-xs"&gt;
        &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
        &lt;Input placeholder="Search contracts..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Title&lt;/TableHead&gt;
                &lt;TableHead&gt;Project&lt;/TableHead&gt;
                &lt;TableHead&gt;Client&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Created&lt;/TableHead&gt;
                &lt;TableHead className="w-[50px]"&gt;&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {filtered.length === 0 &amp;amp;&amp;amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={6} className="text-center text-muted-foreground py-12"&gt;&lt;FileSignature className="h-8 w-8 mx-auto mb-2 opacity-40
              )}
              {filtered.map(c =&gt; (
                &lt;TableRow key={c.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{c.title}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{c.projects?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{c.projects?.companies?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    {c.is_signed
                      ? &lt;Badge className="bg-emerald-500/10 text-emerald-600" variant="outline"&gt;Signed&lt;/Badge&gt;
                      : &lt;Badge variant="outline"&gt;Pending&lt;/Badge&gt;}
                  &lt;/TableCell&gt;
                  &lt;TableCell className="text-xs text-muted-foreground"&gt;{format(new Date(c.created_at), "MMM d, yyyy")}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    {c.file_url &amp;amp;&amp;amp; (
                      &lt;Button variant="ghost" size="icon" className="h-7 w-7" asChild&gt;
                        &lt;a href={c.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;ExternalLink className="h-3.5 w-3.5" /&gt;&lt;/a&gt;
                      &lt;/Button&gt;
                    )}
                  &lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminContracts;
```

src/pages/admin/AdminDashboard.tsx

```
import { useEffect, useState } from "react";
import { useAuth } from "@/hooks/useAuth";
import { supabase } from "@/integrations/supabase/client";
import KPICards from "@/components/admin/dashboard/KPICards";
import ProjectPipeline from "@/components/admin/dashboard/ProjectPipeline";
import ActivityFeed from "@/components/admin/dashboard/ActivityFeed";
import ClientActivityFeed from "@/components/admin/dashboard/ClientActivityFeed";
import TodaysTasks from "@/components/admin/dashboard/TodaysTasks";
import FinancialChart from "@/components/admin/dashboard/FinancialChart";
import NewLeadsTable from "@/components/admin/dashboard/NewLeadsTable";
import QuickActions from "@/components/admin/dashboard/QuickActions";
import ClientTicketsWidget from "@/components/admin/dashboard/ClientTicketsWidget";

const AdminDashboard = () =&gt; {
  const { user } = useAuth();
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [companies, setCompanies] = useState&lt;any[]&gt;([]);
  const [activities, setActivities] = useState&lt;any[]&gt;([]);
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [clientActivities, setClientActivities] = useState&lt;any[]&gt;([]);
  const [tasks, setTasks] = useState&lt;any[]&gt;([]);

  const fetchData = async () =&gt; {
    const [projRes, compRes, actRes, payRes, taskRes] = await Promise.all([
      supabase.from("projects").select("*, companies(name, country, language)"),
      supabase.from("companies").select("id, name"),
      supabase.from("activity_log").select("*, projects(name, companies(name, country))").order("created_at", { ascending: false }).limit(20),
      supabase.from("admin_payments").select("*"),
      supabase.from("admin_tasks").select("*, projects(name, companies(name))").order("due_date", { ascending: true }),
    ]);
    setProjects(projRes.data || []);
    setCompanies(compRes.data || []);
    setActivities(actRes.data || []);
    setPayments(payRes.data || []);

    // Build tasks with project name
    const taskList = (taskRes.data || []).map((t: any) =&gt; ({
      ...t,
      project_name: t.projects?.name || t.projects?.companies?.name || null,
    }));
    setTasks(taskList);

    // Build client activity feed from activity_log entries
    const clientTypes = ["approval", "upload", "revision", "message", "invoice", "download", "view"];
    const clientActs = (actRes.data || [])
      .filter((a: any) =&gt; clientTypes.includes(a.log_type) || a.log_type === "file_upload")
      .map((a: any) =&gt; ({
        id: a.id,
        client_name: a.projects?.companies?.name || "Client",
        client_country: a.projects?.companies?.country || null,
        company_id: a.project_id ? (projRes.data || []).find((p: any) =&gt; p.id === a.project_id)?.company_id : null,
        project_name: a.projects?.name || "Project",
        action: a.description,
        action_type: a.log_type === "file_upload" ? "upload" : a.log_type,
        created_at: a.created_at,
      }));
    setClientActivities(clientActs);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  const activeStages = ["In Progress", "Preview Sent", "Revision", "Final Review"];
  const activeCount = projects.filter((p) =&gt; activeStages.includes(p.stage)).length;
  const revenue = payments.filter((p) =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const pendingPayments = payments.filter((p) =&gt; p.status !== "paid").reduce((s, p) =&gt; s + Number(p.amount_total || 0), 0);
  const thirtyDaysAgo = new Date(); thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);
  const newLeads = projects.filter((p) =&gt; (p.stage === "Lead" || p.stage === "briefing") &amp;&amp; new Date(p.created_at) &gt;= thirtyDaysAgo).length;
  const delayedProjects = projects.filter((p) =&gt; p.stage === "Delayed" || (p.delivery_date &amp;&amp; new Date(p.delivery_date) &lt; new Date() &amp;&amp; p.stage !== "Co
  const proposalsSent = projects.filter((p) =&gt; p.stage === "Budget Sent" || p.stage === "Proposal Sent").length;

  return (
    &lt;div className="p-6 space-y-6 max-w-[1400px] mx-auto"&gt;
      {/* Header */}
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;
            Welcome back, {user?.user_metadata?.full_name?.split(" ")[0] || "Admin"}
          &lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Here's what's happening with your studio today.&lt;/p&gt;
        &lt;/div&gt;
        &lt;QuickActions /&gt;
      &lt;/div&gt;

      {/* KPI Cards */}
      &lt;KPICards
        totalProjects={projects.length}
        activeProjects={activeCount}
        totalClients={companies.length}
        newLeads={newLeads}
        revenue={revenue}
        pendingPayments={pendingPayments}
        delayedProjects={delayedProjects}
        proposalsSent={proposalsSent}
      /&gt;

      {/* Project Pipeline */}
      &lt;ProjectPipeline projects={projects} onProjectMoved={fetchData} /&gt;
```

```
      {/* Activity + Client Activity + Today's Tasks */}
      &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
        &lt;ActivityFeed activities={activities} /&gt;
        &lt;ClientActivityFeed activities={clientActivities} /&gt;
      &lt;/div&gt;

      {/* Tickets + Today's Tasks */}
      &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
        &lt;ClientTicketsWidget /&gt;
        &lt;TodaysTasks tasks={tasks} projects={projects} onRefresh={fetchData} /&gt;
      &lt;/div&gt;

      {/* Financial Performance */}
      &lt;FinancialChart payments={payments} projects={projects} /&gt;

      {/* New Leads */}
      &lt;NewLeadsTable /&gt;
    &lt;/div&gt;
  );
};

export default AdminDashboard;
```

src/pages/admin/AdminDrawings.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Upload, FileImage, Download, Star, Clock, Plus, ChevronRight } from "lucide-react";
import { toast } from "sonner";
import { format } from "date-fns";

const AdminDrawings = () =&gt; {
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [selectedProject, setSelectedProject] = useState&lt;string&gt;("");
  const [drawings, setDrawings] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [newDrawingName, setNewDrawingName] = useState("");
  const [newNotes, setNewNotes] = useState("");
  const [file, setFile] = useState&lt;File | null&gt;(null);
  const [uploading, setUploading] = useState(false);

  useEffect(() =&gt; {
    supabase.from("projects").select("id, name, companies(name)").then(({ data }) =&gt; {
      setProjects(data || []);
    });
  }, []);

  useEffect(() =&gt; {
    if (selectedProject) fetchDrawings();
  }, [selectedProject]);

  const fetchDrawings = async () =&gt; {
    const { data } = await supabase
      .from("drawing_versions")
      .select("*")
      .eq("project_id", selectedProject)
      .order("drawing_name", { ascending: true })
      .order("version_number", { ascending: false });
    setDrawings(data || []);
  };

  const handleUpload = async () =&gt; {
    if (!file || !selectedProject || !newDrawingName.trim()) return;
    setUploading(true);

    const ext = file.name.split(".").pop();
    const path = `${selectedProject}/${Date.now()}.${ext}`;
    const { error: uploadErr } = await supabase.storage
      .from("project-files")
      .upload(path, file);

    if (uploadErr) {
      toast.error("Upload failed");
      setUploading(false);
      return;
    }

    const { data: urlData } = supabase.storage.from("project-files").getPublicUrl(path);

    // Get next version number for this drawing name
    const existing = drawings.filter(
      (d) =&gt; d.drawing_name.toLowerCase() === newDrawingName.trim().toLowerCase()
    );
    const nextVersion = existing.length &gt; 0
      ? Math.max(...existing.map((d) =&gt; d.version_number)) + 1
      : 1;

    // Unmark previous current
    if (existing.length &gt; 0) {
      await supabase
        .from("drawing_versions")
        .update({ is_current: false })
        .eq("project_id", selectedProject)
        .eq("drawing_name", newDrawingName.trim());
    }

    const { error } = await supabase.from("drawing_versions").insert({
      project_id: selectedProject,
      drawing_name: newDrawingName.trim(),
      version_number: nextVersion,
      file_url: urlData.publicUrl,
      file_name: file.name,
      revision_notes: newNotes || null,
      is_current: true,
    });

    if (!error) {
      toast.success(`Uploaded v${nextVersion}`);
      setDialogOpen(false);
      setNewDrawingName("");
      setNewNotes("");
      setFile(null);
      fetchDrawings();
    } else {
      toast.error("Failed to save version");
```

```
  }
  setUploading(false);
};

const handleSetCurrent = async (drawing: any) =&gt; {
  await supabase
    .from("drawing_versions")
    .update({ is_current: false })
    .eq("project_id", drawing.project_id)
    .eq("drawing_name", drawing.drawing_name);

  await supabase
    .from("drawing_versions")
    .update({ is_current: true })
    .eq("id", drawing.id);

  toast.success(`v${drawing.version_number} set as current`);
  fetchDrawings();
};

// Group drawings by name
const grouped: Record&lt;string, any[]&gt; = {};
drawings.forEach((d) =&gt; {
  if (!grouped[d.drawing_name]) grouped[d.drawing_name] = [];
  grouped[d.drawing_name].push(d);
});

return (
  &lt;div className="p-6 space-y-6 max-w-[1200px] mx-auto"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Drawing Version Control&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Manage drawing versions for each project&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
        &lt;DialogTrigger asChild&gt;
          &lt;Button className="gap-2" disabled={!selectedProject}&gt;
            &lt;Plus className="h-4 w-4" /&gt; Upload Drawing
          &lt;/Button&gt;
        &lt;/DialogTrigger&gt;
        &lt;DialogContent&gt;
          &lt;DialogHeader&gt;
            &lt;DialogTitle&gt;Upload New Drawing Version&lt;/DialogTitle&gt;
          &lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            &lt;Input
              placeholder="Drawing name (e.g. Floor Plan, Elevations)"
              value={newDrawingName}
              onChange={(e) =&gt; setNewDrawingName(e.target.value)}
            /&gt;
            &lt;div className="border-2 border-dashed rounded-lg p-6 text-center"&gt;
              &lt;input
                type="file"
                accept=".pdf,.dwg,.dxf,.png,.jpg,.jpeg"
                onChange={(e) =&gt; setFile(e.target.files?.[0] || null)}
                className="hidden"
                id="drawing-upload"
              /&gt;
              &lt;label htmlFor="drawing-upload" className="cursor-pointer space-y-2"&gt;
                &lt;Upload className="h-8 w-8 mx-auto text-muted-foreground" /&gt;
                &lt;p className="text-sm text-muted-foreground"&gt;
                  {file ? file.name : "Click to select file (PDF, DWG, DXF, PNG, JPG)"}
                &lt;/p&gt;
              &lt;/label&gt;
            &lt;/div&gt;
            &lt;Textarea
              placeholder="Revision notes — what changed?"
              value={newNotes}
              onChange={(e) =&gt; setNewNotes(e.target.value)}
              rows={3}
            /&gt;
            &lt;Button
              className="w-full"
              onClick={handleUpload}
              disabled={!file || !newDrawingName.trim() || uploading}
            &gt;
              {uploading ? "Uploading..." : "Upload Version"}
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/DialogContent&gt;
      &lt;/Dialog&gt;
    &lt;/div&gt;

    {/* Project Selector */}
    &lt;Select value={selectedProject} onValueChange={setSelectedProject}&gt;
      &lt;SelectTrigger className="max-w-sm"&gt;
        &lt;SelectValue placeholder="Select a project" /&gt;
      &lt;/SelectTrigger&gt;
      &lt;SelectContent&gt;
        {projects.map((p) =&gt; (
          &lt;SelectItem key={p.id} value={p.id}&gt;
            {p.name || p.companies?.name || "Untitled"}
          &lt;/SelectItem&gt;
        ))}
      &lt;/SelectContent&gt;
    &lt;/Select&gt;

    {!selectedProject &amp;&amp; (
      &lt;p className="text-sm text-muted-foreground text-center py-16"&gt;Select a project to view drawings&lt;/p&gt;
    )}
```

```
      {selectedProject &amp;&amp; Object.keys(grouped).length === 0 &amp;&amp; (
        &lt;p className="text-sm text-muted-foreground text-center py-16"&gt;No drawings uploaded yet&lt;/p&gt;
      )}

      {/* Drawing Groups */}
      &lt;div className="space-y-6"&gt;
        {Object.entries(grouped).map(([name, versions]) =&gt; (
          &lt;Card key={name} className="border-none shadow-sm"&gt;
            &lt;CardHeader className="pb-3"&gt;
              &lt;div className="flex items-center gap-2"&gt;
                &lt;FileImage className="h-4 w-4 text-muted-foreground" /&gt;
                &lt;CardTitle className="text-base font-semibold"&gt;{name}&lt;/CardTitle&gt;
                &lt;Badge variant="secondary" className="ml-auto"&gt;{versions.length} versions&lt;/Badge&gt;
              &lt;/div&gt;
            &lt;/CardHeader&gt;
            &lt;CardContent&gt;
              &lt;ScrollArea className="max-h-[300px]"&gt;
                &lt;div className="space-y-2"&gt;
                  {versions.map((v) =&gt; (
                    &lt;div
                      key={v.id}
                      className={`flex items-center gap-3 p-3 rounded-lg border transition-colors ${
                        v.is_current ? "border-primary/30 bg-primary/5" : "border-border hover:bg-muted/50"
                      }`}
                    &gt;
                      &lt;div className="flex items-center justify-center h-9 w-9 rounded-md bg-muted text-xs font-bold text-muted-foreground shrink-0"&gt;
                        v{v.version_number}
                      &lt;/div&gt;
                      &lt;div className="flex-1 min-w-0"&gt;
                        &lt;div className="flex items-center gap-2"&gt;
                          &lt;p className="text-sm font-medium text-foreground truncate"&gt;{v.file_name}&lt;/p&gt;
                          {v.is_current &amp;&amp; (
                            &lt;Badge className="text-[10px] bg-primary/10 text-primary border-primary/20" variant="outline"&gt;
                              &lt;Star className="h-2.5 w-2.5 mr-0.5" /&gt; Current
                            &lt;/Badge&gt;
                          )}
                        &lt;/div&gt;
                        {v.revision_notes &amp;&amp; (
                          &lt;p className="text-xs text-muted-foreground mt-0.5 truncate"&gt;{v.revision_notes}&lt;/p&gt;
                        )}
                        &lt;div className="flex items-center gap-2 mt-1 text-xs text-muted-foreground"&gt;
                          &lt;Clock className="h-3 w-3" /&gt;
                          {format(new Date(v.created_at), "MMM d, yyyy 'at' h:mm a")}
                        &lt;/div&gt;
                      &lt;/div&gt;
                      &lt;div className="flex items-center gap-1 shrink-0"&gt;
                        {!v.is_current &amp;&amp; (
                          &lt;Button
                            variant="ghost"
                            size="sm"
                            className="h-7 text-xs gap-1"
                            onClick={() =&gt; handleSetCurrent(v)}
                          &gt;
                            &lt;Star className="h-3 w-3" /&gt; Set Current
                          &lt;/Button&gt;
                        )}
                        &lt;Button
                          variant="ghost"
                          size="icon"
                          className="h-7 w-7"
                          asChild
                        &gt;
                          &lt;a href={v.file_url} target="_blank" rel="noopener noreferrer" download&gt;
                            &lt;Download className="h-3.5 w-3.5" /&gt;
                          &lt;/a&gt;
                        &lt;/Button&gt;
                      &lt;/div&gt;
                    &lt;/div&gt;
                  ))}
                &lt;/div&gt;
              &lt;/ScrollArea&gt;
            &lt;/CardContent&gt;
          &lt;/Card&gt;
        ))}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminDrawings;
```

src/pages/admin/AdminFiles.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Search, FolderOpen, Download, FileIcon, ChevronRight, Building2 } from "lucide-react";
import { format } from "date-fns";

const CATEGORY_LABELS: Record&lt;string, string&gt; = {
  pre_construction: "■ Pre-Construction",
  rev_00: "■ REV 00",
  rev_01: "■ REV 01",
  rev_02: "■ REV 02",
  rev_03: "■ REV 03 (Final)",
  final_delivery: "■ Final Delivery",
  general: "■ General",
};

const CATEGORY_ORDER = ["pre_construction", "rev_00", "rev_01", "rev_02", "rev_03", "final_delivery", "general"];

const AdminFiles = () =&gt; {
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [search, setSearch] = useState("");
  const [openProjects, setOpenProjects] = useState&lt;Set&lt;string&gt;&gt;(new Set());

  useEffect(() =&gt; {
    supabase.from("project_files").select("*, projects(name, companies(name))").order("created_at", { ascending: false })
      .then(({ data }) =&gt; setFiles(data || []));
  }, []);

  const filtered = files.filter(f =&gt;
    (f.file_name || "").toLowerCase().includes(search.toLowerCase()) ||
    (f.projects?.name || "").toLowerCase().includes(search.toLowerCase()) ||
    (f.projects?.companies?.name || "").toLowerCase().includes(search.toLowerCase())
  );

  // Group by project
  const byProject = filtered.reduce((acc: Record&lt;string, { project: any; files: any[] }&gt;, f) =&gt; {
    const pid = f.project_id;
    if (!acc[pid]) {
      acc[pid] = { project: { id: pid, name: f.projects?.name || "Unnamed Project", company: f.projects?.companies?.name || "" }, files: [] };
    }
    acc[pid].files.push(f);
    return acc;
  }, {});

  const formatSize = (bytes: number | null) =&gt; {
    if (!bytes) return "—";
    if (bytes &lt; 1024) return `${bytes} B`;
    if (bytes &lt; 1048576) return `${(bytes / 1024).toFixed(1)} KB`;
    return `${(bytes / 1048576).toFixed(1)} MB`;
  };

  const toggleProject = (pid: string) =&gt; {
    setOpenProjects(prev =&gt; {
      const next = new Set(prev);
      next.has(pid) ? next.delete(pid) : next.add(pid);
      return next;
    });
  };

  const downloadAll = (projectFiles: any[]) =&gt; {
    projectFiles.forEach((f, i) =&gt; {
      setTimeout(() =&gt; {
        const a = document.createElement("a");
        a.href = f.file_url;
        a.target = "_blank";
        a.rel = "noopener noreferrer";
        a.click();
      }, i * 300);
    });
  };

  const projectEntries = Object.entries(byProject).sort((a, b) =&gt; (a[1] as any).project.name.localeCompare((b[1] as any).project.name));

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Project Files&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;All files organized by project and revision&lt;/p&gt;
        &lt;/div&gt;
        &lt;Badge variant="secondary"&gt;{files.length} files • {projectEntries.length} projects&lt;/Badge&gt;
      &lt;/div&gt;

      &lt;div className="relative max-w-xs"&gt;
        &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
        &lt;Input placeholder="Search files, projects, companies..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
      &lt;/div&gt;

      {projectEntries.length === 0 &amp;amp;&amp;amp; (
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="py-12 text-center text-muted-foreground"&gt;
            &lt;FolderOpen className="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;
            No files yet
          &lt;/CardContent&gt;
```

```
        &lt;/Card&gt;
      )}

      &lt;div className="space-y-3"&gt;
        {projectEntries.map(([pid, entry]) =&gt; {
          const { project, files: pFiles } = entry as { project: any; files: any[] };
          const isOpen = openProjects.has(pid);
          const categorized = CATEGORY_ORDER.reduce((acc: Record&lt;string, any[]&gt;, cat) =&gt; {
            const catFiles = pFiles.filter(f =&gt; (f.document_category || "general") === cat);
            if (catFiles.length &gt; 0) acc[cat] = catFiles;
            return acc;
          }, {});

          return (
            &lt;Collapsible key={pid} open={isOpen} onOpenChange={() =&gt; toggleProject(pid)}&gt;
              &lt;Card className="border shadow-sm"&gt;
                &lt;CollapsibleTrigger asChild&gt;
                  &lt;CardHeader className="cursor-pointer hover:bg-muted/50 transition-colors py-3 px-4"&gt;
                    &lt;div className="flex items-center justify-between"&gt;
                      &lt;div className="flex items-center gap-3"&gt;
                        &lt;ChevronRight className={`h-4 w-4 text-muted-foreground transition-transform ${isOpen ? "rotate-90" : ""}`} /&gt;
                        &lt;div&gt;
                          &lt;CardTitle className="text-sm font-semibold"&gt;{project.name}&lt;/CardTitle&gt;
                          {project.company &amp;&amp; (
                            &lt;p className="text-xs text-muted-foreground flex items-center gap-1 mt-0.5"&gt;
                              &lt;Building2 className="h-3 w-3" /&gt; {project.company}
                            &lt;/p&gt;
                          )}
                        &lt;/div&gt;
                      &lt;/div&gt;
                      &lt;div className="flex items-center gap-2"&gt;
                        &lt;Badge variant="outline" className="text-xs"&gt;{pFiles.length} files&lt;/Badge&gt;
                        &lt;Button
                          variant="outline"
                          size="sm"
                          className="h-7 gap-1.5 text-xs"
                          onClick={(e) =&gt; { e.stopPropagation(); downloadAll(pFiles); }}
                        &gt;
                          &lt;Download className="h-3 w-3" /&gt; Download All
                        &lt;/Button&gt;
                      &lt;/div&gt;
                    &lt;/div&gt;
                  &lt;/CardHeader&gt;
                &lt;/CollapsibleTrigger&gt;
                &lt;CollapsibleContent&gt;
                  &lt;CardContent className="pt-0 pb-3 px-4 space-y-3"&gt;
                    {Object.entries(categorized).map(([cat, catFiles]) =&gt; (
                      &lt;div key={cat} className="space-y-1"&gt;
                        &lt;p className="text-xs font-medium text-muted-foreground pl-1"&gt;{CATEGORY_LABELS[cat] || cat}&lt;/p&gt;
                        &lt;div className="space-y-0.5"&gt;
                          {catFiles.map((f: any) =&gt; (
                            &lt;div key={f.id} className="flex items-center justify-between py-1.5 px-2 rounded hover:bg-muted/50 group"&gt;
                              &lt;div className="flex items-center gap-2 min-w-0 flex-1"&gt;
                                &lt;FileIcon className="h-3.5 w-3.5 text-muted-foreground flex-shrink-0" /&gt;
                                &lt;span className="text-sm truncate"&gt;{f.file_name}&lt;/span&gt;
                                &lt;Badge variant="outline" className="text-[9px] flex-shrink-0"&gt;{f.file_type || "—"}&lt;/Badge&gt;
                              &lt;/div&gt;
                              &lt;div className="flex items-center gap-3 text-xs text-muted-foreground flex-shrink-0"&gt;
                                &lt;span&gt;{formatSize(f.file_size)}&lt;/span&gt;
                                &lt;span&gt;{format(new Date(f.created_at), "MMM d, yyyy")}&lt;/span&gt;
                                &lt;Button variant="ghost" size="icon" className="h-6 w-6 opacity-0 group-hover:opacity-100 transition-opacity" asChild&gt;
                                  &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-3 w-3" /&gt;&lt;/a&gt;
                                &lt;/Button&gt;
                              &lt;/div&gt;
                            &lt;/div&gt;
                          ))}
                        &lt;/div&gt;
                      &lt;/div&gt;
                    ))}
                  &lt;/CardContent&gt;
                &lt;/CollapsibleContent&gt;
              &lt;/Card&gt;
            &lt;/Collapsible&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminFiles;
```

src/pages/admin/AdminFinanceOverview.tsx

```
import { useEffect, useState, useMemo } from "react";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Progress } from "@/components/ui/progress";
import {
  DollarSign, TrendingUp, Clock, CheckCircle2, Receipt,
  ExternalLink, FileText, Download, Bell,
} from "lucide-react";
import { format, startOfMonth, endOfMonth, addDays, subMonths } from "date-fns";
import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer,
} from "recharts";
import { toast } from "sonner";

const AdminFinanceOverview = () =&gt; {
  const navigate = useNavigate();
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [milestones, setMilestones] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);

  const fetchData = async () =&gt; {
    const [payRes, msRes, projRes] = await Promise.all([
      supabase.from("admin_payments").select("*, projects(name, street_name, street_number, companies(name))").order("created_at", { ascending: false }),
      supabase.from("payment_milestones").select("*, projects(name, street_name, street_number, companies(name))").order("created_at", { ascending: false }),
      supabase.from("projects").select("id, name, street_name, street_number, total_value, currency, companies(name)"),
    ]);
    setPayments(payRes.data || []);
    setMilestones(msRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  // Realtime
  useEffect(() =&gt; {
    const channel = supabase
      .channel("finance-overview")
      .on("postgres_changes", { event: "*", schema: "public", table: "admin_payments" }, () =&gt; fetchData())
      .on("postgres_changes", { event: "*", schema: "public", table: "payment_milestones" }, () =&gt; fetchData())
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, []);

  const now = new Date();
  const monthStart = startOfMonth(now);
  const monthEnd = endOfMonth(now);
  const next30 = addDays(now, 30);

  // KPI calculations
  const totalRevenue = payments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const revenueThisMonth = payments
    .filter(p =&gt; p.status === "paid" &amp;&amp; p.payment_date &amp;&amp; new Date(p.payment_date) &gt;= monthStart &amp;&amp; new Date(p.payment_date) &lt;= monthEnd)
    .reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const outstandingFromPayments = payments.filter(p =&gt; p.status === "pending").reduce((s, p) =&gt; s + Number(p.amount_total || 0), 0);
  const outstandingFromMilestones = milestones.filter(m =&gt; m.status === "pending" || m.status === "overdue").reduce((s, m) =&gt; s + Number(m.amount || 0), 0);
  const outstandingRevenue = outstandingFromPayments + outstandingFromMilestones;
  const paidInvoicesCount = payments.filter(p =&gt; p.status === "paid").length;

  // Outstanding invoices (pending milestones + pending payments)
  const outstandingInvoices = useMemo(() =&gt; {
    const fromMilestones = milestones
      .filter(m =&gt; m.status === "pending" || m.status === "overdue")
      .map(m =&gt; ({
        id: m.id,
        type: "milestone" as const,
        label: m.stage,
        project: [m.projects?.street_number, m.projects?.street_name].filter(Boolean).join(" ") || m.projects?.name || "—",
        client: m.projects?.companies?.name || "—",
        amount: Number(m.amount || 0),
        status: m.status,
        date: m.created_at,
      }));
    const fromPayments = payments
      .filter(p =&gt; p.status === "pending")
      .map(p =&gt; ({
        id: p.id,
        type: "payment" as const,
        label: p.installment_type || "Invoice",
        project: [p.projects?.street_number, p.projects?.street_name].filter(Boolean).join(" ") || p.projects?.name || "—",
        client: p.projects?.companies?.name || "—",
        amount: Number(p.amount_total || 0),
        status: p.status,
        date: p.created_at,
      }));
    return [...fromMilestones, ...fromPayments];
  }, [milestones, payments]);

  // Recent payments (last 10 paid)
  const recentPayments = useMemo(() =&gt;
    payments.filter(p =&gt; p.status === "paid").slice(0, 10), [payments]);

  // Revenue forecast (pending with dates in next 30 days)
  const revenueForecast = useMemo(() =&gt; {
    const pendingMs = milestones
```

```
    .filter(m =&gt; (m.status === "pending" || m.status === "overdue"))
    .reduce((s, m) =&gt; s + Number(m.amount || 0), 0);
  const pendingPay = payments
    .filter(p =&gt; p.status === "pending")
    .reduce((s, p) =&gt; s + Number(p.amount_total || 0), 0);
  return pendingMs + pendingPay;
}, [milestones, payments]);

// Monthly revenue chart (last 6 months)
const monthlyData = useMemo(() =&gt; {
  const months: { label: string; revenue: number }[] = [];
  for (let i = 5; i &gt;= 0; i--) {
    const d = subMonths(now, i);
    const ms = startOfMonth(d);
    const me = endOfMonth(d);
    const rev = payments
      .filter(p =&gt; p.status === "paid" &amp;&amp; p.payment_date &amp;&amp; new Date(p.payment_date) &gt;= ms &amp;&amp; new Date(p.payment_date) &lt;= me)
      .reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
    months.push({ label: format(ms, "MMM yyyy"), revenue: rev });
  }
  return months;
}, [payments]);

const fmtCurrency = (v: number) =&gt; `$${v.toLocaleString("en-US", { minimumFractionDigits: 2, maximumFractionDigits: 2 })}`;

return (
  &lt;div className="p-6 space-y-6 max-w-[1400px] mx-auto"&gt;
    {/* Header */}
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Finance Overview&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Complete financial status of your business&lt;/p&gt;
      &lt;/div&gt;
      &lt;div className="flex items-center gap-2"&gt;
        &lt;Button variant="outline" size="sm" onClick={() =&gt; navigate("/adm/payments")}&gt;
          &lt;Receipt className="h-3.5 w-3.5 mr-1" /&gt; Payments Page
        &lt;/Button&gt;
        &lt;Button variant="outline" size="sm" onClick={() =&gt; {
          toast.info("Export coming soon");
        }}&gt;
          &lt;Download className="h-3.5 w-3.5 mr-1" /&gt; Export Report
        &lt;/Button&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    {/* KPI Cards */}
    &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4"&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4 flex items-center gap-3"&gt;
          &lt;div className="p-2.5 rounded-lg bg-chart-2/15"&gt;
            &lt;DollarSign className="h-5 w-5 text-chart-2" /&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;{fmtCurrency(totalRevenue)}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Total Revenue&lt;/p&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4 flex items-center gap-3"&gt;
          &lt;div className="p-2.5 rounded-lg bg-chart-4/15"&gt;
            &lt;TrendingUp className="h-5 w-5 text-chart-4" /&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;{fmtCurrency(revenueThisMonth)}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Revenue This Month&lt;/p&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4 flex items-center gap-3"&gt;
          &lt;div className="p-2.5 rounded-lg bg-destructive/15"&gt;
            &lt;Clock className="h-5 w-5 text-destructive" /&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;{fmtCurrency(outstandingRevenue)}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Outstanding Revenue&lt;/p&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4 flex items-center gap-3"&gt;
          &lt;div className="p-2.5 rounded-lg bg-primary/15"&gt;
            &lt;CheckCircle2 className="h-5 w-5 text-primary" /&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;{paidInvoicesCount}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Paid Invoices&lt;/p&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;

    {/* Revenue Forecast + Monthly Chart */}
    &lt;div className="grid grid-cols-1 lg:grid-cols-3 gap-6"&gt;
      {/* Forecast card */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;CardTitle className="text-base"&gt;Revenue Forecast&lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent className="space-y-4"&gt;
```

```
      &lt;div className="text-center py-4"&gt;
        &lt;p className="text-3xl font-bold text-foreground"&gt;{fmtCurrency(revenueForecast)}&lt;/p&gt;
        &lt;p className="text-sm text-muted-foreground mt-1"&gt;Projected (pending invoices)&lt;/p&gt;
      &lt;/div&gt;
      &lt;div className="space-y-2"&gt;
        &lt;div className="flex justify-between text-sm"&gt;
          &lt;span className="text-muted-foreground"&gt;Pending Milestones&lt;/span&gt;
          &lt;span className="font-medium"&gt;{fmtCurrency(outstandingFromMilestones)}&lt;/span&gt;
        &lt;/div&gt;
        &lt;div className="flex justify-between text-sm"&gt;
          &lt;span className="text-muted-foreground"&gt;Pending Payments&lt;/span&gt;
          &lt;span className="font-medium"&gt;{fmtCurrency(outstandingFromPayments)}&lt;/span&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      {outstandingRevenue &gt; 0 &amp;&amp; totalRevenue &gt; 0 &amp;&amp; (
        &lt;div&gt;
          &lt;div className="flex justify-between text-xs text-muted-foreground mb-1"&gt;
            &lt;span&gt;Collection Progress&lt;/span&gt;
            &lt;span&gt;{Math.round((totalRevenue / (totalRevenue + outstandingRevenue)) * 100)}%&lt;/span&gt;
          &lt;/div&gt;
          &lt;Progress value={(totalRevenue / (totalRevenue + outstandingRevenue)) * 100} className="h-2" /&gt;
        &lt;/div&gt;
      )}
    &lt;/CardContent&gt;
  &lt;/Card&gt;

  {/* Monthly Revenue Chart */}
  &lt;Card className="border-none shadow-sm lg:col-span-2"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;CardTitle className="text-base"&gt;Monthly Revenue&lt;/CardTitle&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent&gt;
      &lt;div className="h-[250px]"&gt;
        &lt;ResponsiveContainer width="100%" height="100%"&gt;
          &lt;BarChart data={monthlyData}&gt;
            &lt;CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" /&gt;
            &lt;XAxis dataKey="label" tick={{ fontSize: 12 }} stroke="hsl(var(--muted-foreground))" /&gt;
            &lt;YAxis tick={{ fontSize: 12 }} stroke="hsl(var(--muted-foreground))" tickFormatter={(v) =&gt; `$${(v / 1000).toFixed(0)}k`} /&gt;
            &lt;Tooltip
              formatter={(value: number) =&gt; [fmtCurrency(value), "Revenue"]}
              contentStyle={{
                background: "hsl(var(--card))",
                border: "1px solid hsl(var(--border))",
                borderRadius: "8px",
                fontSize: "12px",
              }}
            /&gt;
            &lt;Bar dataKey="revenue" fill="hsl(var(--chart-2))" radius={[4, 4, 0, 0]} /&gt;
          &lt;/BarChart&gt;
        &lt;/ResponsiveContainer&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Outstanding Invoices */}
&lt;Card className="border-none shadow-sm"&gt;
  &lt;CardHeader className="pb-3"&gt;
    &lt;CardTitle className="text-base"&gt;Outstanding Invoices ({outstandingInvoices.length})&lt;/CardTitle&gt;
  &lt;/CardHeader&gt;
  &lt;CardContent className="p-0"&gt;
    &lt;Table&gt;
      &lt;TableHeader&gt;
        &lt;TableRow&gt;
          &lt;TableHead&gt;Invoice&lt;/TableHead&gt;
          &lt;TableHead&gt;Project&lt;/TableHead&gt;
          &lt;TableHead&gt;Client&lt;/TableHead&gt;
          &lt;TableHead&gt;Amount&lt;/TableHead&gt;
          &lt;TableHead&gt;Status&lt;/TableHead&gt;
          &lt;TableHead&gt;Action&lt;/TableHead&gt;
        &lt;/TableRow&gt;
      &lt;/TableHeader&gt;
      &lt;TableBody&gt;
        {outstandingInvoices.length === 0 &amp;&amp; (
          &lt;TableRow&gt;
            &lt;TableCell colSpan={6} className="text-center text-muted-foreground py-12"&gt;
              &lt;CheckCircle2 className="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;
              No outstanding invoices
            &lt;/TableCell&gt;
          &lt;/TableRow&gt;
        )}
        {outstandingInvoices.map((inv) =&gt; (
          &lt;TableRow key={inv.id}&gt;
            &lt;TableCell className="font-medium"&gt;{inv.label}&lt;/TableCell&gt;
            &lt;TableCell className="text-muted-foreground"&gt;{inv.project}&lt;/TableCell&gt;
            &lt;TableCell className="text-muted-foreground"&gt;{inv.client}&lt;/TableCell&gt;
            &lt;TableCell className="font-medium"&gt;{fmtCurrency(inv.amount)}&lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;Badge variant="outline" className={
                inv.status === "overdue"
                  ? "bg-destructive/15 text-destructive border-destructive/30"
                  : "bg-chart-3/15 text-chart-3 border-chart-3/30"
              }&gt;
                {inv.status}
              &lt;/Badge&gt;
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;Button
                size="sm"
                variant="ghost"
```

```
                    className="text-xs"
                    onClick={() =&gt; toast.info(`Reminder for "${inv.label}" would be sent`)}
                  &gt;
                    &lt;Bell className="h-3 w-3 mr-1" /&gt; Remind
                  &lt;/Button&gt;
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Recent Payments */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;CardTitle className="text-base"&gt;Recent Payments&lt;/CardTitle&gt;
          &lt;Button variant="ghost" size="sm" className="text-xs" onClick={() =&gt; navigate("/adm/payments")}&gt;
            View All &lt;ExternalLink className="h-3 w-3 ml-1" /&gt;
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead&gt;Project&lt;/TableHead&gt;
              &lt;TableHead&gt;Client&lt;/TableHead&gt;
              &lt;TableHead&gt;Amount&lt;/TableHead&gt;
              &lt;TableHead&gt;Method&lt;/TableHead&gt;
              &lt;TableHead&gt;Date&lt;/TableHead&gt;
            &lt;/TableRow&gt;
          &lt;/TableHeader&gt;
          &lt;TableBody&gt;
            {recentPayments.length === 0 &amp;&amp; (
              &lt;TableRow&gt;
                &lt;TableCell colSpan={5} className="text-center text-muted-foreground py-12"&gt;
                  No payments yet
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            )}
            {recentPayments.map((p) =&gt; (
              &lt;TableRow key={p.id}&gt;
                &lt;TableCell className="font-medium"&gt;
                  {[p.projects?.street_number, p.projects?.street_name].filter(Boolean).join(" ") || p.projects?.name || "—"}
                &lt;/TableCell&gt;
                &lt;TableCell className="text-muted-foreground"&gt;{p.projects?.companies?.name || "—"}&lt;/TableCell&gt;
                &lt;TableCell className="font-medium text-chart-2"&gt;{fmtCurrency(Number(p.amount_paid || 0))}&lt;/TableCell&gt;
                &lt;TableCell&gt;
                  &lt;Badge variant="secondary" className="text-xs"&gt;{p.payment_method || "—"}&lt;/Badge&gt;
                &lt;/TableCell&gt;
                &lt;TableCell className="text-muted-foreground text-sm"&gt;
                  {p.payment_date ? format(new Date(p.payment_date), "MMM d, yyyy") : "—"}
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/div&gt;
  );
};

export default AdminFinanceOverview;
```

src/pages/admin/AdminFreelancers.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Users, FolderOpen, DollarSign } from "lucide-react";
import { format } from "date-fns";

const AdminFreelancers = () =&gt; {
  const [freelancers, setFreelancers] = useState&lt;any[]&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetchData = async () =&gt; {
      // Get all freelancer roles
      const { data: roles } = await supabase
        .from("user_roles")
        .select("user_id")
        .eq("role", "freelancer");

      if (!roles || roles.length === 0) {
        setLoading(false);
        return;
      }

      const freelancerIds = roles.map(r =&gt; r.user_id);

      // Get profiles, assignments, and payments in parallel
      const [profilesRes, assignmentsRes, paymentsRes] = await Promise.all([
        supabase
          .from("profiles")
          .select("id, full_name, email, avatar_url")
          .in("id", freelancerIds),
        supabase
          .from("freelancer_assignments")
          .select("freelancer_id, project_id, deadline, projects(display_address, name, stage, status, companies(name))")
          .in("freelancer_id", freelancerIds),
        supabase
          .from("freelancer_payments")
          .select("freelancer_id, agreed_amount, paid_amount, status, due_date, project_id, projects(display_address, name)")
          .in("freelancer_id", freelancerIds),
      ]);

      const profiles = profilesRes.data || [];
      const assignments = assignmentsRes.data || [];
      const payments = paymentsRes.data || [];

      const freelancerData = profiles.map(profile =&gt; {
        const myAssignments = assignments.filter((a: any) =&gt; a.freelancer_id === profile.id);
        const myPayments = payments.filter((p: any) =&gt; p.freelancer_id === profile.id);

        const activeProjects = myAssignments.filter(
          (a: any) =&gt; a.projects?.status !== "completed" &amp;&amp; a.projects?.status !== "cancelled"
        ).length;

        const totalAgreed = myPayments.reduce((s: number, p: any) =&gt; s + Number(p.agreed_amount || 0), 0);
        const totalPaid = myPayments.reduce((s: number, p: any) =&gt; s + Number(p.paid_amount || 0), 0);

        return {
          ...profile,
          assignments: myAssignments,
          payments: myPayments,
          activeProjects,
          totalAgreed,
          totalPaid,
          totalReceivable: Math.max(totalAgreed - totalPaid, 0),
        };
      });

      setFreelancers(freelancerData);
      setLoading(false);
    };

    fetchData();
  }, []);

  if (loading) {
    return &lt;div className="p-6 text-muted-foreground"&gt;Loading...&lt;/div&gt;;
  }

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Freelancers&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;View all freelancers, their assigned projects and payment status&lt;/p&gt;
      &lt;/div&gt;

      {/* Summary */}
      &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4"&gt;
        &lt;Card&gt;
          &lt;CardContent className="flex items-center gap-4 p-6"&gt;
            &lt;div className="w-12 h-12 rounded-lg bg-primary/10 flex items-center justify-center"&gt;
              &lt;Users className="h-6 w-6 text-primary" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;Total Freelancers&lt;/p&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;{freelancers.length}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
```

```
  &lt;/Card&gt;
  &lt;Card&gt;
    &lt;CardContent className="flex items-center gap-4 p-6"&gt;
      &lt;div className="w-12 h-12 rounded-lg bg-chart-2/10 flex items-center justify-center"&gt;
        &lt;DollarSign className="h-6 w-6 text-chart-2" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Total Paid&lt;/p&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;
          ${freelancers.reduce((s, f) =&gt; s + f.totalPaid, 0).toLocaleString(undefined, { minimumFractionDigits: 2 })}
        &lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  &lt;Card&gt;
    &lt;CardContent className="flex items-center gap-4 p-6"&gt;
      &lt;div className="w-12 h-12 rounded-lg bg-destructive/10 flex items-center justify-center"&gt;
        &lt;DollarSign className="h-6 w-6 text-destructive" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Total Receivable&lt;/p&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;
          ${freelancers.reduce((s, f) =&gt; s + f.totalReceivable, 0).toLocaleString(undefined, { minimumFractionDigits: 2 })}
        &lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Freelancer Cards */}
{freelancers.length === 0 ? (
  &lt;Card&gt;
    &lt;CardContent className="p-8 text-center text-muted-foreground"&gt;
      No freelancers registered yet
    &lt;/CardContent&gt;
  &lt;/Card&gt;
) : (
  &lt;div className="space-y-6"&gt;
    {freelancers.map(f =&gt; (
      &lt;Card key={f.id}&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center justify-between"&gt;
            &lt;div className="flex items-center gap-3"&gt;
              &lt;div className="w-10 h-10 rounded-full bg-primary flex items-center justify-center"&gt;
                {f.avatar_url ? (
                  &lt;img src={f.avatar_url} alt="" className="w-10 h-10 rounded-full object-cover" /&gt;
                ) : (
                  &lt;span className="text-primary-foreground font-semibold text-sm"&gt;
                    {f.full_name?.charAt(0)?.toUpperCase() || "F"}
                  &lt;/span&gt;
                )}
              &lt;/div&gt;
              &lt;div&gt;
                &lt;CardTitle className="text-lg"&gt;{f.full_name || "Unnamed"}&lt;/CardTitle&gt;
                &lt;p className="text-xs text-muted-foreground"&gt;{f.email}&lt;/p&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div className="flex items-center gap-4 text-sm"&gt;
              &lt;div className="text-center"&gt;
                &lt;p className="text-muted-foreground text-xs"&gt;Active&lt;/p&gt;
                &lt;p className="font-bold text-foreground"&gt;{f.activeProjects}&lt;/p&gt;
              &lt;/div&gt;
              &lt;div className="text-center"&gt;
                &lt;p className="text-muted-foreground text-xs"&gt;Paid&lt;/p&gt;
                &lt;p className="font-bold text-chart-2"&gt;${f.totalPaid.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
              &lt;/div&gt;
              &lt;div className="text-center"&gt;
                &lt;p className="text-muted-foreground text-xs"&gt;Receivable&lt;/p&gt;
                &lt;p className="font-bold text-destructive"&gt;${f.totalReceivable.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          {/* Assigned Projects */}
          {f.assignments.length &gt; 0 &amp;&amp; (
            &lt;div className="space-y-2"&gt;
              &lt;p className="text-xs font-medium text-muted-foreground uppercase tracking-wider"&gt;Assigned Projects&lt;/p&gt;
              &lt;div className="space-y-2"&gt;
                {f.assignments.map((a: any, i: number) =&gt; {
                  const projectLabel = a.projects?.display_address || a.projects?.name || "Unknown";
                  const stage = a.projects?.stage || "—";
                  const company = a.projects?.companies?.name || "";
                  const stageColors: Record&lt;string, string&gt; = {
                    "In Progress": "bg-chart-2/15 text-chart-2 border-chart-2/30",
                    "Completed": "bg-chart-2/15 text-chart-2 border-chart-2/30",
                    "Revision": "bg-chart-5/15 text-chart-5 border-chart-5/30",
                    "Delayed": "bg-destructive/15 text-destructive border-destructive/30",
                  };
                  const badgeClass = stageColors[stage] || "bg-muted text-muted-foreground border-border";

                  return (
                    &lt;div key={i} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
                      &lt;div className="flex items-center gap-2"&gt;
                        &lt;FolderOpen className="h-4 w-4 text-muted-foreground" /&gt;
                        &lt;div&gt;
                          &lt;span className="text-sm font-medium text-foreground"&gt;{projectLabel}&lt;/span&gt;
                          {company &amp;&amp; &lt;span className="text-xs text-muted-foreground ml-2"&gt;({company})&lt;/span&gt;}
                        &lt;/div&gt;
                      &lt;/div&gt;
                      &lt;div className="flex items-center gap-3"&gt;
```

```
                    {a.deadline &amp;&amp; (
                      &lt;span className="text-xs text-muted-foreground"&gt;
                        Due: {format(new Date(a.deadline), "MMM dd")}
                      &lt;/span&gt;
                    )}
                    &lt;Badge className={`${badgeClass} border rounded-full px-3 py-1 text-xs font-semibold`}&gt;
                      {stage}
                    &lt;/Badge&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              );
            })}
          &lt;/div&gt;
        &lt;/div&gt;
      )}

      {/* Payments */}
      {f.payments.length &gt; 0 &amp;&amp; (
        &lt;div className="space-y-2 mt-4"&gt;
          &lt;p className="text-xs font-medium text-muted-foreground uppercase tracking-wider"&gt;Payments&lt;/p&gt;
          &lt;div className="space-y-2"&gt;
            {f.payments.map((p: any, i: number) =&gt; {
              const agreed = Number(p.agreed_amount || 0);
              const paid = Number(p.paid_amount || 0);
              const status = paid &lt;= 0 ? "pending" : paid &lt; agreed ? "partial" : "paid";
              const statusConfig: Record&lt;string, { label: string; className: string }&gt; = {
                pending: { label: "Pending", className: "bg-destructive/15 text-destructive border-destructive/30" },
                partial: { label: "Partial", className: "bg-chart-4/15 text-chart-4 border-chart-4/30" },
                paid: { label: "Paid", className: "bg-chart-2/15 text-chart-2 border-chart-2/30" },
              };
              const config = statusConfig[status];
              const projectLabel = p.projects?.display_address || p.projects?.name || "—";

              return (
                &lt;div key={i} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
                  &lt;div className="text-sm"&gt;
                    &lt;span className="font-medium text-foreground"&gt;{projectLabel}&lt;/span&gt;
                    &lt;span className="text-muted-foreground ml-2"&gt;
                      ${agreed.toLocaleString(undefined, { minimumFractionDigits: 2 })}
                    &lt;/span&gt;
                  &lt;/div&gt;
                  &lt;div className="flex items-center gap-3"&gt;
                    {p.due_date &amp;&amp; (
                      &lt;span className="text-xs text-muted-foreground"&gt;
                        {format(new Date(p.due_date), "MMM dd")}
                      &lt;/span&gt;
                    )}
                    &lt;Badge className={`${config.className} border rounded-full px-3 py-1 text-xs font-semibold`}&gt;
                      {config.label}
                    &lt;/Badge&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              );
            })}
          &lt;/div&gt;
        &lt;/div&gt;
      )}

      {f.assignments.length === 0 &amp;&amp; f.payments.length === 0 &amp;&amp; (
        &lt;p className="text-sm text-muted-foreground"&gt;No assignments or payments yet&lt;/p&gt;
      )}
    &lt;/CardContent&gt;
  &lt;/Card&gt;
))}
&lt;/div&gt;
)}
&lt;/div&gt;
);
};

export default AdminFreelancers;
```

src/pages/admin/AdminGantt.tsx

```
import { useEffect, useState, useMemo } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { useLanguage } from "@/hooks/useLanguage";
import { ChevronLeft, ChevronRight, GanttChart, CalendarDays } from "lucide-react";
import { format, addDays, startOfWeek, startOfMonth, endOfMonth, differenceInDays, isSameDay, isSameMonth, getDay } from "date-fns";
import { PROJECT_STAGE_COLORS as STAGE_COLOR_MAP, getStageLabel } from "@/constants/projectStages";

const PROJECT_STAGE_COLORS: Record&lt;string, string&gt; = Object.fromEntries(
  Object.entries(STAGE_COLOR_MAP).map(([k, v]) =&gt; [k, v.dot])
);

const AdminGantt = () =&gt; {
  const { t } = useLanguage();
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [currentWeekStart, setCurrentWeekStart] = useState(() =&gt; startOfWeek(new Date(), { weekStartsOn: 1 }));
  const [viewMode, setViewMode] = useState&lt;"week" | "month"&gt;("week");
  const [calendarMonth, setCalendarMonth] = useState(new Date());
  const [selectedDate, setSelectedDate] = useState&lt;Date | null&gt;(null);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      const { data } = await supabase
        .from("projects")
        .select("id, stage, start_date, delivery_date, companies(name), street_name, street_number, city, display_address")
        .order("start_date", { ascending: true });
      setProjects(data || []);
    };
    fetch();
  }, []);

  // Gantt logic
  const daysCount = viewMode === "week" ? 7 : 28;
  const days = useMemo(() =&gt; Array.from({ length: daysCount }, (_, i) =&gt; addDays(currentWeekStart, i)), [currentWeekStart, daysCount]);

  const navigate = (dir: number) =&gt; {
    setCurrentWeekStart(prev =&gt; addDays(prev, dir &gt; 0 ? daysCount : -daysCount));
  };

  const rangeEnd = addDays(currentWeekStart, daysCount - 1);

  const visibleProjects = projects.filter(p =&gt; {
    if (!p.start_date &amp;&amp; !p.delivery_date) return false;
    const start = p.start_date ? new Date(p.start_date) : new Date(p.delivery_date);
    const end = p.delivery_date ? new Date(p.delivery_date) : new Date(p.start_date);
    return start &lt;= rangeEnd &amp;&amp; end &gt;= currentWeekStart;
  });

  const getBarStyle = (project: any) =&gt; {
    const start = new Date(project.start_date || project.delivery_date);
    const end = new Date(project.delivery_date || project.start_date);
    const startOffset = Math.max(0, differenceInDays(start, currentWeekStart));
    const endOffset = Math.min(daysCount - 1, differenceInDays(end, currentWeekStart));
    const width = endOffset - startOffset + 1;
    return { left: `${(startOffset / daysCount) * 100}%`, width: `${(width / daysCount) * 100}%` };
  };

  const projectName = (p: any) =&gt; {
    const addr = p.display_address || [p.street_number, p.street_name].filter(Boolean).join(" ");
    return addr || p.companies?.name || "—";
  };

  const todayOffset = differenceInDays(new Date(), currentWeekStart);
  const todayInRange = todayOffset &gt;= 0 &amp;&amp; todayOffset &lt; daysCount;

  // Calendar logic
  const monthStart = startOfMonth(calendarMonth);
  const monthEnd = endOfMonth(calendarMonth);
  const calendarStartDay = getDay(monthStart); // 0=Sun
  const totalDaysInMonth = differenceInDays(monthEnd, monthStart) + 1;
  const calendarDays = Array.from({ length: 42 }, (_, i) =&gt; {
    const day = addDays(monthStart, i - calendarStartDay);
    return day;
  });

  const getProjectsOnDate = (date: Date) =&gt; {
    return projects.filter(p =&gt; {
      if (!p.start_date &amp;&amp; !p.delivery_date) return false;
      const start = p.start_date ? new Date(p.start_date) : new Date(p.delivery_date);
      const end = p.delivery_date ? new Date(p.delivery_date) : new Date(p.start_date);
      return date &gt;= start &amp;&amp; date &lt;= end;
    });
  };

  const navigateCalendar = (dir: number) =&gt; {
    setCalendarMonth(prev =&gt; {
      const d = new Date(prev);
      d.setMonth(d.getMonth() + dir);
      return d;
    });
  };

  const selectedDateProjects = selectedDate ? getProjectsOnDate(selectedDate) : [];

  return (
```

```
&lt;div className="p-6 space-y-6"&gt;
  &lt;div&gt;
    &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.gantt")} &amp; Calendar&lt;/h1&gt;
    &lt;p className="text-sm text-muted-foreground"&gt;Visualize project timelines and overlapping work&lt;/p&gt;
  &lt;/div&gt;

  &lt;Tabs defaultValue="gantt"&gt;
    &lt;TabsList&gt;
      &lt;TabsTrigger value="gantt" className="gap-2"&gt;&lt;GanttChart className="h-4 w-4" /&gt;Gantt&lt;/TabsTrigger&gt;
      &lt;TabsTrigger value="calendar" className="gap-2"&gt;&lt;CalendarDays className="h-4 w-4" /&gt;Calendar&lt;/TabsTrigger&gt;
    &lt;/TabsList&gt;

    {/* GANTT TAB */}
    &lt;TabsContent value="gantt" className="space-y-4"&gt;
      &lt;div className="flex items-center justify-end gap-2"&gt;
        &lt;Select value={viewMode} onValueChange={v =&gt; setViewMode(v as any)}&gt;
          &lt;SelectTrigger className="w-[140px]"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
          &lt;SelectContent&gt;
            &lt;SelectItem value="week"&gt;{t("admin.weekView")}&lt;/SelectItem&gt;
            &lt;SelectItem value="month"&gt;{t("admin.monthView")}&lt;/SelectItem&gt;
          &lt;/SelectContent&gt;
        &lt;/Select&gt;
        &lt;Button variant="outline" size="icon" onClick={() =&gt; navigate(-1)}&gt;&lt;ChevronLeft className="h-4 w-4" /&gt;&lt;/Button&gt;
        &lt;Button variant="outline" size="sm" onClick={() =&gt; setCurrentWeekStart(startOfWeek(new Date(), { weekStartsOn: 1 }))}&gt;{t("admin.today")}&lt;/Button&gt;
        &lt;Button variant="outline" size="icon" onClick={() =&gt; navigate(1)}&gt;&lt;ChevronRight className="h-4 w-4" /&gt;&lt;/Button&gt;
      &lt;/div&gt;

      &lt;Card&gt;
        &lt;CardContent className="p-0 overflow-x-auto"&gt;
          &lt;div className="min-w-[800px]"&gt;
            &lt;div className="flex border-b border-border"&gt;
              &lt;div className="w-[220px] flex-shrink-0 p-3 text-sm font-medium text-muted-foreground border-r border-border"&gt;
                {t("admin.projects")}
              &lt;/div&gt;
              &lt;div class="flex-1 flex"&gt;
                {days.map((day, i) =&gt; {
                  const isToday = isSameDay(day, new Date());
                  return (
                    &lt;div key={i} className={`flex-1 text-center py-2 text-xs border-r border-border last:border-r-0 ${isToday ? "bg-primary/10 font-bold text-primary"
                      &lt;div&gt;{format(day, viewMode === "week" ? "EEE" : "dd")}&lt;/div&gt;
                      &lt;div className="font-medium"&gt;{format(day, viewMode === "week" ? "dd" : "MMM")}&lt;/div&gt;
                    &lt;/div&gt;
                  );
                })}
              &lt;/div&gt;
            &lt;/div&gt;

            {visibleProjects.length === 0 ? (
              &lt;div className="p-8 text-center text-muted-foreground"&gt;{t("admin.noProjects")}&lt;/div&gt;
            ) : (
              visibleProjects.map(p =&gt; {
                const barStyle = getBarStyle(p);
                const stageColor = PROJECT_STAGE_COLORS[p.stage] || "bg-muted-foreground";
                return (
                  &lt;div key={p.id} className="flex border-b border-border last:border-b-0 hover:bg-muted/30"&gt;
                    &lt;div className="w-[220px] flex-shrink-0 p-3 text-sm font-medium text-foreground border-r border-border truncate flex items-center gap-2"&gt;
                      &lt;span className={`w-2 h-2 rounded-full flex-shrink-0 ${stageColor}`} /&gt;
                      {projectName(p)}
                    &lt;/div&gt;
                    &lt;div className="flex-1 relative h-12"&gt;
                      {todayInRange &amp;&amp; (
                        &lt;div className="absolute top-0 bottom-0 w-0.5 bg-destructive z-10" style={{ left: `${((todayOffset + 0.5) / daysCount) * 100}%` }} /&gt;
                      )}
                      &lt;div className={`absolute top-2 h-8 rounded-md ${stageColor} opacity-80 hover:opacity-100 transition-opacity flex items-center px-2 min-w-[20px]
                        &lt;span className="text-xs font-medium text-primary-foreground truncate"&gt;{p.companies?.name || ""}&lt;/span&gt;
                      &lt;/div&gt;
                    &lt;/div&gt;
                  &lt;/div&gt;
                );
              })
            )}
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      &lt;div className="flex flex-wrap gap-3"&gt;
        {Object.entries(PROJECT_STAGE_COLORS).map(([stage, color]) =&gt; (
          &lt;div key={stage} className="flex items-center gap-1.5 text-xs text-muted-foreground"&gt;
            &lt;span className={`w-3 h-3 rounded-sm ${color}`} /&gt;
            {getStageLabel(stage)}
          &lt;/div&gt;
        ))}
      &lt;/div&gt;
    &lt;/TabsContent&gt;

    {/* CALENDAR TAB */}
    &lt;TabsContent value="calendar" className="space-y-4"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;h2 className="text-lg font-semibold text-foreground"&gt;{format(calendarMonth, "MMMM yyyy")}&lt;/h2&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Button variant="outline" size="icon" onClick={() =&gt; navigateCalendar(-1)}&gt;&lt;ChevronLeft className="h-4 w-4" /&gt;&lt;/Button&gt;
          &lt;Button variant="outline" size="sm" onClick={() =&gt; { setCalendarMonth(new Date()); setSelectedDate(new Date()); }}&gt;Today&lt;/Button&gt;
          &lt;Button variant="outline" size="icon" onClick={() =&gt; navigateCalendar(1)}&gt;&lt;ChevronRight className="h-4 w-4" /&gt;&lt;/Button&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;Card&gt;
        &lt;CardContent className="p-4"&gt;
          {/* Day headers */}
          &lt;div className="grid grid-cols-7 gap-1 mb-1"&gt;
            {["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"].map(d =&gt; (
```

```
                &lt;div key={d} className="text-center text-xs font-medium text-muted-foreground py-2"&gt;{d}&lt;/div&gt;
              ))}
            &lt;/div&gt;
            {/* Days grid */}
            &lt;div className="grid grid-cols-7 gap-1"&gt;
              {calendarDays.map((day, i) =&gt; {
                const inMonth = isSameMonth(day, calendarMonth);
                const isToday = isSameDay(day, new Date());
                const isSelected = selectedDate &amp;&amp; isSameDay(day, selectedDate);
                const dayProjects = getProjectsOnDate(day);
                const hasProjects = dayProjects.length &gt; 0;

                return (
                  &lt;button
                    key={i}
                    onClick={() =&gt; setSelectedDate(day)}
                    className={`relative p-2 h-16 rounded-md text-sm transition-colors text-left
                      ${!inMonth ? "text-muted-foreground/40" : "text-foreground"}
                      ${isToday ? "ring-2 ring-primary" : ""}
                      ${isSelected ? "bg-primary/10" : "hover:bg-muted/50"}
                    `}
                  &gt;
                    &lt;span className={`text-xs ${isToday ? "font-bold text-primary" : ""}`}&gt;{format(day, "d")}&lt;/span&gt;
                    {hasProjects &amp;&amp; inMonth &amp;&amp; (
                      &lt;div className="flex gap-0.5 mt-1 flex-wrap"&gt;
                        {dayProjects.slice(0, 3).map((p, j) =&gt; (
                          &lt;span key={j} className={`w-2 h-2 rounded-full ${PROJECT_STAGE_COLORS[p.stage] || "bg-muted-foreground"}`} /&gt;
                        ))}
                        {dayProjects.length &gt; 3 &amp;&amp; &lt;span className="text-[10px] text-muted-foreground"&gt;+{dayProjects.length - 3}&lt;/span&gt;}
                      &lt;/div&gt;
                    )}
                  &lt;/button&gt;
                );
              })}
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;

        {/* Selected date details */}
        {selectedDate &amp;&amp; (
          &lt;Card&gt;
            &lt;CardContent className="p-4"&gt;
              &lt;h3 className="font-semibold text-foreground mb-3"&gt;{format(selectedDate, "EEEE, MMMM d, yyyy")}&lt;/h3&gt;
              {selectedDateProjects.length === 0 ? (
                &lt;p className="text-sm text-muted-foreground"&gt;No active projects on this date&lt;/p&gt;
              ) : (
                &lt;div className="space-y-2"&gt;
                  {selectedDateProjects.map(p =&gt; (
                    &lt;div key={p.id} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
                      &lt;div className="flex items-center gap-2"&gt;
                        &lt;span className={`w-2.5 h-2.5 rounded-full ${PROJECT_STAGE_COLORS[p.stage] || "bg-muted-foreground"}`} /&gt;
                        &lt;span className="font-medium text-sm text-foreground"&gt;{projectName(p)}&lt;/span&gt;
                      &lt;/div&gt;
                      &lt;div className="flex items-center gap-3 text-xs text-muted-foreground"&gt;
                        &lt;span&gt;{getStageLabel(p.stage || "—")}&lt;/span&gt;
                        &lt;span&gt;{p.companies?.name}&lt;/span&gt;
                      &lt;/div&gt;
                    &lt;/div&gt;
                  ))}
                  &lt;p className="text-xs text-muted-foreground mt-2"&gt;{selectedDateProjects.length} project(s) active&lt;/p&gt;
                &lt;/div&gt;
              )}
            &lt;/CardContent&gt;
          &lt;/Card&gt;
        )}
      &lt;/TabsContent&gt;
    &lt;/Tabs&gt;
  &lt;/div&gt;
  );
};

export default AdminGantt;
```

src/pages/admin/AdminLeads.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuTrigger } from "@/components/ui/dropdown-menu";
import { Search, MoreHorizontal, UserPlus, Send, Phone, Inbox, CheckCircle } from "lucide-react";
import { formatDistanceToNow } from "date-fns";
import { toast } from "sonner";

const AdminLeads = () =&gt; {
  const [leads, setLeads] = useState&lt;any[]&gt;([]);
  const [search, setSearch] = useState("");

  const fetchLeads = async () =&gt; {
    const { data } = await supabase.from("quote_requests").select("*").order("created_at", { ascending: false });
    setLeads(data || []);
  };

  useEffect(() =&gt; { fetchLeads(); }, []);

  const handleConvert = async (lead: any) =&gt; {
    const { data: company, error: compErr } = await supabase
      .from("companies").insert({ name: lead.company_name || lead.full_name, email: lead.email, phone: lead.phone }).select().single();
    if (compErr || !company) { toast.error("Failed to create client"); return; }
    const { error: projErr } = await supabase.from("projects").insert({
      company_id: company.id, name: `${lead.full_name} - ${lead.service_type}`,
      service_type: lead.service_type, stage: "Lead", description: lead.description, address: lead.project_address,
    });
    if (!projErr) {
      await supabase.from("quote_requests").update({ processed: true }).eq("id", lead.id);
      toast.success("Lead converted to project"); fetchLeads();
    } else toast.error("Failed to create project");
  };

  const serviceLabel = (s: string) =&gt; s?.replace(/_/g, " ").replace(/\b\w/g, (c) =&gt; c.toUpperCase()) || "—";

  const filtered = leads.filter(l =&gt;
    (l.full_name || "").toLowerCase().includes(search.toLowerCase()) ||
    (l.email || "").toLowerCase().includes(search.toLowerCase())
  );

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Leads&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Incoming leads from the intake form&lt;/p&gt;
        &lt;/div&gt;
        &lt;Badge variant="secondary"&gt;{leads.filter(l =&gt; !l.processed).length} new&lt;/Badge&gt;
      &lt;/div&gt;
      &lt;div className="relative max-w-xs"&gt;
        &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
        &lt;Input placeholder="Search leads..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Name&lt;/TableHead&gt;
                &lt;TableHead&gt;Email&lt;/TableHead&gt;
                &lt;TableHead&gt;Phone&lt;/TableHead&gt;
                &lt;TableHead&gt;Service Type&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Received&lt;/TableHead&gt;
                &lt;TableHead className="w-[50px]"&gt;&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {filtered.length === 0 &amp;amp;&amp;amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={7} className="text-center text-muted-foreground py-12"&gt;&lt;Inbox className="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;N
              )}
              {filtered.map(lead =&gt; (
                &lt;TableRow key={lead.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{lead.full_name}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{lead.email}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{lead.phone || "—"}&lt;/TableCell&gt;
                  &lt;TableCell&gt;{serviceLabel(lead.service_type)}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    {lead.processed
                      ? &lt;Badge variant="secondary"&gt;&lt;CheckCircle className="h-3 w-3 mr-1" /&gt;Processed&lt;/Badge&gt;
                      : &lt;Badge className="bg-blue-500/10 text-blue-600 border-blue-500/20" variant="outline"&gt;New&lt;/Badge&gt;}
                  &lt;/TableCell&gt;
                  &lt;TableCell className="text-xs text-muted-foreground"&gt;{formatDistanceToNow(new Date(lead.created_at), { addSuffix: true })}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    &lt;DropdownMenu&gt;
                      &lt;DropdownMenuTrigger asChild&gt;&lt;Button variant="ghost" size="icon" className="h-7 w-7"&gt;&lt;MoreHorizontal className="h-4 w-4" /&gt;&lt;/Butto
                      &lt;DropdownMenuContent align="end"&gt;
                        &lt;DropdownMenuItem onClick={() =&gt; handleConvert(lead)} className="gap-2"&gt;&lt;UserPlus className="h-3.5 w-3.5" /&gt;Convert to Project&lt;/Dro
                        &lt;DropdownMenuItem onClick={() =&gt; window.open(`mailto:${lead.email}?subject=Proposal`, "_blank")} className="gap-2"&gt;&lt;Send className="h-3.5
                        &lt;DropdownMenuItem onClick={() =&gt; lead.phone &amp;amp;&amp;amp; window.open(`tel:${lead.phone}`)} className="gap-2"&gt;&lt;Phone className="h-3.5 w-3.5"
                      &lt;/DropdownMenuContent&gt;
                    &lt;/DropdownMenu&gt;
                  &lt;/TableCell&gt;
                &lt;/TableRow&gt;
```

```
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminLeads;
```

src/pages/admin/AdminLogin.tsx

```
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { z } from "zod";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { toast } from "sonner";
import { useAuth } from "@/hooks/useAuth";
import { useAdminRole } from "@/hooks/useAdminRole";
import { lovable } from "@/integrations/lovable/index";
import daraLogo from "@/assets/dara-logo.jpg";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Card, CardContent } from "@/components/ui/card";
import { Separator } from "@/components/ui/separator";
import { Loader2, Mail, Lock } from "lucide-react";

const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(6),
});

type LoginFormData = z.infer&lt;typeof loginSchema&gt;;

const AdminLogin = () =&gt; {
  const [isLoading, setIsLoading] = useState(false);
  const { user, signIn } = useAuth();
  const { isAdmin, loading: roleLoading } = useAdminRole();
  const navigate = useNavigate();

  useEffect(() =&gt; {
    if (user &amp;&amp; !roleLoading &amp;&amp; isAdmin) {
      navigate("/adm");
    }
  }, [user, isAdmin, roleLoading, navigate]);

  const form = useForm&lt;LoginFormData&gt;({
    resolver: zodResolver(loginSchema),
    defaultValues: { email: "", password: "" },
  });

  const handleLogin = async (data: LoginFormData) =&gt; {
    setIsLoading(true);
    try {
      const { error } = await signIn(data.email, data.password);
      if (error) {
        toast.error("Credenciais inválidas.");
      }
    } catch {
      toast.error("Erro inesperado.");
    } finally {
      setIsLoading(false);
    }
  };

  const handleGoogleLogin = async () =&gt; {
    setIsLoading(true);
    const { error } = await lovable.auth.signInWithOAuth("google", {
      redirect_uri: window.location.origin,
    });
    if (error) { toast.error("Erro ao entrar com Google"); setIsLoading(false); }
  };

  return (
    &lt;div className="min-h-screen flex items-center justify-center bg-muted/30"&gt;
      &lt;div className="w-full max-w-sm mx-auto px-4"&gt;
        {/* Logo &amp; Brand */}
        &lt;div className="text-center mb-8"&gt;
          &lt;img src={daraLogo} alt="DARA Studio" className="h-14 w-14 rounded-lg mx-auto mb-4" /&gt;
          &lt;h1 className="font-serif text-lg font-semibold text-primary"&gt;
            &lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt; | Drafting &amp; 3D Support
          &lt;/h1&gt;
          &lt;p className="text-xs text-muted-foreground mt-1"&gt;Painel Administrativo&lt;/p&gt;
        &lt;/div&gt;

        &lt;Card&gt;
          &lt;CardContent className="p-6 space-y-4"&gt;
            &lt;Button variant="outline" className="w-full" onClick={handleGoogleLogin} disabled={isLoading}&gt;
              &lt;svg className="w-5 h-5 mr-2" viewBox="0 0 24 24"&gt;&lt;path d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92a5.06 5.06 0 01-2.2 3.32v2.77h3.57c2.08-1.9
              Entrar com Google
            &lt;/Button&gt;

            &lt;div className="relative"&gt;
              &lt;div className="absolute inset-0 flex items-center"&gt;&lt;Separator /&gt;&lt;/div&gt;
              &lt;div className="relative flex justify-center text-xs uppercase"&gt;&lt;span className="bg-card px-2 text-muted-foreground"&gt;ou&lt;/span&gt;&lt;/div&gt;
            &lt;/div&gt;

            &lt;form onSubmit={form.handleSubmit(handleLogin)} className="space-y-4"&gt;
              &lt;div class="space-y-2"&gt;
                &lt;Label class="text-xs"&gt;Email&lt;/Label&gt;
                &lt;div class="relative"&gt;
                  &lt;Mail class="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
                  &lt;Input type="email" class="pl-10" placeholder="admin@darastudio.com" {...form.register("email")} /&gt;
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;div class="space-y-2"&gt;
                &lt;Label class="text-xs"&gt;Senha&lt;/Label&gt;
                &lt;div class="relative"&gt;
```

```
                &lt;Lock className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
                &lt;Input type="password" className="pl-10" placeholder="••••••••" {...form.register("password")} /&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;Button type="submit" className="w-full" disabled={isLoading}&gt;
              {isLoading ? &lt;&gt;&lt;Loader2 className="mr-2 h-4 w-4 animate-spin" /&gt;Entrando...&lt;/&gt; : "Entrar"}
            &lt;/Button&gt;
          &lt;/form&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      &lt;p className="text-center text-xs text-muted-foreground mt-6"&gt;
        Acesso restrito a administradores
      &lt;/p&gt;
    &lt;/div&gt;
  &lt;/div&gt;
);
};

export default AdminLogin;
```

src/pages/admin/AdminMap.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Badge } from "@/components/ui/badge";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { MapPin } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const STAGE_COLORS: Record&lt;string, string&gt; = {
  "Lead": "#6b7280",
  "Budget Sent": "#3b82f6",
  "Budget Approved": "#10b981",
  "Waiting Payment": "#f59e0b",
  "In Progress": "#8b5cf6",
  "Preview Sent": "#06b6d4",
  "Revision": "#f97316",
  "Final Review": "#14b8a6",
  "Completed": "#22c55e",
  "On Hold": "#9ca3af",
  "Cancelled": "#dc2626",
  "Delayed": "#ef4444",
};

let MapContainer: any, TileLayer: any, Marker: any, Popup: any, L: any;

const AdminMap = () =&gt; {
  const { t } = useLanguage();
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [mapReady, setMapReady] = useState(false);
  const [selectedCity, setSelectedCity] = useState&lt;string | null&gt;(null);

  useEffect(() =&gt; {
    Promise.all([
      import("react-leaflet"),
      import("leaflet"),
      import("leaflet/dist/leaflet.css"),
    ]).then(([rl, leaflet]) =&gt; {
      MapContainer = rl.MapContainer;
      TileLayer = rl.TileLayer;
      Marker = rl.Marker;
      Popup = rl.Popup;
      L = leaflet.default;
      setMapReady(true);
    });
  }, []);

  useEffect(() =&gt; {
    const fetchProjects = async () =&gt; {
      const { data } = await supabase
        .from("projects")
        .select("*, companies(name)")
        .not("latitude", "is", null)
        .not("longitude", "is", null);
      setProjects(data || []);
    };
    fetchProjects();
  }, []);

  const createIcon = (color: string) =&gt;
    L.divIcon({
      className: "",
      html: `&lt;div style="width:24px;height:24px;border-radius:50%;background:${color};border:3px solid white;box-shadow:0 2px 6px rgba(0,0,0,0.3);"&gt;&lt;/div&gt;`,
      iconSize: [24, 24],
      iconAnchor: [12, 12],
    });

  // Massachusetts center
  const maCenter: [number, number] = [42.4072, -71.3824];

  // Get unique cities
  const cities = [...new Set(projects.map(p =&gt; p.city).filter(Boolean))].sort();
  const displayedProjects = selectedCity ? projects.filter(p =&gt; p.city === selectedCity) : projects;

  if (!mapReady) {
    return (
      &lt;div className="p-6 space-y-4"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.mapView")}&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;{t("admin.loadingMap")}&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    );
  }

  if (projects.length === 0) {
    return (
      &lt;div className="p-6 space-y-4"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.mapView")}&lt;/h1&gt;
        &lt;/div&gt;
        &lt;Card&gt;
          &lt;CardContent className="py-12 text-center"&gt;
            &lt;MapPin className="h-12 w-12 mx-auto text-muted-foreground mb-4" /&gt;
            &lt;p className="text-muted-foreground"&gt;{t("admin.noProjectsWithLocation")}&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;
    );
```

```
  }

  return (
    &lt;div className="p-6 space-y-4"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-bold text-foreground"&gt;{t("admin.mapView")}&lt;/h1&gt;
        &lt;p className="text-muted-foreground"&gt;{displayedProjects.length} {t("admin.projectsWithLocation")}&lt;/p&gt;
      &lt;/div&gt;

      {/* City filter buttons */}
      &lt;div className="flex flex-wrap gap-2"&gt;
        &lt;Button
          variant={selectedCity === null ? "default" : "outline"}
          size="sm"
          onClick={() =&gt; setSelectedCity(null)}
        &gt;
          {t("admin.allCities")} ({projects.length})
        &lt;/Button&gt;
        {cities.map(city =&gt; {
          const count = projects.filter(p =&gt; p.city === city).length;
          return (
            &lt;Button
              key={city}
              variant={selectedCity === city ? "default" : "outline"}
              size="sm"
              onClick={() =&gt; setSelectedCity(city)}
            &gt;
              {city} ({count})
            &lt;/Button&gt;
          );
        })}
      &lt;/div&gt;

      &lt;div className="flex flex-wrap gap-2 mb-2"&gt;
        {Object.entries(STAGE_COLORS).map(([stage, color]) =&gt; (
          &lt;div key={stage} className="flex items-center gap-1.5 text-xs text-muted-foreground"&gt;
            &lt;div className="w-3 h-3 rounded-full" style={{ background: color }} /&gt;
            {stage}
          &lt;/div&gt;
        ))}
      &lt;/div&gt;

      &lt;div className="rounded-lg border overflow-hidden" style={{ height: "calc(100vh - 300px)" }}&gt;
        &lt;MapContainer center={maCenter} zoom={9} style={{ height: "100%", width: "100%" }}&gt;
          &lt;TileLayer
            attribution='&amp;copy; &lt;a href="https://www.openstreetmap.org/copyright"&gt;OpenStreetMap&lt;/a&gt;'
            url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
          /&gt;
          {displayedProjects.map((p) =&gt; (
            &lt;Marker
              key={p.id}
              position={[p.latitude, p.longitude]}
              icon={createIcon(STAGE_COLORS[p.stage] || "#6b7280")}
            &gt;
              &lt;Popup&gt;
                &lt;div className="space-y-1"&gt;
                  &lt;p className="font-semibold"&gt;{[p.street_number, p.street_name].filter(Boolean).join(" ") || t("admin.noAddress")}&lt;/p&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;{p.companies?.name}&lt;/p&gt;
                  {p.city &amp;&amp; &lt;p className="text-xs"&gt;{p.city}, {p.state}&lt;/p&gt;}
                  &lt;Badge variant="outline"&gt;{p.stage || "—"}&lt;/Badge&gt;
                  {p.total_value &gt; 0 &amp;&amp; (
                    &lt;p className="text-xs"&gt;{p.currency === "BRL" ? "R$" : "$"}{Number(p.total_value).toLocaleString()}&lt;/p&gt;
                  )}
                &lt;/div&gt;
              &lt;/Popup&gt;
            &lt;/Marker&gt;
          ))}
        &lt;/MapContainer&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminMap;
```

src/pages/admin/AdminMilestones.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Plus } from "lucide-react";
import { toast } from "sonner";

const MILESTONE_NAMES = ["Preview 01", "Preview 02", "Revision 01", "Revision 02", "Final Delivery"];
const STATUSES = ["open", "sent", "approved"];

const AdminMilestones = () =&gt; {
  const [milestones, setMilestones] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [form, setForm] = useState({ project_id: "", name: "Preview 01", status: "open", sent_date: "", approved_date: "", notes: "" });

  const fetchData = async () =&gt; {
    const [msRes, projRes] = await Promise.all([
      supabase.from("milestones").select("*, projects(name)").order("created_at", { ascending: false }),
      supabase.from("projects").select("id, name").order("name"),
    ]);
    setMilestones(msRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  const handleSave = async () =&gt; {
    if (!form.project_id) { toast.error("Selecione um projeto"); return; }
    const { error } = await supabase.from("milestones").insert({
      project_id: form.project_id, name: form.name, status: form.status,
      sent_date: form.sent_date || null, approved_date: form.approved_date || null,
      notes: form.notes || null,
    });
    if (error) { toast.error(error.message); return; }
    toast.success("Milestone criado");
    setDialogOpen(false); fetchData();
  };

  const updateStatus = async (id: string, status: string) =&gt; {
    const update: any = { status };
    if (status === "sent") update.sent_date = new Date().toISOString().split("T")[0];
    if (status === "approved") update.approved_date = new Date().toISOString().split("T")[0];
    await supabase.from("milestones").update(update).eq("id", id);
    fetchData();
  };

  const statusBadge = (s: string) =&gt; {
    if (s === "approved") return &lt;Badge className="bg-chart-2/10 text-chart-2 border-chart-2/20"&gt;Aprovado&lt;/Badge&gt;;
    if (s === "sent") return &lt;Badge variant="secondary"&gt;Enviado&lt;/Badge&gt;;
    return &lt;Badge variant="outline"&gt;Aberto&lt;/Badge&gt;;
  };

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Milestones&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Acompanhamento de entregas por projeto&lt;/p&gt;
        &lt;/div&gt;
        &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
          &lt;DialogTrigger asChild&gt;&lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;Novo Milestone&lt;/Button&gt;&lt;/DialogTrigger&gt;
          &lt;DialogContent&gt;
            &lt;DialogHeader&gt;&lt;DialogTitle&gt;Novo Milestone&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
            &lt;div className="space-y-4"&gt;
              &lt;div&gt;&lt;Label&gt;Projeto *&lt;/Label&gt;
                &lt;Select value={form.project_id} onValueChange={v =&gt; setForm({ ...form, project_id: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Selecione" /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{projects.map(p =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{p.name}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Milestone&lt;/Label&gt;
                &lt;Select value={form.name} onValueChange={v =&gt; setForm({ ...form, name: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{MILESTONE_NAMES.map(n =&gt; &lt;SelectItem key={n} value={n}&gt;{n}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Notas&lt;/Label&gt;&lt;Textarea value={form.notes} onChange={e =&gt; setForm({ ...form, notes: e.target.value })} /&gt;&lt;/div&gt;
              &lt;Button className="w-full" onClick={handleSave}&gt;Criar&lt;/Button&gt;
            &lt;/div&gt;
          &lt;/DialogContent&gt;
        &lt;/Dialog&gt;
      &lt;/div&gt;

      &lt;Card&gt;
        &lt;CardContent className="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Projeto&lt;/TableHead&gt;
                &lt;TableHead&gt;Milestone&lt;/TableHead&gt;
```

```
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Enviado&lt;/TableHead&gt;
                &lt;TableHead&gt;Aprovado&lt;/TableHead&gt;
                &lt;TableHead&gt;Ações&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {milestones.map(m =&gt; (
                &lt;TableRow key={m.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{m.projects?.name}&lt;/TableCell&gt;
                  &lt;TableCell&gt;{m.name}&lt;/TableCell&gt;
                  &lt;TableCell&gt;{statusBadge(m.status)}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{m.sent_date || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{m.approved_date || "—"}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    &lt;div className="flex gap-1"&gt;
                      {m.status === "open" &amp;&amp; &lt;Button variant="outline" size="sm" onClick={() =&gt; updateStatus(m.id, "sent")}&gt;Enviar&lt;/Button&gt;}
                      {m.status === "sent" &amp;&amp; &lt;Button variant="outline" size="sm" onClick={() =&gt; updateStatus(m.id, "approved")}&gt;Aprovar&lt;/Button&gt;}
                    &lt;/div&gt;
                  &lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
              {milestones.length === 0 &amp;&amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={6} className="text-center text-muted-foreground py-8"&gt;Nenhum milestone&lt;/TableCell&gt;&lt;/TableRow&gt;
              )}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminMilestones;
```

src/pages/admin/AdminPayments.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Plus } from "lucide-react";
import { toast } from "sonner";
import { Separator } from "@/components/ui/separator";
import { useLanguage } from "@/hooks/useLanguage";

const INSTALLMENT_TYPES = ["Entry", "Final", "Extra"];
const PAYMENT_METHODS = ["Stripe", "Nomad", "Wise", "Transfer"];
const PRICE_TABLES = ["W/ Stripe (1.515)", "W/O Stripe (1.31)", "Full", "REL", "Partner"];
const STATUSES = ["pending", "partial", "paid"];

const AdminPayments = () =&gt; {
  const { t } = useLanguage();
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [form, setForm] = useState({
    project_id: "", installment_type: "Entry", amount_total: "", amount_paid: "",
    status: "pending", payment_method: "Stripe", stripe_fee_percent: "2.9", payment_date: "",
    value_brl: "", price_table: "", third_party_costs: "", payment_notes: "",
  });

  const fetchData = async () =&gt; {
    const [payRes, projRes] = await Promise.all([
      supabase.from("admin_payments").select("*, projects(name, display_address, total_value, entry_payment_percent, currency, street_name, street_number, companies(name))")
      supabase.from("projects").select("id, name, display_address, total_value, entry_payment_percent, currency, street_name, street_number, companies(name)").order("created
    ]);
    setPayments(payRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  // Realtime subscription for payment changes (admin_payments + payment_milestones)
  useEffect(() =&gt; {
    const channel = supabase
      .channel('admin-payments-changes')
      .on('postgres_changes', { event: '*', schema: 'public', table: 'admin_payments' }, () =&gt; {
        fetchData();
      })
      .on('postgres_changes', { event: '*', schema: 'public', table: 'payment_milestones' }, () =&gt; {
        fetchData();
      })
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, []);

  const selectedProject = projects.find(p =&gt; p.id === form.project_id);
  const projectTotal = Number(selectedProject?.total_value || 0);
  const entryPct = Number(selectedProject?.entry_payment_percent || 50);
  const entryAmount = projectTotal * entryPct / 100;
  const finalAmount = projectTotal - entryAmount;

  const handleProjectSelect = (projectId: string) =&gt; {
    const proj = projects.find(p =&gt; p.id === projectId);
    const total = Number(proj?.total_value || 0);
    const ePct = Number(proj?.entry_payment_percent || 50);
    const suggestedAmount = form.installment_type === "Entry" ? (total * ePct / 100) : form.installment_type === "Final" ? (total - total * ePct / 100) : 0;
    setForm(f =&gt; ({ ...f, project_id: projectId, amount_total: suggestedAmount.toString() }));
  };

  const handleTypeChange = (type: string) =&gt; {
    const suggestedAmount = type === "Entry" ? entryAmount : type === "Final" ? finalAmount : 0;
    setForm(f =&gt; ({ ...f, installment_type: type, amount_total: suggestedAmount.toString() }));
  };

  const handleSave = async () =&gt; {
    if (!form.project_id) { toast.error(t("admin.selectProject")); return; }
    const { error } = await supabase.from("admin_payments").insert({
      project_id: form.project_id, installment_type: form.installment_type,
      amount_total: Number(form.amount_total) || 0, amount_paid: Number(form.amount_paid) || 0,
      status: form.status, payment_method: form.payment_method,
      stripe_fee_percent: Number(form.stripe_fee_percent) || 0,
      payment_date: form.payment_date || null,
      value_brl: form.value_brl ? Number(form.value_brl) : null,
      price_table: form.price_table || null,
      third_party_costs: form.third_party_costs ? Number(form.third_party_costs) : 0,
      payment_notes: form.payment_notes || null,
    });
    if (error) { toast.error(error.message); return; }
    await updateProjectPaymentStage(form.project_id);
    toast.success(t("admin.paymentRegistered"));
    setDialogOpen(false); fetchData();
  };

  const updateProjectPaymentStage = async (projectId: string) =&gt; {
    const proj = projects.find(p =&gt; p.id === projectId);
    if (!proj) return;
    const { data: allPayments } = await supabase.from("admin_payments").select("amount_paid, status").eq("project_id", projectId);
```

```
  const totalPaid = (allPayments || []).filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const total = Number(proj.total_value || 0);
  const ePct = Number(proj.entry_payment_percent || 50);
  const entry = total * ePct / 100;

  let paymentStage = "Not Sent";
  if (totalPaid &gt;= total &amp;&amp; total &gt; 0) paymentStage = "Paid";
  else if (totalPaid &gt;= entry &amp;&amp; entry &gt; 0) paymentStage = "Entry Paid";
  else if (totalPaid &gt; 0) paymentStage = "Waiting Final";

  await supabase.from("projects").update({ payment_stage: paymentStage }).eq("id", projectId);
};

// Compute per-project financial summaries from projects data
const projectFinancials = projects.map(proj =&gt; {
  const total = Number(proj.total_value || 0);
  const ePct = Number(proj.entry_payment_percent || 50);
  const entry = total * ePct / 100;
  const final_ = total - entry;
  const projPayments = payments.filter(p =&gt; p.project_id === proj.id);
  const totalPaid = projPayments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const balance = total - totalPaid;
  const sym = proj.currency === "BRL" ? "R$" : "$";
  const addr = [proj.street_number, proj.street_name].filter(Boolean).join(" ") || proj.display_address || proj.name || "—";
  const company = proj.companies?.name || "";
  return { ...proj, total, entry, final_, totalPaid, balance, sym, addr, company };
}).filter(p =&gt; p.total &gt; 0);

const totalReceived = payments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
const totalPending = projectFinancials.reduce((s, p) =&gt; s + p.balance, 0);

const statusBadge = (s: string) =&gt; {
  if (s === "paid") return &lt;Badge className="bg-chart-2/15 text-chart-2 border-chart-2/30"&gt;{t("admin.paidStatus")}&lt;/Badge&gt;;
  if (s === "partial") return &lt;Badge variant="secondary"&gt;{t("admin.partialStatus")}&lt;/Badge&gt;;
  return &lt;Badge variant="outline"&gt;{t("admin.pendingStatus")}&lt;/Badge&gt;;
};

const getProjectLabel = (p: any) =&gt; {
  const addr = [p.street_number, p.street_name].filter(Boolean).join(" ");
  const company = p.companies?.name || "";
  return addr ? `${addr} — ${company}` : p.display_address || p.name || company || "—";
};

return (
  &lt;div className="p-6 space-y-6"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.payments")}&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{t("admin.financialControl")}&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
        &lt;DialogTrigger asChild&gt;&lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;{t("admin.newPayment")}&lt;/Button&gt;&lt;/DialogTrigger&gt;
        &lt;DialogContent className="max-w-lg"&gt;
          &lt;DialogHeader&gt;&lt;DialogTitle&gt;{t("admin.registerPayment")}&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            &lt;div&gt;&lt;Label&gt;{t("admin.project")} *&lt;/Label&gt;
              &lt;Select value={form.project_id || "none"} onValueChange={v =&gt; v !== "none" &amp;&amp; handleProjectSelect(v)}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue placeholder={t("admin.select")} /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;{projects.map(p =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{getProjectLabel(p)}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
              &lt;/Select&gt;
            &lt;/div&gt;

            {selectedProject &amp;&amp; (
              &lt;div className="grid grid-cols-3 gap-2 text-sm"&gt;
                &lt;div className="p-2 rounded bg-primary/10 border border-primary/20"&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;{t("admin.totalValue")}&lt;/p&gt;
                  &lt;p class="font-semibold text-primary"&gt;${projectTotal.toLocaleString()}&lt;/p&gt;
                &lt;/div&gt;
                &lt;div className="p-2 rounded bg-chart-3/10 border border-chart-3/20"&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;{t("admin.entryAmount")} ({entryPct}%)&lt;/p&gt;
                  &lt;p class="font-semibold text-chart-3"&gt;${entryAmount.toLocaleString()}&lt;/p&gt;
                &lt;/div&gt;
                &lt;div class="p-2 rounded bg-chart-4/10 border border-chart-4/20"&gt;
                  &lt;p class="text-xs text-muted-foreground"&gt;{t("admin.finalAmount")}&lt;/p&gt;
                  &lt;p class="font-semibold text-chart-4"&gt;${finalAmount.toLocaleString()}&lt;/p&gt;
                &lt;/div&gt;
              &lt;/div&gt;
            )}

            &lt;div class="grid grid-cols-2 gap-4"&gt;
              &lt;div&gt;&lt;Label&gt;{t("admin.type")}&lt;/Label&gt;
                &lt;Select value={form.installment_type} onValueChange={handleTypeChange}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{INSTALLMENT_TYPES.map(t =&gt; &lt;SelectItem key={t} value={t}&gt;{t}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;{t("admin.method")}&lt;/Label&gt;
                &lt;Select value={form.payment_method} onValueChange={v =&gt; setForm({ ...form, payment_method: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{PAYMENT_METHODS.map(m =&gt; &lt;SelectItem key={m} value={m}&gt;{m}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div class="grid grid-cols-2 gap-4"&gt;
              &lt;div&gt;&lt;Label&gt;{t("admin.amountTotal")} ($)&lt;/Label&gt;&lt;Input type="number" value={form.amount_total} onChange={e =&gt; setForm({ ...form, amou
              &lt;div&gt;&lt;Label&gt;{t("admin.amountPaid")} ($)&lt;/Label&gt;&lt;Input type="number" value={form.amount_paid} onChange={e =&gt; setForm({ ...form, amount
            &lt;/div&gt;
            &lt;div class="grid grid-cols-2 gap-4"&gt;
              &lt;div&gt;&lt;Label&gt;{t("admin.stripeFee")} (%)&lt;/Label&gt;&lt;Input type="number" step="0.1" value={form.stripe_fee_percent} onChange={e =&gt; setForm(
              &lt;div&gt;&lt;Label&gt;{t("admin.status")}&lt;/Label&gt;
                &lt;Select value={form.status} onValueChange={v =&gt; setForm({ ...form, status: v })}&gt;
```

```
              &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;{STATUSES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{s === "pending" ? t("admin.pendingStatus") : s === "partial" ? t("admin.p
            &lt;/Select&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;{t("admin.paymentDate")}&lt;/Label&gt;&lt;Input type="date" value={form.payment_date} onChange={e =&gt; setForm({ ...form, payment_date

        &lt;Separator /&gt;
        &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Detalhes Financeiros&lt;/p&gt;
        &lt;div className="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;Valor BRL (R$)&lt;/Label&gt;&lt;Input type="number" value={form.value_brl} onChange={e =&gt; setForm({ ...form, value_brl: e.target.v
          &lt;div&gt;&lt;Label&gt;Tabela de Preço&lt;/Label&gt;
            &lt;Select value={form.price_table || "none"} onValueChange={v =&gt; setForm({ ...form, price_table: v === "none" ? "" : v })}&gt;
              &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Selecionar" /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;
                &lt;SelectItem value="none"&gt;Nenhuma&lt;/SelectItem&gt;
                {PRICE_TABLES.map(pt =&gt; &lt;SelectItem key={pt} value={pt}&gt;{pt}&lt;/SelectItem&gt;)}
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;Custos com Terceiros (Emily)&lt;/Label&gt;&lt;Input type="number" value={form.third_party_costs} onChange={e =&gt; setForm({ ...form, t
        &lt;div&gt;&lt;Label&gt;Observações de Pagamento&lt;/Label&gt;
          &lt;textarea className="w-full min-h-[80px] rounded-md border border-input bg-background px-3 py-2 text-sm" value={form.payment_notes} onChange={e =&gt; setF
        &lt;/div&gt;
        &lt;Button className="w-full" onClick={handleSave}&gt;{t("admin.register")}&lt;/Button&gt;
      &lt;/div&gt;
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;

&lt;div className="grid grid-cols-1 sm:grid-cols-3 gap-4"&gt;
  &lt;Card className="bg-chart-2/10 border-chart-2/20"&gt;&lt;CardHeader className="pb-2"&gt;&lt;CardTitle className="text-sm text-muted-foreground"&gt;{t("admin.total
    &lt;CardContent&gt;&lt;p className="text-2xl font-bold text-chart-2"&gt;${totalReceived.toLocaleString()}&lt;/p&gt;&lt;/CardContent&gt;&lt;/Card&gt;
  &lt;Card className="bg-destructive/10 border-destructive/20"&gt;&lt;CardHeader className="pb-2"&gt;&lt;CardTitle className="text-sm text-muted-foreground"&gt;{t("adm
    &lt;CardContent&gt;&lt;p className="text-2xl font-bold text-destructive"&gt;${totalPending.toLocaleString()}&lt;/p&gt;&lt;/CardContent&gt;&lt;/Card&gt;
  &lt;Card&gt;&lt;CardHeader className="pb-2"&gt;&lt;CardTitle className="text-sm text-muted-foreground"&gt;{t("admin.records")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
    &lt;CardContent&gt;&lt;p className="text-2xl font-bold text-foreground"&gt;{payments.length}&lt;/p&gt;&lt;/CardContent&gt;&lt;/Card&gt;
&lt;/div&gt;

{/* Project Financial Summary */}
&lt;Card&gt;
  &lt;CardHeader&gt;&lt;CardTitle className="text-sm"&gt;{t("admin.projectFinancials")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
  &lt;CardContent className="p-0"&gt;
    &lt;Table&gt;
      &lt;TableHeader&gt;
        &lt;TableRow&gt;
          &lt;TableHead&gt;{t("admin.project")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.totalValue")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.entryAmount")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.paid")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.balance")}&lt;/TableHead&gt;
        &lt;/TableRow&gt;
      &lt;/TableHeader&gt;
      &lt;TableBody&gt;
        {projectFinancials.map(p =&gt; (
          &lt;TableRow key={p.id}&gt;
            &lt;TableCell className="font-medium"&gt;
              &lt;div&gt;{p.addr}&lt;/div&gt;
              {p.company &amp;&amp; &lt;div className="text-xs text-muted-foreground"&gt;{p.company}&lt;/div&gt;}
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;span className="inline-flex items-center px-2 py-0.5 rounded text-xs font-medium bg-primary/10 text-primary border border-primary/20"&gt;
                {p.sym}{p.total.toLocaleString()}
              &lt;/span&gt;
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;span className="inline-flex items-center px-2 py-0.5 rounded text-xs font-medium bg-chart-3/10 text-chart-3 border border-chart-3/20"&gt;
                {p.sym}{p.entry.toLocaleString()} (50%)
              &lt;/span&gt;
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;span className="inline-flex items-center px-2 py-0.5 rounded text-xs font-medium bg-chart-2/15 text-chart-2 border border-chart-2/30"&gt;
                {p.sym}{p.totalPaid.toLocaleString()}
              &lt;/span&gt;
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;span className={`inline-flex items-center px-2 py-0.5 rounded text-xs font-medium border ${p.balance &gt; 0 ? "bg-destructive/15 text-destructive bor
                {p.sym}{p.balance.toLocaleString()}
              &lt;/span&gt;
            &lt;/TableCell&gt;
          &lt;/TableRow&gt;
        ))}
        {projectFinancials.length === 0 &amp;&amp; (
          &lt;TableRow&gt;&lt;TableCell colSpan={5} className="text-center text-muted-foreground py-8"&gt;{t("admin.noProjects")}&lt;/TableCell&gt;&lt;/TableRow&gt;
        )}
      &lt;/TableBody&gt;
    &lt;/Table&gt;
  &lt;/CardContent&gt;
&lt;/Card&gt;

{/* Payment Records */}
&lt;Card&gt;
  &lt;CardHeader&gt;&lt;CardTitle className="text-sm"&gt;{t("admin.paymentRecords")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
  &lt;CardContent className="p-0"&gt;
    &lt;Table&gt;
      &lt;TableHeader&gt;
        &lt;TableRow&gt;
          &lt;TableHead&gt;{t("admin.project")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.type")}&lt;/TableHead&gt;
```

```
                &lt;TableHead&gt;{t("admin.total")}&lt;/TableHead&gt;
                &lt;TableHead&gt;BRL&lt;/TableHead&gt;
                &lt;TableHead&gt;{t("admin.paid")}&lt;/TableHead&gt;
                &lt;TableHead&gt;Tabela&lt;/TableHead&gt;
                &lt;TableHead&gt;Terceiros&lt;/TableHead&gt;
                &lt;TableHead&gt;{t("admin.status")}&lt;/TableHead&gt;
                &lt;TableHead&gt;{t("admin.date")}&lt;/TableHead&gt;
                &lt;TableHead&gt;Notas&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {payments.map(p =&gt; (
                &lt;TableRow key={p.id}&gt;
                  &lt;TableCell className="font-medium"&gt;
                    {[p.projects?.street_number, p.projects?.street_name].filter(Boolean).join(" ") || p.projects?.display_address || p.projects?.name || "—"}
                  &lt;/TableCell&gt;
                  &lt;TableCell&gt;{p.installment_type}&lt;/TableCell&gt;
                  &lt;TableCell&gt;${Number(p.amount_total).toLocaleString()}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{p.value_brl ? `R$${Number(p.value_brl).toLocaleString()}` : "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-chart-2"&gt;${Number(p.amount_paid).toLocaleString()}&lt;/TableCell&gt;
                  &lt;TableCell className="text-xs text-muted-foreground"&gt;{p.price_table || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{p.third_party_costs ? `R$${Number(p.third_party_costs).toLocaleString()}` : "—"}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    &lt;Select
                      value={p.status}
                      onValueChange={async (newStatus) =&gt; {
                        const updatePayload: any = { status: newStatus };
                        if (newStatus === "paid" &amp;&amp; !p.payment_date) {
                          updatePayload.payment_date = new Date().toISOString().split("T")[0];
                          updatePayload.amount_paid = Number(p.amount_total);
                        }
                        if (newStatus === "pending") {
                          updatePayload.amount_paid = 0;
                        }
                        const { error } = await supabase.from("admin_payments").update(updatePayload).eq("id", p.id);
                        if (error) { toast.error(error.message); return; }
                        await updateProjectPaymentStage(p.project_id);
                        toast.success(`Payment marked as ${newStatus}`);
                        fetchData();
                      }}
                    &gt;
                      &lt;SelectTrigger className="h-7 w-[110px] text-xs"&gt;
                        &lt;SelectValue /&gt;
                      &lt;/SelectTrigger&gt;
                      &lt;SelectContent&gt;
                        &lt;SelectItem value="pending"&gt;{t("admin.pendingStatus")}&lt;/SelectItem&gt;
                        &lt;SelectItem value="paid"&gt;{t("admin.paidStatus")}&lt;/SelectItem&gt;
                        &lt;SelectItem value="cancelled"&gt;Cancelled&lt;/SelectItem&gt;
                      &lt;/SelectContent&gt;
                    &lt;/Select&gt;
                  &lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{p.payment_date || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-xs text-muted-foreground max-w-[200px] truncate"&gt;{p.payment_notes || "—"}&lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
              {payments.length === 0 &amp;&amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={10} className="text-center text-muted-foreground py-8"&gt;{t("admin.noPayments")}&lt;/TableCell&gt;&lt;/TableRow&gt;
              )}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminPayments;
```

src/pages/admin/AdminPermits.tsx

```
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { ShieldCheck } from "lucide-react";

const AdminPermits = () =&gt; {
  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Permits&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Track permit submissions and approvals&lt;/p&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="flex flex-col items-center justify-center py-16"&gt;
          &lt;ShieldCheck className="h-12 w-12 text-muted-foreground/30 mb-4" /&gt;
          &lt;h3 className="text-lg font-medium text-foreground mb-2"&gt;Permit Tracking&lt;/h3&gt;
          &lt;p className="text-sm text-muted-foreground text-center max-w-md"&gt;
            Track permit submissions, approvals, and inspection schedules for all projects. This module will be connected to project milestones.
          &lt;/p&gt;
          &lt;Badge variant="secondary" className="mt-4"&gt;Coming Soon&lt;/Badge&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminPermits;
```

src/pages/admin/AdminProjectDetail.tsx

```
import { useEffect, useState, useCallback, useRef } from "react";
import { useParams, useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Separator } from "@/components/ui/separator";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import {
  ArrowLeft, Building2, Calendar, DollarSign, FileImage, FolderOpen,
  MessageSquare, Download, Star, Clock, MapPin, Ruler,
  CreditCard, Receipt, Ticket, Globe, Languages, Mail, Phone, User,
  Plus, Save, Trash2, Lock, Users, Eye, Upload, StickyNote, Monitor,
  CheckCircle2, AlertTriangle, XCircle, ExternalLink, Hash, Camera, HardHat, Pencil,
} from "lucide-react";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { format } from "date-fns";
import { getCountryFlag, getLanguageLabel } from "@/utils/countryFlags";
import { toast } from "sonner";
import ProjectKanbanBoard from "@/components/admin/ProjectKanbanBoard";
import { Progress } from "@/components/ui/progress";
import { PROJECT_STAGES, getStageLabel } from "@/constants/projectStages";
import ProjectTimeline from "@/components/admin/ProjectTimeline";
import ProjectDocuments from "@/components/admin/ProjectDocuments";

const VISIBILITY_OPTIONS = [
  { value: "private", label: "Private", icon: Lock, color: "text-muted-foreground" },
  { value: "freelancer", label: "Team", icon: Users, color: "text-chart-4" },
  { value: "client", label: "Client Visible", icon: Eye, color: "text-chart-2" },
];

interface DateNote {
  date: string;
  note: string;
  visibility: "private" | "freelancer" | "client";
}

// Inline Site Updates Tab component
const SiteUpdatesTab = ({ projectId }: { projectId: string }) =&gt; {
  const [updates, setUpdates] = useState&lt;any[]&gt;([]);
  const [uploading, setUploading] = useState(false);
  const [files, setFiles] = useState&lt;File[]&gt;([]);
  const [description, setDescription] = useState("");

  const fetchUpdates = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("site_updates")
      .select("*")
      .eq("project_id", projectId)
      .order("created_at", { ascending: false });
    setUpdates(data || []);
  }, [projectId]);

  useEffect(() =&gt; { fetchUpdates(); }, [fetchUpdates]);

  useEffect(() =&gt; {
    const channel = supabase
      .channel(`site-updates-${projectId}`)
      .on("postgres_changes", { event: "*", schema: "public", table: "site_updates", filter: `project_id=eq.${projectId}` }, () =&gt; fetchUpdates())
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, [projectId, fetchUpdates]);

  const handleUpload = async () =&gt; {
    if (files.length === 0) { toast.error("Select at least one file"); return; }
    setUploading(true);
    try {
      const { data: { user } } = await supabase.auth.getUser();
      for (const file of files) {
        const ext = file.name.split(".").pop();
        const path = `${projectId}/${Date.now()}-${Math.random().toString(36).slice(2)}.${ext}`;
        const { error: uploadError } = await supabase.storage.from("site-updates").upload(path, file);
        if (uploadError) { toast.error(`Upload failed: ${file.name}`); continue; }
        const { data: urlData } = supabase.storage.from("site-updates").getPublicUrl(path);
        await supabase.from("site_updates").insert({
          project_id: projectId,
          uploaded_by: user?.id || null,
          media_url: urlData.publicUrl,
          description: description || null,
        });
      }
      toast.success(`${files.length} update(s) uploaded`);
      setFiles([]);
      setDescription("");
      fetchUpdates();
    } catch {
      toast.error("Upload failed");
    } finally {
      setUploading(false);
    }
  };
```

```
  return (
    &lt;div className="space-y-6"&gt;
      {/* Upload area */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;CardTitle className="text-base flex items-center gap-2"&gt;
            &lt;HardHat className="h-4 w-4" /&gt; Upload Site Update
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent className="space-y-4"&gt;
          &lt;label className="flex flex-col items-center justify-center w-full h-32 border-2 border-dashed rounded-lg cursor-pointer hover:bg-muted/50 transition-colors bor
            &lt;div className="flex flex-col items-center"&gt;
              &lt;Camera className="h-8 w-8 text-muted-foreground mb-2" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;
                {files.length &gt; 0 ? `${files.length} file(s) selected` : "Tap to take photo or select files"}
              &lt;/p&gt;
            &lt;/div&gt;
            &lt;input type="file" className="hidden" multiple accept="image/*,video/*" capture="environment" onChange={e =&gt; setFiles(Array.from(e.target.files || []))} /&
          &lt;/label&gt;
          &lt;Textarea value={description} onChange={e =&gt; setDescription(e.target.value)} placeholder="Describe what's happening on site..." /&gt;
          &lt;Button onClick={handleUpload} disabled={uploading || files.length === 0}&gt;
            {uploading ? "Uploading..." : "Upload"}
          &lt;/Button&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Timeline */}
      {updates.length === 0 ? (
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="text-center py-12"&gt;
            &lt;HardHat className="h-10 w-10 mx-auto mb-3 text-muted-foreground/40" /&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;No site updates yet&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      ) : (
        &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4"&gt;
          {updates.map(update =&gt; {
            const isVideo = update.media_url?.match(/\.(mp4|mov|webm|avi)$/i);
            return (
              &lt;Card key={update.id} className="border-none shadow-sm overflow-hidden"&gt;
                &lt;div className="aspect-video bg-muted"&gt;
                  {isVideo ? (
                    &lt;video src={update.media_url} controls className="w-full h-full object-cover" /&gt;
                  ) : (
                    &lt;img src={update.media_url} alt="Site update" className="w-full h-full object-cover" /&gt;
                  )}
                &lt;/div&gt;
                &lt;CardContent className="p-3 space-y-1"&gt;
                  &lt;span className="text-xs text-muted-foreground"&gt;
                    {format(new Date(update.created_at), "MMM d, yyyy · h:mm a")}
                  &lt;/span&gt;
                  {update.description &amp;&amp; &lt;p className="text-sm"&gt;{update.description}&lt;/p&gt;}
                &lt;/CardContent&gt;
              &lt;/Card&gt;
            );
          })}
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
};

const AdminProjectDetail = () =&gt; {
  const { id } = useParams&lt;{ id: string }&gt;();
  const navigate = useNavigate();
  const coverInputRef = useRef&lt;HTMLInputElement&gt;(null);
  const [project, setProject] = useState&lt;any&gt;(null);
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [drawings, setDrawings] = useState&lt;any[]&gt;([]);
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [messages, setMessages] = useState&lt;any[]&gt;([]);
  const [milestones, setMilestones] = useState&lt;any[]&gt;([]);
  const [tickets, setTickets] = useState&lt;any[]&gt;([]);
  const [services, setServices] = useState&lt;any[]&gt;([]);
  const [allServices, setAllServices] = useState&lt;any[]&gt;([]);
  const [selectedServiceIds, setSelectedServiceIds] = useState&lt;string[]&gt;([]);
  const [activityLog, setActivityLog] = useState&lt;any[]&gt;([]);
  const [paymentMilestones, setPaymentMilestones] = useState&lt;any[]&gt;([]);
  const [homeowners, setHomeowners] = useState&lt;any[]&gt;([]);
  const [loading, setLoading] = useState(true);
  const [hasChanges, setHasChanges] = useState(false);
  const [saving, setSaving] = useState(false);

  // Editable project fields
  const [form, setForm] = useState&lt;any&gt;({});
  const [dateNotes, setDateNotes] = useState&lt;DateNote[]&gt;([]);
  const [internalNotes, setInternalNotes] = useState("");

  const loadData = useCallback(async () =&gt; {
    if (!id) return;
    const [proj, f, d, pay, msg, ms, tk, svc, allSvc, psvc, logs, pmRes, hoRes] = await Promise.all([
      supabase.from("projects").select("*, companies(*)").eq("id", id).single(),
      supabase.from("project_files").select("*").eq("project_id", id).order("created_at", { ascending: false }),
      supabase.from("drawing_versions").select("*").eq("project_id", id).order("drawing_name").order("version_number", { ascending: false }),
      supabase.from("admin_payments").select("*").eq("project_id", id).order("created_at", { ascending: false }),
      supabase.from("messages").select("*").eq("project_id", id).order("created_at", { ascending: true }),
      supabase.from("milestones").select("*").eq("project_id", id).order("created_at"),
      supabase.from("client_tickets").select("*, companies(name)").eq("project_id", id).order("updated_at", { ascending: false }),
      supabase.from("project_services").select("service_id, services(name, category)").eq("project_id", id),
      supabase.from("services").select("*").order("category").order("name"),
      supabase.from("project_services").select("service_id").eq("project_id", id),
```

```
    supabase.from("activity_log").select("*").eq("project_id", id).order("log_date", { ascending: false }),
    supabase.from("payment_milestones").select("*").eq("project_id", id).order("created_at"),
    supabase.from("project_homeowners").select("*").eq("project_id", id).order("created_at"),
  ]);

  const p = proj.data;
  setProject(p);
  setFiles(f.data || []);
  setDrawings(d.data || []);
  setPayments(pay.data || []);
  setMessages(msg.data || []);
  setMilestones(ms.data || []);
  setTickets(tk.data || []);
  setServices((svc.data || []).map((s: any) =&gt; s.services).filter(Boolean));
  setAllServices(allSvc.data || []);
  setSelectedServiceIds((psvc.data || []).map((s: any) =&gt; s.service_id));
  setActivityLog(logs.data || []);
  setPaymentMilestones(pmRes.data || []);
  setHomeowners(hoRes.data || []);
  if (p) {
    setForm({
      stage: p.stage || "Lead",
      service_type: p.service_type || "",
      square_feet: p.square_feet?.toString() || "",
      start_date: p.start_date || "",
      delivery_date: p.delivery_date || "",
      preview_delivery_date: p.preview_delivery_date || "",
      revision_return_date: p.revision_return_date || "",
      final_delivery_date: p.final_delivery_date || "",
      total_value: p.total_value?.toString() || "",
      currency: p.currency || "USD",
      entry_payment_percent: p.entry_payment_percent?.toString() || "50",
      payment_stage: p.payment_stage || "Not Sent",
      street_number: p.street_number || "",
      street_name: p.street_name || "",
      city: p.city || "",
      state: p.state || "",
      zip_code: p.zip_code || "",
      country: p.country || "USA",
      homeowner_name: (p as any).homeowner_name || "",
      homeowner_phone: (p as any).homeowner_phone || "",
      homeowner_email: (p as any).homeowner_email || "",
      homeowner_address: (p as any).homeowner_address || "",
    });
    setDateNotes(Array.isArray(p.date_notes) ? (p.date_notes as unknown as DateNote[]) : []);
    setInternalNotes(p.internal_notes || "");
  }
  setLoading(false);
  setHasChanges(false);
}, [id]);

useEffect(() =&gt; { loadData(); }, [loadData]);

// Realtime: refresh when payments change
useEffect(() =&gt; {
  if (!id) return;
  const channel = supabase
    .channel(`project-payments-${id}`)
    .on('postgres_changes', { event: '*', schema: 'public', table: 'admin_payments', filter: `project_id=eq.${id}` }, () =&gt; loadData())
    .on('postgres_changes', { event: '*', schema: 'public', table: 'payment_milestones', filter: `project_id=eq.${id}` }, () =&gt; loadData())
    .subscribe();
  return () =&gt; { supabase.removeChannel(channel); };
}, [id, loadData]);

const updateForm = (field: string, value: string) =&gt; {
  setForm((f: any) =&gt; ({ ...f, [field]: value }));
  setHasChanges(true);
};

// Auto-save stage changes immediately to DB
const updateStage = async (newStage: string) =&gt; {
  setForm((f: any) =&gt; ({ ...f, stage: newStage }));
  if (!id) return;
  const { error } = await supabase.from("projects").update({ stage: newStage }).eq("id", id);
  if (error) { toast.error(error.message); return; }
  // Auto-add timeline event for key stage transitions
  if (newStage === "approved") {
    const today = new Date().toISOString().split("T")[0];
    await supabase.from("project_timeline").insert({
      project_id: id,
      revision_number: -1,
      event_type: "project_approved",
      event_date: today,
      notes: "Project approved (auto-tracked from stage change)",
    });
  }
  toast.success(`Stage → ${getStageLabel(newStage)}`);
  loadData();
};

// Auto-sync stage from timeline (derived, not manual)
const handleTimelineStageSync = useCallback(async (derivedStage: string) =&gt; {
  if (!id || form.stage === derivedStage) return;
  // Only auto-sync if the derived stage is "further" in the pipeline
  const stageIndex = (s: string) =&gt; PROJECT_STAGES.indexOf(s as any);
  if (stageIndex(derivedStage) &gt; stageIndex(form.stage)) {
    setForm((f: any) =&gt; ({ ...f, stage: derivedStage }));
    await supabase.from("projects").update({ stage: derivedStage }).eq("id", id);
  }
}, [id, form.stage]);
```

```
const handleSave = async () =&gt; {
  if (!id) return;
  setSaving(true);
  const payload: any = {
    stage: form.stage,
    service_type: form.service_type || null,
    square_feet: form.square_feet ? Number(form.square_feet) : null,
    start_date: form.start_date || null,
    delivery_date: form.delivery_date || null,
    preview_delivery_date: form.preview_delivery_date || null,
    revision_return_date: form.revision_return_date || null,
    final_delivery_date: form.final_delivery_date || null,
    total_value: form.total_value ? Number(form.total_value) : 0,
    currency: form.currency,
    entry_payment_percent: Number(form.entry_payment_percent) || 50,
    payment_stage: form.payment_stage,
    internal_notes: internalNotes || null,
    date_notes: dateNotes,
    street_number: form.street_number || null,
    street_name: form.street_name || null,
    city: form.city || null,
    state: form.state || null,
    zip_code: form.zip_code || null,
    country: form.country || null,
    homeowner_name: form.homeowner_name || null,
    homeowner_phone: form.homeowner_phone || null,
    homeowner_email: form.homeowner_email || null,
    homeowner_address: form.homeowner_address || null,
  };

  const { error } = await supabase.from("projects").update(payload).eq("id", id);
  if (error) { toast.error(error.message); setSaving(false); return; }

  // Sync services
  await supabase.from("project_services").delete().eq("project_id", id);
  if (selectedServiceIds.length &gt; 0) {
    await supabase.from("project_services").insert(
      selectedServiceIds.map(sid =&gt; ({ project_id: id, service_id: sid }))
    );
  }

  toast.success("Project saved!");
  setSaving(false);
  loadData();
};

if (loading) return &lt;div className="p-6 text-muted-foreground"&gt;Loading...&lt;/div&gt;;
if (!project) return &lt;div className="p-6 text-muted-foreground"&gt;Project not found&lt;/div&gt;;

const totalPaidPayments = payments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
const totalPaidMilestones = paymentMilestones.filter(m =&gt; m.status === "paid").reduce((s: number, m: any) =&gt; s + Number(m.amount || 0), 0);
const totalPaid = totalPaidPayments + totalPaidMilestones;
const totalValue = Number(form.total_value || 0);
const isBRL = (form.currency || "USD") === "BRL";
const fmtVal = (v: number) =&gt; isBRL
  ? `R$${v.toLocaleString("pt-BR", { minimumFractionDigits: 2, maximumFractionDigits: 2 })}`
  : `$${v.toLocaleString("en-US", { minimumFractionDigits: 2, maximumFractionDigits: 2 })}`;

const entryPct = Number(form.entry_payment_percent || 50);
const entryAmt = totalValue * entryPct / 100;
const finalAmt = totalValue - entryAmt;
const isOverdue = project.delivery_date &amp;&amp; new Date(project.delivery_date) &lt; new Date() &amp;&amp; project.stage !== "Completed";
const areaUnit = project.country === "Brazil" ? "m²" : "ft²";

const remaining = Math.max(0, totalValue - totalPaid);
const paidPct = totalValue &gt; 0 ? Math.round((totalPaid / totalValue) * 100) : 0;

// Use form values (editable) for address display
const addrStreet = [form.street_number, form.street_name].filter(Boolean).join(" ");
const fullAddress = [addrStreet, form.city, form.state, form.country].filter(Boolean).join(", ");
const projectTitle = addrStreet || project.name || "Untitled Project";
const projectNumber = project.project_number || "";

// Google Maps link from form (live updated)
const mapsQuery = [form.street_number, form.street_name, form.city, form.state, form.zip_code].filter(Boolean).join("+");
const mapsUrl = mapsQuery ? `https://www.google.com/maps/search/?api=1&amp;query=${encodeURIComponent(mapsQuery)}` : null;

// Health status
const getHealthStatus = () =&gt; {
  if (project.stage === "Completed") return { label: "Completed", color: "text-chart-2", icon: CheckCircle2, bg: "bg-chart-2/15" };
  if (project.stage === "Cancelled") return { label: "Cancelled", color: "text-muted-foreground", icon: XCircle, bg: "bg-muted" };
  if (isOverdue) return { label: "Delayed", color: "text-destructive", icon: XCircle, bg: "bg-destructive/15" };
  const deadline = project.delivery_date ? new Date(project.delivery_date) : null;
  if (deadline) {
    const daysLeft = (deadline.getTime() - Date.now()) / (1000 * 60 * 60 * 24);
    if (daysLeft &lt;= 7 &amp;&amp; daysLeft &gt; 0) return { label: "Attention Needed", color: "text-chart-3", icon: AlertTriangle, bg: "bg-chart-3/15" };
  }
  return { label: "On Track", color: "text-chart-2", icon: CheckCircle2, bg: "bg-chart-2/15" };
};
const health = getHealthStatus();

// Group drawings by name
const drawingGroups: Record&lt;string, any[]&gt; = {};
drawings.forEach(d =&gt; {
  if (!drawingGroups[d.drawing_name]) drawingGroups[d.drawing_name] = [];
  drawingGroups[d.drawing_name].push(d);
});

const paymentStatusColor = (status: string) =&gt; {
  if (status === "paid") return "bg-chart-2/15 text-chart-2 border-chart-2/30";
```

```
  if (status === "pending") return "bg-chart-3/15 text-chart-3 border-chart-3/30";
  return "bg-destructive/15 text-destructive border-destructive/30";
};

const toggleService = (serviceId: string) =&gt; {
  setSelectedServiceIds(prev =&gt;
    prev.includes(serviceId) ? prev.filter(id =&gt; id !== serviceId) : [...prev, serviceId]
  );
  setHasChanges(true);
};

const addDateNote = () =&gt; {
  setDateNotes(prev =&gt; [...prev, { date: new Date().toISOString().split("T")[0], note: "", visibility: "private" }]);
  setHasChanges(true);
};

const updateDateNote = (index: number, field: keyof DateNote, value: string) =&gt; {
  setDateNotes(prev =&gt; prev.map((n, i) =&gt; i === index ? { ...n, [field]: value } : n));
  setHasChanges(true);
};

const removeDateNote = (index: number) =&gt; {
  setDateNotes(prev =&gt; prev.filter((_, i) =&gt; i !== index));
  setHasChanges(true);
};

// Group services by category
const servicesByCategory: Record&lt;string, any[]&gt; = {};
allServices.forEach(s =&gt; {
  const cat = s.category || "Other";
  if (!servicesByCategory[cat]) servicesByCategory[cat] = [];
  servicesByCategory[cat].push(s);
});

return (
  &lt;div className="p-6 space-y-6 max-w-[1400px] mx-auto"&gt;
    {/* Back + Save */}
    &lt;div className="flex items-center justify-between"&gt;
      &lt;Button variant="ghost" className="gap-2 -ml-2" onClick={() =&gt; navigate("/adm/projects")}&gt;
        &lt;ArrowLeft className="h-4 w-4" /&gt; Back to Projects
      &lt;/Button&gt;
      {hasChanges &amp;amp;&amp;amp; (
        &lt;Button onClick={handleSave} disabled={saving} className="gap-2"&gt;
          &lt;Save className="h-4 w-4" /&gt; {saving ? "Saving..." : "Save Changes"}
        &lt;/Button&gt;
      )}
    &lt;/div&gt;

    {/* Header */}
    &lt;div className="flex items-start gap-4"&gt;
      &lt;div className="relative group/avatar"&gt;
        &lt;Avatar className="h-16 w-16 border border-border"&gt;
          &lt;AvatarImage src={project.cover_image || undefined} className="object-cover" /&gt;
          &lt;AvatarFallback className="bg-muted text-muted-foreground text-lg"&gt;
            {projectTitle.substring(0, 2).toUpperCase()}
          &lt;/AvatarFallback&gt;
        &lt;/Avatar&gt;
        &lt;button
          onClick={() =&gt; coverInputRef.current?.click()}
          className="absolute inset-0 flex items-center justify-center rounded-full bg-foreground/50 opacity-0 group-hover/avatar:opacity-100 transition-opacity"
        &gt;
          &lt;Pencil className="h-4 w-4 text-background" /&gt;
        &lt;/button&gt;
        &lt;input
          ref={coverInputRef}
          type="file"
          accept="image/*"
          className="hidden"
          onChange={async (e) =&gt; {
            const file = e.target.files?.[0];
            if (!file || !id) return;
            const ext = file.name.split(".").pop();
            const path = `${id}.${ext}`;
            const { error: uploadError } = await supabase.storage.from("portfolio").upload(path, file, { upsert: true });
            if (uploadError) { toast.error(uploadError.message); return; }
            const { data: urlData } = supabase.storage.from("portfolio").getPublicUrl(path);
            const coverUrl = urlData.publicUrl + "?t=" + Date.now();
            await supabase.from("projects").update({ cover_image: coverUrl }).eq("id", id);
            setProject((prev: any) =&gt; prev ? { ...prev, cover_image: coverUrl } : prev);
            toast.success("Cover image updated");
          }}
        /&gt;
      &lt;/div&gt;
      &lt;div className="flex-1 min-w-0"&gt;
        &lt;div className="flex items-center gap-3 flex-wrap"&gt;
          {projectNumber &amp;amp;&amp;amp; (
            &lt;span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-md bg-muted text-xs font-mono font-semibold text-muted-foreground border border-border"&
              &lt;Hash className="h-3 w-3" /&gt;
              {projectNumber}
            &lt;/span&gt;
          )}
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;{projectTitle}&lt;/h1&gt;
          &lt;span className={`inline-flex items-center gap-1.5 px-2.5 py-1 rounded-full text-xs font-medium ${health.bg} ${health.color}`}&gt;
            &lt;health.icon className="h-3 w-3" /&gt;
            {health.label}
          &lt;/span&gt;
        &lt;/div&gt;
        &lt;div className="flex items-center gap-4 mt-1 text-sm text-muted-foreground flex-wrap"&gt;
          &lt;span
            className="flex items-center gap-1 hover:text-primary cursor-pointer"
            onClick={() =&gt; navigate(`/adm/clients/${project.company_id}`)}
```

```
        &gt;
          &lt;Building2 className="h-3.5 w-3.5" /&gt;
          {project.companies?.name} {getCountryFlag(project.companies?.country)}
        &lt;/span&gt;
        {fullAddress &amp;&amp; (
          &lt;span className="flex items-center gap-1"&gt;
            &lt;MapPin className="h-3.5 w-3.5" /&gt;
            {fullAddress}
          &lt;/span&gt;
        )}
        {mapsUrl &amp;&amp; (
          &lt;a
            href={mapsUrl}
            target="_blank"
            rel="noopener noreferrer"
            className="flex items-center gap-1 text-primary hover:underline text-xs"
          &gt;
            &lt;ExternalLink className="h-3 w-3" /&gt;
            Open in Maps
          &lt;/a&gt;
        )}
        &lt;span className="flex items-center gap-1"&gt;
          &lt;Badge variant="outline" className="text-xs"&gt;{getStageLabel(form.stage)}&lt;/Badge&gt;
        &lt;/span&gt;
        {project.delivery_date &amp;&amp; (
          &lt;span className={`flex items-center gap-1 ${isOverdue ? "text-destructive" : ""}`}&gt;
            &lt;Calendar className="h-3.5 w-3.5" /&gt;
            {format(new Date(project.delivery_date), "MMM d")}
            {isOverdue &amp;&amp; " ■"}
          &lt;/span&gt;
        )}
      &lt;/div&gt;
      {/* Financial summary in header */}
      &lt;div className="flex items-center gap-4 mt-2 flex-wrap"&gt;
        &lt;div className="flex items-center gap-6 text-sm"&gt;
          &lt;span className="text-muted-foreground"&gt;Budget: &lt;span className="font-semibold text-foreground"&gt;{fmtVal(totalValue)}&lt;/span&gt;&lt;/span&gt;
          &lt;span className="text-muted-foreground"&gt;Paid: &lt;span className="font-semibold text-chart-2"&gt;{fmtVal(totalPaid)}&lt;/span&gt;&lt;/span&gt;
          &lt;span className="text-muted-foreground"&gt;Remaining: &lt;span className={`font-semibold ${remaining &gt; 0 ? "text-chart-3" : "text-chart-2"}`}&gt;{fmtVal(
        &lt;/div&gt;
        {totalValue &gt; 0 &amp;&amp; (
          &lt;div className="flex items-center gap-2 min-w-[160px]"&gt;
            &lt;Progress value={paidPct} className="h-2 flex-1" /&gt;
            &lt;span className="text-xs text-muted-foreground font-medium"&gt;{paidPct}%&lt;/span&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  &lt;/div&gt;
&lt;/div&gt;

{/* Tabs */}
&lt;Tabs defaultValue="overview"&gt;
  &lt;TabsList className="flex-wrap h-auto gap-1"&gt;
    &lt;TabsTrigger value="overview"&gt;Overview&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="progress"&gt;Progress&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="services"&gt;Services&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="timeline"&gt;Timeline&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="drawings"&gt;Drawings&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="files"&gt;Files&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="site-updates"&gt;Site Updates&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="finance"&gt;Finance&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="notes"&gt;Notes&lt;/TabsTrigger&gt;
    &lt;TabsTrigger value="client-portal"&gt;Client Portal&lt;/TabsTrigger&gt;
  &lt;/TabsList&gt;

  {/* ============ OVERVIEW ============ */}
  &lt;TabsContent value="overview"&gt;
    &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
      {/* Client Info */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;CardTitle className="text-base flex items-center gap-2"&gt;
            &lt;Building2 className="h-4 w-4" /&gt; Client Information
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent className="space-y-3 text-sm"&gt;
          &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;User className="h-3.5 w-3.5" /&gt; Owne
          &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;Building2 className="h-3.5 w-3.5" /&gt;
          &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;Mail className="h-3.5 w-3.5" /&gt; Emai
          &lt;div className="flex justify-between"&gt;&lt;span class="text-muted-foreground flex items-center gap-1"&gt;&lt;Phone className="h-3.5 w-3.5" /&gt; Pho
          &lt;div className="flex justify-between"&gt;&lt;span class="text-muted-foreground flex items-center gap-1"&gt;&lt;Globe className="h-3.5 w-3.5" /&gt; Cou
          &lt;div className="flex justify-between"&gt;&lt;span class="text-muted-foreground flex items-center gap-1"&gt;&lt;Languages className="h-3.5 w-3.5" /&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Project Details (editable) */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;CardTitle className="text-base flex items-center gap-2"&gt;
            &lt;FolderOpen className="h-4 w-4" /&gt; Project Details
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent className="space-y-4 text-sm"&gt;
          &lt;div&gt;
            &lt;Label className="text-xs text-muted-foreground"&gt;Project Stage&lt;/Label&gt;
            &lt;Select value={form.stage} onValueChange={v =&gt; updateStage(v)}&gt;
              &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;{PROJECT_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{getStageLabel(s)}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/div&gt;
          &lt;div&gt;
```

```
      &lt;Label className="text-xs text-muted-foreground"&gt;Service Type&lt;/Label&gt;
      &lt;Input className="mt-1" value={form.service_type} onChange={e =&gt; updateForm("service_type", e.target.value)} placeholder="Ex: Remodeling, 3D Kitchen.
    &lt;/div&gt;
    &lt;div&gt;
      &lt;Label className="text-xs text-muted-foreground"&gt;Project Area ({areaUnit})&lt;/Label&gt;
      &lt;Input className="mt-1" type="number" value={form.square_feet} onChange={e =&gt; updateForm("square_feet", e.target.value)} placeholder={project.country
    &lt;/div&gt;
    &lt;div class="flex justify-between text-muted-foreground"&gt;
      &lt;span&gt;Revisions&lt;/span&gt;
      &lt;span class="text-foreground"&gt;{project.revision_count}&lt;/span&gt;
    &lt;/div&gt;
  &lt;/CardContent&gt;
&lt;/Card&gt;

{/* Project Address (editable) */}
&lt;Card class="border-none shadow-sm lg:col-span-2"&gt;
  &lt;CardHeader class="pb-3"&gt;
    &lt;CardTitle class="text-base flex items-center gap-2"&gt;
      &lt;MapPin class="h-4 w-4" /&gt; Project Address
    &lt;/CardTitle&gt;
  &lt;/CardHeader&gt;
  &lt;CardContent class="space-y-4 text-sm"&gt;
    &lt;div class="grid grid-cols-2 lg:grid-cols-4 gap-4"&gt;
      &lt;div class="lg:col-span-2"&gt;
        &lt;Label class="text-xs text-muted-foreground"&gt;Street&lt;/Label&gt;
        &lt;div class="flex gap-2 mt-1"&gt;
          &lt;Input class="w-24" value={form.street_number} onChange={e =&gt; updateForm("street_number", e.target.value)} placeholder="No." /&gt;
          &lt;Input class="flex-1" value={form.street_name} onChange={e =&gt; updateForm("street_name", e.target.value)} placeholder="Street name" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;Label class="text-xs text-muted-foreground"&gt;City&lt;/Label&gt;
        &lt;Input class="mt-1" value={form.city} onChange={e =&gt; updateForm("city", e.target.value)} placeholder="City" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;Label class="text-xs text-muted-foreground"&gt;State&lt;/Label&gt;
        &lt;Input class="mt-1" value={form.state} onChange={e =&gt; updateForm("state", e.target.value)} placeholder="State" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;Label class="text-xs text-muted-foreground"&gt;Zip Code&lt;/Label&gt;
        &lt;Input class="mt-1" value={form.zip_code} onChange={e =&gt; updateForm("zip_code", e.target.value)} placeholder="Zip" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;Label class="text-xs text-muted-foreground"&gt;Country&lt;/Label&gt;
        &lt;Select value={form.country} onValueChange={v =&gt; updateForm("country", v)}&gt;
          &lt;SelectTrigger class="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
          &lt;SelectContent&gt;
            &lt;SelectItem value="USA"&gt;USA&lt;/SelectItem&gt;
            &lt;SelectItem value="Brazil"&gt;Brazil&lt;/SelectItem&gt;
            &lt;SelectItem value="Other"&gt;Other&lt;/SelectItem&gt;
          &lt;/SelectContent&gt;
        &lt;/Select&gt;
      &lt;/div&gt;
    &lt;/div&gt;
    {fullAddress &amp;&amp; (
      &lt;div class="flex items-center gap-3 p-3 rounded-md bg-muted/30 border border-border"&gt;
        &lt;MapPin class="h-4 w-4 text-muted-foreground flex-shrink-0" /&gt;
        &lt;span class="text-sm text-foreground"&gt;{fullAddress}&lt;/span&gt;
        {mapsUrl &amp;&amp; (
          &lt;a href={mapsUrl} target="_blank" rel="noopener noreferrer" class="ml-auto flex items-center gap-1 text-primary hover:underline text-xs whitespa
            &lt;ExternalLink class="h-3 w-3" /&gt; Open in Maps
          &lt;/a&gt;
        )}
      &lt;/div&gt;
    )}
  &lt;/CardContent&gt;
&lt;/Card&gt;

{/* Homeowner / Final Client */}
&lt;Card class="border-none shadow-sm lg:col-span-2"&gt;
  &lt;CardHeader class="pb-3"&gt;
    &lt;div class="flex items-center justify-between"&gt;
      &lt;CardTitle class="text-base flex items-center gap-2"&gt;
        &lt;User class="h-4 w-4" /&gt; Homeowner / Final Client
      &lt;/CardTitle&gt;
      &lt;Button
        size="sm"
        variant="outline"
        onClick={async () =&gt; {
          const { error } = await supabase.from("project_homeowners").insert({ project_id: id!, name: "" });
          if (error) { toast.error(error.message); return; }
          loadData();
        }}
      &gt;
        &lt;Plus class="h-3.5 w-3.5 mr-1" /&gt; Add Person
      &lt;/Button&gt;
    &lt;/div&gt;
  &lt;/CardHeader&gt;
  &lt;CardContent class="space-y-4 text-sm"&gt;
    {homeowners.length === 0 &amp;&amp; (
      &lt;p class="text-center text-muted-foreground py-4"&gt;No homeowners added yet&lt;/p&gt;
    )}
    {homeowners.map((ho: any, idx: number) =&gt; (
      &lt;div key={ho.id} class="p-4 rounded-lg border bg-muted/20 space-y-3"&gt;
        &lt;div class="flex items-center justify-between"&gt;
          &lt;span class="text-xs font-semibold text-muted-foreground uppercase"&gt;Person {idx + 1}&lt;/span&gt;
          &lt;Button
            variant="ghost"
            size="icon"
            class="h-6 w-6 text-destructive hover:text-destructive"
            onClick={async () =&gt; {
```

```
                  await supabase.from("project_homeowners").delete().eq("id", ho.id);
                  loadData();
                }}
              &gt;
                &lt;Trash2 className="h-3.5 w-3.5" /&gt;
              &lt;/Button&gt;
            &lt;/div&gt;
            &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-3"&gt;
              &lt;div&gt;
                &lt;Label className="text-xs text-muted-foreground"&gt;Full Name&lt;/Label&gt;
                &lt;Input
                  className="mt-1"
                  defaultValue={ho.name}
                  placeholder="Name"
                  onBlur={async (e) =&gt; {
                    if (e.target.value !== ho.name) {
                      await supabase.from("project_homeowners").update({ name: e.target.value }).eq("id", ho.id);
                      loadData();
                    }
                  }}
                /&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;Label className="text-xs text-muted-foreground"&gt;Phone&lt;/Label&gt;
                &lt;Input
                  className="mt-1"
                  defaultValue={ho.phone || ""}
                  placeholder="+1 555-0000"
                  onBlur={async (e) =&gt; {
                    if (e.target.value !== (ho.phone || "")) {
                      await supabase.from("project_homeowners").update({ phone: e.target.value || null }).eq("id", ho.id);
                      loadData();
                    }
                  }}
                /&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;Label className="text-xs text-muted-foreground"&gt;Email&lt;/Label&gt;
                &lt;Input
                  className="mt-1"
                  defaultValue={ho.email || ""}
                  placeholder="email@example.com"
                  onBlur={async (e) =&gt; {
                    if (e.target.value !== (ho.email || "")) {
                      await supabase.from("project_homeowners").update({ email: e.target.value || null }).eq("id", ho.id);
                      loadData();
                    }
                  }}
                /&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;Label className="text-xs text-muted-foreground"&gt;Address (optional)&lt;/Label&gt;
                &lt;Input
                  className="mt-1"
                  defaultValue={ho.address || ""}
                  placeholder="Home address"
                  onBlur={async (e) =&gt; {
                    if (e.target.value !== (ho.address || "")) {
                      await supabase.from("project_homeowners").update({ address: e.target.value || null }).eq("id", ho.id);
                      loadData();
                    }
                  }}
                /&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        ))}
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/div&gt;
&lt;/TabsContent&gt;

{/* ============ PROGRESS (Kanban) ============ */}
&lt;TabsContent value="progress"&gt;
  &lt;ProjectKanbanBoard
    projectId={id!}
    onStageSync={(stage) =&gt; {
      updateStage(stage);
    }}
  /&gt;
&lt;/TabsContent&gt;

{/* ============ SERVICES ============ */}
&lt;TabsContent value="services"&gt;
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;CardTitle className="text-base"&gt;Project Services&lt;/CardTitle&gt;
      &lt;p className="text-xs text-muted-foreground"&gt;Click tags to add or remove services&lt;/p&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent className="space-y-6"&gt;
      {Object.entries(servicesByCategory).map(([cat, svcs]) =&gt; (
        &lt;div key={cat}&gt;
          &lt;p className="text-xs font-semibold text-muted-foreground uppercase tracking-wider mb-3"&gt;{cat}&lt;/p&gt;
          &lt;div className="flex flex-wrap gap-2"&gt;
            {svcs.map(s =&gt; {
              const isSelected = selectedServiceIds.includes(s.id);
              return (
                &lt;button
                  key={s.id}
                  type="button"
```

```
                  onClick={() =&gt; toggleService(s.id)}
                  className={`px-3 py-1.5 rounded-full text-sm font-medium border transition-colors ${
                    isSelected
                      ? "bg-primary text-primary-foreground border-primary"
                      : "bg-muted/50 text-muted-foreground border-border hover:border-primary/50 hover:text-foreground"
                  }`}
                &gt;
                  {s.name}
                &lt;/button&gt;
              );
            })}
          &lt;/div&gt;
        &lt;/div&gt;
      ))}
      {allServices.length === 0 &amp;&amp; (
        &lt;p className="text-center text-muted-foreground py-8"&gt;No services configured&lt;/p&gt;
      )}
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/TabsContent&gt;

{/* ============ TIMELINE ============ */}
&lt;TabsContent value="timeline"&gt;
  &lt;ProjectTimeline
    projectId={id!}
    deliveryDate={form.delivery_date}
    dates={{
      start_date: form.start_date,
      delivery_date: form.delivery_date,
      preview_delivery_date: form.preview_delivery_date,
      revision_return_date: form.revision_return_date,
      final_delivery_date: form.final_delivery_date,
    }}
    onDateChange={(field, value) =&gt; updateForm(field, value)}
    onStageSync={handleTimelineStageSync}
  /&gt;
  {/* Activity Log (existing notes) */}
  &lt;Card className="border-none shadow-sm mt-6"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;CardTitle className="text-base"&gt;Project Activity Log&lt;/CardTitle&gt;
        &lt;Button variant="outline" size="sm" onClick={addDateNote}&gt;
          &lt;Plus className="h-3 w-3 mr-1" /&gt; Add Note
        &lt;/Button&gt;
      &lt;/div&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent className="space-y-3"&gt;
      {dateNotes.length === 0 &amp;&amp; activityLog.length === 0 &amp;&amp; (
        &lt;div className="text-center py-8 text-muted-foreground text-sm border border-dashed rounded-md"&gt;
          No activity notes yet. Click "Add Note" to start.
        &lt;/div&gt;
      )}
      {dateNotes.sort((a, b) =&gt; (b.date || "").localeCompare(a.date || "")).map((note, idx) =&gt; {
        const visOpt = VISIBILITY_OPTIONS.find(v =&gt; v.value === note.visibility) || VISIBILITY_OPTIONS[0];
        const VisIcon = visOpt.icon;
        return (
          &lt;div key={`note-${idx}`} className="grid grid-cols-[130px_1fr_150px_36px] items-center gap-3 p-3 rounded-md border bg-muted/30"&gt;
            &lt;Input type="date" value={note.date} onChange={e =&gt; updateDateNote(idx, "date", e.target.value)} className="text-sm" /&gt;
            &lt;Input value={note.note} onChange={e =&gt; updateDateNote(idx, "note", e.target.value)} placeholder="Describe the event..." className="text-sm" /&gt;
            &lt;Select value={note.visibility} onValueChange={v =&gt; updateDateNote(idx, "visibility", v)}&gt;
              &lt;SelectTrigger className="text-sm"&gt;
                &lt;div className="flex items-center gap-1.5"&gt;
                  &lt;VisIcon className={`h-3.5 w-3.5 ${visOpt.color}`} /&gt;
                  &lt;span&gt;{visOpt.label}&lt;/span&gt;
                &lt;/div&gt;
              &lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;
                {VISIBILITY_OPTIONS.map(v =&gt; (
                  &lt;SelectItem key={v.value} value={v.value}&gt;
                    &lt;div className="flex items-center gap-1.5"&gt;
                      &lt;v.icon className={`h-3.5 w-3.5 ${v.color}`} /&gt;
                      &lt;span&gt;{v.label}&lt;/span&gt;
                    &lt;/div&gt;
                  &lt;/SelectItem&gt;
                ))}
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
            &lt;Button type="button" variant="ghost" size="icon" className="h-8 w-8 text-destructive" onClick={() =&gt; removeDateNote(idx)}&gt;
              &lt;Trash2 className="h-3.5 w-3.5" /&gt;
            &lt;/Button&gt;
          &lt;/div&gt;
        );
      })}
      {activityLog.map(log =&gt; (
        &lt;div key={log.id} className="flex items-center gap-3 p-3 rounded-md border bg-card"&gt;
          &lt;span className="text-xs text-muted-foreground w-[100px] shrink-0"&gt;{format(new Date(log.log_date), "MMM d, yyyy")}&lt;/span&gt;
          &lt;span className="text-sm flex-1"&gt;{log.description}&lt;/span&gt;
          &lt;Badge variant="outline" className="text-[10px]"&gt;{log.log_type}&lt;/Badge&gt;
        &lt;/div&gt;
      ))}
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/TabsContent&gt;

{/* ============ DRAWINGS ============ */}
&lt;TabsContent value="drawings"&gt;
  &lt;div className="space-y-4"&gt;
    {Object.keys(drawingGroups).length === 0 &amp;&amp; (
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="text-center text-muted-foreground py-12"&gt;
          &lt;FileImage className="h-10 w-10 mx-auto mb-3 opacity-40" /&gt;
```

```
          &lt;p className="text-sm"&gt;No drawings yet&lt;/p&gt;
          &lt;p className="text-xs text-muted-foreground/70 mt-1"&gt;Drawings and version history will appear here&lt;/p&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    )}
    {Object.entries(drawingGroups).map(([name, versions]) =&gt; (
      &lt;Card key={name} className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-2"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;FileImage className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-sm"&gt;{name}&lt;/CardTitle&gt;
            &lt;Badge variant="secondary" className="ml-auto text-[10px]"&gt;{versions.length} version{versions.length &gt; 1 ? "s" : ""}&lt;/Badge&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          &lt;div className="space-y-2"&gt;
            {versions.map(v =&gt; (
              &lt;div key={v.id} className={`flex items-center gap-3 p-2.5 rounded-md border transition-colors ${v.is_current ? "border-primary/30 bg-primary/5" : "b
                &lt;div className="h-8 w-8 rounded bg-muted flex items-center justify-center text-xs font-bold text-muted-foreground"&gt;v{v.version_number}&lt;/div&
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;p className="text-sm font-medium truncate"&gt;{v.file_name}&lt;/p&gt;
                  {v.revision_notes &amp;&amp; &lt;p className="text-xs text-muted-foreground truncate"&gt;{v.revision_notes}&lt;/p&gt;}
                  &lt;div className="flex items-center gap-2 text-xs text-muted-foreground"&gt;
                    &lt;span className="flex items-center gap-1"&gt;&lt;Clock className="h-3 w-3" /&gt;{format(new Date(v.created_at), "MMM d, yyyy")}&lt;/span&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
                &lt;div className="flex items-center gap-1"&gt;
                  {v.is_current &amp;&amp; &lt;Badge variant="outline" className="text-[10px] bg-primary/10 text-primary"&gt;&lt;Star className="h-2.5 w-2.5 mr-0.5"
                  &lt;Button variant="ghost" size="icon" className="h-7 w-7" asChild&gt;
                    &lt;a href={v.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-3.5 w-3.5" /&gt;&lt;/a&gt;
                  &lt;/Button&gt;
                &lt;/div&gt;
              &lt;/div&gt;
            ))}
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    ))}
  &lt;/div&gt;
&lt;/TabsContent&gt;

{/* ============ FILES ============ */}
&lt;TabsContent value="files"&gt;
  &lt;ProjectDocuments projectId={id!} /&gt;
&lt;/TabsContent&gt;

{/* ============ SITE UPDATES ============ */}
&lt;TabsContent value="site-updates"&gt;
  &lt;SiteUpdatesTab projectId={id!} /&gt;
&lt;/TabsContent&gt;

{/* ============ FINANCE ============ */}
&lt;TabsContent value="finance"&gt;
  &lt;div className="space-y-6"&gt;
    {/* Budget config */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;CardTitle className="text-base"&gt;Project Budget&lt;/CardTitle&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="space-y-4"&gt;
        &lt;div className="grid grid-cols-2 lg:grid-cols-4 gap-4"&gt;
          &lt;div&gt;
            &lt;Label className="text-xs text-muted-foreground"&gt;Currency&lt;/Label&gt;
            &lt;Select value={form.currency} onValueChange={v =&gt; updateForm("currency", v)}&gt;
              &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;
                &lt;SelectItem value="USD"&gt;USD ($)&lt;/SelectItem&gt;
                &lt;SelectItem value="BRL"&gt;BRL (R$)&lt;/SelectItem&gt;
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label className="text-xs text-muted-foreground"&gt;Total Value&lt;/Label&gt;
            &lt;Input type="number" className="mt-1" value={form.total_value} onChange={e =&gt; updateForm("total_value", e.target.value)} /&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label className="text-xs text-muted-foreground"&gt;Entry %&lt;/Label&gt;
            &lt;Input type="number" className="mt-1" value={form.entry_payment_percent} onChange={e =&gt; updateForm("entry_payment_percent", e.target.value)} /&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label className="text-xs text-muted-foreground"&gt;Payment Stage&lt;/Label&gt;
            &lt;Select value={form.payment_stage} onValueChange={v =&gt; updateForm("payment_stage", v)}&gt;
              &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;
                {["Not Sent", "Waiting Entry", "Entry Paid", "Waiting Final", "Paid", "Overdue", "Refunded"].map(s =&gt; (
                  &lt;SelectItem key={s} value={s}&gt;{s}&lt;/SelectItem&gt;
                ))}
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Payment summary cards */}
    &lt;div className="grid grid-cols-1 sm:grid-cols-3 gap-4"&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4"&gt;
          &lt;p className="text-xs text-muted-foreground mb-1"&gt;Entry Payment ({entryPct}%)&lt;/p&gt;
          &lt;p className="text-lg font-bold text-foreground"&gt;{fmtVal(entryAmt)}&lt;/p&gt;
```

```
      &lt;Badge variant="outline" className="mt-1 text-[10px]"&gt;
        {totalPaid &gt;= entryAmt &amp;&amp; totalValue &gt; 0 ? "Paid" : "Pending"}
      &lt;/Badge&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="p-4"&gt;
      &lt;p className="text-xs text-muted-foreground mb-1"&gt;Final Payment ({100 - entryPct}%)&lt;/p&gt;
      &lt;p className="text-lg font-bold text-foreground"&gt;{fmtVal(finalAmt)}&lt;/p&gt;
      &lt;Badge variant="outline" className="mt-1 text-[10px]"&gt;
        {totalPaid &gt;= totalValue &amp;&amp; totalValue &gt; 0 ? "Paid" : "Pending"}
      &lt;/Badge&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="p-4"&gt;
      &lt;p className="text-xs text-muted-foreground mb-1"&gt;Balance Due&lt;/p&gt;
      &lt;p className={`text-lg font-bold ${totalPaid &gt;= totalValue &amp;&amp; totalValue &gt; 0 ? "text-chart-2" : "text-destructive"}`}&gt;
        {fmtVal(Math.max(0, totalValue - totalPaid))}
      &lt;/p&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Payment Milestones */}
&lt;Card className="border-none shadow-sm"&gt;
  &lt;CardHeader className="pb-3"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;CardTitle className="text-base flex items-center gap-2"&gt;
        &lt;Receipt className="h-4 w-4" /&gt; Payment Milestones
      &lt;/CardTitle&gt;
      {paymentMilestones.length === 0 &amp;&amp; totalValue &gt; 0 &amp;&amp; (
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Select
            onValueChange={async (template) =&gt; {
              const templates: Record&lt;string, { stage: string; percentage: number }[]&gt; = {
                "entry_final": [
                  { stage: "Entry Payment", percentage: 50 },
                  { stage: "Final Payment", percentage: 50 },
                ],
                "architecture": [
                  { stage: "Concept Design", percentage: 40 },
                  { stage: "Construction Documents", percentage: 30 },
                  { stage: "Permit Submission", percentage: 30 },
                ],
                "thirds": [
                  { stage: "Phase 1", percentage: 33 },
                  { stage: "Phase 2", percentage: 34 },
                  { stage: "Phase 3", percentage: 33 },
                ],
              };
              const milestones = templates[template];
              if (!milestones) return;
              const rows = milestones.map(m =&gt; ({
                project_id: id!,
                stage: m.stage,
                percentage: m.percentage,
                amount: Math.round(totalValue * m.percentage / 100 * 100) / 100,
                status: "pending",
              }));
              const { error } = await supabase.from("payment_milestones").insert(rows);
              if (error) { toast.error(error.message); return; }
              // Auto-add timeline event: Invoice Sent
              const today = new Date().toISOString().split("T")[0];
              await supabase.from("project_timeline").insert({
                project_id: id!,
                revision_number: -1,
                event_type: "invoice_sent_auto",
                event_date: today,
                notes: `Invoices created: ${milestones.map(m =&gt; m.stage).join(", ")}`,
              });
              toast.success("Milestones created from template");
              loadData();
            }}
          &gt;
            &lt;SelectTrigger className="h-8 w-[220px] text-xs"&gt;
              &lt;SelectValue placeholder="Select Payment Template" /&gt;
            &lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              &lt;SelectItem value="entry_final"&gt;Entry / Final (50/50)&lt;/SelectItem&gt;
              &lt;SelectItem value="architecture"&gt;Architecture Milestones (40/30/30)&lt;/SelectItem&gt;
              &lt;SelectItem value="thirds"&gt;Three Phases (33/34/33)&lt;/SelectItem&gt;
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
        &lt;/div&gt;
      )}
      {paymentMilestones.length &gt; 0 &amp;&amp; (
        &lt;Button
          size="sm"
          variant="ghost"
          className="text-xs text-destructive hover:text-destructive"
          onClick={async () =&gt; {
            if (!confirm("Delete all milestones and choose a new template?")) return;
            const { error } = await supabase.from("payment_milestones").delete().eq("project_id", id!);
            if (error) { toast.error(error.message); return; }
            toast.success("Milestones cleared");
            loadData();
          }}
        &gt;
          &lt;Trash2 className="h-3.5 w-3.5 mr-1" /&gt; Reset
        &lt;/Button&gt;
```

```
    )}
  &lt;/div&gt;
&lt;/CardHeader&gt;
&lt;CardContent&gt;
  {paymentMilestones.length === 0 ? (
    &lt;div className="text-center text-muted-foreground py-8"&gt;
      &lt;Receipt className="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;
      &lt;p className="text-sm"&gt;No payment milestones yet&lt;/p&gt;
      &lt;p className="text-xs text-muted-foreground/70 mt-1"&gt;
        {totalValue &gt; 0 ? "Click \"Generate Default Milestones\" to create them" : "Set a project budget first"}
      &lt;/p&gt;
    &lt;/div&gt;
  ) : (
    &lt;div className="space-y-4"&gt;
      {/* Progress */}
      {(() =&gt; {
        const msPaid = paymentMilestones.filter((m: any) =&gt; m.status === "paid").reduce((s: number, m: any) =&gt; s + Number(m.amount || 0), 0);
        const msTotal = paymentMilestones.reduce((s: number, m: any) =&gt; s + Number(m.amount || 0), 0);
        const msPct = msTotal &gt; 0 ? Math.round((msPaid / msTotal) * 100) : 0;
        return (
          &lt;div className="flex items-center gap-3"&gt;
            &lt;Progress value={msPct} className="h-2.5 flex-1" /&gt;
            &lt;span className="text-sm font-semibold text-muted-foreground min-w-[50px] text-right"&gt;{msPct}% Paid&lt;/span&gt;
          &lt;/div&gt;
        );
      })()}
      {/* Milestone rows */}
      &lt;div className="space-y-2"&gt;
        {paymentMilestones.map((ms: any) =&gt; (
          &lt;div key={ms.id} className="flex items-center gap-4 p-3 rounded-lg border bg-card"&gt;
            &lt;div className="flex-1 min-w-0"&gt;
              &lt;p className="text-sm font-medium"&gt;{ms.stage}&lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground"&gt;{ms.percentage}% — {fmtVal(Number(ms.amount))}&lt;/p&gt;
            &lt;/div&gt;
            &lt;Select
              value={ms.status}
              onValueChange={async (newStatus) =&gt; {
                const updatePayload: any = { status: newStatus };
                const today = new Date().toISOString().split("T")[0];
                if (newStatus === "paid") updatePayload.payment_date = today;
                if (newStatus === "pending") updatePayload.payment_date = null;
                const { error } = await supabase.from("payment_milestones").update(updatePayload).eq("id", ms.id);
                if (error) { toast.error(error.message); return; }

                // Auto-create admin_payments record when milestone is paid
                if (newStatus === "paid") {
                  await supabase.from("admin_payments").insert({
                    project_id: id!,
                    installment_type: ms.stage,
                    amount_total: Number(ms.amount),
                    amount_paid: Number(ms.amount),
                    status: "paid",
                    payment_method: "Bank Transfer",
                    payment_date: today,
                    payment_notes: `Auto-generated from milestone: ${ms.stage} (${ms.percentage}%)`,
                  });
                  // Auto-add timeline event: Payment Received
                  await supabase.from("project_timeline").insert({
                    project_id: id!,
                    revision_number: -1,
                    event_type: "payment_received_auto",
                    event_date: today,
                    notes: `${ms.stage} — ${fmtVal(Number(ms.amount))} paid`,
                  });
                }

                // Remove auto-generated payment if reverted to pending
                if (newStatus === "pending") {
                  await supabase.from("admin_payments")
                    .delete()
                    .eq("project_id", id!)
                    .eq("installment_type", ms.stage)
                    .like("payment_notes", `Auto-generated from milestone: ${ms.stage}%`);
                  // Remove auto timeline event
                  await supabase.from("project_timeline")
                    .delete()
                    .eq("project_id", id!)
                    .eq("event_type", "payment_received_auto")
                    .like("notes", `${ms.stage}%`);
                }

                toast.success(`${ms.stage} marked as ${newStatus}`);
                loadData();
              }}
            &gt;
              &lt;SelectTrigger className={`h-7 w-[110px] text-xs border ${
                ms.status === "paid" ? "bg-chart-2/15 text-chart-2 border-chart-2/30" :
                ms.status === "overdue" ? "bg-destructive/15 text-destructive border-destructive/30" :
                "bg-chart-3/15 text-chart-3 border-chart-3/30"
              }`}&gt;
                &lt;SelectValue /&gt;
              &lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;
                &lt;SelectItem value="pending"&gt;Pending&lt;/SelectItem&gt;
                &lt;SelectItem value="paid"&gt;Paid&lt;/SelectItem&gt;
                &lt;SelectItem value="overdue"&gt;Overdue&lt;/SelectItem&gt;
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
            {ms.payment_date &amp;&amp; (
              &lt;span className="text-xs text-muted-foreground"&gt;{format(new Date(ms.payment_date), "MMM d, yyyy")}&lt;/span&gt;
```

```
                )}
              &lt;/div&gt;
            ))}
          &lt;/div&gt;
        &lt;/div&gt;
      )}
    &lt;/CardContent&gt;
  &lt;/Card&gt;

  {/* Invoices table */}
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;CardTitle className="text-base"&gt;Payment Records&lt;/CardTitle&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent className="p-0"&gt;
      &lt;Table&gt;
        &lt;TableHeader&gt;
          &lt;TableRow&gt;
            &lt;TableHead&gt;Type&lt;/TableHead&gt;
            &lt;TableHead&gt;Total&lt;/TableHead&gt;
            &lt;TableHead&gt;Paid&lt;/TableHead&gt;
            &lt;TableHead&gt;Net Received&lt;/TableHead&gt;
            &lt;TableHead&gt;Method&lt;/TableHead&gt;
            &lt;TableHead&gt;Status&lt;/TableHead&gt;
            &lt;TableHead&gt;Date&lt;/TableHead&gt;
          &lt;/TableRow&gt;
        &lt;/TableHeader&gt;
        &lt;TableBody&gt;
          {payments.length === 0 &amp;&amp; (
            &lt;TableRow&gt;&lt;TableCell colSpan={7} className="text-center text-muted-foreground py-8"&gt;No payments recorded&lt;/TableCell&gt;&lt;/TableRow&gt;
          )}
          {payments.map(p =&gt; (
            &lt;TableRow key={p.id}&gt;
              &lt;TableCell className="font-medium"&gt;{p.installment_type}&lt;/TableCell&gt;
              &lt;TableCell&gt;{fmtVal(Number(p.amount_total))}&lt;/TableCell&gt;
              &lt;TableCell className="text-chart-2"&gt;{fmtVal(Number(p.amount_paid))}&lt;/TableCell&gt;
              &lt;TableCell className="text-muted-foreground"&gt;{p.net_received != null ? fmtVal(Number(p.net_received)) : "—"}&lt;/TableCell&gt;
              &lt;TableCell className="text-muted-foreground text-xs"&gt;{p.payment_method || "—"}&lt;/TableCell&gt;
              &lt;TableCell&gt;
                &lt;Select
                  value={p.status}
                  onValueChange={async (newStatus) =&gt; {
                    const updatePayload: any = { status: newStatus };
                    if (newStatus === "paid" &amp;&amp; !p.payment_date) {
                      updatePayload.payment_date = new Date().toISOString().split("T")[0];
                      updatePayload.amount_paid = Number(p.amount_total);
                    }
                    if (newStatus === "pending") {
                      updatePayload.amount_paid = 0;
                    }
                    const { error } = await supabase.from("admin_payments").update(updatePayload).eq("id", p.id);
                    if (error) { toast.error(error.message); return; }
                    // Recalculate project payment_stage
                    const { data: allPay } = await supabase.from("admin_payments").select("amount_paid, status").eq("project_id", id);
                    const newTotalPaid = (allPay || []).filter((pp: any) =&gt; pp.status === "paid").reduce((s: number, pp: any) =&gt; s + Number(pp.amount_paid ||
                    const projTotal = Number(form.total_value || 0);
                    const projEntry = projTotal * Number(form.entry_payment_percent || 50) / 100;
                    let newPayStage = "Not Sent";
                    if (newTotalPaid &gt;= projTotal &amp;&amp; projTotal &gt; 0) newPayStage = "Paid";
                    else if (newTotalPaid &gt;= projEntry &amp;&amp; projEntry &gt; 0) newPayStage = "Entry Paid";
                    else if (newTotalPaid &gt; 0) newPayStage = "Waiting Final";
                    await supabase.from("projects").update({ payment_stage: newPayStage }).eq("id", id);
                    toast.success(`Payment marked as ${newStatus}`);
                    loadData();
                  }}
                &gt;
                  &lt;SelectTrigger className={`h-7 w-[110px] text-xs border ${paymentStatusColor(p.status)}`}&gt;
                    &lt;SelectValue /&gt;
                  &lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;
                    &lt;SelectItem value="pending"&gt;Pending&lt;/SelectItem&gt;
                    &lt;SelectItem value="paid"&gt;Paid&lt;/SelectItem&gt;
                    &lt;SelectItem value="cancelled"&gt;Cancelled&lt;/SelectItem&gt;
                  &lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/TableCell&gt;
              &lt;TableCell className="text-xs text-muted-foreground"&gt;{p.payment_date ? format(new Date(p.payment_date), "MMM d, yyyy") : "—"}&lt;/TableCell&gt;
            &lt;/TableRow&gt;
          ))}
        &lt;/TableBody&gt;
      &lt;/Table&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;
&lt;/TabsContent&gt;

{/* ============ NOTES ============ */}
&lt;TabsContent value="notes"&gt;
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;CardTitle className="text-base flex items-center gap-2"&gt;
        &lt;StickyNote className="h-4 w-4" /&gt; Internal Notes
      &lt;/CardTitle&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent&gt;
      &lt;Textarea
        value={internalNotes}
        onChange={e =&gt; { setInternalNotes(e.target.value); setHasChanges(true); }}
        placeholder="Write internal notes about this project..."
        className="min-h-[200px]"
      /&gt;
```

```
            &lt;/CardContent&gt;
          &lt;/Card&gt;
        &lt;/TabsContent&gt;

        {/* ============ CLIENT PORTAL ============ */}
        &lt;TabsContent value="client-portal"&gt;
          &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
            {/* Messages */}
            &lt;Card className="border-none shadow-sm"&gt;
              &lt;CardHeader className="pb-3"&gt;
                &lt;CardTitle className="text-base flex items-center gap-2"&gt;
                  &lt;MessageSquare className="h-4 w-4" /&gt; Client Messages ({messages.length})
                &lt;/CardTitle&gt;
              &lt;/CardHeader&gt;
              &lt;CardContent className="p-4"&gt;
                &lt;ScrollArea className="max-h-[400px]"&gt;
                  {messages.length === 0 &amp;amp;&amp; &lt;p className="text-center text-muted-foreground py-8"&gt;No messages yet&lt;/p&gt;}
                  &lt;div className="space-y-3"&gt;
                    {messages.map(m =&gt; (
                      &lt;div key={m.id} className={`flex ${m.is_from_admin ? "justify-end" : "justify-start"}`}&gt;
                        &lt;div className={`max-w-[80%] p-3 rounded-lg text-sm ${m.is_from_admin ? "bg-primary text-primary-foreground" : "bg-muted"}`}&gt;
                          &lt;p&gt;{m.content}&lt;/p&gt;
                          &lt;p className={`text-[10px] mt-1 ${m.is_from_admin ? "text-primary-foreground/70" : "text-muted-foreground"}`}&gt;
                            {format(new Date(m.created_at), "MMM d, h:mm a")}
                          &lt;/p&gt;
                        &lt;/div&gt;
                      &lt;/div&gt;
                    ))}
                  &lt;/div&gt;
                &lt;/ScrollArea&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;

            {/* Tickets */}
            &lt;Card className="border-none shadow-sm"&gt;
              &lt;CardHeader className="pb-3"&gt;
                &lt;CardTitle className="text-base flex items-center gap-2"&gt;
                  &lt;Ticket className="h-4 w-4" /&gt; Support Tickets ({tickets.length})
                &lt;/CardTitle&gt;
              &lt;/CardHeader&gt;
              &lt;CardContent&gt;
                {tickets.length === 0 ? (
                  &lt;p className="text-center text-muted-foreground py-8"&gt;No tickets&lt;/p&gt;
                ) : (
                  &lt;div className="space-y-2"&gt;
                    {tickets.map(t =&gt; (
                      &lt;div
                        key={t.id}
                        className="flex items-center gap-3 p-3 rounded-lg border hover:bg-muted/30 cursor-pointer transition-colors"
                        onClick={() =&gt; navigate(`/adm/tickets?id=${t.id}`)}
                      &gt;
                        &lt;Ticket className="h-4 w-4 text-muted-foreground shrink-0" /&gt;
                        &lt;div className="flex-1 min-w-0"&gt;
                          &lt;p className="text-sm font-medium truncate"&gt;{t.subject}&lt;/p&gt;
                          &lt;p class="text-xs text-muted-foreground"&gt;{format(new Date(t.updated_at), "MMM d, yyyy")}&lt;/p&gt;
                        &lt;/div&gt;
                        &lt;Badge variant={t.status === "open" ? "destructive" : "outline"} className="text-[10px]"&gt;{t.status}&lt;/Badge&gt;
                      &lt;/div&gt;
                    ))}
                  &lt;/div&gt;
                )}
              &lt;/CardContent&gt;
            &lt;/Card&gt;

            {/* Client-visible files */}
            &lt;Card className="border-none shadow-sm lg:col-span-2"&gt;
              &lt;CardHeader className="pb-3"&gt;
                &lt;CardTitle className="text-base flex items-center gap-2"&gt;
                  &lt;FolderOpen className="h-4 w-4" /&gt; Files Shared with Client
                &lt;/CardTitle&gt;
              &lt;/CardHeader&gt;
              &lt;CardContent&gt;
                {files.length === 0 ? (
                  &lt;p className="text-center text-muted-foreground py-8"&gt;No files shared&lt;/p&gt;
                ) : (
                  &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-3"&gt;
                    {files.slice(0, 6).map(f =&gt; (
                      &lt;div key={f.id} className="flex items-center gap-3 p-3 rounded-lg border bg-muted/30"&gt;
                        &lt;FolderOpen className="h-4 w-4 text-muted-foreground" /&gt;
                        &lt;div class="flex-1 min-w-0"&gt;
                          &lt;p class="text-sm font-medium truncate"&gt;{f.file_name}&lt;/p&gt;
                          &lt;p class="text-xs text-muted-foreground"&gt;{format(new Date(f.created_at), "MMM d, yyyy")}&lt;/p&gt;
                        &lt;/div&gt;
                        &lt;Button variant="ghost" size="icon" class="h-7 w-7" asChild&gt;
                          &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download class="h-3.5 w-3.5" /&gt;&lt;/a&gt;
                        &lt;/Button&gt;
                      &lt;/div&gt;
                    ))}
                  &lt;/div&gt;
                )}
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          &lt;/div&gt;
        &lt;/TabsContent&gt;
      &lt;/Tabs&gt;
    &lt;/div&gt;
  );
};

export default AdminProjectDetail;
```

src/pages/admin/AdminProjects.tsx

```
import { useEffect, useState, useRef } from "react";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Checkbox } from "@/components/ui/checkbox";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { Plus, Pencil, Search, Copy, Camera, FolderOpen, Trash2, Share2, Lock, Users, Eye, ArrowUpDown, ArrowUp, ArrowDown, ChevronDown, ChevronRight, Archive } from "lucide
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { toast } from "sonner";
import { Separator } from "@/components/ui/separator";
import { useLanguage } from "@/hooks/useLanguage";

interface Company {
  id: string;
  name: string;
  street_name?: string | null;
  street_number?: string | null;
  unit?: string | null;
  neighborhood?: string | null;
  city?: string | null;
  state?: string | null;
  zip_code?: string | null;
  country?: string | null;
}

interface Service { id: string; name: string; slug: string; category?: string | null; }
interface Package { id: string; name: string; description: string | null; }

interface DateNote {
  date: string;
  note: string;
  visibility: "freelancer" | "private" | "client";
}

import { PROJECT_STAGES, PROJECT_STAGE_COLORS as SHARED_STAGE_COLORS, getStageLabel, COMPLETED_STAGES, isActiveStage } from "@/constants/projectStages";

const SERVICE_CATEGORIES_ORDER = ["Design Services", "Drafting &amp; Technical Documentation", "Visualization"];

const PAYMENT_STAGES = [
  "Not Sent", "Waiting Entry", "Entry Paid",
  "Waiting Final", "Paid", "Overdue", "Refunded",
];

const COUNTRIES = ["USA", "Brazil", "Other"];
const CURRENCIES = ["USD", "BRL"];

const emptyForm = {
  company_id: "", stage: "WAITING_CLIENT_INFO", payment_stage: "Not Sent",
  start_date: "", delivery_date: "", total_value: "",
  street_name: "", street_number: "", unit: "", neighborhood: "",
  city: "", state: "Massachusetts", zip_code: "", country: "USA",
  latitude: "", longitude: "",
  payment_effective_date: "", preview_delivery_date: "",
  preview_sent_date: "", revision_return_date: "", final_delivery_date: "",
  revision_count: "0", currency: "USD", entry_payment_percent: "50",
  package_id: "", square_feet: "", service_type: "",
};

const PROJECT_STAGE_COLORS = SHARED_STAGE_COLORS;

const PAYMENT_STAGE_COLORS: Record&lt;string, { dot: string; bg: string; text: string; border: string }&gt; = {
  "Not Sent": { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" },
  "Waiting Entry": { dot: "bg-chart-3", bg: "bg-chart-3/15", text: "text-chart-3", border: "border-chart-3/30" },
  "Entry Paid": { dot: "bg-chart-4", bg: "bg-chart-4/15", text: "text-chart-4", border: "border-chart-4/30" },
  "Waiting Final": { dot: "bg-chart-3", bg: "bg-chart-3/15", text: "text-chart-3", border: "border-chart-3/30" },
  "Paid": { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
  "Overdue": { dot: "bg-destructive", bg: "bg-destructive/15", text: "text-destructive", border: "border-destructive/30" },
  "Refunded": { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" },
};

const VISIBILITY_OPTIONS = [
  { value: "private", label: "Privado", icon: Lock, color: "text-muted-foreground" },
  { value: "freelancer", label: "Freelancer", icon: Users, color: "text-chart-4" },
  { value: "client", label: "Área do Cliente", icon: Eye, color: "text-chart-2" },
];

const AdminProjects = () =&gt; {
  const navigate = useNavigate();
  const { t } = useLanguage();
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [companies, setCompanies] = useState&lt;Company[]&gt;([]);
  const [services, setServices] = useState&lt;Service[]&gt;([]);
  const [packages, setPackages] = useState&lt;Package[]&gt;([]);
  const [projectServices, setProjectServices] = useState&lt;Record&lt;string, string[]&gt;&gt;({});
  const [search, setSearch] = useState("");
  const [stageFilter, setStageFilter] = useState("all");
  const [companyFilter, setCompanyFilter] = useState("all");
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editing, setEditing] = useState&lt;any&gt;(null);
  const [form, setForm] = useState(emptyForm);
```

```
const [selectedServices, setSelectedServices] = useState&lt;string[]&gt;([]);
const [payments, setPayments] = useState&lt;Record&lt;string, number&gt;&gt;({});
const [milestoneBudgets, setMilestoneBudgets] = useState&lt;Record&lt;string, number&gt;&gt;({});
const [coverFile, setCoverFile] = useState&lt;File | null&gt;(null);
const [coverPreview, setCoverPreview] = useState&lt;string | null&gt;(null);
const coverInputRef = useRef&lt;HTMLInputElement&gt;(null);
const [dateNotes, setDateNotes] = useState&lt;DateNote[]&gt;([]);
const [inlineEditId, setInlineEditId] = useState&lt;string | null&gt;(null);
const [inlineEditField, setInlineEditField] = useState&lt;"stage" | "payment_stage" | null&gt;(null);
const [countryStep, setCountryStep] = useState&lt;boolean&gt;(false);
const [sortColumn, setSortColumn] = useState&lt;string | null&gt;(null);
const [sortDir, setSortDir] = useState&lt;"asc" | "desc"&gt;("asc");
const [completedOpen, setCompletedOpen] = useState(false);

const toggleSort = (col: string) =&gt; {
  if (sortColumn === col) {
    setSortDir(d =&gt; d === "asc" ? "desc" : "asc");
  } else {
    setSortColumn(col);
    setSortDir("asc");
  }
};

const SortIcon = ({ col }: { col: string }) =&gt; {
  if (sortColumn !== col) return &lt;ArrowUpDown className="h-3 w-3 text-muted-foreground/40" /&gt;;
  return sortDir === "asc"
    ? &lt;ArrowUp className="h-3 w-3 text-primary" /&gt;
    : &lt;ArrowDown className="h-3 w-3 text-primary" /&gt;;
};

const isBrazil = form.country === "Brazil";
const areaLabel = isBrazil ? "Área (m²)" : "Metragem (sq ft)";
const areaUnit = isBrazil ? "m²" : "ft²";
const areaPlaceholder = isBrazil ? "Ex: 120" : "Ex: 720";

const handleInlineSave = async (projectId: string, field: "stage" | "payment_stage", value: string) =&gt; {
  const { error } = await supabase.from("projects").update({ [field]: value }).eq("id", projectId);
  if (error) { toast.error(error.message); return; }
  toast.success("Atualizado!");
  setInlineEditId(null);
  setInlineEditField(null);
  fetchData();
};

const uploadCoverImage = async (projectId: string): Promise&lt;string | null&gt; =&gt; {
  if (!coverFile) return editing?.cover_image || null;
  const ext = coverFile.name.split(".").pop();
  const path = `${projectId}.${ext}`;
  const { error } = await supabase.storage.from("portfolio").upload(path, coverFile, { upsert: true });
  if (error) { toast.error("Erro ao enviar imagem"); return null; }
  const { data } = supabase.storage.from("portfolio").getPublicUrl(path);
  return `${data.publicUrl}?t=${Date.now()}`;
};

const handleCoverChange = (e: React.ChangeEvent&lt;HTMLInputElement&gt;) =&gt; {
  const file = e.target.files?.[0];
  if (!file) return;
  setCoverFile(file);
  setCoverPreview(URL.createObjectURL(file));
};

const fetchData = async () =&gt; {
  const [projRes, compRes, svcRes, pkgRes, psRes, payRes] = await Promise.all([
    supabase.from("projects").select("*, companies(name)").order("created_at", { ascending: false }),
    supabase.from("companies").select("id, name, street_name, street_number, unit, neighborhood, city, state, zip_code, country").order("name"),
    supabase.from("services").select("*").order("name"),
    supabase.from("packages").select("*"),
    supabase.from("project_services").select("project_id, service_id"),
    supabase.from("payment_milestones").select("project_id, amount, status"),
  ]);
  setProjects(projRes.data || []);
  setCompanies(compRes.data || []);
  setServices(svcRes.data || []);
  setPackages(pkgRes.data || []);

  const psMap: Record&lt;string, string[]&gt; = {};
  (psRes.data || []).forEach((ps: any) =&gt; {
    if (!psMap[ps.project_id]) psMap[ps.project_id] = [];
    psMap[ps.project_id].push(ps.service_id);
  });
  setProjectServices(psMap);

  // Use payment_milestones as source of truth
  const payMap: Record&lt;string, number&gt; = {};
  const budgetMap: Record&lt;string, number&gt; = {};
  (payRes.data || []).forEach((m: any) =&gt; {
    budgetMap[m.project_id] = (budgetMap[m.project_id] || 0) + Number(m.amount || 0);
    if (m.status === "paid") {
      payMap[m.project_id] = (payMap[m.project_id] || 0) + Number(m.amount || 0);
    }
  });
  setPayments(payMap);
  setMilestoneBudgets(budgetMap);
};

useEffect(() =&gt; { fetchData(); }, []);

const copyFromCompany = () =&gt; {
  const comp = companies.find(c =&gt; c.id === form.company_id);
  if (!comp) { toast.error(t("admin.selectCompanyFirst")); return; }
  setForm(f =&gt; ({
```

```
    ...f,
    street_name: comp.street_name || "", street_number: comp.street_number || "",
    unit: comp.unit || "", neighborhood: comp.neighborhood || "",
    city: comp.city || "", state: comp.state || "Massachusetts",
    zip_code: comp.zip_code || "", country: comp.country || "USA",
  }));
  toast.success(t("admin.addressCopied"));
};

const handleSave = async () =&gt; {
  if (!form.company_id) {
    toast.error("Please select a company/client before saving.");
    return;
  }
  const payload: any = {
    company_id: form.company_id,
    service_type: form.service_type?.trim() || "Technical Drafting",
    square_feet: form.square_feet ? Number(form.square_feet) : null,
    stage: form.stage,
    payment_stage: form.payment_stage,
    start_date: form.start_date || null,
    delivery_date: form.delivery_date || null,
    total_value: form.total_value ? Number(form.total_value) : 0,
    street_name: form.street_name || null, street_number: form.street_number || null,
    unit: form.unit || null, neighborhood: form.neighborhood || null,
    city: form.city || null, state: form.state || null,
    zip_code: form.zip_code || null, country: form.country || null,
    latitude: form.latitude ? Number(form.latitude) : null,
    longitude: form.longitude ? Number(form.longitude) : null,
    payment_effective_date: form.payment_effective_date || null,
    preview_delivery_date: form.preview_delivery_date || null,
    preview_sent_date: form.preview_sent_date || null,
    revision_return_date: form.revision_return_date || null,
    final_delivery_date: form.final_delivery_date || null,
    revision_count: Number(form.revision_count) || 0,
    currency: form.currency,
    entry_payment_percent: Number(form.entry_payment_percent) || 50,
    package_id: form.package_id &amp;amp;&amp; form.package_id !== "none" ? form.package_id : null,
    date_notes: dateNotes,
  };

  let projectId = editing?.id;
  if (editing) {
    const coverUrl = await uploadCoverImage(editing.id);
    if (coverUrl) payload.cover_image = coverUrl;
    const { error } = await supabase.from("projects").update(payload).eq("id", editing.id);
    if (error) { toast.error(error.message); return; }
  } else {
    const { data, error } = await supabase.from("projects").insert(payload).select("id").single();
    if (error) { toast.error(error.message); return; }
    projectId = data.id;
    if (coverFile) {
      const coverUrl = await uploadCoverImage(projectId);
      if (coverUrl) await supabase.from("projects").update({ cover_image: coverUrl }).eq("id", projectId);
    }
  }

  await supabase.from("project_services").delete().eq("project_id", projectId);
  if (selectedServices.length &gt; 0) {
    await supabase.from("project_services").insert(
      selectedServices.map(sid =&gt; ({ project_id: projectId, service_id: sid }))
    );
  }

  toast.success(editing ? t("admin.projectUpdated") : t("admin.projectCreated"));
  setDialogOpen(false); setEditing(null); setCoverFile(null); setCoverPreview(null); fetchData();
};

const handleDeleteProject = async (id: string) =&gt; {
  if (!confirm("Tem certeza que deseja excluir este projeto?")) return;
  // Delete related records first
  await supabase.from("project_services").delete().eq("project_id", id);
  await supabase.from("activity_log").delete().eq("project_id", id);
  await supabase.from("admin_payments").delete().eq("project_id", id);
  await supabase.from("milestones").delete().eq("project_id", id);
  await supabase.from("project_files").delete().eq("project_id", id);
  await supabase.from("messages").delete().eq("project_id", id);
  await supabase.from("freelancer_assignments").delete().eq("project_id", id);
  await supabase.from("freelancer_contracts").delete().eq("project_id", id);
  await supabase.from("freelancer_deliveries").delete().eq("project_id", id);
  await supabase.from("freelancer_payments").delete().eq("project_id", id);
  await supabase.from("quotes").delete().eq("project_id", id);
  const { error } = await supabase.from("projects").delete().eq("id", id);
  if (error) { toast.error(error.message); return; }
  toast.success("Projeto excluído");
  fetchData();
};

const handleDuplicateProject = async (p: any) =&gt; {
  const payload: any = {
    company_id: p.company_id,
    service_type: p.service_type || "Technical Drafting",
    square_feet: p.square_feet,
    stage: "Lead",
    payment_stage: "Not Sent",
    start_date: null,
    delivery_date: null,
    total_value: p.total_value || 0,
    street_name: p.street_name,
    street_number: p.street_number,
```

```
    unit: p.unit,
    neighborhood: p.neighborhood,
    city: p.city,
    state: p.state,
    zip_code: p.zip_code,
    country: p.country,
    currency: p.currency || "USD",
    entry_payment_percent: p.entry_payment_percent || 50,
    package_id: p.package_id || null,
    revision_count: 0,
  };
  const { data, error } = await supabase.from("projects").insert(payload).select("id").single();
  if (error) { toast.error(error.message); return; }
  // Copy project services
  const existingServices = projectServices[p.id] || [];
  if (existingServices.length &gt; 0) {
    await supabase.from("project_services").insert(
      existingServices.map(sid =&gt; ({ project_id: data.id, service_id: sid }))
    );
  }
  toast.success("Projeto duplicado!");
  fetchData();
};

const openEdit = (p: any) =&gt; {
  setEditing(p);
  setCoverPreview(p.cover_image || null);
  setCoverFile(null);
  setForm({
    company_id: p.company_id || "", stage: p.stage || "Lead",
    payment_stage: p.payment_stage || "Not Sent",
    start_date: p.start_date || "", delivery_date: p.delivery_date || "",
    total_value: p.total_value?.toString() || "",
    street_name: p.street_name || "", street_number: p.street_number || "",
    unit: p.unit || "", neighborhood: p.neighborhood || "",
    city: p.city || "", state: p.state || "Massachusetts",
    zip_code: p.zip_code || "", country: p.country || "USA",
    latitude: p.latitude?.toString() || "", longitude: p.longitude?.toString() || "",
    payment_effective_date: p.payment_effective_date || "",
    preview_delivery_date: p.preview_delivery_date || "",
    preview_sent_date: p.preview_sent_date || "",
    revision_return_date: p.revision_return_date || "",
    final_delivery_date: p.final_delivery_date || "",
    revision_count: p.revision_count?.toString() || "0",
    currency: p.currency || "USD",
    entry_payment_percent: p.entry_payment_percent?.toString() || "50",
    package_id: p.package_id || "",
    square_feet: p.square_feet?.toString() || "",
    service_type: p.service_type || "Technical Drafting",
  });
  setSelectedServices(projectServices[p.id] || []);
  // Load date_notes from project
  const notes = Array.isArray(p.date_notes) ? p.date_notes as DateNote[] : [];
  setDateNotes(notes);
  setCountryStep(false);
  setDialogOpen(true);
};

const openNew = () =&gt; {
  setEditing(null);
  setForm({ ...emptyForm, company_id: companies[0]?.id || "" });
  setSelectedServices([]);
  setCoverFile(null); setCoverPreview(null);
  setDateNotes([]);
  setCountryStep(true);
  setDialogOpen(true);
};

const selectCountryForNew = (country: "Brazil" | "USA") =&gt; {
  if (country === "Brazil") {
    setForm(f =&gt; ({ ...f, country: "Brazil", currency: "BRL", state: "" }));
  } else {
    setForm(f =&gt; ({ ...f, country: "USA", currency: "USD", state: "Massachusetts" }));
  }
  setCountryStep(false);
};

const addDateNote = () =&gt; {
  setDateNotes(prev =&gt; [...prev, { date: "", note: "", visibility: "private" }]);
};

const updateDateNote = (index: number, field: keyof DateNote, value: string) =&gt; {
  setDateNotes(prev =&gt; prev.map((n, i) =&gt; i === index ? { ...n, [field]: value } : n));
};

const removeDateNote = (index: number) =&gt; {
  setDateNotes(prev =&gt; prev.filter((_, i) =&gt; i !== index));
};

const totalValue = Number(form.total_value) || 0;
const entryPercent = Number(form.entry_payment_percent) || 50;
const entryAmount = totalValue * entryPercent / 100;
const finalAmount = totalValue - entryAmount;

const filtered = projects
  .filter(p =&gt; {
    const addr = [p.street_number, p.street_name].filter(Boolean).join(" ") || p.display_address || "";
    const name = p.name || "";
    const projectNum = p.project_number || "";
    const searchLower = search.toLowerCase();
    return name.toLowerCase().includes(searchLower)
```

```
      || addr.toLowerCase().includes(searchLower)
      || (p.companies?.name || "").toLowerCase().includes(searchLower)
      || projectNum.toLowerCase().includes(searchLower);
  })
  .filter(p =&gt; stageFilter === "all" || p.stage === stageFilter)
  .filter(p =&gt; companyFilter === "all" || p.company_id === companyFilter);

const sorted = [...filtered].sort((a, b) =&gt; {
  if (!sortColumn) return 0;
  const dir = sortDir === "asc" ? 1 : -1;
  switch (sortColumn) {
    case "project": return dir * getProjectDisplayName(a).localeCompare(getProjectDisplayName(b));
    case "client": return dir * ((a.companies?.name || "").localeCompare(b.companies?.name || ""));
    case "stage": return dir * ((a.stage || "").localeCompare(b.stage || ""));
    case "deadline": {
      const da = a.delivery_date ? new Date(a.delivery_date).getTime() : Infinity;
      const db = b.delivery_date ? new Date(b.delivery_date).getTime() : Infinity;
      return dir * (da - db);
    }
    case "budget": return dir * (Number(a.total_value || 0) - Number(b.total_value || 0));
    default: return 0;
  }
});

const getProjectDisplayName = (p: any) =&gt; {
  const addr = [p.street_number, p.street_name].filter(Boolean).join(" ");
  return addr || p.name || p.display_address || "Untitled Project";
};

const getFinancialHealth = (p: any): { icon: string; label: string; color: string } =&gt; {
  const budget = milestoneBudgets[p.id] || Number(p.total_value || 0);
  if (budget === 0) return { icon: "■", label: "No Budget", color: "text-muted-foreground" };
  const paid = payments[p.id] || 0;
  const rem = budget - paid;
  if (rem &lt;= 0) return { icon: "■", label: "Paid", color: "text-chart-2" };
  if (paid &gt; 0) return { icon: "■", label: "Partial", color: "text-chart-3" };
  if (p.delivery_date &amp;&amp; new Date(p.delivery_date) &lt; new Date()) return { icon: "■", label: "Overdue", color: "text-destructive" };
  return { icon: "■", label: "Unpaid", color: "text-destructive" };
};

const handleArchiveProject = async (id: string) =&gt; {
  const { error } = await supabase.from("projects").update({ stage: "CANCELED" }).eq("id", id);
  if (error) { toast.error(error.message); return; }
  toast.success("Project archived");
  fetchData();
};

// Split projects into 3 groups
const activeProjects = sorted.filter(p =&gt; isActiveStage(p.stage || ""));
const awaitingPayment = sorted.filter(p =&gt; {
  if (!COMPLETED_STAGES.includes(p.stage as any)) return false;
  const budget = milestoneBudgets[p.id] || Number(p.total_value || 0);
  const paid = payments[p.id] || 0;
  return budget &gt; 0 &amp;&amp; (budget - paid) &gt; 0;
});
const completedProjects = sorted.filter(p =&gt; {
  if (!COMPLETED_STAGES.includes(p.stage as any)) return false;
  const budget = milestoneBudgets[p.id] || Number(p.total_value || 0);
  const paid = payments[p.id] || 0;
  return budget === 0 || (budget - paid) &lt;= 0;
});

const currencySymbol = (c: string) =&gt; c === "BRL" ? "R$" : "$";
const getCountryFlag = (country: string | null | undefined) =&gt; {
  if (country === "Brazil") return "■■";
  if (country === "USA") return "■■";
  return "";
};

const renderStageBadge = (stage: string, colors: Record&lt;string, { dot: string; bg: string; text: string; border: string }&gt;) =&gt; {
  const c = colors[stage] || { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" };
  return (
    &lt;span className={`inline-flex items-center gap-1.5 px-2 py-0.5 rounded-full text-xs font-medium border ${c.bg} ${c.text} ${c.border}`}&gt;
      &lt;span className={`w-2 h-2 rounded-full ${c.dot}`} /&gt;
      {getStageLabel(stage) || "—"}
    &lt;/span&gt;
  );
};

const renderProjectRow = (p: any, showArchive = false) =&gt; {
  const tv = Number(p.total_value || 0);
  const isBRL = (p.currency || "USD") === "BRL";
  const formattedBudget = isBRL
    ? `R$${tv.toLocaleString("pt-BR", { minimumFractionDigits: 0 })}`
    : `$${tv.toLocaleString("en-US", { minimumFractionDigits: 0 })}`;
  const isInlineStage = inlineEditId === p.id &amp;&amp; inlineEditField === "stage";
  const isOverdue = p.delivery_date &amp;&amp; new Date(p.delivery_date) &lt; new Date() &amp;&amp; p.stage !== "Completed";
  const health = getFinancialHealth(p);

  return (
    &lt;TableRow key={p.id} className="group"&gt;
      &lt;TableCell&gt;
        &lt;div className="flex items-center gap-3"&gt;
          &lt;span className="text-sm" title={health.label}&gt;{health.icon}&lt;/span&gt;
          &lt;div className="relative group/avatar"&gt;
            &lt;Avatar className="h-9 w-9 border border-border flex-shrink-0"&gt;
              &lt;AvatarImage src={p.cover_image || undefined} className="object-cover" /&gt;
              &lt;AvatarFallback className="bg-muted text-muted-foreground text-xs"&gt;
                {getProjectDisplayName(p).substring(0, 2).toUpperCase()}
              &lt;/AvatarFallback&gt;
            &lt;/Avatar&gt;
```

```
      &lt;button
        onClick={(e) =&gt; {
          e.stopPropagation();
          const input = document.createElement("input");
          input.type = "file";
          input.accept = "image/*";
          input.onchange = async (ev) =&gt; {
            const file = (ev.target as HTMLInputElement).files?.[0];
            if (!file) return;
            const ext = file.name.split(".").pop();
            const path = `${p.id}.${ext}`;
            const { error: uploadError } = await supabase.storage.from("portfolio").upload(path, file, { upsert: true });
            if (uploadError) { toast.error(uploadError.message); return; }
            const { data: urlData } = supabase.storage.from("portfolio").getPublicUrl(path);
            const coverUrl = urlData.publicUrl + "?t=" + Date.now();
            await supabase.from("projects").update({ cover_image: coverUrl }).eq("id", p.id);
            toast.success("Cover updated");
            fetchData();
          };
          input.click();
        }}
        className="absolute inset-0 flex items-center justify-center rounded-full bg-foreground/50 opacity-0 group-hover/avatar:opacity-100 transition-opacity"
      &gt;
        &lt;Pencil className="h-3 w-3 text-background" /&gt;
      &lt;/button&gt;
    &lt;/div&gt;
    &lt;div className="min-w-0"&gt;
      &lt;div className="flex items-center gap-2"&gt;
        {p.project_number &amp;&amp; (
          &lt;span className="text-[10px] font-mono font-semibold text-muted-foreground bg-muted px-1.5 py-0.5 rounded border border-border"&gt;
            {p.project_number}
          &lt;/span&gt;
        )}
        &lt;p className="text-sm font-medium text-foreground truncate hover:text-primary cursor-pointer" onClick={() =&gt; navigate(`/adm/projects/${p.id}`)}&gt;
          {getProjectDisplayName(p)}
        &lt;/p&gt;
      &lt;/div&gt;
      &lt;p className="text-xs text-muted-foreground truncate"&gt;
        {p.service_type || "—"} {p.city ? `· ${p.city}, ${p.state || ""}` : ""}
      &lt;/p&gt;
    &lt;/div&gt;
  &lt;/div&gt;
&lt;/TableCell&gt;
&lt;TableCell&gt;
  &lt;p className="text-sm text-foreground hover:text-primary cursor-pointer truncate" onClick={() =&gt; p.company_id &amp;&amp; navigate(`/adm/clients/${p.company_i
    {p.companies?.name || "—"} {getCountryFlag(p.country)}
  &lt;/p&gt;
&lt;/TableCell&gt;
&lt;TableCell&gt;
  {isInlineStage ? (
    &lt;Select defaultValue={p.stage} onValueChange={v =&gt; handleInlineSave(p.id, "stage", v)}&gt;
      &lt;SelectTrigger className="h-8 text-xs w-[150px]"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
      &lt;SelectContent&gt;{PROJECT_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{getStageLabel(s)}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
    &lt;/Select&gt;
  ) : (
    &lt;div className="flex items-center gap-1"&gt;
      {renderStageBadge(p.stage || "—", PROJECT_STAGE_COLORS)}
      &lt;Button variant="ghost" size="icon" className="h-6 w-6 opacity-0 group-hover:opacity-100" onClick={() =&gt; { setInlineEditId(p.id); setInlineEditField("sta
        &lt;Pencil className="h-3 w-3 text-muted-foreground" /&gt;
      &lt;/Button&gt;
    &lt;/div&gt;
  )}
&lt;/TableCell&gt;
&lt;TableCell&gt;
  {p.delivery_date ? (
    &lt;div className="flex items-center gap-1.5"&gt;
      &lt;span className={`text-sm ${isOverdue ? "text-destructive font-medium" : "text-muted-foreground"}`}&gt;
        {new Date(p.delivery_date).toLocaleDateString("en-US", { month: "short", day: "numeric" })}
      &lt;/span&gt;
      {isOverdue &amp;&amp; (
        &lt;span className="inline-flex items-center gap-0.5 px-1.5 py-0.5 rounded text-[10px] font-medium bg-destructive/10 text-destructive border border-destructi
      )}
    &lt;/div&gt;
  ) : (
    &lt;span className="text-sm text-muted-foreground"&gt;—&lt;/span&gt;
  )}
&lt;/TableCell&gt;
&lt;TableCell&gt;
  &lt;div&gt;
    &lt;span className="text-sm font-medium text-foreground"&gt;{formattedBudget}&lt;/span&gt;
    {tv &gt; 0 &amp;&amp; (() =&gt; {
      const paid = payments[p.id] || 0;
      const rem = Math.max(0, tv - paid);
      return (
        &lt;div className="flex items-center gap-2 mt-0.5"&gt;
          &lt;span className="text-[10px] text-muted-foreground"&gt;
            Rem: {isBRL ? `R$${rem.toLocaleString("pt-BR")}` : `$${rem.toLocaleString("en-US")}`}
          &lt;/span&gt;
        &lt;/div&gt;
      );
    })()}
  &lt;/div&gt;
&lt;/TableCell&gt;
&lt;TableCell&gt;
  &lt;div className="flex gap-1 opacity-0 group-hover:opacity-100 transition-opacity"&gt;
    &lt;Button variant="ghost" size="icon" onClick={() =&gt; navigate(`/adm/projects/${p.id}`)} title="Edit"&gt;&lt;Pencil className="h-4 w-4" /&gt;&lt;/Button&gt;
    &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleDuplicateProject(p)} title="Duplicate"&gt;&lt;Copy className="h-4 w-4" /&gt;&lt;/Button&gt;
    {showArchive &amp;&amp; (
      &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleArchiveProject(p.id)} title="Archive"&gt;&lt;Archive className="h-4 w-4 text-muted-foreground" /
    )}
```

```
          &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleDeleteProject(p.id)} title="Delete"&gt;&lt;Trash2 className="h-4 w-4 text-destructive" /&gt;&lt;/B
        &lt;/div&gt;
      &lt;/TableCell&gt;
    &lt;/TableRow&gt;
  );
};

const renderTableHeader = () =&gt; (
  &lt;TableHeader&gt;
    &lt;TableRow&gt;
      &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("project")}&gt;
        &lt;span className="flex items-center gap-1"&gt;Project &lt;SortIcon col="project" /&gt;&lt;/span&gt;
      &lt;/TableHead&gt;
      &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("client")}&gt;
        &lt;span className="flex items-center gap-1"&gt;Client &lt;SortIcon col="client" /&gt;&lt;/span&gt;
      &lt;/TableHead&gt;
      &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("stage")}&gt;
        &lt;span className="flex items-center gap-1"&gt;Stage &lt;SortIcon col="stage" /&gt;&lt;/span&gt;
      &lt;/TableHead&gt;
      &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("deadline")}&gt;
        &lt;span className="flex items-center gap-1"&gt;Deadline &lt;SortIcon col="deadline" /&gt;&lt;/span&gt;
      &lt;/TableHead&gt;
      &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("budget")}&gt;
        &lt;span className="flex items-center gap-1"&gt;Budget &lt;SortIcon col="budget" /&gt;&lt;/span&gt;
      &lt;/TableHead&gt;
      &lt;TableHead className="w-[100px]"&gt;&lt;/TableHead&gt;
    &lt;/TableRow&gt;
  &lt;/TableHeader&gt;
);

return (
  &lt;div className="p-6 space-y-6"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.projects")}&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{t("admin.manageProjects")}&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={dialogOpen} onOpenChange={(o) =&gt; { setDialogOpen(o); if (!o) setEditing(null); }}&gt;
        &lt;DialogTrigger asChild&gt;
          &lt;Button onClick={openNew}&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;{t("admin.newProject")}&lt;/Button&gt;
        &lt;/DialogTrigger&gt;
        &lt;DialogContent className="max-w-4xl max-h-[90vh] overflow-y-auto"&gt;
          &lt;DialogHeader&gt;&lt;DialogTitle className="text-lg font-semibold"&gt;{editing ? t("admin.editProject") : t("admin.newProject")}&lt;/DialogTitle&gt;&lt;/Dialo

          {countryStep &amp;&amp; !editing ? (
            &lt;div className="py-8 space-y-6"&gt;
              &lt;p className="text-center text-sm text-muted-foreground"&gt;Este projeto é para qual país?&lt;/p&gt;
              &lt;div className="flex justify-center gap-6"&gt;
                &lt;button
                  type="button"
                  onClick={() =&gt; selectCountryForNew("Brazil")}
                  className="flex flex-col items-center gap-3 p-6 rounded-lg border-2 border-border hover:border-primary transition-colors cursor-pointer bg-card"
                &gt;
                  &lt;span className="text-5xl"&gt;■■&lt;/span&gt;
                  &lt;span className="text-sm font-medium text-foreground"&gt;Brasil&lt;/span&gt;
                  &lt;span className="text-xs text-muted-foreground"&gt;R$ · m²&lt;/span&gt;
                &lt;/button&gt;
                &lt;button
                  type="button"
                  onClick={() =&gt; selectCountryForNew("USA")}
                  className="flex flex-col items-center gap-3 p-6 rounded-lg border-2 border-border hover:border-primary transition-colors cursor-pointer bg-card"
                &gt;
                  &lt;span className="text-5xl"&gt;■■&lt;/span&gt;
                  &lt;span className="text-sm font-medium text-foreground"&gt;Estados Unidos&lt;/span&gt;
                  &lt;span class="text-xs text-muted-foreground"&gt;USD · ft²&lt;/span&gt;
                &lt;/button&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          ) : (
            &lt;&gt;
              &lt;Tabs defaultValue="info" className="w-full"&gt;
                &lt;TabsList className="w-full grid grid-cols-4"&gt;
                  &lt;TabsTrigger value="info"&gt;Info&lt;/TabsTrigger&gt;
                  &lt;TabsTrigger value="address"&gt;{isBrazil ? "Endereço" : "Address"}&lt;/TabsTrigger&gt;
                  &lt;TabsTrigger value="dates"&gt;Datas &amp; Notas&lt;/TabsTrigger&gt;
                  &lt;TabsTrigger value="financial"&gt;Financeiro&lt;/TabsTrigger&gt;
                &lt;/TabsList&gt;

                {/* TAB: Info */}
                &lt;TabsContent value="info" className="space-y-4 mt-4"&gt;
                  &lt;div className="flex flex-col items-center gap-2"&gt;
                    &lt;div className="relative cursor-pointer" onClick={() =&gt; coverInputRef.current?.click()}&gt;
                      &lt;Avatar className="h-20 w-20 border-2 border-border overflow-hidden"&gt;
                        &lt;AvatarImage src={coverPreview || undefined} className="object-cover" /&gt;
                        &lt;AvatarFallback className="bg-muted text-muted-foreground"&gt;
                          &lt;FolderOpen className="h-8 w-8" /&gt;
                        &lt;/AvatarFallback&gt;
                      &lt;/Avatar&gt;
                      &lt;div className="absolute bottom-0 right-0 bg-primary text-primary-foreground rounded-full p-1"&gt;
                        &lt;Camera className="h-3 w-3" /&gt;
                      &lt;/div&gt;
                    &lt;/div&gt;
                    &lt;p className="text-xs text-muted-foreground"&gt;Foto do projeto&lt;/p&gt;
                    &lt;input ref={coverInputRef} type="file" accept="image/*" className="hidden" onChange={handleCoverChange} /&gt;
                  &lt;/div&gt;

                  {/* Country indicator */}
                  &lt;div className="flex items-center gap-2 p-2 rounded-md bg-muted/50 border border-border"&gt;
                    &lt;span className="text-lg"&gt;{isBrazil ? "■■" : "■■"}&lt;/span&gt;
                    &lt;span class="text-sm text-muted-foreground"&gt;{isBrazil ? "Brasil — R$ · m²" : "Estados Unidos — USD · ft²"}&lt;/span&gt;
                  &lt;/div&gt;
```

```
  &lt;div&gt;&lt;Label&gt;{t("admin.company")}&lt;/Label&gt;
    &lt;Select value={form.company_id} onValueChange={v =&gt; setForm({ ...form, company_id: v })}&gt;
      &lt;SelectTrigger&gt;&lt;SelectValue placeholder={t("admin.select")} /&gt;&lt;/SelectTrigger&gt;
      &lt;SelectContent&gt;{companies.map(c =&gt; &lt;SelectItem key={c.id} value={c.id}&gt;{c.name}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
    &lt;/Select&gt;
  &lt;/div&gt;
  &lt;div&gt;&lt;Label&gt;{t("admin.projectStage")}&lt;/Label&gt;
    &lt;Select value={form.stage} onValueChange={v =&gt; setForm({ ...form, stage: v })}&gt;
      &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
      &lt;SelectContent&gt;{PROJECT_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{getStageLabel(s)}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
    &lt;/Select&gt;
  &lt;/div&gt;
  &lt;div class="grid grid-cols-2 gap-4"&gt;
    &lt;div&gt;&lt;Label&gt;Tipo de Serviço&lt;/Label&gt;
      &lt;Input value={form.service_type} onChange={e =&gt; setForm({ ...form, service_type: e.target.value })} placeholder="Ex: Remodeling, 3D Kitchen..."
    &lt;/div&gt;
    &lt;div&gt;&lt;Label&gt;{areaLabel}&lt;/Label&gt;&lt;Input type="number" value={form.square_feet} onChange={e =&gt; setForm({ ...form, square_feet: e.t
  &lt;/div&gt;
  &lt;div&gt;&lt;Label&gt;{t("admin.package")}&lt;/Label&gt;
    &lt;Select value={form.package_id || "none"} onValueChange={v =&gt; setForm({ ...form, package_id: v === "none" ? "" : v })}&gt;
      &lt;SelectTrigger&gt;&lt;SelectValue placeholder={t("admin.select")} /&gt;&lt;/SelectTrigger&gt;
      &lt;SelectContent&gt;
        &lt;SelectItem value="none"&gt;{t("admin.none")}&lt;/SelectItem&gt;
        {packages.map(p =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{p.name}&lt;/SelectItem&gt;)}
      &lt;/SelectContent&gt;
    &lt;/Select&gt;
  &lt;/div&gt;
  &lt;div&gt;
    &lt;Label&gt;{t("admin.services")}&lt;/Label&gt;
    &lt;div class="space-y-4 mt-2"&gt;
      {SERVICE_CATEGORIES_ORDER.map(cat =&gt; {
        const catServices = services.filter(s =&gt; s.category === cat);
        if (catServices.length === 0) return null;
        return (
          &lt;div key={cat}&gt;
            &lt;p class="text-xs font-semibold text-muted-foreground uppercase tracking-wider mb-2"&gt;{cat}&lt;/p&gt;
            &lt;div class="grid grid-cols-2 gap-2"&gt;
              {catServices.map(s =&gt; (
                &lt;div key={s.id} class="flex items-center gap-2"&gt;
                  &lt;Checkbox
                    checked={selectedServices.includes(s.id)}
                    onCheckedChange={(checked) =&gt; {
                      setSelectedServices(prev =&gt;
                        checked ? [...prev, s.id] : prev.filter(id =&gt; id !== s.id)
                      );
                    }}
                  /&gt;
                  &lt;span class="text-sm"&gt;{s.name}&lt;/span&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        );
      })}
    &lt;/div&gt;
  &lt;/div&gt;
&lt;/TabsContent&gt;

{/* TAB: Address */}
&lt;TabsContent value="address" class="space-y-4 mt-4"&gt;
  &lt;div class="flex items-center justify-between"&gt;
    &lt;p class="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;{isBrazil ? "Endereço do Projeto" : t("admin.projectAddress")
    &lt;Button type="button" variant="outline" size="sm" onClick={copyFromCompany}&gt;
      &lt;Copy class="h-3 w-3 mr-1" /&gt;{t("admin.copyFromCompany")}
    &lt;/Button&gt;
  &lt;/div&gt;
  &lt;div class="grid grid-cols-3 gap-4"&gt;
    &lt;div&gt;&lt;Label&gt;{isBrazil ? "Número" : t("admin.number")}&lt;/Label&gt;&lt;Input value={form.street_number} onChange={e =&gt; setForm({ ...form
    &lt;div class="col-span-2"&gt;&lt;Label&gt;{isBrazil ? "Rua" : t("admin.street")}&lt;/Label&gt;&lt;Input value={form.street_name} onChange={e =&gt;
  &lt;/div&gt;
  &lt;div class="grid grid-cols-2 gap-4"&gt;
    &lt;div&gt;&lt;Label&gt;{isBrazil ? "Complemento" : t("admin.unit")}&lt;/Label&gt;&lt;Input value={form.unit} onChange={e =&gt; setForm({ ...form, unit
    &lt;div&gt;&lt;Label&gt;{isBrazil ? "Bairro" : t("admin.neighborhood")}&lt;/Label&gt;&lt;Input value={form.neighborhood} onChange={e =&gt; setForm({ ..
  &lt;/div&gt;
  &lt;div class="grid grid-cols-3 gap-4"&gt;
    &lt;div&gt;&lt;Label&gt;{isBrazil ? "Cidade" : t("admin.city")}&lt;/Label&gt;&lt;Input value={form.city} onChange={e =&gt; setForm({ ...form, city: e.t
    &lt;div&gt;&lt;Label&gt;{isBrazil ? "Estado" : t("admin.state")}&lt;/Label&gt;&lt;Input value={form.state} onChange={e =&gt; setForm({ ...form, state:
    &lt;div&gt;&lt;Label&gt;{isBrazil ? "CEP" : t("admin.zip")}&lt;/Label&gt;&lt;Input value={form.zip_code} onChange={e =&gt; setForm({ ...form, zip_code:
  &lt;/div&gt;
  &lt;div&gt;&lt;Label&gt;{isBrazil ? "País" : t("admin.countryLabel")}&lt;/Label&gt;
    &lt;Select value={form.country} onValueChange={v =&gt; {
      const newCurrency = v === "Brazil" ? "BRL" : "USD";
      const newState = v === "Brazil" ? "" : "Massachusetts";
      setForm({ ...form, country: v, currency: newCurrency, state: newState });
    }}&gt;
      &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
      &lt;SelectContent&gt;{COUNTRIES.map(c =&gt; &lt;SelectItem key={c} value={c}&gt;{c}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
    &lt;/Select&gt;
  &lt;/div&gt;
&lt;/TabsContent&gt;

{/* TAB: Dates &amp; Notes */}
&lt;TabsContent value="dates" class="space-y-4 mt-4"&gt;
  &lt;h3 class="text-lg font-semibold"&gt;Project Timeline&lt;/h3&gt;
  &lt;div class="grid grid-cols-2 gap-4"&gt;
    &lt;div&gt;
      &lt;Label class="text-sm font-medium"&gt;Start Date&lt;/Label&gt;
      &lt;Input type="date" class="w-full" value={form.start_date} onChange={e =&gt; setForm({ ...form, start_date: e.target.value })} /&gt;
    &lt;/div&gt;
    &lt;div&gt;
```

```
          &lt;Label className="text-sm font-medium"&gt;Project Deadline&lt;/Label&gt;
          &lt;Input type="date" className="w-full" value={form.delivery_date} onChange={e =&gt; setForm({ ...form, delivery_date: e.target.value })} /&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label className="text-sm font-medium"&gt;Preview Delivery&lt;/Label&gt;
          &lt;Input type="date" className="w-full" value={form.preview_delivery_date} onChange={e =&gt; setForm({ ...form, preview_delivery_date: e.target.valu
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label className="text-sm font-medium"&gt;Client Revision Return&lt;/Label&gt;
          &lt;Input type="date" className="w-full" value={form.revision_return_date} onChange={e =&gt; setForm({ ...form, revision_return_date: e.target.value
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label className="text-sm font-medium"&gt;Final Delivery&lt;/Label&gt;
          &lt;Input type="date" className="w-full" value={form.final_delivery_date} onChange={e =&gt; setForm({ ...form, final_delivery_date: e.target.value })
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;Separator /&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Histórico / Notas de Acompanhamento&lt;/p&gt;
        &lt;Button type="button" variant="outline" size="sm" onClick={addDateNote}&gt;
          &lt;Plus className="h-3 w-3 mr-1" /&gt; Adicionar Nota
        &lt;/Button&gt;
      &lt;/div&gt;
      &lt;p className="text-xs text-muted-foreground"&gt;
        Registre cada evento do projeto com data, descrição e quem pode ver.
      &lt;/p&gt;

      {dateNotes.length === 0 &amp;amp;&amp; (
        &lt;div className="text-center py-6 text-muted-foreground text-sm border border-dashed rounded-md"&gt;
          Nenhuma nota registrada. Clique em "Adicionar Nota" para começar.
        &lt;/div&gt;
      )}

      &lt;div className="space-y-3"&gt;
        {dateNotes.sort((a, b) =&gt; (a.date || "").localeCompare(b.date || "")).map((note, idx) =&gt; {
          const visOpt = VISIBILITY_OPTIONS.find(v =&gt; v.value === note.visibility) || VISIBILITY_OPTIONS[0];
          const VisIcon = visOpt.icon;
          return (
            &lt;div key={idx} className="grid grid-cols-[140px_1fr_160px_40px] items-center gap-3 p-3 rounded-md border bg-muted/30"&gt;
              &lt;Input type="date" value={note.date} onChange={e =&gt; updateDateNote(idx, "date", e.target.value)} className="text-sm" /&gt;
              &lt;Input value={note.note} onChange={e =&gt; updateDateNote(idx, "note", e.target.value)} placeholder="Ex: Cliente pagou entrada, Início do 3D..
              &lt;Select value={note.visibility} onValueChange={v =&gt; updateDateNote(idx, "visibility", v)}&gt;
                &lt;SelectTrigger className="text-sm"&gt;
                  &lt;div className="flex items-center gap-1.5"&gt;
                    &lt;VisIcon className={`h-3.5 w-3.5 ${visOpt.color}`} /&gt;
                    &lt;span&gt;{visOpt.label}&lt;/span&gt;
                  &lt;/div&gt;
                &lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;
                  {VISIBILITY_OPTIONS.map(v =&gt; (
                    &lt;SelectItem key={v.value} value={v.value}&gt;
                      &lt;div className="flex items-center gap-1.5"&gt;
                        &lt;v.icon className={`h-3.5 w-3.5 ${v.color}`} /&gt;
                        &lt;span&gt;{v.label}&lt;/span&gt;
                      &lt;/div&gt;
                    &lt;/SelectItem&gt;
                  ))}
                &lt;/SelectContent&gt;
              &lt;/Select&gt;
              &lt;Button type="button" variant="ghost" size="icon" className="h-8 w-8 text-destructive" onClick={() =&gt; removeDateNote(idx)}&gt;
                &lt;Trash2 className="h-3.5 w-3.5" /&gt;
              &lt;/Button&gt;
            &lt;/div&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/TabsContent&gt;

    {/* TAB: Financial */}
    &lt;TabsContent value="financial" className="space-y-4 mt-4"&gt;
      &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;{t("admin.financials")}&lt;/p&gt;
      &lt;div className="grid grid-cols-2 gap-4"&gt;
        &lt;div&gt;&lt;Label&gt;{t("admin.currency")}&lt;/Label&gt;
          &lt;Select value={form.currency} onValueChange={v =&gt; setForm({ ...form, currency: v })}&gt;
            &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;{CURRENCIES.map(c =&gt; &lt;SelectItem key={c} value={c}&gt;{c}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
          &lt;/Select&gt;
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;{t("admin.totalValue")} ({isBrazil ? `R$/${areaUnit}` : `USD/${areaUnit}`})&lt;/Label&gt;&lt;Input type="number" value={form.to
      &lt;/div&gt;
      &lt;div&gt;&lt;Label&gt;{t("admin.paymentStage")}&lt;/Label&gt;
        &lt;Select value={form.payment_stage} onValueChange={v =&gt; setForm({ ...form, payment_stage: v })}&gt;
          &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
          &lt;SelectContent&gt;{PAYMENT_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{s}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
        &lt;/Select&gt;
      &lt;/div&gt;
      &lt;div grid grid-cols-2 gap-4"&gt;
        &lt;div className="p-3 rounded-md bg-chart-3/10 border border-chart-3/20"&gt;
          &lt;p class="text-xs text-muted-foreground"&gt;{t("admin.entryAmount")} (50%)&lt;/p&gt;
          &lt;p class="text-lg font-semibold text-chart-3"&gt;{currencySymbol(form.currency)}{entryAmount.toLocaleString()}&lt;/p&gt;
        &lt;/div&gt;
        &lt;div class="p-3 rounded-md bg-chart-4/10 border border-chart-4/20"&gt;
          &lt;p class="text-xs text-muted-foreground"&gt;{t("admin.finalAmount")} (50%)&lt;/p&gt;
          &lt;p class="text-lg font-semibold text-chart-4"&gt;{currencySymbol(form.currency)}{finalAmount.toLocaleString()}&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/TabsContent&gt;
  &lt;/Tabs&gt;

  &lt;Button className="w-full mt-4" onClick={handleSave}&gt;{editing ? t("admin.save") : t("admin.create")}&lt;/Button&gt;
```

```
        &lt;/&gt;
      )}
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;

{/* Filters */}
&lt;div className="flex flex-wrap gap-3"&gt;
  &lt;div className="relative max-w-xs"&gt;
    &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
    &lt;Input placeholder={t("admin.search")} className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
  &lt;/div&gt;
  &lt;Select value={stageFilter} onValueChange={setStageFilter}&gt;
    &lt;SelectTrigger className="w-[160px]"&gt;&lt;SelectValue placeholder={t("admin.stage")} /&gt;&lt;/SelectTrigger&gt;
    &lt;SelectContent&gt;
      &lt;SelectItem value="all"&gt;{t("admin.allStages")}&lt;/SelectItem&gt;
      {PROJECT_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{getStageLabel(s)}&lt;/SelectItem&gt;)}
    &lt;/SelectContent&gt;
  &lt;/Select&gt;
  &lt;Select value={companyFilter} onValueChange={setCompanyFilter}&gt;
    &lt;SelectTrigger className="w-[200px]"&gt;&lt;SelectValue placeholder={t("admin.company")} /&gt;&lt;/SelectTrigger&gt;
    &lt;SelectContent&gt;
      &lt;SelectItem value="all"&gt;{t("admin.allCompanies")}&lt;/SelectItem&gt;
      {companies.map(c =&gt; &lt;SelectItem key={c.id} value={c.id}&gt;{c.name}&lt;/SelectItem&gt;)}
    &lt;/SelectContent&gt;
  &lt;/Select&gt;
&lt;/div&gt;

{/* Section: Active Projects */}
&lt;div className="space-y-2"&gt;
  &lt;div className="flex items-center gap-2"&gt;
    &lt;h2 className="text-sm font-semibold text-foreground uppercase tracking-wider"&gt;Active Projects&lt;/h2&gt;
    &lt;Badge variant="secondary" className="text-xs"&gt;{activeProjects.length}&lt;/Badge&gt;
  &lt;/div&gt;
  &lt;Card&gt;
    &lt;CardContent className="p-0"&gt;
      &lt;Table&gt;
        {renderTableHeader()}
        &lt;TableBody&gt;
          {activeProjects.length === 0 &amp;amp;&amp;amp; (
            &lt;TableRow&gt;&lt;TableCell colSpan={6} className="py-8 text-center text-muted-foreground"&gt;No active projects&lt;/TableCell&gt;&lt;/TableRow&gt;
          )}
          {activeProjects.map(p =&gt; renderProjectRow(p))}
        &lt;/TableBody&gt;
      &lt;/Table&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Section: Awaiting Payment */}
{awaitingPayment.length &gt; 0 &amp;amp;&amp;amp; (
  &lt;div className="space-y-2"&gt;
    &lt;div className="flex items-center gap-2"&gt;
      &lt;h2 className="text-sm font-semibold text-chart-3 uppercase tracking-wider"&gt;■ Awaiting Payment&lt;/h2&gt;
      &lt;Badge variant="secondary" className="text-xs"&gt;{awaitingPayment.length}&lt;/Badge&gt;
    &lt;/div&gt;
    &lt;Card className="border-chart-3/30"&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          {renderTableHeader()}
          &lt;TableBody&gt;
            {awaitingPayment.map(p =&gt; renderProjectRow(p))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/div&gt;
)}

{/* Section: Completed Projects (Collapsible) */}
{completedProjects.length &gt; 0 &amp;amp;&amp;amp; (
  &lt;Collapsible open={completedOpen} onOpenChange={setCompletedOpen}&gt;
    &lt;div className="space-y-2"&gt;
      &lt;CollapsibleTrigger className="flex items-center gap-2 cursor-pointer hover:opacity-80 transition-opacity"&gt;
        {completedOpen ? &lt;ChevronDown className="h-4 w-4 text-muted-foreground" /&gt; : &lt;ChevronRight className="h-4 w-4 text-muted-foreground" /&gt;}
        &lt;h2 className="text-sm font-semibold text-chart-2 uppercase tracking-wider"&gt;■ Completed Projects&lt;/h2&gt;
        &lt;Badge variant="secondary" className="text-xs"&gt;{completedProjects.length}&lt;/Badge&gt;
      &lt;/CollapsibleTrigger&gt;
      &lt;CollapsibleContent&gt;
        &lt;Card className="border-chart-2/30"&gt;
          &lt;CardContent className="p-0"&gt;
            &lt;Table&gt;
              &lt;TableHeader&gt;
                &lt;TableRow&gt;
                  &lt;TableHead&gt;Project ID&lt;/TableHead&gt;
                  &lt;TableHead&gt;Project&lt;/TableHead&gt;
                  &lt;TableHead&gt;Client&lt;/TableHead&gt;
                  &lt;TableHead&gt;Completed&lt;/TableHead&gt;
                  &lt;TableHead&gt;Total Paid&lt;/TableHead&gt;
                  &lt;TableHead className="w-[100px]"&gt;&lt;/TableHead&gt;
                &lt;/TableRow&gt;
              &lt;/TableHeader&gt;
              &lt;TableBody&gt;
                {completedProjects.map((p: any) =&gt; {
                  const paid = payments[p.id] || 0;
                  const isBRL = (p.currency || "USD") === "BRL";
                  const formattedPaid = isBRL
                    ? `R$${paid.toLocaleString("pt-BR")}`
                    : `$${paid.toLocaleString("en-US")}`;
                  return (
```

```
                        &lt;TableRow key={p.id} className="group"&gt;
                          &lt;TableCell&gt;
                            &lt;span className="text-xs font-mono text-muted-foreground"&gt;{p.project_number || "—"}&lt;/span&gt;
                          &lt;/TableCell&gt;
                          &lt;TableCell&gt;
                            &lt;p className="text-sm font-medium text-foreground hover:text-primary cursor-pointer" onClick={() =&gt; navigate(`/adm/projects/${p.id}`)}&gt
                              {getProjectDisplayName(p)}
                            &lt;/p&gt;
                          &lt;/TableCell&gt;
                          &lt;TableCell&gt;
                            &lt;p className="text-sm text-foreground"&gt;{p.companies?.name || "—"} {getCountryFlag(p.country)}&lt;/p&gt;
                          &lt;/TableCell&gt;
                          &lt;TableCell&gt;
                            &lt;span className="text-sm text-muted-foreground"&gt;
                              {p.final_delivery_date
                                ? new Date(p.final_delivery_date).toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" })
                                : p.updated_at
                                  ? new Date(p.updated_at).toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" })
                                  : "—"}
                            &lt;/span&gt;
                          &lt;/TableCell&gt;
                          &lt;TableCell&gt;
                            &lt;span className="text-sm font-medium text-chart-2"&gt;{formattedPaid}&lt;/span&gt;
                          &lt;/TableCell&gt;
                          &lt;TableCell&gt;
                            &lt;div className="flex gap-1 opacity-0 group-hover:opacity-100 transition-opacity"&gt;
                              &lt;Button variant="ghost" size="icon" onClick={() =&gt; navigate(`/adm/projects/${p.id}`)} title="View"&gt;&lt;Eye className="h-4 w-4" /&gt;
                              &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleArchiveProject(p.id)} title="Archive"&gt;&lt;Archive className="h-4 w-4 text-m
                            &lt;/div&gt;
                          &lt;/TableCell&gt;
                        &lt;/TableRow&gt;
                      );
                    })}
                  &lt;/TableBody&gt;
                &lt;/Table&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          &lt;/CollapsibleContent&gt;
        &lt;/div&gt;
      &lt;/Collapsible&gt;
    )}
  &lt;/div&gt;
);
};

export default AdminProjects;
```

src/pages/admin/AdminProposals.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Search, FileText } from "lucide-react";
import { format } from "date-fns";

const AdminProposals = () =&gt; {
  const [quotes, setQuotes] = useState&lt;any[]&gt;([]);
  const [search, setSearch] = useState("");

  useEffect(() =&gt; {
    supabase.from("quotes").select("*, companies(name), projects(name)").order("created_at", { ascending: false })
      .then(({ data }) =&gt; setQuotes(data || []));
  }, []);

  const statusColor: Record&lt;string, string&gt; = {
    draft: "bg-muted text-muted-foreground", sent: "bg-blue-500/10 text-blue-600",
    approved: "bg-emerald-500/10 text-emerald-600", paid: "bg-green-500/10 text-green-700",
  };

  const filtered = quotes.filter(q =&gt;
    (q.title || "").toLowerCase().includes(search.toLowerCase()) ||
    (q.companies?.name || "").toLowerCase().includes(search.toLowerCase())
  );

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Proposals&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Manage client proposals and quotes&lt;/p&gt;
        &lt;/div&gt;
        &lt;Badge variant="secondary"&gt;{quotes.length} total&lt;/Badge&gt;
      &lt;/div&gt;
      &lt;div className="relative max-w-xs"&gt;
        &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
        &lt;Input placeholder="Search proposals..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Title&lt;/TableHead&gt;
                &lt;TableHead&gt;Client&lt;/TableHead&gt;
                &lt;TableHead&gt;Project&lt;/TableHead&gt;
                &lt;TableHead&gt;Amount&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Date&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {filtered.length === 0 &amp;&amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={6} className="text-center text-muted-foreground py-12"&gt;&lt;FileText className="h-8 w-8 mx-auto mb-2 opacity-40" /&g
              )}
              {filtered.map(q =&gt; (
                &lt;TableRow key={q.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{q.title}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{q.companies?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{q.projects?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="font-medium"&gt;${Number(q.amount).toLocaleString()}&lt;/TableCell&gt;
                  &lt;TableCell&gt;&lt;Badge variant="outline" className={statusColor[q.status] || ""}&gt;{q.status}&lt;/Badge&gt;&lt;/TableCell&gt;
                  &lt;TableCell className="text-xs text-muted-foreground"&gt;{format(new Date(q.created_at), "MMM d, yyyy")}&lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminProposals;
```

src/pages/admin/AdminSettings.tsx

```
import { useState, useEffect } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Skeleton } from "@/components/ui/skeleton";
import { toast } from "sonner";
import { Settings, Palette, Globe, Bell, Shield, Save, Loader2 } from "lucide-react";

const AdminSettings = () =&gt; {
  const [settings, setSettings] = useState&lt;any&gt;(null);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      const { data } = await supabase.from("studio_settings").select("*").limit(1).single();
      if (data) setSettings(data);
      setLoading(false);
    };
    fetch();
  }, []);

  const handleSave = async () =&gt; {
    if (!settings) return;
    setSaving(true);
    const { error } = await supabase.from("studio_settings").update({
      studio_name: settings.studio_name,
      tagline: settings.tagline,
      logo_url: settings.logo_url,
      primary_color: settings.primary_color,
      accent_color: settings.accent_color,
      contact_email: settings.contact_email,
      contact_phone: settings.contact_phone,
      custom_domain: settings.custom_domain,
    }).eq("id", settings.id);
    if (error) toast.error("Failed to save settings");
    else toast.success("Settings saved successfully");
    setSaving(false);
  };

  if (loading) return &lt;div className="p-6 space-y-4"&gt;&lt;Skeleton className="h-8 w-48" /&gt;&lt;Skeleton className="h-64" /&gt;&lt;/div&gt;;

  return (
    &lt;div className="p-6 space-y-6 max-w-[800px]"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Settings&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;System configuration and white-label branding&lt;/p&gt;
      &lt;/div&gt;

      {/* Branding / White Label */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;Palette className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base"&gt;White Label Branding&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent className="space-y-4"&gt;
          &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
            &lt;div&gt;
              &lt;Label&gt;Studio Name&lt;/Label&gt;
              &lt;Input value={settings?.studio_name || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, studio_name: e.target.value }))} /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Tagline&lt;/Label&gt;
              &lt;Input value={settings?.tagline || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, tagline: e.target.value }))} placeholder="e.g. Drafting &amp; 3
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label&gt;Logo URL&lt;/Label&gt;
            &lt;Input value={settings?.logo_url || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, logo_url: e.target.value }))} placeholder="https://..." /&gt;
          &lt;/div&gt;
          &lt;div class="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
            &lt;div&gt;
              &lt;Label&gt;Primary Color&lt;/Label&gt;
              &lt;div class="flex gap-2"&gt;
                &lt;Input type="color" value={settings?.primary_color || "#b8860b"} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, primary_color: e.target.value }))}
                &lt;Input value={settings?.primary_color || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, primary_color: e.target.value }))} class="flex-1" /
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Accent Color&lt;/Label&gt;
              &lt;div class="flex gap-2"&gt;
                &lt;Input type="color" value={settings?.accent_color || "#f5a623"} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, accent_color: e.target.value }))} cl
                &lt;Input value={settings?.accent_color || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, accent_color: e.target.value }))} class="flex-1" /&g
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* General */}
      &lt;Card class="border-none shadow-sm"&gt;
        &lt;CardHeader class="pb-3"&gt;
          &lt;div class="flex items-center gap-2"&gt;
            &lt;Settings class="h-4 w-4 text-muted-foreground" /&gt;
```

```
            &lt;CardTitle className="text-base"&gt;General&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent className="space-y-4"&gt;
          &lt;div&gt;&lt;Label&gt;Contact Email&lt;/Label&gt;&lt;Input value={settings?.contact_email || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, contact_em
          &lt;div&gt;&lt;Label&gt;Contact Phone&lt;/Label&gt;&lt;Input value={settings?.contact_phone || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, contact_ph
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Domain */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;Globe className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base"&gt;Custom Domain&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          &lt;div&gt;&lt;Label&gt;Domain&lt;/Label&gt;&lt;Input value={settings?.custom_domain || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, custom_domain: e.
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;&lt;Bell className="h-4 w-4 text-muted-foreground" /&gt;&lt;CardTitle className="text-base"&gt;Notifications&lt;/Car
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;&lt;p className="text-sm text-muted-foreground"&gt;Email notification settings will be available here.&lt;/p&gt;&lt;/CardContent&gt;
      &lt;/Card&gt;

      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;&lt;Shield className="h-4 w-4 text-muted-foreground" /&gt;&lt;CardTitle className="text-base"&gt;Security&lt;/CardTi
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;&lt;p className="text-sm text-muted-foreground"&gt;Authentication and access control settings.&lt;/p&gt;&lt;/CardContent&gt;
      &lt;/Card&gt;

      &lt;Button onClick={handleSave} disabled={saving}&gt;
        {saving ? &lt;&gt;&lt;Loader2 className="mr-2 h-4 w-4 animate-spin" /&gt;Saving...&lt;/&gt; : &lt;&gt;&lt;Save className="mr-2 h-4 w-4" /&gt;Save Settings&lt;/&gt;}
      &lt;/Button&gt;
    &lt;/div&gt;
  );
};

export default AdminSettings;
```

src/pages/admin/AdminSiteUpdates.tsx

```
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { Badge } from "@/components/ui/badge";
import { Plus, Upload, Camera, HardHat, ExternalLink, Calendar } from "lucide-react";
import { toast } from "sonner";
import { format } from "date-fns";

const AdminSiteUpdates = () =&gt; {
  const navigate = useNavigate();
  const [updates, setUpdates] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [uploading, setUploading] = useState(false);
  const [selectedProject, setSelectedProject] = useState("");
  const [description, setDescription] = useState("");
  const [files, setFiles] = useState&lt;File[]&gt;([]);
  const [filterProject, setFilterProject] = useState("all");

  const fetchData = async () =&gt; {
    const [updRes, projRes] = await Promise.all([
      supabase.from("site_updates").select("*, projects(name, street_name, street_number, companies(name))").order("created_at", { ascending: false }),
      supabase.from("projects").select("id, name, street_name, street_number, companies(name)").order("created_at", { ascending: false }),
    ]);
    setUpdates(updRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  useEffect(() =&gt; {
    const channel = supabase
      .channel("site-updates-realtime")
      .on("postgres_changes", { event: "*", schema: "public", table: "site_updates" }, () =&gt; fetchData())
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, []);

  const getProjectLabel = (p: any) =&gt; {
    const addr = [p.street_number, p.street_name].filter(Boolean).join(" ");
    return addr || p.name || p.companies?.name || "—";
  };

  const handleUpload = async () =&gt; {
    if (!selectedProject || files.length === 0) {
      toast.error("Select a project and at least one file");
      return;
    }
    setUploading(true);
    try {
      const { data: { user } } = await supabase.auth.getUser();
      for (const file of files) {
        const ext = file.name.split(".").pop();
        const path = `${selectedProject}/${Date.now()}-${Math.random().toString(36).slice(2)}.${ext}`;
        const { error: uploadError } = await supabase.storage.from("site-updates").upload(path, file);
        if (uploadError) { toast.error(`Upload failed: ${file.name}`); continue; }
        const { data: urlData } = supabase.storage.from("site-updates").getPublicUrl(path);

        await supabase.from("site_updates").insert({
          project_id: selectedProject,
          uploaded_by: user?.id || null,
          media_url: urlData.publicUrl,
          description: description || null,
        });
      }
      toast.success(`${files.length} update(s) uploaded`);
      setDialogOpen(false);
      setFiles([]);
      setDescription("");
      fetchData();
    } catch {
      toast.error("Upload failed");
    } finally {
      setUploading(false);
    }
  };

  const filtered = filterProject === "all"
    ? updates
    : updates.filter(u =&gt; u.project_id === filterProject);

  // Group by date
  const grouped: Record&lt;string, any[]&gt; = {};
  filtered.forEach(u =&gt; {
    const day = format(new Date(u.created_at), "yyyy-MM-dd");
    if (!grouped[day]) grouped[day] = [];
    grouped[day].push(u);
  });

  return (
```

```
&lt;div className="p-6 space-y-6 max-w-[1400px] mx-auto"&gt;
  &lt;div className="flex items-center justify-between"&gt;
    &lt;div&gt;
      &lt;h1 className="text-xl font-semibold text-foreground"&gt;Site Updates&lt;/h1&gt;
      &lt;p className="text-sm text-muted-foreground"&gt;Construction progress photos and updates&lt;/p&gt;
    &lt;/div&gt;
    &lt;div className="flex items-center gap-3"&gt;
      &lt;Select value={filterProject} onValueChange={setFilterProject}&gt;
        &lt;SelectTrigger className="w-[220px]"&gt;
          &lt;SelectValue placeholder="Filter by project" /&gt;
        &lt;/SelectTrigger&gt;
        &lt;SelectContent&gt;
          &lt;SelectItem value="all"&gt;All Projects&lt;/SelectItem&gt;
          {projects.map(p =&gt; (
            &lt;SelectItem key={p.id} value={p.id}&gt;{getProjectLabel(p)}&lt;/SelectItem&gt;
          ))}
        &lt;/SelectContent&gt;
      &lt;/Select&gt;
      &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
        &lt;DialogTrigger asChild&gt;
          &lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt; New Update&lt;/Button&gt;
        &lt;/DialogTrigger&gt;
        &lt;DialogContent&gt;
          &lt;DialogHeader&gt;&lt;DialogTitle&gt;Upload Site Update&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            &lt;div&gt;
              &lt;Label&gt;Project *&lt;/Label&gt;
              &lt;Select value={selectedProject || "none"} onValueChange={v =&gt; v !== "none" &amp;&amp; setSelectedProject(v)}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Select project" /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;
                  {projects.map(p =&gt; (
                    &lt;SelectItem key={p.id} value={p.id}&gt;{getProjectLabel(p)}&lt;/SelectItem&gt;
                  ))}
                &lt;/SelectContent&gt;
              &lt;/Select&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Photos / Videos *&lt;/Label&gt;
              &lt;div className="mt-1"&gt;
                &lt;label className="flex flex-col items-center justify-center w-full h-32 border-2 border-dashed rounded-lg cursor-pointer hover:bg-muted/50 transition-
                  &lt;div className="flex flex-col items-center"&gt;
                    &lt;Camera className="h-8 w-8 text-muted-foreground mb-2" /&gt;
                    &lt;p className="text-sm text-muted-foreground"&gt;
                      {files.length &gt; 0 ? `${files.length} file(s) selected` : "Tap to take photo or select files"}
                    &lt;/p&gt;
                  &lt;/div&gt;
                  &lt;input
                    type="file"
                    className="hidden"
                    multiple
                    accept="image/*,video/*"
                    capture="environment"
                    onChange={e =&gt; setFiles(Array.from(e.target.files || []))}
                  /&gt;
                &lt;/label&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Description&lt;/Label&gt;
              &lt;Textarea
                value={description}
                onChange={e =&gt; setDescription(e.target.value)}
                placeholder="Describe what's happening on site..."
                className="mt-1"
              /&gt;
            &lt;/div&gt;
            &lt;Button className="w-full" onClick={handleUpload} disabled={uploading}&gt;
              {uploading ? (
                &lt;&gt;
                  &lt;Upload className="h-4 w-4 mr-2 animate-spin" /&gt; Uploading...
                &lt;/&gt;
              ) : (
                &lt;&gt;
                  &lt;Upload className="h-4 w-4 mr-2" /&gt; Upload Update
                &lt;/&gt;
              )}
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/DialogContent&gt;
      &lt;/Dialog&gt;
    &lt;/div&gt;
  &lt;/div&gt;

  {/* Stats */}
  &lt;div className="grid grid-cols-1 sm:grid-cols-3 gap-4"&gt;
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-4 flex items-center gap-3"&gt;
        &lt;div className="p-2.5 rounded-lg bg-primary/15"&gt;
          &lt;HardHat className="h-5 w-5 text-primary" /&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;p className="text-2xl font-bold text-foreground"&gt;{updates.length}&lt;/p&gt;
          &lt;p className="text-xs text-muted-foreground"&gt;Total Updates&lt;/p&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-4 flex items-center gap-3"&gt;
        &lt;div className="p-2.5 rounded-lg bg-chart-2/15"&gt;
          &lt;Camera className="h-5 w-5 text-chart-2" /&gt;
        &lt;/div&gt;
```

```
            &lt;div&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;
                {new Set(updates.map(u =&gt; u.project_id)).size}
              &lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground"&gt;Projects with Updates&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="p-2.5 rounded-lg bg-chart-4/15"&gt;
              &lt;Calendar className="h-5 w-5 text-chart-4" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;
                {updates.filter(u =&gt; {
                  const d = new Date(u.created_at);
                  const now = new Date();
                  return d.toDateString() === now.toDateString();
                }).length}
              &lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground"&gt;Today's Updates&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;

      {/* Timeline */}
      {Object.keys(grouped).length === 0 ? (
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="text-center py-16"&gt;
            &lt;HardHat className="h-12 w-12 mx-auto mb-3 text-muted-foreground/40" /&gt;
            &lt;p className="text-muted-foreground"&gt;No site updates yet&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground/70 mt-1"&gt;Upload progress photos from the construction site&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      ) : (
        &lt;div className="space-y-6"&gt;
          {Object.entries(grouped).map(([date, items]) =&gt; (
            &lt;div key={date}&gt;
              &lt;div className="flex items-center gap-2 mb-3"&gt;
                &lt;Calendar className="h-4 w-4 text-muted-foreground" /&gt;
                &lt;span className="text-sm font-medium text-muted-foreground"&gt;
                  {format(new Date(date), "MMMM d, yyyy")}
                &lt;/span&gt;
                &lt;Badge variant="secondary" className="text-xs"&gt;{items.length}&lt;/Badge&gt;
              &lt;/div&gt;
              &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4"&gt;
                {items.map(update =&gt; {
                  const projectLabel = [update.projects?.street_number, update.projects?.street_name].filter(Boolean).join(" ") || update.projects?.name || "—";
                  const isVideo = update.media_url?.match(/\.(mp4|mov|webm|avi)$/i);
                  return (
                    &lt;Card key={update.id} className="border-none shadow-sm overflow-hidden"&gt;
                      &lt;div className="aspect-video bg-muted relative"&gt;
                        {isVideo ? (
                          &lt;video src={update.media_url} controls className="w-full h-full object-cover" /&gt;
                        ) : (
                          &lt;img src={update.media_url} alt="Site update" className="w-full h-full object-cover" /&gt;
                        )}
                      &lt;/div&gt;
                      &lt;CardContent className="p-3 space-y-1.5"&gt;
                        &lt;div className="flex items-center justify-between"&gt;
                          &lt;span
                            className="text-sm font-medium text-primary hover:underline cursor-pointer"
                            onClick={() =&gt; {
                              const proj = projects.find(p =&gt; p.id === update.project_id);
                              if (proj) navigate(`/adm/projects/${proj.id}`);
                            }}
                          &gt;
                            {projectLabel}
                          &lt;/span&gt;
                          &lt;span className="text-xs text-muted-foreground"&gt;
                            {format(new Date(update.created_at), "h:mm a")}
                          &lt;/span&gt;
                        &lt;/div&gt;
                        {update.projects?.companies?.name &amp;&amp; (
                          &lt;p className="text-xs text-muted-foreground"&gt;{update.projects.companies.name}&lt;/p&gt;
                        )}
                        {update.description &amp;&amp; (
                          &lt;p className="text-sm text-foreground"&gt;{update.description}&lt;/p&gt;
                        )}
                      &lt;/CardContent&gt;
                    &lt;/Card&gt;
                  );
                })}
              &lt;/div&gt;
            &lt;/div&gt;
          ))}
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
};

export default AdminSiteUpdates;
```

src/pages/admin/AdminTickets.tsx

```
import { useEffect, useState, useRef } from "react";
import { useSearchParams } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Avatar, AvatarFallback } from "@/components/ui/avatar";
import { Plus, MessageSquare, Send, Ticket } from "lucide-react";
import { toast } from "sonner";
import { formatDistanceToNow, format } from "date-fns";

const STATUS_STYLES: Record&lt;string, { label: string; variant: "default" | "secondary" | "outline" | "destructive" }&gt; = {
  open: { label: "Open", variant: "destructive" },
  waiting: { label: "Waiting", variant: "secondary" },
  resolved: { label: "Resolved", variant: "outline" },
};

const AdminTickets = () =&gt; {
  const { user } = useAuth();
  const [searchParams, setSearchParams] = useSearchParams();
  const [tickets, setTickets] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [selectedTicket, setSelectedTicket] = useState&lt;any&gt;(null);
  const [messages, setMessages] = useState&lt;any[]&gt;([]);
  const [newMsg, setNewMsg] = useState("");
  const [createOpen, setCreateOpen] = useState(false);
  const [form, setForm] = useState({ project_id: "", subject: "", message: "" });
  const scrollRef = useRef&lt;HTMLDivElement&gt;(null);

  const fetchTickets = async () =&gt; {
    const { data } = await supabase
      .from("client_tickets")
      .select("*, projects(name, company_id), companies(name)")
      .order("updated_at", { ascending: false });
    setTickets(data || []);
    return data || [];
  };

  const fetchProjects = async () =&gt; {
    const { data } = await supabase.from("projects").select("id, name, company_id").order("name");
    setProjects(data || []);
  };

  const fetchMessages = async (ticketId: string) =&gt; {
    const { data } = await supabase
      .from("client_ticket_messages")
      .select("*")
      .eq("ticket_id", ticketId)
      .order("created_at", { ascending: true });
    setMessages(data || []);
    setTimeout(() =&gt; scrollRef.current?.scrollTo({ top: scrollRef.current.scrollHeight }), 100);
  };

  useEffect(() =&gt; {
    fetchTickets().then((tks) =&gt; {
      const urlId = searchParams.get("id");
      if (urlId) {
        const found = tks.find((t: any) =&gt; t.id === urlId);
        if (found) openTicket(found);
      }
    });
    fetchProjects();
  }, []);

  const openTicket = (ticket: any) =&gt; {
    setSelectedTicket(ticket);
    fetchMessages(ticket.id);
  };

  const handleCreate = async () =&gt; {
    if (!form.project_id || !form.subject.trim()) {
      toast.error("Project and subject are required");
      return;
    }
    const proj = projects.find((p) =&gt; p.id === form.project_id);
    const { data: ticket, error } = await supabase
      .from("client_tickets")
      .insert({
        project_id: form.project_id,
        company_id: proj?.company_id,
        subject: form.subject,
        created_by: user?.id,
      })
      .select()
      .single();
    if (error) { toast.error(error.message); return; }
    if (form.message.trim() &amp;amp;&amp;amp; ticket) {
      await supabase.from("client_ticket_messages").insert({
        ticket_id: ticket.id,
        sender_id: user?.id,
        is_from_admin: true,
```

```
      content: form.message,
    });
  }
  toast.success("Ticket created");
  setCreateOpen(false);
  setForm({ project_id: "", subject: "", message: "" });
  fetchTickets();
};

const handleSendMsg = async () =&gt; {
  if (!newMsg.trim() || !selectedTicket) return;
  const { error } = await supabase.from("client_ticket_messages").insert({
    ticket_id: selectedTicket.id,
    sender_id: user?.id,
    is_from_admin: true,
    content: newMsg,
  });
  if (error) { toast.error(error.message); return; }
  await supabase.from("client_tickets").update({ updated_at: new Date().toISOString() }).eq("id", selectedTicket.id);
  setNewMsg("");
  fetchMessages(selectedTicket.id);
};

const handleStatusChange = async (ticketId: string, status: string) =&gt; {
  await supabase.from("client_tickets").update({ status }).eq("id", ticketId);
  fetchTickets();
  if (selectedTicket?.id === ticketId) setSelectedTicket({ ...selectedTicket, status });
};

return (
  &lt;div className="p-6 space-y-6"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Client Tickets&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Manage client support conversations&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={createOpen} onOpenChange={setCreateOpen}&gt;
        &lt;DialogTrigger asChild&gt;
          &lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;New Ticket&lt;/Button&gt;
        &lt;/DialogTrigger&gt;
        &lt;DialogContent&gt;
          &lt;DialogHeader&gt;&lt;DialogTitle&gt;Create Ticket&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            &lt;div&gt;
              &lt;Label&gt;Project *&lt;/Label&gt;
              &lt;Select value={form.project_id} onValueChange={(v) =&gt; setForm({ ...form, project_id: v })}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Select project" /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;{projects.map((p) =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{p.name || "Untitled"}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
              &lt;/Select&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Subject *&lt;/Label&gt;
              &lt;Input value={form.subject} onChange={(e) =&gt; setForm({ ...form, subject: e.target.value })} placeholder="e.g. Concept Design Feedback" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Initial Message&lt;/Label&gt;
              &lt;Textarea value={form.message} onChange={(e) =&gt; setForm({ ...form, message: e.target.value })} placeholder="Optional first message..." /&gt;
            &lt;/div&gt;
            &lt;Button className="w-full" onClick={handleCreate}&gt;Create Ticket&lt;/Button&gt;
          &lt;/div&gt;
        &lt;/DialogContent&gt;
      &lt;/Dialog&gt;
    &lt;/div&gt;

    &lt;div className="grid grid-cols-1 lg:grid-cols-[1fr_1.2fr] gap-6"&gt;
      {/* Ticket list */}
      &lt;div className="space-y-2"&gt;
        {tickets.length === 0 &amp;amp;&amp; (
          &lt;div className="text-center text-muted-foreground py-12"&gt;
            &lt;Ticket className="h-10 w-10 mx-auto mb-2 text-muted-foreground/40" /&gt;
            No tickets yet
          &lt;/div&gt;
        )}
        {tickets.map((t) =&gt; {
          const style = STATUS_STYLES[t.status] || STATUS_STYLES.open;
          const isActive = selectedTicket?.id === t.id;
          return (
            &lt;Card
              key={t.id}
              className={`cursor-pointer transition-colors ${isActive ? "ring-2 ring-primary" : "hover:bg-muted/30"}`}
              onClick={() =&gt; openTicket(t)}
            &gt;
              &lt;CardContent className="flex items-start gap-3 py-3"&gt;
                &lt;div className="p-1.5 rounded-md bg-accent text-accent-foreground mt-0.5"&gt;
                  &lt;MessageSquare className="h-4 w-4" /&gt;
                &lt;/div&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;p className="text-sm font-medium text-foreground line-clamp-1"&gt;{t.subject}&lt;/p&gt;
                  &lt;p className="text-xs text-muted-foreground truncate"&gt;
                    {t.companies?.name} · {t.projects?.name}
                  &lt;/p&gt;
                  &lt;p className="text-[10px] text-muted-foreground mt-1"&gt;
                    {formatDistanceToNow(new Date(t.updated_at), { addSuffix: true })}
                  &lt;/p&gt;
                &lt;/div&gt;
                &lt;Select value={t.status} onValueChange={(v) =&gt; handleStatusChange(t.id, v)}&gt;
                  &lt;SelectTrigger className="w-auto h-6 text-[10px] border-none p-0" onClick={(e) =&gt; e.stopPropagation()}&gt;
                    &lt;Badge variant={style.variant} className="text-[10px] cursor-pointer"&gt;{style.label}&lt;/Badge&gt;
                  &lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;
                    &lt;SelectItem value="open"&gt;Open&lt;/SelectItem&gt;
```

```
                &lt;SelectItem value="waiting"&gt;Waiting&lt;/SelectItem&gt;
                &lt;SelectItem value="resolved"&gt;Resolved&lt;/SelectItem&gt;
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      );
    })}
  &lt;/div&gt;

  {/* Conversation thread */}
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="p-0"&gt;
      {!selectedTicket ? (
        &lt;div className="flex flex-col items-center justify-center py-20 text-muted-foreground"&gt;
          &lt;MessageSquare className="h-10 w-10 mb-2 text-muted-foreground/30" /&gt;
          &lt;p className="text-sm"&gt;Select a ticket to view the conversation&lt;/p&gt;
        &lt;/div&gt;
      ) : (
        &lt;div className="flex flex-col h-[500px]"&gt;
          {/* Thread header */}
          &lt;div className="px-4 py-3 border-b"&gt;
            &lt;h3 className="text-sm font-semibold text-foreground"&gt;{selectedTicket.subject}&lt;/h3&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;
              {selectedTicket.companies?.name} · {selectedTicket.projects?.name}
            &lt;/p&gt;
          &lt;/div&gt;

          {/* Messages */}
          &lt;ScrollArea className="flex-1 px-4 py-3" ref={scrollRef}&gt;
            {messages.length === 0 &amp;amp;&amp; (
              &lt;p className="text-sm text-muted-foreground text-center py-8"&gt;No messages yet&lt;/p&gt;
            )}
            &lt;div className="space-y-3"&gt;
              {messages.map((m) =&gt; (
                &lt;div key={m.id} className={`flex gap-2 ${m.is_from_admin ? "justify-end" : "justify-start"}`}&gt;
                  {!m.is_from_admin &amp;amp;&amp; (
                    &lt;Avatar className="h-7 w-7 shrink-0"&gt;
                      &lt;AvatarFallback className="text-[10px] bg-primary/10 text-primary"&gt;CL&lt;/AvatarFallback&gt;
                    &lt;/Avatar&gt;
                  )}
                  &lt;div className={`max-w-[75%] rounded-lg px-3 py-2 text-sm ${
                    m.is_from_admin
                      ? "bg-primary text-primary-foreground"
                      : "bg-muted text-foreground"
                  }`}&gt;
                    &lt;p&gt;{m.content}&lt;/p&gt;
                    &lt;p className={`text-[10px] mt-1 ${m.is_from_admin ? "text-primary-foreground/70" : "text-muted-foreground"}`}&gt;
                      {format(new Date(m.created_at), "MMM d, h:mm a")}
                    &lt;/p&gt;
                  &lt;/div&gt;
                  {m.is_from_admin &amp;amp;&amp; (
                    &lt;Avatar className="h-7 w-7 shrink-0"&gt;
                      &lt;AvatarFallback className="text-[10px] bg-accent text-accent-foreground"&gt;AD&lt;/AvatarFallback&gt;
                    &lt;/Avatar&gt;
                  )}
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          &lt;/ScrollArea&gt;

          {/* Input */}
          &lt;div className="px-4 py-3 border-t flex gap-2"&gt;
            &lt;Input
              placeholder="Type a reply..."
              value={newMsg}
              onChange={(e) =&gt; setNewMsg(e.target.value)}
              onKeyDown={(e) =&gt; e.key === "Enter" &amp;amp;&amp; !e.shiftKey &amp;amp;&amp; handleSendMsg()}
              className="flex-1"
            /&gt;
            &lt;Button size="icon" onClick={handleSendMsg} disabled={!newMsg.trim()}&gt;
              &lt;Send className="h-4 w-4" /&gt;
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      )}
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;
&lt;/div&gt;
);
};

export default AdminTickets;
```

src/pages/admin/Dashboard.tsx

```
// Re-export existing AdminDashboard as Dashboard
export { default } from "@/pages/admin/AdminDashboard";
```

src/pages/dashboard

src/pages/dashboard/ClientProjectDetail.tsx

```
import { useEffect, useState } from "react";
import { useParams, Link } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { Progress } from "@/components/ui/progress";
import { Skeleton } from "@/components/ui/skeleton";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Textarea } from "@/components/ui/textarea";
import { toast } from "sonner";
import {
  ArrowLeft, MapPin, DollarSign, Clock, CheckCircle2, FileText,
  Download, Send, AlertCircle, Upload,
} from "lucide-react";
import { getStageLabel, PROJECT_STAGE_COLORS } from "@/constants/projectStages";
import ClientTimeline from "@/components/dashboard/ClientTimeline";
import ClientProjectDocuments from "@/components/dashboard/ClientProjectDocuments";

const ClientProjectDetail = () =&gt; {
  const { id } = useParams();
  const { user } = useAuth();
  const [project, setProject] = useState&lt;any&gt;(null);
  const [milestones, setMilestones] = useState&lt;any[]&gt;([]);
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [drawings, setDrawings] = useState&lt;any[]&gt;([]);
  const [messages, setMessages] = useState&lt;any[]&gt;([]);
  const [newMessage, setNewMessage] = useState("");
  const [sending, setSending] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    if (!id) return;
    const fetchAll = async () =&gt; {
      const [pRes, mRes, fRes, dRes, msgRes] = await Promise.all([
        supabase.from("projects").select("*, companies(name)").eq("id", id).single(),
        supabase.from("payment_milestones").select("*").eq("project_id", id).order("created_at"),
        supabase.from("project_files").select("*").eq("project_id", id).order("created_at", { ascending: false }),
        supabase.from("drawing_versions").select("*").eq("project_id", id).eq("is_current", true).order("created_at", { ascending: false }),
        supabase.from("messages").select("*").eq("project_id", id).order("created_at", { ascending: true }),
      ]);
      if (pRes.data) setProject(pRes.data);
      setMilestones(mRes.data || []);
      setFiles(fRes.data || []);
      setDrawings(dRes.data || []);
      setMessages(msgRes.data || []);
      setLoading(false);
    };
    fetchAll();
  }, [id]);

  const sendMessage = async () =&gt; {
    if (!newMessage.trim() || !user || !id) return;
    setSending(true);
    const { error } = await supabase.from("messages").insert({
      project_id: id, content: newMessage.trim(), sender_id: user.id, is_from_admin: false,
    });
    if (error) { toast.error("Failed to send message"); }
    else {
      setMessages(prev =&gt; [...prev, { id: Date.now().toString(), content: newMessage.trim(), sender_id: user.id, is_from_admin: false, created_at: new Date().toISOString(
      setNewMessage("");
      toast.success("Message sent");
    }
    setSending(false);
  };

  if (loading) {
    return &lt;div className="p-6 lg:p-8 space-y-4"&gt;&lt;Skeleton className="h-10 w-64" /&gt;&lt;Skeleton className="h-48" /&gt;&lt;Skeleton className="h-64" /&gt;&lt;/div
  }

  if (!project) {
    return (
      &lt;div className="p-6 lg:p-8 text-center py-20"&gt;
        &lt;AlertCircle className="h-12 w-12 text-muted-foreground/50 mx-auto mb-4" /&gt;
        &lt;p className="text-muted-foreground"&gt;Project not found&lt;/p&gt;
        &lt;Button variant="outline" asChild className="mt-4"&gt;&lt;Link to="/dashboard/projects"&gt;Back to Projects&lt;/Link&gt;&lt;/Button&gt;
      &lt;/div&gt;
    );
  }

  // Stage-based progress
  const stage = project.stage || "briefing";
  const stageLabel = getStageLabel(stage);
  const stageColors = PROJECT_STAGE_COLORS[stage] || { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" };

  // Financial calculations
  const totalValue = Number(project.total_value || 0);
  const totalPaid = milestones.filter(m =&gt; m.status === "paid").reduce((s, m) =&gt; s + Number(m.amount), 0);
  const remaining = totalValue - totalPaid;
  const paymentProgress = totalValue &gt; 0 ? Math.round((totalPaid / totalValue) * 100) : 0;
  const location = [project.display_address, project.city, project.state].filter(Boolean).join(", ");
```

```
return (
  &lt;div className="p-6 lg:p-8 space-y-6 max-w-5xl"&gt;
    {/* Back + Header */}
    &lt;div&gt;
      &lt;Button variant="ghost" size="sm" asChild className="mb-3"&gt;
        &lt;Link to="/dashboard/projects"&gt;&lt;ArrowLeft className="mr-2 h-4 w-4" /&gt;Back to Projects&lt;/Link&gt;
      &lt;/Button&gt;
      &lt;div className="flex flex-col sm:flex-row sm:items-start sm:justify-between gap-4"&gt;
        &lt;div&gt;
          &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;
            {project.display_address || project.name || "Untitled Project"}
          &lt;/h1&gt;
          {location &amp;&amp; (
            &lt;div className="flex items-center gap-1 mt-1 text-muted-foreground"&gt;
              &lt;MapPin className="h-3.5 w-3.5" /&gt;&lt;span className="text-sm"&gt;{location}&lt;/span&gt;
            &lt;/div&gt;
          )}
        &lt;/div&gt;
        &lt;Badge className={`${stageColors.bg} ${stageColors.text} ${stageColors.border} border text-sm px-3 py-1`}&gt;
          &lt;span className={`h-2 w-2 rounded-full ${stageColors.dot} mr-2 inline-block`} /&gt;
          {stageLabel}
        &lt;/Badge&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Financial Summary */}
    {totalValue &gt; 0 &amp;&amp; (
      &lt;div className="grid grid-cols-1 sm:grid-cols-3 gap-4"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-xs text-muted-foreground uppercase tracking-wider mb-1"&gt;Project Value&lt;/p&gt;
            &lt;p className="text-xl font-bold text-foreground"&gt;${totalValue.toLocaleString()}&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-xs text-muted-foreground uppercase tracking-wider mb-1"&gt;Total Paid&lt;/p&gt;
            &lt;p className="text-xl font-bold text-chart-2"&gt;${totalPaid.toLocaleString()}&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-xs text-muted-foreground uppercase tracking-wider mb-1"&gt;Remaining&lt;/p&gt;
            &lt;p className="text-xl font-bold text-foreground"&gt;${remaining.toLocaleString()}&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;
    )}

    {/* Payment Progress */}
    {totalValue &gt; 0 &amp;&amp; (
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-5"&gt;
          &lt;div className="flex items-center justify-between mb-2"&gt;
            &lt;span className="text-sm font-medium text-foreground"&gt;Payment Progress&lt;/span&gt;
            &lt;span class="text-sm font-bold text-primary"&gt;{paymentProgress}%&lt;/span&gt;
          &lt;/div&gt;
          &lt;Progress value={paymentProgress} className="h-2.5" /&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    )}

    {/* Tabs */}
    &lt;Tabs defaultValue="timeline"&gt;
      &lt;TabsList&gt;
        &lt;TabsTrigger value="timeline"&gt;Timeline&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="invoices"&gt;Invoices ({milestones.length})&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="files"&gt;Files ({files.length + drawings.length})&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="messages"&gt;Messages ({messages.length})&lt;/TabsTrigger&gt;
      &lt;/TabsList&gt;

      {/* Timeline Tab */}
      &lt;TabsContent value="timeline" className="mt-4"&gt;
        {id &amp;&amp; &lt;ClientTimeline projectId={id} /&gt;}
      &lt;/TabsContent&gt;

      {/* Invoices Tab */}
      &lt;TabsContent value="invoices" className="mt-4"&gt;
        {milestones.length === 0 ? (
          &lt;Card className="border-dashed"&gt;&lt;CardContent className="py-10 text-center text-muted-foreground"&gt;No invoices yet&lt;/CardContent&gt;&lt;/Card&gt;
        ) : (
          &lt;Card className="border-none shadow-sm overflow-hidden"&gt;
            &lt;Table&gt;
              &lt;TableHeader&gt;
                &lt;TableRow&gt;&lt;TableHead&gt;Milestone&lt;/TableHead&gt;&lt;TableHead&gt;Amount&lt;/TableHead&gt;&lt;TableHead&gt;Status&lt;/TableHead&gt;&lt;TableHead
              &lt;/TableHeader&gt;
              &lt;TableBody&gt;
                {milestones.map(m =&gt; (
                  &lt;TableRow key={m.id}&gt;
                    &lt;TableCell className="font-medium"&gt;{m.stage} ({m.percentage}%)&lt;/TableCell&gt;
                    &lt;TableCell&gt;${Number(m.amount).toLocaleString()}&lt;/TableCell&gt;
                    &lt;TableCell&gt;
                      &lt;Badge variant={m.status === "paid" ? "default" : m.status === "overdue" ? "destructive" : "secondary"}&gt;
                        {m.status === "paid" ? &lt;&gt;&lt;CheckCircle2 className="h-3 w-3 mr-1" /&gt;Paid&lt;/&gt; : m.status === "overdue" ? &lt;&gt;&lt;AlertCircle clas
                      &lt;/Badge&gt;
                    &lt;/TableCell&gt;
                    &lt;TableCell&gt;
                      {m.status !== "paid" &amp;&amp; (
                        &lt;Button size="sm" variant="outline" className="text-xs"&gt;
                          &lt;DollarSign className="h-3 w-3 mr-1" /&gt;Pay Online
                        &lt;/Button&gt;
```

```
                    )}
                  &lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/Card&gt;
      )}
    &lt;/TabsContent&gt;

    {/* Files Tab */}
    &lt;TabsContent value="files" className="mt-4"&gt;
      &lt;ClientProjectDocuments projectId={id!} /&gt;
    &lt;/TabsContent&gt;

    {/* Messages Tab */}
    &lt;TabsContent value="messages" className="mt-4"&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4"&gt;
          &lt;div className="space-y-3 max-h-80 overflow-y-auto mb-4"&gt;
            {messages.length === 0 ? (
              &lt;p className="text-sm text-muted-foreground text-center py-8"&gt;No messages yet. Start a conversation below.&lt;/p&gt;
            ) : (
              messages.map(msg =&gt; (
                &lt;div key={msg.id} className={`flex ${msg.is_from_admin ? "justify-start" : "justify-end"}`}&gt;
                  &lt;div className={`max-w-[75%] rounded-xl px-4 py-2.5 ${msg.is_from_admin ? "bg-muted/50 text-foreground" : "bg-primary text-primary-foreground"}`}&gt
                    &lt;p className="text-sm"&gt;{msg.content}&lt;/p&gt;
                    &lt;p className={`text-[10px] mt-1 ${msg.is_from_admin ? "text-muted-foreground" : "text-primary-foreground/70"}`}&gt;
                      {new Date(msg.created_at).toLocaleString()}
                    &lt;/p&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              ))
            )}
          &lt;/div&gt;
          &lt;div className="flex gap-2"&gt;
            &lt;Textarea placeholder="Type your message..." value={newMessage} onChange={e =&gt; setNewMessage(e.target.value)} rows={2} className="resize-none" /&gt;
            &lt;Button onClick={sendMessage} disabled={sending || !newMessage.trim()} className="self-end"&gt;
              &lt;Send className="h-4 w-4" /&gt;
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/TabsContent&gt;
  &lt;/Tabs&gt;
&lt;/div&gt;
);
};

export default ClientProjectDetail;
```

src/pages/dashboard/Company.tsx

```
import { Card, CardContent } from "@/components/ui/card";
import { Construction } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const Company = () =&gt; {
  const { t } = useLanguage();
  return (
    &lt;div className="p-6"&gt;
      &lt;h1 className="text-2xl font-serif font-bold text-foreground mb-2"&gt;{t("dashboard.companyInfo")}&lt;/h1&gt;
      &lt;p className="text-muted-foreground mb-6"&gt;{t("dashboard.manageCompany")}&lt;/p&gt;
      &lt;Card className="border-dashed"&gt;
        &lt;CardContent className="flex flex-col items-center justify-center py-12"&gt;
          &lt;Construction className="h-12 w-12 text-muted-foreground/50 mb-4" /&gt;
          &lt;h3 className="text-lg font-medium text-foreground mb-2"&gt;{t("dashboard.comingSoon")}&lt;/h3&gt;
          &lt;p className="text-muted-foreground text-center"&gt;{t("dashboard.comingSoonDesc")}&lt;/p&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default Company;
```

src/pages/dashboard/DashboardHome.tsx

```
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { Skeleton } from "@/components/ui/skeleton";
import { Progress } from "@/components/ui/progress";
import {
  FolderOpen, FileText, MessageSquare, Plus, AlertCircle,
  CheckCircle2, Clock, DollarSign, ArrowRight, Upload, TrendingUp,
} from "lucide-react";

interface Project {
  id: string;
  name: string | null;
  display_address: string | null;
  status: string;
  stage: string | null;
  total_value: number | null;
  created_at: string;
  updated_at: string;
}

interface DashboardStats {
  activeProjects: number;
  completedProjects: number;
  pendingInvoices: number;
  totalFiles: number;
  totalPaid: number;
  totalOutstanding: number;
}

const statusProgress: Record&lt;string, number&gt; = {
  pending: 10,
  in_progress: 50,
  under_review: 80,
  completed: 100,
  cancelled: 0,
};

const statusLabel: Record&lt;string, string&gt; = {
  pending: "Proposal",
  in_progress: "In Progress",
  under_review: "Review",
  completed: "Completed",
  cancelled: "Cancelled",
};

const statusIcon: Record&lt;string, typeof Clock&gt; = {
  pending: Clock,
  in_progress: TrendingUp,
  under_review: FileText,
  completed: CheckCircle2,
  cancelled: AlertCircle,
};

const DashboardHome = () =&gt; {
  const { user } = useAuth();
  const { t } = useLanguage();
  const [projects, setProjects] = useState&lt;Project[]&gt;([]);
  const [stats, setStats] = useState&lt;DashboardStats&gt;({
    activeProjects: 0, completedProjects: 0, pendingInvoices: 0,
    totalFiles: 0, totalPaid: 0, totalOutstanding: 0,
  });
  const [recentActivity, setRecentActivity] = useState&lt;Array&lt;{ id: string; text: string; date: string; type: string }&gt;&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetchData = async () =&gt; {
      if (!user) return;
      try {
        setLoading(true);
        const { data: profile } = await supabase.from("profiles").select("company_id").eq("id", user.id).single();
        if (!profile?.company_id) { setLoading(false); return; }

        const { data: projectsData } = await supabase.from("projects")
          .select("id, name, display_address, status, stage, total_value, created_at, updated_at")
          .eq("company_id", profile.company_id).order("updated_at", { ascending: false });

        const prj = projectsData || [];
        setProjects(prj);

        const projectIds = prj.map(p =&gt; p.id);
        const active = prj.filter(p =&gt; p.status !== "completed" &amp;&amp; p.status !== "cancelled").length;
        const completed = prj.filter(p =&gt; p.status === "completed").length;

        let pendingInvoices = 0;
        let totalPaid = 0;
        let totalOutstanding = 0;
        let totalFiles = 0;

        if (projectIds.length &gt; 0) {
          const { data: milestones } = await supabase.from("payment_milestones")
            .select("amount, status").in("project_id", projectIds);
          if (milestones) {
```

```
          pendingInvoices = milestones.filter(m =&gt; m.status === "pending" || m.status === "overdue").length;
          totalPaid = milestones.filter(m =&gt; m.status === "paid").reduce((s, m) =&gt; s + Number(m.amount), 0);
          totalOutstanding = milestones.filter(m =&gt; m.status !== "paid").reduce((s, m) =&gt; s + Number(m.amount), 0);
        }

        const { count: filesCount } = await supabase.from("project_files")
          .select("*", { count: "exact", head: true }).in("project_id", projectIds);
        const { count: drawingsCount } = await supabase.from("drawing_versions")
          .select("*", { count: "exact", head: true }).in("project_id", projectIds);
        totalFiles = (filesCount || 0) + (drawingsCount || 0);

        // Recent activity from messages + files
        const activity: Array&lt;{ id: string; text: string; date: string; type: string }&gt; = [];
        const { data: recentMsgs } = await supabase.from("messages")
          .select("id, content, created_at, is_from_admin")
          .in("project_id", projectIds).order("created_at", { ascending: false }).limit(3);
        recentMsgs?.forEach(m =&gt; activity.push({
          id: m.id, text: m.is_from_admin ? "New message from team" : "You sent a message",
          date: m.created_at, type: "message",
        }));

        const { data: recentFiles } = await supabase.from("project_files")
          .select("id, file_name, created_at")
          .in("project_id", projectIds).order("created_at", { ascending: false }).limit(3);
        recentFiles?.forEach(f =&gt; activity.push({
          id: f.id, text: `File uploaded: ${f.file_name}`, date: f.created_at, type: "file",
        }));

        activity.sort((a, b) =&gt; new Date(b.date).getTime() - new Date(a.date).getTime());
        setRecentActivity(activity.slice(0, 5));
      }

      setStats({ activeProjects: active, completedProjects: completed, pendingInvoices, totalFiles, totalPaid, totalOutstanding });
    } catch (err) {
      console.error("Error fetching dashboard data:", err);
    } finally {
      setLoading(false);
    }
  };
  fetchData();
}, [user]);

if (loading) {
  return (
    &lt;div className="p-6 lg:p-8 space-y-6"&gt;
      &lt;Skeleton className="h-10 w-64" /&gt;
      &lt;div className="grid grid-cols-2 lg:grid-cols-4 gap-4"&gt;{[1,2,3,4].map(i =&gt; &lt;Skeleton key={i} className="h-28" /&gt;)}&lt;/div&gt;
      &lt;div className="grid grid-cols-1 lg:grid-cols-3 gap-6"&gt;{[1,2,3].map(i =&gt; &lt;Skeleton key={i} className="h-64" /&gt;)}&lt;/div&gt;
    &lt;/div&gt;
  );
}

const firstName = user?.user_metadata?.full_name?.split(" ")[0] || "there";

return (
  &lt;div className="p-6 lg:p-8 space-y-8"&gt;
    {/* Header */}
    &lt;div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;Welcome back, {firstName}&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground mt-1"&gt;Here's what's happening with your projects&lt;/p&gt;
      &lt;/div&gt;
      &lt;div className="flex gap-2"&gt;
        &lt;Button variant="outline" asChild&gt;
          &lt;Link to="/dashboard/projects"&gt;&lt;FolderOpen className="mr-2 h-4 w-4" /&gt;My Projects&lt;/Link&gt;
        &lt;/Button&gt;
        &lt;Button asChild&gt;
          &lt;Link to="/dashboard/new-quote"&gt;&lt;Plus className="mr-2 h-4 w-4" /&gt;New Project&lt;/Link&gt;
        &lt;/Button&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    {/* KPI Cards */}
    &lt;div className="grid grid-cols-2 lg:grid-cols-4 gap-4"&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-5"&gt;
          &lt;div className="flex items-center justify-between mb-3"&gt;
            &lt;span className="text-xs font-medium uppercase tracking-wider text-muted-foreground"&gt;Active Projects&lt;/span&gt;
            &lt;div className="h-9 w-9 rounded-lg bg-primary/10 flex items-center justify-center"&gt;
              &lt;FolderOpen className="h-4 w-4 text-primary" /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;p className="text-3xl font-bold text-foreground"&gt;{stats.activeProjects}&lt;/p&gt;
          &lt;p className="text-xs text-muted-foreground mt-1"&gt;{stats.completedProjects} completed&lt;/p&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-5"&gt;
          &lt;div className="flex items-center justify-between mb-3"&gt;
            &lt;span className="text-xs font-medium uppercase tracking-wider text-muted-foreground"&gt;Pending Invoices&lt;/span&gt;
            &lt;div className="h-9 w-9 rounded-lg bg-destructive/10 flex items-center justify-center"&gt;
              &lt;FileText className="h-4 w-4 text-destructive" /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;p className="text-3xl font-bold text-foreground"&gt;{stats.pendingInvoices}&lt;/p&gt;
          &lt;p className="text-xs text-muted-foreground mt-1"&gt;${stats.totalOutstanding.toLocaleString()} outstanding&lt;/p&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      &lt;Card className="border-none shadow-sm"&gt;
```

```
    &lt;CardContent className="p-5"&gt;
      &lt;div className="flex items-center justify-between mb-3"&gt;
        &lt;span className="text-xs font-medium uppercase tracking-wider text-muted-foreground"&gt;Total Paid&lt;/span&gt;
        &lt;div className="h-9 w-9 rounded-lg bg-chart-1/10 flex items-center justify-center"&gt;
          &lt;DollarSign className="h-4 w-4 text-chart-1" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;p className="text-3xl font-bold text-foreground"&gt;${stats.totalPaid.toLocaleString()}&lt;/p&gt;
      &lt;p className="text-xs text-muted-foreground mt-1"&gt;All time&lt;/p&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;

  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="p-5"&gt;
      &lt;div className="flex items-center justify-between mb-3"&gt;
        &lt;span className="text-xs font-medium uppercase tracking-wider text-muted-foreground"&gt;Files&lt;/span&gt;
        &lt;div class="h-9 w-9 rounded-lg bg-secondary/10 flex items-center justify-center"&gt;
          &lt;Upload class="h-4 w-4 text-secondary" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;p class="text-3xl font-bold text-foreground"&gt;{stats.totalFiles}&lt;/p&gt;
      &lt;p class="text-xs text-muted-foreground mt-1"&gt;Drawings &amp; documents&lt;/p&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Main Content Grid */}
&lt;div class="grid grid-cols-1 lg:grid-cols-3 gap-6"&gt;
  {/* Projects List */}
  &lt;div class="lg:col-span-2 space-y-4"&gt;
    &lt;div class="flex items-center justify-between"&gt;
      &lt;h2 class="text-lg font-semibold text-foreground"&gt;Your Projects&lt;/h2&gt;
      &lt;Button variant="ghost" size="sm" asChild&gt;
        &lt;Link to="/dashboard/projects"&gt;View all &lt;ArrowRight class="ml-1 h-3 w-3" /&gt;&lt;/Link&gt;
      &lt;/Button&gt;
    &lt;/div&gt;

    {projects.length === 0 ? (
      &lt;Card class="border-dashed border-2"&gt;
        &lt;CardContent class="flex flex-col items-center justify-center py-16"&gt;
          &lt;div class="h-16 w-16 rounded-2xl bg-muted/50 flex items-center justify-center mb-4"&gt;
            &lt;FolderOpen class="h-8 w-8 text-muted-foreground/50" /&gt;
          &lt;/div&gt;
          &lt;h3 class="text-lg font-medium text-foreground mb-2"&gt;No projects yet&lt;/h3&gt;
          &lt;p class="text-sm text-muted-foreground text-center mb-6 max-w-sm"&gt;
            Submit a new project request and our team will get back to you with a proposal.
          &lt;/p&gt;
          &lt;Button asChild&gt;&lt;Link to="/dashboard/new-quote"&gt;&lt;Plus class="mr-2 h-4 w-4" /&gt;Submit Project Request&lt;/Link&gt;&lt;/Button&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    ) : (
      &lt;div class="space-y-3"&gt;
        {projects.slice(0, 5).map((project) =&gt; {
          const progress = statusProgress[project.status] || 10;
          const label = statusLabel[project.status] || "Pending";
          const Icon = statusIcon[project.status] || Clock;
          return (
            &lt;Link key={project.id} to={`/dashboard/projects/${project.id}`}&gt;
              &lt;Card class="hover:shadow-md transition-shadow border-none shadow-sm cursor-pointer"&gt;
                &lt;CardContent class="p-4"&gt;
                  &lt;div class="flex items-start justify-between gap-3 mb-3"&gt;
                    &lt;div class="flex-1 min-w-0"&gt;
                      &lt;h3 class="font-medium text-foreground truncate"&gt;
                        {project.display_address || project.name || "Untitled Project"}
                      &lt;/h3&gt;
                      &lt;p class="text-xs text-muted-foreground mt-0.5"&gt;
                        Updated {new Date(project.updated_at).toLocaleDateString()}
                      &lt;/p&gt;
                    &lt;/div&gt;
                    &lt;Badge variant={project.status === "completed" ? "default" : project.status === "cancelled" ? "destructive" : "secondary"} class="flex items
                      &lt;Icon class="h-3 w-3" /&gt;{label}
                    &lt;/Badge&gt;
                  &lt;/div&gt;
                  &lt;div class="flex items-center gap-3"&gt;
                    &lt;Progress value={progress} class="h-1.5 flex-1" /&gt;
                    &lt;span class="text-xs font-medium text-muted-foreground w-8 text-right"&gt;{progress}%&lt;/span&gt;
                  &lt;/div&gt;
                &lt;/CardContent&gt;
              &lt;/Card&gt;
            &lt;/Link&gt;
          );
        })}
      &lt;/div&gt;
    )}
  &lt;/div&gt;

  {/* Right Column */}
  &lt;div class="space-y-6"&gt;
    {/* Recent Activity */}
    &lt;Card class="border-none shadow-sm"&gt;
      &lt;CardHeader class="pb-3"&gt;
        &lt;CardTitle class="text-base"&gt;Recent Activity&lt;/CardTitle&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        {recentActivity.length === 0 ? (
          &lt;p class="text-sm text-muted-foreground py-4 text-center"&gt;No recent activity&lt;/p&gt;
        ) : (
          &lt;div class="space-y-3"&gt;
            {recentActivity.map((activity) =&gt; (
              &lt;div key={activity.id} class="flex items-start gap-3"&gt;
```

```
                &lt;div className="h-7 w-7 rounded-full bg-muted/50 flex items-center justify-center flex-shrink-0 mt-0.5"&gt;
                  {activity.type === "message" ? (
                    &lt;MessageSquare className="h-3.5 w-3.5 text-muted-foreground" /&gt;
                  ) : (
                    &lt;Upload className="h-3.5 w-3.5 text-muted-foreground" /&gt;
                  )}
                &lt;/div&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;p className="text-sm text-foreground truncate"&gt;{activity.text}&lt;/p&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;
                    {new Date(activity.date).toLocaleDateString()}
                  &lt;/p&gt;
                &lt;/div&gt;
              &lt;/div&gt;
            ))}
          &lt;/div&gt;
        )}
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Quick Actions */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;CardTitle className="text-base"&gt;Quick Actions&lt;/CardTitle&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="space-y-2"&gt;
        &lt;Button variant="outline" className="w-full justify-start" asChild&gt;
          &lt;Link to="/dashboard/new-quote"&gt;&lt;Plus className="mr-2 h-4 w-4" /&gt;New Project Request&lt;/Link&gt;
        &lt;/Button&gt;
        &lt;Button variant="outline" className="w-full justify-start" asChild&gt;
          &lt;Link to="/dashboard/invoices"&gt;&lt;FileText className="mr-2 h-4 w-4" /&gt;View Invoices&lt;/Link&gt;
        &lt;/Button&gt;
        &lt;Button variant="outline" className="w-full justify-start" asChild&gt;
          &lt;Link to="/dashboard/messages"&gt;&lt;MessageSquare className="mr-2 h-4 w-4" /&gt;Send Message&lt;/Link&gt;
        &lt;/Button&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/div&gt;
&lt;/div&gt;
&lt;/div&gt;
);
};

export default DashboardHome;
```

src/pages/dashboard/Files.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Skeleton } from "@/components/ui/skeleton";
import { Badge } from "@/components/ui/badge";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Download, FileText, PenTool, FolderOpen, ChevronRight } from "lucide-react";
import { format } from "date-fns";

const DOCUMENT_CATEGORIES = [
  { value: "pre_construction", label: "Pre-Construction" },
  { value: "rev_00", label: "REV 00" },
  { value: "rev_01", label: "REV 01" },
  { value: "rev_02", label: "REV 02" },
  { value: "rev_03", label: "REV 03 (Final)" },
  { value: "final_delivery", label: "Final Delivery" },
  { value: "general", label: "General" },
] as const;

interface FileItem {
  id: string;
  name: string;
  type: "drawing" | "file";
  project_name: string;
  date: string;
  url: string;
  version?: number;
  size?: number | null;
  document_category?: string;
}

const Files = () =&gt; {
  const { user } = useAuth();
  const [files, setFiles] = useState&lt;FileItem[]&gt;([]);
  const [loading, setLoading] = useState(true);
  const [openFolders, setOpenFolders] = useState&lt;Set&lt;string&gt;&gt;(new Set(DOCUMENT_CATEGORIES.map(c =&gt; c.value)));

  useEffect(() =&gt; {
    const fetchData = async () =&gt; {
      if (!user) return;
      const { data: profile } = await supabase.from("profiles").select("company_id").eq("id", user.id).single();
      if (!profile?.company_id) { setLoading(false); return; }
      const { data: projects } = await supabase.from("projects")
        .select("id, name, display_address").eq("company_id", profile.company_id);
      if (!projects || projects.length === 0) { setLoading(false); return; }
      const ids = projects.map(p =&gt; p.id);
      const projectMap = Object.fromEntries(projects.map(p =&gt; [p.id, p.display_address || p.name || "Untitled"]));

      const [fRes, dRes] = await Promise.all([
        supabase.from("project_files").select("*").in("project_id", ids).order("created_at", { ascending: false }),
        supabase.from("drawing_versions").select("*").in("project_id", ids).eq("is_current", true).order("created_at", { ascending: false }),
      ]);

      const items: FileItem[] = [];
      (fRes.data || []).forEach(f =&gt; items.push({
        id: f.id, name: f.file_name, type: "file", project_name: projectMap[f.project_id] || "Unknown",
        date: f.created_at, url: f.file_url, size: f.file_size,
        document_category: (f as any).document_category || "general",
      }));
      (dRes.data || []).forEach(d =&gt; items.push({
        id: d.id, name: d.drawing_name, type: "drawing", project_name: projectMap[d.project_id] || "Unknown",
        date: d.created_at, url: d.file_url, version: d.version_number,
        document_category: "drawings",
      }));
      setFiles(items);
      setLoading(false);
    };
    fetchData();
  }, [user]);

  const toggleFolder = (cat: string) =&gt; {
    setOpenFolders(prev =&gt; {
      const next = new Set(prev);
      next.has(cat) ? next.delete(cat) : next.add(cat);
      return next;
    });
  };

  if (loading) return &lt;div className="p-6 lg:p-8 space-y-4"&gt;&lt;Skeleton className="h-10 w-48" /&gt;&lt;Skeleton className="h-64" /&gt;&lt;/div&gt;;

  const drawingFiles = files.filter(f =&gt; f.type === "drawing");
  const docFiles = files.filter(f =&gt; f.type === "file");

  const grouped = DOCUMENT_CATEGORIES.reduce((acc, cat) =&gt; {
    acc[cat.value] = docFiles.filter(f =&gt; (f.document_category || "general") === cat.value);
    return acc;
  }, {} as Record&lt;string, FileItem[]&gt;);

  const nonEmptyFolders = DOCUMENT_CATEGORIES.filter(cat =&gt; grouped[cat.value].length &gt; 0);

  const formatSize = (bytes: number | null | undefined) =&gt; {
    if (!bytes) return "";
    if (bytes &lt; 1024) return `${bytes} B`;
    if (bytes &lt; 1048576) return `${(bytes / 1024).toFixed(1)} KB`;
    return `${(bytes / 1048576).toFixed(1)} MB`;
  };
```

```
  return (
    &lt;div className="p-6 lg:p-8 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;Files&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{files.length} files available&lt;/p&gt;
      &lt;/div&gt;

      {files.length === 0 ? (
        &lt;Card className="border-dashed border-2"&gt;
          &lt;CardContent className="flex flex-col items-center justify-center py-16"&gt;
            &lt;FolderOpen className="h-12 w-12 text-muted-foreground/40 mb-4" /&gt;
            &lt;p className="text-muted-foreground"&gt;No files available yet&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      ) : (
        &lt;div className="space-y-4"&gt;
          {/* Drawings */}
          {drawingFiles.length &gt; 0 &amp;&amp; (
            &lt;Card className="border-none shadow-sm"&gt;
              &lt;CardHeader className="pb-2"&gt;
                &lt;CardTitle className="text-sm"&gt;Drawings&lt;/CardTitle&gt;
              &lt;/CardHeader&gt;
              &lt;CardContent className="p-0 pb-2"&gt;
                {drawingFiles.map(f =&gt; (
                  &lt;div key={f.id} className="flex items-center gap-3 px-4 py-2 hover:bg-muted/30 transition-colors"&gt;
                    &lt;PenTool className="h-4 w-4 text-primary shrink-0" /&gt;
                    &lt;div className="flex-1 min-w-0"&gt;
                      &lt;p className="text-sm font-medium text-foreground truncate"&gt;{f.name}&lt;/p&gt;
                      &lt;p className="text-[11px] text-muted-foreground"&gt;
                        {f.version &amp;&amp; `v${f.version} · `}{f.project_name} · {format(new Date(f.date), "MMM d, yyyy")}
                      &lt;/p&gt;
                    &lt;/div&gt;
                    &lt;Button size="sm" variant="ghost" asChild&gt;
                      &lt;a href={f.url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-4 w-4" /&gt;&lt;/a&gt;
                    &lt;/Button&gt;
                  &lt;/div&gt;
                ))}
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          )}

          {/* Document Folders */}
          {nonEmptyFolders.length &gt; 0 &amp;&amp; (
            &lt;Card className="border-none shadow-sm"&gt;
              &lt;CardHeader className="pb-2"&gt;
                &lt;div className="flex items-center justify-between"&gt;
                  &lt;CardTitle className="text-sm"&gt;Documents&lt;/CardTitle&gt;
                  &lt;Badge variant="secondary" className="text-[10px]"&gt;{docFiles.length} files&lt;/Badge&gt;
                &lt;/div&gt;
              &lt;/CardHeader&gt;
              &lt;CardContent className="p-0"&gt;
                &lt;div className="divide-y divide-border"&gt;
                  {nonEmptyFolders.map(cat =&gt; {
                    const catFiles = grouped[cat.value];
                    const isOpen = openFolders.has(cat.value);
                    return (
                      &lt;Collapsible key={cat.value} open={isOpen} onOpenChange={() =&gt; toggleFolder(cat.value)}&gt;
                        &lt;CollapsibleTrigger className="flex items-center gap-3 w-full px-4 py-3 hover:bg-muted/50 transition-colors text-left"&gt;
                          &lt;ChevronRight className={`h-4 w-4 text-muted-foreground transition-transform ${isOpen ? "rotate-90" : ""}`} /&gt;
                          &lt;FolderOpen className="h-4 w-4 text-primary" /&gt;
                          &lt;span className="text-sm font-medium text-foreground flex-1"&gt;{cat.label}&lt;/span&gt;
                          &lt;Badge variant="outline" className="text-[10px]"&gt;{catFiles.length}&lt;/Badge&gt;
                        &lt;/CollapsibleTrigger&gt;
                        &lt;CollapsibleContent&gt;
                          &lt;div className="pb-2"&gt;
                            {catFiles.map(f =&gt; (
                              &lt;div key={f.id} className="flex items-center gap-3 px-12 py-2 hover:bg-muted/30 transition-colors"&gt;
                                &lt;FileText className="h-4 w-4 text-muted-foreground shrink-0" /&gt;
                                &lt;div className="flex-1 min-w-0"&gt;
                                  &lt;p className="text-sm font-medium text-foreground truncate"&gt;{f.name}&lt;/p&gt;
                                  &lt;p className="text-[11px] text-muted-foreground"&gt;
                                    {formatSize(f.size)} · {f.project_name} · {format(new Date(f.date), "MMM d, yyyy")}
                                  &lt;/p&gt;
                                &lt;/div&gt;
                                &lt;Button variant="ghost" size="icon" className="h-7 w-7 shrink-0" asChild&gt;
                                  &lt;a href={f.url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-3.5 w-3.5" /&gt;&lt;/a&gt;
                                &lt;/Button&gt;
                              &lt;/div&gt;
                            ))}
                          &lt;/div&gt;
                        &lt;/CollapsibleContent&gt;
                      &lt;/Collapsible&gt;
                    );
                  ))}
                &lt;/div&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          )}
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
};

export default Files;
```

src/pages/dashboard/Invoices.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Skeleton } from "@/components/ui/skeleton";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { CheckCircle2, Clock, AlertCircle, ExternalLink, FileText, DollarSign } from "lucide-react";

interface Invoice {
  id: string;
  stage: string;
  amount: number;
  percentage: number;
  status: string;
  payment_date: string | null;
  project_id: string;
  project_name: string;
}

const Invoices = () =&gt; {
  const { user } = useAuth();
  const [invoices, setInvoices] = useState&lt;Invoice[]&gt;([]);
  const [loading, setLoading] = useState(true);
  const [totalPaid, setTotalPaid] = useState(0);
  const [totalPending, setTotalPending] = useState(0);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      if (!user) return;
      const { data: profile } = await supabase.from("profiles").select("company_id").eq("id", user.id).single();
      if (!profile?.company_id) { setLoading(false); return; }
      const { data: projects } = await supabase.from("projects")
        .select("id, name, display_address").eq("company_id", profile.company_id);
      if (!projects || projects.length === 0) { setLoading(false); return; }
      const ids = projects.map(p =&gt; p.id);
      const { data: milestones } = await supabase.from("payment_milestones")
        .select("*").in("project_id", ids).order("created_at");
      const projectMap = Object.fromEntries(projects.map(p =&gt; [p.id, p.display_address || p.name || "Untitled"]));
      const inv: Invoice[] = (milestones || []).map(m =&gt; ({
        ...m, project_name: projectMap[m.project_id] || "Unknown",
      }));
      setInvoices(inv);
      setTotalPaid(inv.filter(i =&gt; i.status === "paid").reduce((s, i) =&gt; s + Number(i.amount), 0));
      setTotalPending(inv.filter(i =&gt; i.status !== "paid").reduce((s, i) =&gt; s + Number(i.amount), 0));
      setLoading(false);
    };
    fetch();
  }, [user]);

  if (loading) return &lt;div className="p-6 lg:p-8 space-y-4"&gt;&lt;Skeleton className="h-10 w-48" /&gt;&lt;Skeleton className="h-64" /&gt;&lt;/div&gt;;

  return (
    &lt;div className="p-6 lg:p-8 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;Invoices&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{invoices.length} total invoices&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="grid grid-cols-1 sm:grid-cols-3 gap-4"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="h-10 w-10 rounded-lg bg-chart-1/10 flex items-center justify-center"&gt;
              &lt;DollarSign className="h-5 w-5 text-chart-1" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-xs text-muted-foreground"&gt;Total Paid&lt;/p&gt;
              &lt;p className="text-lg font-bold text-foreground"&gt;${totalPaid.toLocaleString()}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="h-10 w-10 rounded-lg bg-destructive/10 flex items-center justify-center"&gt;
              &lt;Clock className="h-5 w-5 text-destructive" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-xs text-muted-foreground"&gt;Outstanding&lt;/p&gt;
              &lt;p className="text-lg font-bold text-foreground"&gt;${totalPending.toLocaleString()}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="h-10 w-10 rounded-lg bg-primary/10 flex items-center justify-center"&gt;
              &lt;FileText className="h-5 w-5 text-primary" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-xs text-muted-foreground"&gt;Total Invoices&lt;/p&gt;
              &lt;p className="text-lg font-bold text-foreground"&gt;{invoices.length}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;

      {invoices.length === 0 ? (
        &lt;Card className="border-dashed"&gt;&lt;CardContent className="py-10 text-center text-muted-foreground"&gt;No invoices yet&lt;/CardContent&gt;&lt;/Card&gt;
```

```
      ) : (
        &lt;Card className="border-none shadow-sm overflow-hidden"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Invoice&lt;/TableHead&gt;
                &lt;TableHead&gt;Project&lt;/TableHead&gt;
                &lt;TableHead&gt;Amount&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {invoices.map(inv =&gt; (
                &lt;TableRow key={inv.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{inv.stage} ({inv.percentage}%)&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{inv.project_name}&lt;/TableCell&gt;
                  &lt;TableCell className="font-medium"&gt;${Number(inv.amount).toLocaleString()}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    &lt;Badge variant={inv.status === "paid" ? "default" : inv.status === "overdue" ? "destructive" : "secondary"}&gt;
                      {inv.status === "paid" ? &lt;&gt;&lt;CheckCircle2 className="h-3 w-3 mr-1" /&gt;Paid&lt;/&gt; : inv.status === "overdue" ? &lt;&gt;&lt;AlertCircle clas
                    &lt;/Badge&gt;
                  &lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    {inv.status !== "paid" &amp;&amp; (
                      &lt;Button size="sm" variant="outline" className="text-xs"&gt;
                        &lt;ExternalLink className="h-3 w-3 mr-1" /&gt;Pay Online
                      &lt;/Button&gt;
                    )}
                  &lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/Card&gt;
      )}
    &lt;/div&gt;
  );
};

export default Invoices;
```

src/pages/dashboard/Messages.tsx

```
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Skeleton } from "@/components/ui/skeleton";
import { Badge } from "@/components/ui/badge";
import { MessageSquare, ArrowRight, FolderOpen } from "lucide-react";

interface ProjectThread {
  project_id: string;
  project_name: string;
  lastMessage: string;
  lastDate: string;
  unread: number;
  total: number;
}

const Messages = () =&gt; {
  const { user } = useAuth();
  const [threads, setThreads] = useState&lt;ProjectThread[]&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      if (!user) return;
      const { data: profile } = await supabase.from("profiles").select("company_id").eq("id", user.id).single();
      if (!profile?.company_id) { setLoading(false); return; }
      const { data: projects } = await supabase.from("projects")
        .select("id, name, display_address").eq("company_id", profile.company_id);
      if (!projects || projects.length === 0) { setLoading(false); return; }
      const ids = projects.map(p =&gt; p.id);
      const { data: msgs } = await supabase.from("messages").select("*").in("project_id", ids).order("created_at", { ascending: false });
      const projectMap = Object.fromEntries(projects.map(p =&gt; [p.id, p.display_address || p.name || "Untitled"]));
      const threadMap: Record&lt;string, ProjectThread&gt; = {};
      (msgs || []).forEach(m =&gt; {
        if (!threadMap[m.project_id]) {
          threadMap[m.project_id] = {
            project_id: m.project_id,
            project_name: projectMap[m.project_id] || "Unknown",
            lastMessage: m.content,
            lastDate: m.created_at,
            unread: 0,
            total: 0,
          };
        }
        threadMap[m.project_id].total++;
      });
      setThreads(Object.values(threadMap).sort((a, b) =&gt; new Date(b.lastDate).getTime() - new Date(a.lastDate).getTime()));
      setLoading(false);
    };
    fetch();
  }, [user]);

  if (loading) return &lt;div className="p-6 lg:p-8 space-y-4"&gt;&lt;Skeleton className="h-10 w-48" /&gt;&lt;Skeleton className="h-64" /&gt;&lt;/div&gt;;

  return (
    &lt;div className="p-6 lg:p-8 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;Messages&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Communicate with our team about your projects&lt;/p&gt;
      &lt;/div&gt;

      {threads.length === 0 ? (
        &lt;Card className="border-dashed border-2"&gt;
          &lt;CardContent className="flex flex-col items-center justify-center py-16"&gt;
            &lt;MessageSquare className="h-12 w-12 text-muted-foreground/40 mb-4" /&gt;
            &lt;h3 className="text-lg font-medium text-foreground mb-2"&gt;No messages yet&lt;/h3&gt;
            &lt;p className="text-sm text-muted-foreground text-center mb-4"&gt;Messages will appear here once you have active projects.&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      ) : (
        &lt;div className="space-y-2"&gt;
          {threads.map(thread =&gt; (
            &lt;Link key={thread.project_id} to={`/dashboard/projects/${thread.project_id}`}&gt;
              &lt;Card className="hover:shadow-md transition-shadow border-none shadow-sm cursor-pointer"&gt;
                &lt;CardContent className="p-4 flex items-center justify-between gap-3"&gt;
                  &lt;div className="flex items-center gap-3 flex-1 min-w-0"&gt;
                    &lt;div className="h-10 w-10 rounded-lg bg-primary/10 flex items-center justify-center flex-shrink-0"&gt;
                      &lt;FolderOpen className="h-5 w-5 text-primary" /&gt;
                    &lt;/div&gt;
                    &lt;div className="flex-1 min-w-0"&gt;
                      &lt;div className="flex items-center gap-2"&gt;
                        &lt;h3 className="font-medium text-foreground truncate"&gt;{thread.project_name}&lt;/h3&gt;
                        &lt;Badge variant="secondary" className="text-[10px]"&gt;{thread.total} msgs&lt;/Badge&gt;
                      &lt;/div&gt;
                      &lt;p className="text-xs text-muted-foreground truncate mt-0.5"&gt;{thread.lastMessage}&lt;/p&gt;
                    &lt;/div&gt;
                  &lt;/div&gt;
                  &lt;div className="flex items-center gap-2 flex-shrink-0"&gt;
                    &lt;span className="text-xs text-muted-foreground"&gt;{new Date(thread.lastDate).toLocaleDateString()}&lt;/span&gt;
                    &lt;ArrowRight className="h-4 w-4 text-muted-foreground" /&gt;
                  &lt;/div&gt;
                &lt;/CardContent&gt;
              &lt;/Card&gt;
            &lt;/Link&gt;
          ))}
```

```
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
};

export default Messages;
```

src/pages/dashboard/NewQuote.tsx

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { toast } from "sonner";
import { z } from "zod";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { ArrowLeft, Send, Loader2 } from "lucide-react";
import { Link } from "react-router-dom";

const quoteSchema = z.object({
  projectAddress: z.string().min(1, "Project address is required"),
  projectCity: z.string().min(1, "City is required"),
  projectState: z.string().min(1, "State is required"),
  projectType: z.string().min(1, "Project type is required"),
  estimatedSize: z.string().min(1, "Estimated size is required"),
  description: z.string().optional(),
});

type QuoteFormData = z.infer&lt;typeof quoteSchema&gt;;

const NewQuote = () =&gt; {
  const { user } = useAuth();
  const { t } = useLanguage();
  const navigate = useNavigate();
  const [isSubmitting, setIsSubmitting] = useState(false);

  const PROJECT_TYPES_KEYS = [
    "projectType.singleFamily", "projectType.multiFamily", "projectType.commercial",
    "projectType.renovation", "projectType.addition", "projectType.adu",
    "projectType.deck", "projectType.garage", "projectType.other",
  ];

  const { register, handleSubmit, setValue, formState: { errors } } = useForm&lt;QuoteFormData&gt;({
    resolver: zodResolver(quoteSchema),
    defaultValues: { projectAddress: "", projectCity: "", projectState: "", projectType: "", estimatedSize: "", description: "" },
  });

  const onSubmit = async (data: QuoteFormData) =&gt; {
    if (!user) { toast.error("You must be logged in"); return; }
    setIsSubmitting(true);
    try {
      const fullAddress = `${data.projectAddress}, ${data.projectCity}, ${data.projectState}`;
      const description = `PROJECT TYPE: ${data.projectType}\nESTIMATED SIZE: ${data.estimatedSize} sq ft\nADDRESS: ${fullAddress}\n\nADDITIONAL NOTES:\n${data.description |
      const { error } = await supabase.from("quote_requests").insert({
        full_name: user.user_metadata?.full_name || user.email || "Client",
        email: user.email || "",
        phone: null, company_name: null, service_type: "technical_drafting",
        project_address: fullAddress, description,
      });
      if (error) throw error;
      toast.success("Quote request submitted successfully!");
      navigate("/dashboard");
    } catch (err) {
      console.error("Error submitting quote:", err);
      toast.error("Failed to submit quote request.");
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    &lt;div className="p-6 max-w-2xl mx-auto"&gt;
      &lt;div className="mb-6"&gt;
        &lt;Button variant="ghost" asChild className="mb-4"&gt;
          &lt;Link to="/dashboard"&gt;&lt;ArrowLeft className="mr-2 h-4 w-4" /&gt;{t("dashboard.backToDashboard")}&lt;/Link&gt;
        &lt;/Button&gt;
        &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;{t("newQuote.title")}&lt;/h1&gt;
        &lt;p className="text-muted-foreground"&gt;{t("newQuote.subtitle")}&lt;/p&gt;
      &lt;/div&gt;
      &lt;Card&gt;
        &lt;CardHeader&gt;
          &lt;CardTitle&gt;{t("newQuote.projectDetails")}&lt;/CardTitle&gt;
          &lt;CardDescription&gt;{t("newQuote.projectDetailsDesc")}&lt;/CardDescription&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          &lt;form onSubmit={handleSubmit(onSubmit)} className="space-y-6"&gt;
            &lt;div className="space-y-2"&gt;
              &lt;Label htmlFor="projectAddress"&gt;{t("newQuote.projectAddress")} *&lt;/Label&gt;
              &lt;Input id="projectAddress" placeholder="123 Main Street" {...register("projectAddress")} /&gt;
              {errors.projectAddress &amp;&amp; &lt;p className="text-sm text-destructive"&gt;{errors.projectAddress.message}&lt;/p&gt;}
            &lt;/div&gt;
            &lt;div className="grid grid-cols-2 gap-4"&gt;
              &lt;div className="space-y-2"&gt;
                &lt;Label htmlFor="projectCity"&gt;{t("newQuote.city")} *&lt;/Label&gt;
                &lt;Input id="projectCity" placeholder="Boston" {...register("projectCity")} /&gt;
                {errors.projectCity &amp;&amp; &lt;p className="text-sm text-destructive"&gt;{errors.projectCity.message}&lt;/p&gt;}
              &lt;/div&gt;
              &lt;div className="space-y-2"&gt;
                &lt;Label htmlFor="projectState"&gt;{t("newQuote.state")} *&lt;/Label&gt;
```

```
                &lt;Input id="projectState" placeholder="MA" {...register("projectState")} /&gt;
                {errors.projectState &amp;&amp; &lt;p className="text-sm text-destructive"&gt;{errors.projectState.message}&lt;/p&gt;}
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div className="space-y-2"&gt;
              &lt;Label htmlFor="projectType"&gt;{t("newQuote.projectType")} *&lt;/Label&gt;
              &lt;Select onValueChange={(value) =&gt; setValue("projectType", value)}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue placeholder={t("newQuote.selectProjectType")} /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;
                  {PROJECT_TYPES_KEYS.map((key) =&gt; (&lt;SelectItem key={key} value={t(key)}&gt;{t(key)}&lt;/SelectItem&gt;))}
                &lt;/SelectContent&gt;
              &lt;/Select&gt;
              {errors.projectType &amp;&amp; &lt;p className="text-sm text-destructive"&gt;{errors.projectType.message}&lt;/p&gt;}
            &lt;/div&gt;
            &lt;div className="space-y-2"&gt;
              &lt;Label htmlFor="estimatedSize"&gt;{t("newQuote.estimatedSize")} *&lt;/Label&gt;
              &lt;Input id="estimatedSize" placeholder="2000" type="number" {...register("estimatedSize")} /&gt;
              {errors.estimatedSize &amp;&amp; &lt;p className="text-sm text-destructive"&gt;{errors.estimatedSize.message}&lt;/p&gt;}
            &lt;/div&gt;
            &lt;div className="space-y-2"&gt;
              &lt;Label htmlFor="description"&gt;{t("newQuote.additionalNotes")}&lt;/Label&gt;
              &lt;Textarea id="description" placeholder={t("newQuote.additionalNotesPlaceholder")} rows={4} {...register("description")} /&gt;
            &lt;/div&gt;
            &lt;div className="flex justify-end gap-4 pt-4"&gt;
              &lt;Button type="button" variant="outline" asChild&gt;&lt;Link to="/dashboard"&gt;{t("newQuote.cancel")}&lt;/Link&gt;&lt;/Button&gt;
              &lt;Button type="submit" disabled={isSubmitting}&gt;
                {isSubmitting ? (&lt;&gt;&lt;Loader2 className="mr-2 h-4 w-4 animate-spin" /&gt;{t("newQuote.submitting")}&lt;/&gt;) : (&lt;&gt;&lt;Send className="mr-2 h-4
              &lt;/Button&gt;
            &lt;/div&gt;
          &lt;/form&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default NewQuote;
```

src/pages/dashboard/Profile.tsx

```
import ProfileEditor from "@/components/profile/ProfileEditor";

const Profile = () =&gt; {
  return &lt;ProfileEditor /&gt;;
};

export default Profile;
```

src/pages/dashboard/Projects.tsx

```
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { Progress } from "@/components/ui/progress";
import { Skeleton } from "@/components/ui/skeleton";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import {
  FolderOpen, Plus, Clock, TrendingUp, FileText, CheckCircle2, AlertCircle, MapPin,
} from "lucide-react";

interface Project {
  id: string;
  name: string | null;
  display_address: string | null;
  city: string | null;
  state: string | null;
  status: string;
  total_value: number | null;
  service_type: string;
  created_at: string;
  updated_at: string;
}

const statusProgress: Record&lt;string, number&gt; = {
  pending: 10, in_progress: 50, under_review: 80, completed: 100, cancelled: 0,
};
const statusLabel: Record&lt;string, string&gt; = {
  pending: "Proposal", in_progress: "In Progress", under_review: "Review", completed: "Completed", cancelled: "Cancelled",
};
const statusIcon: Record&lt;string, typeof Clock&gt; = {
  pending: Clock, in_progress: TrendingUp, under_review: FileText, completed: CheckCircle2, cancelled: AlertCircle,
};

const Projects = () =&gt; {
  const { user } = useAuth();
  const [projects, setProjects] = useState&lt;Project[]&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      if (!user) return;
      const { data: profile } = await supabase.from("profiles").select("company_id").eq("id", user.id).single();
      if (!profile?.company_id) { setLoading(false); return; }
      const { data } = await supabase.from("projects")
        .select("id, name, display_address, city, state, status, total_value, service_type, created_at, updated_at")
        .eq("company_id", profile.company_id).order("updated_at", { ascending: false });
      setProjects(data || []);
      setLoading(false);
    };
    fetch();
  }, [user]);

  if (loading) {
    return &lt;div className="p-6 lg:p-8 space-y-4"&gt;&lt;Skeleton className="h-10 w-48" /&gt;&lt;div className="space-y-3"&gt;{[1,2,3].map(i =&gt; &lt;Skeleton key={i} cla
  }

  const active = projects.filter(p =&gt; p.status !== "completed" &amp;&amp; p.status !== "cancelled");
  const completed = projects.filter(p =&gt; p.status === "completed");
  const all = projects;

  const renderProjects = (list: Project[]) =&gt; {
    if (list.length === 0) {
      return (
        &lt;Card className="border-dashed border-2"&gt;
          &lt;CardContent className="flex flex-col items-center justify-center py-16"&gt;
            &lt;FolderOpen className="h-12 w-12 text-muted-foreground/40 mb-4" /&gt;
            &lt;p className="text-muted-foreground"&gt;No projects found&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      );
    }
    return (
      &lt;div className="space-y-3"&gt;
        {list.map(project =&gt; {
          const progress = statusProgress[project.status] || 10;
          const label = statusLabel[project.status] || "Pending";
          const Icon = statusIcon[project.status] || Clock;
          const location = [project.city, project.state].filter(Boolean).join(", ");
          return (
            &lt;Link key={project.id} to={`/dashboard/projects/${project.id}`}&gt;
              &lt;Card className="hover:shadow-md transition-shadow border-none shadow-sm cursor-pointer"&gt;
                &lt;CardContent className="p-5"&gt;
                  &lt;div className="flex items-start justify-between gap-3 mb-3"&gt;
                    &lt;div className="flex-1 min-w-0"&gt;
                      &lt;h3 className="font-medium text-foreground"&gt;
                        {project.display_address || project.name || "Untitled Project"}
                      &lt;/h3&gt;
                      {location &amp;&amp; (
                        &lt;div className="flex items-center gap-1 mt-1 text-muted-foreground"&gt;
                          &lt;MapPin className="h-3 w-3" /&gt;&lt;span className="text-xs"&gt;{location}&lt;/span&gt;
                        &lt;/div&gt;
                      )}
                    &lt;/div&gt;
                    &lt;div className="flex items-center gap-2 flex-shrink-0"&gt;
```

```
                    {project.total_value ? (
                      &lt;span className="text-sm font-medium text-foreground"&gt;${Number(project.total_value).toLocaleString()}&lt;/span&gt;
                    ) : null}
                    &lt;Badge variant={project.status === "completed" ? "default" : project.status === "cancelled" ? "destructive" : "secondary"} className="flex items-cen
                      &lt;Icon className="h-3 w-3" /&gt;{label}
                    &lt;/Badge&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
                &lt;div className="flex items-center gap-3"&gt;
                  &lt;Progress value={progress} className="h-1.5 flex-1" /&gt;
                  &lt;span className="text-xs font-medium text-muted-foreground w-8 text-right"&gt;{progress}%&lt;/span&gt;
                &lt;/div&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          &lt;/Link&gt;
        );
      })}
    &lt;/div&gt;
  );
};

return (
  &lt;div className="p-6 lg:p-8 space-y-6"&gt;
    &lt;div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;My Projects&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{projects.length} total projects&lt;/p&gt;
      &lt;/div&gt;
      &lt;Button asChild&gt;&lt;Link to="/dashboard/new-quote"&gt;&lt;Plus className="mr-2 h-4 w-4" /&gt;New Project&lt;/Link&gt;&lt;/Button&gt;
    &lt;/div&gt;

    &lt;Tabs defaultValue="active"&gt;
      &lt;TabsList&gt;
        &lt;TabsTrigger value="active"&gt;Active ({active.length})&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="completed"&gt;Completed ({completed.length})&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="all"&gt;All ({all.length})&lt;/TabsTrigger&gt;
      &lt;/TabsList&gt;
      &lt;TabsContent value="active" className="mt-4"&gt;{renderProjects(active)}&lt;/TabsContent&gt;
      &lt;TabsContent value="completed" className="mt-4"&gt;{renderProjects(completed)}&lt;/TabsContent&gt;
      &lt;TabsContent value="all" className="mt-4"&gt;{renderProjects(all)}&lt;/TabsContent&gt;
    &lt;/Tabs&gt;
  &lt;/div&gt;
);
};

export default Projects;
```

src/pages/dashboard/Quotes.tsx

```
import { Card, CardContent } from "@/components/ui/card";
import { Construction } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const Quotes = () =&gt; {
  const { t } = useLanguage();
  return (
    &lt;div className="p-6"&gt;
      &lt;h1 className="text-2xl font-serif font-bold text-foreground mb-2"&gt;{t("dashboard.quotesInvoices")}&lt;/h1&gt;
      &lt;p className="text-muted-foreground mb-6"&gt;{t("dashboard.viewQuotes")}&lt;/p&gt;
      &lt;Card className="border-dashed"&gt;
        &lt;CardContent className="flex flex-col items-center justify-center py-12"&gt;
          &lt;Construction className="h-12 w-12 text-muted-foreground/50 mb-4" /&gt;
          &lt;h3 className="text-lg font-medium text-foreground mb-2"&gt;{t("dashboard.comingSoon")}&lt;/h3&gt;
          &lt;p className="text-muted-foreground text-center"&gt;{t("dashboard.comingSoonDesc")}&lt;/p&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default Quotes;
```

src/pages/freelancer

src/pages/freelancer/Dashboard.tsx

```
// Re-export existing FreelancerHome as Dashboard
export { default } from "@/pages/freelancer/FreelancerHome";
```

src/pages/freelancer/FreelancerCalendar.tsx

```
import { useEffect, useState } from "react";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Calendar } from "@/components/ui/calendar";
import { CalendarDays, MapPin } from "lucide-react";
import { format, isSameDay } from "date-fns";

interface Deadline {
  projectName: string;
  address: string | null;
  deadline: Date;
  status: string;
}

const FreelancerCalendar = () =&gt; {
  const { user } = useAuth();
  const { t } = useLanguage();
  const [deadlines, setDeadlines] = useState&lt;Deadline[]&gt;([]);
  const [selectedDate, setSelectedDate] = useState&lt;Date | undefined&gt;(new Date());
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      if (!user) return;
      const { data } = await supabase.from("freelancer_assignments").select("deadline, projects(name, address, status)").eq("freelancer_id", user.id).not("deadline", "is", n
      const mapped = (data || []).map((a: any) =&gt; ({ projectName: a.projects?.name || "Unknown", address: a.projects?.address || null, deadline: new Date(a.deadline), sta
      setDeadlines(mapped);
      setLoading(false);
    };
    fetch();
  }, [user]);

  const deadlineDates = deadlines.map((d) =&gt; d.deadline);
  const selectedDeadlines = selectedDate ? deadlines.filter((d) =&gt; isSameDay(d.deadline, selectedDate)) : [];

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;{t("freelancer.calendar")}&lt;/h1&gt;
      &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
        &lt;Card&gt;
          &lt;CardContent className="p-4"&gt;
            &lt;Calendar mode="single" selected={selectedDate} onSelect={setSelectedDate} modifiers={{ deadline: deadlineDates }} modifiersClassNames={{ deadline: "bg-primar
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card&gt;
          &lt;CardHeader&gt;
            &lt;CardTitle className="flex items-center gap-2 text-lg"&gt;&lt;CalendarDays className="h-5 w-5" /&gt;{selectedDate ? format(selectedDate, "MMMM dd, yyyy") : t(
          &lt;/CardHeader&gt;
          &lt;CardContent&gt;
            {selectedDeadlines.length === 0 ? &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.noDeadlinesOnDate")}&lt;/p&gt; : (
              &lt;div className="space-y-3"&gt;
                {selectedDeadlines.map((d, i) =&gt; (
                  &lt;div key={i} className="p-4 rounded-lg border border-border"&gt;
                    &lt;p className="font-medium text-foreground"&gt;{d.projectName}&lt;/p&gt;
                    {d.address &amp;&amp; &lt;p className="text-xs text-muted-foreground flex items-center gap-1 mt-1"&gt;&lt;MapPin className="h-3 w-3" /&gt; {d.address}&lt
                    &lt;Badge variant="outline" className="mt-2"&gt;{d.status.replace("_", " ")}&lt;/Badge&gt;
                  &lt;/div&gt;
                ))}
              &lt;/div&gt;
            )}
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;
      &lt;Card&gt;
        &lt;CardHeader&gt;&lt;CardTitle className="text-lg"&gt;{t("freelancer.allUpcomingDeadlines")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
        &lt;CardContent&gt;
          {loading ? &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.loading")}&lt;/p&gt; : deadlines.filter(d =&gt; d.deadline &gt;= new Date()).length ==
            &lt;p class="text-sm text-muted-foreground"&gt;{t("freelancer.noUpcoming")}&lt;/p&gt;
          ) : (
            &lt;div className="space-y-2"&gt;
              {deadlines.filter(d =&gt; d.deadline &gt;= new Date()).sort((a, b) =&gt; a.deadline.getTime() - b.deadline.getTime()).map((d, i) =&gt; (
                &lt;div key={i} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
                  &lt;span className="font-medium text-sm text-foreground"&gt;{d.projectName}&lt;/span&gt;
                  &lt;Badge variant="outline"&gt;{format(d.deadline, "MMM dd, yyyy")}&lt;/Badge&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          )}
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default FreelancerCalendar;
```

src/pages/freelancer/FreelancerContracts.tsx

```
import { useEffect, useState } from "react";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { FileText, Download, CheckCircle, XCircle } from "lucide-react";

const FreelancerContracts = () =&gt; {
  const { user } = useAuth();
  const { t } = useLanguage();
  const [contracts, setContracts] = useState&lt;any[]&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      if (!user) return;
      const { data } = await supabase.from("freelancer_contracts").select("*, projects(name)").eq("freelancer_id", user.id).order("created_at", { ascending: false });
      setContracts(data || []);
      setLoading(false);
    };
    fetch();
  }, [user]);

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;{t("freelancer.contracts")}&lt;/h1&gt;
      {loading ? &lt;p className="text-muted-foreground"&gt;{t("freelancer.loading")}&lt;/p&gt; : contracts.length === 0 ? (
        &lt;Card&gt;&lt;CardContent className="p-8 text-center"&gt;&lt;FileText className="h-12 w-12 mx-auto text-muted-foreground mb-4" /&gt;&lt;p className="text-muted-for
      ) : (
        &lt;div className="space-y-3"&gt;
          {contracts.map((c) =&gt; (
            &lt;Card key={c.id}&gt;
              &lt;CardContent className="flex items-center justify-between p-5"&gt;
                &lt;div className="flex items-center gap-4"&gt;
                  &lt;div className="w-10 h-10 rounded-lg bg-muted flex items-center justify-center"&gt;&lt;FileText className="h-5 w-5 text-muted-foreground" /&gt;&lt;/div&
                  &lt;div&gt;&lt;p className="font-medium text-foreground"&gt;{c.title}&lt;/p&gt;&lt;p className="text-xs text-muted-foreground"&gt;{c.projects?.name || t("f
                &lt;/div&gt;
                &lt;div className="flex items-center gap-3"&gt;
                  {c.is_signed ? (
                    &lt;Badge variant="outline" className="gap-1"&gt;&lt;CheckCircle className="h-3 w-3" /&gt; {t("freelancer.signed")}&lt;/Badge&gt;
                  ) : (
                    &lt;Badge variant="secondary" className="gap-1"&gt;&lt;XCircle className="h-3 w-3" /&gt; {t("freelancer.unsigned")}&lt;/Badge&gt;
                  )}
                  {c.file_url &amp;&amp; (
                    &lt;a href={c.file_url} target="_blank" rel="noopener noreferrer"&gt;
                      &lt;Button variant="outline" size="sm"&gt;&lt;Download className="h-3 w-3 mr-1" /&gt; {t("freelancer.download")}&lt;/Button&gt;
                    &lt;/a&gt;
                  )}
                &lt;/div&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          ))}
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
};

export default FreelancerContracts;
```

src/pages/freelancer/FreelancerHome.tsx

```
import { useEffect, useState } from "react";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { FolderOpen, DollarSign, CalendarDays, Wallet } from "lucide-react";
import { format } from "date-fns";

const FreelancerHome = () =&gt; {
  const { user } = useAuth();
  const { t } = useLanguage();
  const [stats, setStats] = useState({
    activeProjects: 0,
    totalReceivable: 0,
    totalAlreadyPaid: 0,
    nextPaymentLabel: "—",
    upcomingDeadlines: [] as { projectName: string; deadline: string }[],
  });
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetchStats = async () =&gt; {
      if (!user) return;
      const [assignmentsRes, paymentsRes] = await Promise.all([
        supabase
          .from("freelancer_assignments")
          .select("project_id, deadline, projects(name, status, display_address)")
          .eq("freelancer_id", user.id),
        supabase
          .from("freelancer_payments")
          .select("amount, status, due_date, agreed_amount, paid_amount")
          .eq("freelancer_id", user.id),
      ]);

      const assignments = assignmentsRes.data || [];
      const payments = paymentsRes.data || [];

      const activeProjects = assignments.filter(
        (a: any) =&gt; a.projects?.status !== "completed" &amp;&amp; a.projects?.status !== "cancelled"
      ).length;

      const totalReceivable = payments.reduce((s, p: any) =&gt; {
        const agreed = Number(p.agreed_amount || p.amount || 0);
        const paid = Number(p.paid_amount || 0);
        return s + Math.max(agreed - paid, 0);
      }, 0);

      const totalAlreadyPaid = payments.reduce((s, p: any) =&gt; s + Number(p.paid_amount || 0), 0);

      // Next payment
      const pendingPayments = payments
        .filter((p: any) =&gt; {
          const agreed = Number(p.agreed_amount || p.amount || 0);
          const paid = Number(p.paid_amount || 0);
          return paid &lt; agreed &amp;&amp; p.due_date;
        })
        .sort((a, b) =&gt; new Date(a.due_date!).getTime() - new Date(b.due_date!).getTime());

      const nextPaymentLabel = pendingPayments[0]
        ? `$${Number((pendingPayments[0] as any).agreed_amount || pendingPayments[0].amount || 0).toLocaleString(undefined, { minimumFractionDigits: 2 })} — ${format(new Dat
        : "—";

      const upcomingDeadlines = assignments
        .filter((a: any) =&gt; a.deadline &amp;&amp; new Date(a.deadline) &gt; new Date())
        .sort((a: any, b: any) =&gt; new Date(a.deadline).getTime() - new Date(b.deadline).getTime())
        .slice(0, 5)
        .map((a: any) =&gt; ({
          projectName: a.projects?.display_address || a.projects?.name || "Unknown",
          deadline: a.deadline!,
        }));

      setStats({ activeProjects, totalReceivable, totalAlreadyPaid, nextPaymentLabel, upcomingDeadlines });
      setLoading(false);
    };
    fetchStats();
  }, [user]);

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;
          {t("freelancer.welcome")}, {user?.user_metadata?.full_name?.split(" ")[0] || "Freelancer"}!
        &lt;/h1&gt;
        &lt;p className="text-xs text-muted-foreground"&gt;{t("freelancer.overviewDesc")}&lt;/p&gt;
      &lt;/div&gt;

      {/* Stats Grid */}
      &lt;div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4"&gt;
        &lt;Card&gt;
          &lt;CardContent className="flex items-center gap-4 p-6"&gt;
            &lt;div className="w-12 h-12 rounded-lg bg-primary/10 flex items-center justify-center"&gt;
              &lt;FolderOpen className="h-6 w-6 text-primary" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.activeProjects")}&lt;/p&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;{stats.activeProjects}&lt;/p&gt;
            &lt;/div&gt;
```

```
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card&gt;
          &lt;CardContent className="flex items-center gap-4 p-6"&gt;
            &lt;div className="w-12 h-12 rounded-lg bg-destructive/10 flex items-center justify-center"&gt;
              &lt;Wallet className="h-6 w-6 text-destructive" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.totalReceivable")}&lt;/p&gt;
              &lt;p class="text-2xl font-bold text-foreground"&gt;${stats.totalReceivable.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card&gt;
          &lt;CardContent className="flex items-center gap-4 p-6"&gt;
            &lt;div className="w-12 h-12 rounded-lg bg-chart-2/10 flex items-center justify-center"&gt;
              &lt;DollarSign className="h-6 w-6 text-chart-2" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.totalAlreadyPaid")}&lt;/p&gt;
              &lt;p class="text-2xl font-bold text-foreground"&gt;${stats.totalAlreadyPaid.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card&gt;
          &lt;CardContent className="flex items-center gap-4 p-6"&gt;
            &lt;div className="w-12 h-12 rounded-lg bg-chart-4/10 flex items-center justify-center"&gt;
              &lt;CalendarDays className="h-6 w-6 text-chart-4" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.nextPayment")}&lt;/p&gt;
              &lt;p class="text-lg font-bold text-foreground"&gt;{stats.nextPaymentLabel}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;

      {/* Upcoming Deadlines */}
      &lt;Card&gt;
        &lt;CardHeader&gt;
          &lt;CardTitle className="flex items-center gap-2"&gt;
            &lt;CalendarDays className="h-5 w-5" /&gt;
            {t("freelancer.upcomingDeadlines")}
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          {stats.upcomingDeadlines.length === 0 ? (
            &lt;p className="text-muted-foreground text-sm"&gt;{t("freelancer.noUpcomingDeadlines")}&lt;/p&gt;
          ) : (
            &lt;div className="space-y-3"&gt;
              {stats.upcomingDeadlines.map((d, i) =&gt; (
                &lt;div key={i} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
                  &lt;span className="font-medium text-foreground"&gt;{d.projectName}&lt;/span&gt;
                  &lt;Badge variant="outline"&gt;{format(new Date(d.deadline), "MMM dd, yyyy")}&lt;/Badge&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          )}
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default FreelancerHome;
```

src/pages/freelancer/FreelancerPayments.tsx

```
import { useEffect, useState } from "react";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { DollarSign, Wallet, Clock } from "lucide-react";
import { format } from "date-fns";

const getPaymentStatus = (agreed: number, paid: number) =&gt; {
  if (paid &lt;= 0) return "pending";
  if (paid &lt; agreed) return "partial";
  return "paid";
};

const statusConfig: Record&lt;string, { label: string; className: string }&gt; = {
  pending: { label: "Pending", className: "bg-destructive/15 text-destructive border-destructive/30" },
  partial: { label: "Partial", className: "bg-chart-4/15 text-chart-4 border-chart-4/30" },
  paid: { label: "Paid", className: "bg-chart-2/15 text-chart-2 border-chart-2/30" },
};

const FreelancerPayments = () =&gt; {
  const { user } = useAuth();
  const { t } = useLanguage();
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetchPayments = async () =&gt; {
      if (!user) return;
      const { data } = await supabase
        .from("freelancer_payments")
        .select("*, projects(name, display_address)")
        .eq("freelancer_id", user.id)
        .order("created_at", { ascending: false });
      setPayments(data || []);
      setLoading(false);
    };
    fetchPayments();
  }, [user]);

  // Compute totals
  const totalReceivable = payments.reduce((s, p) =&gt; {
    const agreed = Number((p as any).agreed_amount || p.amount || 0);
    const paid = Number((p as any).paid_amount || 0);
    return s + Math.max(agreed - paid, 0);
  }, 0);

  const totalAlreadyPaid = payments.reduce((s, p) =&gt; s + Number((p as any).paid_amount || 0), 0);

  // Next payment: first pending/partial with due_date
  const nextPayment = payments
    .filter(p =&gt; {
      const status = getPaymentStatus(Number((p as any).agreed_amount || p.amount || 0), Number((p as any).paid_amount || 0));
      return status !== "paid" &amp;&amp; p.due_date;
    })
    .sort((a, b) =&gt; new Date(a.due_date).getTime() - new Date(b.due_date).getTime())[0];

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;{t("freelancer.payments")}&lt;/h1&gt;
        &lt;p className="text-xs text-muted-foreground"&gt;{t("freelancer.paymentHistory")}&lt;/p&gt;
      &lt;/div&gt;

      {/* Summary Cards */}
      &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4"&gt;
        &lt;Card&gt;
          &lt;CardContent className="flex items-center gap-4 p-6"&gt;
            &lt;div className="w-12 h-12 rounded-lg bg-destructive/10 flex items-center justify-center"&gt;
              &lt;Wallet className="h-6 w-6 text-destructive" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.totalReceivable")}&lt;/p&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;${totalReceivable.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card&gt;
          &lt;CardContent className="flex items-center gap-4 p-6"&gt;
            &lt;div className="w-12 h-12 rounded-lg bg-chart-2/10 flex items-center justify-center"&gt;
              &lt;DollarSign className="h-6 w-6 text-chart-2" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.totalAlreadyPaid")}&lt;/p&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;${totalAlreadyPaid.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card&gt;
          &lt;CardContent className="flex items-center gap-4 p-6"&gt;
            &lt;div className="w-12 h-12 rounded-lg bg-chart-4/10 flex items-center justify-center"&gt;
              &lt;Clock className="h-6 w-6 text-chart-4" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.nextPayment")}&lt;/p&gt;
              &lt;p className="text-lg font-bold text-foreground"&gt;
                {nextPayment
```

```
                ? `$${Number((nextPayment as any).agreed_amount || nextPayment.amount || 0).toLocaleString(undefined, { minimumFractionDigits: 2 })} — ${format(new Date(ne
                : "—"}
            &lt;/p&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;

    {/* Payment List */}
    &lt;Card&gt;
      &lt;CardHeader&gt;
        &lt;CardTitle&gt;{t("freelancer.paymentHistory")}&lt;/CardTitle&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        {loading ? (
          &lt;p className="text-muted-foreground text-sm"&gt;{t("freelancer.loading")}&lt;/p&gt;
        ) : payments.length === 0 ? (
          &lt;p className="text-muted-foreground text-sm"&gt;{t("freelancer.noPayments")}&lt;/p&gt;
        ) : (
          &lt;div className="space-y-4"&gt;
            {payments.map((p) =&gt; {
              const agreed = Number((p as any).agreed_amount || p.amount || 0);
              const paid = Number((p as any).paid_amount || 0);
              const remaining = Math.max(agreed - paid, 0);
              const status = getPaymentStatus(agreed, paid);
              const config = statusConfig[status];
              const projectLabel = p.projects?.display_address || p.projects?.name || p.description || "Payment";

              return (
                &lt;div key={p.id} className="p-4 rounded-lg border border-border space-y-3"&gt;
                  &lt;div className="flex items-start justify-between"&gt;
                    &lt;div&gt;
                      &lt;p className="font-semibold text-foreground"&gt;{t("freelancer.project")}: {projectLabel}&lt;/p&gt;
                      {p.due_date &amp;&amp; (
                        &lt;p className="text-xs text-muted-foreground mt-1"&gt;
                          {t("freelancer.due")}: {format(new Date(p.due_date), "MMM dd, yyyy")}
                        &lt;/p&gt;
                      )}
                      {(p as any).payment_method &amp;&amp; (
                        &lt;p className="text-xs text-muted-foreground"&gt;
                          {t("freelancer.paymentMethod")}: {(p as any).payment_method}
                        &lt;/p&gt;
                      )}
                    &lt;/div&gt;
                    &lt;Badge className={`${config.className} border rounded-full px-3 py-1 text-xs font-semibold`}&gt;
                      {status === "pending" ? t("freelancer.pending") : status === "partial" ? t("freelancer.partial") : t("freelancer.paid")}
                    &lt;/Badge&gt;
                  &lt;/div&gt;
                  &lt;div className="grid grid-cols-3 gap-4 text-sm"&gt;
                    &lt;div&gt;
                      &lt;p className="text-muted-foreground"&gt;{t("freelancer.agreedAmount")}&lt;/p&gt;
                      &lt;p class="font-bold text-foreground"&gt;${agreed.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
                    &lt;/div&gt;
                    &lt;div&gt;
                      &lt;p class="text-muted-foreground"&gt;{t("freelancer.paidAmount")}&lt;/p&gt;
                      &lt;p class="font-bold text-foreground"&gt;${paid.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
                    &lt;/div&gt;
                    &lt;div&gt;
                      &lt;p class="text-muted-foreground"&gt;{t("freelancer.remainingAmount")}&lt;/p&gt;
                      &lt;p class="font-bold text-foreground"&gt;${remaining.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt;
                    &lt;/div&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              );
            })}
          &lt;/div&gt;
        )}
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/div&gt;
);
};

export default FreelancerPayments;
```

src/pages/freelancer/FreelancerProfile.tsx

```
import ProfileEditor from "@/components/profile/ProfileEditor";

const FreelancerProfile = () =&gt; {
  return &lt;ProfileEditor /&gt;;
};

export default FreelancerProfile;
```

src/pages/freelancer/FreelancerProjects.tsx

```
import { useEffect, useState } from "react";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { MapPin, Eye, Upload, FileText, FolderOpen } from "lucide-react";
import { toast } from "sonner";
import { format } from "date-fns";

const statusColors: Record&lt;string, string&gt; = {
  pending: "bg-muted text-muted-foreground",
  in_progress: "bg-primary/10 text-primary",
  under_review: "bg-accent text-accent-foreground",
  completed: "bg-primary/20 text-primary",
  cancelled: "bg-destructive/10 text-destructive",
};

const FreelancerProjects = () =&gt; {
  const { user } = useAuth();
  const { t } = useLanguage();
  const [assignments, setAssignments] = useState&lt;any[]&gt;([]);
  const [selectedProject, setSelectedProject] = useState&lt;any | null&gt;(null);
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [deliveries, setDeliveries] = useState&lt;any[]&gt;([]);
  const [loading, setLoading] = useState(true);
  const [deliveryName, setDeliveryName] = useState("");
  const [deliveryNotes, setDeliveryNotes] = useState("");
  const [deliveryUrl, setDeliveryUrl] = useState("");
  const [submitting, setSubmitting] = useState(false);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      if (!user) return;
      const { data } = await supabase.from("freelancer_assignments").select("*, projects(id, name, address, description, status, service_type, created_at)").eq("freelancer_i
      setAssignments(data || []);
      setLoading(false);
    };
    fetch();
  }, [user]);

  const openProject = async (assignment: any) =&gt; {
    setSelectedProject(assignment);
    const projectId = assignment.projects?.id;
    if (!projectId) return;
    const [filesRes, deliveriesRes] = await Promise.all([
      supabase.from("project_files").select("*").eq("project_id", projectId),
      supabase.from("freelancer_deliveries").select("*").eq("project_id", projectId).eq("freelancer_id", user!.id),
    ]);
    setFiles(filesRes.data || []);
    setDeliveries(deliveriesRes.data || []);
  };

  const handleSubmitDelivery = async () =&gt; {
    if (!deliveryName || !deliveryUrl || !selectedProject?.projects?.id) { toast.error("Please fill in the file name and URL"); return; }
    setSubmitting(true);
    const { error } = await supabase.from("freelancer_deliveries").insert({ freelancer_id: user!.id, project_id: selectedProject.projects.id, file_name: deliveryName, file_u
    if (error) { toast.error("Failed to submit delivery"); } else {
      toast.success("Delivery submitted!");
      setDeliveryName(""); setDeliveryNotes(""); setDeliveryUrl("");
      const { data } = await supabase.from("freelancer_deliveries").select("*").eq("project_id", selectedProject.projects.id).eq("freelancer_id", user!.id);
      setDeliveries(data || []);
    }
    setSubmitting(false);
  };

  if (selectedProject) {
    const proj = selectedProject.projects;
    return (
      &lt;div className="p-6 space-y-6"&gt;
        &lt;Button variant="ghost" onClick={() =&gt; setSelectedProject(null)}&gt;{t("freelancer.backToProjects")}&lt;/Button&gt;
        &lt;Card&gt;
          &lt;CardHeader&gt;&lt;CardTitle className="flex items-center gap-2"&gt;&lt;MapPin className="h-5 w-5" /&gt;{proj.name}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
          &lt;CardContent className="space-y-3"&gt;
            &lt;div className="grid grid-cols-2 gap-4 text-sm"&gt;
              &lt;div&gt;&lt;span className="text-muted-foreground"&gt;{t("freelancer.address")}:&lt;/span&gt; {proj.address || "N/A"}&lt;/div&gt;
              &lt;div&gt;&lt;span className="text-muted-foreground"&gt;{t("freelancer.status")}:&lt;/span&gt; &lt;Badge className={statusColors[proj.status]}&gt;{proj.status
              &lt;div&gt;&lt;span className="text-muted-foreground"&gt;{t("freelancer.service")}:&lt;/span&gt; {proj.service_type.replace(/_/g, " ")}&lt;/div&gt;
              &lt;div&gt;&lt;span className="text-muted-foreground"&gt;{t("freelancer.deadline")}:&lt;/span&gt; {selectedProject.deadline ? format(new Date(selectedProject.d
            &lt;/div&gt;
            {proj.description &amp;&amp; &lt;p className="text-sm text-muted-foreground mt-2"&gt;{proj.description}&lt;/p&gt;}
            {selectedProject.notes &amp;&amp; &lt;div className="p-3 rounded-lg bg-muted/50 text-sm"&gt;&lt;strong&gt;{t("freelancer.notes")}:&lt;/strong&gt; {selectedProjec
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card&gt;
          &lt;CardHeader&gt;&lt;CardTitle className="text-lg"&gt;{t("freelancer.referenceFiles")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
          &lt;CardContent&gt;
            {files.length === 0 ? &lt;p className="text-sm text-muted-foreground"&gt;{t("freelancer.noFiles")}&lt;/p&gt; : (
              &lt;div className="space-y-2"&gt;
                {files.map((f) =&gt; (
                  &lt;a key={f.id} href={f.file_url} target="_blank" rel="noopener noreferrer" className="flex items-center gap-2 p-2 rounded hover:bg-muted/50 text-sm text-
                    &lt;FileText className="h-4 w-4 text-muted-foreground" /&gt;{f.file_name}
                  &lt;/a&gt;
                ))}
```

```
              &lt;/div&gt;
            )}
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card&gt;
          &lt;CardHeader&gt;&lt;CardTitle className="text-lg"&gt;{t("freelancer.submitDelivery")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
          &lt;CardContent className="space-y-4"&gt;
            &lt;div className="space-y-2"&gt;&lt;Label&gt;{t("freelancer.fileName")}&lt;/Label&gt;&lt;Input value={deliveryName} onChange={(e) =&gt; setDeliveryName(e.target
            &lt;div className="space-y-2"&gt;&lt;Label&gt;{t("freelancer.fileUrl")}&lt;/Label&gt;&lt;Input value={deliveryUrl} onChange={(e) =&gt; setDeliveryUrl(e.target.va
            &lt;div className="space-y-2"&gt;&lt;Label&gt;{t("freelancer.notesOptional")}&lt;/Label&gt;&lt;Textarea value={deliveryNotes} onChange={(e) =&gt; setDeliveryNote
            &lt;Button onClick={handleSubmitDelivery} disabled={submitting}&gt;&lt;Upload className="h-4 w-4 mr-2" /&gt;{submitting ? t("freelancer.submitting") : t("freelan
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        {deliveries.length &gt; 0 &amp;&amp; (
          &lt;Card&gt;
            &lt;CardHeader&gt;&lt;CardTitle className="text-lg"&gt;{t("freelancer.yourDeliveries")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
            &lt;CardContent&gt;
              &lt;div className="space-y-2"&gt;
                {deliveries.map((d) =&gt; (
                  &lt;div key={d.id} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
                    &lt;div&gt;&lt;p className="font-medium text-sm text-foreground"&gt;{d.file_name}&lt;/p&gt;{d.notes &amp;&amp; &lt;p className="text-xs text-muted-foregr
                    &lt;a href={d.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Button variant="outline" size="sm"&gt;&lt;Eye className="h-3 w-3 mr-1" /&gt; {t(
                  &lt;/div&gt;
                ))}
              &lt;/div&gt;
            &lt;/CardContent&gt;
          &lt;/Card&gt;
        )}
      &lt;/div&gt;
    );
  }

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;{t("freelancer.myProjects")}&lt;/h1&gt;
      {loading ? &lt;p className="text-muted-foreground"&gt;{t("freelancer.loading")}&lt;/p&gt; : assignments.length === 0 ? (
        &lt;Card&gt;&lt;CardContent className="p-8 text-center"&gt;&lt;FolderOpen className="h-12 w-12 mx-auto text-muted-foreground mb-4" /&gt;&lt;p class="text-muted-f
      ) : (
        &lt;div class="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
          {assignments.map((a) =&gt; (
            &lt;Card key={a.id} class="cursor-pointer hover:shadow-md transition-shadow" onClick={() =&gt; openProject(a)}&gt;
              &lt;CardContent class="p-5"&gt;
                &lt;div class="flex items-start justify-between mb-2"&gt;
                  &lt;h3 class="font-semibold text-foreground"&gt;{a.projects?.name || "Unknown"}&lt;/h3&gt;
                  &lt;Badge class={statusColors[a.projects?.status || "pending"]}&gt;{(a.projects?.status || "pending").replace("_", " ")}&lt;/Badge&gt;
                &lt;/div&gt;
                {a.projects?.address &amp;&amp; &lt;p class="text-sm text-muted-foreground flex items-center gap-1 mb-2"&gt;&lt;MapPin class="h-3 w-3" /&gt; {a.proje
                {a.deadline &amp;&amp; &lt;p class="text-xs text-muted-foreground"&gt;{t("freelancer.deadline")}: {format(new Date(a.deadline), "MMM dd, yyyy")}&lt;/p&gt
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          ))}
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
};

export default FreelancerProjects;
```

src/pages/public

src/pages/public/EstimateRequest.tsx

```
import { useState, useCallback } from "react";
import { useNavigate } from "react-router-dom";
import { toast } from "sonner";
import PublicLayout from "@/components/layout/PublicLayout";
import { Button } from "@/components/ui/button";
import { Progress } from "@/components/ui/progress";
import { supabase } from "@/integrations/supabase/client";
import { ArrowLeft, ArrowRight, Send } from "lucide-react";

import {
  TOTAL_STEPS,
  STEP_LABELS,
  PROJECT_TYPES,
  CLIENT_TYPES,
  SIZE_OPTIONS,
  SIZE_OPTIONS_BR,
  FLOOR_OPTIONS,
  RUSH_OPTIONS,
  OPTIONAL_SERVICES,
  DRAFTING_SERVICES,
  DESIGN_SERVICES,
  CONSTRUCTION_TYPE,
  SITE_CONDITIONS,
  PROFESSIONAL_TYPES,
} from "@/components/estimate/types";
import { useEstimateCalculator } from "@/components/estimate/useEstimateCalculator";
import { useConfidenceScore } from "@/components/estimate/useConfidenceScore";
import EstimatePreview from "@/components/estimate/EstimatePreview";
import StepLocation from "@/components/estimate/steps/StepLocation";
import StepClientInfo from "@/components/estimate/steps/StepClientInfo";
import StepOverview from "@/components/estimate/steps/StepOverview";
import StepDetails from "@/components/estimate/steps/StepDetails";
import StepProjectType from "@/components/estimate/steps/StepProjectType";
// StepConditions removed from flow
import StepServices from "@/components/estimate/steps/StepServices";
import StepProgram from "@/components/estimate/steps/StepProgram";
import StepUpload from "@/components/estimate/steps/StepUpload";
import StepTimeline from "@/components/estimate/steps/StepTimeline";
import StepAgreement from "@/components/estimate/steps/StepAgreement";

const EstimateRequest = () =&gt; {
  const navigate = useNavigate();
  const [step, setStep] = useState(1);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [files, setFiles] = useState&lt;File[]&gt;([]);
  const [errors, setErrors] = useState&lt;Record&lt;string, string&gt;&gt;({});

  const [form, setForm] = useState({
    country: "",
    city: "",
    state: "",
    fullName: "",
    email: "",
    phone: "",
    clientType: "",
    companyName: "",
    companyAddress: "",
    projectDescription: "",
    projectSize: "",
    customSqft: undefined as number | undefined,
    floors: [] as string[],
    lotSize: "",
    projectType: "",
    constructionType: "",
    siteCondition: "",
    draftingServices: [] as string[],
    designServices: [] as string[],
    optionalServices: [] as string[],
    rooms: {} as Record&lt;string, number&gt;,
    specialRequirements: "",
    rushService: "",
    agreeTerms: false,
    notes: "",
  });

  const setField = useCallback((field: string, value: any) =&gt; {
    setForm((prev) =&gt; ({ ...prev, [field]: value }));
    setErrors((prev) =&gt; { const next = { ...prev }; delete next[field]; return next; });
  }, []);

  const estimate = useEstimateCalculator({
    country: form.country,
    projectType: form.projectType,
    projectSize: form.projectSize,
    customSqft: form.customSqft,
    floors: form.floors,
    rushService: form.rushService,
    optionalServices: form.optionalServices,
    draftingServices: form.draftingServices,
  });

  const confidence = useConfidenceScore({
    city: form.city,
```

```
    state: form.state,
    projectType: form.projectType,
    projectSize: form.projectSize,
    customSqft: form.customSqft,
    rooms: form.rooms,
    filesCount: files.length,
    timeline: form.rushService || "standard",
  });

  // New step order:
  // 1=Location, 2=About You, 3=Details, 4=Project Type, 5=Services,
  // 6=Program, 7=Files, 8=Overview, 9=Rush Fees, 10=Review
  const validateStep = (): boolean =&gt; {
    const errs: Record&lt;string, string&gt; = {};
    switch (step) {
      case 1:
        if (!form.country) errs.country = "Required";
        if (!form.city) errs.city = "Required";
        if (!form.state) errs.state = "Required";
        break;
      case 2:
        if (!form.fullName || form.fullName.length &lt; 2) errs.fullName = "Name is required";
        if (!form.email || !/\S+@\S+\.\S+/.test(form.email)) errs.email = "Valid email required";
        if (!form.phone || form.phone.length &lt; 7) errs.phone = "Phone is required";
        if (!form.clientType) errs.clientType = "Required";
        if (PROFESSIONAL_TYPES.includes(form.clientType)) {
          if (!form.companyName) errs.companyName = "Company name is required";
          if (!form.companyAddress) errs.companyAddress = "Company address is required";
        }
        break;
      case 3:
        if (!form.projectSize) errs.projectSize = "Required";
        if (form.projectSize === "custom" &amp;&amp; (!form.customSqft || form.customSqft &lt; 10)) errs.customSqft = "Enter valid size";
        if (form.floors.length === 0) errs.floors = "Select at least one option";
        break;
      case 4:
        if (!form.projectType) errs.projectType = "Required";
        break;
      case 5: break;
      case 6: break;
      case 7: break;
      case 8: break;
      case 9: break;
      case 10:
        if (!form.agreeTerms) errs.agreeTerms = "You must agree to continue";
        break;
    }
    setErrors(errs);
    return Object.keys(errs).length === 0;
  };

  const next = () =&gt; { if (validateStep() &amp;&amp; step &lt; TOTAL_STEPS) setStep(step + 1); };
  const prev = () =&gt; { if (step &gt; 1) setStep(step - 1); };

  const onSubmit = async () =&gt; {
    if (!validateStep()) return;
    setIsSubmitting(true);

    try {
      const fileUrls: string[] = [];
      for (const file of files) {
        const path = `leads/${Date.now()}-${file.name}`;
        const { error: uploadErr } = await supabase.storage.from("project-files").upload(path, file);
        if (!uploadErr) {
          const { data: urlData } = supabase.storage.from("project-files").getPublicUrl(path);
          fileUrls.push(urlData.publicUrl);
        }
      }

      const projLabel = PROJECT_TYPES.find((p) =&gt; p.value === form.projectType)?.label || form.projectType;
      const clientLabel = CLIENT_TYPES.find((c) =&gt; c.value === form.clientType)?.label || form.clientType;
      const sizeOpts = form.country === "BR" ? SIZE_OPTIONS_BR : SIZE_OPTIONS;
      const sizeLabel = sizeOpts.find((s) =&gt; s.value === form.projectSize)?.label || `${form.customSqft} sqft`;
      const floorLabels = form.floors.map(f =&gt; FLOOR_OPTIONS.find(fo =&gt; fo.value === f)?.label).filter(Boolean).join(", ");
      const rushLabel = RUSH_OPTIONS.find(r =&gt; r.value === form.rushService)?.label || "Standard";
      const constructionLabel = CONSTRUCTION_TYPE.find((c) =&gt; c.value === form.constructionType)?.label || "";
      const siteLabel = SITE_CONDITIONS.find((s) =&gt; s.value === form.siteCondition)?.label || "";
      const draftingLabels = form.draftingServices.map((id) =&gt; DRAFTING_SERVICES.find((s) =&gt; s.id === id)?.label).filter(Boolean).join(", ");
      const designLabels = form.designServices.map((id) =&gt; DESIGN_SERVICES.find((s) =&gt; s.id === id)?.label).filter(Boolean).join(", ");
      const optLabels = form.optionalServices.map((id) =&gt; OPTIONAL_SERVICES.find((s) =&gt; s.id === id)?.label).filter(Boolean).join(", ");

      const roomsSummary = Object.entries(form.rooms).filter(([, v]) =&gt; v &gt; 0).map(([k, v]) =&gt; `${k}: ${v}`).join(", ");

      const description = `
COUNTRY: ${form.country === "BR" ? "Brazil" : "United States"}
CLIENT TYPE: ${clientLabel}
${form.companyName ? `COMPANY: ${form.companyName}` : ""}
${form.companyAddress ? `COMPANY ADDRESS: ${form.companyAddress}` : ""}
PROJECT LOCATION: ${form.city}, ${form.state}

PROJECT TYPE: ${projLabel}
PROJECT SIZE: ${sizeLabel}
FLOORS: ${floorLabels}
${constructionLabel ? `CONSTRUCTION: ${constructionLabel}` : ""}
${siteLabel ? `SITE: ${siteLabel}` : ""}

DRAFTING SERVICES: ${draftingLabels || "None selected"}
DESIGN SERVICES: ${designLabels || "None selected"}
3D SERVICES: ${optLabels || "None selected"}

${roomsSummary ? `PROGRAM: ${roomsSummary}` : ""}
```

```
${form.specialRequirements ? `SPECIAL REQUIREMENTS: ${form.specialRequirements}` : ""}

DELIVERY: ${rushLabel}

ESTIMATED FEE: $${estimate.low.toLocaleString()} – $${estimate.high.toLocaleString()}
CONFIDENCE: ${confidence}%
${form.projectDescription ? `DESCRIPTION: ${form.projectDescription}` : ""}
${fileUrls.length &gt; 0 ? `UPLOADED FILES:\n${fileUrls.join("\n")}` : ""}
${form.notes ? `NOTES: ${form.notes}` : ""}
      `.trim();

      const { error } = await supabase.from("quote_requests").insert({
        full_name: form.fullName,
        email: form.email,
        phone: form.phone,
        company_name: form.companyName || null,
        service_type: "architecture" as const,
        project_address: `${form.city}, ${form.state}`,
        description,
        file_url: fileUrls[0] || null,
      });

      if (error) throw error;

      navigate("/thank-you", {
        state: {
          projectAddress: `${form.city}, ${form.state}`,
          projectName: `${form.fullName} – ${projLabel}`,
          email: form.email,
          estimatedPrice: `$${estimate.low.toLocaleString()} – $${estimate.high.toLocaleString()}`,
        },
      });
    } catch (err) {
      console.error("Submit error:", err);
      toast.error("Error submitting request. Please try again.");
    } finally {
      setIsSubmitting(false);
    }
  };

  const progress = (step / TOTAL_STEPS) * 100;

  return (
    &lt;PublicLayout&gt;
      &lt;div className="container mx-auto px-4 py-12"&gt;
        &lt;div className="text-center mb-8"&gt;
          &lt;h1 className="text-3xl md:text-4xl font-serif font-bold text-foreground mb-2"&gt;
            Get Your Project Estimate
          &lt;/h1&gt;
          &lt;p className="text-muted-foreground"&gt;
            Answer a few questions and receive a personalized estimate in minutes.
          &lt;/p&gt;
        &lt;/div&gt;

        {/* Progress */}
        &lt;div className="max-w-4xl mx-auto mb-8"&gt;
          &lt;Progress value={progress} className="h-2 mb-3" /&gt;
          &lt;div className="flex justify-between text-xs text-muted-foreground"&gt;
            {STEP_LABELS.map((label, i) =&gt; (
              &lt;span
                key={i}
                className={`hidden lg:block ${step === i + 1 ? "text-primary font-semibold" : step &gt; i + 1 ? "text-primary/60" : ""}`}
              &gt;
                {label}
              &lt;/span&gt;
            ))}
            &lt;span className="lg:hidden text-primary font-semibold"&gt;
              Step {step} of {TOTAL_STEPS}: {STEP_LABELS[step - 1]}
            &lt;/span&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;div className="max-w-4xl mx-auto grid grid-cols-1 lg:grid-cols-3 gap-8"&gt;
          {/* Form */}
          &lt;div className="lg:col-span-2"&gt;
            &lt;div className="rounded-xl border border-border bg-card p-6 md:p-8 min-h-[400px]"&gt;
              {step === 1 &amp;&amp; &lt;StepLocation country={form.country} city={form.city} state={form.state} onFieldChange={setField} errors={errors} /&gt;}
              {step === 2 &amp;&amp; (
                &lt;StepClientInfo
                  fullName={form.fullName}
                  email={form.email}
                  phone={form.phone}
                  clientType={form.clientType}
                  companyName={form.companyName}
                  companyAddress={form.companyAddress}
                  onFieldChange={setField}
                  errors={errors}
                /&gt;
              )}
              {step === 3 &amp;&amp; (
                &lt;StepDetails
                  country={form.country}
                  projectSize={form.projectSize}
                  customSqft={form.customSqft}
                  floors={form.floors}
                  lotSize={form.lotSize}
                  onFieldChange={setField}
                  errors={errors}
                /&gt;
              )}
```

```
              {step === 4 &amp;&amp; &lt;StepProjectType value={form.projectType} onChange={(v) =&gt; setField("projectType", v)} error={errors.projectType} /&gt;}
              {step === 5 &amp;&amp; (
                &lt;StepServices
                  draftingServices={form.draftingServices}
                  designServices={form.designServices}
                  optionalServices={form.optionalServices}
                  onFieldChange={setField}
                  errors={errors}
                /&gt;
              )}
              {step === 6 &amp;&amp; &lt;StepProgram rooms={form.rooms} specialRequirements={form.specialRequirements} onFieldChange={setField} /&gt;}
              {step === 7 &amp;&amp; &lt;StepUpload files={files} onFilesChange={setFiles} /&gt;}
              {step === 8 &amp;&amp; &lt;StepOverview value={form.projectDescription} onChange={(v) =&gt; setField("projectDescription", v)} /&gt;}
              {step === 9 &amp;&amp; (
                &lt;StepTimeline
                  rushService={form.rushService}
                  uploadedFileNames={files.map(f =&gt; f.name)}
                  onFieldChange={setField}
                  error={errors.rushService}
                /&gt;
              )}
              {step === 10 &amp;&amp; &lt;StepAgreement agreeTerms={form.agreeTerms} notes={form.notes} onFieldChange={setField} errors={errors} /&gt;}

              {/* Navigation */}
              &lt;div className="flex justify-between pt-8"&gt;
                &lt;Button type="button" variant="outline" onClick={prev} disabled={step === 1}&gt;
                  &lt;ArrowLeft className="w-4 h-4 mr-2" /&gt; Back
                &lt;/Button&gt;
                {step &lt; TOTAL_STEPS ? (
                  &lt;Button type="button" onClick={next}&gt;
                    Next &lt;ArrowRight className="w-4 h-4 ml-2" /&gt;
                  &lt;/Button&gt;
                ) : (
                  &lt;Button onClick={onSubmit} disabled={isSubmitting}&gt;
                    {isSubmitting ? "Submitting..." : (
                      &lt;&gt;Submit Request &lt;Send className="w-4 h-4 ml-2" /&gt;&lt;/&gt;
                    )}
                  &lt;/Button&gt;
                )}
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          {/* Live Estimate Sidebar */}
          &lt;div className="lg:col-span-1"&gt;
            &lt;div className="sticky top-24"&gt;
              &lt;EstimatePreview
                low={estimate.low}
                high={estimate.high}
                total={estimate.total}
                currency={estimate.currency}
                breakdown={estimate.breakdown}
                confidence={confidence}
                canCalculate={estimate.canCalculate}
                complexityLabel={estimate.complexityLabel}
              /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/PublicLayout&gt;
  );
};

export default EstimateRequest;
```

src/pages/public/Home.tsx

```
import PublicLayout from "@/components/layout/PublicLayout";
import HeroSection from "@/components/home/HeroSection";
import ServicesGrid from "@/components/home/ServicesGrid";
import PortfolioPreview from "@/components/home/PortfolioPreview";
import CTASection from "@/components/home/CTASection";

const Index = () =&gt; {
  return (
    &lt;PublicLayout&gt;
      &lt;HeroSection /&gt;
      &lt;ServicesGrid /&gt;
      &lt;PortfolioPreview /&gt;
      &lt;CTASection /&gt;
    &lt;/PublicLayout&gt;
  );
};

export default Index;
```

src/pages/public/HowWeWork.tsx

```
import PublicLayout from "@/components/layout/PublicLayout";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Separator } from "@/components/ui/separator";
import {
  CheckCircle2,
  FileCheck,
  Pencil,
  FileText,
  Headphones,
  Check,
  X,
  Shield,
  MessageSquare,
  Award
} from "lucide-react";
import ProcessStep from "@/components/how-we-work/ProcessStep";
import { useLanguage } from "@/hooks/useLanguage";

const HowWeWork = () =&gt; {
  const { t } = useLanguage();

  const processSteps = [
    {
      number: "1",
      title: t("process.step1.title"),
      description: t("process.step1.desc"),
      icon: FileCheck,
      detailedInfo: t("process.step1.detail"),
    },
    {
      number: "2",
      title: t("process.step2.title"),
      description: t("process.step2.desc"),
      timeline: t("process.step2.timeline"),
      note: t("process.step2.note"),
      icon: Pencil,
      detailedInfo: t("process.step2.detail"),
    },
    {
      number: "3",
      title: t("process.step3.title"),
      description: t("process.step3.desc"),
      note: t("process.step3.note"),
      icon: FileText,
      detailedInfo: t("process.step3.detail"),
    },
    {
      number: "4",
      title: t("process.step4.title"),
      description: t("process.step4.desc"),
      timeline: t("process.step4.timeline"),
      note: t("process.step4.note"),
      icon: CheckCircle2,
      detailedInfo: t("process.step4.detail"),
    },
    {
      number: "5",
      title: t("process.step5.title"),
      description: t("process.step5.desc"),
      icon: Headphones,
      detailedInfo: t("process.step5.detail"),
    },
  ];

  const includedItems = Array.from({ length: 9 }, (_, i) =&gt; t(`included.${i + 1}`));
  const notIncludedItems = Array.from({ length: 11 }, (_, i) =&gt; t(`notIncluded.${i + 1}`));
  const clientResponsibilities = Array.from({ length: 3 }, (_, i) =&gt; t(`responsibility.${i + 1}`));
  const communicationFeatures = Array.from({ length: 4 }, (_, i) =&gt; t(`communication.${i + 1}`));
  const whyDara = Array.from({ length: 5 }, (_, i) =&gt; t(`whyDara.${i + 1}`));

  return (
    &lt;PublicLayout&gt;
      &lt;div className="container mx-auto px-4 py-12"&gt;
        &lt;div className="max-w-4xl mx-auto"&gt;
          {/* Header */}
          &lt;div className="text-center mb-12"&gt;
            &lt;h1 className="text-3xl md:text-4xl font-serif font-bold text-foreground mb-4"&gt;
              {t("howWeWork.title")}
            &lt;/h1&gt;
            &lt;p className="text-lg text-muted-foreground"&gt;
              {t("howWeWork.subtitle")}
            &lt;/p&gt;
          &lt;/div&gt;

          {/* Our Process */}
          &lt;section className="mb-12"&gt;
            &lt;h2 className="text-2xl font-serif font-semibold text-foreground mb-2"&gt;
              {t("howWeWork.ourProcess")}
            &lt;/h2&gt;
            &lt;p className="text-muted-foreground mb-6"&gt;
              {t("howWeWork.ourProcessDesc")}
            &lt;/p&gt;

            &lt;div className="space-y-4"&gt;
              {processSteps.map((step) =&gt; (
                &lt;ProcessStep
                  key={step.number}
                  number={step.number}
```

```
        title={step.title}
        description={step.description}
        timeline={step.timeline}
        note={step.note}
        icon={step.icon}
        detailedInfo={step.detailedInfo}
      /&gt;
    ))}
  &lt;/div&gt;
&lt;/section&gt;

&lt;Separator className="my-8" /&gt;

{/* What's Included */}
&lt;section className="mb-12"&gt;
  &lt;div className="flex items-center gap-3 mb-4"&gt;
    &lt;Check className="w-6 h-6 text-primary" /&gt;
    &lt;h2 className="text-2xl font-serif font-semibold text-foreground"&gt;
      {t("howWeWork.whatsIncluded")}
    &lt;/h2&gt;
  &lt;/div&gt;
  &lt;p className="text-muted-foreground mb-4"&gt;
    {t("howWeWork.whatsIncludedDesc")}
  &lt;/p&gt;

  &lt;Card&gt;
    &lt;CardContent className="p-6"&gt;
      &lt;ul className="grid md:grid-cols-2 gap-3"&gt;
        {includedItems.map((item, index) =&gt; (
          &lt;li key={index} className="flex items-start gap-2"&gt;
            &lt;Check className="w-4 h-4 text-primary mt-0.5 flex-shrink-0" /&gt;
            &lt;span className="text-foreground"&gt;{item}&lt;/span&gt;
          &lt;/li&gt;
        ))}
      &lt;/ul&gt;
      &lt;p className="text-sm text-muted-foreground mt-4 italic"&gt;
        {t("howWeWork.drawingsNote")}
      &lt;/p&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/section&gt;

{/* What's Not Included */}
&lt;section className="mb-12"&gt;
  &lt;div className="flex items-center gap-3 mb-4"&gt;
    &lt;X className="w-6 h-6 text-destructive" /&gt;
    &lt;h2 className="text-2xl font-serif font-semibold text-foreground"&gt;
      {t("howWeWork.whatsNotIncluded")}
    &lt;/h2&gt;
  &lt;/div&gt;
  &lt;p className="text-muted-foreground mb-4"&gt;
    {t("howWeWork.whatsNotIncludedDesc")}
  &lt;/p&gt;

  &lt;Card className="border-destructive/20 bg-destructive/5"&gt;
    &lt;CardContent className="p-6"&gt;
      &lt;ul className="grid md:grid-cols-2 gap-3"&gt;
        {notIncludedItems.map((item, index) =&gt; (
          &lt;li key={index} className="flex items-start gap-2"&gt;
            &lt;X className="w-4 h-4 text-destructive mt-0.5 flex-shrink-0" /&gt;
            &lt;span className="text-foreground"&gt;{item}&lt;/span&gt;
          &lt;/li&gt;
        ))}
      &lt;/ul&gt;
      &lt;p className="text-sm text-muted-foreground mt-4 italic"&gt;
        {t("howWeWork.licensedNote")}
      &lt;/p&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/section&gt;

&lt;Separator className="my-8" /&gt;

{/* Compliance &amp; Responsibility */}
&lt;section className="mb-12"&gt;
  &lt;div className="flex items-center gap-3 mb-4"&gt;
    &lt;Shield className="w-6 h-6 text-primary" /&gt;
    &lt;h2 className="text-2xl font-serif font-semibold text-foreground"&gt;
      {t("howWeWork.compliance")}
    &lt;/h2&gt;
  &lt;/div&gt;
  &lt;p className="text-muted-foreground mb-4"&gt;
    {t("howWeWork.complianceDesc")}
  &lt;/p&gt;

  &lt;Card&gt;
    &lt;CardHeader&gt;
      &lt;CardTitle className="text-lg"&gt;{t("howWeWork.clientResponsible")}&lt;/CardTitle&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent&gt;
      &lt;ul className="space-y-2"&gt;
        {clientResponsibilities.map((item, index) =&gt; (
          &lt;li key={index} className="flex items-start gap-2"&gt;
            &lt;Check className="w-4 h-4 text-primary mt-0.5 flex-shrink-0" /&gt;
            &lt;span className="text-foreground"&gt;{item}&lt;/span&gt;
          &lt;/li&gt;
        ))}
      &lt;/ul&gt;
      &lt;p className="text-sm text-muted-foreground mt-4 italic"&gt;
        {t("howWeWork.complianceNote")}
      &lt;/p&gt;
```

```
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          &lt;/section&gt;

          {/* Meetings &amp; Communication */}
          &lt;section className="mb-12"&gt;
            &lt;div className="flex items-center gap-3 mb-4"&gt;
              &lt;MessageSquare className="w-6 h-6 text-primary" /&gt;
              &lt;h2 className="text-2xl font-serif font-semibold text-foreground"&gt;
                {t("howWeWork.meetings")}
              &lt;/h2&gt;
            &lt;/div&gt;
            &lt;p className="text-muted-foreground mb-4"&gt;
              {t("howWeWork.meetingsDesc")}
            &lt;/p&gt;

            &lt;Card&gt;
              &lt;CardContent className="p-6"&gt;
                &lt;ul className="grid md:grid-cols-2 gap-3"&gt;
                  {communicationFeatures.map((item, index) =&gt; (
                    &lt;li key={index} className="flex items-start gap-2"&gt;
                      &lt;Check className="w-4 h-4 text-primary mt-0.5 flex-shrink-0" /&gt;
                      &lt;span className="text-foreground"&gt;{item}&lt;/span&gt;
                    &lt;/li&gt;
                  ))}
                &lt;/ul&gt;
                &lt;p className="text-sm text-muted-foreground mt-4"&gt;
                  {t("howWeWork.meetingsNote")}
                &lt;/p&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          &lt;/section&gt;

          &lt;Separator className="my-8" /&gt;

          {/* Why DA•RA Studio */}
          &lt;section className="mb-8"&gt;
            &lt;div className="flex items-center gap-3 mb-4"&gt;
              &lt;Award className="w-6 h-6 text-primary" /&gt;
              &lt;h2 className="text-2xl font-serif font-semibold text-foreground"&gt;
                {t("howWeWork.whyDara")}
              &lt;/h2&gt;
            &lt;/div&gt;

            &lt;Card className="bg-primary/5 border-primary/20"&gt;
              &lt;CardContent className="p-6"&gt;
                &lt;ul className="grid md:grid-cols-2 gap-3"&gt;
                  {whyDara.map((item, index) =&gt; (
                    &lt;li key={index} className="flex items-start gap-2"&gt;
                      &lt;Check className="w-5 h-5 text-primary mt-0.5 flex-shrink-0" /&gt;
                      &lt;span className="text-foreground font-medium"&gt;{item}&lt;/span&gt;
                    &lt;/li&gt;
                  ))}
                &lt;/ul&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          &lt;/section&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/PublicLayout&gt;
  );
};

export default HowWeWork;
```

src/pages/public/NotFound.tsx

```
import { useLocation } from "react-router-dom";
import { useEffect } from "react";
import { useLanguage } from "@/hooks/useLanguage";

const NotFound = () =&gt; {
  const location = useLocation();
  const { t } = useLanguage();

  useEffect(() =&gt; {
    console.error("404 Error: User attempted to access non-existent route:", location.pathname);
  }, [location.pathname]);

  return (
    &lt;div className="flex min-h-screen items-center justify-center bg-muted"&gt;
      &lt;div className="text-center"&gt;
        &lt;h1 className="mb-4 text-4xl font-bold"&gt;{t("notFound.title")}&lt;/h1&gt;
        &lt;p className="mb-4 text-xl text-muted-foreground"&gt;{t("notFound.message")}&lt;/p&gt;
        &lt;a href="/" className="text-primary underline hover:text-primary/90"&gt;
          {t("notFound.goHome")}
        &lt;/a&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default NotFound;
```

src/pages/public/ThankYou.tsx

```
import { useEffect } from "react";
import { Link, useLocation, useNavigate } from "react-router-dom";
import { useLanguage } from "@/hooks/useLanguage";
import PublicLayout from "@/components/layout/PublicLayout";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { CheckCircle, Home, LogIn, Mail, MapPin } from "lucide-react";

interface ThankYouState {
  projectAddress: string;
  projectName: string;
  email: string;
}

const ThankYou = () =&gt; {
  const { t } = useLanguage();
  const location = useLocation();
  const navigate = useNavigate();
  const state = location.state as ThankYouState | null;

  useEffect(() =&gt; {
    // Redirect if accessed directly without state
    if (!state) {
      navigate("/");
    }
  }, [state, navigate]);

  if (!state) {
    return null;
  }

  return (
    &lt;PublicLayout&gt;
      &lt;div className="container mx-auto px-4 py-16"&gt;
        &lt;div className="max-w-2xl mx-auto"&gt;
          &lt;Card className="text-center"&gt;
            &lt;CardHeader className="pb-4"&gt;
              &lt;div className="mx-auto mb-4"&gt;
                &lt;div className="w-20 h-20 rounded-full bg-primary/10 flex items-center justify-center mx-auto"&gt;
                  &lt;CheckCircle className="w-10 h-10 text-primary" /&gt;
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;CardTitle className="text-2xl md:text-3xl font-serif"&gt;
                {t("thankyou.title")}
              &lt;/CardTitle&gt;
            &lt;/CardHeader&gt;
            &lt;CardContent className="space-y-6"&gt;
              &lt;p className="text-lg text-muted-foreground"&gt;
                {t("thankyou.message")}
              &lt;/p&gt;

              {/* Project Details */}
              &lt;div className="bg-accent/50 rounded-lg p-6 space-y-4"&gt;
                &lt;p className="text-sm text-muted-foreground"&gt;
                  {t("thankyou.projectCreated")}
                &lt;/p&gt;
                &lt;div className="flex items-center justify-center gap-2 text-lg font-semibold"&gt;
                  &lt;MapPin className="h-5 w-5 text-primary" /&gt;
                  &lt;span&gt;{state.projectAddress}&lt;/span&gt;
                &lt;/div&gt;
              &lt;/div&gt;

              {/* Timeline */}
              &lt;div className="flex items-center justify-center gap-2 text-muted-foreground"&gt;
                &lt;Mail className="h-4 w-4" /&gt;
                &lt;p&gt;{t("thankyou.proposalTime")}&lt;/p&gt;
              &lt;/div&gt;

              {/* Portal Access Info */}
              &lt;div className="bg-primary/5 rounded-lg p-4 border border-primary/20"&gt;
                &lt;p className="text-sm"&gt;
                  {t("thankyou.portalAccess")}
                &lt;/p&gt;
                &lt;p className="text-sm font-medium mt-2 text-primary"&gt;
                  {state.email}
                &lt;/p&gt;
              &lt;/div&gt;

              {/* Action Buttons */}
              &lt;div className="flex flex-col sm:flex-row gap-4 justify-center pt-4"&gt;
                &lt;Button variant="outline" asChild&gt;
                  &lt;Link to="/"&gt;
                    &lt;Home className="w-4 h-4 mr-2" /&gt;
                    {t("thankyou.goHome")}
                  &lt;/Link&gt;
                &lt;/Button&gt;
                &lt;Button asChild&gt;
                  &lt;Link to="/auth"&gt;
                    &lt;LogIn className="w-4 h-4 mr-2" /&gt;
                    {t("thankyou.goPortal")}
                  &lt;/Link&gt;
                &lt;/Button&gt;
              &lt;/div&gt;
            &lt;/CardContent&gt;
          &lt;/Card&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/PublicLayout&gt;
```

```
  );
};

export default ThankYou;
```

src/pages/auth

src/pages/auth/FreelancerLogin.tsx

```
// Re-export existing FreelancerAuth page as FreelancerLogin
export { default } from "@/pages/FreelancerAuth";
```

src/pages/auth/Login.tsx

```
// Re-export existing Auth page as Login
export { default } from "@/pages/Auth";
```

src/components/admin

src/components/admin/AddressAutocomplete.tsx

```
import { useState, useRef, useEffect } from "react";
import { Input } from "@/components/ui/input";
import { Command, CommandEmpty, CommandGroup, CommandItem, CommandList } from "@/components/ui/command";
import { Popover, PopoverContent, PopoverTrigger } from "@/components/ui/popover";
import { MapPin } from "lucide-react";

interface AddressResult {
  display_name: string;
  address: {
    road?: string;
    house_number?: string;
    city?: string;
    town?: string;
    village?: string;
    state?: string;
    postcode?: string;
    country?: string;
    country_code?: string;
    suburb?: string;
    neighbourhood?: string;
  };
  lat: string;
  lon: string;
}

interface AddressData {
  street_name: string;
  street_number: string;
  neighborhood: string;
  city: string;
  state: string;
  zip_code: string;
  country: string;
  latitude: number;
  longitude: number;
}

interface Props {
  onSelect: (data: AddressData) =&gt; void;
  placeholder?: string;
}

const AddressAutocomplete = ({ onSelect, placeholder = "Search address..." }: Props) =&gt; {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState&lt;AddressResult[]&gt;([]);
  const [open, setOpen] = useState(false);
  const debounceRef = useRef&lt;NodeJS.Timeout&gt;();

  useEffect(() =&gt; {
    if (query.length &lt; 3) { setResults([]); return; }
    clearTimeout(debounceRef.current);
    debounceRef.current = setTimeout(async () =&gt; {
      try {
        const res = await fetch(
          `https://nominatim.openstreetmap.org/search?format=json&amp;addressdetails=1&amp;limit=5&amp;q=${encodeURIComponent(query)}`,
          { headers: { "Accept-Language": "en" } }
        );
        const data = await res.json();
        setResults(data);
        setOpen(data.length &gt; 0);
      } catch { setResults([]); }
    }, 400);
    return () =&gt; clearTimeout(debounceRef.current);
  }, [query]);

  const handleSelect = (r: AddressResult) =&gt; {
    const a = r.address;
    const cityName = a.city || a.town || a.village || "";
    let country = "Other";
    if (a.country_code === "us") country = "USA";
    else if (a.country_code === "br") country = "Brazil";

    onSelect({
      street_name: a.road || "",
      street_number: a.house_number || "",
      neighborhood: a.suburb || a.neighbourhood || "",
      city: cityName,
      state: a.state || "",
      zip_code: a.postcode || "",
      country,
      latitude: parseFloat(r.lat),
      longitude: parseFloat(r.lon),
    });
    setQuery(r.display_name);
    setOpen(false);
  };

  return (
    &lt;Popover open={open} onOpenChange={setOpen}&gt;
      &lt;PopoverTrigger asChild&gt;
        &lt;div className="relative"&gt;
          &lt;MapPin className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
          &lt;Input
```

```
            placeholder={placeholder}
            value={query}
            onChange={(e) =&gt; { setQuery(e.target.value); setOpen(true); }}
            className="pl-10"
          /&gt;
        &lt;/div&gt;
      &lt;/PopoverTrigger&gt;
      &lt;PopoverContent className="p-0 w-[var(--radix-popover-trigger-width)]" align="start"&gt;
        &lt;Command&gt;
          &lt;CommandList&gt;
            &lt;CommandEmpty&gt;No results found.&lt;/CommandEmpty&gt;
            &lt;CommandGroup&gt;
              {results.map((r, i) =&gt; (
                &lt;CommandItem key={i} onSelect={() =&gt; handleSelect(r)} className="cursor-pointer"&gt;
                  &lt;MapPin className="h-4 w-4 mr-2 shrink-0 text-muted-foreground" /&gt;
                  &lt;span className="truncate text-sm"&gt;{r.display_name}&lt;/span&gt;
                &lt;/CommandItem&gt;
              ))}
            &lt;/CommandGroup&gt;
          &lt;/CommandList&gt;
        &lt;/Command&gt;
      &lt;/PopoverContent&gt;
    &lt;/Popover&gt;
  );
};

export default AddressAutocomplete;
```

src/components/admin/AdminLayout.tsx

```
import { useEffect } from "react";
import { Outlet, useNavigate, Link } from "react-router-dom";
import daraLogo from "@/assets/dara-logo.jpg";
import { useAuth } from "@/hooks/useAuth";
import { useAdminRole } from "@/hooks/useAdminRole";
import { SidebarProvider, SidebarTrigger, SidebarInset } from "@/components/ui/sidebar";
import AdminSidebar from "./AdminSidebar";
import { Skeleton } from "@/components/ui/skeleton";
import LanguageSelector from "@/components/LanguageSelector";
import { useLanguage } from "@/hooks/useLanguage";

const AdminLayout = () =&gt; {
  const { user, loading: authLoading } = useAuth();
  const { isAdmin, loading: roleLoading } = useAdminRole();
  const navigate = useNavigate();
  const { t } = useLanguage();
  const loading = authLoading || roleLoading;

  useEffect(() =&gt; {
    if (!loading &amp;&amp; !user) {
      navigate("/adm/login");
    } else if (!loading &amp;&amp; user &amp;&amp; !roleLoading &amp;&amp; !isAdmin) {
      navigate("/");
    }
  }, [user, loading, isAdmin, roleLoading, navigate]);

  if (loading) {
    return (
      &lt;div className="min-h-screen flex items-center justify-center bg-background"&gt;
        &lt;div className="space-y-4 w-full max-w-md p-8"&gt;
          &lt;Skeleton className="h-8 w-3/4 mx-auto" /&gt;
          &lt;Skeleton className="h-4 w-1/2 mx-auto" /&gt;
          &lt;Skeleton className="h-32 w-full" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    );
  }

  if (!user) return null;

  if (!isAdmin) return null;

  return (
    &lt;SidebarProvider&gt;
      &lt;div className="min-h-screen flex w-full"&gt;
        &lt;AdminSidebar /&gt;
        &lt;SidebarInset className="flex flex-col flex-1"&gt;
          &lt;header className="h-14 border-b border-border flex items-center justify-between px-4 bg-background"&gt;
            &lt;div className="flex items-center gap-4"&gt;
              &lt;SidebarTrigger /&gt;
              &lt;Link to="/adm" className="flex items-center gap-2"&gt;
                &lt;img src={daraLogo} alt="DARA Studio" className="h-8 w-8 rounded" /&gt;
                &lt;span className="font-serif text-lg font-semibold text-primary"&gt;&lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt; | Drafting &amp; 3D Support&
              &lt;/Link&gt;
              &lt;span className="text-xs font-medium text-destructive bg-destructive/10 px-2 py-0.5 rounded"&gt;Admin&lt;/span&gt;
            &lt;/div&gt;
            &lt;LanguageSelector /&gt;
          &lt;/header&gt;
          &lt;main className="flex-1 overflow-auto bg-muted/30"&gt;
            &lt;Outlet /&gt;
          &lt;/main&gt;
        &lt;/SidebarInset&gt;
      &lt;/div&gt;
    &lt;/SidebarProvider&gt;
  );
};

export default AdminLayout;
```

src/components/admin/AdminSidebar.tsx

```
import { useNavigate, useLocation } from "react-router-dom";
import { useAuth } from "@/hooks/useAuth";
import {
  Sidebar, SidebarContent, SidebarGroup, SidebarGroupContent, SidebarGroupLabel,
  SidebarMenu, SidebarMenuButton, SidebarMenuItem, SidebarFooter,
} from "@/components/ui/sidebar";
import {
  LayoutDashboard, FolderKanban, Building2, UserPlus, FileText, FileSignature,
  Receipt, PenTool, FolderOpen, BrainCircuit, Globe, BarChart3,
  Settings, LogOut, Ticket, PieChart, HardHat,
} from "lucide-react";

const sidebarGroups = [
  {
    label: "Dashboard",
    items: [
      { title: "Dashboard", url: "/adm", icon: LayoutDashboard },
    ],
  },
  {
    label: "CRM",
    items: [
      { title: "Leads", url: "/adm/leads", icon: UserPlus },
      { title: "Companies", url: "/adm/companies", icon: Building2 },
    ],
  },
  {
    label: "Projects",
    items: [
      { title: "All Projects", url: "/adm/projects", icon: FolderKanban },
      { title: "Drawings", url: "/adm/drawings", icon: PenTool },
      { title: "Files", url: "/adm/files", icon: FolderOpen },
      { title: "Site Updates", url: "/adm/site-updates", icon: HardHat },
    ],
  },
  {
    label: "Finance",
    items: [
      { title: "Finance Overview", url: "/adm/finance-overview", icon: PieChart },
      { title: "Proposals", url: "/adm/proposals", icon: FileText },
      { title: "Contracts", url: "/adm/contracts", icon: FileSignature },
      { title: "Invoices", url: "/adm/payments", icon: Receipt },
    ],
  },
  {
    label: "Clients",
    items: [
      { title: "Client Portal", url: "/adm/client-portal", icon: Globe },
      { title: "Client Tickets", url: "/adm/tickets", icon: Ticket },
    ],
  },
  {
    label: "Tools",
    items: [
      { title: "AI Code Check", url: "/adm/ai-code-check", icon: BrainCircuit },
    ],
  },
  {
    label: "Insights",
    items: [
      { title: "Analytics", url: "/adm/analytics", icon: BarChart3 },
    ],
  },
  {
    label: "Settings",
    items: [
      { title: "Settings", url: "/adm/settings", icon: Settings },
    ],
  },
];

const AdminSidebar = () =&gt; {
  const navigate = useNavigate();
  const location = useLocation();
  const { signOut } = useAuth();

  return (
    &lt;Sidebar&gt;
      &lt;SidebarContent&gt;
        {sidebarGroups.map((group) =&gt; (
          &lt;SidebarGroup key={group.label}&gt;
            &lt;SidebarGroupLabel className="text-xs uppercase tracking-wider text-muted-foreground/70"&gt;{group.label}&lt;/SidebarGroupLabel&gt;
            &lt;SidebarGroupContent&gt;
              &lt;SidebarMenu&gt;
                {group.items.map((item) =&gt; (
                  &lt;SidebarMenuItem key={item.url}&gt;
                    &lt;SidebarMenuButton
                      onClick={() =&gt; navigate(item.url)}
                      isActive={location.pathname === item.url || (item.url !== "/adm" &amp;&amp; location.pathname.startsWith(item.url + "/"))}
                    &gt;
                      &lt;item.icon className="h-4 w-4" /&gt;
                      &lt;span&gt;{item.title}&lt;/span&gt;
                    &lt;/SidebarMenuButton&gt;
                  &lt;/SidebarMenuItem&gt;
                ))}
              &lt;/SidebarMenu&gt;
            &lt;/SidebarGroupContent&gt;
          &lt;/SidebarGroup&gt;
```

```
        ))}
      &lt;/SidebarContent&gt;
      &lt;SidebarFooter&gt;
        &lt;SidebarMenu&gt;
          &lt;SidebarMenuItem&gt;
            &lt;SidebarMenuButton onClick={async () =&gt; { await signOut(); navigate("/"); }}&gt;
              &lt;LogOut className="h-4 w-4" /&gt;
              &lt;span&gt;Sign Out&lt;/span&gt;
            &lt;/SidebarMenuButton&gt;
          &lt;/SidebarMenuItem&gt;
        &lt;/SidebarMenu&gt;
      &lt;/SidebarFooter&gt;
    &lt;/Sidebar&gt;
  );
};

export default AdminSidebar;
```

src/components/admin/CompanyEmployees.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Separator } from "@/components/ui/separator";
import { Plus, Trash2, Pencil, X, Check, UserPlus } from "lucide-react";
import { toast } from "sonner";

interface Employee {
  id: string;
  company_id: string;
  name: string;
  phone: string | null;
  email: string | null;
  instagram: string | null;
  linkedin: string | null;
}

const emptyEmployee = { name: "", phone: "", email: "", instagram: "", linkedin: "" };

const CompanyEmployees = ({ companyId }: { companyId: string }) =&gt; {
  const [employees, setEmployees] = useState&lt;Employee[]&gt;([]);
  const [adding, setAdding] = useState(false);
  const [editingId, setEditingId] = useState&lt;string | null&gt;(null);
  const [form, setForm] = useState(emptyEmployee);

  const fetch = async () =&gt; {
    const { data } = await supabase
      .from("company_employees")
      .select("*")
      .eq("company_id", companyId)
      .order("name");
    setEmployees((data as Employee[]) || []);
  };

  useEffect(() =&gt; { fetch(); }, [companyId]);

  const handleSave = async () =&gt; {
    if (!form.name.trim()) { toast.error("Nome é obrigatório"); return; }

    if (editingId) {
      const { error } = await supabase.from("company_employees").update({
        name: form.name, phone: form.phone || null, email: form.email || null,
        instagram: form.instagram || null, linkedin: form.linkedin || null,
      }).eq("id", editingId);
      if (error) { toast.error(error.message); return; }
      toast.success("Funcionário atualizado");
    } else {
      const { error } = await supabase.from("company_employees").insert({
        company_id: companyId, name: form.name,
        phone: form.phone || null, email: form.email || null,
        instagram: form.instagram || null, linkedin: form.linkedin || null,
      });
      if (error) { toast.error(error.message); return; }
      toast.success("Funcionário adicionado");
    }
    setForm(emptyEmployee); setAdding(false); setEditingId(null);
    fetch();
  };

  const handleDelete = async (id: string) =&gt; {
    if (!confirm("Excluir funcionário?")) return;
    const { error } = await supabase.from("company_employees").delete().eq("id", id);
    if (error) { toast.error(error.message); return; }
    toast.success("Funcionário excluído"); fetch();
  };

  const startEdit = (e: Employee) =&gt; {
    setEditingId(e.id);
    setAdding(true);
    setForm({ name: e.name, phone: e.phone || "", email: e.email || "", instagram: e.instagram || "", linkedin: e.linkedin || "" });
  };

  const cancel = () =&gt; { setAdding(false); setEditingId(null); setForm(emptyEmployee); };

  return (
    &lt;div className="space-y-4"&gt;
      {employees.length === 0 &amp;&amp; !adding &amp;&amp; (
        &lt;p className="text-sm text-muted-foreground text-center py-4"&gt;Nenhum funcionário cadastrado&lt;/p&gt;
      )}

      {employees.map(emp =&gt; (
        &lt;div key={emp.id} className="flex items-start justify-between p-3 rounded-md border border-border bg-muted/30"&gt;
          &lt;div className="space-y-0.5"&gt;
            &lt;p className="font-medium text-sm text-foreground"&gt;{emp.name}&lt;/p&gt;
            {emp.email &amp;&amp; &lt;p className="text-xs text-muted-foreground"&gt;{emp.email}&lt;/p&gt;}
            {emp.phone &amp;&amp; &lt;p className="text-xs text-muted-foreground"&gt;{emp.phone}&lt;/p&gt;}
            &lt;div className="flex gap-3 text-xs text-muted-foreground"&gt;
              {emp.instagram &amp;&amp; &lt;span&gt;@{emp.instagram.replace("@", "")}&lt;/span&gt;}
              {emp.linkedin &amp;&amp; &lt;span&gt;LinkedIn&lt;/span&gt;}
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div className="flex gap-1"&gt;
            &lt;Button variant="ghost" size="icon" className="h-7 w-7" onClick={() =&gt; startEdit(emp)}&gt;&lt;Pencil className="h-3.5 w-3.5" /&gt;&lt;/Button&gt;
            &lt;Button variant="ghost" size="icon" className="h-7 w-7" onClick={() =&gt; handleDelete(emp.id)}&gt;&lt;Trash2 className="h-3.5 w-3.5 text-destructive" /&gt;&l
          &lt;/div&gt;
        &lt;/div&gt;
```

```
      ))}

      {adding ? (
        &lt;div className="space-y-3 p-3 rounded-md border border-border"&gt;
          &lt;div&gt;&lt;Label&gt;Nome *&lt;/Label&gt;&lt;Input value={form.name} onChange={e =&gt; setForm({ ...form, name: e.target.value })} /&gt;&lt;/div&gt;
          &lt;div className="grid grid-cols-2 gap-3"&gt;
            &lt;div&gt;&lt;Label&gt;Telefone&lt;/Label&gt;&lt;Input value={form.phone} onChange={e =&gt; setForm({ ...form, phone: e.target.value })} /&gt;&lt;/div&gt;
            &lt;div&gt;&lt;Label&gt;Email&lt;/Label&gt;&lt;Input type="email" value={form.email} onChange={e =&gt; setForm({ ...form, email: e.target.value })} /&gt;&lt;/div
          &lt;/div&gt;
          &lt;div className="grid grid-cols-2 gap-3"&gt;
            &lt;div&gt;&lt;Label&gt;Instagram&lt;/Label&gt;&lt;Input value={form.instagram} onChange={e =&gt; setForm({ ...form, instagram: e.target.value })} placeholder="@
            &lt;div&gt;&lt;Label&gt;LinkedIn&lt;/Label&gt;&lt;Input value={form.linkedin} onChange={e =&gt; setForm({ ...form, linkedin: e.target.value })} placeholder="URL"
          &lt;/div&gt;
          &lt;div className="flex gap-2"&gt;
            &lt;Button size="sm" onClick={handleSave}&gt;&lt;Check className="h-3.5 w-3.5 mr-1" /&gt;{editingId ? "Salvar" : "Adicionar"}&lt;/Button&gt;
            &lt;Button size="sm" variant="outline" onClick={cancel}&gt;&lt;X className="h-3.5 w-3.5 mr-1" /&gt;Cancelar&lt;/Button&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      ) : (
        &lt;Button variant="outline" size="sm" className="w-full" onClick={() =&gt; setAdding(true)}&gt;
          &lt;UserPlus className="h-4 w-4 mr-2" /&gt;Adicionar Funcionário
        &lt;/Button&gt;
      )}
    &lt;/div&gt;
  );
};

export default CompanyEmployees;
```

src/components/admin/ProjectDocuments.tsx

```
import { useCallback, useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import {
  Download, Upload, FolderOpen, FileText, ChevronRight, Trash2, Plus,
} from "lucide-react";
import { format } from "date-fns";
import { toast } from "sonner";

const DOCUMENT_CATEGORIES = [
  { value: "pre_construction", label: "Pre-Construction", icon: "■" },
  { value: "rev_00", label: "REV 00", icon: "■" },
  { value: "rev_01", label: "REV 01", icon: "■" },
  { value: "rev_02", label: "REV 02", icon: "■" },
  { value: "rev_03", label: "REV 03 (Final)", icon: "■" },
  { value: "final_delivery", label: "Final Delivery", icon: "■" },
  { value: "general", label: "General", icon: "■" },
] as const;

type DocCategory = typeof DOCUMENT_CATEGORIES[number]["value"];

interface ProjectDocumentsProps {
  projectId: string;
}

const ProjectDocuments = ({ projectId }: ProjectDocumentsProps) =&gt; {
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [openFolders, setOpenFolders] = useState&lt;Set&lt;string&gt;&gt;(new Set(["pre_construction", "rev_00"]));
  const [uploading, setUploading] = useState(false);
  const [uploadCategory, setUploadCategory] = useState&lt;DocCategory&gt;("general");
  const [selectedFiles, setSelectedFiles] = useState&lt;FileList | null&gt;(null);

  const loadFiles = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("project_files")
      .select("*")
      .eq("project_id", projectId)
      .order("created_at", { ascending: false });
    setFiles(data || []);
  }, [projectId]);

  useEffect(() =&gt; { loadFiles(); }, [loadFiles]);

  const toggleFolder = (cat: string) =&gt; {
    setOpenFolders(prev =&gt; {
      const next = new Set(prev);
      next.has(cat) ? next.delete(cat) : next.add(cat);
      return next;
    });
  };

  const handleUpload = async () =&gt; {
    if (!selectedFiles || selectedFiles.length === 0) return;
    setUploading(true);
    try {
      const { data: { user } } = await supabase.auth.getUser();
      for (const file of Array.from(selectedFiles)) {
        const ext = file.name.split(".").pop();
        const path = `${projectId}/${Date.now()}-${Math.random().toString(36).slice(2)}.${ext}`;
        const { error: uploadError } = await supabase.storage.from("project-files").upload(path, file);
        if (uploadError) { toast.error(`Upload failed: ${file.name}`); continue; }
        const { data: urlData } = supabase.storage.from("project-files").getPublicUrl(path);
        await supabase.from("project_files").insert({
          project_id: projectId,
          file_name: file.name,
          file_url: urlData.publicUrl,
          file_type: file.type || ext,
          file_size: file.size,
          uploaded_by: user?.id || null,
          document_category: uploadCategory,
        });
      }
      toast.success("Files uploaded");
      setSelectedFiles(null);
      loadFiles();
    } catch { toast.error("Upload error"); }
    setUploading(false);
  };

  const handleDelete = async (fileId: string) =&gt; {
    const { error } = await supabase.from("project_files").delete().eq("id", fileId);
    if (error) { toast.error("Delete failed"); return; }
    toast.success("File deleted");
    loadFiles();
  };

  const handleCategoryChange = async (fileId: string, newCategory: string) =&gt; {
    const { error } = await supabase.from("project_files").update({ document_category: newCategory }).eq("id", fileId);
    if (error) { toast.error("Update failed"); return; }
    loadFiles();
  };

  const grouped = DOCUMENT_CATEGORIES.reduce((acc, cat) =&gt; {
```

```
  acc[cat.value] = files.filter(f =&gt; (f.document_category || "general") === cat.value);
  return acc;
}, {} as Record&lt;string, any[]&gt;);

const formatSize = (bytes: number | null) =&gt; {
  if (!bytes) return "—";
  if (bytes &lt; 1024) return `${bytes} B`;
  if (bytes &lt; 1048576) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / 1048576).toFixed(1)} MB`;
};

return (
  &lt;div className="space-y-4"&gt;
    {/* Upload Bar */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-4"&gt;
        &lt;div className="flex items-center gap-3 flex-wrap"&gt;
          &lt;Select value={uploadCategory} onValueChange={(v) =&gt; setUploadCategory(v as DocCategory)}&gt;
            &lt;SelectTrigger className="w-48"&gt;
              &lt;SelectValue placeholder="Select folder" /&gt;
            &lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              {DOCUMENT_CATEGORIES.map(c =&gt; (
                &lt;SelectItem key={c.value} value={c.value}&gt;{c.icon} {c.label}&lt;/SelectItem&gt;
              ))}
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
          &lt;Input
            type="file"
            multiple
            className="max-w-xs"
            onChange={e =&gt; setSelectedFiles(e.target.files)}
          /&gt;
          &lt;Button onClick={handleUpload} disabled={uploading || !selectedFiles?.length} size="sm"&gt;
            &lt;Upload className="h-3.5 w-3.5 mr-1.5" /&gt;
            {uploading ? "Uploading..." : "Upload"}
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Folder Tree */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-2"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;CardTitle className="text-base"&gt;Project Documents&lt;/CardTitle&gt;
          &lt;Badge variant="secondary"&gt;{files.length} files&lt;/Badge&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;div className="divide-y divide-border"&gt;
          {DOCUMENT_CATEGORIES.map(cat =&gt; {
            const catFiles = grouped[cat.value];
            const isOpen = openFolders.has(cat.value);
            return (
              &lt;Collapsible key={cat.value} open={isOpen} onOpenChange={() =&gt; toggleFolder(cat.value)}&gt;
                &lt;CollapsibleTrigger className="flex items-center gap-3 w-full px-4 py-3 hover:bg-muted/50 transition-colors text-left"&gt;
                  &lt;ChevronRight className={`h-4 w-4 text-muted-foreground transition-transform ${isOpen ? "rotate-90" : ""}`} /&gt;
                  &lt;FolderOpen className="h-4 w-4 text-primary" /&gt;
                  &lt;span className="text-sm font-medium text-foreground flex-1"&gt;{cat.label}&lt;/span&gt;
                  &lt;Badge variant="outline" className="text-[10px]"&gt;{catFiles.length}&lt;/Badge&gt;
                &lt;/CollapsibleTrigger&gt;
                &lt;CollapsibleContent&gt;
                  {catFiles.length === 0 ? (
                    &lt;div className="px-12 py-4 text-xs text-muted-foreground italic"&gt;No files in this folder&lt;/div&gt;
                  ) : (
                    &lt;div className="pb-2"&gt;
                      {catFiles.map(f =&gt; (
                        &lt;div key={f.id} className="flex items-center gap-3 px-12 py-2 hover:bg-muted/30 transition-colors group"&gt;
                          &lt;FileText className="h-4 w-4 text-muted-foreground shrink-0" /&gt;
                          &lt;div className="flex-1 min-w-0"&gt;
                            &lt;p className="text-sm font-medium text-foreground truncate"&gt;{f.file_name}&lt;/p&gt;
                            &lt;p className="text-[11px] text-muted-foreground"&gt;
                              {formatSize(f.file_size)} · {format(new Date(f.created_at), "MMM d, yyyy")}
                            &lt;/p&gt;
                          &lt;/div&gt;
                          &lt;Select
                            value={f.document_category || "general"}
                            onValueChange={(v) =&gt; handleCategoryChange(f.id, v)}
                          &gt;
                            &lt;SelectTrigger className="w-32 h-7 text-[11px] opacity-0 group-hover:opacity-100 transition-opacity"&gt;
                              &lt;SelectValue /&gt;
                            &lt;/SelectTrigger&gt;
                            &lt;SelectContent&gt;
                              {DOCUMENT_CATEGORIES.map(c =&gt; (
                                &lt;SelectItem key={c.value} value={c.value} className="text-xs"&gt;{c.label}&lt;/SelectItem&gt;
                              ))}
                            &lt;/SelectContent&gt;
                          &lt;/Select&gt;
                          &lt;Button variant="ghost" size="icon" className="h-7 w-7 shrink-0" asChild&gt;
                            &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-3.5 w-3.5" /&gt;&lt;/a&gt;
                          &lt;/Button&gt;
                          &lt;Button
                            variant="ghost"
                            size="icon"
                            className="h-7 w-7 shrink-0 opacity-0 group-hover:opacity-100 text-destructive hover:text-destructive"
                            onClick={() =&gt; handleDelete(f.id)}
                          &gt;
                            &lt;Trash2 className="h-3.5 w-3.5" /&gt;
                          &lt;/Button&gt;
                        &lt;/div&gt;
```

```
                        ))}
                      &lt;/div&gt;
                    )}
                  &lt;/CollapsibleContent&gt;
                &lt;/Collapsible&gt;
              );
            })}
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default ProjectDocuments;
```

src/components/admin/ProjectKanbanBoard.tsx

```
import { useEffect, useState, useRef, useCallback } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { Popover, PopoverContent, PopoverTrigger } from "@/components/ui/popover";
import { Calendar } from "@/components/ui/calendar";
import { Checkbox } from "@/components/ui/checkbox";
import { ScrollArea } from "@/components/ui/scroll-area";
import {
  Plus, GripVertical, CalendarIcon, User, Flag, Pencil, Trash2,
  CheckCircle2, Clock, AlertTriangle, Eye,
} from "lucide-react";
import { format } from "date-fns";
import { cn } from "@/lib/utils";
import { toast } from "sonner";

import { KANBAN_STAGES, KANBAN_STAGE_COLORS as STAGE_COLORS, getStageLabel } from "@/constants/projectStages";

const PRIORITIES = [
  { value: "low", label: "Low", color: "text-muted-foreground bg-muted border-border" },
  { value: "medium", label: "Medium", color: "text-chart-4 bg-chart-4/15 border-chart-4/30" },
  { value: "high", label: "High", color: "text-chart-3 bg-chart-3/15 border-chart-3/30" },
  { value: "urgent", label: "Urgent", color: "text-destructive bg-destructive/15 border-destructive/30" },
];

interface ProjectTask {
  id: string;
  project_id: string;
  title: string;
  assigned_to: string | null;
  due_date: string | null;
  priority: string;
  stage: string;
  is_completed: boolean;
  completed_at: string | null;
  requires_client_review: boolean;
  sort_order: number;
  created_at: string;
  updated_at: string;
}

interface Props {
  projectId: string;
  onStageSync?: (stage: string) =&gt; void;
}

const emptyTask = {
  title: "",
  assigned_to: "",
  due_date: undefined as Date | undefined,
  priority: "medium",
  stage: "WAITING_CLIENT_INFO",
  requires_client_review: false,
};

export default function ProjectKanbanBoard({ projectId, onStageSync }: Props) {
  const [tasks, setTasks] = useState&lt;ProjectTask[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editingTask, setEditingTask] = useState&lt;ProjectTask | null&gt;(null);
  const [form, setForm] = useState(emptyTask);
  const [dragTaskId, setDragTaskId] = useState&lt;string | null&gt;(null);
  const [dragOverStage, setDragOverStage] = useState&lt;string | null&gt;(null);

  const fetchTasks = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("project_tasks")
      .select("*")
      .eq("project_id", projectId)
      .order("sort_order")
      .order("created_at");
    setTasks((data as ProjectTask[]) || []);
  }, [projectId]);

  useEffect(() =&gt; { fetchTasks(); }, [fetchTasks]);

  // Realtime subscription
  useEffect(() =&gt; {
    const channel = supabase
      .channel(`project_tasks_${projectId}`)
      .on("postgres_changes", { event: "*", schema: "public", table: "project_tasks", filter: `project_id=eq.${projectId}` }, () =&gt; {
        fetchTasks();
      })
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, [projectId, fetchTasks]);

  const syncProjectStage = useCallback(async (updatedTasks: ProjectTask[]) =&gt; {
    // Count tasks per stage (excluding completed tasks)
    const activeTasks = updatedTasks.filter(t =&gt; !t.is_completed);
    if (activeTasks.length === 0) return;

    const stageCounts: Record&lt;string, number&gt; = {};
    activeTasks.forEach(t =&gt; {
```

```
    stageCounts[t.stage] = (stageCounts[t.stage] || 0) + 1;
  });

  // Find the stage with the most tasks
  let maxStage = "Lead";
  let maxCount = 0;
  for (const [stage, count] of Object.entries(stageCounts)) {
    if (count &gt; maxCount) {
      maxCount = count;
      maxStage = stage;
    }
  }

  onStageSync?.(maxStage);
}, [onStageSync]);

const openNew = (stage: string) =&gt; {
  setEditingTask(null);
  setForm({ ...emptyTask, stage });
  setDialogOpen(true);
};

const openEdit = (task: ProjectTask) =&gt; {
  setEditingTask(task);
  setForm({
    title: task.title,
    assigned_to: task.assigned_to || "",
    due_date: task.due_date ? new Date(task.due_date) : undefined,
    priority: task.priority,
    stage: task.stage,
    requires_client_review: task.requires_client_review,
  });
  setDialogOpen(true);
};

const handleSave = async () =&gt; {
  if (!form.title.trim()) { toast.error("Task title is required"); return; }

  const payload = {
    project_id: projectId,
    title: form.title.trim(),
    assigned_to: form.assigned_to?.trim() || null,
    due_date: form.due_date ? format(form.due_date, "yyyy-MM-dd") : null,
    priority: form.priority,
    stage: form.stage,
    requires_client_review: form.requires_client_review,
  };

  if (editingTask) {
    const { error } = await supabase.from("project_tasks").update(payload).eq("id", editingTask.id);
    if (error) { toast.error(error.message); return; }
    toast.success("Task updated");
  } else {
    const maxOrder = tasks.filter(t =&gt; t.stage === form.stage).reduce((max, t) =&gt; Math.max(max, t.sort_order), -1);
    const { error } = await supabase.from("project_tasks").insert({ ...payload, sort_order: maxOrder + 1 });
    if (error) { toast.error(error.message); return; }
    toast.success("Task created");
  }

  setDialogOpen(false);
  await fetchTasks();
};

const handleDelete = async (taskId: string) =&gt; {
  if (!confirm("Delete this task?")) return;
  await supabase.from("project_tasks").delete().eq("id", taskId);
  toast.success("Task deleted");
  fetchTasks();
};

const handleToggleComplete = async (task: ProjectTask) =&gt; {
  const nowCompleted = !task.is_completed;
  await supabase.from("project_tasks").update({
    is_completed: nowCompleted,
    completed_at: nowCompleted ? new Date().toISOString() : null,
  }).eq("id", task.id);

  // Log to activity_log when completed
  if (nowCompleted) {
    await supabase.from("activity_log").insert({
      project_id: projectId,
      description: `Task "${task.title}" completed`,
      log_type: "task_completed",
      log_date: new Date().toISOString(),
    });
  }

  await fetchTasks();
};

const handleDragStart = (e: React.DragEvent, taskId: string) =&gt; {
  setDragTaskId(taskId);
  e.dataTransfer.effectAllowed = "move";
  e.dataTransfer.setData("text/plain", taskId);
};

const handleDragOver = (e: React.DragEvent, stage: string) =&gt; {
  e.preventDefault();
  e.dataTransfer.dropEffect = "move";
  setDragOverStage(stage);
};
```

```
const handleDragLeave = () =&gt; {
  setDragOverStage(null);
};

const handleDrop = async (e: React.DragEvent, targetStage: string) =&gt; {
  e.preventDefault();
  setDragOverStage(null);
  const taskId = e.dataTransfer.getData("text/plain");
  if (!taskId) return;

  const task = tasks.find(t =&gt; t.id === taskId);
  if (!task || task.stage === targetStage) { setDragTaskId(null); return; }

  const maxOrder = tasks.filter(t =&gt; t.stage === targetStage).reduce((max, t) =&gt; Math.max(max, t.sort_order), -1);

  await supabase.from("project_tasks").update({
    stage: targetStage,
    sort_order: maxOrder + 1,
  }).eq("id", taskId);

  // Log stage change
  await supabase.from("activity_log").insert({
    project_id: projectId,
    description: `Task "${task.title}" moved to ${targetStage}`,
    log_type: "task_moved",
    log_date: new Date().toISOString(),
  });

  setDragTaskId(null);
  const updatedTasks = tasks.map(t =&gt; t.id === taskId ? { ...t, stage: targetStage } : t);
  setTasks(updatedTasks);
  syncProjectStage(updatedTasks);
  fetchTasks();
};

const getPriorityStyle = (priority: string) =&gt; {
  return PRIORITIES.find(p =&gt; p.value === priority)?.color || PRIORITIES[1].color;
};

const getTasksByStage = (stage: string) =&gt;
  tasks.filter(t =&gt; t.stage === stage).sort((a, b) =&gt; a.sort_order - b.sort_order);

const isOverdue = (dueDate: string | null) =&gt; {
  if (!dueDate) return false;
  return new Date(dueDate) &lt; new Date();
};

return (
  &lt;div className="space-y-4"&gt;
    &lt;ScrollArea className="w-full"&gt;
      &lt;div className="flex gap-3 pb-4 min-w-max"&gt;
        {KANBAN_STAGES.map(stage =&gt; {
          const stageTasks = getTasksByStage(stage);
          const colors = STAGE_COLORS[stage] || STAGE_COLORS["Lead"];
          const isDragTarget = dragOverStage === stage;

          return (
            &lt;div
              key={stage}
              className={cn(
                "w-[260px] flex-shrink-0 rounded-xl border transition-all",
                colors.bg,
                colors.border,
                isDragTarget &amp;&amp; "ring-2 ring-primary/40 scale-[1.01]"
              )}
              onDragOver={e =&gt; handleDragOver(e, stage)}
              onDragLeave={handleDragLeave}
              onDrop={e =&gt; handleDrop(e, stage)}
            &gt;
              {/* Column header */}
              &lt;div className={cn("px-3 py-2.5 rounded-t-xl flex items-center justify-between", colors.header)}&gt;
                &lt;div className="flex items-center gap-2"&gt;
                  &lt;h3 className="text-xs font-semibold text-foreground uppercase tracking-wider"&gt;{getStageLabel(stage)}&lt;/h3&gt;
                  &lt;span className="text-[10px] font-medium text-muted-foreground bg-background/60 rounded-full px-1.5 py-0.5"&gt;
                    {stageTasks.length}
                  &lt;/span&gt;
                &lt;/div&gt;
                &lt;Button variant="ghost" size="icon" className="h-6 w-6" onClick={() =&gt; openNew(stage)}&gt;
                  &lt;Plus className="h-3.5 w-3.5" /&gt;
                &lt;/Button&gt;
              &lt;/div&gt;

              {/* Cards */}
              &lt;div className="p-2 space-y-2 min-h-[80px]"&gt;
                {stageTasks.map(task =&gt; (
                  &lt;div
                    key={task.id}
                    draggable
                    onDragStart={e =&gt; handleDragStart(e, task.id)}
                    className={cn(
                      "group rounded-lg border bg-card p-3 cursor-grab active:cursor-grabbing shadow-sm hover:shadow-md transition-all",
                      task.is_completed &amp;&amp; "opacity-60",
                      dragTaskId === task.id &amp;&amp; "opacity-40 scale-95"
                    )}
                  &gt;
                    &lt;div className="flex items-start gap-2"&gt;
                      &lt;button
                        onClick={() =&gt; handleToggleComplete(task)}
                        className="mt-0.5 shrink-0"
                      &gt;
                        &lt;CheckCircle2 className={cn(
```

```
                "h-4 w-4 transition-colors",
                task.is_completed ? "text-chart-2 fill-chart-2/20" : "text-muted-foreground/40 hover:text-chart-2"
              )} /&gt;
            &lt;/button&gt;
            &lt;div className="flex-1 min-w-0"&gt;
              &lt;p className={cn(
                "text-sm font-medium text-foreground leading-tight",
                task.is_completed &amp;&amp; "line-through text-muted-foreground"
              )}&gt;
                {task.title}
              &lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="flex gap-0.5 opacity-0 group-hover:opacity-100 transition-opacity"&gt;
              &lt;Button variant="ghost" size="icon" className="h-5 w-5" onClick={() =&gt; openEdit(task)}&gt;
                &lt;Pencil className="h-3 w-3" /&gt;
              &lt;/Button&gt;
              &lt;Button variant="ghost" size="icon" className="h-5 w-5" onClick={() =&gt; handleDelete(task.id)}&gt;
                &lt;Trash2 className="h-3 w-3 text-destructive" /&gt;
              &lt;/Button&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          &lt;div className="flex items-center gap-2 mt-2 flex-wrap"&gt;
            {task.assigned_to &amp;&amp; (
              &lt;span className="inline-flex items-center gap-1 text-[10px] text-muted-foreground bg-muted rounded-full px-1.5 py-0.5"&gt;
                &lt;User className="h-2.5 w-2.5" /&gt;
                {task.assigned_to}
              &lt;/span&gt;
            )}
            {task.due_date &amp;&amp; (
              &lt;span className={cn(
                "inline-flex items-center gap-1 text-[10px] rounded-full px-1.5 py-0.5",
                isOverdue(task.due_date) &amp;&amp; !task.is_completed
                  ? "text-destructive bg-destructive/10"
                  : "text-muted-foreground bg-muted"
              )}&gt;
                &lt;Clock className="h-2.5 w-2.5" /&gt;
                {format(new Date(task.due_date), "MMM d")}
              &lt;/span&gt;
            )}
            &lt;span className={cn(
              "inline-flex items-center gap-1 text-[10px] font-medium rounded-full px-1.5 py-0.5 border",
              getPriorityStyle(task.priority)
            )}&gt;
              &lt;Flag className="h-2.5 w-2.5" /&gt;
              {task.priority}
            &lt;/span&gt;
            {task.requires_client_review &amp;&amp; (
              &lt;span className="inline-flex items-center gap-1 text-[10px] font-medium text-chart-5 bg-chart-5/15 border-chart-5/30 rounded-full px-1.5 py-0.5
                &lt;Eye className="h-2.5 w-2.5" /&gt;
                Client Review
              &lt;/span&gt;
            )}
          &lt;/div&gt;
        &lt;/div&gt;
      ))}

      {stageTasks.length === 0 &amp;&amp; (
        &lt;div className="text-center py-6 text-muted-foreground/40 text-xs"&gt;
          Drop tasks here
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  &lt;/div&gt;
);
})}
&lt;/div&gt;
&lt;/ScrollArea&gt;

{/* Task Dialog */}
&lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
  &lt;DialogContent className="max-w-md"&gt;
    &lt;DialogHeader&gt;
      &lt;DialogTitle&gt;{editingTask ? "Edit Task" : "New Task"}&lt;/DialogTitle&gt;
    &lt;/DialogHeader&gt;
    &lt;div className="space-y-4"&gt;
      &lt;div&gt;
        &lt;Label&gt;Task Name&lt;/Label&gt;
        &lt;Input
          value={form.title}
          onChange={e =&gt; setForm(f =&gt; ({ ...f, title: e.target.value }))}
          placeholder="e.g. Kitchen Layout"
          className="mt-1"
        /&gt;
      &lt;/div&gt;
      &lt;div className="grid grid-cols-2 gap-4"&gt;
        &lt;div&gt;
          &lt;Label&gt;Assigned To&lt;/Label&gt;
          &lt;Input
            value={form.assigned_to}
            onChange={e =&gt; setForm(f =&gt; ({ ...f, assigned_to: e.target.value }))}
            placeholder="e.g. Emily"
            className="mt-1"
          /&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label&gt;Priority&lt;/Label&gt;
          &lt;Select value={form.priority} onValueChange={v =&gt; setForm(f =&gt; ({ ...f, priority: v }))}&gt;
            &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
```

```
                {PRIORITIES.map(p =&gt; (
                  &lt;SelectItem key={p.value} value={p.value}&gt;
                    &lt;span className="flex items-center gap-2"&gt;
                      &lt;Flag className={cn("h-3 w-3", p.color.split(" ")[0])} /&gt;
                      {p.label}
                    &lt;/span&gt;
                  &lt;/SelectItem&gt;
                ))}
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label&gt;Due Date&lt;/Label&gt;
          &lt;Popover&gt;
            &lt;PopoverTrigger asChild&gt;
              &lt;Button
                variant="outline"
                className={cn(
                  "w-full mt-1 justify-start text-left font-normal",
                  !form.due_date &amp;&amp; "text-muted-foreground"
                )}
              &gt;
                &lt;CalendarIcon className="h-4 w-4 mr-2" /&gt;
                {form.due_date ? format(form.due_date, "PPP") : "Pick a date"}
              &lt;/Button&gt;
            &lt;/PopoverTrigger&gt;
            &lt;PopoverContent className="w-auto p-0" align="start"&gt;
              &lt;Calendar
                mode="single"
                selected={form.due_date}
                onSelect={d =&gt; setForm(f =&gt; ({ ...f, due_date: d }))}
                initialFocus
                className={cn("p-3 pointer-events-auto")}
              /&gt;
            &lt;/PopoverContent&gt;
          &lt;/Popover&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label&gt;Stage&lt;/Label&gt;
          &lt;Select value={form.stage} onValueChange={v =&gt; setForm(f =&gt; ({ ...f, stage: v }))}&gt;
            &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              {KANBAN_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{getStageLabel(s)}&lt;/SelectItem&gt;)}
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
        &lt;/div&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Checkbox
            checked={form.requires_client_review}
            onCheckedChange={v =&gt; setForm(f =&gt; ({ ...f, requires_client_review: !!v }))}
          /&gt;
          &lt;Label className="text-sm font-normal"&gt;Requires Client Review&lt;/Label&gt;
        &lt;/div&gt;
        &lt;Button className="w-full" onClick={handleSave}&gt;
          {editingTask ? "Save Changes" : "Create Task"}
        &lt;/Button&gt;
      &lt;/div&gt;
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;
);
}
```

src/components/admin/ProjectTimeline.tsx

```
import { useEffect, useState, useCallback } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Badge } from "@/components/ui/badge";
import { Textarea } from "@/components/ui/textarea";
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Progress } from "@/components/ui/progress";
import {
  Plus, CheckCircle2, Clock, DollarSign, FileText, Send,
  MessageSquare, Star, Package, Truck, ChevronDown, ChevronsUpDown,
  Trash2, RotateCcw, AlertTriangle,
} from "lucide-react";
import { format } from "date-fns";
import { cn } from "@/lib/utils";
import { toast } from "sonner";
import { useIsMobile } from "@/hooks/use-mobile";
import {
  REV_EVENTS, FINAL_EVENTS, TimelineEvent, STATUS_CONFIG, ALERT_CONFIG,
  getRevisionStatus, getRevisionAlert, getOverallAlert,
  getTimelineProgress, deriveStageFromTimeline, getRevisionDelay,
} from "./timeline/timelineUtils";

interface Props {
  projectId: string;
  deliveryDate?: string;
  dates?: {
    start_date?: string;
    delivery_date?: string;
    preview_delivery_date?: string;
    revision_return_date?: string;
    final_delivery_date?: string;
  };
  onDateChange?: (field: string, value: string) =&gt; void;
  onStageSync?: (stage: string) =&gt; void;
}

export default function ProjectTimeline({ projectId, deliveryDate, dates, onDateChange, onStageSync }: Props) {
  const [events, setEvents] = useState&lt;TimelineEvent[]&gt;([]);
  const [maxRevision, setMaxRevision] = useState(3);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [extraRevAmount, setExtraRevAmount] = useState("");
  const [addEventDialog, setAddEventDialog] = useState(false);
  const [newEvent, setNewEvent] = useState({ revision_number: 0, event_type: "", event_date: "", notes: "" });
  const [openRevisions, setOpenRevisions] = useState&lt;Record&lt;number, boolean&gt;&gt;({ 0: true });
  const [finalizationOpen, setFinalizationOpen] = useState(true);
  const isMobile = useIsMobile();

  const fetchEvents = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("project_timeline")
      .select("*")
      .eq("project_id", projectId)
      .order("revision_number")
      .order("created_at");
    const list = (data as TimelineEvent[]) || [];
    setEvents(list);
    const highestRev = list.reduce((m, e) =&gt; Math.max(m, e.revision_number), 3);
    setMaxRevision(highestRev &lt; 3 ? 3 : highestRev);
  }, [projectId]);

  useEffect(() =&gt; { fetchEvents(); }, [fetchEvents]);

  // Auto-sync stage from timeline events
  useEffect(() =&gt; {
    if (!onStageSync || events.length === 0) return;
    const derivedStage = deriveStageFromTimeline(events, maxRevision);
    if (derivedStage) onStageSync(derivedStage);
  }, [events, maxRevision, onStageSync]);

  // Auto-log activity on event changes
  const logActivity = async (description: string) =&gt; {
    await supabase.from("activity_log").insert({
      project_id: projectId,
      log_type: "timeline_event",
      description,
    });
  };

  const toggleAllRevisions = (expand: boolean) =&gt; {
    const newState: Record&lt;number, boolean&gt; = {};
    for (let i = 0; i &lt;= maxRevision; i++) newState[i] = expand;
    setOpenRevisions(newState);
    setFinalizationOpen(expand);
  };

  const markEvent = async (revNum: number, eventType: string) =&gt; {
    const existing = events.find(e =&gt; e.revision_number === revNum &amp;amp;&amp;amp; e.event_type === eventType);
    if (existing) return; // Use delete button instead
    await supabase.from("project_timeline").insert({
      project_id: projectId,
      revision_number: revNum,
      event_type: eventType,
      event_date: format(new Date(), "yyyy-MM-dd"),
```

```
  });
  // Find label
  const allEvents = revNum === -1 ? FINAL_EVENTS : REV_EVENTS(revNum);
  const evDef = allEvents.find(e =&gt; e.type === eventType);
  await logActivity(`${evDef?.label || eventType} – ${format(new Date(), "MMM d")}`);
  toast.success("Event recorded");
  fetchEvents();
};

const deleteEvent = async (eventId: string, label: string) =&gt; {
  await supabase.from("project_timeline").delete().eq("id", eventId);
  await logActivity(`${label} removed – ${format(new Date(), "MMM d")}`);
  toast.success("Event deleted");
  fetchEvents();
};

const resetEvent = async (eventId: string, label: string) =&gt; {
  await supabase.from("project_timeline").update({ event_date: null }).eq("id", eventId);
  await logActivity(`${label} reset – ${format(new Date(), "MMM d")}`);
  toast.success("Event reset");
  fetchEvents();
};

const updateEventDate = async (eventId: string, date: string) =&gt; {
  await supabase.from("project_timeline").update({ event_date: date }).eq("id", eventId);
  fetchEvents();
};

const handleAddExtraRevision = async () =&gt; {
  const newRev = maxRevision + 1;
  const revEvents = REV_EVENTS(newRev);
  await supabase.from("project_timeline").insert(
    revEvents.map(e =&gt; ({
      project_id: projectId,
      revision_number: newRev,
      event_type: e.type,
      event_date: null,
      notes: null,
    }))
  );
  if (extraRevAmount &amp;&amp; Number(extraRevAmount) &gt; 0) {
    await supabase.from("payment_milestones").insert({
      project_id: projectId,
      stage: `Extra Revision REV ${String(newRev).padStart(2, "0")}`,
      percentage: 0,
      amount: Number(extraRevAmount),
      status: "pending",
    });
  }
  await logActivity(`Extra Revision REV ${String(newRev).padStart(2, "0")} added – ${format(new Date(), "MMM d")}`);
  toast.success(`Extra Revision REV ${String(newRev).padStart(2, "0")} added`);
  setDialogOpen(false);
  setExtraRevAmount("");
  setOpenRevisions(prev =&gt; ({ ...prev, [newRev]: true }));
  fetchEvents();
};

const handleAddCustomEvent = async () =&gt; {
  if (!newEvent.event_type.trim()) { toast.error("Event type required"); return; }
  await supabase.from("project_timeline").insert({
    project_id: projectId,
    revision_number: newEvent.revision_number,
    event_type: newEvent.event_type,
    event_date: newEvent.event_date || null,
    notes: newEvent.notes || null,
  });
  await logActivity(`Custom event: ${newEvent.event_type} – ${format(new Date(), "MMM d")}`);
  toast.success("Event added");
  setAddEventDialog(false);
  setNewEvent({ revision_number: 0, event_type: "", event_date: "", notes: "" });
  fetchEvents();
};

const getEventRecord = (revNum: number, eventType: string) =&gt;
  events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === eventType);

// Overall stats
const overallAlert = getOverallAlert(events, deliveryDate || dates?.delivery_date);
const alertCfg = ALERT_CONFIG[overallAlert];
const progressPct = getTimelineProgress(events, maxRevision);

const renderEventRow = (ev: { type: string; label: string; icon: React.ElementType }, revNum: number) =&gt; {
  const record = getEventRecord(revNum, ev.type);
  const Icon = ev.icon;
  return (
    &lt;div
      key={ev.type}
      className={cn(
        "flex items-center gap-2 px-3 py-2 rounded-md transition-colors group",
        record?.event_date ? "bg-chart-2/5" : "bg-transparent hover:bg-muted/50"
      )}
    &gt;
      {/* Click to mark */}
      &lt;button
        className={cn(
          "h-6 w-6 rounded-full flex items-center justify-center shrink-0 transition-colors",
          record?.event_date
            ? "bg-chart-2/20 text-chart-2"
            : "bg-muted text-muted-foreground hover:bg-primary/20 hover:text-primary cursor-pointer"
        )}
        onClick={() =&gt; !record?.event_date &amp;&amp; markEvent(revNum, ev.type)}
```

```
        disabled={!!record?.event_date}
      &gt;
        {record?.event_date ? &lt;CheckCircle2 className="h-3.5 w-3.5" /&gt; : &lt;Icon className="h-3.5 w-3.5" /&gt;}
      &lt;/button&gt;

      &lt;div className="flex-1 min-w-0"&gt;
        &lt;p className={cn("text-sm", record?.event_date ? "text-foreground font-medium" : "text-muted-foreground")}&gt;
          {ev.label}
        &lt;/p&gt;
      &lt;/div&gt;

      {record?.event_date &amp;&amp; (
        &lt;span className="text-xs text-muted-foreground shrink-0 hidden sm:block"&gt;
          {format(new Date(record.event_date), "MMM d, yyyy")}
        &lt;/span&gt;
      )}

      {record &amp;&amp; (
        &lt;&gt;
          &lt;Input
            type="date"
            className="w-[120px] h-7 text-xs hidden sm:block"
            value={record.event_date || ""}
            onClick={e =&gt; e.stopPropagation()}
            onChange={e =&gt; updateEventDate(record.id, e.target.value)}
          /&gt;
          &lt;div className="flex items-center gap-0.5 opacity-0 group-hover:opacity-100 transition-opacity"&gt;
            &lt;Button
              variant="ghost"
              size="icon"
              className="h-6 w-6"
              title="Reset event"
              onClick={() =&gt; resetEvent(record.id, ev.label)}
            &gt;
              &lt;RotateCcw className="h-3 w-3 text-muted-foreground" /&gt;
            &lt;/Button&gt;
            &lt;Button
              variant="ghost"
              size="icon"
              className="h-6 w-6"
              title="Delete event"
              onClick={() =&gt; deleteEvent(record.id, ev.label)}
            &gt;
              &lt;Trash2 className="h-3 w-3 text-destructive" /&gt;
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/&gt;
      )}
    &lt;/div&gt;
  );
};

const renderRevisionBlock = (revNum: number) =&gt; {
  const revEvents = REV_EVENTS(revNum);
  const isExtra = revNum &gt; 3;
  const revLabel = `REV ${String(revNum).padStart(2, "0")}`;
  const status = getRevisionStatus(revNum, events);
  const statusCfg = STATUS_CONFIG[status];
  const alert = getRevisionAlert(revNum, events);
  const delay = getRevisionDelay(revNum, events);
  const completedCount = revEvents.filter(ev =&gt; getEventRecord(revNum, ev.type)?.event_date).length;
  const isOpen = openRevisions[revNum] ?? false;

  return (
    &lt;Collapsible
      key={`rev-${revNum}`}
      open={isOpen}
      onOpenChange={(open) =&gt; setOpenRevisions(prev =&gt; ({ ...prev, [revNum]: open }))}
    &gt;
      &lt;Card className={cn(
        "border shadow-sm",
        isExtra &amp;&amp; "border-l-4 border-l-chart-4",
        alert === "delayed" &amp;&amp; "border-destructive/50",
        alert === "waiting_client" &amp;&amp; "border-yellow-400/50 dark:border-yellow-600/50"
      )}&gt;
        &lt;CollapsibleTrigger asChild&gt;
          &lt;CardHeader className="pb-2 cursor-pointer hover:bg-muted/30 transition-colors"&gt;
            &lt;CardTitle className="text-sm flex items-center gap-2 flex-wrap"&gt;
              &lt;Package className="h-4 w-4 text-muted-foreground" /&gt;
              {revLabel}
              {revNum === 0 &amp;&amp; &lt;Badge variant="secondary" className="text-[10px]"&gt;Início&lt;/Badge&gt;}
              {revNum === 3 &amp;&amp; &lt;Badge variant="secondary" className="text-[10px]"&gt;Final&lt;/Badge&gt;}
              {isExtra &amp;&amp; &lt;Badge className="text-[10px] bg-chart-4/15 text-chart-4 border-chart-4/30"&gt;Extra&lt;/Badge&gt;}
              &lt;Badge className={cn("text-[10px] border-0", statusCfg.classes)}&gt;
                {statusCfg.icon} {statusCfg.label}
              &lt;/Badge&gt;
              {alert &amp;&amp; (
                &lt;Badge className={cn("text-[10px] border-0", ALERT_CONFIG[alert].classes)}&gt;
                  {ALERT_CONFIG[alert].icon} {ALERT_CONFIG[alert].label}
                &lt;/Badge&gt;
              )}
              {delay?.delayed &amp;&amp; (
                &lt;span className="text-[10px] text-destructive font-medium"&gt;
                  +{delay.days}d late
                &lt;/span&gt;
              )}
              &lt;span className="ml-auto text-xs text-muted-foreground font-normal"&gt;{completedCount}/{revEvents.length}&lt;/span&gt;
              &lt;ChevronDown className={cn("h-4 w-4 text-muted-foreground transition-transform", isOpen &amp;&amp; "rotate-180")} /&gt;
            &lt;/CardTitle&gt;
          &lt;/CardHeader&gt;
```

```
        &lt;/CollapsibleTrigger&gt;
        &lt;CollapsibleContent&gt;
          &lt;CardContent className="space-y-0.5 pt-0"&gt;
            {revEvents.map(ev =&gt; renderEventRow(ev, revNum))}
          &lt;/CardContent&gt;
        &lt;/CollapsibleContent&gt;
      &lt;/Card&gt;
    &lt;/Collapsible&gt;
  );
};

const renderFinalBlock = () =&gt; (
  &lt;Collapsible open={finalizationOpen} onOpenChange={setFinalizationOpen}&gt;
    &lt;Card className="border shadow-sm"&gt;
      &lt;CollapsibleTrigger asChild&gt;
        &lt;CardHeader className="pb-2 cursor-pointer hover:bg-muted/30 transition-colors"&gt;
          &lt;CardTitle className="text-sm flex items-center gap-2"&gt;
            &lt;Star className="h-4 w-4 text-chart-2" /&gt; Finalização
            &lt;ChevronDown className={cn("h-4 w-4 text-muted-foreground ml-auto transition-transform", finalizationOpen &amp;&amp; "rotate-180")} /&gt;
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
      &lt;/CollapsibleTrigger&gt;
      &lt;CollapsibleContent&gt;
        &lt;CardContent className="space-y-0.5 pt-0"&gt;
          {FINAL_EVENTS.map(ev =&gt; {
            const isAutoType = ev.type === "invoice_sent_auto" || ev.type === "payment_received_auto";
            const autoRecords = isAutoType ? events.filter(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === ev.type) : [];
            const Icon = ev.icon;

            if (isAutoType &amp;&amp; autoRecords.length &gt; 0) {
              return autoRecords.map(ar =&gt; (
                &lt;div key={ar.id} className="flex items-center gap-2 px-3 py-2 rounded-md bg-chart-2/5"&gt;
                  &lt;div className="h-6 w-6 rounded-full flex items-center justify-center shrink-0 bg-chart-2/20 text-chart-2"&gt;
                    &lt;CheckCircle2 className="h-3.5 w-3.5" /&gt;
                  &lt;/div&gt;
                  &lt;div className="flex-1 min-w-0"&gt;
                    &lt;p className="text-sm text-foreground font-medium"&gt;{ev.label}&lt;/p&gt;
                    {ar.notes &amp;&amp; &lt;p className="text-xs text-muted-foreground truncate"&gt;{ar.notes}&lt;/p&gt;}
                  &lt;/div&gt;
                  {ar.event_date &amp;&amp; (
                    &lt;span className="text-xs text-muted-foreground shrink-0"&gt;
                      {format(new Date(ar.event_date), "MMM d, yyyy")}
                    &lt;/span&gt;
                  )}
                &lt;/div&gt;
              ));
            }

            if (isAutoType) {
              return (
                &lt;div key={ev.type} className="flex items-center gap-2 px-3 py-2 rounded-md bg-transparent opacity-50"&gt;
                  &lt;div className="h-6 w-6 rounded-full flex items-center justify-center shrink-0 bg-muted text-muted-foreground"&gt;
                    &lt;Icon className="h-3.5 w-3.5" /&gt;
                  &lt;/div&gt;
                  &lt;p className="text-sm text-muted-foreground"&gt;{ev.label}&lt;/p&gt;
                  &lt;span className="text-[10px] text-muted-foreground ml-auto"&gt;Auto-tracked&lt;/span&gt;
                &lt;/div&gt;
              );
            }

            return renderEventRow(ev, -1);
          })}
        &lt;/CardContent&gt;
      &lt;/CollapsibleContent&gt;
    &lt;/Card&gt;
  &lt;/Collapsible&gt;
);

const revisionNumbers = Array.from({ length: maxRevision + 1 }, (_, i) =&gt; i);

return (
  &lt;div className="space-y-4"&gt;
    {/* Overall Status Bar */}
    &lt;Card className="border shadow-sm"&gt;
      &lt;CardContent className="py-4 space-y-3"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;div className="flex items-center gap-3"&gt;
            &lt;Badge className={cn("text-xs border-0", alertCfg.classes)}&gt;
              {alertCfg.icon} {alertCfg.label}
            &lt;/Badge&gt;
            &lt;span className="text-sm text-muted-foreground"&gt;Timeline Progress&lt;/span&gt;
          &lt;/div&gt;
          &lt;span className="text-sm font-semibold"&gt;{progressPct}%&lt;/span&gt;
        &lt;/div&gt;
        &lt;Progress value={progressPct} className="h-2" /&gt;
        {overallAlert === "overdue" &amp;&amp; (
          &lt;div className="flex items-center gap-2 text-xs text-destructive"&gt;
            &lt;AlertTriangle className="h-3.5 w-3.5" /&gt;
            Project is past deadline
          &lt;/div&gt;
        )}
        {overallAlert === "waiting_client" &amp;&amp; (
          &lt;div className="flex items-center gap-2 text-xs text-yellow-700 dark:text-yellow-400"&gt;
            &lt;Clock className="h-3.5 w-3.5" /&gt;
            Waiting for client feedback
          &lt;/div&gt;
        )}
        {overallAlert === "delayed" &amp;&amp; (
          &lt;div className="flex items-center gap-2 text-xs text-destructive"&gt;
            &lt;AlertTriangle className="h-3.5 w-3.5" /&gt;
            One or more revisions are delayed
```

```
      &lt;/div&gt;
    )}
  &lt;/CardContent&gt;
&lt;/Card&gt;

{/* Date fields */}
{dates &amp;&amp; onDateChange &amp;&amp; (
  &lt;Card className="border shadow-sm"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;CardTitle className="text-base"&gt;Key Dates&lt;/CardTitle&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent&gt;
      &lt;div className="grid grid-cols-2 lg:grid-cols-3 gap-4"&gt;
        &lt;div&gt;
          &lt;Label className="text-xs text-muted-foreground"&gt;Start Date&lt;/Label&gt;
          &lt;Input type="date" className="mt-1" value={dates.start_date || ""} onChange={e =&gt; onDateChange("start_date", e.target.value)} /&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label className="text-xs text-muted-foreground"&gt;Preview Delivery&lt;/Label&gt;
          &lt;Input type="date" className="mt-1" value={dates.preview_delivery_date || ""} onChange={e =&gt; onDateChange("preview_delivery_date", e.target.value)} /&g
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label className="text-xs text-muted-foreground"&gt;Client Revision Return&lt;/Label&gt;
          &lt;Input type="date" className="mt-1" value={dates.revision_return_date || ""} onChange={e =&gt; onDateChange("revision_return_date", e.target.value)} /&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label className="text-xs text-muted-foreground"&gt;Final Delivery&lt;/Label&gt;
          &lt;Input type="date" className="mt-1" value={dates.final_delivery_date || ""} onChange={e =&gt; onDateChange("final_delivery_date", e.target.value)} /&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label className="text-xs text-muted-foreground"&gt;Project Deadline&lt;/Label&gt;
          &lt;Input type="date" className="mt-1" value={dates.delivery_date || ""} onChange={e =&gt; onDateChange("delivery_date", e.target.value)} /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
)}

{/* Expand/Collapse All */}
&lt;div className="flex items-center justify-between"&gt;
  &lt;Button variant="ghost" size="sm" onClick={() =&gt; toggleAllRevisions(true)} className="text-xs gap-1"&gt;
    &lt;ChevronsUpDown className="h-3.5 w-3.5" /&gt; Expand All
  &lt;/Button&gt;
  &lt;Button variant="ghost" size="sm" onClick={() =&gt; toggleAllRevisions(false)} className="text-xs gap-1"&gt;
    Collapse All
  &lt;/Button&gt;
&lt;/div&gt;

{/* Revision blocks in responsive grid */}
&lt;div className={cn("grid gap-4", isMobile ? "grid-cols-1" : "grid-cols-1 lg:grid-cols-2")}&gt;
  {revisionNumbers.map(revNum =&gt; renderRevisionBlock(revNum))}
  {renderFinalBlock()}
&lt;/div&gt;

{/* Actions */}
&lt;div className="flex gap-2"&gt;
  &lt;Button variant="outline" onClick={() =&gt; setDialogOpen(true)}&gt;
    &lt;Plus className="h-3.5 w-3.5 mr-1" /&gt; Add Extra Revision
  &lt;/Button&gt;
  &lt;Button variant="ghost" onClick={() =&gt; setAddEventDialog(true)}&gt;
    &lt;Plus className="h-3.5 w-3.5 mr-1" /&gt; Add Custom Event
  &lt;/Button&gt;
&lt;/div&gt;

{/* Extra Revision Dialog */}
&lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
  &lt;DialogContent className="max-w-sm"&gt;
    &lt;DialogHeader&gt;
      &lt;DialogTitle&gt;Add Extra Revision (REV {String(maxRevision + 1).padStart(2, "0")})&lt;/DialogTitle&gt;
    &lt;/DialogHeader&gt;
    &lt;div className="space-y-4"&gt;
      &lt;div&gt;
        &lt;Label&gt;Invoice Amount (optional)&lt;/Label&gt;
        &lt;Input type="number" placeholder="e.g. 500" value={extraRevAmount} onChange={e =&gt; setExtraRevAmount(e.target.value)} className="mt-1" /&gt;
        &lt;p className="text-xs text-muted-foreground mt-1"&gt;A payment milestone will be created for this extra revision.&lt;/p&gt;
      &lt;/div&gt;
      &lt;Button className="w-full" onClick={handleAddExtraRevision}&gt;Create Extra Revision&lt;/Button&gt;
    &lt;/div&gt;
  &lt;/DialogContent&gt;
&lt;/Dialog&gt;

{/* Custom Event Dialog */}
&lt;Dialog open={addEventDialog} onOpenChange={setAddEventDialog}&gt;
  &lt;DialogContent className="max-w-sm"&gt;
    &lt;DialogHeader&gt;
      &lt;DialogTitle&gt;Add Custom Event&lt;/DialogTitle&gt;
    &lt;/DialogHeader&gt;
    &lt;div className="space-y-4"&gt;
      &lt;div&gt;
        &lt;Label&gt;Event Type&lt;/Label&gt;
        &lt;Input value={newEvent.event_type} onChange={e =&gt; setNewEvent(n =&gt; ({ ...n, event_type: e.target.value }))} placeholder="e.g. Client Meeting" classNam
      &lt;/div&gt;
      &lt;div&gt;
        &lt;Label&gt;Revision #&lt;/Label&gt;
        &lt;Select value={String(newEvent.revision_number)} onValueChange={v =&gt; setNewEvent(n =&gt; ({ ...n, revision_number: Number(v) }))}&gt;
          &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
          &lt;SelectContent&gt;
            &lt;SelectItem value="-1"&gt;Finalização&lt;/SelectItem&gt;
            {Array.from({ length: maxRevision + 1 }, (_, i) =&gt; (
              &lt;SelectItem key={i} value={String(i)}&gt;REV {String(i).padStart(2, "0")}&lt;/SelectItem&gt;
            ))}
```

```
                &lt;/SelectContent&gt;
              &lt;/Select&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Date&lt;/Label&gt;
              &lt;Input type="date" value={newEvent.event_date} onChange={e =&gt; setNewEvent(n =&gt; ({ ...n, event_date: e.target.value }))} className="mt-1" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Notes&lt;/Label&gt;
              &lt;Textarea value={newEvent.notes} onChange={e =&gt; setNewEvent(n =&gt; ({ ...n, notes: e.target.value }))} className="mt-1" /&gt;
            &lt;/div&gt;
            &lt;Button className="w-full" onClick={handleAddCustomEvent}&gt;Add Event&lt;/Button&gt;
          &lt;/div&gt;
        &lt;/DialogContent&gt;
      &lt;/Dialog&gt;
    &lt;/div&gt;
  );
}
```

src/components/admin/dashboard/ActivityFeed.tsx

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { MessageSquare, FileUp, CreditCard, CheckCircle } from "lucide-react";
import { formatDistanceToNow } from "date-fns";

interface ActivityFeedProps {
  activities: any[];
}

const iconMap: Record&lt;string, any&gt; = {
  message: MessageSquare,
  file_upload: FileUp,
  payment: CreditCard,
  default: CheckCircle,
};

const ActivityFeed = ({ activities }: ActivityFeedProps) =&gt; {
  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;CardTitle className="text-base font-semibold"&gt;Recent Activity&lt;/CardTitle&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;div className="space-y-3 max-h-[300px] overflow-y-auto"&gt;
          {activities.length === 0 &amp;amp;&amp; (
            &lt;p className="text-sm text-muted-foreground text-center py-6"&gt;No recent activity&lt;/p&gt;
          )}
          {activities.slice(0, 8).map((a) =&gt; {
            const Icon = iconMap[a.log_type] || iconMap.default;
            return (
              &lt;div key={a.id} className="flex items-start gap-3 p-2 rounded-md hover:bg-muted/50 transition-colors"&gt;
                &lt;div className="p-1.5 rounded-md bg-primary/10 text-primary mt-0.5"&gt;
                  &lt;Icon className="h-3.5 w-3.5" /&gt;
                &lt;/div&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;p className="text-sm text-foreground line-clamp-1"&gt;{a.description}&lt;/p&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;
                    {formatDistanceToNow(new Date(a.created_at), { addSuffix: true })}
                  &lt;/p&gt;
                &lt;/div&gt;
                &lt;Badge variant="outline" className="text-[10px] shrink-0"&gt;{a.log_type}&lt;/Badge&gt;
              &lt;/div&gt;
            );
          })}
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
};

export default ActivityFeed;
```

src/components/admin/dashboard/ClientActivityFeed.tsx

```
import { useNavigate } from "react-router-dom";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Avatar, AvatarFallback } from "@/components/ui/avatar";
import { ScrollArea } from "@/components/ui/scroll-area";
import {
  CheckCircle,
  Upload,
  RotateCcw,
  MessageSquare,
  FileCheck,
  Download,
  Eye,
  Bell,
} from "lucide-react";
import { formatDistanceToNow } from "date-fns";
import { getCountryFlag } from "@/utils/countryFlags";

interface ClientActivity {
  id: string;
  client_name: string;
  client_country?: string | null;
  company_id?: string | null;
  project_name: string;
  action: string;
  action_type: string;
  created_at: string;
}

interface ClientActivityFeedProps {
  activities: ClientActivity[];
}

const ACTION_CONFIG: Record&lt;string, { icon: any; color: string; label: string }&gt; = {
  approval: { icon: CheckCircle, color: "text-emerald-500 bg-emerald-500/10", label: "Approved" },
  upload: { icon: Upload, color: "text-blue-500 bg-blue-500/10", label: "Uploaded" },
  revision: { icon: RotateCcw, color: "text-amber-500 bg-amber-500/10", label: "Revision" },
  message: { icon: MessageSquare, color: "text-violet-500 bg-violet-500/10", label: "Message" },
  invoice: { icon: FileCheck, color: "text-emerald-600 bg-emerald-600/10", label: "Invoice" },
  download: { icon: Download, color: "text-cyan-500 bg-cyan-500/10", label: "Download" },
  view: { icon: Eye, color: "text-muted-foreground bg-muted", label: "Viewed" },
  default: { icon: Bell, color: "text-muted-foreground bg-muted", label: "Activity" },
};

const getInitials = (name: string) =&gt;
  name.split(" ").map((w) =&gt; w[0]).join("").toUpperCase().slice(0, 2);

const ClientActivityFeed = ({ activities }: ClientActivityFeedProps) =&gt; {
  const navigate = useNavigate();
  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;CardTitle className="text-base font-semibold"&gt;Client Activity&lt;/CardTitle&gt;
          {activities.length &gt; 0 &amp;&amp; (
            &lt;Badge variant="secondary" className="text-[10px]"&gt;
              {activities.length} recent
            &lt;/Badge&gt;
          )}
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;ScrollArea className="h-[340px] pr-3"&gt;
          {activities.length === 0 &amp;&amp; (
            &lt;div className="flex flex-col items-center justify-center py-12 text-center"&gt;
              &lt;Bell className="h-8 w-8 text-muted-foreground/40 mb-2" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;No client activity yet&lt;/p&gt;
            &lt;/div&gt;
          )}
          &lt;div className="space-y-1"&gt;
            {activities.map((a) =&gt; {
              const config = ACTION_CONFIG[a.action_type] || ACTION_CONFIG.default;
              const Icon = config.icon;
              return (
                &lt;div
                  key={a.id}
                  className="flex items-start gap-3 p-2.5 rounded-lg hover:bg-muted/50 transition-colors group"
                &gt;
                  &lt;Avatar className="h-8 w-8 shrink-0 mt-0.5"&gt;
                    &lt;AvatarFallback className="text-[10px] font-medium bg-primary/10 text-primary"&gt;
                      {getInitials(a.client_name)}
                    &lt;/AvatarFallback&gt;
                  &lt;/Avatar&gt;
                  &lt;div class="flex-1 min-w-0"&gt;
                    &lt;p class="text-sm leading-snug"&gt;
                      &lt;span
                        class="font-medium text-foreground hover:text-primary cursor-pointer hover:underline"
                        onClick={() =&gt; a.company_id &amp;&amp; navigate(`/adm/clients/${a.company_id}`)}
                      &gt;{a.client_name} {getCountryFlag(a.client_country)}&lt;/span&gt;{" "}
                      &lt;span class="text-muted-foreground"&gt;{a.action}&lt;/span&gt;
                    &lt;/p&gt;
                    &lt;p class="text-xs text-muted-foreground mt-0.5 truncate"&gt;{a.project_name}&lt;/p&gt;
                  &lt;/div&gt;
                  &lt;div class="flex flex-col items-end gap-1 shrink-0"&gt;
                    &lt;div class={`p-1 rounded-md ${config.color}`}&gt;
                      &lt;Icon class="h-3.5 w-3.5" /&gt;
                    &lt;/div&gt;
                    &lt;span class="text-[10px] text-muted-foreground whitespace-nowrap"&gt;
```

```
                      {formatDistanceToNow(new Date(a.created_at), { addSuffix: true })}
                    &lt;/span&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              );
            })}
          &lt;/div&gt;
        &lt;/ScrollArea&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
};

export default ClientActivityFeed;
```

src/components/admin/dashboard/ClientTicketsWidget.tsx

```
import { useEffect, useState } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Button } from "@/components/ui/button";
import { supabase } from "@/integrations/supabase/client";
import { Ticket, MessageSquare, ArrowRight } from "lucide-react";
import { formatDistanceToNow } from "date-fns";
import { useNavigate } from "react-router-dom";

interface TicketRow {
  id: string;
  subject: string;
  status: string;
  updated_at: string;
  projects: { name: string } | null;
  companies: { name: string } | null;
}

const STATUS_STYLES: Record&lt;string, { label: string; variant: "default" | "secondary" | "outline" | "destructive" }&gt; = {
  open: { label: "Open", variant: "destructive" },
  waiting: { label: "Waiting", variant: "secondary" },
  resolved: { label: "Resolved", variant: "outline" },
};

const ClientTicketsWidget = () =&gt; {
  const [tickets, setTickets] = useState&lt;TicketRow[]&gt;([]);
  const navigate = useNavigate();

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      const { data } = await supabase
        .from("client_tickets")
        .select("id, subject, status, updated_at, projects(name), companies(name)")
        .order("updated_at", { ascending: false })
        .limit(8);
      setTickets((data as any[]) || []);
    };
    fetch();
  }, []);

  const openCount = tickets.filter((t) =&gt; t.status === "open").length;

  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;CardTitle className="text-base font-semibold"&gt;Client Tickets&lt;/CardTitle&gt;
          &lt;div className="flex items-center gap-2"&gt;
            {openCount &gt; 0 &amp;&amp; (
              &lt;Badge variant="destructive" className="text-[10px]"&gt;
                {openCount} open
              &lt;/Badge&gt;
            )}
            &lt;Button variant="ghost" size="sm" className="h-7 text-xs" onClick={() =&gt; navigate("/adm/tickets")}&gt;
              View All &lt;ArrowRight className="h-3 w-3 ml-1" /&gt;
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;ScrollArea className="h-[300px] pr-3"&gt;
          {tickets.length === 0 &amp;&amp; (
            &lt;div className="flex flex-col items-center justify-center py-12 text-center"&gt;
              &lt;Ticket className="h-8 w-8 text-muted-foreground/40 mb-2" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;No tickets yet&lt;/p&gt;
            &lt;/div&gt;
          )}
          &lt;div className="space-y-1"&gt;
            {tickets.map((t) =&gt; {
              const style = STATUS_STYLES[t.status] || STATUS_STYLES.open;
              return (
                &lt;div
                  key={t.id}
                  className="flex items-start gap-3 p-2.5 rounded-lg hover:bg-muted/50 transition-colors cursor-pointer group"
                  onClick={() =&gt; navigate(`/adm/tickets?id=${t.id}`)}
                &gt;
                  &lt;div className="p-1.5 rounded-md bg-accent text-accent-foreground mt-0.5"&gt;
                    &lt;MessageSquare className="h-3.5 w-3.5" /&gt;
                  &lt;/div&gt;
                  &lt;div className="flex-1 min-w-0"&gt;
                    &lt;p className="text-sm font-medium text-foreground line-clamp-1"&gt;{t.subject}&lt;/p&gt;
                    &lt;p className="text-xs text-muted-foreground truncate"&gt;
                      {t.companies?.name || "Client"} · {t.projects?.name || "Project"}
                    &lt;/p&gt;
                  &lt;/div&gt;
                  &lt;div className="flex flex-col items-end gap-1 shrink-0"&gt;
                    &lt;Badge variant={style.variant} className="text-[10px]"&gt;{style.label}&lt;/Badge&gt;
                    &lt;span className="text-[10px] text-muted-foreground"&gt;
                      {formatDistanceToNow(new Date(t.updated_at), { addSuffix: true })}
                    &lt;/span&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              );
            })}
          &lt;/div&gt;
        &lt;/ScrollArea&gt;
      &lt;/CardContent&gt;
```

```
    &lt;/Card&gt;
  );
};

export default ClientTicketsWidget;
```

src/components/admin/dashboard/FinancialChart.tsx

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import {
  BarChart, Bar, LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Legend,
} from "recharts";
import { DollarSign, TrendingUp, BarChart3 } from "lucide-react";

interface FinancialChartProps {
  payments: any[];
  projects: any[];
}

const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];

const tooltipStyle = {
  backgroundColor: "hsl(var(--card))",
  border: "1px solid hsl(var(--border))",
  borderRadius: "8px",
  fontSize: 12,
};

const FinancialChart = ({ payments, projects }: FinancialChartProps) =&gt; {
  // 1. Revenue per Month
  const monthlyRevenue: Record&lt;string, number&gt; = {};
  payments.forEach((p) =&gt; {
    if (p.status === "paid") {
      const date = new Date(p.payment_date || p.created_at);
      const key = months[date.getMonth()];
      monthlyRevenue[key] = (monthlyRevenue[key] || 0) + Number(p.amount_paid || 0);
    }
  });
  const revenueData = months
    .filter((m) =&gt; monthlyRevenue[m])
    .map((m) =&gt; ({ month: m, revenue: monthlyRevenue[m] }));

  // 2. Projects Started vs Completed per Month
  const projectMonthly: Record&lt;string, { started: number; completed: number }&gt; = {};
  projects.forEach((p) =&gt; {
    if (p.start_date) {
      const m = months[new Date(p.start_date).getMonth()];
      if (!projectMonthly[m]) projectMonthly[m] = { started: 0, completed: 0 };
      projectMonthly[m].started++;
    }
    if (p.stage === "Completed" &amp;&amp; p.updated_at) {
      const m = months[new Date(p.updated_at).getMonth()];
      if (!projectMonthly[m]) projectMonthly[m] = { started: 0, completed: 0 };
      projectMonthly[m].completed++;
    }
  });
  const projectData = months
    .filter((m) =&gt; projectMonthly[m])
    .map((m) =&gt; ({ month: m, ...projectMonthly[m] }));

  // 3. Pipeline Value
  const pipelineTotal = projects
    .filter((p) =&gt; p.stage !== "Completed" &amp;&amp; p.stage !== "Lead" &amp;&amp; p.stage !== "briefing")
    .reduce((s, p) =&gt; s + Number(p.total_value || 0), 0);
  const totalRevenue = payments
    .filter((p) =&gt; p.status === "paid")
    .reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const totalPending = payments
    .filter((p) =&gt; p.status !== "paid")
    .reduce((s, p) =&gt; s + Number(p.amount_total || 0), 0);

  const fmt = (v: number) =&gt;
    v &gt;= 1000 ? `$${(v / 1000).toFixed(1)}k` : `$${v.toFixed(0)}`;

  return (
    &lt;div className="space-y-6"&gt;
      {/* Revenue per Month */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;DollarSign className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base font-semibold"&gt;Revenue per Month&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          {revenueData.length === 0 ? (
            &lt;p className="text-sm text-muted-foreground text-center py-10"&gt;No payment data yet&lt;/p&gt;
          ) : (
            &lt;ResponsiveContainer width="100%" height={220}&gt;
              &lt;BarChart data={revenueData}&gt;
                &lt;CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" /&gt;
                &lt;XAxis dataKey="month" tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" /&gt;
                &lt;YAxis tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" tickFormatter={fmt} /&gt;
                &lt;Tooltip contentStyle={tooltipStyle} formatter={(v: number) =&gt; [`$${v.toLocaleString()}`, "Revenue"]} /&gt;
                &lt;Bar dataKey="revenue" fill="hsl(var(--chart-2))" radius={[4, 4, 0, 0]} /&gt;
              &lt;/BarChart&gt;
            &lt;/ResponsiveContainer&gt;
          )}
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Projects Started vs Completed */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
```

```
            &lt;TrendingUp className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base font-semibold"&gt;Projects Started vs Completed&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          {projectData.length === 0 ? (
            &lt;p className="text-sm text-muted-foreground text-center py-10"&gt;No project timeline data yet&lt;/p&gt;
          ) : (
            &lt;ResponsiveContainer width="100%" height={220}&gt;
              &lt;LineChart data={projectData}&gt;
                &lt;CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" /&gt;
                &lt;XAxis dataKey="month" tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" /&gt;
                &lt;YAxis tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" allowDecimals={false} /&gt;
                &lt;Tooltip contentStyle={tooltipStyle} /&gt;
                &lt;Legend wrapperStyle={{ fontSize: 12 }} /&gt;
                &lt;Line type="monotone" dataKey="started" stroke="hsl(var(--chart-1))" strokeWidth={2} dot={{ r: 4 }} name="Started" /&gt;
                &lt;Line type="monotone" dataKey="completed" stroke="hsl(var(--chart-2))" strokeWidth={2} dot={{ r: 4 }} name="Completed" /&gt;
              &lt;/LineChart&gt;
            &lt;/ResponsiveContainer&gt;
          )}
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Pipeline Value Summary */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;BarChart3 className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base font-semibold"&gt;Revenue Pipeline&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          &lt;div className="grid grid-cols-3 gap-4"&gt;
            &lt;div className="text-center p-4 rounded-lg bg-muted/50"&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;{fmt(totalRevenue)}&lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground mt-1"&gt;Total Received&lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="text-center p-4 rounded-lg bg-muted/50"&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;{fmt(totalPending)}&lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground mt-1"&gt;Pending Payments&lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="text-center p-4 rounded-lg bg-muted/50"&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;{fmt(pipelineTotal)}&lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground mt-1"&gt;Pipeline Value&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default FinancialChart;
```

src/components/admin/dashboard/KPICards.tsx

```
import { useNavigate } from "react-router-dom";
import { Card, CardContent } from "@/components/ui/card";
import { TrendingUp, TrendingDown, Minus, UserPlus, FolderKanban, FileText, DollarSign, AlertTriangle, MessageSquareWarning } from "lucide-react";

interface KPICardsProps {
  totalProjects: number;
  activeProjects: number;
  totalClients: number;
  newLeads: number;
  revenue: number;
  pendingPayments: number;
  delayedProjects?: number;
  proposalsSent?: number;
  clientTickets?: number;
}

const KPICards = ({
  activeProjects,
  newLeads,
  revenue,
  delayedProjects = 0,
  proposalsSent = 0,
  clientTickets = 0,
}: KPICardsProps) =&gt; {
  const navigate = useNavigate();

  const cards = [
    {
      label: "Leads Received", sublabel: "Last 30 days", value: newLeads,
      icon: UserPlus, color: "text-blue-600", bg: "bg-blue-50 dark:bg-blue-950/40",
      iconBg: "bg-blue-100 dark:bg-blue-900/50", borderAccent: "border-l-blue-500",
      change: "+4", changeType: "up" as const, url: "/adm/leads",
    },
    {
      label: "Active Projects", sublabel: "In progress", value: activeProjects,
      icon: FolderKanban, color: "text-emerald-600", bg: "bg-emerald-50 dark:bg-emerald-950/40",
      iconBg: "bg-emerald-100 dark:bg-emerald-900/50", borderAccent: "border-l-emerald-500",
      change: "+2", changeType: "up" as const, url: "/adm/projects",
    },
    {
      label: "Proposals Sent", sublabel: "This month", value: proposalsSent,
      icon: FileText, color: "text-amber-600", bg: "bg-amber-50 dark:bg-amber-950/40",
      iconBg: "bg-amber-100 dark:bg-amber-900/50", borderAccent: "border-l-amber-500",
      change: "0", changeType: "neutral" as const, url: "/adm/proposals",
    },
    {
      label: "Revenue Pipeline", sublabel: "Total value", value: `$${revenue.toLocaleString()}`,
      icon: DollarSign, color: "text-purple-600", bg: "bg-purple-50 dark:bg-purple-950/40",
      iconBg: "bg-purple-100 dark:bg-purple-900/50", borderAccent: "border-l-purple-500",
      change: "+12%", changeType: "up" as const, url: "/adm/analytics",
    },
    {
      label: "Delayed Projects", sublabel: "Needs attention", value: delayedProjects,
      icon: AlertTriangle, color: "text-red-600", bg: "bg-red-50 dark:bg-red-950/40",
      iconBg: "bg-red-100 dark:bg-red-900/50", borderAccent: "border-l-red-500",
      change: delayedProjects &gt; 0 ? `${delayedProjects}` : "0",
      changeType: delayedProjects &gt; 0 ? "down" as const : "neutral" as const,
      url: "/adm/projects",
    },
    {
      label: "Client Tickets", sublabel: "Open requests", value: clientTickets,
      icon: MessageSquareWarning, color: "text-orange-600", bg: "bg-orange-50 dark:bg-orange-950/40",
      iconBg: "bg-orange-100 dark:bg-orange-900/50", borderAccent: "border-l-orange-500",
      change: clientTickets &gt; 0 ? `${clientTickets} open` : "0",
      changeType: clientTickets &gt; 0 ? "down" as const : "neutral" as const,
      url: "/adm/tickets",
    },
  ];

  return (
    &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-6 gap-4"&gt;
      {cards.map((card) =&gt; (
        &lt;Card
          key={card.label}
          onClick={() =&gt; navigate(card.url)}
          className={`border-none shadow-sm hover:shadow-md transition-all border-l-4 ${card.borderAccent} ${card.bg} cursor-pointer`}
        &gt;
          &lt;CardContent className="p-4"&gt;
            &lt;div className="flex items-start justify-between mb-3"&gt;
              &lt;div className={`p-2 rounded-lg ${card.iconBg}`}&gt;
                &lt;card.icon className={`h-4 w-4 ${card.color}`} /&gt;
              &lt;/div&gt;
              &lt;div className="flex items-center gap-1 text-[11px] font-medium"&gt;
                {card.changeType === "up" &amp;&amp; &lt;TrendingUp className="h-3 w-3 text-emerald-500" /&gt;}
                {card.changeType === "down" &amp;&amp; &lt;TrendingDown className="h-3 w-3 text-red-500" /&gt;}
                {card.changeType === "neutral" &amp;&amp; &lt;Minus className="h-3 w-3 text-muted-foreground" /&gt;}
                &lt;span className={
                  card.changeType === "up" ? "text-emerald-600" :
                  card.changeType === "down" ? "text-red-600" :
                  "text-muted-foreground"
                }&gt;
                  {card.change}
                &lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;p className="text-2xl font-bold text-foreground leading-none mb-1"&gt;{card.value}&lt;/p&gt;
            &lt;p className="text-xs font-medium text-muted-foreground"&gt;{card.label}&lt;/p&gt;
            &lt;p className="text-[10px] text-muted-foreground/70"&gt;{card.sublabel}&lt;/p&gt;
```

```
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      ))}
    &lt;/div&gt;
  );
};

export default KPICards;
```

src/components/admin/dashboard/NewLeadsTable.tsx

```
import { useEffect, useState } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuTrigger } from "@/components/ui/dropdown-menu";
import { formatDistanceToNow } from "date-fns";
import { supabase } from "@/integrations/supabase/client";
import { MoreHorizontal, UserPlus, Send, Phone, Inbox } from "lucide-react";
import { toast } from "sonner";

const NewLeadsTable = () =&gt; {
  const [leads, setLeads] = useState&lt;any[]&gt;([]);

  const fetchLeads = async () =&gt; {
    const { data } = await supabase
      .from("quote_requests")
      .select("*")
      .order("created_at", { ascending: false })
      .limit(10);
    setLeads(data || []);
  };

  useEffect(() =&gt; {
    fetchLeads();
  }, []);

  const handleConvert = async (lead: any) =&gt; {
    // Create company + project from lead
    const { data: company, error: compErr } = await supabase
      .from("companies")
      .insert({ name: lead.company_name || lead.full_name, email: lead.email, phone: lead.phone })
      .select()
      .single();

    if (compErr || !company) {
      toast.error("Failed to create client");
      return;
    }

    const { error: projErr } = await supabase.from("projects").insert({
      company_id: company.id,
      name: `${lead.full_name} - ${lead.service_type}`,
      service_type: lead.service_type,
      stage: "Lead",
      description: lead.description,
      address: lead.project_address,
    });

    if (!projErr) {
      await supabase.from("quote_requests").update({ processed: true }).eq("id", lead.id);
      toast.success("Lead converted to project");
      fetchLeads();
    } else {
      toast.error("Failed to create project");
    }
  };

  const handleMarkProcessed = async (id: string) =&gt; {
    await supabase.from("quote_requests").update({ processed: true }).eq("id", id);
    toast.success("Marked as processed");
    fetchLeads();
  };

  const serviceLabel = (s: string) =&gt;
    s?.replace(/_/g, " ").replace(/\b\w/g, (c) =&gt; c.toUpperCase()) || "—";

  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Inbox className="h-4 w-4 text-muted-foreground" /&gt;
          &lt;CardTitle className="text-base font-semibold"&gt;New Leads&lt;/CardTitle&gt;
          &lt;Badge variant="secondary" className="ml-auto"&gt;
            {leads.filter((l) =&gt; !l.processed).length} new
          &lt;/Badge&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead className="text-xs"&gt;Lead Name&lt;/TableHead&gt;
              &lt;TableHead className="text-xs"&gt;Project Type&lt;/TableHead&gt;
              &lt;TableHead className="text-xs"&gt;Status&lt;/TableHead&gt;
              &lt;TableHead className="text-xs"&gt;Received&lt;/TableHead&gt;
              &lt;TableHead className="text-xs w-[50px]"&gt;&lt;/TableHead&gt;
            &lt;/TableRow&gt;
          &lt;/TableHeader&gt;
          &lt;TableBody&gt;
            {leads.length === 0 &amp;amp;&amp; (
              &lt;TableRow&gt;
                &lt;TableCell colSpan={5} className="text-center text-sm text-muted-foreground py-8"&gt;
                  No leads from intake form yet
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            )}
            {leads.map((lead) =&gt; (
```

```
              &lt;TableRow key={lead.id}&gt;
                &lt;TableCell&gt;
                  &lt;div&gt;
                    &lt;p className="text-sm font-medium text-foreground"&gt;{lead.full_name}&lt;/p&gt;
                    &lt;p className="text-xs text-muted-foreground"&gt;{lead.email}&lt;/p&gt;
                  &lt;/div&gt;
                &lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;
                  {serviceLabel(lead.service_type)}
                &lt;/TableCell&gt;
                &lt;TableCell&gt;
                  {lead.processed ? (
                    &lt;Badge variant="secondary" className="text-[10px]"&gt;Processed&lt;/Badge&gt;
                  ) : (
                    &lt;Badge className="text-[10px] bg-chart-1/10 text-chart-1 border-chart-1/20" variant="outline"&gt;
                      New
                    &lt;/Badge&gt;
                  )}
                &lt;/TableCell&gt;
                &lt;TableCell className="text-xs text-muted-foreground"&gt;
                  {formatDistanceToNow(new Date(lead.created_at), { addSuffix: true })}
                &lt;/TableCell&gt;
                &lt;TableCell&gt;
                  &lt;DropdownMenu&gt;
                    &lt;DropdownMenuTrigger asChild&gt;
                      &lt;Button variant="ghost" size="icon" className="h-7 w-7"&gt;
                        &lt;MoreHorizontal className="h-4 w-4" /&gt;
                      &lt;/Button&gt;
                    &lt;/DropdownMenuTrigger&gt;
                    &lt;DropdownMenuContent align="end"&gt;
                      &lt;DropdownMenuItem onClick={() =&gt; handleConvert(lead)} className="gap-2"&gt;
                        &lt;UserPlus className="h-3.5 w-3.5" /&gt; Convert to Project
                      &lt;/DropdownMenuItem&gt;
                      &lt;DropdownMenuItem onClick={() =&gt; window.open(`mailto:${lead.email}?subject=Proposal`, "_blank")} className="gap-2"&gt;
                        &lt;Send className="h-3.5 w-3.5" /&gt; Send Proposal
                      &lt;/DropdownMenuItem&gt;
                      &lt;DropdownMenuItem onClick={() =&gt; lead.phone &amp;&amp; window.open(`tel:${lead.phone}`, "_blank")} className="gap-2"&gt;
                        &lt;Phone className="h-3.5 w-3.5" /&gt; Schedule Call
                      &lt;/DropdownMenuItem&gt;
                      {!lead.processed &amp;&amp; (
                        &lt;DropdownMenuItem onClick={() =&gt; handleMarkProcessed(lead.id)} className="gap-2 text-muted-foreground"&gt;
                          Mark as Processed
                        &lt;/DropdownMenuItem&gt;
                      )}
                    &lt;/DropdownMenuContent&gt;
                  &lt;/DropdownMenu&gt;
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
};

export default NewLeadsTable;
```

src/components/admin/dashboard/ProjectPipeline.tsx

```
import { useState, useRef, DragEvent } from "react";
import { useNavigate } from "react-router-dom";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { ScrollArea, ScrollBar } from "@/components/ui/scroll-area";
import { Calendar, DollarSign, GripVertical, ImageIcon } from "lucide-react";
import { getCountryFlag } from "@/utils/countryFlags";
import { supabase } from "@/integrations/supabase/client";
import { toast } from "sonner";

const PIPELINE_STAGES = [
  { key: "Lead", color: "bg-slate-400", dot: "bg-slate-500" },
  { key: "Discovery Call", color: "bg-sky-400", dot: "bg-sky-500" },
  { key: "Proposal Sent", color: "bg-amber-400", dot: "bg-amber-500" },
  { key: "Contract Signed", color: "bg-violet-400", dot: "bg-violet-500" },
  { key: "Concept Design", color: "bg-indigo-400", dot: "bg-indigo-500" },
  { key: "Design Development", color: "bg-blue-400", dot: "bg-blue-500" },
  { key: "Construction Documents", color: "bg-cyan-400", dot: "bg-cyan-500" },
  { key: "Permit Submission", color: "bg-teal-400", dot: "bg-teal-500" },
  { key: "Construction", color: "bg-emerald-400", dot: "bg-emerald-500" },
  { key: "Completed", color: "bg-green-500", dot: "bg-green-600" },
];

interface ProjectPipelineProps {
  projects: any[];
  onProjectMoved?: () =&gt; void;
}

const ProjectPipeline = ({ projects, onProjectMoved }: ProjectPipelineProps) =&gt; {
  const navigate = useNavigate();
  const [draggedId, setDraggedId] = useState&lt;string | null&gt;(null);
  const [dragOverStage, setDragOverStage] = useState&lt;string | null&gt;(null);
  const scrollRef = useRef&lt;HTMLDivElement&gt;(null);

  const grouped: Record&lt;string, any[]&gt; = {};
  PIPELINE_STAGES.forEach((s) =&gt; (grouped[s.key] = []));
  projects.forEach((p) =&gt; {
    const stage = p.stage || "Lead";
    if (grouped[stage]) {
      grouped[stage].push(p);
    } else {
      // Map old stages to new pipeline
      if (stage === "briefing" || stage === "Budget Sent") grouped["Lead"].push(p);
      else if (stage === "Budget Approved") grouped["Proposal Sent"].push(p);
      else if (stage === "In Progress") grouped["Design Development"].push(p);
      else if (stage === "Preview Sent" || stage === "Revision" || stage === "Final Review") grouped["Construction Documents"].push(p);
      else grouped["Lead"].push(p);
    }
  });

  const handleDragStart = (e: DragEvent, projectId: string) =&gt; {
    setDraggedId(projectId);
    e.dataTransfer.effectAllowed = "move";
    e.dataTransfer.setData("text/plain", projectId);
  };

  const handleDragOver = (e: DragEvent, stageKey: string) =&gt; {
    e.preventDefault();
    e.dataTransfer.dropEffect = "move";
    setDragOverStage(stageKey);
  };

  const handleDragLeave = () =&gt; {
    setDragOverStage(null);
  };

  const handleDrop = async (e: DragEvent, stageKey: string) =&gt; {
    e.preventDefault();
    setDragOverStage(null);
    const projectId = e.dataTransfer.getData("text/plain");
    if (!projectId) return;

    const { error } = await supabase
      .from("projects")
      .update({ stage: stageKey })
      .eq("id", projectId);

    if (error) {
      toast.error("Failed to move project");
    } else {
      toast.success(`Moved to ${stageKey}`);
      onProjectMoved?.();
    }
    setDraggedId(null);
  };

  const handleDragEnd = () =&gt; {
    setDraggedId(null);
    setDragOverStage(null);
  };

  const getStatusTag = (p: any) =&gt; {
    if (p.delivery_date &amp;&amp; new Date(p.delivery_date) &lt; new Date() &amp;&amp; p.stage !== "Completed") {
      return &lt;Badge variant="destructive" className="text-[9px] px-1.5 py-0"&gt;Overdue&lt;/Badge&gt;;
    }
    if (p.stage === "Completed") {
      return &lt;Badge className="text-[9px] px-1.5 py-0 bg-emerald-100 text-emerald-700 dark:bg-emerald-900/40 dark:text-emerald-300 border-0"&gt;Done&lt;/Badge&gt;;
    }
```

```
  return &lt;Badge variant="secondary" className="text-[9px] px-1.5 py-0"&gt;Active&lt;/Badge&gt;;
};

return (
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardHeader className="pb-2"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;CardTitle className="text-base font-semibold"&gt;Project Pipeline&lt;/CardTitle&gt;
        &lt;span className="text-xs text-muted-foreground"&gt;{projects.length} projects&lt;/span&gt;
      &lt;/div&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent className="p-0"&gt;
      &lt;ScrollArea className="w-full" ref={scrollRef}&gt;
        &lt;div className="flex gap-3 p-4 min-w-max"&gt;
          {PIPELINE_STAGES.map((stage) =&gt; {
            const items = grouped[stage.key] || [];
            const isOver = dragOverStage === stage.key;
            return (
              &lt;div
                key={stage.key}
                className={`flex-shrink-0 w-[240px] rounded-lg transition-colors ${
                  isOver ? "bg-primary/5 ring-2 ring-primary/20" : "bg-muted/30"
                }`}
                onDragOver={(e) =&gt; handleDragOver(e, stage.key)}
                onDragLeave={handleDragLeave}
                onDrop={(e) =&gt; handleDrop(e, stage.key)}
              &gt;
                {/* Column header */}
                &lt;div className="flex items-center gap-2 p-3 pb-2"&gt;
                  &lt;span className={`w-2.5 h-2.5 rounded-full ${stage.dot}`} /&gt;
                  &lt;span className="text-xs font-semibold text-foreground uppercase tracking-wide truncate"&gt;
                    {stage.key}
                  &lt;/span&gt;
                  &lt;Badge variant="outline" className="text-[10px] h-4 px-1.5 ml-auto shrink-0"&gt;
                    {items.length}
                  &lt;/Badge&gt;
                &lt;/div&gt;

                {/* Cards */}
                &lt;div className="space-y-2 p-2 pt-0 min-h-[100px]"&gt;
                  {items.map((p) =&gt; (
                    &lt;div
                      key={p.id}
                      draggable
                      onDragStart={(e) =&gt; handleDragStart(e, p.id)}
                      onDragEnd={handleDragEnd}
                      className={`group rounded-lg border bg-card p-3 cursor-grab active:cursor-grabbing shadow-sm hover:shadow-md transition-all ${
                        draggedId === p.id ? "opacity-40 scale-95" : "opacity-100"
                      }`}
                    &gt;
                      {/* Top row: thumbnail + grip */}
                      &lt;div className="flex items-start gap-2 mb-2"&gt;
                        &lt;div className="w-8 h-8 rounded bg-muted flex items-center justify-center flex-shrink-0 overflow-hidden"&gt;
                          {p.cover_image ? (
                            &lt;img src={p.cover_image} alt="" className="w-full h-full object-cover" /&gt;
                          ) : (
                            &lt;ImageIcon className="h-3.5 w-3.5 text-muted-foreground/50" /&gt;
                          )}
                        &lt;/div&gt;
                        &lt;div className="flex-1 min-w-0"&gt;
                          &lt;p
                            className="text-xs font-semibold text-foreground truncate leading-tight hover:text-primary cursor-pointer"
                            onClick={(e) =&gt; { e.stopPropagation(); navigate(`/adm/projects/${p.id}`); }}
                          &gt;
                            {p.name || "Untitled Project"}
                          &lt;/p&gt;
                          &lt;p className="text-[11px] text-muted-foreground truncate"&gt;
                            &lt;span
                              className="hover:text-primary cursor-pointer hover:underline"
                              onClick={(e) =&gt; { e.stopPropagation(); if (p.company_id) navigate(`/adm/clients/${p.company_id}`); }}
                            &gt;
                              {p.companies?.name || "No client"}
                            &lt;/span&gt; {getCountryFlag(p.companies?.country)}
                          &lt;/p&gt;
                        &lt;/div&gt;
                        &lt;GripVertical className="h-3.5 w-3.5 text-muted-foreground/30 group-hover:text-muted-foreground/60 shrink-0 mt-0.5" /&gt;
                      &lt;/div&gt;

                      {/* Meta row */}
                      &lt;div className="flex items-center gap-2 flex-wrap"&gt;
                        {p.total_value &gt; 0 &amp;amp;&amp; (
                          &lt;span className="flex items-center gap-0.5 text-[10px] text-muted-foreground"&gt;
                            &lt;DollarSign className="h-2.5 w-2.5" /&gt;
                            {Number(p.total_value).toLocaleString()}
                          &lt;/span&gt;
                        )}
                        {p.delivery_date &amp;amp;&amp; (
                          &lt;span className="flex items-center gap-0.5 text-[10px] text-muted-foreground"&gt;
                            &lt;Calendar className="h-2.5 w-2.5" /&gt;
                            {new Date(p.delivery_date).toLocaleDateString("en-US", { month: "short", day: "numeric" })}
                          &lt;/span&gt;
                        )}
                        &lt;span className="ml-auto"&gt;{getStatusTag(p)}&lt;/span&gt;
                      &lt;/div&gt;
                    &lt;/div&gt;
                  ))}

                  {items.length === 0 &amp;amp;&amp; (
                    &lt;div className="flex items-center justify-center h-[80px] border border-dashed border-border/50 rounded-lg"&gt;
                      &lt;p class="text-[11px] text-muted-foreground/50"&gt;Drop here&lt;/p&gt;
                    &lt;/div&gt;
```

```
                    )}
                  &lt;/div&gt;
                &lt;/div&gt;
              );
            })}
          &lt;/div&gt;
          &lt;ScrollBar orientation="horizontal" /&gt;
        &lt;/ScrollArea&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
};

export default ProjectPipeline;
```

src/components/admin/dashboard/QuickActions.tsx

```
import { useNavigate } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { FolderKanban, Send, Upload, UserPlus, Receipt } from "lucide-react";

const QuickActions = () =&gt; {
  const navigate = useNavigate();

  const actions = [
    { label: "New Project", icon: FolderKanban, url: "/adm/projects" },
    { label: "Send Proposal", icon: Send, url: "/adm/projects" },
    { label: "Upload Drawings", icon: Upload, url: "/adm/drawings" },
    { label: "Invite Client", icon: UserPlus, url: "/adm/companies" },
    { label: "Create Invoice", icon: Receipt, url: "/adm/payments" },
  ];

  return (
    &lt;div className="flex flex-wrap gap-2"&gt;
      {actions.map((action) =&gt; (
        &lt;Button
          key={action.label}
          variant="outline"
          onClick={() =&gt; navigate(action.url)}
          className="gap-2 h-9 px-4 text-sm font-medium"
        &gt;
          &lt;action.icon className="h-4 w-4" /&gt;
          {action.label}
        &lt;/Button&gt;
      ))}
    &lt;/div&gt;
  );
};

export default QuickActions;
```

src/components/admin/dashboard/TodaysTasks.tsx

```
import { useState } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Checkbox } from "@/components/ui/checkbox";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { CalendarDays, Plus, Flag, Filter } from "lucide-react";
import { supabase } from "@/integrations/supabase/client";
import { toast } from "sonner";

interface Task {
  id: string;
  title: string;
  project_id: string | null;
  priority: string;
  due_date: string;
  is_completed: boolean;
  project_name?: string;
}

interface TodaysTasksProps {
  tasks: Task[];
  projects: { id: string; name: string | null; companies?: { name: string } | null }[];
  onRefresh: () =&gt; void;
}

const priorityConfig: Record&lt;string, { label: string; className: string }&gt; = {
  high: { label: "High", className: "bg-destructive/10 text-destructive border-destructive/20" },
  medium: { label: "Medium", className: "bg-yellow-500/10 text-yellow-700 border-yellow-500/20" },
  low: { label: "Low", className: "bg-muted text-muted-foreground border-border" },
};

const TodaysTasks = ({ tasks, projects, onRefresh }: TodaysTasksProps) =&gt; {
  const [filter, setFilter] = useState&lt;string&gt;("all");
  const [dialogOpen, setDialogOpen] = useState(false);
  const [newTitle, setNewTitle] = useState("");
  const [newProjectId, setNewProjectId] = useState("");
  const [newPriority, setNewPriority] = useState("medium");
  const [newDueDate, setNewDueDate] = useState(new Date().toISOString().split("T")[0]);

  const filtered = tasks.filter((t) =&gt; {
    if (filter === "all") return true;
    if (filter === "pending") return !t.is_completed;
    if (filter === "completed") return t.is_completed;
    return t.priority === filter;
  });

  const handleToggle = async (task: Task) =&gt; {
    const nowCompleted = !task.is_completed;
    const { error } = await supabase
      .from("admin_tasks")
      .update({
        is_completed: nowCompleted,
        completed_at: nowCompleted ? new Date().toISOString() : null,
      })
      .eq("id", task.id);
    if (!error) onRefresh();
    else toast.error("Failed to update task");
  };

  const handleAdd = async () =&gt; {
    if (!newTitle.trim()) return;
    const { error } = await supabase.from("admin_tasks").insert({
      title: newTitle.trim(),
      project_id: newProjectId || null,
      priority: newPriority,
      due_date: newDueDate,
    });
    if (!error) {
      toast.success("Task added");
      setNewTitle("");
      setNewProjectId("");
      setNewPriority("medium");
      setNewDueDate(new Date().toISOString().split("T")[0]);
      setDialogOpen(false);
      onRefresh();
    } else {
      toast.error("Failed to add task");
    }
  };

  const pendingCount = tasks.filter((t) =&gt; !t.is_completed).length;

  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;CalendarDays className="h-4 w-4 text-muted-foreground" /&gt;
          &lt;CardTitle className="text-base font-semibold"&gt;Today's Tasks&lt;/CardTitle&gt;
          &lt;Badge variant="secondary" className="ml-auto"&gt;{pendingCount} pending&lt;/Badge&gt;
        &lt;/div&gt;
        &lt;div className="flex items-center gap-2 pt-2"&gt;
          &lt;Filter className="h-3.5 w-3.5 text-muted-foreground" /&gt;
          &lt;Select value={filter} onValueChange={setFilter}&gt;
            &lt;SelectTrigger className="h-7 text-xs w-[120px]"&gt;
              &lt;SelectValue /&gt;
```

```
            &lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              &lt;SelectItem value="all"&gt;All&lt;/SelectItem&gt;
              &lt;SelectItem value="pending"&gt;Pending&lt;/SelectItem&gt;
              &lt;SelectItem value="completed"&gt;Completed&lt;/SelectItem&gt;
              &lt;SelectItem value="high"&gt;High Priority&lt;/SelectItem&gt;
              &lt;SelectItem value="medium"&gt;Medium Priority&lt;/SelectItem&gt;
              &lt;SelectItem value="low"&gt;Low Priority&lt;/SelectItem&gt;
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
          &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
            &lt;DialogTrigger asChild&gt;
              &lt;Button variant="ghost" size="sm" className="h-7 ml-auto gap-1 text-xs"&gt;
                &lt;Plus className="h-3.5 w-3.5" /&gt; Add
              &lt;/Button&gt;
            &lt;/DialogTrigger&gt;
            &lt;DialogContent className="max-w-md"&gt;
              &lt;DialogHeader&gt;
                &lt;DialogTitle&gt;New Task&lt;/DialogTitle&gt;
              &lt;/DialogHeader&gt;
              &lt;div className="space-y-3"&gt;
                &lt;Input placeholder="Task title" value={newTitle} onChange={(e) =&gt; setNewTitle(e.target.value)} /&gt;
                &lt;Select value={newProjectId} onValueChange={setNewProjectId}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Related project (optional)" /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;
                    {projects.map((p) =&gt; (
                      &lt;SelectItem key={p.id} value={p.id}&gt;{p.name || p.companies?.name || "Untitled"}&lt;/SelectItem&gt;
                    ))}
                  &lt;/SelectContent&gt;
                &lt;/Select&gt;
                &lt;div className="flex gap-3"&gt;
                  &lt;Select value={newPriority} onValueChange={setNewPriority}&gt;
                    &lt;SelectTrigger className="flex-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                    &lt;SelectContent&gt;
                      &lt;SelectItem value="high"&gt;High&lt;/SelectItem&gt;
                      &lt;SelectItem value="medium"&gt;Medium&lt;/SelectItem&gt;
                      &lt;SelectItem value="low"&gt;Low&lt;/SelectItem&gt;
                    &lt;/SelectContent&gt;
                  &lt;/Select&gt;
                  &lt;Input type="date" className="flex-1" value={newDueDate} onChange={(e) =&gt; setNewDueDate(e.target.value)} /&gt;
                &lt;/div&gt;
                &lt;Button className="w-full" onClick={handleAdd} disabled={!newTitle.trim()}&gt;Add Task&lt;/Button&gt;
              &lt;/div&gt;
            &lt;/DialogContent&gt;
          &lt;/Dialog&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;div className="space-y-1.5 max-h-[280px] overflow-y-auto"&gt;
          {filtered.length === 0 &amp;amp;&amp; (
            &lt;p className="text-sm text-muted-foreground text-center py-6"&gt;
              {filter === "all" ? "No tasks yet — add one!" : "No tasks match this filter"}
            &lt;/p&gt;
          )}
          {filtered.map((task) =&gt; {
            const prio = priorityConfig[task.priority] || priorityConfig.medium;
            return (
              &lt;div
                key={task.id}
                className={`flex items-center gap-3 p-2 rounded-md hover:bg-muted/50 transition-colors ${task.is_completed ? "opacity-50" : ""}`}
              &gt;
                &lt;Checkbox checked={task.is_completed} onCheckedChange={() =&gt; handleToggle(task)} /&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;p className={`text-sm font-medium truncate ${task.is_completed ? "line-through text-muted-foreground" : "text-foreground"}`}&gt;
                    {task.title}
                  &lt;/p&gt;
                  {task.project_name &amp;amp;&amp; (
                    &lt;p className="text-xs text-muted-foreground truncate"&gt;{task.project_name}&lt;/p&gt;
                  )}
                &lt;/div&gt;
                &lt;div className="flex items-center gap-1.5 shrink-0"&gt;
                  &lt;Badge variant="outline" className={`text-[10px] gap-1 ${prio.className}`}&gt;
                    &lt;Flag className="h-2.5 w-2.5" /&gt;
                    {prio.label}
                  &lt;/Badge&gt;
                  &lt;span className="text-[10px] text-muted-foreground whitespace-nowrap"&gt;{task.due_date}&lt;/span&gt;
                &lt;/div&gt;
              &lt;/div&gt;
            );
          })}
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
};

export default TodaysTasks;
```

src/components/admin/timeline/timelineUtils.ts

```
import { LucideIcon } from "lucide-react";
import {
  CheckCircle2, Clock, DollarSign, FileText, Send,
  MessageSquare, Star, Package, Truck,
} from "lucide-react";

// ■■ Event definitions ■■

export interface EventDef {
  type: string;
  label: string;
  icon: LucideIcon;
}

export const REV_EVENTS = (rev: number): EventDef[] =&gt; {
  if (rev === 0) return [
    { type: "estimate_created", label: "Estimate Created", icon: FileText },
    { type: "initial_invoice_sent", label: "Initial Invoice Sent", icon: Send },
    { type: "initial_payment_received", label: "Initial Payment Received", icon: DollarSign },
    { type: "rev_started", label: `REV 0${rev} Started`, icon: Clock },
    { type: "rev_preview_sent", label: `REV 0${rev} Preview Sent`, icon: Send },
    { type: "client_feedback", label: `Client Feedback REV 0${rev}`, icon: MessageSquare },
  ];
  return [
    { type: "rev_started", label: `REV ${String(rev).padStart(2, "0")} Started`, icon: Clock },
    { type: "rev_preview_sent", label: `REV ${String(rev).padStart(2, "0")} Preview Sent`, icon: Send },
    { type: "client_feedback", label: `Client Feedback REV ${String(rev).padStart(2, "0")}`, icon: MessageSquare },
  ];
};

export const FINAL_EVENTS: EventDef[] = [
  { type: "invoice_sent_auto", label: "Invoice Sent", icon: Send },
  { type: "payment_received_auto", label: "Payment Received", icon: DollarSign },
  { type: "project_approved", label: "Project Approved", icon: Star },
  { type: "final_documents", label: "Final Documents", icon: FileText },
  { type: "final_payment", label: "Final Payment", icon: DollarSign },
  { type: "project_completed", label: "Project Completed", icon: CheckCircle2 },
  { type: "project_delivered", label: "Project Delivered", icon: Truck },
];

// ■■ Types ■■

export interface TimelineEvent {
  id: string;
  project_id: string;
  revision_number: number;
  event_type: string;
  event_date: string | null;
  notes: string | null;
  created_at: string;
}

export type RevStatus = "not_started" | "in_progress" | "completed";
export type AlertLevel = "on_track" | "waiting_client" | "delayed" | "overdue";

export const STATUS_CONFIG: Record&lt;RevStatus, { label: string; icon: string; classes: string }&gt; = {
  not_started: { label: "Not Started", icon: "■", classes: "bg-muted text-muted-foreground" },
  in_progress: { label: "In Progress", icon: "■", classes: "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yellow-400" },
  completed: { label: "Completed", icon: "■", classes: "bg-chart-2/15 text-chart-2" },
};

export const ALERT_CONFIG: Record&lt;AlertLevel, { label: string; icon: string; classes: string }&gt; = {
  on_track: { label: "On Track", icon: "■", classes: "bg-chart-2/15 text-chart-2" },
  waiting_client: { label: "Waiting Client", icon: "■", classes: "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yellow-400" },
  delayed: { label: "Delayed", icon: "■", classes: "bg-destructive/15 text-destructive" },
  overdue: { label: "Overdue", icon: "■", classes: "bg-destructive/15 text-destructive" },
};

// ■■ Status logic ■■

export function getRevisionStatus(revNum: number, events: TimelineEvent[]): RevStatus {
  const revEvents = REV_EVENTS(revNum);
  const completed = revEvents.filter(ev =&gt;
    events.some(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === ev.type &amp;&amp; e.event_date)
  ).length;
  if (completed === 0) return "not_started";
  if (completed === revEvents.length) return "completed";
  return "in_progress";
}

// ■■ Alert logic ■■

const WAITING_CLIENT_DAYS = 3;
const DELAYED_DAYS = 5;

export function getRevisionAlert(revNum: number, events: TimelineEvent[]): AlertLevel | null {
  const revEvents = REV_EVENTS(revNum);
  const status = getRevisionStatus(revNum, events);
  if (status === "completed" || status === "not_started") return null;

  const getRecord = (type: string) =&gt;
    events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === type);

  // Check: Preview sent but no feedback
  const previewSent = getRecord("rev_preview_sent");
  const feedback = getRecord("client_feedback");
  if (previewSent?.event_date &amp;&amp; !feedback?.event_date) {
    const daysSince = Math.floor((Date.now() - new Date(previewSent.event_date).getTime()) / (1000 * 60 * 60 * 24));
```

```
    if (daysSince &gt;= WAITING_CLIENT_DAYS) return "waiting_client";
  }

  // Check: Rev started but no preview sent
  const revStarted = getRecord("rev_started");
  const previewSentCheck = getRecord("rev_preview_sent");
  if (revStarted?.event_date &amp;&amp; !previewSentCheck?.event_date) {
    const daysSince = Math.floor((Date.now() - new Date(revStarted.event_date).getTime()) / (1000 * 60 * 60 * 24));
    if (daysSince &gt;= DELAYED_DAYS) return "delayed";
  }

  return null;
}

export function getOverallAlert(events: TimelineEvent[], deliveryDate?: string): AlertLevel {
  // Check overdue
  if (deliveryDate) {
    const deadline = new Date(deliveryDate);
    const isCompleted = events.some(e =&gt; e.event_type === "project_completed" &amp;&amp; e.event_date);
    const isDelivered = events.some(e =&gt; e.event_type === "project_delivered" &amp;&amp; e.event_date);
    if (deadline &lt; new Date() &amp;&amp; !isCompleted &amp;&amp; !isDelivered) return "overdue";
  }

  // Check all revisions for worst alert
  const maxRev = events.reduce((m, e) =&gt; Math.max(m, e.revision_number), 3);
  let worstAlert: AlertLevel = "on_track";
  for (let i = 0; i &lt;= maxRev; i++) {
    const alert = getRevisionAlert(i, events);
    if (alert === "delayed") return "delayed";
    if (alert === "waiting_client") worstAlert = "waiting_client";
  }
  return worstAlert;
}

// ■■ Timeline progress ■■

export function getTimelineProgress(events: TimelineEvent[], maxRevision: number): number {
  let totalEvents = 0;
  let completedEvents = 0;

  for (let i = 0; i &lt;= maxRevision; i++) {
    const revEvents = REV_EVENTS(i);
    totalEvents += revEvents.length;
    completedEvents += revEvents.filter(ev =&gt;
      events.some(e =&gt; e.revision_number === i &amp;&amp; e.event_type === ev.type &amp;&amp; e.event_date)
    ).length;
  }

  // Final events (non-auto only)
  const manualFinals = FINAL_EVENTS.filter(e =&gt; e.type !== "invoice_sent_auto" &amp;&amp; e.type !== "payment_received_auto");
  totalEvents += manualFinals.length;
  completedEvents += manualFinals.filter(ev =&gt;
    events.some(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === ev.type &amp;&amp; e.event_date)
  ).length;

  return totalEvents &gt; 0 ? Math.round((completedEvents / totalEvents) * 100) : 0;
}

// ■■ Auto stage derivation ■■

export function deriveStageFromTimeline(events: TimelineEvent[], maxRevision: number): string | null {
  const has = (rev: number, type: string) =&gt;
    events.some(e =&gt; e.revision_number === rev &amp;&amp; e.event_type === type &amp;&amp; e.event_date);
  const hasFinal = (type: string) =&gt;
    events.some(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === type &amp;&amp; e.event_date);

  // Check from end to start
  if (hasFinal("project_delivered")) return "DELIVERED";
  if (hasFinal("project_completed")) return "COMPLETED";
  if (hasFinal("project_approved")) return "APPROVED";

  // Check revisions from highest to lowest
  for (let rev = maxRevision; rev &gt;= 0; rev--) {
    if (has(rev, "rev_preview_sent")) return `PREVIEW_SENT_REV_${String(rev).padStart(2, "0")}`;
    if (has(rev, "rev_started")) return `IN_PROGRESS_REV_${String(rev).padStart(2, "0")}`;
  }

  // Check initial events
  if (has(0, "initial_payment_received")) return "READY_TO_START";
  if (has(0, "initial_invoice_sent")) return "WAITING_INITIAL_PAYMENT";

  return null;
}

// ■■ Deadline helpers ■■

export function getRevisionDelay(revNum: number, events: TimelineEvent[], expectedDaysPerRev: number = 3): { delayed: boolean; days: number } | null {
  const revStarted = events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === "rev_started" &amp;&amp; e.event_date);
  const previewSent = events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === "rev_preview_sent" &amp;&amp; e.event_date);

  if (!revStarted?.event_date) return null;
  if (previewSent?.event_date) return { delayed: false, days: 0 };

  const startDate = new Date(revStarted.event_date);
  const expected = new Date(startDate);
  expected.setDate(expected.getDate() + expectedDaysPerRev);

  const today = new Date();
  if (today &gt; expected) {
    const delayDays = Math.floor((today.getTime() - expected.getTime()) / (1000 * 60 * 60 * 24));
    return { delayed: true, days: delayDays };
```

```
  }
  return { delayed: false, days: 0 };
}
```

src/components/dashboard

src/components/dashboard/ClientProjectDocuments.tsx

```
import { useCallback, useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Download, FolderOpen, FileText, ChevronRight, PenTool } from "lucide-react";
import { format } from "date-fns";

const DOCUMENT_CATEGORIES = [
  { value: "pre_construction", label: "Pre-Construction" },
  { value: "rev_00", label: "REV 00" },
  { value: "rev_01", label: "REV 01" },
  { value: "rev_02", label: "REV 02" },
  { value: "rev_03", label: "REV 03 (Final)" },
  { value: "final_delivery", label: "Final Delivery" },
  { value: "general", label: "General" },
] as const;

interface ClientProjectDocumentsProps {
  projectId: string;
}

const ClientProjectDocuments = ({ projectId }: ClientProjectDocumentsProps) =&gt; {
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [drawings, setDrawings] = useState&lt;any[]&gt;([]);
  const [openFolders, setOpenFolders] = useState&lt;Set&lt;string&gt;&gt;(new Set(DOCUMENT_CATEGORIES.map(c =&gt; c.value)));

  const loadData = useCallback(async () =&gt; {
    const [fRes, dRes] = await Promise.all([
      supabase.from("project_files").select("*").eq("project_id", projectId).order("created_at", { ascending: false }),
      supabase.from("drawing_versions").select("*").eq("project_id", projectId).eq("is_current", true).order("created_at", { ascending: false }),
    ]);
    setFiles(fRes.data || []);
    setDrawings(dRes.data || []);
  }, [projectId]);

  useEffect(() =&gt; { loadData(); }, [loadData]);

  const toggleFolder = (cat: string) =&gt; {
    setOpenFolders(prev =&gt; {
      const next = new Set(prev);
      next.has(cat) ? next.delete(cat) : next.add(cat);
      return next;
    });
  };

  const grouped = DOCUMENT_CATEGORIES.reduce((acc, cat) =&gt; {
    acc[cat.value] = files.filter(f =&gt; (f.document_category || "general") === cat.value);
    return acc;
  }, {} as Record&lt;string, any[]&gt;);

  const totalCount = files.length + drawings.length;

  const formatSize = (bytes: number | null) =&gt; {
    if (!bytes) return "";
    if (bytes &lt; 1024) return `${bytes} B`;
    if (bytes &lt; 1048576) return `${(bytes / 1024).toFixed(1)} KB`;
    return `${(bytes / 1048576).toFixed(1)} MB`;
  };

  if (totalCount === 0) {
    return (
      &lt;Card className="border-dashed"&gt;
        &lt;CardContent className="py-10 text-center text-muted-foreground"&gt;
          &lt;FolderOpen className="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;
          No files available yet
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    );
  }

  const hasDrawings = drawings.length &gt; 0;
  const nonEmptyFolders = DOCUMENT_CATEGORIES.filter(cat =&gt; grouped[cat.value].length &gt; 0);

  return (
    &lt;div className="space-y-4"&gt;
      {/* Drawings Section */}
      {hasDrawings &amp;&amp; (
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardHeader className="pb-2"&gt;
            &lt;CardTitle className="text-sm"&gt;Drawings&lt;/CardTitle&gt;
          &lt;/CardHeader&gt;
          &lt;CardContent className="p-0 pb-2"&gt;
            {drawings.map(d =&gt; (
              &lt;div key={d.id} className="flex items-center gap-3 px-4 py-2 hover:bg-muted/30 transition-colors"&gt;
                &lt;PenTool className="h-4 w-4 text-primary shrink-0" /&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;p className="text-sm font-medium text-foreground truncate"&gt;{d.drawing_name}&lt;/p&gt;
                  &lt;p className="text-[11px] text-muted-foreground"&gt;
                    v{d.version_number} · {format(new Date(d.created_at), "MMM d, yyyy")}
                  &lt;/p&gt;
                &lt;/div&gt;
```

```
                &lt;Button size="sm" variant="ghost" asChild&gt;
                  &lt;a href={d.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-4 w-4" /&gt;&lt;/a&gt;
                &lt;/Button&gt;
              &lt;/div&gt;
            ))}
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      )}

      {/* Document Folders */}
      {nonEmptyFolders.length &gt; 0 &amp;&amp; (
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardHeader className="pb-2"&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;CardTitle className="text-sm"&gt;Documents&lt;/CardTitle&gt;
              &lt;Badge variant="secondary" className="text-[10px]"&gt;{files.length} files&lt;/Badge&gt;
            &lt;/div&gt;
          &lt;/CardHeader&gt;
          &lt;CardContent className="p-0"&gt;
            &lt;div className="divide-y divide-border"&gt;
              {nonEmptyFolders.map(cat =&gt; {
                const catFiles = grouped[cat.value];
                const isOpen = openFolders.has(cat.value);
                return (
                  &lt;Collapsible key={cat.value} open={isOpen} onOpenChange={() =&gt; toggleFolder(cat.value)}&gt;
                    &lt;CollapsibleTrigger className="flex items-center gap-3 w-full px-4 py-3 hover:bg-muted/50 transition-colors text-left"&gt;
                      &lt;ChevronRight className={`h-4 w-4 text-muted-foreground transition-transform ${isOpen ? "rotate-90" : ""}`} /&gt;
                      &lt;FolderOpen className="h-4 w-4 text-primary" /&gt;
                      &lt;span className="text-sm font-medium text-foreground flex-1"&gt;{cat.label}&lt;/span&gt;
                      &lt;Badge variant="outline" className="text-[10px]"&gt;{catFiles.length}&lt;/Badge&gt;
                    &lt;/CollapsibleTrigger&gt;
                    &lt;CollapsibleContent&gt;
                      &lt;div className="pb-2"&gt;
                        {catFiles.map(f =&gt; (
                          &lt;div key={f.id} className="flex items-center gap-3 px-12 py-2 hover:bg-muted/30 transition-colors"&gt;
                            &lt;FileText className="h-4 w-4 text-muted-foreground shrink-0" /&gt;
                            &lt;div className="flex-1 min-w-0"&gt;
                              &lt;p className="text-sm font-medium text-foreground truncate"&gt;{f.file_name}&lt;/p&gt;
                              &lt;p className="text-[11px] text-muted-foreground"&gt;
                                {formatSize(f.file_size)} · {format(new Date(f.created_at), "MMM d, yyyy")}
                              &lt;/p&gt;
                            &lt;/div&gt;
                            &lt;Button variant="ghost" size="icon" className="h-7 w-7 shrink-0" asChild&gt;
                              &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-3.5 w-3.5" /&gt;&lt;/a&gt;
                            &lt;/Button&gt;
                          &lt;/div&gt;
                        ))}
                      &lt;/div&gt;
                    &lt;/CollapsibleContent&gt;
                  &lt;/Collapsible&gt;
                );
              })}
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      )}
    &lt;/div&gt;
  );
};

export default ClientProjectDocuments;
```

src/components/dashboard/ClientTimeline.tsx

```
import { useEffect, useState, useCallback } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Progress } from "@/components/ui/progress";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import {
  CheckCircle2, Clock, DollarSign, FileText, Send,
  MessageSquare, Star, Package, Truck, ChevronDown, AlertTriangle,
} from "lucide-react";
import { format } from "date-fns";
import { cn } from "@/lib/utils";
import { useIsMobile } from "@/hooks/use-mobile";
import {
  REV_EVENTS, FINAL_EVENTS, TimelineEvent, STATUS_CONFIG, ALERT_CONFIG,
  getRevisionStatus, getRevisionAlert, getOverallAlert,
  getTimelineProgress, getRevisionDelay,
} from "@/components/admin/timeline/timelineUtils";

interface Props {
  projectId: string;
  deliveryDate?: string;
}

export default function ClientTimeline({ projectId, deliveryDate }: Props) {
  const [events, setEvents] = useState&lt;TimelineEvent[]&gt;([]);
  const [maxRevision, setMaxRevision] = useState(3);
  const [openRevisions, setOpenRevisions] = useState&lt;Record&lt;number, boolean&gt;&gt;({ 0: true });
  const [finalizationOpen, setFinalizationOpen] = useState(true);
  const isMobile = useIsMobile();

  const fetchEvents = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("project_timeline")
      .select("*")
      .eq("project_id", projectId)
      .order("revision_number")
      .order("created_at");
    const list = (data as TimelineEvent[]) || [];
    setEvents(list);
    const highestRev = list.reduce((m, e) =&gt; Math.max(m, e.revision_number), 3);
    setMaxRevision(highestRev &lt; 3 ? 3 : highestRev);
  }, [projectId]);

  useEffect(() =&gt; { fetchEvents(); }, [fetchEvents]);

  const getEventRecord = (revNum: number, eventType: string) =&gt;
    events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === eventType);

  // Find the "next step" for the client
  const getNextStep = (): string | null =&gt; {
    for (let rev = 0; rev &lt;= maxRevision; rev++) {
      const alert = getRevisionAlert(rev, events);
      if (alert === "waiting_client") {
        return `Waiting for your feedback on REV ${String(rev).padStart(2, "0")}`;
      }
    }
    // Check if there's an in-progress revision
    for (let rev = 0; rev &lt;= maxRevision; rev++) {
      const status = getRevisionStatus(rev, events);
      if (status === "in_progress") {
        const previewSent = events.find(e =&gt; e.revision_number === rev &amp;&amp; e.event_type === "rev_preview_sent" &amp;&amp; e.event_date);
        if (previewSent) return `Please review REV ${String(rev).padStart(2, "0")} preview`;
        return `REV ${String(rev).padStart(2, "0")} is being worked on`;
      }
    }
    return null;
  };

  const renderEvent = (ev: { type: string; label: string; icon: React.ElementType }, record: TimelineEvent | undefined) =&gt; {
    const Icon = ev.icon;
    const done = !!record?.event_date;
    return (
      &lt;div key={ev.type} className={cn("flex items-center gap-3 px-3 py-2 rounded-md", done ? "bg-chart-2/5" : "bg-transparent")}&gt;
        &lt;div className={cn(
          "h-6 w-6 rounded-full flex items-center justify-center shrink-0",
          done ? "bg-chart-2/20 text-chart-2" : "bg-muted text-muted-foreground"
        )}&gt;
          {done ? &lt;CheckCircle2 className="h-3.5 w-3.5" /&gt; : &lt;Icon className="h-3.5 w-3.5" /&gt;}
        &lt;/div&gt;
        &lt;p className={cn("text-sm flex-1", done ? "text-foreground font-medium" : "text-muted-foreground")}&gt;{ev.label}&lt;/p&gt;
        {record?.event_date &amp;&amp; (
          &lt;span className="text-xs text-muted-foreground shrink-0"&gt;
            {format(new Date(record.event_date), "MMM d, yyyy")}
          &lt;/span&gt;
        )}
      &lt;/div&gt;
    );
  };

  const renderRevisionBlock = (revNum: number) =&gt; {
    const revEvents = REV_EVENTS(revNum);
    const isExtra = revNum &gt; 3;
    const revLabel = `REV ${String(revNum).padStart(2, "0")}`;
    const status = getRevisionStatus(revNum, events);
    const statusCfg = STATUS_CONFIG[status];
    const alert = getRevisionAlert(revNum, events);
    const completedCount = revEvents.filter(ev =&gt; getEventRecord(revNum, ev.type)?.event_date).length;
```

```
  const isOpen = openRevisions[revNum] ?? false;

  return (
    &lt;Collapsible
      key={`rev-${revNum}`}
      open={isOpen}
      onOpenChange={(open) =&gt; setOpenRevisions(prev =&gt; ({ ...prev, [revNum]: open }))}
    &gt;
      &lt;Card className={cn(
        "border shadow-sm",
        isExtra &amp;&amp; "border-l-4 border-l-chart-4",
        alert === "waiting_client" &amp;&amp; "border-yellow-400/50 dark:border-yellow-600/50"
      )}&gt;
        &lt;CollapsibleTrigger asChild&gt;
          &lt;CardHeader className="pb-2 cursor-pointer hover:bg-muted/30 transition-colors"&gt;
            &lt;CardTitle className="text-sm flex items-center gap-2"&gt;
              &lt;Package className="h-4 w-4 text-muted-foreground" /&gt;
              {revLabel}
              {revNum === 0 &amp;&amp; &lt;Badge variant="secondary" className="text-[10px]"&gt;Initial&lt;/Badge&gt;}
              {revNum === 3 &amp;&amp; &lt;Badge variant="secondary" className="text-[10px]"&gt;Final&lt;/Badge&gt;}
              {isExtra &amp;&amp; &lt;Badge className="text-[10px] bg-chart-4/15 text-chart-4 border-chart-4/30"&gt;Extra&lt;/Badge&gt;}
              &lt;Badge className={cn("text-[10px] ml-1 border-0", statusCfg.classes)}&gt;
                {statusCfg.icon} {statusCfg.label}
              &lt;/Badge&gt;
              {alert === "waiting_client" &amp;&amp; (
                &lt;Badge className={cn("text-[10px] border-0", ALERT_CONFIG.waiting_client.classes)}&gt;
                  Action Required
                &lt;/Badge&gt;
              )}
              &lt;span className="ml-auto text-xs text-muted-foreground font-normal"&gt;{completedCount}/{revEvents.length}&lt;/span&gt;
              &lt;ChevronDown className={cn("h-4 w-4 text-muted-foreground transition-transform", isOpen &amp;&amp; "rotate-180")} /&gt;
            &lt;/CardTitle&gt;
          &lt;/CardHeader&gt;
        &lt;/CollapsibleTrigger&gt;
        &lt;CollapsibleContent&gt;
          &lt;CardContent className="space-y-1 pt-0"&gt;
            {revEvents.map(ev =&gt; renderEvent(ev, getEventRecord(revNum, ev.type)))}
          &lt;/CardContent&gt;
        &lt;/CollapsibleContent&gt;
      &lt;/Card&gt;
    &lt;/Collapsible&gt;
  );
};

const renderFinalBlock = () =&gt; (
  &lt;Collapsible open={finalizationOpen} onOpenChange={setFinalizationOpen}&gt;
    &lt;Card className="border shadow-sm"&gt;
      &lt;CollapsibleTrigger asChild&gt;
        &lt;CardHeader className="pb-2 cursor-pointer hover:bg-muted/30 transition-colors"&gt;
          &lt;CardTitle className="text-sm flex items-center gap-2"&gt;
            &lt;Star className="h-4 w-4 text-chart-2" /&gt; Completion
            &lt;ChevronDown className={cn("h-4 w-4 text-muted-foreground ml-auto transition-transform", finalizationOpen &amp;&amp; "rotate-180")} /&gt;
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
      &lt;/CollapsibleTrigger&gt;
      &lt;CollapsibleContent&gt;
        &lt;CardContent className="space-y-1 pt-0"&gt;
          {FINAL_EVENTS.map(ev =&gt; {
            const isAutoType = ev.type === "invoice_sent_auto" || ev.type === "payment_received_auto";
            if (isAutoType) {
              const autoRecords = events.filter(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === ev.type);
              if (autoRecords.length &gt; 0) {
                return autoRecords.map(ar =&gt; (
                  &lt;div key={ar.id} className="flex items-center gap-3 px-3 py-2 rounded-md bg-chart-2/5"&gt;
                    &lt;div className="h-6 w-6 rounded-full flex items-center justify-center shrink-0 bg-chart-2/20 text-chart-2"&gt;
                      &lt;CheckCircle2 className="h-3.5 w-3.5" /&gt;
                    &lt;/div&gt;
                    &lt;div className="flex-1 min-w-0"&gt;
                      &lt;p className="text-sm text-foreground font-medium"&gt;{ev.label}&lt;/p&gt;
                      {ar.notes &amp;&amp; &lt;p className="text-xs text-muted-foreground truncate"&gt;{ar.notes}&lt;/p&gt;}
                    &lt;/div&gt;
                    {ar.event_date &amp;&amp; (
                      &lt;span className="text-xs text-muted-foreground shrink-0"&gt;
                        {format(new Date(ar.event_date), "MMM d, yyyy")}
                      &lt;/span&gt;
                    )}
                  &lt;/div&gt;
                ));
              }
              return (
                &lt;div key={ev.type} className="flex items-center gap-3 px-3 py-2 rounded-md bg-transparent opacity-50"&gt;
                  &lt;div className="h-6 w-6 rounded-full flex items-center justify-center shrink-0 bg-muted text-muted-foreground"&gt;
                    &lt;ev.icon className="h-3.5 w-3.5" /&gt;
                  &lt;/div&gt;
                  &lt;p className="text-sm text-muted-foreground"&gt;{ev.label}&lt;/p&gt;
                &lt;/div&gt;
              );
            }
            return renderEvent(ev, events.find(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === ev.type));
          })}
        &lt;/CardContent&gt;
      &lt;/CollapsibleContent&gt;
    &lt;/Card&gt;
  &lt;/Collapsible&gt;
);

const overallAlert = getOverallAlert(events, deliveryDate);
const alertCfg = ALERT_CONFIG[overallAlert];
const progressPct = getTimelineProgress(events, maxRevision);
const nextStep = getNextStep();
const revisionNumbers = Array.from({ length: maxRevision + 1 }, (_, i) =&gt; i);
```

```
  return (
    &lt;div className="space-y-4"&gt;
      {/* Overall status */}
      &lt;Card className="border shadow-sm"&gt;
        &lt;CardContent className="py-4 space-y-3"&gt;
          &lt;div className="flex items-center justify-between"&gt;
            &lt;div className="flex items-center gap-3"&gt;
              &lt;Badge className={cn("text-xs border-0", alertCfg.classes)}&gt;
                {alertCfg.icon} {alertCfg.label}
              &lt;/Badge&gt;
              &lt;span className="text-sm text-muted-foreground"&gt;Project Progress&lt;/span&gt;
            &lt;/div&gt;
            &lt;span className="text-sm font-semibold"&gt;{progressPct}%&lt;/span&gt;
          &lt;/div&gt;
          &lt;Progress value={progressPct} className="h-2" /&gt;
          {nextStep &amp;&amp; (
            &lt;div className="flex items-center gap-2 text-xs text-yellow-700 dark:text-yellow-400 bg-yellow-50 dark:bg-yellow-900/20 px-3 py-2 rounded-md"&gt;
              &lt;AlertTriangle className="h-3.5 w-3.5 shrink-0" /&gt;
              {nextStep}
            &lt;/div&gt;
          )}
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Revision grid */}
      &lt;div className={cn("grid gap-4", isMobile ? "grid-cols-1" : "grid-cols-1 lg:grid-cols-2")}&gt;
        {revisionNumbers.map(revNum =&gt; renderRevisionBlock(revNum))}
        {renderFinalBlock()}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

src/components/dashboard/DashboardLayout.tsx

```
import { useEffect } from "react";
import { Outlet, useNavigate, Link } from "react-router-dom";
import daraLogo from "@/assets/dara-logo.jpg";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { SidebarProvider, SidebarTrigger, SidebarInset } from "@/components/ui/sidebar";
import DashboardSidebar from "./DashboardSidebar";
import { Skeleton } from "@/components/ui/skeleton";

const DashboardLayout = () =&gt; {
  const { user, loading } = useAuth();
  const { t } = useLanguage();
  const navigate = useNavigate();

  useEffect(() =&gt; {
    if (!loading &amp;&amp; !user) {
      navigate("/auth");
    }
  }, [user, loading, navigate]);

  if (loading) {
    return (
      &lt;div className="min-h-screen flex items-center justify-center bg-background"&gt;
        &lt;div className="space-y-4 w-full max-w-md p-8"&gt;
          &lt;Skeleton className="h-8 w-3/4 mx-auto" /&gt;
          &lt;Skeleton className="h-4 w-1/2 mx-auto" /&gt;
          &lt;Skeleton className="h-32 w-full" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    );
  }

  if (!user) {
    return null;
  }

  return (
    &lt;SidebarProvider&gt;
      &lt;div className="min-h-screen flex w-full"&gt;
        &lt;DashboardSidebar /&gt;
        &lt;SidebarInset className="flex flex-col flex-1"&gt;
          {/* Top Header */}
          &lt;header className="h-14 border-b border-border flex items-center justify-between px-4 bg-background"&gt;
            &lt;div className="flex items-center gap-4"&gt;
              &lt;SidebarTrigger /&gt;
              &lt;Link to="/" className="flex items-center gap-2"&gt;
                &lt;img src={daraLogo} alt="DARA Studio" className="h-8 w-8 rounded" /&gt;
                &lt;span className="font-serif text-lg font-semibold text-primary"&gt;&lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt; | Drafting &amp; 3D Support&
              &lt;/Link&gt;
            &lt;/div&gt;
          &lt;/header&gt;

          {/* Main Content */}
          &lt;main className="flex-1 overflow-auto bg-muted/30"&gt;
            &lt;Outlet /&gt;
          &lt;/main&gt;
        &lt;/SidebarInset&gt;
      &lt;/div&gt;
    &lt;/SidebarProvider&gt;
  );
};

export default DashboardLayout;
```

src/components/dashboard/DashboardSidebar.tsx

```
import { NavLink, useLocation } from "react-router-dom";
import { LayoutDashboard, FolderOpen, FileText, MessageSquare, User, Building2, LogOut, Plus, Receipt, Upload } from "lucide-react";
import { useAuth } from "@/hooks/useAuth";
import { Button } from "@/components/ui/button";
import { Sidebar, SidebarContent, SidebarGroup, SidebarGroupContent, SidebarGroupLabel, SidebarMenu, SidebarMenuButton, SidebarMenuItem, SidebarHeader, SidebarFooter, useSid
import { cn } from "@/lib/utils";

const DashboardSidebar = () =&gt; {
  const { state } = useSidebar();
  const collapsed = state === "collapsed";
  const location = useLocation();
  const { signOut, user } = useAuth();

  const mainNavItems = [
    { title: "Dashboard", url: "/dashboard", icon: LayoutDashboard },
    { title: "My Projects", url: "/dashboard/projects", icon: FolderOpen },
    { title: "Invoices", url: "/dashboard/invoices", icon: Receipt },
    { title: "Files", url: "/dashboard/files", icon: Upload },
    { title: "Messages", url: "/dashboard/messages", icon: MessageSquare },
  ];

  const settingsNavItems = [
    { title: "Profile", url: "/dashboard/profile", icon: User },
    { title: "Company", url: "/dashboard/company", icon: Building2 },
  ];

  const isActive = (path: string) =&gt; {
    if (path === "/dashboard") return location.pathname === "/dashboard";
    return location.pathname.startsWith(path);
  };

  const initials = user?.user_metadata?.full_name
    ? user.user_metadata.full_name.split(" ").map((n: string) =&gt; n[0]).join("").toUpperCase().slice(0, 2)
    : user?.email?.charAt(0).toUpperCase() || "C";

  return (
    &lt;Sidebar collapsible="icon"&gt;
      &lt;SidebarHeader className="border-b border-sidebar-border p-4"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;div className="w-9 h-9 rounded-full bg-primary flex items-center justify-center flex-shrink-0"&gt;
            &lt;span className="text-primary-foreground font-semibold text-sm"&gt;{initials}&lt;/span&gt;
          &lt;/div&gt;
          {!collapsed &amp;&amp; (
            &lt;div className="flex flex-col min-w-0"&gt;
              &lt;span className="font-semibold text-sm text-sidebar-foreground truncate"&gt;{user?.user_metadata?.full_name || "Client"}&lt;/span&gt;
              &lt;span className="text-xs text-sidebar-foreground/60 truncate"&gt;{user?.email}&lt;/span&gt;
            &lt;/div&gt;
          )}
        &lt;/div&gt;
      &lt;/SidebarHeader&gt;
      &lt;SidebarContent&gt;
        &lt;div className="p-3"&gt;
          &lt;Button asChild className="w-full" size={collapsed ? "icon" : "default"}&gt;
            &lt;NavLink to="/dashboard/new-quote"&gt;
              &lt;Plus className={cn("h-4 w-4", !collapsed &amp;&amp; "mr-2")} /&gt;
              {!collapsed &amp;&amp; "New Project"}
            &lt;/NavLink&gt;
          &lt;/Button&gt;
        &lt;/div&gt;
        &lt;SidebarGroup&gt;
          &lt;SidebarGroupLabel&gt;Menu&lt;/SidebarGroupLabel&gt;
          &lt;SidebarGroupContent&gt;
            &lt;SidebarMenu&gt;
              {mainNavItems.map((item) =&gt; (
                &lt;SidebarMenuItem key={item.url}&gt;
                  &lt;SidebarMenuButton asChild isActive={isActive(item.url)} tooltip={item.title}&gt;
                    &lt;NavLink to={item.url}&gt;&lt;item.icon className="h-4 w-4" /&gt;&lt;span&gt;{item.title}&lt;/span&gt;&lt;/NavLink&gt;
                  &lt;/SidebarMenuButton&gt;
                &lt;/SidebarMenuItem&gt;
              ))}
            &lt;/SidebarMenu&gt;
          &lt;/SidebarGroupContent&gt;
        &lt;/SidebarGroup&gt;
        &lt;SidebarGroup&gt;
          &lt;SidebarGroupLabel&gt;Settings&lt;/SidebarGroupLabel&gt;
          &lt;SidebarGroupContent&gt;
            &lt;SidebarMenu&gt;
              {settingsNavItems.map((item) =&gt; (
                &lt;SidebarMenuItem key={item.url}&gt;
                  &lt;SidebarMenuButton asChild isActive={isActive(item.url)} tooltip={item.title}&gt;
                    &lt;NavLink to={item.url}&gt;&lt;item.icon className="h-4 w-4" /&gt;&lt;span&gt;{item.title}&lt;/span&gt;&lt;/NavLink&gt;
                  &lt;/SidebarMenuButton&gt;
                &lt;/SidebarMenuItem&gt;
              ))}
            &lt;/SidebarMenu&gt;
          &lt;/SidebarGroupContent&gt;
        &lt;/SidebarGroup&gt;
      &lt;/SidebarContent&gt;
      &lt;SidebarFooter className="border-t border-sidebar-border p-3"&gt;
        &lt;Button variant="ghost" className="w-full justify-start text-sidebar-foreground/70 hover:text-sidebar-foreground" onClick={signOut}&gt;
          &lt;LogOut className={cn("h-4 w-4", !collapsed &amp;&amp; "mr-2")} /&gt;
          {!collapsed &amp;&amp; "Sign Out"}
        &lt;/Button&gt;
      &lt;/SidebarFooter&gt;
    &lt;/Sidebar&gt;
  );
};
```

```
export default DashboardSidebar;
```

src/components/dashboard/ProjectCard.tsx

```
import { Link } from "react-router-dom";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { MapPin, ArrowRight, FolderOpen, MessageSquare, DollarSign } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

interface ProjectCardProps {
  id: string;
  name: string;
  address: string;
  status: "pending" | "in_progress" | "under_review" | "completed" | "cancelled";
  serviceType: string;
  filesCount?: number;
  messagesCount?: number;
  quotesCount?: number;
}

const ProjectCard = ({ id, name, address, status, filesCount = 0, messagesCount = 0, quotesCount = 0 }: ProjectCardProps) =&gt; {
  const { t } = useLanguage();

  const statusConfig = {
    pending: { label: t("projectCard.briefing"), variant: "secondary" as const },
    in_progress: { label: t("projectCard.inDevelopment"), variant: "default" as const },
    under_review: { label: t("projectCard.underReview"), variant: "outline" as const },
    completed: { label: t("projectCard.completed"), variant: "default" as const },
    cancelled: { label: t("projectCard.cancelled"), variant: "destructive" as const },
  };

  const statusInfo = statusConfig[status] || statusConfig.pending;

  return (
    &lt;Card className="hover:border-primary/50 transition-all duration-300 overflow-hidden"&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;div className="p-4 border-b border-border bg-muted/30"&gt;
          &lt;div className="flex items-start justify-between gap-2"&gt;
            &lt;div className="flex-1 min-w-0"&gt;
              &lt;h3 className="font-semibold text-foreground truncate"&gt;{name}&lt;/h3&gt;
              &lt;div className="flex items-center gap-1 mt-1 text-muted-foreground"&gt;
                &lt;MapPin className="h-3 w-3 flex-shrink-0" /&gt;
                &lt;span className="text-sm truncate"&gt;{address}&lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;Badge variant={statusInfo.variant} className="flex-shrink-0"&gt;{statusInfo.label}&lt;/Badge&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div className="p-4 grid grid-cols-3 gap-4 border-b border-border"&gt;
          &lt;div className="flex items-center gap-2 text-sm text-muted-foreground"&gt;&lt;FolderOpen className="h-4 w-4" /&gt;&lt;span&gt;{filesCount} {t("projectCard.files
          &lt;div className="flex items-center gap-2 text-sm text-muted-foreground"&gt;&lt;MessageSquare className="h-4 w-4" /&gt;&lt;span&gt;{messagesCount} {t("projectCard
          &lt;div className="flex items-center gap-2 text-sm text-muted-foreground"&gt;&lt;DollarSign className="h-4 w-4" /&gt;&lt;span&gt;{quotesCount} {t("projectCard.quot
        &lt;/div&gt;
        &lt;div className="p-4"&gt;
          &lt;Button asChild className="w-full"&gt;
            &lt;Link to={`/dashboard/projects/${id}`}&gt;{t("dashboard.openProject")}&lt;ArrowRight className="ml-2 h-4 w-4" /&gt;&lt;/Link&gt;
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
};

export default ProjectCard;
```

src/components/freelancer

src/components/freelancer/FreelancerLayout.tsx

```
import { useEffect } from "react";
import { Outlet, useNavigate, Link } from "react-router-dom";
import daraLogo from "@/assets/dara-logo.jpg";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { useFreelancerRole } from "@/hooks/useFreelancerRole";
import { SidebarProvider, SidebarTrigger, SidebarInset } from "@/components/ui/sidebar";
import FreelancerSidebar from "./FreelancerSidebar";
import { Skeleton } from "@/components/ui/skeleton";

const FreelancerLayout = () =&gt; {
  const { user, loading: authLoading } = useAuth();
  const { t } = useLanguage();
  const { isFreelancer, loading: roleLoading } = useFreelancerRole();
  const navigate = useNavigate();

  const loading = authLoading || roleLoading;

  useEffect(() =&gt; {
    if (!loading &amp;&amp; !user) {
      navigate("/freelancer");
    }
    if (!loading &amp;&amp; user &amp;&amp; !isFreelancer) {
      navigate("/dashboard");
    }
  }, [user, loading, isFreelancer, navigate]);

  if (loading) {
    return (
      &lt;div className="min-h-screen flex items-center justify-center bg-background"&gt;
        &lt;div className="space-y-4 w-full max-w-md p-8"&gt;
          &lt;Skeleton className="h-8 w-3/4 mx-auto" /&gt;
          &lt;Skeleton className="h-4 w-1/2 mx-auto" /&gt;
          &lt;Skeleton className="h-32 w-full" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    );
  }

  if (!user || !isFreelancer) return null;

  return (
    &lt;SidebarProvider&gt;
      &lt;div className="min-h-screen flex w-full"&gt;
        &lt;FreelancerSidebar /&gt;
        &lt;SidebarInset className="flex flex-col flex-1"&gt;
          &lt;header className="h-14 border-b border-border flex items-center justify-between px-4 bg-background"&gt;
            &lt;div className="flex items-center gap-4"&gt;
              &lt;SidebarTrigger /&gt;
              &lt;Link to="/" className="flex items-center gap-2"&gt;
                &lt;img src={daraLogo} alt="DARA Studio" className="h-8 w-8 rounded" /&gt;
                &lt;span className="font-serif text-lg font-semibold text-primary"&gt;&lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt; | Drafting &amp; 3D Support&
              &lt;/Link&gt;
              &lt;span className="text-xs font-medium text-muted-foreground bg-muted px-2 py-0.5 rounded"&gt;Freelancer&lt;/span&gt;
            &lt;/div&gt;
          &lt;/header&gt;
          &lt;main className="flex-1 overflow-auto bg-muted/30"&gt;
            &lt;Outlet /&gt;
          &lt;/main&gt;
        &lt;/SidebarInset&gt;
      &lt;/div&gt;
    &lt;/SidebarProvider&gt;
  );
};

export default FreelancerLayout;
```

src/components/freelancer/FreelancerSidebar.tsx

```
import { NavLink, useLocation } from "react-router-dom";
import { LayoutDashboard, FolderOpen, DollarSign, FileText, CalendarDays, LogOut, User } from "lucide-react";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { Button } from "@/components/ui/button";
import { Sidebar, SidebarContent, SidebarGroup, SidebarGroupContent, SidebarGroupLabel, SidebarMenu, SidebarMenuButton, SidebarMenuItem, SidebarHeader, SidebarFooter, useSid
import { cn } from "@/lib/utils";

const FreelancerSidebar = () =&gt; {
  const { state } = useSidebar();
  const collapsed = state === "collapsed";
  const location = useLocation();
  const { signOut, user } = useAuth();
  const { t } = useLanguage();

  const navItems = [
    { title: t("freelancer.dashboard"), url: "/work", icon: LayoutDashboard },
    { title: t("freelancer.projects"), url: "/work/projects", icon: FolderOpen },
    { title: t("freelancer.payments"), url: "/work/payments", icon: DollarSign },
    { title: t("freelancer.contracts"), url: "/work/contracts", icon: FileText },
    { title: t("freelancer.calendar"), url: "/work/calendar", icon: CalendarDays },
  ];

  const settingsItems = [
    { title: t("profile.title"), url: "/work/profile", icon: User },
  ];

  const isActive = (path: string) =&gt; {
    if (path === "/work") return location.pathname === "/work";
    return location.pathname.startsWith(path);
  };

  return (
    &lt;Sidebar collapsible="icon"&gt;
      &lt;SidebarHeader className="border-b border-sidebar-border p-4"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;div className="w-8 h-8 rounded-full bg-primary flex items-center justify-center"&gt;
            &lt;span className="text-primary-foreground font-semibold text-sm"&gt;{user?.email?.charAt(0).toUpperCase() || "F"}&lt;/span&gt;
          &lt;/div&gt;
          {!collapsed &amp;&amp; (
            &lt;div className="flex flex-col"&gt;
              &lt;span className="font-semibold text-sm text-sidebar-foreground truncate max-w-[140px]"&gt;{user?.user_metadata?.full_name || "Freelancer"}&lt;/span&gt;
              &lt;span className="text-xs text-sidebar-foreground/60 truncate max-w-[140px]"&gt;{user?.email}&lt;/span&gt;
            &lt;/div&gt;
          )}
        &lt;/div&gt;
      &lt;/SidebarHeader&gt;
      &lt;SidebarContent&gt;
        &lt;SidebarGroup&gt;
          &lt;SidebarGroupLabel&gt;{t("freelancer.workspaceLabel")}&lt;/SidebarGroupLabel&gt;
          &lt;SidebarGroupContent&gt;
            &lt;SidebarMenu&gt;
              {navItems.map((item) =&gt; (
                &lt;SidebarMenuItem key={item.url}&gt;
                  &lt;SidebarMenuButton asChild isActive={isActive(item.url)} tooltip={item.title}&gt;
                    &lt;NavLink to={item.url}&gt;&lt;item.icon className="h-4 w-4" /&gt;&lt;span&gt;{item.title}&lt;/span&gt;&lt;/NavLink&gt;
                  &lt;/SidebarMenuButton&gt;
                &lt;/SidebarMenuItem&gt;
              ))}
            &lt;/SidebarMenu&gt;
          &lt;/SidebarGroupContent&gt;
        &lt;/SidebarGroup&gt;
        &lt;SidebarGroup&gt;
          &lt;SidebarGroupLabel&gt;{t("dashboard.settings")}&lt;/SidebarGroupLabel&gt;
          &lt;SidebarGroupContent&gt;
            &lt;SidebarMenu&gt;
              {settingsItems.map((item) =&gt; (
                &lt;SidebarMenuItem key={item.url}&gt;
                  &lt;SidebarMenuButton asChild isActive={isActive(item.url)} tooltip={item.title}&gt;
                    &lt;NavLink to={item.url}&gt;&lt;item.icon className="h-4 w-4" /&gt;&lt;span&gt;{item.title}&lt;/span&gt;&lt;/NavLink&gt;
                  &lt;/SidebarMenuButton&gt;
                &lt;/SidebarMenuItem&gt;
              ))}
            &lt;/SidebarMenu&gt;
          &lt;/SidebarGroupContent&gt;
        &lt;/SidebarGroup&gt;
      &lt;/SidebarContent&gt;
      &lt;SidebarFooter className="border-t border-sidebar-border p-3"&gt;
        &lt;Button variant="ghost" className="w-full justify-start text-sidebar-foreground/70 hover:text-sidebar-foreground" onClick={signOut}&gt;
          &lt;LogOut className={cn("h-4 w-4", !collapsed &amp;&amp; "mr-2")} /&gt;
          {!collapsed &amp;&amp; t("freelancer.signOut")}
        &lt;/Button&gt;
      &lt;/SidebarFooter&gt;
    &lt;/Sidebar&gt;
  );
};

export default FreelancerSidebar;
```

src/components/home

src/components/home/CTASection.tsx

```
import { Link } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { ArrowRight, Phone } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const CTASection = () =&gt; {
  const { t } = useLanguage();

  return (
    &lt;section className="py-20 bg-primary"&gt;
      &lt;div className="container mx-auto px-4"&gt;
        &lt;div className="max-w-3xl mx-auto text-center space-y-6"&gt;
          &lt;h2 className="font-serif text-3xl md:text-4xl font-bold text-primary-foreground"&gt;
            {t("cta.title")}
          &lt;/h2&gt;
          &lt;p className="text-lg text-primary-foreground/80"&gt;
            {t("cta.subtitle")}
          &lt;/p&gt;
          &lt;div className="flex flex-col sm:flex-row gap-4 justify-center pt-4"&gt;
            &lt;Button size="lg" variant="secondary" asChild&gt;
              &lt;Link to="/orcamento"&gt;
                {t("cta.proposal")}
                &lt;ArrowRight className="ml-2 h-4 w-4" /&gt;
              &lt;/Link&gt;
            &lt;/Button&gt;
            &lt;Button
              size="lg"
              variant="outline"
              className="bg-transparent border-primary-foreground text-primary-foreground hover:bg-primary-foreground hover:text-primary"
              asChild
            &gt;
              &lt;a href="https://wa.link/lsghnf" target="_blank" rel="noopener noreferrer"&gt;
                &lt;Phone className="mr-2 h-4 w-4" /&gt;
                {t("cta.call")}
              &lt;/a&gt;
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/section&gt;
  );
};

export default CTASection;
```

src/components/home/HeroSection.tsx

```
import { Link } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { ArrowRight } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const HeroSection = () =&gt; {
  const { t } = useLanguage();

  return (
    &lt;section className="relative min-h-[80vh] flex items-center justify-center bg-gradient-to-br from-background via-background to-accent"&gt;
      {/* Background Pattern */}
      &lt;div className="absolute inset-0 opacity-5"&gt;
        &lt;div
          className="absolute inset-0"
          style={{
            backgroundImage: `url("data:image/svg+xml,%3Csvg width='60' height='60' viewBox='0 0 60 60' xmlns='http://www.w3.org/2000/svg'%3E%3Cg fill='none' fill-rule='even
          }}
        /&gt;
      &lt;/div&gt;

      &lt;div className="container mx-auto px-4 relative z-10"&gt;
        &lt;div className="max-w-3xl mx-auto text-center space-y-8"&gt;
          {/* Badge */}
          &lt;div className="inline-flex items-center rounded-full bg-accent px-4 py-1.5 text-sm font-medium text-accent-foreground"&gt;
            {t("hero.badge")}
          &lt;/div&gt;

          {/* Main Heading */}
          &lt;h1 className="font-serif text-4xl md:text-5xl lg:text-6xl font-bold text-foreground leading-tight"&gt;
            {t("hero.title")}{" "}
            &lt;span className="text-primary"&gt;{t("hero.titleHighlight")}&lt;/span&gt;
          &lt;/h1&gt;

          {/* Subtitle */}
          &lt;p className="text-lg md:text-xl text-muted-foreground max-w-2xl mx-auto"&gt;
            {t("hero.subtitle")}
          &lt;/p&gt;

          {/* CTAs */}
          &lt;div className="flex flex-col sm:flex-row gap-4 justify-center pt-4"&gt;
            &lt;Button size="lg" asChild&gt;
              &lt;Link to="/orcamento"&gt;
                {t("hero.cta.quote")}
                &lt;ArrowRight className="ml-2 h-4 w-4" /&gt;
              &lt;/Link&gt;
            &lt;/Button&gt;
            &lt;Button size="lg" variant="outline" asChild&gt;
              &lt;Link to="/portfolio"&gt;{t("hero.cta.portfolio")}&lt;/Link&gt;
            &lt;/Button&gt;
          &lt;/div&gt;

          {/* Stats */}
          &lt;div className="grid grid-cols-3 gap-8 pt-12 border-t border-border mt-12"&gt;
            &lt;div className="text-center"&gt;
              &lt;p className="text-3xl md:text-4xl font-bold text-primary"&gt;150+&lt;/p&gt;
              &lt;p className="text-sm text-muted-foreground mt-1"&gt;{t("hero.stats.projects")}&lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="text-center"&gt;
              &lt;p className="text-3xl md:text-4xl font-bold text-primary"&gt;10+&lt;/p&gt;
              &lt;p className="text-sm text-muted-foreground mt-1"&gt;{t("hero.stats.experience")}&lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="text-center"&gt;
              &lt;p className="text-3xl md:text-4xl font-bold text-primary"&gt;98%&lt;/p&gt;
              &lt;p className="text-sm text-muted-foreground mt-1"&gt;{t("hero.stats.clients")}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/section&gt;
  );
};

export default HeroSection;
```

src/components/home/PortfolioPreview.tsx

```
import { Link } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { ArrowRight, Image as ImageIcon } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const PortfolioPreview = () =&gt; {
  const { t } = useLanguage();

  const portfolioItems = [
    { id: 1, titleKey: "portfolio.item1.title", categoryKey: "portfolio.item1.category", location: "Worcester, MA" },
    { id: 2, titleKey: "portfolio.item2.title", categoryKey: "portfolio.item2.category", location: "Boston, MA" },
    { id: 3, titleKey: "portfolio.item3.title", categoryKey: "portfolio.item3.category", location: "Cambridge, MA" },
  ];

  return (
    &lt;section className="py-20 bg-background"&gt;
      &lt;div className="container mx-auto px-4"&gt;
        &lt;div className="text-center mb-6"&gt;
          &lt;h2 className="font-serif text-3xl md:text-4xl font-bold text-foreground mb-4"&gt;{t("portfolio.title")}&lt;/h2&gt;
          &lt;p className="text-muted-foreground max-w-xl mx-auto"&gt;{t("portfolio.subtitle")}&lt;/p&gt;
        &lt;/div&gt;
        &lt;p className="text-sm font-medium text-primary mb-8 text-center"&gt;{t("portfolio.badge")}&lt;/p&gt;
        &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-6"&gt;
          {portfolioItems.map((item) =&gt; (
            &lt;Card key={item.id} className="group overflow-hidden border-border hover:border-primary/50 transition-all duration-300"&gt;
              &lt;CardContent className="p-0"&gt;
                &lt;div className="aspect-[4/3] bg-muted flex items-center justify-center relative overflow-hidden"&gt;
                  &lt;ImageIcon className="h-12 w-12 text-muted-foreground/50" /&gt;
                &lt;/div&gt;
                &lt;div className="p-5 space-y-3"&gt;
                  &lt;Badge variant="secondary" className="text-xs font-medium"&gt;{t(item.categoryKey)}&lt;/Badge&gt;
                  &lt;h3 className="font-semibold text-lg text-foreground group-hover:text-primary transition-colors"&gt;{t(item.titleKey)}&lt;/h3&gt;
                  &lt;p className="text-sm text-muted-foreground"&gt;{item.location}&lt;/p&gt;
                  &lt;Button variant="link" className="p-0 h-auto text-primary font-medium" asChild&gt;
                    &lt;Link to={`/portfolio/${item.id}`}&gt;{t("portfolio.viewProject")}&lt;ArrowRight className="ml-1 h-4 w-4" /&gt;&lt;/Link&gt;
                  &lt;/Button&gt;
                &lt;/div&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          ))}
        &lt;/div&gt;
        &lt;div className="text-center mt-10"&gt;
          &lt;Button variant="outline" asChild&gt;
            &lt;Link to="/portfolio"&gt;{t("portfolio.viewAll")}&lt;ArrowRight className="ml-2 h-4 w-4" /&gt;&lt;/Link&gt;
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/section&gt;
  );
};

export default PortfolioPreview;
```

src/components/home/ServicesGrid.tsx

```
import { Link } from "react-router-dom";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { PenTool, FileText, Home, Calculator, ArrowRight } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const services = [
  {
    icon: PenTool,
    titleEn: "Architectural Design &amp; Construction Documents",
    titlePt: "Design Arquitetônico e Documentos de Construção",
    descriptionEn:
      "Complete architectural packages including floor plans, elevations, sections, detailing, and construction-ready documentation aligned with U.S. codes and permitting re
    descriptionPt:
      "Pacotes arquitetônicos completos incluindo plantas baixas, elevações, cortes, detalhamento e documentação pronta para construção alinhada com códigos americanos.",
    slug: "technical_drafting",
  },
  {
    icon: Home,
    titleEn: "Wood Framing Design &amp; Layout",
    titlePt: "Design e Layout de Estrutura Wood Frame",
    descriptionEn:
      "Technical framing layouts for residential wood construction, supporting builders and engineers with precise load path visualization and structural coordination.",
    descriptionPt:
      "Layouts técnicos de estrutura para construção residencial em madeira, apoiando construtores e engenheiros com visualização precisa de cargas e coordenação estrutural.
    slug: "wood_frame_structures",
  },
  {
    icon: FileText,
    titleEn: "3D Visualization &amp; Concept Development",
    titlePt: "Visualização 3D e Desenvolvimento de Conceito",
    descriptionEn:
      "High-quality 3D modeling and visual studies to support client presentations, design refinement, and city submissions.",
    descriptionPt:
      "Modelagem 3D de alta qualidade e estudos visuais para apoiar apresentações a clientes, refinamento de design e submissões municipais.",
    slug: "executive_project",
  },
  {
    icon: Calculator,
    titleEn: "Project Estimation &amp; Documentation Review",
    titlePt: "Estimativa de Projeto e Revisão de Documentação",
    descriptionEn:
      "Preliminary quantity take-offs and documentation analysis to support financial planning and project feasibility.",
    descriptionPt:
      "Levantamentos preliminares de quantidades e análise de documentação para apoiar planejamento financeiro e viabilidade do projeto.",
    slug: "project_budgeting",
  },
];

const ServicesGrid = () =&gt; {
  const { language, t } = useLanguage();

  return (
    &lt;section className="py-20 bg-card"&gt;
      &lt;div className="container mx-auto px-4"&gt;
        {/* Section Header */}
        &lt;div className="text-center max-w-2xl mx-auto mb-16"&gt;
          &lt;h2 className="font-serif text-3xl md:text-4xl font-bold text-foreground mb-4"&gt;
            {t("services.title")}
          &lt;/h2&gt;
          &lt;p className="text-muted-foreground"&gt;
            {t("services.subtitle")}
          &lt;/p&gt;
        &lt;/div&gt;

        {/* Services Cards */}
        &lt;div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6"&gt;
          {services.map((service) =&gt; (
            &lt;Card
              key={service.slug}
              className="group hover:shadow-lg transition-all duration-300 border-border hover:border-primary/50"
            &gt;
              &lt;CardHeader&gt;
                &lt;div className="w-12 h-12 rounded-lg bg-accent flex items-center justify-center mb-4 group-hover:bg-primary transition-colors"&gt;
                  &lt;service.icon className="h-6 w-6 text-accent-foreground group-hover:text-primary-foreground transition-colors" /&gt;
                &lt;/div&gt;
                &lt;CardTitle className="text-xl"&gt;
                  {language === "en" ? service.titleEn : service.titlePt}
                &lt;/CardTitle&gt;
              &lt;/CardHeader&gt;
              &lt;CardContent&gt;
                &lt;CardDescription className="text-muted-foreground mb-4"&gt;
                  {language === "en" ? service.descriptionEn : service.descriptionPt}
                &lt;/CardDescription&gt;
                &lt;Link
                  to="/servicos"
                  className="inline-flex items-center text-sm font-medium text-primary hover:underline"
                &gt;
                  {t("services.learnMore")}
                  &lt;ArrowRight className="ml-1 h-4 w-4" /&gt;
                &lt;/Link&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          ))}
        &lt;/div&gt;

        {/* CTA */}
        &lt;div className="text-center mt-12"&gt;
```

```
          &lt;Button size="lg" asChild&gt;
            &lt;Link to="/orcamento"&gt;
              {t("services.cta")}
              &lt;ArrowRight className="ml-2 h-4 w-4" /&gt;
            &lt;/Link&gt;
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/section&gt;
  );
};

export default ServicesGrid;
```

src/components/how-we-work

src/components/how-we-work/ProcessStep.tsx

```
import { useState, forwardRef } from "react";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Info, ChevronDown, ChevronUp } from "lucide-react";
import { LucideIcon } from "lucide-react";
import { cn } from "@/lib/utils";

interface ProcessStepProps {
  number: string;
  title: string;
  description: string;
  timeline?: string;
  note?: string;
  icon: LucideIcon;
  detailedInfo: string;
}

const ProcessStep = forwardRef&lt;HTMLDivElement, ProcessStepProps&gt;(({
  number,
  title,
  description,
  timeline,
  note,
  icon: Icon,
  detailedInfo,
}, ref) =&gt; {
  const [isExpanded, setIsExpanded] = useState(false);

  return (
    &lt;Card className="border-l-4 border-l-primary"&gt;
      &lt;CardContent className="p-4 md:p-6"&gt;
        &lt;div className="flex items-start gap-4"&gt;
          &lt;div className="flex-shrink-0 w-10 h-10 rounded-full bg-primary/10 flex items-center justify-center"&gt;
            &lt;Icon className="w-5 h-5 text-primary" /&gt;
          &lt;/div&gt;
          &lt;div className="flex-1"&gt;
            &lt;div className="flex items-center gap-2 mb-1 flex-wrap"&gt;
              &lt;span className="text-sm font-bold text-primary"&gt;{number}.&lt;/span&gt;
              &lt;h3 className="font-semibold text-foreground"&gt;{title}&lt;/h3&gt;
              {timeline &amp;&amp; (
                &lt;span className="text-xs bg-muted px-2 py-1 rounded-full text-muted-foreground"&gt;
                  {timeline}
                &lt;/span&gt;
              )}
              &lt;Button
                variant="ghost"
                size="sm"
                className="h-6 w-6 p-0 rounded-full ml-auto"
                onClick={() =&gt; setIsExpanded(!isExpanded)}
                aria-label={isExpanded ? "Hide details" : "Show details"}
              &gt;
                {isExpanded ? (
                  &lt;ChevronUp className="h-4 w-4 text-primary" /&gt;
                ) : (
                  &lt;Info className="h-4 w-4 text-primary" /&gt;
                )}
              &lt;/Button&gt;
            &lt;/div&gt;
            &lt;p className="text-muted-foreground"&gt;{description}&lt;/p&gt;
            {note &amp;&amp; (
              &lt;p className="text-sm text-muted-foreground/80 italic mt-1"&gt;
                {note}
              &lt;/p&gt;
            )}

            {/* Expandable detailed info */}
            &lt;div
              className={cn(
                "overflow-hidden transition-all duration-300 ease-in-out",
                isExpanded ? "max-h-96 opacity-100 mt-4" : "max-h-0 opacity-0"
              )}
            &gt;
              &lt;div className="bg-muted/50 rounded-lg p-4 border border-border"&gt;
                &lt;p className="text-sm text-muted-foreground leading-relaxed"&gt;
                  {detailedInfo}
                &lt;/p&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
});

ProcessStep.displayName = "ProcessStep";

export default ProcessStep;
```

src/components/layout

src/components/layout/Footer.tsx

```
import { Link } from "react-router-dom";
import { Mail, Phone, MapPin } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const Footer = () =&gt; {
  const currentYear = new Date().getFullYear();
  const { t } = useLanguage();

  const services = [t("footer.service1"), t("footer.service2"), t("footer.service3"), t("footer.service4")];

  return (
    &lt;footer className="bg-secondary text-secondary-foreground"&gt;
      &lt;div className="container mx-auto px-4 py-12"&gt;
        &lt;div className="grid grid-cols-1 md:grid-cols-4 gap-8"&gt;
          &lt;div className="space-y-4"&gt;
            &lt;span className="font-serif text-2xl font-bold text-primary-foreground"&gt;DA•RA Studio&lt;/span&gt;
            &lt;p className="text-sm text-secondary-foreground/80"&gt;{t("footer.tagline")}&lt;/p&gt;
          &lt;/div&gt;
          &lt;div className="space-y-4"&gt;
            &lt;h3 className="font-semibold text-lg"&gt;{t("footer.quickLinks")}&lt;/h3&gt;
            &lt;ul className="space-y-2"&gt;
              &lt;li&gt;&lt;Link to="/servicos" className="text-sm text-secondary-foreground/80 hover:text-primary-foreground transition-colors"&gt;{t("footer.services")}&lt
              &lt;li&gt;&lt;Link to="/portfolio" className="text-sm text-secondary-foreground/80 hover:text-primary-foreground transition-colors"&gt;{t("nav.portfolio")}&lt;
              &lt;li&gt;&lt;Link to="/orcamento" className="text-sm text-secondary-foreground/80 hover:text-primary-foreground transition-colors"&gt;{t("nav.requestQuote")}&
              &lt;li&gt;&lt;Link to="/auth" className="text-sm text-secondary-foreground/80 hover:text-primary-foreground transition-colors"&gt;{t("nav.clientArea")}&lt;/Lin
            &lt;/ul&gt;
          &lt;/div&gt;
          &lt;div className="space-y-4"&gt;
            &lt;h3 className="font-semibold text-lg"&gt;{t("footer.services")}&lt;/h3&gt;
            &lt;ul className="space-y-2"&gt;
              {services.map((service) =&gt; (&lt;li key={service} className="text-sm text-secondary-foreground/80"&gt;{service}&lt;/li&gt;))}
            &lt;/ul&gt;
          &lt;/div&gt;
          &lt;div className="space-y-4"&gt;
            &lt;h3 className="font-semibold text-lg"&gt;{t("footer.contact")}&lt;/h3&gt;
            &lt;ul className="space-y-3"&gt;
              &lt;li class="flex items-center gap-2 text-sm text-secondary-foreground/80"&gt;&lt;Mail class="h-4 w-4" /&gt;darastudiooficial@gmail.com&lt;/li&gt;
              &lt;li class="flex items-center gap-2 text-sm text-secondary-foreground/80"&gt;&lt;Phone class="h-4 w-4" /&gt;+1 (508) 523-6056&lt;/li&gt;
              &lt;li class="flex items-center gap-2 text-sm text-secondary-foreground/80"&gt;&lt;MapPin class="h-4 w-4" /&gt;Worcester, MA&lt;/li&gt;
            &lt;/ul&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div className="mt-12 pt-8 border-t border-secondary-foreground/20"&gt;
          &lt;p className="text-center text-sm text-secondary-foreground/60"&gt;© {currentYear} DA•RA Studio. {t("footer.rights")}&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/footer&gt;
  );
};

export default Footer;
```

src/components/layout/Header.tsx

```
import { useState } from "react";
import { Link } from "react-router-dom";
import { Menu, X } from "lucide-react";
import daraLogo from "@/assets/dara-logo.jpg";
import { Button } from "@/components/ui/button";
import { useLanguage } from "@/hooks/useLanguage";
import LanguageSelector from "@/components/LanguageSelector";

const Header = () =&gt; {
  const [isMenuOpen, setIsMenuOpen] = useState(false);
  const { t } = useLanguage();

  const navLinks = [
    { href: "/how-we-work", label: t("nav.howWeWork") },
    { href: "/portfolio", label: t("nav.portfolio") },
    { href: "/orcamento", label: t("nav.requestQuote") },
    { href: "/contato", label: t("nav.contact") },
  ];

  return (
    &lt;header className="sticky top-0 z-50 w-full border-b border-border bg-background/95 backdrop-blur supports-[backdrop-filter]:bg-background/60"&gt;
      &lt;div className="container mx-auto flex h-16 items-center justify-between px-4"&gt;
        {/* Logo */}
        &lt;Link to="/" className="flex items-center gap-2"&gt;
          &lt;img src={daraLogo} alt="DARA Studio" className="h-9 w-9 rounded" /&gt;
          &lt;span className="font-serif text-lg font-semibold text-primary"&gt;
            &lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt; | Drafting &amp; 3D Support
          &lt;/span&gt;
        &lt;/Link&gt;

        {/* Desktop Navigation */}
        &lt;nav className="hidden md:flex items-center gap-6"&gt;
          {navLinks.map((link) =&gt; (
            &lt;Link
              key={link.href}
              to={link.href}
              className="text-sm font-medium text-muted-foreground transition-colors hover:text-primary"
            &gt;
              {link.label}
            &lt;/Link&gt;
          ))}
          &lt;LanguageSelector /&gt;
          &lt;Button asChild&gt;
            &lt;Link to="/auth"&gt;{t("nav.clientArea")}&lt;/Link&gt;
          &lt;/Button&gt;
        &lt;/nav&gt;

        {/* Mobile Menu Button */}
        &lt;div className="flex items-center gap-2 md:hidden"&gt;
          &lt;LanguageSelector /&gt;
          &lt;button
            className="p-2"
            onClick={() =&gt; setIsMenuOpen(!isMenuOpen)}
            aria-label="Toggle menu"
          &gt;
            {isMenuOpen ? (
              &lt;X className="h-6 w-6 text-foreground" /&gt;
            ) : (
              &lt;Menu className="h-6 w-6 text-foreground" /&gt;
            )}
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Mobile Navigation */}
      {isMenuOpen &amp;&amp; (
        &lt;div className="md:hidden border-t border-border bg-background"&gt;
          &lt;nav className="container mx-auto flex flex-col gap-4 px-4 py-4"&gt;
            {navLinks.map((link) =&gt; (
              &lt;Link
                key={link.href}
                to={link.href}
                className="text-sm font-medium text-muted-foreground transition-colors hover:text-primary"
                onClick={() =&gt; setIsMenuOpen(false)}
              &gt;
                {link.label}
              &lt;/Link&gt;
            ))}
            &lt;Button asChild className="w-full"&gt;
              &lt;Link to="/auth" onClick={() =&gt; setIsMenuOpen(false)}&gt;
                {t("nav.clientArea")}
              &lt;/Link&gt;
            &lt;/Button&gt;
          &lt;/nav&gt;
        &lt;/div&gt;
      )}
    &lt;/header&gt;
  );
};

export default Header;
```

src/components/layout/MainLayout.tsx

```
import { ReactNode } from "react";

interface MainLayoutProps {
  children: ReactNode;
}

const MainLayout = ({ children }: MainLayoutProps) =&gt; {
  return &lt;&gt;{children}&lt;/&gt;;
};

export default MainLayout;
```

src/components/layout/PublicLayout.tsx

```
import { ReactNode, forwardRef } from "react";
import Header from "./Header";
import Footer from "./Footer";

interface PublicLayoutProps {
  children: ReactNode;
}

const PublicLayout = forwardRef&lt;HTMLDivElement, PublicLayoutProps&gt;(({ children }, ref) =&gt; {
  return (
    &lt;div ref={ref} className="flex min-h-screen flex-col"&gt;
      &lt;Header /&gt;
      &lt;main className="flex-1"&gt;{children}&lt;/main&gt;
      &lt;Footer /&gt;
    &lt;/div&gt;
  );
});

PublicLayout.displayName = "PublicLayout";

export default PublicLayout;
```

src/components/estimate

src/components/estimate/EstimatePreview.tsx

```
import { DollarSign, TrendingUp, Target, Layers } from "lucide-react";
import { Progress } from "@/components/ui/progress";

interface Props {
  low: number;
  high: number;
  total: number;
  currency: string;
  breakdown: { label: string; value: number }[];
  confidence: number;
  canCalculate: boolean;
  complexityLabel?: string;
}

const EstimatePreview = ({ low, high, currency, confidence, canCalculate, complexityLabel }: Props) =&gt; {
  const fmt = (v: number) =&gt;
    new Intl.NumberFormat("en-US", {
      style: "currency",
      currency,
      maximumFractionDigits: 0,
    }).format(v);

  return (
    &lt;div className="space-y-4"&gt;
      {/* Estimate Card — Client view: only range, no breakdown */}
      {!canCalculate ? (
        &lt;div className="rounded-xl border border-dashed border-border bg-card/50 p-6 text-center"&gt;
          &lt;TrendingUp className="mx-auto h-8 w-8 text-muted-foreground/40 mb-2" /&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;
            Your live estimate will appear here once you provide project size and type.
          &lt;/p&gt;
        &lt;/div&gt;
      ) : (
        &lt;div className="rounded-xl border border-primary/30 bg-gradient-to-br from-primary/5 to-accent/30 p-6 space-y-4"&gt;
          &lt;div className="flex items-center gap-2 text-primary"&gt;
            &lt;DollarSign className="h-5 w-5" /&gt;
            &lt;h3 className="font-semibold text-sm uppercase tracking-wider"&gt;Estimated Design Fee&lt;/h3&gt;
          &lt;/div&gt;

          &lt;div&gt;
            &lt;div className="text-3xl font-bold text-foreground"&gt;
              {fmt(low)} – {fmt(high)}
            &lt;/div&gt;
          &lt;/div&gt;

          {/* Complexity */}
          {complexityLabel &amp;&amp; (
            &lt;div className="flex items-center gap-2 pt-2 border-t border-border/50"&gt;
              &lt;Layers className="h-4 w-4 text-muted-foreground" /&gt;
              &lt;span className="text-sm text-muted-foreground"&gt;Project Complexity&lt;/span&gt;
              &lt;span className="ml-auto text-sm font-semibold text-foreground"&gt;{complexityLabel}&lt;/span&gt;
            &lt;/div&gt;
          )}

          &lt;p className="text-xs text-muted-foreground pt-2"&gt;
            *Approximate estimate. Final pricing may vary based on project complexity.
          &lt;/p&gt;
        &lt;/div&gt;
      )}

      {/* Confidence Score */}
      &lt;div className="rounded-xl border border-border bg-card p-5 space-y-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Target className="h-4 w-4 text-primary" /&gt;
          &lt;h3 className="font-semibold text-sm"&gt;Estimate Confidence&lt;/h3&gt;
        &lt;/div&gt;

        &lt;div className="flex items-center gap-3"&gt;
          &lt;Progress value={confidence} className="h-2.5 flex-1" /&gt;
          &lt;span className="text-lg font-bold text-primary"&gt;{confidence}%&lt;/span&gt;
        &lt;/div&gt;

        {confidence &lt; 80 &amp;&amp; (
          &lt;p className="text-xs text-muted-foreground"&gt;
            Add more project details to improve accuracy.
          &lt;/p&gt;
        )}
        {confidence &gt;= 80 &amp;&amp; confidence &lt; 100 &amp;&amp; (
          &lt;p className="text-xs text-chart-2"&gt;
            Good confidence level. Upload documents to maximize accuracy.
          &lt;/p&gt;
        )}
        {confidence &gt;= 100 &amp;&amp; (
          &lt;p className="text-xs text-chart-2 font-medium"&gt;
            Maximum confidence reached!
          &lt;/p&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default EstimatePreview;
```

src/components/estimate/types.ts

```
import { z } from "zod";

// ■■■ COUNTRY / REGION ■■■
export const COUNTRIES = [
  { value: "US", label: "United States", flag: "■■", unit: "sqft", currency: "USD", postalLabel: "ZIP Code", buildingCode: "IRC / IBC" },
  { value: "BR", label: "Brazil", flag: "■■", unit: "m²", currency: "BRL", postalLabel: "CEP", buildingCode: "NBR" },
] as const;

// ■■■ PROJECT TYPES WITH COMPLEXITY MULTIPLIERS ■■■
export const PROJECT_TYPES = [
  { value: "deck_porch", label: "Deck / Porch / Outdoor Structure", factor: 0.70 },
  { value: "garage", label: "Garage", factor: 0.85 },
  { value: "kitchen_remodel", label: "Kitchen Remodel", factor: 0.95 },
  { value: "bathroom_remodel", label: "Bathroom Remodel", factor: 0.95 },
  { value: "interior_renovation", label: "Interior Renovation", factor: 1.00 },
  { value: "home_addition", label: "Home Addition", factor: 1.10 },
  { value: "single_family", label: "Single Family Home", factor: 1.15 },
  { value: "custom_home", label: "Custom Home", factor: 1.20 },
  { value: "adu", label: "ADU (Accessory Dwelling Unit)", factor: 1.15 },
  { value: "multi_family", label: "Multi-Family Residential", factor: 1.30 },
  { value: "small_commercial", label: "Small Commercial Building", factor: 1.35 },
  { value: "3d_visualization", label: "3D Visualization Only", factor: 0 },
] as const;

// ■■■ CLIENT TYPES ■■■
export const CLIENT_TYPES = [
  { value: "homeowner", label: "Homeowner", requiresCompany: false },
  { value: "contractor", label: "Builder / Contractor", requiresCompany: true },
  { value: "architect", label: "Architect", requiresCompany: true },
  { value: "developer", label: "Developer", requiresCompany: true },
  { value: "investor", label: "Investor", requiresCompany: true },
  { value: "real_estate", label: "Real Estate Agent", requiresCompany: true },
] as const;

export const PROFESSIONAL_TYPES = ["contractor", "architect", "developer", "investor", "real_estate"];

// ■■■ OPTIONAL SERVICES ■■■
export const OPTIONAL_SERVICES = [
  { id: "3d_exterior", label: "3D Exterior Rendering", price: 500 },
  { id: "3d_kitchen", label: "3D Kitchen Design", price: 350 },
  { id: "3d_bathroom", label: "3D Bathroom Design", price: 250 },
] as const;

// ■■■ DRAFTING SERVICES ■■■
export const DRAFTING_SERVICES = [
  { id: "full_construction_set", label: "Full Construction Document Set" },
  { id: "floor_plans_only", label: "Floor Plans Only" },
  { id: "as_built", label: "As-Built Drawings" },
] as const;

// ■■■ DESIGN SERVICES ■■■
export const DESIGN_SERVICES = [
  { id: "architectural_design", label: "Architectural Design" },
  { id: "space_planning", label: "Space Planning" },
  { id: "interior_layout", label: "Interior Layout" },
] as const;

// ■■■ PROJECT SIZE RANGES ■■■
export const SIZE_OPTIONS = [
  { value: "under_800", label: "Under 800 sqft", avg: 600 },
  { value: "800_1200", label: "800 – 1,200 sqft", avg: 1000 },
  { value: "1200_1800", label: "1,200 – 1,800 sqft", avg: 1500 },
  { value: "1800_2500", label: "1,800 – 2,500 sqft", avg: 2150 },
  { value: "2500_3500", label: "2,500 – 3,500 sqft", avg: 3000 },
  { value: "3500_5000", label: "3,500 – 5,000 sqft", avg: 4250 },
  { value: "5000_plus", label: "5,000+ sqft", avg: 6000 },
  { value: "custom", label: "I know the exact size", avg: 0 },
] as const;

export const SIZE_OPTIONS_BR = [
  { value: "under_80", label: "Até 80 m²", avg: 60 },
  { value: "80_150", label: "80 – 150 m²", avg: 115 },
  { value: "150_250", label: "150 – 250 m²", avg: 200 },
  { value: "250_400", label: "250 – 400 m²", avg: 325 },
  { value: "400_plus", label: "400+ m²", avg: 500 },
  { value: "custom", label: "Sei o tamanho exato", avg: 0 },
] as const;

// ■■■ FLOOR OPTIONS (multi-select checkboxes) ■■■
export const FLOOR_OPTIONS = [
  { value: "1_floor", label: "1 Floor", factor: 1.0 },
  { value: "2_floors", label: "2 Floors", factor: 1.0 },
  { value: "3_floors", label: "3 Floors", factor: 1.05 },
  { value: "basement", label: "Basement", factor: 1.10 },
  { value: "finished_basement", label: "Finished Basement", factor: 1.15 },
] as const;

// ■■■ PROJECT CONDITIONS ■■■
export const CONSTRUCTION_TYPE = [
  { value: "new_construction", label: "New Construction" },
  { value: "existing_structure", label: "Existing Structure" },
  { value: "partial_demolition", label: "Partial Demolition + New" },
] as const;

export const SITE_CONDITIONS = [
  { value: "flat", label: "Flat / Standard Lot" },
  { value: "sloped", label: "Sloped / Hillside" },
  { value: "waterfront", label: "Waterfront" },
```

```
  { value: "irregular", label: "Irregular Shape" },
  { value: "unknown", label: "Not Sure" },
] as const;

// ■■■ RUSH FEE OPTIONS ■■■
export const RUSH_OPTIONS = [
  { value: "3_weeks", label: "3 Weeks", rushPercent: 15 },
  { value: "2_weeks", label: "2 Weeks", rushPercent: 25 },
  { value: "1_week", label: "1 Week", rushPercent: 40 },
] as const;

// ■■■ TIMELINE OPTIONS (kept for backwards compat) ■■■
export const TIMELINE_OPTIONS = [
  { value: "standard", label: "Standard Delivery", rushPercent: 0 },
  { value: "3_weeks", label: "3 Weeks", rushPercent: 15 },
  { value: "2_weeks", label: "2 Weeks", rushPercent: 25 },
  { value: "1_week", label: "1 Week", rushPercent: 40 },
] as const;

// ■■■ RUSH REQUIRED DOCUMENTS ■■■
export const RUSH_REQUIRED_DOCS = [
  { id: "plot_plan", label: "Property Survey / Plot Plan" },
  { id: "site_photos", label: "Site Photos" },
  { id: "measurements", label: "Measurements" },
] as const;

// ■■■ UPLOAD DOCUMENT TYPES ■■■
export const UPLOAD_DOC_TYPES = [
  { id: "plot_plan", label: "Property Survey / Plot Plan" },
  { id: "site_photos", label: "Site Photos" },
  { id: "measurements", label: "Measurements" },
  { id: "reference_images", label: "Reference Images" },
  { id: "existing_plans", label: "Existing Plans" },
  { id: "sketches", label: "Sketches / Hand Drawings" },
] as const;

// ■■■ PROGRAM REQUIREMENTS (expanded) ■■■
export const ROOM_TYPES = [
  // Core rooms
  { id: "bedrooms", label: "Bedrooms", defaultCount: 0, category: "Core Rooms" },
  { id: "bathrooms", label: "Bathrooms", defaultCount: 0, category: "Core Rooms" },
  { id: "half_baths", label: "Half Baths", defaultCount: 0, category: "Core Rooms" },
  { id: "living_rooms", label: "Living Rooms", defaultCount: 0, category: "Core Rooms" },
  { id: "family_room", label: "Family Room", defaultCount: 0, category: "Core Rooms" },
  { id: "double_height_living", label: "Double Height Living Room", defaultCount: 0, category: "Core Rooms" },
  // Kitchen &amp; Dining
  { id: "kitchen", label: "Kitchen", defaultCount: 0, category: "Kitchen &amp; Dining" },
  { id: "kitchen_island", label: "Kitchen Island", defaultCount: 0, category: "Kitchen &amp; Dining" },
  { id: "pantry", label: "Pantry", defaultCount: 0, category: "Kitchen &amp; Dining" },
  { id: "butler_pantry", label: "Butler Pantry", defaultCount: 0, category: "Kitchen &amp; Dining" },
  { id: "dining", label: "Dining Room", defaultCount: 0, category: "Kitchen &amp; Dining" },
  // Utility
  { id: "walk_in_closet", label: "Walk-in Closet", defaultCount: 0, category: "Utility &amp; Storage" },
  { id: "mudroom", label: "Mudroom", defaultCount: 0, category: "Utility &amp; Storage" },
  { id: "laundry", label: "Laundry Room", defaultCount: 0, category: "Utility &amp; Storage" },
  { id: "garage_bays", label: "Garage Bays", defaultCount: 0, category: "Utility &amp; Storage" },
  // Outdoor
  { id: "covered_deck", label: "Covered Deck", defaultCount: 0, category: "Outdoor Spaces" },
  { id: "screened_porch", label: "Screened Porch", defaultCount: 0, category: "Outdoor Spaces" },
  { id: "outdoor_kitchen", label: "Outdoor Kitchen", defaultCount: 0, category: "Outdoor Spaces" },
  // Special
  { id: "office", label: "Home Office", defaultCount: 0, category: "Special Features" },
  { id: "fireplace", label: "Fireplace", defaultCount: 0, category: "Special Features" },
  { id: "gym", label: "Gym", defaultCount: 0, category: "Special Features" },
  { id: "wine_cellar", label: "Wine Cellar", defaultCount: 0, category: "Special Features" },
  { id: "sauna", label: "Sauna", defaultCount: 0, category: "Special Features" },
  { id: "elevator", label: "Elevator", defaultCount: 0, category: "Special Features" },
] as const;

// ■■■ CONFIDENCE SCORE WEIGHTS ■■■
export const CONFIDENCE_WEIGHTS = {
  location: 10,
  projectType: 20,
  projectSize: 25,
  programRequirements: 15,
  uploadedDocuments: 20,
  timeline: 10,
} as const;

// ■■■ BASE RATE ■■■
export const BASE_RATE = 1.60; // USD per sqft

// ■■■ RUSH FEE MINIMUM SQFT ■■■
export const RUSH_MIN_SQFT = 1200;

// ■■■ FORM SCHEMA ■■■
export const estimateFormSchema = z.object({
  country: z.string().min(1, "Required"),
  city: z.string().min(1, "City is required"),
  state: z.string().min(1, "State is required"),
  fullName: z.string().min(2, "Name is required"),
  email: z.string().email("Invalid email"),
  phone: z.string().min(7, "Phone is required"),
  clientType: z.string().min(1, "Required"),
  companyName: z.string().optional(),
  companyAddress: z.string().optional(),
  projectDescription: z.string().optional(),
  projectSize: z.string().min(1, "Required"),
  customSqft: z.number().optional(),
  floors: z.array(z.string()),
  lotSize: z.string().optional(),
```

```
  projectType: z.string().min(1, "Required"),
  constructionType: z.string().optional(),
  siteCondition: z.string().optional(),
  draftingServices: z.array(z.string()),
  designServices: z.array(z.string()),
  optionalServices: z.array(z.string()),
  rooms: z.record(z.number()).optional(),
  specialRequirements: z.string().optional(),
  timeline: z.string(),
  rushService: z.string().optional(),
  agreeTerms: z.boolean(),
  notes: z.string().optional(),
});

export type EstimateFormData = z.infer&lt;typeof estimateFormSchema&gt;;

export const TOTAL_STEPS = 10;

export const STEP_LABELS = [
  "Location",
  "About You",
  "Details",
  "Project Type",
  "Services",
  "Program",
  "Files",
  "Overview",
  "Rush Fees",
  "Review",
];

// US States for dropdown
export const US_STATES = [
  "Alabama", "Alaska", "Arizona", "Arkansas", "California", "Colorado",
  "Connecticut", "Delaware", "Florida", "Georgia", "Hawaii", "Idaho",
  "Illinois", "Indiana", "Iowa", "Kansas", "Kentucky", "Louisiana",
  "Maine", "Maryland", "Massachusetts", "Michigan", "Minnesota",
  "Mississippi", "Missouri", "Montana", "Nebraska", "Nevada",
  "New Hampshire", "New Jersey", "New Mexico", "New York",
  "North Carolina", "North Dakota", "Ohio", "Oklahoma", "Oregon",
  "Pennsylvania", "Rhode Island", "South Carolina", "South Dakota",
  "Tennessee", "Texas", "Utah", "Vermont", "Virginia", "Washington",
  "West Virginia", "Wisconsin", "Wyoming",
] as const;

// Brazilian states
export const BR_STATES = [
  "Acre", "Alagoas", "Amapá", "Amazonas", "Bahia", "Ceará",
  "Distrito Federal", "Espírito Santo", "Goiás", "Maranhão",
  "Mato Grosso", "Mato Grosso do Sul", "Minas Gerais", "Pará",
  "Paraíba", "Paraná", "Pernambuco", "Piauí", "Rio de Janeiro",
  "Rio Grande do Norte", "Rio Grande do Sul", "Rondônia", "Roraima",
  "Santa Catarina", "São Paulo", "Sergipe", "Tocantins",
] as const;
```

src/components/estimate/useConfidenceScore.ts

```
import { useMemo } from "react";
import { CONFIDENCE_WEIGHTS } from "./types";

interface ConfidenceInput {
  city: string;
  state: string;
  projectType: string;
  projectSize: string;
  customSqft?: number;
  rooms: Record&lt;string, number&gt;;
  filesCount: number;
  timeline: string;
}

export function useConfidenceScore(input: ConfidenceInput) {
  return useMemo(() =&gt; {
    let score = 0;

    // Location (10%)
    if (input.city &amp;&amp; input.state) {
      score += CONFIDENCE_WEIGHTS.location;
    } else if (input.city || input.state) {
      score += CONFIDENCE_WEIGHTS.location * 0.5;
    }

    // Project Type (20%)
    if (input.projectType) {
      score += CONFIDENCE_WEIGHTS.projectType;
    }

    // Project Size (25%)
    if (input.projectSize) {
      score += CONFIDENCE_WEIGHTS.projectSize;
      if (input.projectSize === "custom" &amp;&amp; input.customSqft) {
        // exact size = full weight already
      }
    }

    // Program Requirements (15%)
    const filledRooms = Object.values(input.rooms).filter((v) =&gt; v &gt; 0).length;
    if (filledRooms &gt;= 4) {
      score += CONFIDENCE_WEIGHTS.programRequirements;
    } else if (filledRooms &gt;= 2) {
      score += CONFIDENCE_WEIGHTS.programRequirements * 0.6;
    } else if (filledRooms &gt;= 1) {
      score += CONFIDENCE_WEIGHTS.programRequirements * 0.3;
    }

    // Uploaded Documents (20%)
    if (input.filesCount &gt;= 3) {
      score += CONFIDENCE_WEIGHTS.uploadedDocuments;
    } else if (input.filesCount &gt;= 1) {
      score += CONFIDENCE_WEIGHTS.uploadedDocuments * (input.filesCount / 3);
    }

    // Timeline (10%)
    if (input.timeline) {
      score += CONFIDENCE_WEIGHTS.timeline;
    }

    return Math.min(Math.round(score), 100);
  }, [input.city, input.state, input.projectType, input.projectSize, input.customSqft, input.rooms, input.filesCount, input.timeline]);
}
```

src/components/estimate/useEstimateCalculator.ts

```
import { useMemo } from "react";
import {
  BASE_RATE,
  PROJECT_TYPES,
  SIZE_OPTIONS,
  SIZE_OPTIONS_BR,
  FLOOR_OPTIONS,
  RUSH_OPTIONS,
  OPTIONAL_SERVICES,
} from "./types";

interface CalcInput {
  country: string;
  projectType: string;
  projectSize: string;
  customSqft?: number;
  floors: string[];
  rushService: string;
  optionalServices: string[];
  draftingServices: string[];
}

export function useEstimateCalculator(input: CalcInput) {
  return useMemo(() =&gt; {
    const { country, projectType, projectSize, customSqft, floors, rushService, optionalServices, draftingServices } = input;

    // Need at minimum projectType and projectSize to calculate
    if (!projectType || !projectSize) {
      return { low: 0, high: 0, total: 0, breakdown: [], currency: "USD", canCalculate: false };
    }

    // Special case: 3D Visualization Only
    if (projectType === "3d_visualization") {
      let vizTotal = 0;
      const vizBreakdown: { label: string; value: number }[] = [];
      for (const id of optionalServices) {
        const svc = OPTIONAL_SERVICES.find((s) =&gt; s.id === id);
        if (svc) {
          vizTotal += svc.price;
          vizBreakdown.push({ label: svc.label, value: svc.price });
        }
      }
      if (vizTotal === 0) {
        vizTotal = 500;
        vizBreakdown.push({ label: "3D Visualization (base)", value: 500 });
      }
      return { low: vizTotal, high: vizTotal, total: vizTotal, breakdown: vizBreakdown, currency: "USD", canCalculate: true };
    }

    // Get area
    const isBR = country === "BR";
    const sizeOpts = isBR ? SIZE_OPTIONS_BR : SIZE_OPTIONS;
    const sizeOpt = sizeOpts.find((s) =&gt; s.value === projectSize);
    let area = projectSize === "custom" &amp;&amp; customSqft ? customSqft : (sizeOpt?.avg || 0);

    // Convert m² to sqft for calculation if Brazil
    if (isBR &amp;&amp; area &gt; 0) {
      area = area * 10.764; // 1 m² = 10.764 sqft
    }

    if (area === 0) return { low: 0, high: 0, total: 0, breakdown: [], currency: "USD", canCalculate: false };

    // Project type factor
    const projOpt = PROJECT_TYPES.find((p) =&gt; p.value === projectType);
    const projFactor = projOpt?.factor || 1;

    // Floor factor (multi-select: use highest factor)
    const selectedFloors = floors || [];
    const floorFactor = selectedFloors.length &gt; 0
      ? Math.max(...selectedFloors.map(f =&gt; FLOOR_OPTIONS.find(fo =&gt; fo.value === f)?.factor || 1))
      : 1;

    // Base cost
    const baseCost = area * BASE_RATE * projFactor * floorFactor;

    // Optional services
    const hasFullSet = draftingServices.includes("full_construction_set");
    let servicesCost = 0;
    const serviceItems: { label: string; value: number }[] = [];
    for (const id of optionalServices) {
      const svc = OPTIONAL_SERVICES.find((s) =&gt; s.id === id);
      if (svc) {
        // 3D Exterior is free if Full Construction Document Set is selected
        const price = (svc.id === "3d_exterior" &amp;&amp; hasFullSet) ? 0 : svc.price;
        servicesCost += price;
        serviceItems.push({ label: svc.label, value: price });
      }
    }

    // Rush fee
    const rushOpt = RUSH_OPTIONS.find(r =&gt; r.value === rushService);
    const rushPercent = rushOpt?.rushPercent || 0;
    const rushFee = rushPercent &gt; 0 ? baseCost * (rushPercent / 100) : 0;

    const total = Math.round(baseCost + servicesCost + rushFee);

    // Build range (±variation)
    const low = Math.round(total * 0.92);
```

```
    const high = Math.round(total * 1.10);

    // Breakdown (for admin only — not shown to client)
    const breakdown: { label: string; value: number }[] = [
      { label: `Base Drafting Fee (${Math.round(area).toLocaleString()} sqft × $${BASE_RATE}/sqft)`, value: Math.round(area * BASE_RATE) },
    ];

    if (projFactor !== 1) {
      breakdown.push({ label: `Complexity (${projOpt?.label} ×${projFactor})`, value: Math.round(area * BASE_RATE * projFactor - area * BASE_RATE) });
    }

    if (floorFactor !== 1) {
      breakdown.push({ label: `Floor Adjustment (×${floorFactor})`, value: Math.round(baseCost - area * BASE_RATE * projFactor) });
    }

    breakdown.push(...serviceItems);

    if (rushFee &gt; 0) {
      breakdown.push({ label: `Rush Fee (+${rushPercent}%)`, value: Math.round(rushFee) });
    }

    // Complexity label for client display
    const complexityLabel = projFactor &lt;= 0.85 ? "Simple" : projFactor &lt;= 1.05 ? "Standard" : projFactor &lt;= 1.20 ? "Moderate" : "Complex";

    return { low, high, total, breakdown, currency: "USD", canCalculate: true, complexityLabel };
  }, [input.country, input.projectType, input.projectSize, input.customSqft, input.floors, input.rushService, input.optionalServices, input.draftingServices]);
}
```

src/components/estimate/steps/StepAgreement.tsx

```
import { Checkbox } from "@/components/ui/checkbox";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { ShieldCheck } from "lucide-react";

interface Props {
  agreeTerms: boolean;
  notes: string;
  onFieldChange: (field: string, value: any) =&gt; void;
  errors: Record&lt;string, string&gt;;
}

const StepAgreement = ({ agreeTerms, notes, onFieldChange, errors }: Props) =&gt; (
  &lt;div className="space-y-6"&gt;
    &lt;div className="text-center mb-6"&gt;
      &lt;ShieldCheck className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
      &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Review &amp; Submit&lt;/h2&gt;
      &lt;p className="text-muted-foreground mt-1"&gt;Almost done! Add any final notes and submit your request.&lt;/p&gt;
    &lt;/div&gt;

    &lt;div className="max-w-lg mx-auto space-y-5"&gt;
      &lt;div&gt;
        &lt;Label&gt;Additional Notes (optional)&lt;/Label&gt;
        &lt;Textarea
          value={notes}
          onChange={(e) =&gt; onFieldChange("notes", e.target.value)}
          rows={4}
          placeholder="Anything else we should know about your project?"
          className="mt-1"
        /&gt;
      &lt;/div&gt;

      &lt;div className="p-4 rounded-lg border border-border bg-muted/30 space-y-3"&gt;
        &lt;h3 className="font-semibold text-sm"&gt;What happens next?&lt;/h3&gt;
        &lt;ul className="text-xs text-muted-foreground space-y-1"&gt;
          &lt;li&gt;• Our team reviews your project details within 2–3 business days&lt;/li&gt;
          &lt;li&gt;• You'll receive a detailed proposal via email&lt;/li&gt;
          &lt;li&gt;• We may reach out for additional information if needed&lt;/li&gt;
          &lt;li&gt;• No commitment required — this is a free estimate&lt;/li&gt;
        &lt;/ul&gt;
      &lt;/div&gt;

      &lt;label className="flex items-start gap-3 cursor-pointer"&gt;
        &lt;Checkbox
          checked={agreeTerms}
          onCheckedChange={(checked) =&gt; onFieldChange("agreeTerms", !!checked)}
          className="mt-0.5"
        /&gt;
        &lt;span className="text-sm text-muted-foreground"&gt;
          I understand this is an approximate estimate and final pricing may vary based on project complexity and detailed requirements. *
        &lt;/span&gt;
      &lt;/label&gt;
      {errors.agreeTerms &amp;&amp; &lt;p className="text-xs text-destructive"&gt;{errors.agreeTerms}&lt;/p&gt;}
    &lt;/div&gt;
  &lt;/div&gt;
);

export default StepAgreement;
```

src/components/estimate/steps/StepClientInfo.tsx

```
import { CLIENT_TYPES, PROFESSIONAL_TYPES } from "../types";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { User } from "lucide-react";

interface Props {
  fullName: string;
  email: string;
  phone: string;
  clientType: string;
  companyName: string;
  companyAddress: string;
  onFieldChange: (field: string, value: string) =&gt; void;
  errors: Record&lt;string, string&gt;;
}

const StepClientInfo = ({ fullName, email, phone, clientType, companyName, companyAddress, onFieldChange, errors }: Props) =&gt; {
  const isProfessional = PROFESSIONAL_TYPES.includes(clientType);

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="text-center mb-6"&gt;
        &lt;User className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
        &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;About You&lt;/h2&gt;
        &lt;p className="text-muted-foreground mt-1"&gt;Tell us about yourself and your role.&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="max-w-lg mx-auto space-y-4"&gt;
        &lt;div&gt;
          &lt;Label&gt;Full Name *&lt;/Label&gt;
          &lt;Input
            value={fullName}
            onChange={(e) =&gt; onFieldChange("fullName", e.target.value)}
            placeholder="John Smith"
            className={errors.fullName ? "border-destructive" : ""}
          /&gt;
          {errors.fullName &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.fullName}&lt;/p&gt;}
        &lt;/div&gt;

        &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
          &lt;div&gt;
            &lt;Label&gt;Email *&lt;/Label&gt;
            &lt;Input
              type="email"
              value={email}
              onChange={(e) =&gt; onFieldChange("email", e.target.value)}
              placeholder="john@email.com"
              className={errors.email ? "border-destructive" : ""}
            /&gt;
            {errors.email &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.email}&lt;/p&gt;}
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label&gt;Phone *&lt;/Label&gt;
            &lt;Input
              value={phone}
              onChange={(e) =&gt; onFieldChange("phone", e.target.value)}
              placeholder="(555) 123-4567"
              className={errors.phone ? "border-destructive" : ""}
            /&gt;
            {errors.phone &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.phone}&lt;/p&gt;}
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;div&gt;
          &lt;Label&gt;Who are you? *&lt;/Label&gt;
          &lt;div className="grid grid-cols-2 md:grid-cols-3 gap-2 mt-2"&gt;
            {CLIENT_TYPES.map((ct) =&gt; (
              &lt;button
                key={ct.value}
                type="button"
                onClick={() =&gt; onFieldChange("clientType", ct.value)}
                className={`p-3 rounded-lg border text-sm font-medium transition-all ${
                  clientType === ct.value
                    ? "border-primary bg-primary/10 text-primary"
                    : "border-border bg-card hover:border-primary/40"
                }`}
              &gt;
                {ct.label}
              &lt;/button&gt;
            ))}
          &lt;/div&gt;
          {errors.clientType &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.clientType}&lt;/p&gt;}
        &lt;/div&gt;

        {isProfessional &amp;&amp; (
          &lt;div className="space-y-4 p-4 rounded-lg border border-primary/20 bg-primary/5"&gt;
            &lt;p className="text-sm font-semibold text-foreground"&gt;Company Information&lt;/p&gt;
            &lt;div&gt;
              &lt;Label&gt;Company Name *&lt;/Label&gt;
              &lt;Input
                value={companyName}
                onChange={(e) =&gt; onFieldChange("companyName", e.target.value)}
                placeholder="Company name"
                className={errors.companyName ? "border-destructive" : ""}
              /&gt;
              {errors.companyName &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.companyName}&lt;/p&gt;}
            &lt;/div&gt;
            &lt;div&gt;
```

```
              &lt;Label&gt;Company Address *&lt;/Label&gt;
              &lt;Input
                value={companyAddress}
                onChange={(e) =&gt; onFieldChange("companyAddress", e.target.value)}
                placeholder="123 Main St, City, State"
                className={errors.companyAddress ? "border-destructive" : ""}
              /&gt;
              {errors.companyAddress &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.companyAddress}&lt;/p&gt;}
            &lt;/div&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default StepClientInfo;
```

src/components/estimate/steps/StepConditions.tsx

```
import { CONSTRUCTION_TYPE, SITE_CONDITIONS } from "../types";
import { Label } from "@/components/ui/label";
import { Mountain } from "lucide-react";

interface Props {
  constructionType: string;
  siteCondition: string;
  onFieldChange: (field: string, value: string) =&gt; void;
}

const StepConditions = ({ constructionType, siteCondition, onFieldChange }: Props) =&gt; (
  &lt;div className="space-y-6"&gt;
    &lt;div className="text-center mb-6"&gt;
      &lt;Mountain className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
      &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Project Conditions&lt;/h2&gt;
      &lt;p className="text-muted-foreground mt-1"&gt;Tell us about the site and construction context.&lt;/p&gt;
    &lt;/div&gt;

    &lt;div className="max-w-lg mx-auto space-y-5"&gt;
      &lt;div&gt;
        &lt;Label&gt;Construction Type&lt;/Label&gt;
        &lt;div className="grid grid-cols-1 gap-2 mt-2"&gt;
          {CONSTRUCTION_TYPE.map((opt) =&gt; (
            &lt;button
              key={opt.value}
              type="button"
              onClick={() =&gt; onFieldChange("constructionType", opt.value)}
              className={`p-3 rounded-lg border text-sm font-medium transition-all text-left ${
                constructionType === opt.value
                  ? "border-primary bg-primary/10 text-primary"
                  : "border-border bg-card hover:border-primary/40"
              }`}
            &gt;
              {opt.label}
            &lt;/button&gt;
          ))}
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div&gt;
        &lt;Label&gt;Site Conditions&lt;/Label&gt;
        &lt;div className="grid grid-cols-2 gap-2 mt-2"&gt;
          {SITE_CONDITIONS.map((opt) =&gt; (
            &lt;button
              key={opt.value}
              type="button"
              onClick={() =&gt; onFieldChange("siteCondition", opt.value)}
              className={`p-3 rounded-lg border text-sm font-medium transition-all ${
                siteCondition === opt.value
                  ? "border-primary bg-primary/10 text-primary"
                  : "border-border bg-card hover:border-primary/40"
              }`}
            &gt;
              {opt.label}
            &lt;/button&gt;
          ))}
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;p className="text-xs text-muted-foreground"&gt;
        These fields are optional but help us better assess project complexity.
      &lt;/p&gt;
    &lt;/div&gt;
  &lt;/div&gt;
);

export default StepConditions;
```

src/components/estimate/steps/StepDetails.tsx

```
import { SIZE_OPTIONS, SIZE_OPTIONS_BR, FLOOR_OPTIONS } from "../types";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Checkbox } from "@/components/ui/checkbox";
import { Ruler } from "lucide-react";

interface Props {
  country: string;
  projectSize: string;
  customSqft: number | undefined;
  floors: string[];
  lotSize: string;
  onFieldChange: (field: string, value: any) =&gt; void;
  errors: Record&lt;string, string&gt;;
}

const StepDetails = ({ country, projectSize, customSqft, floors, lotSize, onFieldChange, errors }: Props) =&gt; {
  const isBR = country === "BR";
  const sizeOptions = isBR ? SIZE_OPTIONS_BR : SIZE_OPTIONS;
  const unitLabel = isBR ? "m²" : "sqft";

  const toggleFloor = (value: string) =&gt; {
    const current = floors || [];
    const updated = current.includes(value)
      ? current.filter(f =&gt; f !== value)
      : [...current, value];
    onFieldChange("floors", updated);
  };

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="text-center mb-6"&gt;
        &lt;Ruler className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
        &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Project Details&lt;/h2&gt;
        &lt;p className="text-muted-foreground mt-1"&gt;Size and structure information.&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="max-w-lg mx-auto space-y-5"&gt;
        &lt;div&gt;
          &lt;Label&gt;Estimated Project Size ({unitLabel}) *&lt;/Label&gt;
          &lt;div className="grid grid-cols-2 gap-2 mt-2"&gt;
            {sizeOptions.map((opt) =&gt; (
              &lt;button
                key={opt.value}
                type="button"
                onClick={() =&gt; onFieldChange("projectSize", opt.value)}
                className={`p-3 rounded-lg border text-sm font-medium transition-all ${
                  projectSize === opt.value
                    ? "border-primary bg-primary/10 text-primary"
                    : "border-border bg-card hover:border-primary/40"
                }`}
              &gt;
                {opt.label}
              &lt;/button&gt;
            ))}
          &lt;/div&gt;
          {errors.projectSize &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.projectSize}&lt;/p&gt;}
        &lt;/div&gt;

        {projectSize === "custom" &amp;&amp; (
          &lt;div&gt;
            &lt;Label&gt;Exact {unitLabel} *&lt;/Label&gt;
            &lt;Input
              type="number"
              value={customSqft || ""}
              onChange={(e) =&gt; onFieldChange("customSqft", e.target.value ? Number(e.target.value) : undefined)}
              placeholder={isBR ? "Ex: 200" : "e.g. 2400"}
              min={10}
              className={errors.customSqft ? "border-destructive" : ""}
            /&gt;
            {errors.customSqft &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.customSqft}&lt;/p&gt;}
          &lt;/div&gt;
        )}

        &lt;div&gt;
          &lt;Label&gt;Number of Floors *&lt;/Label&gt;
          &lt;p className="text-xs text-muted-foreground mb-2"&gt;Select all that apply (e.g. 2 Floors + Basement)&lt;/p&gt;
          &lt;div className="space-y-2"&gt;
            {FLOOR_OPTIONS.map((opt) =&gt; (
              &lt;label
                key={opt.value}
                className={`flex items-center gap-3 p-3 rounded-lg border cursor-pointer transition-all ${
                  floors.includes(opt.value)
                    ? "border-primary bg-primary/10"
                    : "border-border hover:border-primary/40"
                }`}
              &gt;
                &lt;Checkbox
                  checked={floors.includes(opt.value)}
                  onCheckedChange={() =&gt; toggleFloor(opt.value)}
                /&gt;
                &lt;span className="text-sm font-medium"&gt;{opt.label}&lt;/span&gt;
              &lt;/label&gt;
            ))}
          &lt;/div&gt;
          {errors.floors &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.floors}&lt;/p&gt;}
        &lt;/div&gt;
```

```
        &lt;div&gt;
          &lt;Label&gt;Lot Size (optional)&lt;/Label&gt;
          &lt;Input
            value={lotSize}
            onChange={(e) =&gt; onFieldChange("lotSize", e.target.value)}
            placeholder={isBR ? "Ex: 300 m²" : "e.g. 0.25 acres or 10,890 sqft"}
          /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default StepDetails;
```

src/components/estimate/steps/StepLocation.tsx

```
import { US_STATES, BR_STATES, COUNTRIES } from "../types";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { MapPin } from "lucide-react";

interface Props {
  country: string;
  city: string;
  state: string;
  onFieldChange: (field: string, value: string) =&gt; void;
  errors: Record&lt;string, string&gt;;
}

const StepLocation = ({ country, city, state, onFieldChange, errors }: Props) =&gt; {
  const states = country === "BR" ? BR_STATES : US_STATES;
  const countryData = COUNTRIES.find(c =&gt; c.value === country);

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="text-center mb-6"&gt;
        &lt;MapPin className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
        &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Project Location&lt;/h2&gt;
        &lt;p className="text-muted-foreground mt-1"&gt;Where is your project located?&lt;/p&gt;
      &lt;/div&gt;

      {/* Country selection */}
      &lt;div className="flex justify-center gap-4 max-w-md mx-auto"&gt;
        {COUNTRIES.map((c) =&gt; (
          &lt;button
            key={c.value}
            type="button"
            onClick={() =&gt; {
              onFieldChange("country", c.value);
              onFieldChange("state", "");
            }}
            className={`flex-1 flex flex-col items-center gap-2 p-5 rounded-xl border-2 transition-all ${
              country === c.value
                ? "border-primary bg-primary/5 ring-1 ring-primary/30"
                : "border-border bg-card hover:border-primary/40"
            }`}
          &gt;
            &lt;span className="text-4xl"&gt;{c.flag}&lt;/span&gt;
            &lt;span className="text-sm font-medium text-foreground"&gt;{c.label}&lt;/span&gt;
            &lt;span className="text-xs text-muted-foreground"&gt;{c.currency} · {c.unit}&lt;/span&gt;
          &lt;/button&gt;
        ))}
      &lt;/div&gt;
      {errors.country &amp;&amp; &lt;p className="text-xs text-destructive text-center"&gt;{errors.country}&lt;/p&gt;}

      {country &amp;&amp; (
        &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4 max-w-lg mx-auto pt-4"&gt;
          &lt;div&gt;
            &lt;Label&gt;{country === "BR" ? "Cidade" : "City"} *&lt;/Label&gt;
            &lt;Input
              value={city}
              onChange={(e) =&gt; onFieldChange("city", e.target.value)}
              placeholder={country === "BR" ? "Ex: São Paulo" : "e.g. Austin"}
              className={errors.city ? "border-destructive" : ""}
            /&gt;
            {errors.city &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.city}&lt;/p&gt;}
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label&gt;{country === "BR" ? "Estado" : "State"} *&lt;/Label&gt;
            &lt;Select value={state} onValueChange={(v) =&gt; onFieldChange("state", v)}&gt;
              &lt;SelectTrigger className={errors.state ? "border-destructive" : ""}&gt;
                &lt;SelectValue placeholder={country === "BR" ? "Selecione" : "Select state"} /&gt;
              &lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;
                {states.map((s) =&gt; (
                  &lt;SelectItem key={s} value={s}&gt;{s}&lt;/SelectItem&gt;
                ))}
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
            {errors.state &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{errors.state}&lt;/p&gt;}
          &lt;/div&gt;
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
};

export default StepLocation;
```

src/components/estimate/steps/StepOverview.tsx

```
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { FileText } from "lucide-react";

interface Props {
  value: string;
  onChange: (v: string) =&gt; void;
}

const StepOverview = ({ value, onChange }: Props) =&gt; (
  &lt;div className="space-y-6"&gt;
    &lt;div className="text-center mb-6"&gt;
      &lt;FileText className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
      &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Project Overview&lt;/h2&gt;
      &lt;p className="text-muted-foreground mt-1"&gt;
        Briefly describe your project. This helps us understand your vision.
      &lt;/p&gt;
    &lt;/div&gt;

    &lt;div className="max-w-lg mx-auto"&gt;
      &lt;Label&gt;Project Description (optional)&lt;/Label&gt;
      &lt;Textarea
        value={value}
        onChange={(e) =&gt; onChange(e.target.value)}
        rows={5}
        placeholder="Describe your project — goals, ideas, any special requirements..."
        className="mt-2"
      /&gt;
      &lt;p className="text-xs text-muted-foreground mt-2"&gt;
        You can skip this step if you prefer to share details later.
      &lt;/p&gt;
    &lt;/div&gt;
  &lt;/div&gt;
);

export default StepOverview;
```

src/components/estimate/steps/StepProgram.tsx

```
import { ROOM_TYPES } from "../types";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { LayoutGrid } from "lucide-react";

interface Props {
  rooms: Record&lt;string, number&gt;;
  specialRequirements: string;
  onFieldChange: (field: string, value: any) =&gt; void;
}

const StepProgram = ({ rooms, specialRequirements, onFieldChange }: Props) =&gt; {
  const setRoom = (id: string, count: number) =&gt; {
    onFieldChange("rooms", { ...rooms, [id]: Math.max(0, count) });
  };

  const categories = [...new Set(ROOM_TYPES.map(r =&gt; r.category))];

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="text-center mb-6"&gt;
        &lt;LayoutGrid className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
        &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Program Requirements&lt;/h2&gt;
        &lt;p className="text-muted-foreground mt-1"&gt;What spaces does your project need?&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="max-w-lg mx-auto space-y-6"&gt;
        {categories.map(cat =&gt; (
          &lt;div key={cat}&gt;
            &lt;p className="text-xs font-semibold text-muted-foreground uppercase tracking-wider mb-2"&gt;{cat}&lt;/p&gt;
            &lt;div className="space-y-2"&gt;
              {ROOM_TYPES.filter(r =&gt; r.category === cat).map((room) =&gt; (
                &lt;div key={room.id} className="flex items-center justify-between p-3 rounded-lg border border-border"&gt;
                  &lt;span className="text-sm font-medium"&gt;{room.label}&lt;/span&gt;
                  &lt;div className="flex items-center gap-2"&gt;
                    &lt;button
                      type="button"
                      onClick={() =&gt; setRoom(room.id, (rooms[room.id] || 0) - 1)}
                      className="w-8 h-8 rounded-full border border-border flex items-center justify-center text-sm hover:bg-muted transition-colors"
                    &gt;
                      –
                    &lt;/button&gt;
                    &lt;Input
                      type="number"
                      value={rooms[room.id] || 0}
                      onChange={(e) =&gt; setRoom(room.id, Number(e.target.value))}
                      className="w-16 text-center"
                      min={0}
                    /&gt;
                    &lt;button
                      type="button"
                      onClick={() =&gt; setRoom(room.id, (rooms[room.id] || 0) + 1)}
                      className="w-8 h-8 rounded-full border border-border flex items-center justify-center text-sm hover:bg-muted transition-colors"
                    &gt;
                      +
                    &lt;/button&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        ))}

        &lt;div className="pt-4"&gt;
          &lt;Label&gt;Special Requirements (optional)&lt;/Label&gt;
          &lt;Textarea
            value={specialRequirements}
            onChange={(e) =&gt; onFieldChange("specialRequirements", e.target.value)}
            rows={3}
            placeholder="e.g. ADA accessibility, home theater, specific materials..."
            className="mt-1"
          /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default StepProgram;
```

src/components/estimate/steps/StepProjectType.tsx

```
import { PROJECT_TYPES } from "../types";
import { Home } from "lucide-react";

interface Props {
  value: string;
  onChange: (v: string) =&gt; void;
  error?: string;
}

const StepProjectType = ({ value, onChange, error }: Props) =&gt; (
  &lt;div className="space-y-6"&gt;
    &lt;div className="text-center mb-6"&gt;
      &lt;Home className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
      &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Project Type&lt;/h2&gt;
      &lt;p className="text-muted-foreground mt-1"&gt;Select the type that best describes your project.&lt;/p&gt;
    &lt;/div&gt;

    &lt;div className="grid grid-cols-1 sm:grid-cols-2 gap-3 max-w-xl mx-auto"&gt;
      {PROJECT_TYPES.map((pt) =&gt; (
        &lt;button
          key={pt.value}
          type="button"
          onClick={() =&gt; onChange(pt.value)}
          className={`p-4 rounded-lg border text-left transition-all ${
            value === pt.value
              ? "border-primary bg-primary/10 text-primary ring-1 ring-primary/30"
              : "border-border bg-card hover:border-primary/40"
          }`}
        &gt;
          &lt;span className="font-medium text-sm"&gt;{pt.label}&lt;/span&gt;
        &lt;/button&gt;
      ))}
    &lt;/div&gt;
    {error &amp;&amp; &lt;p className="text-xs text-destructive text-center mt-2"&gt;{error}&lt;/p&gt;}
  &lt;/div&gt;
);

export default StepProjectType;
```

src/components/estimate/steps/StepServices.tsx

```
import { DRAFTING_SERVICES, DESIGN_SERVICES, OPTIONAL_SERVICES } from "../types";
import { Label } from "@/components/ui/label";
import { Checkbox } from "@/components/ui/checkbox";
import { PenTool } from "lucide-react";

interface Props {
  draftingServices: string[];
  designServices: string[];
  optionalServices: string[];
  onFieldChange: (field: string, value: string[]) =&gt; void;
  errors: Record&lt;string, string&gt;;
}

const toggle = (arr: string[], id: string) =&gt;
  arr.includes(id) ? arr.filter((v) =&gt; v !== id) : [...arr, id];

const StepServices = ({ draftingServices, designServices, optionalServices, onFieldChange }: Props) =&gt; {
  const hasFullSet = draftingServices.includes("full_construction_set");

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="text-center mb-6"&gt;
        &lt;PenTool className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
        &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Services Needed&lt;/h2&gt;
        &lt;p className="text-muted-foreground mt-1"&gt;Select all services you need for this project.&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="max-w-lg mx-auto space-y-6"&gt;
        &lt;div&gt;
          &lt;Label className="text-base font-semibold"&gt;Drafting &amp; Documentation&lt;/Label&gt;
          &lt;div className="space-y-2 mt-2"&gt;
            {DRAFTING_SERVICES.map((svc) =&gt; (
              &lt;label key={svc.id} className="flex items-center gap-3 p-3 rounded-lg border border-border hover:border-primary/40 cursor-pointer transition-all"&gt;
                &lt;Checkbox
                  checked={draftingServices.includes(svc.id)}
                  onCheckedChange={() =&gt; onFieldChange("draftingServices", toggle(draftingServices, svc.id))}
                /&gt;
                &lt;span className="text-sm"&gt;{svc.label}&lt;/span&gt;
              &lt;/label&gt;
            ))}
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;div&gt;
          &lt;Label className="text-base font-semibold"&gt;Design Services&lt;/Label&gt;
          &lt;div className="space-y-2 mt-2"&gt;
            {DESIGN_SERVICES.map((svc) =&gt; (
              &lt;label key={svc.id} className="flex items-center gap-3 p-3 rounded-lg border border-border hover:border-primary/40 cursor-pointer transition-all"&gt;
                &lt;Checkbox
                  checked={designServices.includes(svc.id)}
                  onCheckedChange={() =&gt; onFieldChange("designServices", toggle(designServices, svc.id))}
                /&gt;
                &lt;span className="text-sm"&gt;{svc.label}&lt;/span&gt;
              &lt;/label&gt;
            ))}
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;div&gt;
          &lt;Label className="text-base font-semibold"&gt;3D Visualization (optional add-ons)&lt;/Label&gt;
          &lt;div className="space-y-2 mt-2"&gt;
            {OPTIONAL_SERVICES.map((svc) =&gt; {
              const isFreeExterior = svc.id === "3d_exterior" &amp;&amp; hasFullSet;
              const displayPrice = isFreeExterior ? 0 : svc.price;
              return (
                &lt;label key={svc.id} className="flex items-center justify-between gap-3 p-3 rounded-lg border border-border hover:border-primary/40 cursor-pointer transiti
                  &lt;div className="flex items-center gap-3"&gt;
                    &lt;Checkbox
                      checked={optionalServices.includes(svc.id)}
                      onCheckedChange={() =&gt; onFieldChange("optionalServices", toggle(optionalServices, svc.id))}
                    /&gt;
                    &lt;span className="text-sm"&gt;{svc.label}&lt;/span&gt;
                  &lt;/div&gt;
                  &lt;span className={`text-xs font-semibold ${isFreeExterior ? "text-chart-2" : "text-primary"}`}&gt;
                    {isFreeExterior ? "Included" : `+$${displayPrice}`}
                  &lt;/span&gt;
                &lt;/label&gt;
              );
            })}
          &lt;/div&gt;
          {hasFullSet &amp;&amp; (
            &lt;p className="text-xs text-chart-2 mt-2"&gt;
              ✓ 3D Exterior Rendering is included with Full Construction Document Set.
            &lt;/p&gt;
          )}
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default StepServices;
```

src/components/estimate/steps/StepTimeline.tsx

```
import { RUSH_OPTIONS, RUSH_REQUIRED_DOCS, UPLOAD_DOC_TYPES } from "../types";
import { Clock, AlertTriangle, CheckCircle2, XCircle } from "lucide-react";

interface Props {
  rushService: string;
  uploadedFileNames: string[];
  onFieldChange: (field: string, value: string) =&gt; void;
  error?: string;
}

const StepTimeline = ({ rushService, uploadedFileNames, onFieldChange, error }: Props) =&gt; {
  // Check if required docs are uploaded
  const hasPlotPlan = uploadedFileNames.some(n =&gt;
    n.toLowerCase().includes("survey") || n.toLowerCase().includes("plot") || n.toLowerCase().includes("plat")
  );
  const hasSitePhotos = uploadedFileNames.some(n =&gt;
    n.toLowerCase().includes("photo") || n.toLowerCase().includes("img") || n.toLowerCase().includes("pic")
  );
  const hasMeasurements = uploadedFileNames.some(n =&gt;
    n.toLowerCase().includes("measure") || n.toLowerCase().includes("dimension")
  );

  // Simple eligibility: at least docs uploaded (we check count &gt;= 3 as proxy)
  const rushEligible = uploadedFileNames.length &gt;= 3;

  const docChecks = [
    { id: "plot_plan", label: "Property Survey / Plot Plan uploaded", met: hasPlotPlan || uploadedFileNames.length &gt;= 1 },
    { id: "site_photos", label: "Site Photos uploaded", met: hasSitePhotos || uploadedFileNames.length &gt;= 2 },
    { id: "measurements", label: "Measurements provided", met: hasMeasurements || uploadedFileNames.length &gt;= 3 },
  ];

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="text-center mb-6"&gt;
        &lt;Clock className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
        &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;Rush Fees&lt;/h2&gt;
        &lt;p className="text-muted-foreground mt-1"&gt;Need your project sooner? Rush service is optional.&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="max-w-lg mx-auto space-y-5"&gt;
        {/* Delivery Information */}
        &lt;div className="p-4 rounded-lg border border-border bg-muted/30 space-y-3"&gt;
          &lt;h3 className="font-semibold text-sm text-foreground"&gt;Standard Delivery Timeline&lt;/h3&gt;
          &lt;div className="space-y-2 text-sm text-muted-foreground"&gt;
            &lt;div&gt;
              &lt;span className="font-medium text-foreground"&gt;Design Preview&lt;/span&gt;
              &lt;p className="text-xs"&gt;8–16 Business Days&lt;/p&gt;
              &lt;p className="text-xs"&gt;Initial layout and visual direction are delivered for review.&lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="pt-2"&gt;
              &lt;span className="font-medium text-foreground"&gt;Final Drawing Set&lt;/span&gt;
              &lt;p className="text-xs"&gt;25–30 Business Days After Approval&lt;/p&gt;
              &lt;p className="text-xs"&gt;Complete architectural drawing package delivered in digital format.&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;p className="text-xs text-muted-foreground italic"&gt;
            Timeline starts from receipt of required project information including proposal approval, initial payment and site documentation.
            Timeline may vary depending on project complexity, requested revisions and technical coordination.
          &lt;/p&gt;
        &lt;/div&gt;

        {/* Documentation Check */}
        &lt;div&gt;
          &lt;p className="text-sm font-semibold text-foreground mb-3"&gt;Do you have the following information?&lt;/p&gt;
          &lt;div class="space-y-2"&gt;
            {docChecks.map((doc) =&gt; (
              &lt;div key={doc.id} className="flex items-center gap-2 p-2 rounded-md border border-border"&gt;
                {doc.met ? (
                  &lt;CheckCircle2 className="h-4 w-4 text-chart-2 shrink-0" /&gt;
                ) : (
                  &lt;XCircle className="h-4 w-4 text-muted-foreground shrink-0" /&gt;
                )}
                &lt;span className={`text-sm ${doc.met ? "text-foreground" : "text-muted-foreground"}`}&gt;{doc.label}&lt;/span&gt;
              &lt;/div&gt;
            ))}
          &lt;/div&gt;
        &lt;/div&gt;

        {/* Rush Options */}
        {rushEligible ? (
          &lt;div&gt;
            &lt;p className="text-sm font-semibold text-foreground mb-2"&gt;Rush Service (optional)&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground mb-3"&gt;Rush service prioritizes your project in our workflow and reduces the delivery timeline.&lt;/p&gt;

            &lt;button
              type="button"
              onClick={() =&gt; onFieldChange("rushService", "")}
              className={`w-full p-4 rounded-lg border text-left transition-all mb-2 ${
                !rushService
                  ? "border-primary bg-primary/10 text-primary ring-1 ring-primary/30"
                  : "border-border bg-card hover:border-primary/40"
              }`}
            &gt;
              &lt;span className="font-medium text-sm"&gt;Standard Delivery (no rush fee)&lt;/span&gt;
            &lt;/button&gt;

            {RUSH_OPTIONS.map((opt) =&gt; (
              &lt;button
```

```
                key={opt.value}
                type="button"
                onClick={() =&gt; onFieldChange("rushService", opt.value)}
                className={`w-full p-4 rounded-lg border text-left transition-all mb-2 ${
                  rushService === opt.value
                    ? "border-primary bg-primary/10 text-primary ring-1 ring-primary/30"
                    : "border-border bg-card hover:border-primary/40"
                }`}
              &gt;
                &lt;div className="flex items-center justify-between"&gt;
                  &lt;span className="font-medium text-sm"&gt;{opt.label}&lt;/span&gt;
                  &lt;span className="text-xs font-semibold text-chart-3 bg-chart-3/15 px-2 py-0.5 rounded"&gt;
                    +{opt.rushPercent}%
                  &lt;/span&gt;
                &lt;/div&gt;
              &lt;/button&gt;
            ))}
          &lt;/div&gt;
        ) : (
          &lt;div className="flex gap-2 p-4 rounded-lg bg-destructive/5 border border-destructive/20"&gt;
            &lt;AlertTriangle className="h-5 w-5 text-destructive mt-0.5 shrink-0" /&gt;
            &lt;div&gt;
              &lt;p className="text-sm font-medium text-destructive"&gt;Rush Service is not available&lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground mt-1"&gt;
                Required documentation is missing. Upload Property Survey, Site Photos and Measurements in the Files step to enable rush delivery.
              &lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        )}
        {error &amp;&amp; &lt;p className="text-xs text-destructive mt-1"&gt;{error}&lt;/p&gt;}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default StepTimeline;
```

src/components/estimate/steps/StepUpload.tsx

```
import { useCallback } from "react";
import { UPLOAD_DOC_TYPES } from "../types";
import { Upload, X, FileIcon } from "lucide-react";
import { Button } from "@/components/ui/button";

interface Props {
  files: File[];
  onFilesChange: (files: File[]) =&gt; void;
}

const StepUpload = ({ files, onFilesChange }: Props) =&gt; {
  const handleDrop = useCallback(
    (e: React.DragEvent) =&gt; {
      e.preventDefault();
      onFilesChange([...files, ...Array.from(e.dataTransfer.files)]);
    },
    [files, onFilesChange]
  );

  const handleSelect = (e: React.ChangeEvent&lt;HTMLInputElement&gt;) =&gt; {
    if (e.target.files) onFilesChange([...files, ...Array.from(e.target.files)]);
  };

  const removeFile = (idx: number) =&gt; onFilesChange(files.filter((_, i) =&gt; i !== idx));

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="text-center mb-6"&gt;
        &lt;Upload className="mx-auto h-10 w-10 text-primary mb-3" /&gt;
        &lt;h2 className="text-2xl font-serif font-bold text-foreground"&gt;File Uploads&lt;/h2&gt;
        &lt;p className="text-muted-foreground mt-1"&gt;Upload any documents that help us understand your project.&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="max-w-lg mx-auto space-y-4"&gt;
        &lt;div className="text-xs text-muted-foreground space-y-1"&gt;
          &lt;p className="font-semibold text-sm text-foreground mb-1"&gt;Helpful documents include:&lt;/p&gt;
          {UPLOAD_DOC_TYPES.map((doc) =&gt; (
            &lt;p key={doc.id}&gt;• {doc.label}&lt;/p&gt;
          ))}
        &lt;/div&gt;

        &lt;div
          onDrop={handleDrop}
          onDragOver={(e) =&gt; e.preventDefault()}
          className="border-2 border-dashed border-border rounded-xl p-8 text-center hover:border-primary/50 transition-colors cursor-pointer"
          onClick={() =&gt; document.getElementById("file-input")?.click()}
        &gt;
          &lt;Upload className="mx-auto h-8 w-8 text-muted-foreground/50 mb-2" /&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;
            Drag &amp; drop files here or &lt;span className="text-primary font-medium"&gt;browse&lt;/span&gt;
          &lt;/p&gt;
          &lt;p className="text-xs text-muted-foreground mt-1"&gt;PDF, JPG, PNG, DWG — Max 20MB per file&lt;/p&gt;
          &lt;input id="file-input" type="file" multiple className="hidden" onChange={handleSelect} accept=".pdf,.jpg,.jpeg,.png,.dwg,.dxf,.svg" /&gt;
        &lt;/div&gt;

        {files.length &gt; 0 &amp;&amp; (
          &lt;div className="space-y-2"&gt;
            {files.map((file, idx) =&gt; (
              &lt;div key={idx} className="flex items-center justify-between p-2 rounded-lg border border-border bg-muted/30"&gt;
                &lt;div className="flex items-center gap-2"&gt;
                  &lt;FileIcon className="h-4 w-4 text-muted-foreground" /&gt;
                  &lt;span className="text-sm truncate max-w-[200px]"&gt;{file.name}&lt;/span&gt;
                  &lt;span className="text-xs text-muted-foreground"&gt;({(file.size / 1024).toFixed(0)} KB)&lt;/span&gt;
                &lt;/div&gt;
                &lt;Button variant="ghost" size="sm" onClick={() =&gt; removeFile(idx)}&gt;
                  &lt;X className="h-4 w-4" /&gt;
                &lt;/Button&gt;
              &lt;/div&gt;
            ))}
          &lt;/div&gt;
        )}

        &lt;p className="text-xs text-muted-foreground"&gt;Uploading documents significantly improves estimate accuracy.&lt;/p&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default StepUpload;
```

src/components/orcamento

src/components/orcamento/StepClientInfo.tsx

```
import { UseFormRegister, FieldErrors, UseFormWatch, UseFormSetValue } from "react-hook-form";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { RadioGroup, RadioGroupItem } from "@/components/ui/radio-group";
import { useLanguage } from "@/hooks/useLanguage";
import { FormData } from "./types";

interface StepClientInfoProps {
  register: UseFormRegister&lt;FormData&gt;;
  errors: FieldErrors&lt;FormData&gt;;
  watch: UseFormWatch&lt;FormData&gt;;
  setValue: UseFormSetValue&lt;FormData&gt;;
}

const clientTypeKeys = [
  { value: "cliente_final", key: "clientType.homeowner" },
  { value: "construtor", key: "clientType.contractor" },
  { value: "arquiteto", key: "clientType.architect" },
  { value: "engenheiro", key: "clientType.engineer" },
  { value: "real_estate", key: "clientType.realEstate" },
];

const StepClientInfo = ({ register, errors, watch, setValue }: StepClientInfoProps) =&gt; {
  const { t } = useLanguage();
  const clientType = watch("clientType");
  const isContractor = clientType === "construtor";

  return (
    &lt;div className="space-y-4"&gt;
      &lt;div&gt;
        &lt;Label className="text-base font-semibold"&gt;{t("form.iAmA")} *&lt;/Label&gt;
        &lt;RadioGroup value={watch("clientType")} onValueChange={(value) =&gt; setValue("clientType", value as FormData["clientType"])} className="mt-3 grid grid-cols-1 sm:
          {clientTypeKeys.map((type) =&gt; (
            &lt;div key={type.value} className="flex items-center space-x-2 p-2 rounded-md hover:bg-muted/50 transition-colors"&gt;
              &lt;RadioGroupItem value={type.value} id={`client-${type.value}`} /&gt;
              &lt;Label htmlFor={`client-${type.value}`} className="cursor-pointer text-sm font-normal"&gt;{t(type.key)}&lt;/Label&gt;
            &lt;/div&gt;
          ))}
        &lt;/RadioGroup&gt;
        {errors.clientType &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.clientType.message}&lt;/p&gt;}
      &lt;/div&gt;

      &lt;div className="pt-4 border-t border-border"&gt;
        &lt;h4 className="font-medium text-foreground mb-4"&gt;{t("form.contactInfo")}&lt;/h4&gt;
        &lt;div className="space-y-4"&gt;
          &lt;div&gt;
            &lt;Label htmlFor="fullName"&gt;{t("form.fullName")} *&lt;/Label&gt;
            &lt;Input id="fullName" placeholder={t("form.fullNamePlaceholder")} {...register("fullName")} className="mt-1" /&gt;
            {errors.fullName &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.fullName.message}&lt;/p&gt;}
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label htmlFor="email"&gt;{t("form.email")} *&lt;/Label&gt;
            &lt;Input id="email" type="email" placeholder="your@email.com" {...register("email")} className="mt-1" /&gt;
            {errors.email &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.email.message}&lt;/p&gt;}
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label htmlFor="phone"&gt;{t("form.phone")} *&lt;/Label&gt;
            &lt;Input id="phone" placeholder="(555) 123-4567" {...register("phone")} className="mt-1" /&gt;
            {errors.phone &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.phone.message}&lt;/p&gt;}
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      {clientType &amp;&amp; clientType !== "cliente_final" &amp;&amp; (
        &lt;div className="space-y-4 pt-4 border-t border-border"&gt;
          &lt;h4 className="font-medium text-foreground"&gt;
            {t("form.companyInfo")} {isContractor &amp;&amp; &lt;span className="text-destructive"&gt;*&lt;/span&gt;}
          &lt;/h4&gt;
          &lt;div&gt;
            &lt;Label htmlFor="companyName"&gt;{t("form.companyName")} {isContractor ? "*" : t("form.optional")}&lt;/Label&gt;
            &lt;Input id="companyName" placeholder={t("form.companyNamePlaceholder")} {...register("companyName")} className="mt-1" /&gt;
            {errors.companyName &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.companyName.message}&lt;/p&gt;}
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label htmlFor="companyAddress"&gt;{t("form.companyAddress")}&lt;/Label&gt;
            &lt;Input id="companyAddress" placeholder={t("form.companyAddressPlaceholder")} {...register("companyAddress")} className="mt-1" /&gt;
          &lt;/div&gt;
          &lt;div class="grid grid-cols-2 gap-4"&gt;
            &lt;div&gt;
              &lt;Label htmlFor="companyInstagram"&gt;{t("form.instagram")}&lt;/Label&gt;
              &lt;Input id="companyInstagram" placeholder="@yourcompany" {...register("companyInstagram")} class="mt-1" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label htmlFor="companyPhone"&gt;{t("form.companyPhone")}&lt;/Label&gt;
              &lt;Input id="companyPhone" placeholder="(555) 123-4567" {...register("companyPhone")} class="mt-1" /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label htmlFor="companyEmail"&gt;{t("form.companyEmail")}&lt;/Label&gt;
            &lt;Input id="companyEmail" type="email" placeholder="contact@company.com" {...register("companyEmail")} class="mt-1" /&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      )}
```

```
    &lt;/div&gt;
  );
};

export default StepClientInfo;
```

src/components/orcamento/StepDetails.tsx

```
import { UseFormRegister, FieldErrors, UseFormWatch, UseFormSetValue } from "react-hook-form";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Checkbox } from "@/components/ui/checkbox";
import { RadioGroup, RadioGroupItem } from "@/components/ui/radio-group";
import { useLanguage } from "@/hooks/useLanguage";
import { FormData } from "./types";

interface StepDetailsProps {
  register: UseFormRegister&lt;FormData&gt;;
  errors: FieldErrors&lt;FormData&gt;;
  watch: UseFormWatch&lt;FormData&gt;;
  setValue: UseFormSetValue&lt;FormData&gt;;
}

const timelineKeys = [
  { id: "asap", key: "timeline.asap" },
  { id: "within_30_days", key: "timeline.within30" },
  { id: "1_3_months", key: "timeline.1to3months" },
  { id: "3_plus_months", key: "timeline.3plusMonths" },
  { id: "flexible", key: "timeline.flexible" },
];

const budgetKeys = [
  { id: "under_2500", key: "budget.under2500" },
  { id: "2500_5000", key: "budget.2500to5000" },
  { id: "5000_10000", key: "budget.5000to10000" },
  { id: "10000_plus", key: "budget.10000plus" },
  { id: "not_sure", key: "budget.notSure" },
];

const referralKeys = [
  { id: "google", key: "referral.google" },
  { id: "referral", key: "referral.referral" },
  { id: "social_media", key: "referral.socialMedia" },
  { id: "contractor", key: "referral.contractor" },
  { id: "returning_client", key: "referral.returningClient" },
  { id: "other", key: "referral.other" },
];

const StepDetails = ({ register, errors, watch, setValue }: StepDetailsProps) =&gt; {
  const { t } = useLanguage();

  return (
    &lt;div className="space-y-8"&gt;
      &lt;div className="space-y-3"&gt;
        &lt;Label className="text-base font-semibold"&gt;{t("form.desiredTimeline")}&lt;/Label&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{t("form.whenStart")}&lt;/p&gt;
        &lt;RadioGroup value={watch("deadline") || ""} onValueChange={(value) =&gt; setValue("deadline", value)} className="grid grid-cols-2 sm:grid-cols-3 gap-3"&gt;
          {timelineKeys.map((option) =&gt; (
            &lt;div key={option.id} className="flex items-center space-x-2"&gt;
              &lt;RadioGroupItem value={option.id} id={`timeline-${option.id}`} /&gt;
              &lt;Label htmlFor={`timeline-${option.id}`} className="text-sm cursor-pointer"&gt;{t(option.key)}&lt;/Label&gt;
            &lt;/div&gt;
          ))}
        &lt;/RadioGroup&gt;
      &lt;/div&gt;

      &lt;div className="space-y-3"&gt;
        &lt;Label className="text-base font-semibold"&gt;{t("form.estimatedInvestment")}&lt;/Label&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{t("form.investmentDesc")}&lt;/p&gt;
        &lt;RadioGroup value={watch("budget") || ""} onValueChange={(value) =&gt; setValue("budget", value)} className="grid grid-cols-2 sm:grid-cols-3 gap-3"&gt;
          {budgetKeys.map((option) =&gt; (
            &lt;div key={option.id} className="flex items-center space-x-2"&gt;
              &lt;RadioGroupItem value={option.id} id={`budget-${option.id}`} /&gt;
              &lt;Label htmlFor={`budget-${option.id}`} className="text-sm cursor-pointer"&gt;{t(option.key)}&lt;/Label&gt;
            &lt;/div&gt;
          ))}
        &lt;/RadioGroup&gt;
        &lt;p className="text-xs text-muted-foreground italic"&gt;{t("form.investmentNote")}&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="space-y-3"&gt;
        &lt;Label className="text-base font-semibold"&gt;{t("form.howHear")}&lt;/Label&gt;
        &lt;RadioGroup value={watch("howDidYouFindUs") || ""} onValueChange={(value) =&gt; setValue("howDidYouFindUs", value)} className="grid grid-cols-2 sm:grid-cols-3 gap
          {referralKeys.map((option) =&gt; (
            &lt;div key={option.id} className="flex items-center space-x-2"&gt;
              &lt;RadioGroupItem value={option.id} id={`referral-${option.id}`} /&gt;
              &lt;Label htmlFor={`referral-${option.id}`} className="text-sm cursor-pointer"&gt;{t(option.key)}&lt;/Label&gt;
            &lt;/div&gt;
          ))}
        &lt;/RadioGroup&gt;
      &lt;/div&gt;

      &lt;div className="space-y-3"&gt;
        &lt;Label htmlFor="description" className="text-base font-semibold"&gt;{t("form.additionalNotes")}&lt;/Label&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{t("form.additionalNotesDesc")}&lt;/p&gt;
        &lt;Textarea id="description" placeholder={t("form.additionalNotesPlaceholder")} {...register("description")} className="min-h-[120px]" /&gt;
      &lt;/div&gt;

      &lt;div className="border-t pt-6"&gt;
        &lt;div className="flex items-start space-x-3"&gt;
          &lt;Checkbox id="acceptTerms" checked={watch("acceptTerms")} onCheckedChange={(checked) =&gt; setValue("acceptTerms", checked as boolean)} className="mt-0.5" /&gt;
          &lt;Label htmlFor="acceptTerms" className="text-sm cursor-pointer leading-relaxed"&gt;
            {t("form.agreement")}{" "}
            &lt;span className="text-primary underline"&gt;{t("form.termsOfUse")}&lt;/span&gt; {t("form.and")}{" "}
            &lt;span className="text-primary underline"&gt;{t("form.privacyPolicy")}&lt;/span&gt;. *
          &lt;/Label&gt;
```

```
        &lt;/div&gt;
        {errors.acceptTerms &amp;&amp; &lt;p className="text-sm text-destructive mt-2"&gt;{errors.acceptTerms.message}&lt;/p&gt;}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default StepDetails;
```

src/components/orcamento/StepLocation.tsx

```
import { UseFormRegister, FieldErrors } from "react-hook-form";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { FormData } from "./types";
import { useLanguage } from "@/hooks/useLanguage";

interface StepLocationProps {
  register: UseFormRegister&lt;FormData&gt;;
  errors: FieldErrors&lt;FormData&gt;;
}

const StepLocation = ({ register, errors }: StepLocationProps) =&gt; {
  const { t } = useLanguage();
  return (
    &lt;div className="space-y-4"&gt;
      &lt;div&gt;
        &lt;Label htmlFor="projectAddress"&gt;{t("form.projectAddress")} *&lt;/Label&gt;
        &lt;Input id="projectAddress" placeholder={t("form.streetAddress")} {...register("projectAddress")} className="mt-1" /&gt;
        {errors.projectAddress &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.projectAddress.message}&lt;/p&gt;}
      &lt;/div&gt;
      &lt;div className="grid grid-cols-2 gap-4"&gt;
        &lt;div&gt;
          &lt;Label htmlFor="projectCity"&gt;{t("form.city")} *&lt;/Label&gt;
          &lt;Input id="projectCity" placeholder={t("form.city")} {...register("projectCity")} className="mt-1" /&gt;
          {errors.projectCity &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.projectCity.message}&lt;/p&gt;}
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label htmlFor="projectState"&gt;{t("form.state")} *&lt;/Label&gt;
          &lt;Input id="projectState" placeholder={t("form.statePlaceholder")} {...register("projectState")} className="mt-1" /&gt;
          {errors.projectState &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.projectState.message}&lt;/p&gt;}
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;Label htmlFor="projectZipCode"&gt;{t("form.zipCode")}&lt;/Label&gt;
        &lt;Input id="projectZipCode" placeholder={t("form.zipPlaceholder")} {...register("projectZipCode")} className="mt-1" /&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default StepLocation;
```

src/components/orcamento/StepProject.tsx

```
import { UseFormRegister, FieldErrors, UseFormWatch, UseFormSetValue } from "react-hook-form";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { RadioGroup, RadioGroupItem } from "@/components/ui/radio-group";
import { Checkbox } from "@/components/ui/checkbox";
import { Textarea } from "@/components/ui/textarea";
import { Separator } from "@/components/ui/separator";
import { AlertTriangle } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";
import { FormData, PROJECT_TYPES, PROJECT_LEVELS, BUILDING_STATUS_OPTIONS, REQUIRED_SPACES, DARA_SERVICES } from "./types";

interface StepProjectProps {
  register: UseFormRegister&lt;FormData&gt;;
  errors: FieldErrors&lt;FormData&gt;;
  watch: UseFormWatch&lt;FormData&gt;;
  setValue: UseFormSetValue&lt;FormData&gt;;
}

const projectTypeKeys = [
  "projectType.singleFamily", "projectType.multiFamily", "projectType.commercial",
  "projectType.renovation", "projectType.addition", "projectType.adu",
  "projectType.deck", "projectType.garage", "projectType.other",
];

const buildingStatusKeys = [
  { value: "occupied", key: "buildingStatus.occupied" },
  { value: "under_construction", key: "buildingStatus.underConstruction" },
  { value: "vacant", key: "buildingStatus.vacant" },
  { value: "new_construction", key: "buildingStatus.newConstruction" },
];

const levelKeys = [
  { id: "basement", key: "level.basement" },
  { id: "first_floor", key: "level.firstFloor" },
  { id: "second_floor", key: "level.secondFloor" },
  { id: "attic_loft", key: "level.atticLoft" },
  { id: "garage_level", key: "level.garageLevel" },
];

const spacesCategoryKeys: Record&lt;string, string&gt; = {
  privateSpaces: "spaces.private",
  bathrooms: "spaces.bathrooms",
  socialAreas: "spaces.social",
  kitchenSupport: "spaces.kitchen",
  workWellness: "spaces.work",
  circulationStorage: "spaces.circulation",
  technicalAreas: "spaces.technical",
  outdoorAreas: "spaces.outdoor",
};

const daraServiceCategoryKeys: Record&lt;string, string&gt; = {
  architecture: "daraService.architecture",
  interiors: "daraService.interiors",
  visualization: "daraService.visualization",
  documentation: "daraService.documentation",
  construction: "daraService.construction",
  additional: "daraService.additional",
};

const StepProject = ({ register, errors, watch, setValue }: StepProjectProps) =&gt; {
  const { t } = useLanguage();
  const projectType = watch("projectType");
  const requiredSpaces = watch("requiredSpaces") || [];
  const selectedDaraServices = watch("selectedDaraServices") || [];
  const projectLevels = watch("projectLevels") || [];

  const handleSpaceChange = (spaceId: string, checked: boolean) =&gt; {
    const current = requiredSpaces || [];
    setValue("requiredSpaces", checked ? [...current, spaceId] : current.filter(s =&gt; s !== spaceId));
  };

  const handleDaraServiceChange = (serviceId: string, checked: boolean) =&gt; {
    const current = selectedDaraServices || [];
    setValue("selectedDaraServices", checked ? [...current, serviceId] : current.filter(s =&gt; s !== serviceId));
  };

  const handleLevelChange = (levelId: string, checked: boolean) =&gt; {
    const current = projectLevels || [];
    setValue("projectLevels", checked ? [...current, levelId] : current.filter(l =&gt; l !== levelId));
  };

  return (
    &lt;div className="space-y-8"&gt;
      {/* STEP 1 — PROJECT INFORMATION */}
      &lt;section className="space-y-6"&gt;
        &lt;div className="flex items-center gap-3"&gt;
          &lt;span className="flex items-center justify-center w-8 h-8 rounded-full bg-primary text-primary-foreground text-sm font-semibold"&gt;1&lt;/span&gt;
          &lt;h3 className="text-lg font-semibold text-foreground"&gt;{t("form.projectInfo")}&lt;/h3&gt;
        &lt;/div&gt;

        &lt;div&gt;
          &lt;Label className="text-base font-medium"&gt;{t("form.projectType")} *&lt;/Label&gt;
          &lt;p className="text-sm text-muted-foreground mb-3"&gt;{t("form.selectOne")}&lt;/p&gt;
          &lt;RadioGroup value={watch("projectType")} onValueChange={(value) =&gt; setValue("projectType", value)} className="grid grid-cols-1 sm:grid-cols-2 gap-2"&gt;
            {projectTypeKeys.map((key) =&gt; {
              const label = t(key);
              return (
                &lt;div key={key} className="flex items-center space-x-2 p-2 rounded-md hover:bg-muted/50 transition-colors"&gt;
```

```
            &lt;RadioGroupItem value={label} id={key} /&gt;
            &lt;Label htmlFor={key} className="cursor-pointer text-sm font-normal"&gt;{label}&lt;/Label&gt;
          &lt;/div&gt;
        );
      })}
    &lt;/RadioGroup&gt;
    {errors.projectType &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.projectType.message}&lt;/p&gt;}
  &lt;/div&gt;

  {projectType === t("projectType.other") &amp;&amp; (
    &lt;div&gt;
      &lt;Label htmlFor="otherProjectType"&gt;{t("form.ifOtherDescribe")}&lt;/Label&gt;
      &lt;Input id="otherProjectType" placeholder={t("form.describeProjectType")} {...register("otherProjectType")} className="mt-1" /&gt;
    &lt;/div&gt;
  )}

  &lt;div&gt;
    &lt;Label className="text-base font-medium"&gt;{t("form.hasExistingPlans")} *&lt;/Label&gt;
    &lt;RadioGroup value={watch("hasExistingProject")} onValueChange={(value) =&gt; setValue("hasExistingProject", value as "yes" | "no")} className="mt-2 flex gap-6"&
      &lt;div className="flex items-center space-x-2"&gt;&lt;RadioGroupItem value="yes" id="hasProject-yes" /&gt;&lt;Label htmlFor="hasProject-yes" className="cursor-p
      &lt;div className="flex items-center space-x-2"&gt;&lt;RadioGroupItem value="no" id="hasProject-no" /&gt;&lt;Label htmlFor="hasProject-no" className="cursor-poin
    &lt;/RadioGroup&gt;
    {errors.hasExistingProject &amp;&amp; &lt;p className="text-sm text-destructive mt-1"&gt;{errors.hasExistingProject.message}&lt;/p&gt;}
  &lt;/div&gt;

  &lt;div&gt;
    &lt;Label className="text-base font-medium"&gt;{t("form.hasEngineer")}&lt;/Label&gt;
    &lt;RadioGroup value={watch("hasEngineer")} onValueChange={(value) =&gt; setValue("hasEngineer", value as "yes" | "no")} className="mt-2 flex flex-wrap gap-4"&gt;
      &lt;div className="flex items-center space-x-2"&gt;&lt;RadioGroupItem value="yes" id="engineer-yes" /&gt;&lt;Label htmlFor="engineer-yes" className="cursor-point
      &lt;div className="flex items-center space-x-2"&gt;&lt;RadioGroupItem value="no" id="engineer-no" /&gt;&lt;Label htmlFor="engineer-no" className="cursor-pointer"
    &lt;/RadioGroup&gt;
  &lt;/div&gt;

  &lt;div&gt;
    &lt;Label className="text-base font-medium"&gt;{t("form.buildingStatus")}&lt;/Label&gt;
    &lt;RadioGroup value={watch("buildingStatus")} onValueChange={(value) =&gt; setValue("buildingStatus", value)} className="mt-2 grid grid-cols-2 gap-2"&gt;
      {buildingStatusKeys.map((option) =&gt; (
        &lt;div key={option.value} className="flex items-center space-x-2 p-2 rounded-md hover:bg-muted/50 transition-colors"&gt;
          &lt;RadioGroupItem value={option.value} id={`status-${option.value}`} /&gt;
          &lt;Label htmlFor={`status-${option.value}`} className="cursor-pointer text-sm"&gt;{t(option.key)}&lt;/Label&gt;
        &lt;/div&gt;
      ))}
    &lt;/RadioGroup&gt;
  &lt;/div&gt;
&lt;/section&gt;

&lt;Separator /&gt;

{/* STEP 2 — PROJECT SIZE &amp; LEVELS */}
&lt;section className="space-y-6"&gt;
  &lt;div class="flex items-center gap-3"&gt;
    &lt;span class="flex items-center justify-center w-8 h-8 rounded-full bg-primary text-primary-foreground text-sm font-semibold"&gt;2&lt;/span&gt;
    &lt;h3 class="text-lg font-semibold text-foreground"&gt;{t("form.projectSizeLevels")}&lt;/h3&gt;
  &lt;/div&gt;
  &lt;div class="grid grid-cols-1 sm:grid-cols-2 gap-4"&gt;
    &lt;div&gt;
      &lt;Label htmlFor="existingArea"&gt;{t("form.existingArea")}&lt;/Label&gt;
      &lt;p class="text-sm text-muted-foreground mb-1"&gt;{t("form.sqftOptional")}&lt;/p&gt;
      &lt;Input id="existingArea" placeholder="e.g., 1,800" {...register("existingArea")} /&gt;
    &lt;/div&gt;
    &lt;div&gt;
      &lt;Label htmlFor="estimatedSize"&gt;{t("form.proposedArea")} *&lt;/Label&gt;
      &lt;p class="text-sm text-muted-foreground mb-1"&gt;{t("form.proposedAreaDesc")}&lt;/p&gt;
      &lt;Input id="estimatedSize" placeholder="e.g., 2,500 sq ft" {...register("estimatedSize")} /&gt;
      {errors.estimatedSize &amp;&amp; &lt;p class="text-sm text-destructive mt-1"&gt;{errors.estimatedSize.message}&lt;/p&gt;}
    &lt;/div&gt;
  &lt;/div&gt;
  &lt;div&gt;
    &lt;Label class="text-base font-medium"&gt;{t("form.projectLevels")}&lt;/Label&gt;
    &lt;div class="mt-2 grid grid-cols-2 sm:grid-cols-3 gap-2"&gt;
      {levelKeys.map((level) =&gt; (
        &lt;div key={level.id} class="flex items-center space-x-2 p-2 rounded-md hover:bg-muted/50 transition-colors"&gt;
          &lt;Checkbox id={level.id} checked={projectLevels.includes(level.id)} onCheckedChange={(checked) =&gt; handleLevelChange(level.id, checked as boolean)} /&gt;
          &lt;Label htmlFor={level.id} class="cursor-pointer text-sm font-normal"&gt;{t(level.key)}&lt;/Label&gt;
        &lt;/div&gt;
      ))}
    &lt;/div&gt;
  &lt;/div&gt;
&lt;/section&gt;

&lt;Separator /&gt;

{/* STEP 3 — FILE UPLOADS &amp; REFERENCES */}
&lt;section class="space-y-6"&gt;
  &lt;div class="flex items-center gap-3"&gt;
    &lt;span class="flex items-center justify-center w-8 h-8 rounded-full bg-primary text-primary-foreground text-sm font-semibold"&gt;3&lt;/span&gt;
    &lt;h3 class="text-lg font-semibold text-foreground"&gt;{t("form.fileUploads")}&lt;/h3&gt;
  &lt;/div&gt;
  &lt;div&gt;
    &lt;Label htmlFor="referenceNotes"&gt;{t("form.uploadDesc")}&lt;/Label&gt;
    &lt;p class="text-sm text-muted-foreground mb-2"&gt;{t("form.acceptsFormats")}&lt;/p&gt;
    &lt;Textarea id="referenceNotes" placeholder={t("form.referenceNotesPlaceholder")} class="min-h-[80px]" {...register("referenceNotes")} /&gt;
  &lt;/div&gt;
  &lt;div&gt;
    &lt;Label htmlFor="referenceLinks"&gt;{t("form.referenceLinks")}&lt;/Label&gt;
    &lt;p class="text-sm text-muted-foreground mb-2"&gt;{t("form.referenceLinksDesc")}&lt;/p&gt;
    &lt;Textarea id="referenceLinks" placeholder={t("form.referenceLinksPlaceholder")} class="min-h-[60px]" {...register("referenceLinks")} /&gt;
  &lt;/div&gt;
&lt;/section&gt;

&lt;Separator /&gt;
```

```
      {/* STEP 4 — PROGRAM OF REQUIREMENTS */}
      &lt;section className="space-y-6"&gt;
        &lt;div className="flex items-center gap-3"&gt;
          &lt;span className="flex items-center justify-center w-8 h-8 rounded-full bg-primary text-primary-foreground text-sm font-semibold"&gt;4&lt;/span&gt;
          &lt;h3 className="text-lg font-semibold text-foreground"&gt;{t("form.programRequirements")}&lt;/h3&gt;
        &lt;/div&gt;
        &lt;p className="text-sm text-muted-foreground -mt-4"&gt;{t("form.programRequirementsDesc")}&lt;/p&gt;

        {Object.entries(REQUIRED_SPACES).map(([key, category]) =&gt; (
          &lt;div key={key} className="space-y-2"&gt;
            &lt;h4 className="font-medium text-foreground text-sm"&gt;{t(spacesCategoryKeys[key])}&lt;/h4&gt;
            &lt;div className="grid grid-cols-2 sm:grid-cols-3 gap-1"&gt;
              {category.spaces.map((space) =&gt; (
                &lt;div key={space.id} className="flex items-center space-x-2 p-2 rounded-md hover:bg-muted/50 transition-colors"&gt;
                  &lt;Checkbox id={space.id} checked={requiredSpaces.includes(space.id)} onCheckedChange={(checked) =&gt; handleSpaceChange(space.id, checked as boolean)} /&
                  &lt;Label htmlFor={space.id} className="cursor-pointer text-sm font-normal"&gt;{space.label}&lt;/Label&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        ))}

        &lt;div&gt;
          &lt;Label htmlFor="otherSpace"&gt;{t("form.other")}&lt;/Label&gt;
          &lt;Input id="otherSpace" placeholder={t("form.otherPlaceholder")} {...register("otherSpace")} className="mt-1" /&gt;
        &lt;/div&gt;
      &lt;/section&gt;

      &lt;Separator /&gt;

      {/* STEP 5 — SERVICES OFFERED BY DARA */}
      &lt;section className="space-y-6"&gt;
        &lt;div className="flex items-center gap-3"&gt;
          &lt;span className="flex items-center justify-center w-8 h-8 rounded-full bg-primary text-primary-foreground text-sm font-semibold"&gt;5&lt;/span&gt;
          &lt;h3 className="text-lg font-semibold text-foreground"&gt;{t("form.servicesOffered")}&lt;/h3&gt;
        &lt;/div&gt;
        &lt;div className="-mt-4"&gt;
          &lt;p className="text-sm font-medium text-foreground"&gt;{t("form.designDrafting")}&lt;/p&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;{t("form.selectServices")}&lt;/p&gt;
        &lt;/div&gt;

        {Object.entries(DARA_SERVICES).map(([key, category]) =&gt; (
          &lt;div key={key} className="space-y-2"&gt;
            &lt;h4 className="font-medium text-primary text-sm uppercase tracking-wide"&gt;{t(daraServiceCategoryKeys[key])}&lt;/h4&gt;
            &lt;div className="grid grid-cols-1 gap-1"&gt;
              {category.services.map((service) =&gt; (
                &lt;div key={service.id} className="flex items-center space-x-2 p-2 rounded-md hover:bg-muted/50 transition-colors"&gt;
                  &lt;Checkbox id={service.id} checked={selectedDaraServices.includes(service.id)} onCheckedChange={(checked) =&gt; handleDaraServiceChange(service.id, check
                  &lt;Label htmlFor={service.id} className="cursor-pointer text-sm font-normal"&gt;{service.label}&lt;/Label&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        ))}

        &lt;div className="bg-muted/50 border border-border rounded-lg p-4 mt-6"&gt;
          &lt;div className="flex items-start gap-3"&gt;
            &lt;AlertTriangle className="w-5 h-5 text-amber-500 mt-0.5 flex-shrink-0" /&gt;
            &lt;div&gt;
              &lt;h5 className="font-semibold text-foreground text-sm mb-2"&gt;{t("form.disclaimer")}&lt;/h5&gt;
              &lt;p className="text-sm text-muted-foreground leading-relaxed"&gt;{t("form.disclaimerText")}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/section&gt;
    &lt;/div&gt;
  );
};

export default StepProject;
```

src/components/orcamento/types.ts

```
import { z } from "zod";

export const CLIENT_TYPES = [
  { value: "cliente_final", label: "Homeowner" },
  { value: "construtor", label: "General Contractor" },
  { value: "arquiteto", label: "Architect" },
  { value: "engenheiro", label: "Engineer" },
  { value: "real_estate", label: "Real Estate Developer" },
] as const;

export const PROJECT_TYPES = [
  "Single-Family Residential",
  "Multi-Family Residential",
  "Commercial",
  "Renovation / Remodel",
  "Addition / Extension",
  "ADU (Accessory Dwelling Unit)",
  "Deck",
  "Garage (Attached / Detached)",
  "Other",
] as const;

export const PROJECT_LEVELS = [
  { id: "basement", label: "Basement" },
  { id: "first_floor", label: "First Floor" },
  { id: "second_floor", label: "Second Floor" },
  { id: "attic_loft", label: "Attic / Loft" },
  { id: "garage_level", label: "Garage Level" },
] as const;

export const BUILDING_STATUS_OPTIONS = [
  { value: "occupied", label: "Occupied" },
  { value: "under_construction", label: "Under Construction" },
  { value: "vacant", label: "Vacant" },
  { value: "new_construction", label: "N/A (New Construction)" },
] as const;

export const REQUIRED_SPACES = {
  privateSpaces: {
    title: "Private Spaces",
    spaces: [
      { id: "primary_suite", label: "Primary Suite" },
      { id: "secondary_bedroom", label: "Secondary Bedroom" },
      { id: "guest_bedroom", label: "Guest Bedroom" },
      { id: "flexible_room", label: "Flexible / Multi-Purpose Room" },
    ],
  },
  bathrooms: {
    title: "Bathrooms",
    spaces: [
      { id: "primary_bathroom", label: "Primary Bathroom" },
      { id: "full_bathroom", label: "Full Bathroom" },
      { id: "secondary_bathroom", label: "Secondary Bathroom" },
      { id: "powder_room", label: "Powder Room" },
    ],
  },
  socialAreas: {
    title: "Social Areas",
    spaces: [
      { id: "living_room", label: "Living Room" },
      { id: "family_room", label: "Family Room / TV Room" },
      { id: "dining_room", label: "Dining Room" },
      { id: "game_room", label: "Game Room" },
      { id: "library", label: "Library / Lounge" },
    ],
  },
  kitchenSupport: {
    title: "Kitchen &amp; Support Spaces",
    spaces: [
      { id: "kitchen", label: "Kitchen" },
      { id: "pantry", label: "Pantry" },
      { id: "laundry", label: "Laundry Room" },
      { id: "mudroom", label: "Mudroom / Service Entry" },
      { id: "utility_room", label: "Utility / Storage Room" },
    ],
  },
  workWellness: {
    title: "Work &amp; Wellness",
    spaces: [
      { id: "home_office", label: "Home Office" },
      { id: "home_gym", label: "Home Gym / Wellness Area" },
    ],
  },
  circulationStorage: {
    title: "Circulation &amp; Storage",
    spaces: [
      { id: "entry_hall", label: "Entry Hall / Foyer" },
      { id: "hallways", label: "Hallways / Circulation" },
      { id: "walk_in_closet", label: "Walk-In Closet" },
      { id: "linen_closet", label: "Linen Closet" },
    ],
  },
  technicalAreas: {
    title: "Technical Areas",
    spaces: [
      { id: "mechanical_room", label: "Mechanical Room (HVAC / Boiler)" },
      { id: "electrical_panel", label: "Electrical Panel / Technical Room" },
      { id: "water_heater", label: "Water Heater Closet" },
```

```
    ],
  },
  outdoorAreas: {
    title: "Outdoor Areas",
    spaces: [
      { id: "porch_terrace", label: "Porch / Terrace" },
      { id: "covered_porch", label: "Covered Porch" },
      { id: "screened_porch", label: "Screened Porch" },
      { id: "deck", label: "Deck" },
      { id: "yard_patio", label: "Yard / Patio" },
      { id: "garage", label: "Garage" },
      { id: "exterior_storage", label: "Exterior Storage" },
      { id: "trash_recycling", label: "Trash &amp; Recycling Area" },
    ],
  },
} as const;

export const TIMELINE_OPTIONS = [
  { id: "asap", label: "ASAP" },
  { id: "within_30_days", label: "Within 30 days" },
  { id: "1_3_months", label: "1–3 months" },
  { id: "3_plus_months", label: "3+ months" },
  { id: "flexible", label: "Flexible" },
] as const;

export const BUDGET_OPTIONS = [
  { id: "under_2500", label: "Under $2,500" },
  { id: "2500_5000", label: "$2,500 – $5,000" },
  { id: "5000_10000", label: "$5,000 – $10,000" },
  { id: "10000_plus", label: "$10,000+" },
  { id: "not_sure", label: "Not sure yet" },
] as const;

export const REFERRAL_OPTIONS = [
  { id: "google", label: "Google" },
  { id: "referral", label: "Referral" },
  { id: "social_media", label: "Social Media" },
  { id: "contractor", label: "Contractor" },
  { id: "returning_client", label: "Returning Client" },
  { id: "other", label: "Other" },
] as const;

export const DARA_SERVICES = {
  architecture: {
    title: "ARCHITECTURAL DESIGN &amp; DRAFTING",
    services: [
      { id: "residential_arch", label: "Residential architectural design layouts" },
      { id: "floor_plans", label: "Technical floor plans for design coordination and permitting support" },
      { id: "sections_elevations", label: "Sections and elevations for visualization and documentation" },
      { id: "construction_details", label: "Construction details (design intent only)" },
      { id: "layout_studies", label: "Layout and space organization studies" },
      { id: "addition_renovation", label: "Addition and renovation design concepts" },
    ],
  },
  interiors: {
    title: "INTERIOR DESIGN &amp; SPACE PLANNING",
    services: [
      { id: "interior_design", label: "Interior design concept development" },
      { id: "furniture_layout", label: "Furniture layout and space planning" },
      { id: "kitchen_bath", label: "Kitchen and bathroom design layouts" },
      { id: "custom_millwork", label: "Custom millwork design drawings" },
      { id: "finishes_materials", label: "Finishes and materials selection assistance" },
      { id: "fixtures_specification", label: "Fixtures, hardware, and plumbing specification support" },
    ],
  },
  visualization: {
    title: "3D VISUALIZATION",
    services: [
      { id: "exterior_renders", label: "Exterior renderings for presentation purposes" },
      { id: "interior_renders", label: "Interior renderings for design validation" },
    ],
  },
  documentation: {
    title: "TECHNICAL DRAFTING &amp; DOCUMENTATION",
    services: [
      { id: "pdf_to_cad", label: "PDF to CAD conversion (technical redrawing)" },
      { id: "vectorization", label: "Vectorization of existing or legacy drawings" },
      { id: "technical_sheets", label: "Technical drawing set for coordination and review" },
    ],
  },
  construction: {
    title: "CONSTRUCTION COORDINATION SUPPORT",
    services: [
      { id: "project_adjustments", label: "Design adjustments for existing construction conditions" },
      { id: "as_built", label: "As-built drawings based on provided field information" },
    ],
  },
  additional: {
    title: "ADDITIONAL DESIGN SERVICES",
    services: [
      { id: "outdoor_design", label: "Outdoor living design concepts (deck, pool, patio)" },
      { id: "presentation_boards", label: "Presentation boards for clients and investors" },
    ],
  },
} as const;

export const formSchema = z.object({
  // Step 1 - Project Location
  projectAddress: z.string().min(1, "Address is required"),
  projectCity: z.string().min(1, "City is required"),
  projectState: z.string().min(1, "State is required"),
```

```
  projectZipCode: z.string().optional(),
  // Step 2 - Project Type &amp; Info
  projectType: z.string().min(1, "Project type is required"),
  otherProjectType: z.string().optional(),
  hasExistingProject: z.enum(["yes", "no"], {
    required_error: "Please indicate if you have existing plans",
  }),
  hasEngineer: z.enum(["yes", "no"]).optional(),
  buildingStatus: z.string().optional(),

  // Project Size &amp; Levels
  existingArea: z.string().optional(),
  estimatedSize: z.string().min(1, "Estimated project area is required"),
  projectLevels: z.array(z.string()).optional(),

  // File Uploads &amp; References
  referenceNotes: z.string().optional(),
  referenceLinks: z.string().optional(),

  // Program of Requirements
  requiredSpaces: z.array(z.string()).optional(),
  otherSpace: z.string().optional(),

  // DARA Services
  selectedDaraServices: z.array(z.string()).optional(),

  // Legacy fields (kept for compatibility)
  totalArea: z.string().optional(),
  numberOfFloors: z.string().optional(),
  desiredServices: z.array(z.string()).optional(),

  // Step 3 - Client Info
  clientType: z.enum(["cliente_final", "construtor", "arquiteto", "engenheiro", "real_estate"], {
    required_error: "Please select your client type",
  }),
  fullName: z.string().min(2, "Full name is required"),
  email: z.string().email("Invalid email address"),
  phone: z.string().min(10, "Invalid phone number"),
  // Company fields (required for construtor)
  companyName: z.string().optional(),
  companyAddress: z.string().optional(),
  companyInstagram: z.string().optional(),
  companyPhone: z.string().optional(),
  companyEmail: z.string().optional(),

  // Step 4 - Additional Details
  deadline: z.string().optional(),
  budget: z.string().optional(),
  description: z.string().optional(),
  howDidYouFindUs: z.string().optional(),
  acceptTerms: z.boolean().refine(val =&gt; val === true, {
    message: "You must accept the terms",
  }),
}).refine((data) =&gt; {
  // Company name is required for General Contractor
  if (data.clientType === "construtor" &amp;&amp; !data.companyName) {
    return false;
  }
  return true;
}, {
  message: "Company name is required for General Contractors",
  path: ["companyName"],
});

export type FormData = z.infer&lt;typeof formSchema&gt;;

export const STEPS = [
  { id: 1, title: "Location" },
  { id: 2, title: "Project" },
  { id: 3, title: "Your Info" },
  { id: 4, title: "Details" },
] as const;

// Helper function to get all spaces as flat array for submission
export const getAllSpacesFlat = () =&gt; {
  const allSpaces: { id: string; label: string }[] = [];
  Object.values(REQUIRED_SPACES).forEach(category =&gt; {
    allSpaces.push(...category.spaces);
  });
  return allSpaces;
};
```

src/components/profile

src/components/profile/ProfileEditor.tsx

```
import { useState, useEffect, useRef } from "react";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { Skeleton } from "@/components/ui/skeleton";
import { Camera, Save, User } from "lucide-react";
import { useToast } from "@/hooks/use-toast";

interface ProfileData {
  full_name: string;
  email: string;
  phone: string;
  instagram: string;
  avatar_url: string | null;
}

const ProfileEditor = () =&gt; {
  const { user } = useAuth();
  const { t } = useLanguage();
  const { toast } = useToast();
  const fileInputRef = useRef&lt;HTMLInputElement&gt;(null);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [uploading, setUploading] = useState(false);
  const [profile, setProfile] = useState&lt;ProfileData&gt;({
    full_name: "",
    email: "",
    phone: "",
    instagram: "",
    avatar_url: null,
  });

  useEffect(() =&gt; {
    const fetchProfile = async () =&gt; {
      if (!user) return;
      const { data, error } = await supabase
        .from("profiles")
        .select("full_name, email, phone, instagram, avatar_url")
        .eq("id", user.id)
        .single();

      if (data) {
        setProfile({
          full_name: data.full_name || "",
          email: data.email || user.email || "",
          phone: data.phone || "",
          instagram: (data as any).instagram || "",
          avatar_url: data.avatar_url || null,
        });
      }
      setLoading(false);
    };
    fetchProfile();
  }, [user]);

  const handleSave = async () =&gt; {
    if (!user) return;
    setSaving(true);
    const { error } = await supabase
      .from("profiles")
      .update({
        full_name: profile.full_name,
        email: profile.email,
        phone: profile.phone,
        instagram: profile.instagram,
      })
      .eq("id", user.id);

    if (error) {
      toast({ title: t("profile.errorSaving"), description: error.message, variant: "destructive" });
    } else {
      // Also update auth metadata so name shows in header
      await supabase.auth.updateUser({ data: { full_name: profile.full_name } });
      toast({ title: t("profile.saved"), description: t("profile.savedDesc") });
    }
    setSaving(false);
  };

  const handleAvatarUpload = async (e: React.ChangeEvent&lt;HTMLInputElement&gt;) =&gt; {
    const file = e.target.files?.[0];
    if (!file || !user) return;

    setUploading(true);
    const fileExt = file.name.split(".").pop();
    const filePath = `${user.id}/avatar.${fileExt}`;

    // Upload to storage
    const { error: uploadError } = await supabase.storage
```

```
    .from("company-avatars")
    .upload(filePath, file, { upsert: true });

  if (uploadError) {
    toast({ title: t("profile.errorUpload"), description: uploadError.message, variant: "destructive" });
    setUploading(false);
    return;
  }

  const { data: urlData } = supabase.storage
    .from("company-avatars")
    .getPublicUrl(filePath);

  const avatarUrl = urlData.publicUrl;

  // Update profile with new avatar URL
  const { error: updateError } = await supabase
    .from("profiles")
    .update({ avatar_url: avatarUrl })
    .eq("id", user.id);

  if (updateError) {
    toast({ title: t("profile.errorSaving"), description: updateError.message, variant: "destructive" });
  } else {
    setProfile((prev) =&gt; ({ ...prev, avatar_url: avatarUrl }));
    toast({ title: t("profile.avatarUpdated") });
  }
  setUploading(false);
};

if (loading) {
  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;Skeleton className="h-8 w-48" /&gt;
      &lt;Skeleton className="h-64 w-full max-w-2xl" /&gt;
    &lt;/div&gt;
  );
}

const initials = profile.full_name
  ? profile.full_name.split(" ").map((n) =&gt; n[0]).join("").toUpperCase().slice(0, 2)
  : "?";

return (
  &lt;div className="p-6 space-y-6 max-w-2xl"&gt;
    &lt;div&gt;
      &lt;h1 className="text-2xl font-serif font-bold text-foreground"&gt;{t("profile.title")}&lt;/h1&gt;
      &lt;p className="text-xs text-muted-foreground"&gt;{t("profile.subtitle")}&lt;/p&gt;
    &lt;/div&gt;

    {/* Avatar Section */}
    &lt;Card&gt;
      &lt;CardHeader&gt;
        &lt;CardTitle className="text-lg"&gt;{t("profile.photo")}&lt;/CardTitle&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="flex items-center gap-6"&gt;
        &lt;div className="relative group"&gt;
          &lt;Avatar className="h-20 w-20"&gt;
            &lt;AvatarImage src={profile.avatar_url || undefined} alt={profile.full_name} /&gt;
            &lt;AvatarFallback className="text-xl bg-primary text-primary-foreground"&gt;{initials}&lt;/AvatarFallback&gt;
          &lt;/Avatar&gt;
          &lt;button
            onClick={() =&gt; fileInputRef.current?.click()}
            disabled={uploading}
            className="absolute inset-0 flex items-center justify-center rounded-full bg-foreground/50 opacity-0 group-hover:opacity-100 transition-opacity"
          &gt;
            &lt;Camera className="h-6 w-6 text-background" /&gt;
          &lt;/button&gt;
          &lt;input
            ref={fileInputRef}
            type="file"
            accept="image/*"
            className="hidden"
            onChange={handleAvatarUpload}
          /&gt;
        &lt;/div&gt;
        &lt;div className="space-y-1"&gt;
          &lt;p className="font-medium text-foreground"&gt;{profile.full_name || t("profile.noName")}&lt;/p&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;{profile.email}&lt;/p&gt;
          &lt;Button variant="outline" size="sm" onClick={() =&gt; fileInputRef.current?.click()} disabled={uploading}&gt;
            &lt;Camera className="h-4 w-4 mr-2" /&gt;
            {uploading ? t("profile.uploading") : t("profile.changePhoto")}
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Personal Info */}
    &lt;Card&gt;
      &lt;CardHeader&gt;
        &lt;CardTitle className="text-lg flex items-center gap-2"&gt;
          &lt;User className="h-5 w-5" /&gt;
          {t("profile.personalInfo")}
        &lt;/CardTitle&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="space-y-4"&gt;
        &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
          &lt;div className="space-y-2"&gt;
            &lt;Label htmlFor="full_name"&gt;{t("profile.fullName")}&lt;/Label&gt;
            &lt;Input
              id="full_name"
```

```
                value={profile.full_name}
                onChange={(e) =&gt; setProfile((p) =&gt; ({ ...p, full_name: e.target.value }))}
                placeholder={t("profile.fullNamePlaceholder")}
              /&gt;
            &lt;/div&gt;
            &lt;div className="space-y-2"&gt;
              &lt;Label htmlFor="email"&gt;{t("profile.email")}&lt;/Label&gt;
              &lt;Input
                id="email"
                type="email"
                value={profile.email}
                onChange={(e) =&gt; setProfile((p) =&gt; ({ ...p, email: e.target.value }))}
                placeholder={t("profile.emailPlaceholder")}
              /&gt;
            &lt;/div&gt;
            &lt;div className="space-y-2"&gt;
              &lt;Label htmlFor="phone"&gt;{t("profile.phone")}&lt;/Label&gt;
              &lt;Input
                id="phone"
                value={profile.phone}
                onChange={(e) =&gt; setProfile((p) =&gt; ({ ...p, phone: e.target.value }))}
                placeholder={t("profile.phonePlaceholder")}
              /&gt;
            &lt;/div&gt;
            &lt;div className="space-y-2"&gt;
              &lt;Label htmlFor="instagram"&gt;Instagram&lt;/Label&gt;
              &lt;Input
                id="instagram"
                value={profile.instagram}
                onChange={(e) =&gt; setProfile((p) =&gt; ({ ...p, instagram: e.target.value }))}
                placeholder="@username"
              /&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          &lt;div className="flex justify-end pt-4"&gt;
            &lt;Button onClick={handleSave} disabled={saving}&gt;
              &lt;Save className="h-4 w-4 mr-2" /&gt;
              {saving ? t("profile.saving") : t("profile.save")}
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default ProfileEditor;
```

src/hooks/use-mobile.tsx

src/hooks/use-mobile.tsx

```
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState&lt;boolean | undefined&gt;(undefined);

  React.useEffect(() =&gt; {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
    const onChange = () =&gt; {
      setIsMobile(window.innerWidth &lt; MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth &lt; MOBILE_BREAKPOINT);
    return () =&gt; mql.removeEventListener("change", onChange);
  }, []);

  return !!isMobile;
}
```

src/hooks/use-toast.ts

src/hooks/use-toast.ts

```
import * as React from "react";

import type { ToastActionElement, ToastProps } from "@/components/ui/toast";

const TOAST_LIMIT = 1;
const TOAST_REMOVE_DELAY = 1000000;

type ToasterToast = ToastProps &amp; {
  id: string;
  title?: React.ReactNode;
  description?: React.ReactNode;
  action?: ToastActionElement;
};

const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
} as const;

let count = 0;

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER;
  return count.toString();
}

type ActionType = typeof actionTypes;

type Action =
  | {
      type: ActionType["ADD_TOAST"];
      toast: ToasterToast;
    }
  | {
      type: ActionType["UPDATE_TOAST"];
      toast: Partial&lt;ToasterToast&gt;;
    }
  | {
      type: ActionType["DISMISS_TOAST"];
      toastId?: ToasterToast["id"];
    }
  | {
      type: ActionType["REMOVE_TOAST"];
      toastId?: ToasterToast["id"];
    };

interface State {
  toasts: ToasterToast[];
}

const toastTimeouts = new Map&lt;string, ReturnType&lt;typeof setTimeout&gt;&gt;();

const addToRemoveQueue = (toastId: string) =&gt; {
  if (toastTimeouts.has(toastId)) {
    return;
  }

  const timeout = setTimeout(() =&gt; {
    toastTimeouts.delete(toastId);
    dispatch({
      type: "REMOVE_TOAST",
      toastId: toastId,
    });
  }, TOAST_REMOVE_DELAY);

  toastTimeouts.set(toastId, timeout);
};

export const reducer = (state: State, action: Action): State =&gt; {
  switch (action.type) {
    case "ADD_TOAST":
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      };

    case "UPDATE_TOAST":
      return {
        ...state,
        toasts: state.toasts.map((t) =&gt; (t.id === action.toast.id ? { ...t, ...action.toast } : t)),
      };

    case "DISMISS_TOAST": {
      const { toastId } = action;

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId);
      } else {
        state.toasts.forEach((toast) =&gt; {
```

```
          addToRemoveQueue(toast.id);
        });
      }

      return {
        ...state,
        toasts: state.toasts.map((t) =&gt;
          t.id === toastId || toastId === undefined
            ? {
                ...t,
                open: false,
              }
            : t,
        ),
      };
    }
    case "REMOVE_TOAST":
      if (action.toastId === undefined) {
        return {
          ...state,
          toasts: [],
        };
      }
      return {
        ...state,
        toasts: state.toasts.filter((t) =&gt; t.id !== action.toastId),
      };
  }
};

const listeners: Array&lt;(state: State) =&gt; void&gt; = [];

let memoryState: State = { toasts: [] };

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action);
  listeners.forEach((listener) =&gt; {
    listener(memoryState);
  });
}

type Toast = Omit&lt;ToasterToast, "id"&gt;;

function toast({ ...props }: Toast) {
  const id = genId();

  const update = (props: ToasterToast) =&gt;
    dispatch({
      type: "UPDATE_TOAST",
      toast: { ...props, id },
    });
  const dismiss = () =&gt; dispatch({ type: "DISMISS_TOAST", toastId: id });

  dispatch({
    type: "ADD_TOAST",
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) =&gt; {
        if (!open) dismiss();
      },
    },
  });

  return {
    id: id,
    dismiss,
    update,
  };
}

function useToast() {
  const [state, setState] = React.useState&lt;State&gt;(memoryState);

  React.useEffect(() =&gt; {
    listeners.push(setState);
    return () =&gt; {
      const index = listeners.indexOf(setState);
      if (index &gt; -1) {
        listeners.splice(index, 1);
      }
    };
  }, [state]);

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) =&gt; dispatch({ type: "DISMISS_TOAST", toastId }),
  };
}

export { useToast, toast };
```

src/hooks/useAdminRole.tsx

src/hooks/useAdminRole.tsx

```
import { useState, useEffect } from "react";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";

export const useAdminRole = () =&gt; {
  const { user } = useAuth();
  const [isAdmin, setIsAdmin] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const checkRole = async () =&gt; {
      if (!user) {
        setIsAdmin(false);
        setLoading(false);
        return;
      }
      const { data, error } = await supabase
        .from("user_roles")
        .select("role")
        .eq("user_id", user.id)
        .eq("role", "admin")
        .maybeSingle();
      setIsAdmin(!!data &amp;&amp; !error);
      setLoading(false);
    };
    checkRole();
  }, [user]);

  return { isAdmin, loading };
};
```

src/hooks/useAuth.tsx

src/hooks/useAuth.tsx

```
import { useState, useEffect, createContext, useContext, ReactNode } from "react";
import { User, Session } from "@supabase/supabase-js";
import { supabase } from "@/integrations/supabase/client";

interface AuthContextType {
  user: User | null;
  session: Session | null;
  loading: boolean;
  signIn: (email: string, password: string) =&gt; Promise&lt;{ error: Error | null }&gt;;
  signUp: (email: string, password: string, fullName: string) =&gt; Promise&lt;{ error: Error | null }&gt;;
  signOut: () =&gt; Promise&lt;void&gt;;
}

const AuthContext = createContext&lt;AuthContextType | undefined&gt;(undefined);

export const AuthProvider = ({ children }: { children: ReactNode }) =&gt; {
  const [user, setUser] = useState&lt;User | null&gt;(null);
  const [session, setSession] = useState&lt;Session | null&gt;(null);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    // Set up auth state listener FIRST
    const { data: { subscription } } = supabase.auth.onAuthStateChange(
      (event, session) =&gt; {
        setSession(session);
        setUser(session?.user ?? null);
        setLoading(false);
      }
    );

    // THEN check for existing session
    supabase.auth.getSession().then(({ data: { session } }) =&gt; {
      setSession(session);
      setUser(session?.user ?? null);
      setLoading(false);
    });

    return () =&gt; subscription.unsubscribe();
  }, []);

  const signIn = async (email: string, password: string) =&gt; {
    const { error } = await supabase.auth.signInWithPassword({
      email,
      password,
    });
    return { error: error as Error | null };
  };

  const signUp = async (email: string, password: string, fullName: string) =&gt; {
    const redirectUrl = `${window.location.origin}/`;

    const { error } = await supabase.auth.signUp({
      email,
      password,
      options: {
        emailRedirectTo: redirectUrl,
        data: {
          full_name: fullName,
        },
      },
    });
    return { error: error as Error | null };
  };

  const signOut = async () =&gt; {
    await supabase.auth.signOut();
  };

  return (
    &lt;AuthContext.Provider value={{ user, session, loading, signIn, signUp, signOut }}&gt;
      {children}
    &lt;/AuthContext.Provider&gt;
  );
};

export const useAuth = () =&gt; {
  const context = useContext(AuthContext);
  if (context === undefined) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  return context;
};
```

src/hooks/useFreelancerRole.tsx

src/hooks/useFreelancerRole.tsx

```
import { useState, useEffect } from "react";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";

export const useFreelancerRole = () =&gt; {
  const { user } = useAuth();
  const [isFreelancer, setIsFreelancer] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const checkRole = async () =&gt; {
      if (!user) {
        setIsFreelancer(false);
        setLoading(false);
        return;
      }

      const { data, error } = await supabase
        .from("user_roles")
        .select("role")
        .eq("user_id", user.id)
        .eq("role", "freelancer")
        .maybeSingle();

      setIsFreelancer(!!data &amp;amp;&amp;amp; !error);
      setLoading(false);
    };

    checkRole();
  }, [user]);

  return { isFreelancer, loading };
};
```

src/hooks/useLanguage.tsx

src/hooks/useLanguage.tsx

```
import { createContext, useContext, useState, useEffect, ReactNode } from "react";
import { translations, Language } from "@/i18n/translations";

interface LanguageContextType {
  language: Language;
  setLanguage: (lang: Language) =&gt; void;
  t: (key: string) =&gt; string;
}

const LanguageContext = createContext&lt;LanguageContextType | undefined&gt;(undefined);

export const LanguageProvider = ({ children }: { children: ReactNode }) =&gt; {
  const [language, setLanguageState] = useState&lt;Language&gt;(() =&gt; {
    const saved = localStorage.getItem("language");
    return (saved as Language) || "en";
  });

  useEffect(() =&gt; {
    localStorage.setItem("language", language);
  }, [language]);

  const setLanguage = (lang: Language) =&gt; {
    setLanguageState(lang);
  };

  const t = (key: string): string =&gt; {
    return translations[language][key] || key;
  };

  return (
    &lt;LanguageContext.Provider value={{ language, setLanguage, t }}&gt;
      {children}
    &lt;/LanguageContext.Provider&gt;
  );
};

export const useLanguage = () =&gt; {
  const context = useContext(LanguageContext);
  if (context === undefined) {
    throw new Error("useLanguage must be used within a LanguageProvider");
  }
  return context;
};
```

src/i18n/translations.ts

src/i18n/translations.ts

```
export type Language = "en" | "pt";

export const translations: Record&lt;Language, Record&lt;string, string&gt;&gt; = {
  en: {
    // Header
    "nav.howWeWork": "How We Work",
    "nav.portfolio": "Portfolio",
    "nav.requestQuote": "Request a Quote",
    "nav.contact": "Contact",
    "nav.clientArea": "Client Area",

    // Hero Section
    "hero.badge": "Excellence in Architectural Projects",
    "hero.title": "Transforming your ideas into",
    "hero.titleHighlight": "exceptional projects",
    "hero.subtitle": "Specialists in technical drafting, executive projects, and wood frame structures. From conception to final detailing.",
    "hero.cta.quote": "Request a Quote",
    "hero.cta.portfolio": "View Portfolio",
    "hero.stats.projects": "Projects Delivered",
    "hero.stats.experience": "Years of Experience",
    "hero.stats.clients": "Satisfied Clients",

    // Services Section
    "services.title": "Design-Led Architectural Solutions",
    "services.subtitle": "We deliver design-driven architectural solutions from concept development to construction-ready documentation, aligned with U.S. building standards
    "services.learnMore": "Learn more",
    "services.cta": "Request Free Quote",

    // Portfolio Section
    "portfolio.title": "Selected Projects",
    "portfolio.subtitle": "A curated selection of residential and commercial projects developed for builders, investors, and private clients across the U.S.",
    "portfolio.badge": "Designed to meet U.S. building codes and construction standards.",
    "portfolio.viewProject": "View Project",
    "portfolio.viewAll": "View all projects",
    "portfolio.item1.title": "Modern Single-Family Residence",
    "portfolio.item1.category": "Residential Architecture",
    "portfolio.item2.title": "Wood Frame Residential Build",
    "portfolio.item2.category": "Wood Framing Layout",
    "portfolio.item3.title": "Commercial Retail Building",
    "portfolio.item3.category": "Commercial Design",

    // CTA Section
    "cta.title": "Let's Build Something Exceptional",
    "cta.subtitle": "Tell us about your project. We'll guide you through the next steps with clarity, strategy, and precision.",
    "cta.proposal": "Request a Proposal",
    "cta.call": "Schedule a Call",

    // Footer
    "footer.tagline": "Transforming ideas into exceptional projects with precision and excellence.",
    "footer.quickLinks": "Quick Links",
    "footer.services": "Services",
    "footer.contact": "Contact",
    "footer.rights": "All rights reserved.",
    "footer.service1": "Technical Drafting",
    "footer.service2": "Executive Project",
    "footer.service3": "Wood Frame Structures",
    "footer.service4": "Project Budgeting",

    // Auth
    "auth.title": "Client Portal",
    "auth.subtitle": "Access your projects, files, and communications",
    "auth.signIn": "Sign In",
    "auth.createAccount": "Create Account",
    "auth.welcomeBack": "Welcome Back",
    "auth.enterCredentials": "Enter your credentials to access your account",
    "auth.register": "Register to access your project portal",
    "auth.email": "Email",
    "auth.password": "Password",
    "auth.fullName": "Full Name",
    "auth.confirmPassword": "Confirm Password",
    "auth.signingIn": "Signing in...",
    "auth.creatingAccount": "Creating account...",
    "auth.forgotPassword": "Forgot your password?",
    "auth.resetPassword": "Reset Password",
    "auth.resetInstructions": "Enter your email and we'll send you a reset link",
    "auth.sendResetLink": "Send Reset Link",
    "auth.sending": "Sending...",
    "auth.backToLogin": "Back to Login",
    "auth.checkEmail": "Check your email for the reset link",

    // Quote Form
    "quote.title": "Request a Quote",
    "quote.subtitle": "Complete the form below to receive a personalized proposal",
    "quote.step.location": "Location",
    "quote.step.project": "Project",
    "quote.step.client": "Your Info",
    "quote.step.details": "Details",
    "quote.locationDesc": "Where will the project be located?",
    "quote.projectDesc": "Tell us about your project",
    "quote.clientDesc": "Your contact information",
    "quote.detailsDesc": "Additional information (optional)",
    "quote.back": "Back",
    "quote.next": "Next",
```

"quote.submit": "Submit Request",
"quote.submitting": "Submitting...",

// Thank You Page
"thankyou.title": "Project Received",
"thankyou.message": "Thank you for your submission.",
"thankyou.projectCreated": "Your project has been created under:",
"thankyou.proposalTime": "You will receive a detailed proposal within 2-3 business days.",
"thankyou.portalAccess": "We've sent you an email with instructions to access your Client Portal.",
"thankyou.goHome": "Return Home",
"thankyou.goPortal": "Go to Client Portal",

// Dashboard
"dashboard.welcome": "Welcome",
"dashboard.welcomeBack": "Welcome back",
"dashboard.overview": "Here's an overview of your projects and activity",
"dashboard.home": "Home",
"dashboard.projects": "Projects",
"dashboard.quotes": "Quotes",
"dashboard.messages": "Messages",
"dashboard.profile": "Profile",
"dashboard.company": "Company",
"dashboard.newQuote": "New Quote",
"dashboard.signOut": "Sign Out",
"dashboard.noProjects": "No projects yet",
"dashboard.noProjectsDesc": "Start by requesting a quote for your first project",
"dashboard.openProject": "Open Project",
"dashboard.activeProjects": "Active Projects",
"dashboard.quotesInvoices": "Quotes & Invoices",
"dashboard.newQuoteRequest": "New Quote Request",
"dashboard.yourProjects": "Your Projects",
"dashboard.requestQuote": "Request a Quote",
"dashboard.myProjects": "My Projects",
"dashboard.viewManageProjects": "View and manage all your projects",
"dashboard.comingSoon": "Coming Soon",
"dashboard.comingSoonDesc": "This section is under development. Check back soon!",
"dashboard.myProfile": "My Profile",
"dashboard.manageProfile": "Manage your personal information",
"dashboard.companyInfo": "Company Information",
"dashboard.manageCompany": "Manage your company details",
"dashboard.communicateTeam": "Communicate with our team about your projects",
"dashboard.viewQuotes": "View your quotes, invoices, and payment history",
"dashboard.menu": "Menu",
"dashboard.settings": "Settings",
"dashboard.backToDashboard": "Back to Dashboard",

// Dashboard New Quote
"newQuote.title": "New Quote Request",
"newQuote.subtitle": "Submit a new project for a quote estimate",
"newQuote.projectDetails": "Project Details",
"newQuote.projectDetailsDesc": "Provide the basic information about your project",
"newQuote.projectAddress": "Project Address",
"newQuote.city": "City",
"newQuote.state": "State",
"newQuote.projectType": "Project Type",
"newQuote.selectProjectType": "Select project type",
"newQuote.estimatedSize": "Estimated Project Size (sq ft)",
"newQuote.additionalNotes": "Additional Notes (optional)",
"newQuote.additionalNotesPlaceholder": "Describe your project requirements, special requests, or any other relevant information...",
"newQuote.cancel": "Cancel",
"newQuote.submitting": "Submitting...",
"newQuote.submitRequest": "Submit Request",

// Project Card
"projectCard.briefing": "Briefing",
"projectCard.inDevelopment": "In Development",
"projectCard.underReview": "Under Review",
"projectCard.completed": "Completed",
"projectCard.cancelled": "Cancelled",
"projectCard.files": "files",
"projectCard.msgs": "msgs",
"projectCard.quotes": "quotes",

// How We Work
"howWeWork.title": "HOW WE WORK",
"howWeWork.subtitle": "Project Process & Professional Scope",
"howWeWork.ourProcess": "Our Process",
"howWeWork.ourProcessDesc": "We follow a structured and efficient workflow designed for builders, developers, and serious homeowners.",
"howWeWork.whatsIncluded": "What's Included",
"howWeWork.whatsIncludedDesc": "Our services focus on architectural design development and drawing documentation. Depending on the selected package, deliverables may inc
"howWeWork.whatsNotIncluded": "What's Not Included",
"howWeWork.whatsNotIncludedDesc": "To maintain clarity and compliance, the following services are not part of our scope unless explicitly contracted:",
"howWeWork.compliance": "Compliance & Responsibility",
"howWeWork.complianceDesc": "Our drawings are intended for design development and permit documentation support. Final approval is subject to review by local authorities
"howWeWork.clientResponsible": "The client is responsible for:",
"howWeWork.complianceNote": "We design in alignment with U.S. residential construction standards; however, final compliance depends on local jurisdiction requirements.",
"howWeWork.meetings": "Meetings & Communication",
"howWeWork.meetingsDesc": "DA•RA Studio operates remotely with a structured digital workflow.",
"howWeWork.meetingsNote": "We have proudly supported builders and investors across multiple U.S. states.",
"howWeWork.whyDara": "Why DA•RA Studio",
"howWeWork.drawingsNote": "All drawings are prepared in accordance with standard U.S. residential drafting practices.",
"howWeWork.licensedNote": "When required, licensed professionals must be hired locally by the client.",

// Process Steps
"process.step1.title": "Proposal Approval",
"process.step1.desc": "Project scope and pricing are confirmed.",
"process.step1.detail": "During this phase, we finalize all project parameters including scope of work, deliverables, timeline estimates, and pricing. You'll receive a d
"process.step2.title": "Design Preview",
"process.step2.desc": "Initial layout and visual direction are delivered for review.",
"process.step2.timeline": "8–16 Business Days",
"process.step2.note": "Timeline starts from receipt of required project information, including proposal approval, initial payment, and site or existing residence documen

```
"process.step2.detail": "Our design team develops the initial conceptual layouts based on your project requirements, site conditions, and design preferences. This includ
"process.step3.title": "Revisions Phase",
"process.step3.desc": "Two (2) revision rounds are included to refine layout and design intent.",
"process.step3.note": "Additional revisions are available upon request.",
"process.step3.detail": "Collaboration is key to a successful project. During this phase, we work closely with you to refine the design based on your feedback. Each revi
"process.step4.title": "Final Drawing Set",
"process.step4.desc": "Complete architectural drawing package is delivered in digital format.",
"process.step4.timeline": "25–30 Business Days After Approval",
"process.step4.note": "Timeline may vary depending on project complexity, requested revisions, and technical coordination during project development.",
"process.step4.detail": "Once the design preview is approved, our team develops the complete construction documentation package. This includes detailed floor plans with
"process.step5.title": "Ongoing Coordination",
"process.step5.desc": "We provide remote support and coordination with your contractor or engineer as needed.",
"process.step5.detail": "Our commitment doesn't end with drawing delivery. We remain available to answer questions from your construction team, clarify design intent, an

// Included Items
"included.1": "Architectural floor plans",
"included.2": "Exterior elevations",
"included.3": "Basic building sections",
"included.4": "Interior layout planning",
"included.5": "Kitchen and bathroom layouts",
"included.6": "Fixture and equipment placement",
"included.7": "Door and window schedules",
"included.8": "Dimensioned construction drawings (for permit submission when applicable)",
"included.9": "3D visualization (if included in selected package)",

// Not Included Items
"notIncluded.1": "Structural engineering calculations",
"notIncluded.2": "Engineer stamp or sealed drawings",
"notIncluded.3": "Load diagrams or structural reinforcement detailing",
"notIncluded.4": "Full electrical system design",
"notIncluded.5": "Full plumbing system design (we indicate fixture locations only)",
"notIncluded.6": "Mechanical/HVAC design",
"notIncluded.7": "Site surveys, plot plans, or topographic mapping",
"notIncluded.8": "Soil testing or geotechnical reports",
"notIncluded.9": "Energy compliance reports (HERS, blower door, etc.)",
"notIncluded.10": "In-person site visits or inspections",
"notIncluded.11": "Material procurement or interior décor shopping lists",

// Client Responsibilities
"responsibility.1": "Hiring licensed structural engineers (if required)",
"responsibility.2": "Coordinating required technical reports",
"responsibility.3": "Verifying local zoning and code restrictions",

// Communication Features
"communication.1": "Full online project management",
"communication.2": "Scheduled video meetings",
"communication.3": "Direct communication during business hours",
"communication.4": "Continuous updates throughout each phase",

// Why DARA
"whyDara.1": "Code-conscious documentation",
"whyDara.2": "Builder-friendly drawing sets",
"whyDara.3": "Efficient turnaround",
"whyDara.4": "Clear scope and professional boundaries",
"whyDara.5": "Trusted by developers and construction teams",

// 404
"notFound.title": "404",
"notFound.message": "Oops! Page not found",
"notFound.goHome": "Return to Home",

// Language Selector
"language.english": "English",
"language.portuguese": "Português",

// Freelancer
"freelancer.workspace": "Freelancer Workspace",
"freelancer.dashboard": "Dashboard",
"freelancer.projects": "Projects",
"freelancer.payments": "Payments",
"freelancer.contracts": "Contracts",
"freelancer.calendar": "Calendar",
"freelancer.workspaceLabel": "Workspace",
"freelancer.welcome": "Welcome",
"freelancer.overviewDesc": "Here's your workspace overview",
"freelancer.activeProjects": "Active Projects",
"freelancer.pendingPayments": "Pending Payments",
"freelancer.totalEarnings": "Total Earnings",
"freelancer.upcomingDeadlines": "Upcoming Deadlines",
"freelancer.noUpcomingDeadlines": "No upcoming deadlines",
"freelancer.myProjects": "My Projects",
"freelancer.loading": "Loading...",
"freelancer.noProjectsAssigned": "No projects assigned yet",
"freelancer.backToProjects": "← Back to Projects",
"freelancer.address": "Address",
"freelancer.status": "Status",
"freelancer.service": "Service",
"freelancer.deadline": "Deadline",
"freelancer.notes": "Notes",
"freelancer.referenceFiles": "Reference Files",
"freelancer.noFiles": "No files available",
"freelancer.submitDelivery": "Submit Delivery",
"freelancer.fileName": "File Name",
"freelancer.fileUrl": "File URL",
"freelancer.notesOptional": "Notes (optional)",
"freelancer.submitting": "Submitting...",
"freelancer.yourDeliveries": "Your Deliveries",
"freelancer.view": "View",
"freelancer.pending": "Pending",
"freelancer.totalPaid": "Total Paid",
```

```
"freelancer.paymentHistory": "Payment History",
"freelancer.noPayments": "No payments yet",
"freelancer.due": "Due",
"freelancer.noDueDate": "No due date",
"freelancer.agreedAmount": "Agreed Amount",
"freelancer.paidAmount": "Paid",
"freelancer.remainingAmount": "Balance",
"freelancer.partial": "Partial",
"freelancer.paid": "Paid",
"freelancer.totalReceivable": "Total Receivable",
"freelancer.totalAlreadyPaid": "Total Already Paid",
"freelancer.nextPayment": "Next Payment",
"freelancer.project": "Project",
"freelancer.paymentMethod": "Method",
"freelancer.noContracts": "No contracts available",
"freelancer.signed": "Signed",
"freelancer.unsigned": "Unsigned",
"freelancer.download": "Download",
"freelancer.unknownProject": "Unknown project",
"freelancer.selectDate": "Select a date",
"freelancer.noDeadlinesOnDate": "No deadlines on this date",
"freelancer.allUpcomingDeadlines": "All Upcoming Deadlines",
"freelancer.noUpcoming": "No upcoming deadlines",
"freelancer.signOut": "Sign Out",

// Freelancer Auth
"freelancerAuth.title": "Freelancer Portal",
"freelancerAuth.subtitle": "Access your workspace, projects, and payments",
"freelancerAuth.welcomeBack": "Welcome, Collaborator",
"freelancerAuth.enterCredentials": "Sign in with the credentials provided by the admin",
"freelancerAuth.signIn": "Access Workspace",
"freelancerAuth.noAccess": "You don't have freelancer access. Contact the administrator.",
"freelancerAuth.invalidCredentials": "Invalid email or password. Please try again.",
"freelancerAuth.accessNote": "Access is provided by the DA•RA Studio team. If you don't have credentials, contact your project manager.",

// Quote Form - Location Step
"form.projectAddress": "Project Address",
"form.streetAddress": "Street address",
"form.city": "City",
"form.state": "State",
"form.statePlaceholder": "e.g., CA, TX, FL",
"form.zipCode": "ZIP Code",
"form.zipPlaceholder": "e.g., 90210",

// Quote Form - Project Step
"form.projectInfo": "Project Information",
"form.projectType": "Project Type",
"form.selectOne": "Select one option",
"form.hasExistingPlans": "Do you have existing architectural plans?",
"form.yes": "Yes",
"form.no": "No",
"form.hasEngineer": "Do you already have an engineer or need a referral?",
"form.haveOne": "I already have one",
"form.needReferral": "I need a referral",
"form.buildingStatus": "Is the existing building occupied or under construction?",
"form.projectSizeLevels": "Project Size &amp; Levels",
"form.existingArea": "Existing Area (if applicable)",
"form.sqftOptional": "sq ft (optional)",
"form.proposedArea": "Proposed / Estimated Project Area",
"form.proposedAreaDesc": "Approximate size of the new or modified area",
"form.projectLevels": "Project Levels (check all that apply):",
"form.fileUploads": "File Uploads &amp; References",
"form.uploadDesc": "Upload photos, sketches, or reference files",
"form.acceptsFormats": "Accepts PDF, JPG, PNG, DWG",
"form.referenceNotesPlaceholder": "Describe your reference files or include notes about the documents you'll be sharing",
"form.referenceLinks": "Reference links (optional)",
"form.referenceLinksDesc": "Google Drive, Dropbox, etc.",
"form.referenceLinksPlaceholder": "Paste links to your files here...",
"form.programRequirements": "Program of Requirements",
"form.programRequirementsDesc": "For residential projects, including ADU when applicable",
"form.other": "Other:",
"form.otherPlaceholder": "Specify other spaces needed",
"form.servicesOffered": "Services Offered by DARA",
"form.designDrafting": "Design &amp; Drafting Services",
"form.selectServices": "Select the services you're interested in",
"form.disclaimer": "Professional Disclaimer",
"form.disclaimerText": "DARA provides design and drafting services only. All drawings are conceptual and illustrative in nature and are not intended to replace architect
"form.ifOtherDescribe": 'If "Other", please describe:',
"form.describeProjectType": "Describe your project type",

// Quote Form - Client Step
"form.iAmA": "I am a...",
"form.contactInfo": "Contact Information",
"form.fullName": "Full Name",
"form.fullNamePlaceholder": "Your full name",
"form.email": "Email",
"form.phone": "Phone Number",
"form.companyInfo": "Company Information",
"form.companyName": "Company Name",
"form.companyNamePlaceholder": "Your company name",
"form.companyAddress": "Company Address",
"form.companyAddressPlaceholder": "Full business address",
"form.instagram": "Instagram",
"form.companyPhone": "Company Phone",
"form.companyEmail": "Company Email",
"form.optional": "(optional)",

// Quote Form - Details Step
"form.desiredTimeline": "Desired Timeline",
"form.whenStart": "When would you like to start?",
"form.estimatedInvestment": "Estimated Investment for Design Services",
```

```
"form.investmentDesc": "Optional — helps us prepare the right proposal",
"form.investmentNote": "This refers to design and drafting services only (not construction costs).",
"form.howHear": "How did you hear about us?",
"form.additionalNotes": "Additional Notes",
"form.additionalNotesDesc": "Tell us any important details about your project",
"form.additionalNotesPlaceholder": "Share any special requirements, preferences, or questions...",
"form.agreement": "I agree to the processing of my information for quote purposes and accept the",
"form.termsOfUse": "Terms of Use",
"form.and": "and",
"form.privacyPolicy": "Privacy Policy",

// Client Types
"clientType.homeowner": "Homeowner",
"clientType.contractor": "General Contractor",
"clientType.architect": "Architect",
"clientType.engineer": "Engineer",
"clientType.realEstate": "Real Estate Developer",

// Project Types
"projectType.singleFamily": "Single-Family Residential",
"projectType.multiFamily": "Multi-Family Residential",
"projectType.commercial": "Commercial",
"projectType.renovation": "Renovation / Remodel",
"projectType.addition": "Addition / Extension",
"projectType.adu": "ADU (Accessory Dwelling Unit)",
"projectType.deck": "Deck",
"projectType.garage": "Garage (Attached / Detached)",
"projectType.other": "Other",

// Building Status
"buildingStatus.occupied": "Occupied",
"buildingStatus.underConstruction": "Under Construction",
"buildingStatus.vacant": "Vacant",
"buildingStatus.newConstruction": "N/A (New Construction)",

// Project Levels
"level.basement": "Basement",
"level.firstFloor": "First Floor",
"level.secondFloor": "Second Floor",
"level.atticLoft": "Attic / Loft",
"level.garageLevel": "Garage Level",

// Timeline Options
"timeline.asap": "ASAP",
"timeline.within30": "Within 30 days",
"timeline.1to3months": "1–3 months",
"timeline.3plusMonths": "3+ months",
"timeline.flexible": "Flexible",

// Budget Options
"budget.under2500": "Under $2,500",
"budget.2500to5000": "$2,500 – $5,000",
"budget.5000to10000": "$5,000 – $10,000",
"budget.10000plus": "$10,000+",
"budget.notSure": "Not sure yet",

// Referral Options
"referral.google": "Google",
"referral.referral": "Referral",
"referral.socialMedia": "Social Media",
"referral.contractor": "Contractor",
"referral.returningClient": "Returning Client",
"referral.other": "Other",

// Required Spaces
"spaces.private": "Private Spaces",
"spaces.bathrooms": "Bathrooms",
"spaces.social": "Social Areas",
"spaces.kitchen": "Kitchen &amp; Support Spaces",
"spaces.work": "Work &amp; Wellness",
"spaces.circulation": "Circulation &amp; Storage",
"spaces.technical": "Technical Areas",
"spaces.outdoor": "Outdoor Areas",

// DARA Services Categories
"daraService.architecture": "ARCHITECTURAL DESIGN &amp; DRAFTING",
"daraService.interiors": "INTERIOR DESIGN &amp; SPACE PLANNING",
"daraService.visualization": "3D VISUALIZATION",
"daraService.documentation": "TECHNICAL DRAFTING &amp; DOCUMENTATION",
"daraService.construction": "CONSTRUCTION COORDINATION SUPPORT",
"daraService.additional": "ADDITIONAL DESIGN SERVICES",

// Admin Panel
"admin.dashboard": "Dashboard",
"admin.studioOverview": "Studio overview",
"admin.projects": "Projects",
"admin.manageProjects": "Manage all studio projects",
"admin.companies": "Companies",
"admin.payments": "Payments",
"admin.milestones": "Milestones",
"admin.activity": "Activity",
"admin.map": "Map",
"admin.signOut": "Sign Out",
"admin.newProject": "New Project",
"admin.editProject": "Edit Project",
"admin.company": "Company",
"admin.select": "Select",
"admin.projectStage": "Project Stage",
"admin.paymentStage": "Payment Stage",
"admin.package": "Package",
```

"admin.none": "None",
"admin.services": "Services",
"admin.projectAddress": "Project Address",
"admin.copyFromCompany": "Copy from Company",
"admin.street": "Street",
"admin.number": "Number",
"admin.unit": "Unit",
"admin.neighborhood": "Neighborhood",
"admin.city": "City",
"admin.state": "State",
"admin.zip": "Zip",
"admin.countryLabel": "Country",
"admin.dates": "Dates",
"admin.startDate": "Start Date",
"admin.deliveryDate": "Delivery Date",
"admin.paymentEffective": "Payment Effective",
"admin.previewDelivery": "Preview Delivery",
"admin.previewSent": "Preview Sent",
"admin.revisionReturn": "Revision Return",
"admin.finalDelivery": "Final Delivery",
"admin.revisionCount": "Revision Count",
"admin.financials": "Financials",
"admin.currency": "Currency",
"admin.totalValue": "Total Value",
"admin.entryAmount": "Entry Amount",
"admin.finalAmount": "Final Amount",
"admin.save": "Save",
"admin.create": "Create",
"admin.search": "Search...",
"admin.stage": "Stage",
"admin.allStages": "All Stages",
"admin.allCompanies": "All Companies",
"admin.address": "Address",
"admin.delivery": "Delivery",
"admin.value": "Value",
"admin.paid": "Paid",
"admin.balance": "Balance",
"admin.noProjects": "No projects found",
"admin.noCompany": "No company",
"admin.selectCompanyFirst": "Select a company first",
"admin.addressCopied": "Address copied from company",
"admin.projectUpdated": "Project updated",
"admin.projectCreated": "Project created",
"admin.delayedProjects": "Delayed Projects",
"admin.waitingPayment": "Waiting Payment",
"admin.overduePayment": "Overdue Payment",
"admin.inProgress": "In Progress",
"admin.cancelled": "Cancelled",
"admin.pendingRevenue": "Pending Revenue",
"admin.receivedThisMonth": "Received This Month",
"admin.mapView": "Map View",
"admin.loadingMap": "Loading map...",
"admin.noProjectsWithLocation": "No projects with location data. Add latitude and longitude to your projects to see them on the map.",
"admin.projectsWithLocation": "projects with location data",
"admin.noAddress": "No address",
"admin.financialControl": "Financial control of projects",
"admin.newPayment": "New Payment",
"admin.registerPayment": "Register Payment",
"admin.project": "Project",
"admin.selectProject": "Select a project",
"admin.type": "Type",
"admin.method": "Method",
"admin.amountTotal": "Total Amount",
"admin.amountPaid": "Amount Paid",
"admin.stripeFee": "Stripe Fee",
"admin.status": "Status",
"admin.paymentDate": "Payment Date",
"admin.register": "Register",
"admin.paymentRegistered": "Payment registered",
"admin.totalReceived": "Total Received",
"admin.pendingLabel": "Pending",
"admin.records": "Records",
"admin.total": "Total",
"admin.net": "Net",
"admin.date": "Date",
"admin.noPayments": "No payments registered",
"admin.paidStatus": "Paid",
"admin.partialStatus": "Partial",
"admin.pendingStatus": "Pending",
"admin.restrictedAccess": "Restricted Access",
"admin.noPermission": "You do not have permission to access this area.",
"admin.backToHome": "Back to home",
"admin.allCities": "All Cities",
"admin.previewSentDate": "Preview Sent",
"admin.projectFinancials": "Project Financial Summary",
"admin.paymentRecords": "Payment Records",

// Profile
"profile.title": "My Profile",
"profile.subtitle": "Manage your personal information",
"profile.photo": "Profile Photo",
"profile.noName": "No name set",
"profile.changePhoto": "Change photo",
"profile.uploading": "Uploading...",
"profile.personalInfo": "Personal Information",
"profile.fullName": "Full Name",
"profile.fullNamePlaceholder": "Your full name",
"profile.email": "Email",
"profile.emailPlaceholder": "your@email.com",
"profile.phone": "Phone",
"profile.phonePlaceholder": "+1 (555) 000-0000",

```
    "profile.save": "Save Changes",
    "profile.saving": "Saving...",
    "profile.saved": "Profile updated",
    "profile.savedDesc": "Your information has been saved successfully.",
    "profile.errorSaving": "Error saving profile",
    "profile.errorUpload": "Error uploading image",
    "profile.avatarUpdated": "Photo updated successfully",
  },
  pt: {
    // Header
    "nav.howWeWork": "Como Trabalhamos",
    "nav.portfolio": "Portfólio",
    "nav.requestQuote": "Solicitar Orçamento",
    "nav.contact": "Contato",
    "nav.clientArea": "Área do Cliente",

    // Hero Section
    "hero.badge": "Excelência em Projetos Arquitetônicos",
    "hero.title": "Transformamos suas ideias em",
    "hero.titleHighlight": "projetos excepcionais",
    "hero.subtitle": "Especialistas em desenho técnico, projetos executivos e estruturas wood frame. Da concepção ao detalhamento final.",
    "hero.cta.quote": "Solicitar Orçamento",
    "hero.cta.portfolio": "Ver Portfólio",
    "hero.stats.projects": "Projetos Entregues",
    "hero.stats.experience": "Anos de Experiência",
    "hero.stats.clients": "Clientes Satisfeitos",

    // Services Section
    "services.title": "Soluções Arquitetônicas Orientadas por Design",
    "services.subtitle": "Entregamos soluções arquitetônicas orientadas por design, desde o desenvolvimento de conceitos até documentação pronta para construção, alinhadas c
    "services.learnMore": "Saiba mais",
    "services.cta": "Solicitar Orçamento Gratuito",

    // Portfolio Section
    "portfolio.title": "Projetos Selecionados",
    "portfolio.subtitle": "Uma seleção curada de projetos residenciais e comerciais desenvolvidos para construtores, investidores e clientes particulares nos EUA.",
    "portfolio.badge": "Projetado para atender aos códigos de construção e padrões americanos.",
    "portfolio.viewProject": "Ver Projeto",
    "portfolio.viewAll": "Ver todos os projetos",
    "portfolio.item1.title": "Residência Unifamiliar Moderna",
    "portfolio.item1.category": "Arquitetura Residencial",
    "portfolio.item2.title": "Construção Residencial Wood Frame",
    "portfolio.item2.category": "Layout Wood Frame",
    "portfolio.item3.title": "Edifício Comercial",
    "portfolio.item3.category": "Design Comercial",

    // CTA Section
    "cta.title": "Vamos Construir Algo Excepcional",
    "cta.subtitle": "Conte-nos sobre seu projeto. Vamos guiá-lo pelos próximos passos com clareza, estratégia e precisão.",
    "cta.proposal": "Solicitar Proposta",
    "cta.call": "Agendar Ligação",

    // Footer
    "footer.tagline": "Transformando ideias em projetos excepcionais com precisão e excelência.",
    "footer.quickLinks": "Links Rápidos",
    "footer.services": "Serviços",
    "footer.contact": "Contato",
    "footer.rights": "Todos os direitos reservados.",
    "footer.service1": "Desenho Técnico",
    "footer.service2": "Projeto Executivo",
    "footer.service3": "Estruturas Wood Frame",
    "footer.service4": "Orçamento de Projetos",

    // Auth
    "auth.title": "Portal do Cliente",
    "auth.subtitle": "Acesse seus projetos, arquivos e comunicações",
    "auth.signIn": "Entrar",
    "auth.createAccount": "Criar Conta",
    "auth.welcomeBack": "Bem-vindo de Volta",
    "auth.enterCredentials": "Insira suas credenciais para acessar sua conta",
    "auth.register": "Registre-se para acessar o portal do seu projeto",
    "auth.email": "Email",
    "auth.password": "Senha",
    "auth.fullName": "Nome Completo",
    "auth.confirmPassword": "Confirmar Senha",
    "auth.signingIn": "Entrando...",
    "auth.creatingAccount": "Criando conta...",
    "auth.forgotPassword": "Esqueceu sua senha?",
    "auth.resetPassword": "Redefinir Senha",
    "auth.resetInstructions": "Insira seu email e enviaremos um link de redefinição",
    "auth.sendResetLink": "Enviar Link",
    "auth.sending": "Enviando...",
    "auth.backToLogin": "Voltar ao Login",
    "auth.checkEmail": "Verifique seu email para o link de redefinição",

    // Quote Form
    "quote.title": "Solicitar Orçamento",
    "quote.subtitle": "Preencha o formulário abaixo para receber uma proposta personalizada",
    "quote.step.location": "Localização",
    "quote.step.project": "Projeto",
    "quote.step.client": "Seus Dados",
    "quote.step.details": "Detalhes",
    "quote.locationDesc": "Onde será o projeto?",
    "quote.projectDesc": "Conte-nos sobre seu projeto",
    "quote.clientDesc": "Suas informações de contato",
    "quote.detailsDesc": "Informações adicionais (opcional)",
    "quote.back": "Voltar",
    "quote.next": "Próximo",
    "quote.submit": "Enviar Solicitação",
    "quote.submitting": "Enviando...",
```

```
// Thank You Page
"thankyou.title": "Projeto Recebido",
"thankyou.message": "Obrigado pelo seu envio.",
"thankyou.projectCreated": "Seu projeto foi criado como:",
"thankyou.proposalTime": "Você receberá uma proposta detalhada em 2-3 dias úteis.",
"thankyou.portalAccess": "Enviamos um email com instruções para acessar seu Portal do Cliente.",
"thankyou.goHome": "Voltar ao Início",
"thankyou.goPortal": "Ir ao Portal do Cliente",

// Dashboard
"dashboard.welcome": "Bem-vindo",
"dashboard.welcomeBack": "Bem-vindo de volta",
"dashboard.overview": "Aqui está uma visão geral dos seus projetos e atividades",
"dashboard.home": "Início",
"dashboard.projects": "Projetos",
"dashboard.quotes": "Orçamentos",
"dashboard.messages": "Mensagens",
"dashboard.profile": "Perfil",
"dashboard.company": "Empresa",
"dashboard.newQuote": "Novo Orçamento",
"dashboard.signOut": "Sair",
"dashboard.noProjects": "Nenhum projeto ainda",
"dashboard.noProjectsDesc": "Comece solicitando um orçamento para o seu primeiro projeto",
"dashboard.openProject": "Abrir Projeto",
"dashboard.activeProjects": "Projetos Ativos",
"dashboard.quotesInvoices": "Orçamentos e Faturas",
"dashboard.newQuoteRequest": "Nova Solicitação de Orçamento",
"dashboard.yourProjects": "Seus Projetos",
"dashboard.requestQuote": "Solicitar Orçamento",
"dashboard.myProjects": "Meus Projetos",
"dashboard.viewManageProjects": "Visualize e gerencie todos os seus projetos",
"dashboard.comingSoon": "Em Breve",
"dashboard.comingSoonDesc": "Esta seção está em desenvolvimento. Volte em breve!",
"dashboard.myProfile": "Meu Perfil",
"dashboard.manageProfile": "Gerencie suas informações pessoais",
"dashboard.companyInfo": "Informações da Empresa",
"dashboard.manageCompany": "Gerencie os detalhes da sua empresa",
"dashboard.communicateTeam": "Comunique-se com nossa equipe sobre seus projetos",
"dashboard.viewQuotes": "Veja seus orçamentos, faturas e histórico de pagamentos",
"dashboard.menu": "Menu",
"dashboard.settings": "Configurações",
"dashboard.backToDashboard": "Voltar ao Painel",

// Dashboard New Quote
"newQuote.title": "Nova Solicitação de Orçamento",
"newQuote.subtitle": "Envie um novo projeto para estimativa de orçamento",
"newQuote.projectDetails": "Detalhes do Projeto",
"newQuote.projectDetailsDesc": "Forneça as informações básicas sobre seu projeto",
"newQuote.projectAddress": "Endereço do Projeto",
"newQuote.city": "Cidade",
"newQuote.state": "Estado",
"newQuote.projectType": "Tipo de Projeto",
"newQuote.selectProjectType": "Selecione o tipo de projeto",
"newQuote.estimatedSize": "Tamanho Estimado do Projeto (sq ft)",
"newQuote.additionalNotes": "Notas Adicionais (opcional)",
"newQuote.additionalNotesPlaceholder": "Descreva os requisitos do seu projeto, solicitações especiais ou qualquer outra informação relevante...",
"newQuote.cancel": "Cancelar",
"newQuote.submitting": "Enviando...",
"newQuote.submitRequest": "Enviar Solicitação",

// Project Card
"projectCard.briefing": "Briefing",
"projectCard.inDevelopment": "Em Desenvolvimento",
"projectCard.underReview": "Em Revisão",
"projectCard.completed": "Concluído",
"projectCard.cancelled": "Cancelado",
"projectCard.files": "arquivos",
"projectCard.msgs": "msgs",
"projectCard.quotes": "orçamentos",

// How We Work
"howWeWork.title": "COMO TRABALHAMOS",
"howWeWork.subtitle": "Processo do Projeto e Escopo Profissional",
"howWeWork.ourProcess": "Nosso Processo",
"howWeWork.ourProcessDesc": "Seguimos um fluxo de trabalho estruturado e eficiente projetado para construtores, incorporadores e proprietários sérios.",
"howWeWork.whatsIncluded": "O que está Incluído",
"howWeWork.whatsIncludedDesc": "Nossos serviços focam no desenvolvimento de design arquitetônico e documentação de desenhos. Dependendo do pacote selecionado, os entregá
"howWeWork.whatsNotIncluded": "O que NÃO está Incluído",
"howWeWork.whatsNotIncludedDesc": "Para manter clareza e conformidade, os seguintes serviços não fazem parte do nosso escopo, a menos que explicitamente contratados:",
"howWeWork.compliance": "Conformidade e Responsabilidade",
"howWeWork.complianceDesc": "Nossos desenhos são destinados ao desenvolvimento de design e suporte à documentação de permissões. A aprovação final está sujeita à revisão
"howWeWork.clientResponsible": "O cliente é responsável por:",
"howWeWork.complianceNote": "Projetamos em conformidade com os padrões de construção residencial dos EUA; no entanto, a conformidade final depende dos requisitos da juri
"howWeWork.meetings": "Reuniões e Comunicação",
"howWeWork.meetingsDesc": "O DA•RA Studio opera remotamente com um fluxo de trabalho digital estruturado.",
"howWeWork.meetingsNote": "Temos orgulho de apoiar construtores e investidores em vários estados dos EUA.",
"howWeWork.whyDara": "Por que DA•RA Studio",
"howWeWork.drawingsNote": "Todos os desenhos são preparados de acordo com práticas padrão de desenho residencial dos EUA.",
"howWeWork.licensedNote": "Quando necessário, profissionais licenciados devem ser contratados localmente pelo cliente.",

// Process Steps
"process.step1.title": "Aprovação da Proposta",
"process.step1.desc": "O escopo e o preço do projeto são confirmados.",
"process.step1.detail": "Durante esta fase, finalizamos todos os parâmetros do projeto, incluindo escopo de trabalho, entregáveis, estimativas de prazo e preços. Você re
"process.step2.title": "Prévia do Design",
"process.step2.desc": "O layout inicial e a direção visual são entregues para revisão.",
"process.step2.timeline": "8–16 Dias Úteis",
"process.step2.note": "O prazo começa a contar a partir do recebimento das informações necessárias do projeto, incluindo aprovação da proposta, pagamento inicial e docum
"process.step2.detail": "Nossa equipe de design desenvolve os layouts conceituais iniciais com base nos requisitos do seu projeto, condições do terreno e preferências de
"process.step3.title": "Fase de Revisões",
"process.step3.desc": "Duas (2) rodadas de revisão estão incluídas para refinar o layout e a intenção do design.",
```

```
"process.step3.note": "Revisões adicionais estão disponíveis mediante solicitação.",
"process.step3.detail": "A colaboração é a chave para um projeto bem-sucedido. Durante esta fase, trabalhamos em estreita colaboração com você para refinar o design com
"process.step4.title": "Conjunto Final de Desenhos",
"process.step4.desc": "O pacote completo de desenhos arquitetônicos é entregue em formato digital.",
"process.step4.timeline": "25–30 Dias Úteis Após Aprovação",
"process.step4.note": "O prazo pode variar dependendo da complexidade do projeto, revisões solicitadas e coordenação técnica durante o desenvolvimento do projeto.",
"process.step4.detail": "Após a aprovação da prévia do design, nossa equipe desenvolve o pacote completo de documentação de construção. Isso inclui plantas baixas detalh
"process.step5.title": "Coordenação Contínua",
"process.step5.desc": "Fornecemos suporte remoto e coordenação com seu empreiteiro ou engenheiro conforme necessário.",
"process.step5.detail": "Nosso compromisso não termina com a entrega dos desenhos. Permanecemos disponíveis para responder perguntas de sua equipe de construção, esclare

// Included Items
"included.1": "Plantas arquitetônicas",
"included.2": "Elevações exteriores",
"included.3": "Seções básicas do edifício",
"included.4": "Planejamento de layout interior",
"included.5": "Layouts de cozinha e banheiro",
"included.6": "Posicionamento de equipamentos e acessórios",
"included.7": "Cronogramas de portas e janelas",
"included.8": "Desenhos de construção dimensionados (para submissão de permissão quando aplicável)",
"included.9": "Visualização 3D (se incluída no pacote selecionado)",

// Not Included Items
"notIncluded.1": "Cálculos de engenharia estrutural",
"notIncluded.2": "Carimbo de engenheiro ou desenhos selados",
"notIncluded.3": "Diagramas de carga ou detalhamento de reforço estrutural",
"notIncluded.4": "Projeto completo de sistema elétrico",
"notIncluded.5": "Projeto completo de sistema hidráulico (indicamos apenas localizações de acessórios)",
"notIncluded.6": "Projeto mecânico/HVAC",
"notIncluded.7": "Levantamentos topográficos, plantas de lote ou mapeamento topográfico",
"notIncluded.8": "Testes de solo ou relatórios geotécnicos",
"notIncluded.9": "Relatórios de conformidade energética (HERS, blower door, etc.)",
"notIncluded.10": "Visitas presenciais ao local ou inspeções",
"notIncluded.11": "Aquisição de materiais ou listas de compras de decoração",

// Client Responsibilities
"responsibility.1": "Contratar engenheiros estruturais licenciados (se necessário)",
"responsibility.2": "Coordenar relatórios técnicos necessários",
"responsibility.3": "Verificar restrições locais de zoneamento e código",

// Communication Features
"communication.1": "Gerenciamento de projetos totalmente online",
"communication.2": "Reuniões por vídeo agendadas",
"communication.3": "Comunicação direta durante o horário comercial",
"communication.4": "Atualizações contínuas em cada fase",

// Why DARA
"whyDara.1": "Documentação consciente de códigos",
"whyDara.2": "Conjuntos de desenhos amigáveis para construtores",
"whyDara.3": "Prazo de entrega eficiente",
"whyDara.4": "Escopo claro e limites profissionais",
"whyDara.5": "Confiado por incorporadores e equipes de construção",

// 404
"notFound.title": "404",
"notFound.message": "Ops! Página não encontrada",
"notFound.goHome": "Voltar ao Início",

// Language Selector
"language.english": "English",
"language.portuguese": "Português",

// Freelancer
"freelancer.workspace": "Área do Freelancer",
"freelancer.dashboard": "Painel",
"freelancer.projects": "Projetos",
"freelancer.payments": "Pagamentos",
"freelancer.contracts": "Contratos",
"freelancer.calendar": "Calendário",
"freelancer.workspaceLabel": "Workspace",
"freelancer.welcome": "Bem-vindo",
"freelancer.overviewDesc": "Aqui está a visão geral do seu workspace",
"freelancer.activeProjects": "Projetos Ativos",
"freelancer.pendingPayments": "Pagamentos Pendentes",
"freelancer.totalEarnings": "Total Recebido",
"freelancer.upcomingDeadlines": "Próximos Prazos",
"freelancer.noUpcomingDeadlines": "Nenhum prazo próximo",
"freelancer.myProjects": "Meus Projetos",
"freelancer.loading": "Carregando...",
"freelancer.noProjectsAssigned": "Nenhum projeto atribuído ainda",
"freelancer.backToProjects": "← Voltar aos Projetos",
"freelancer.address": "Endereço",
"freelancer.status": "Status",
"freelancer.service": "Serviço",
"freelancer.deadline": "Prazo",
"freelancer.notes": "Notas",
"freelancer.referenceFiles": "Arquivos de Referência",
"freelancer.noFiles": "Nenhum arquivo disponível",
"freelancer.submitDelivery": "Enviar Entrega",
"freelancer.fileName": "Nome do Arquivo",
"freelancer.fileUrl": "URL do Arquivo",
"freelancer.notesOptional": "Notas (opcional)",
"freelancer.submitting": "Enviando...",
"freelancer.yourDeliveries": "Suas Entregas",
"freelancer.view": "Ver",
"freelancer.pending": "Pendente",
"freelancer.totalPaid": "Total Pago",
"freelancer.paymentHistory": "Histórico de Pagamentos",
"freelancer.noPayments": "Nenhum pagamento ainda",
"freelancer.due": "Vencimento",
```

```
"freelancer.noDueDate": "Sem data de vencimento",
"freelancer.agreedAmount": "Valor Acordado",
"freelancer.paidAmount": "Pago",
"freelancer.remainingAmount": "Saldo",
"freelancer.partial": "Parcial",
"freelancer.paid": "Pago",
"freelancer.totalReceivable": "Total A Receber",
"freelancer.totalAlreadyPaid": "Total Já Pago",
"freelancer.nextPayment": "Próximo Pagamento",
"freelancer.project": "Projeto",
"freelancer.paymentMethod": "Método",
"freelancer.noContracts": "Nenhum contrato disponível",
"freelancer.signed": "Assinado",
"freelancer.unsigned": "Não Assinado",
"freelancer.download": "Baixar",
"freelancer.unknownProject": "Projeto desconhecido",
"freelancer.selectDate": "Selecione uma data",
"freelancer.noDeadlinesOnDate": "Nenhum prazo nesta data",
"freelancer.allUpcomingDeadlines": "Todos os Próximos Prazos",
"freelancer.noUpcoming": "Nenhum prazo próximo",
"freelancer.signOut": "Sair",

// Freelancer Auth
"freelancerAuth.title": "Portal do Freelancer",
"freelancerAuth.subtitle": "Acesse seu workspace, projetos e pagamentos",
"freelancerAuth.welcomeBack": "Bem-vindo, Colaborador",
"freelancerAuth.enterCredentials": "Entre com as credenciais fornecidas pelo administrador",
"freelancerAuth.signIn": "Acessar Workspace",
"freelancerAuth.noAccess": "Você não tem acesso como freelancer. Contate o administrador.",
"freelancerAuth.invalidCredentials": "Email ou senha inválidos. Tente novamente.",
"freelancerAuth.accessNote": "O acesso é fornecido pela equipe DA•RA Studio. Se você não tem credenciais, contate seu gerente de projeto.",

// Quote Form - Location Step
"form.projectAddress": "Endereço do Projeto",
"form.streetAddress": "Endereço",
"form.city": "Cidade",
"form.state": "Estado",
"form.statePlaceholder": "ex.: SP, RJ, MG",
"form.zipCode": "CEP",
"form.zipPlaceholder": "ex.: 01310-100",

// Quote Form - Project Step
"form.projectInfo": "Informações do Projeto",
"form.projectType": "Tipo de Projeto",
"form.selectOne": "Selecione uma opção",
"form.hasExistingPlans": "Você possui plantas arquitetônicas existentes?",
"form.yes": "Sim",
"form.no": "Não",
"form.hasEngineer": "Você já tem um engenheiro ou precisa de uma indicação?",
"form.haveOne": "Já tenho um",
"form.needReferral": "Preciso de uma indicação",
"form.buildingStatus": "O edifício existente está ocupado ou em construção?",
"form.projectSizeLevels": "Tamanho e Níveis do Projeto",
"form.existingArea": "Área Existente (se aplicável)",
"form.sqftOptional": "m² (opcional)",
"form.proposedArea": "Área do Projeto Proposta / Estimada",
"form.proposedAreaDesc": "Tamanho aproximado da área nova ou modificada",
"form.projectLevels": "Níveis do Projeto (marque todos os que se aplicam):",
"form.fileUploads": "Upload de Arquivos e Referências",
"form.uploadDesc": "Envie fotos, esboços ou arquivos de referência",
"form.acceptsFormats": "Aceita PDF, JPG, PNG, DWG",
"form.referenceNotesPlaceholder": "Descreva seus arquivos de referência ou inclua notas sobre os documentos que você compartilhará",
"form.referenceLinks": "Links de referência (opcional)",
"form.referenceLinksDesc": "Google Drive, Dropbox, etc.",
"form.referenceLinksPlaceholder": "Cole links para seus arquivos aqui...",
"form.programRequirements": "Programa de Necessidades",
"form.programRequirementsDesc": "Para projetos residenciais, incluindo ADU quando aplicável",
"form.other": "Outro:",
"form.otherPlaceholder": "Especifique outros espaços necessários",
"form.servicesOffered": "Serviços Oferecidos pela DARA",
"form.designDrafting": "Serviços de Design e Desenho",
"form.selectServices": "Selecione os serviços de seu interesse",
"form.disclaimer": "Aviso Profissional",
"form.disclaimerText": "A DARA fornece apenas serviços de design e desenho. Todos os desenhos são de natureza conceitual e ilustrativa e não se destinam a substituir doc
"form.ifOtherDescribe": 'Se "Outro", por favor descreva:',
"form.describeProjectType": "Descreva o tipo do seu projeto",

// Quote Form - Client Step
"form.iAmA": "Eu sou...",
"form.contactInfo": "Informações de Contato",
"form.fullName": "Nome Completo",
"form.fullNamePlaceholder": "Seu nome completo",
"form.email": "Email",
"form.phone": "Telefone",
"form.companyInfo": "Informações da Empresa",
"form.companyName": "Nome da Empresa",
"form.companyNamePlaceholder": "Nome da sua empresa",
"form.companyAddress": "Endereço da Empresa",
"form.companyAddressPlaceholder": "Endereço comercial completo",
"form.instagram": "Instagram",
"form.companyPhone": "Telefone da Empresa",
"form.companyEmail": "Email da Empresa",
"form.optional": "(opcional)",

// Quote Form - Details Step
"form.desiredTimeline": "Prazo Desejado",
"form.whenStart": "Quando você gostaria de começar?",
"form.estimatedInvestment": "Investimento Estimado para Serviços de Design",
"form.investmentDesc": "Opcional — nos ajuda a preparar a proposta certa",
"form.investmentNote": "Refere-se apenas a serviços de design e desenho (não custos de construção).",
"form.howHear": "Como você nos encontrou?",
```

```
"form.additionalNotes": "Notas Adicionais",
"form.additionalNotesDesc": "Conte-nos detalhes importantes sobre seu projeto",
"form.additionalNotesPlaceholder": "Compartilhe requisitos especiais, preferências ou perguntas...",
"form.agreement": "Concordo com o processamento das minhas informações para fins de orçamento e aceito os",
"form.termsOfUse": "Termos de Uso",
"form.and": "e",
"form.privacyPolicy": "Política de Privacidade",

// Client Types
"clientType.homeowner": "Proprietário",
"clientType.contractor": "Empreiteiro Geral",
"clientType.architect": "Arquiteto",
"clientType.engineer": "Engenheiro",
"clientType.realEstate": "Incorporador Imobiliário",

// Project Types
"projectType.singleFamily": "Residencial Unifamiliar",
"projectType.multiFamily": "Residencial Multifamiliar",
"projectType.commercial": "Comercial",
"projectType.renovation": "Renovação / Reforma",
"projectType.addition": "Adição / Extensão",
"projectType.adu": "ADU (Unidade Habitacional Acessória)",
"projectType.deck": "Deck",
"projectType.garage": "Garagem (Acoplada / Separada)",
"projectType.other": "Outro",

// Building Status
"buildingStatus.occupied": "Ocupado",
"buildingStatus.underConstruction": "Em Construção",
"buildingStatus.vacant": "Vago",
"buildingStatus.newConstruction": "N/A (Construção Nova)",

// Project Levels
"level.basement": "Subsolo",
"level.firstFloor": "Primeiro Andar",
"level.secondFloor": "Segundo Andar",
"level.atticLoft": "Sótão / Loft",
"level.garageLevel": "Nível da Garagem",

// Timeline Options
"timeline.asap": "O mais rápido possível",
"timeline.within30": "Em até 30 dias",
"timeline.1to3months": "1–3 meses",
"timeline.3plusMonths": "3+ meses",
"timeline.flexible": "Flexível",

// Budget Options
"budget.under2500": "Abaixo de $2.500",
"budget.2500to5000": "$2.500 – $5.000",
"budget.5000to10000": "$5.000 – $10.000",
"budget.10000plus": "$10.000+",
"budget.notSure": "Ainda não tenho certeza",

// Referral Options
"referral.google": "Google",
"referral.referral": "Indicação",
"referral.socialMedia": "Redes Sociais",
"referral.contractor": "Empreiteiro",
"referral.returningClient": "Cliente Retornante",
"referral.other": "Outro",

// Required Spaces
"spaces.private": "Espaços Privativos",
"spaces.bathrooms": "Banheiros",
"spaces.social": "Áreas Sociais",
"spaces.kitchen": "Cozinha e Áreas de Apoio",
"spaces.work": "Trabalho e Bem-estar",
"spaces.circulation": "Circulação e Armazenamento",
"spaces.technical": "Áreas Técnicas",
"spaces.outdoor": "Áreas Externas",

// DARA Services Categories
"daraService.architecture": "DESIGN ARQUITETÔNICO E DESENHO",
"daraService.interiors": "DESIGN DE INTERIORES E PLANEJAMENTO ESPACIAL",
"daraService.visualization": "VISUALIZAÇÃO 3D",
"daraService.documentation": "DESENHO TÉCNICO E DOCUMENTAÇÃO",
"daraService.construction": "SUPORTE À COORDENAÇÃO DE CONSTRUÇÃO",
"daraService.additional": "SERVIÇOS ADICIONAIS DE DESIGN",

// Admin Panel
"admin.dashboard": "Painel",
"admin.studioOverview": "Visão geral do estúdio",
"admin.projects": "Projetos",
"admin.manageProjects": "Gerenciar todos os projetos do estúdio",
"admin.companies": "Empresas",
"admin.payments": "Pagamentos",
"admin.milestones": "Marcos",
"admin.activity": "Atividade",
"admin.map": "Mapa",
"admin.signOut": "Sair",
"admin.newProject": "Novo Projeto",
"admin.editProject": "Editar Projeto",
"admin.company": "Empresa",
"admin.select": "Selecione",
"admin.projectStage": "Etapa do Projeto",
"admin.paymentStage": "Etapa do Pagamento",
"admin.package": "Pacote",
"admin.none": "Nenhum",
"admin.services": "Serviços",
"admin.projectAddress": "Endereço do Projeto",
```

"admin.copyFromCompany": "Copiar da Empresa",
"admin.street": "Rua",
"admin.number": "Número",
"admin.unit": "Unidade",
"admin.neighborhood": "Bairro",
"admin.city": "Cidade",
"admin.state": "Estado",
"admin.zip": "CEP",
"admin.countryLabel": "País",
"admin.dates": "Datas",
"admin.startDate": "Data de Início",
"admin.deliveryDate": "Data de Entrega",
"admin.paymentEffective": "Pagamento Efetivo",
"admin.previewDelivery": "Entrega da Prévia",
"admin.previewSent": "Prévia Enviada",
"admin.revisionReturn": "Retorno da Revisão",
"admin.finalDelivery": "Entrega Final",
"admin.revisionCount": "Contagem de Revisões",
"admin.financials": "Financeiro",
"admin.currency": "Moeda",
"admin.totalValue": "Valor Total",
"admin.entryAmount": "Valor da Entrada",
"admin.finalAmount": "Valor Final",
"admin.save": "Salvar",
"admin.create": "Criar",
"admin.search": "Buscar...",
"admin.stage": "Etapa",
"admin.allStages": "Todas as Etapas",
"admin.allCompanies": "Todas as Empresas",
"admin.address": "Endereço",
"admin.delivery": "Entrega",
"admin.value": "Valor",
"admin.paid": "Pago",
"admin.balance": "Saldo",
"admin.noProjects": "Nenhum projeto encontrado",
"admin.noCompany": "Sem empresa",
"admin.selectCompanyFirst": "Selecione uma empresa primeiro",
"admin.addressCopied": "Endereço copiado da empresa",
"admin.projectUpdated": "Projeto atualizado",
"admin.projectCreated": "Projeto criado",
"admin.delayedProjects": "Projetos Atrasados",
"admin.waitingPayment": "Aguardando Pagamento",
"admin.overduePayment": "Pagamento em Atraso",
"admin.inProgress": "Em Andamento",
"admin.cancelled": "Cancelados",
"admin.pendingRevenue": "Receita Pendente",
"admin.receivedThisMonth": "Recebido Este Mês",
"admin.mapView": "Visualização do Mapa",
"admin.loadingMap": "Carregando mapa...",
"admin.noProjectsWithLocation": "Nenhum projeto com dados de localização. Adicione latitude e longitude aos seus projetos para vê-los no mapa.",
"admin.projectsWithLocation": "projetos com dados de localização",
"admin.noAddress": "Sem endereço",
"admin.financialControl": "Controle financeiro dos projetos",
"admin.newPayment": "Novo Pagamento",
"admin.registerPayment": "Registrar Pagamento",
"admin.project": "Projeto",
"admin.selectProject": "Selecione um projeto",
"admin.type": "Tipo",
"admin.method": "Método",
"admin.amountTotal": "Valor Total",
"admin.amountPaid": "Valor Pago",
"admin.stripeFee": "Taxa Stripe",
"admin.status": "Status",
"admin.paymentDate": "Data do Pagamento",
"admin.register": "Registrar",
"admin.paymentRegistered": "Pagamento registrado",
"admin.totalReceived": "Total Recebido",
"admin.pendingLabel": "Pendente",
"admin.records": "Registros",
"admin.total": "Total",
"admin.net": "Líquido",
"admin.date": "Data",
"admin.noPayments": "Nenhum pagamento registrado",
"admin.paidStatus": "Pago",
"admin.partialStatus": "Parcial",
"admin.pendingStatus": "Pendente",
"admin.restrictedAccess": "Acesso Restrito",
"admin.noPermission": "Você não tem permissão para acessar esta área.",
"admin.backToHome": "Voltar ao início",
"admin.allCities": "Todas as Cidades",
"admin.previewSentDate": "Prévia Enviada",
"admin.projectFinancials": "Resumo Financeiro dos Projetos",
"admin.paymentRecords": "Registros de Pagamento",

// Profile
"profile.title": "Meu Perfil",
"profile.subtitle": "Gerencie suas informações pessoais",
"profile.photo": "Foto de Perfil",
"profile.noName": "Nome não definido",
"profile.changePhoto": "Alterar foto",
"profile.uploading": "Enviando...",
"profile.personalInfo": "Informações Pessoais",
"profile.fullName": "Nome Completo",
"profile.fullNamePlaceholder": "Seu nome completo",
"profile.email": "Email",
"profile.emailPlaceholder": "seu@email.com",
"profile.phone": "Telefone",
"profile.phonePlaceholder": "+55 (11) 99999-0000",
"profile.save": "Salvar Alterações",
"profile.saving": "Salvando...",
"profile.saved": "Perfil atualizado",

```
    "profile.savedDesc": "Suas informações foram salvas com sucesso.",
    "profile.errorSaving": "Erro ao salvar perfil",
    "profile.errorUpload": "Erro ao enviar imagem",
    "profile.avatarUpdated": "Foto atualizada com sucesso",
  },
};
```

src/modules/clients

src/modules/clients/index.ts

```
// Clients module
export {};
```

src/modules/estimates

src/modules/estimates/index.ts

```
// Estimates module
export {};
```

src/modules/freelancers

src/modules/freelancers/index.ts

```
// Freelancers module
export {};
```

src/modules/payments

src/modules/payments/index.ts

```
// Payments module
export {};
```

src/modules/projects

src/modules/projects/index.ts

```
// Projects module
export {};
```

src/modules/proposals

src/modules/proposals/index.ts

```
// Proposals module
export {};
```

src/services/estimateService.ts

src/services/estimateService.ts

```
// src/services/estimateService.ts

export const calculateEstimate = (
  area: number,
  rate: number
) =&gt; {
  return area * rate;
};
```

src/services/index.ts

src/services/index.ts

```
// Centralized Supabase services
export { supabase } from "./supabase/client";
export * as authService from "./supabase/authService";
export * as projectService from "./supabase/projectService";
export * as estimateService from "./supabase/estimateService";
export * as paymentService from "./supabase/paymentService";
export * as freelancerService from "./supabase/freelancerService";
```

src/services/paymentService.ts

src/services/paymentService.ts

```
// src/services/paymentService.ts

export const calculateRemaining = (
  total: number,
  paid: number
) =&gt; {
  return total - paid;
};
```

src/services/projectService.ts

src/services/projectService.ts

```
// src/services/projectService.ts

export const getProjectStatus = (stage: string) =&gt; {
  return stage;
};
```

src/services/supabase

src/services/supabase/authService.ts

```
import { supabase } from "./client";

export const login = async (email: string, password: string) =&gt; {
  return supabase.auth.signInWithPassword({ email, password });
};

export const logout = async () =&gt; {
  return supabase.auth.signOut();
};

export const getUser = async () =&gt; {
  return supabase.auth.getUser();
};

export const onAuthStateChange = (callback: (event: string, session: any) =&gt; void) =&gt; {
  return supabase.auth.onAuthStateChange(callback);
};
```

src/services/supabase/client.ts

```
// Re-export the auto-generated Supabase client
// IMPORTANT: Do NOT create a new client — use the one managed by Lovable Cloud
export { supabase } from "@/integrations/supabase/client";
```

src/services/supabase/estimateService.ts

```
import { supabase } from "./client";

// Uses the quote_requests table (public estimate form submissions)
export const createEstimate = async (data: any) =&gt; {
  const { data: result, error } = await supabase
    .from("quote_requests")
    .insert([data])
    .select();
  return { result, error };
};

export const getEstimates = async () =&gt; {
  const { data, error } = await supabase
    .from("quote_requests")
    .select("*")
    .order("created_at", { ascending: false });
  return { data, error };
};

// Quotes (sent to clients)
export const getQuotes = async () =&gt; {
  const { data, error } = await supabase
    .from("quotes")
    .select("*, companies(name), projects(name)")
    .order("created_at", { ascending: false });
  return { data, error };
};

export const createQuote = async (quote: any) =&gt; {
  const { data, error } = await supabase
    .from("quotes")
    .insert([quote])
    .select();
  return { data, error };
};
```

src/services/supabase/freelancerService.ts

```
import { supabase } from "./client";

// Freelancer assignments
export const getFreelancerAssignments = async (freelancerId?: string) =&gt; {
  let query = supabase
    .from("freelancer_assignments")
    .select("*, projects(name, companies(name))");

  if (freelancerId) {
    query = query.eq("freelancer_id", freelancerId);
  }

  const { data, error } = await query;
  return { data, error };
};

// Freelancer contracts
export const getFreelancerContracts = async (freelancerId: string) =&gt; {
  const { data, error } = await supabase
    .from("freelancer_contracts")
    .select("*, projects(name)")
    .eq("freelancer_id", freelancerId)
    .order("created_at", { ascending: false });
  return { data, error };
};

// Freelancer deliveries
export const createDelivery = async (delivery: any) =&gt; {
  const { data, error } = await supabase
    .from("freelancer_deliveries")
    .insert([delivery])
    .select();
  return { data, error };
};

export const getDeliveries = async (freelancerId: string) =&gt; {
  const { data, error } = await supabase
    .from("freelancer_deliveries")
    .select("*, projects(name)")
    .eq("freelancer_id", freelancerId)
    .order("created_at", { ascending: false });
  return { data, error };
};
```

src/services/supabase/paymentService.ts

```
import { supabase } from "./client";

// Admin payments (studio income)
export const getAdminPayments = async () =&gt; {
  const { data, error } = await supabase
    .from("admin_payments")
    .select("*, projects(name, companies(name))")
    .order("created_at", { ascending: false });
  return { data, error };
};

// Freelancer payments (outgoing)
export const getFreelancerPayments = async (freelancerId?: string) =&gt; {
  let query = supabase
    .from("freelancer_payments")
    .select("*, projects(name)")
    .order("created_at", { ascending: false });

  if (freelancerId) {
    query = query.eq("freelancer_id", freelancerId);
  }

  const { data, error } = await query;
  return { data, error };
};

export const createAdminPayment = async (payment: any) =&gt; {
  const { data, error } = await supabase
    .from("admin_payments")
    .insert([payment])
    .select();
  return { data, error };
};
```

src/services/supabase/projectService.ts

```
import { supabase } from "./client";

export const getProjects = async () =&gt; {
  const { data, error } = await supabase
    .from("projects")
    .select("*, companies(name)");
  return { data, error };
};

export const getProjectById = async (id: string) =&gt; {
  const { data, error } = await supabase
    .from("projects")
    .select("*, companies(name)")
    .eq("id", id)
    .single();
  return { data, error };
};

export const createProject = async (project: any) =&gt; {
  const { data, error } = await supabase
    .from("projects")
    .insert([project])
    .select();
  return { data, error };
};

export const updateProject = async (id: string, updates: any) =&gt; {
  const { data, error } = await supabase
    .from("projects")
    .update(updates)
    .eq("id", id)
    .select();
  return { data, error };
};
```

src/constants/projectStages.ts

src/constants/projectStages.ts

```
// Unified project stages used across the entire system
// SYSTEM VALUES — stored in DB
export const PROJECT_STAGES = [
  "STOPPED",
  "CANCELED",
  "WAITING_INITIAL_PAYMENT",
  "WAITING_CLIENT_INFO",
  "READY_TO_START",
  "IN_PROGRESS_REV_00",
  "PREVIEW_SENT_REV_00",
  "IN_PROGRESS_REV_01",
  "PREVIEW_SENT_REV_01",
  "IN_PROGRESS_REV_02",
  "PREVIEW_SENT_REV_02",
  "IN_PROGRESS_REV_03",
  "PREVIEW_SENT_REV_03",
  "APPROVED",
  "FINALIZING",
  "COMPLETED",
  "DELIVERED",
] as const;

export type ProjectStage = typeof PROJECT_STAGES[number];

// UI LABELS — display only (PT-BR)
export const PROJECT_STAGE_LABELS: Record&lt;ProjectStage, string&gt; = {
  STOPPED: "Parado",
  CANCELED: "Cancelado",
  WAITING_INITIAL_PAYMENT: "Esperando pagamento inicial",
  WAITING_CLIENT_INFO: "Esperando informações do cliente",
  READY_TO_START: "Pronto para iniciar",
  IN_PROGRESS_REV_00: "Em andamento – REV 00",
  PREVIEW_SENT_REV_00: "Preview REV 00 enviado",
  IN_PROGRESS_REV_01: "Em andamento – REV 01",
  PREVIEW_SENT_REV_01: "Preview REV 01 enviado",
  IN_PROGRESS_REV_02: "Em andamento – REV 02",
  PREVIEW_SENT_REV_02: "Preview REV 02 enviado",
  IN_PROGRESS_REV_03: "Em andamento – REV 03 (final)",
  PREVIEW_SENT_REV_03: "Preview REV 03 enviado",
  APPROVED: "Aprovado",
  FINALIZING: "Em finalização",
  COMPLETED: "Concluído",
  DELIVERED: "Entregue",
};

// Helper to get display label for any stage value (graceful fallback)
export const getStageLabel = (stage: string): string =&gt; {
  return PROJECT_STAGE_LABELS[stage as ProjectStage] || stage;
};

// Color tokens for stage badges / indicators
export const PROJECT_STAGE_COLORS: Record&lt;string, { dot: string; bg: string; text: string; border: string }&gt; = {
  STOPPED: { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" },
  CANCELED: { dot: "bg-destructive", bg: "bg-destructive/10", text: "text-destructive", border: "border-destructive/20" },
  WAITING_INITIAL_PAYMENT: { dot: "bg-chart-4", bg: "bg-chart-4/15", text: "text-chart-4", border: "border-chart-4/30" },
  WAITING_CLIENT_INFO: { dot: "bg-chart-4", bg: "bg-chart-4/15", text: "text-chart-4", border: "border-chart-4/30" },
  READY_TO_START: { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
  IN_PROGRESS_REV_00: { dot: "bg-primary", bg: "bg-primary/10", text: "text-primary", border: "border-primary/20" },
  PREVIEW_SENT_REV_00: { dot: "bg-chart-5", bg: "bg-chart-5/15", text: "text-chart-5", border: "border-chart-5/30" },
  IN_PROGRESS_REV_01: { dot: "bg-primary", bg: "bg-primary/10", text: "text-primary", border: "border-primary/20" },
  PREVIEW_SENT_REV_01: { dot: "bg-chart-5", bg: "bg-chart-5/15", text: "text-chart-5", border: "border-chart-5/30" },
  IN_PROGRESS_REV_02: { dot: "bg-primary", bg: "bg-primary/10", text: "text-primary", border: "border-primary/20" },
  PREVIEW_SENT_REV_02: { dot: "bg-chart-5", bg: "bg-chart-5/15", text: "text-chart-5", border: "border-chart-5/30" },
  IN_PROGRESS_REV_03: { dot: "bg-primary", bg: "bg-primary/10", text: "text-primary", border: "border-primary/20" },
  PREVIEW_SENT_REV_03: { dot: "bg-chart-5", bg: "bg-chart-5/15", text: "text-chart-5", border: "border-chart-5/30" },
  APPROVED: { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
  FINALIZING: { dot: "bg-chart-3", bg: "bg-chart-3/15", text: "text-chart-3", border: "border-chart-3/30" },
  COMPLETED: { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
  DELIVERED: { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
};

// Kanban board stages — the workflow stages shown as columns
export const KANBAN_STAGES = [
  "WAITING_INITIAL_PAYMENT",
  "WAITING_CLIENT_INFO",
  "READY_TO_START",
  "IN_PROGRESS_REV_00",
  "PREVIEW_SENT_REV_00",
  "IN_PROGRESS_REV_01",
  "PREVIEW_SENT_REV_01",
  "IN_PROGRESS_REV_02",
  "PREVIEW_SENT_REV_02",
  "IN_PROGRESS_REV_03",
  "PREVIEW_SENT_REV_03",
  "APPROVED",
  "FINALIZING",
  "COMPLETED",
] as const;

export const KANBAN_STAGE_COLORS: Record&lt;string, { bg: string; border: string; header: string }&gt; = {
  WAITING_INITIAL_PAYMENT: { bg: "bg-chart-4/5", border: "border-chart-4/20", header: "bg-chart-4/15" },
  WAITING_CLIENT_INFO: { bg: "bg-chart-4/5", border: "border-chart-4/20", header: "bg-chart-4/15" },
  READY_TO_START: { bg: "bg-chart-2/5", border: "border-chart-2/20", header: "bg-chart-2/15" },
```

```
  IN_PROGRESS_REV_00: { bg: "bg-primary/5", border: "border-primary/20", header: "bg-primary/15" },
  PREVIEW_SENT_REV_00: { bg: "bg-chart-5/5", border: "border-chart-5/20", header: "bg-chart-5/15" },
  IN_PROGRESS_REV_01: { bg: "bg-primary/5", border: "border-primary/20", header: "bg-primary/15" },
  PREVIEW_SENT_REV_01: { bg: "bg-chart-5/5", border: "border-chart-5/20", header: "bg-chart-5/15" },
  IN_PROGRESS_REV_02: { bg: "bg-primary/5", border: "border-primary/20", header: "bg-primary/15" },
  PREVIEW_SENT_REV_02: { bg: "bg-chart-5/5", border: "border-chart-5/20", header: "bg-chart-5/15" },
  IN_PROGRESS_REV_03: { bg: "bg-primary/5", border: "border-primary/20", header: "bg-primary/15" },
  PREVIEW_SENT_REV_03: { bg: "bg-chart-5/5", border: "border-chart-5/20", header: "bg-chart-5/15" },
  APPROVED: { bg: "bg-chart-2/5", border: "border-chart-2/20", header: "bg-chart-2/15" },
  FINALIZING: { bg: "bg-chart-3/5", border: "border-chart-3/20", header: "bg-chart-3/15" },
  COMPLETED: { bg: "bg-chart-2/5", border: "border-chart-2/20", header: "bg-chart-2/15" },
};

// Stages considered "completed" for financial grouping
export const COMPLETED_STAGES: ProjectStage[] = ["COMPLETED", "DELIVERED"];

// Stages considered "active" (not stopped/canceled/completed)
export const isActiveStage = (stage: string): boolean =&gt; {
  return !["STOPPED", "CANCELED", "COMPLETED", "DELIVERED"].includes(stage);
};
```

src/utils/countryFlags.ts

src/utils/countryFlags.ts

```
// Shared utility for country flags
export const getCountryFlag = (country: string | null | undefined): string =&gt; {
  if (country === "Brazil") return "🇧🇷";
  if (country === "USA") return "🇺🇸";
  return "";
};

export const getLanguageLabel = (lang: string | null | undefined): string =&gt; {
  if (lang === "pt") return "Portuguese";
  return "English";
};
```

src/utils/index.ts

src/utils/index.ts

```
// Utility functions
export {};
```

src/types/index.ts

src/types/index.ts

```
// Shared type definitions
export {};
```

supabase/functions/create-client-account

supabase/functions/create-client-account/index.ts

```
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

const corsHeaders = {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type",
};

Deno.serve(async (req) =&gt; {
  if (req.method === "OPTIONS") {
    return new Response("ok", { headers: corsHeaders });
  }

  try {
    // Verify the caller is an admin
    const authHeader = req.headers.get("Authorization");
    if (!authHeader) {
      return new Response(JSON.stringify({ error: "Missing authorization" }), {
        status: 401, headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }

    const supabaseUrl = Deno.env.get("SUPABASE_URL")!;
    const serviceRoleKey = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;

    // Verify caller is admin using their JWT
    const userClient = createClient(supabaseUrl, Deno.env.get("SUPABASE_ANON_KEY")!, {
      global: { headers: { Authorization: authHeader } },
    });
    const { data: { user: caller } } = await userClient.auth.getUser();
    if (!caller) {
      return new Response(JSON.stringify({ error: "Unauthorized" }), {
        status: 401, headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }

    // Check admin role
    const adminClient = createClient(supabaseUrl, serviceRoleKey);
    const { data: roleData } = await adminClient
      .from("user_roles")
      .select("role")
      .eq("user_id", caller.id)
      .eq("role", "admin")
      .maybeSingle();

    if (!roleData) {
      return new Response(JSON.stringify({ error: "Admin access required" }), {
        status: 403, headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }

    const { email, password, full_name, company_id } = await req.json();

    if (!email || !password || !company_id) {
      return new Response(JSON.stringify({ error: "Missing required fields: email, password, company_id" }), {
        status: 400, headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }

    // Check if user already exists
    const { data: existingUsers } = await adminClient.auth.admin.listUsers();
    const existingUser = existingUsers?.users?.find(u =&gt; u.email === email);

    if (existingUser) {
      // User exists — just link their profile to the company
      await adminClient.from("profiles").update({ company_id }).eq("id", existingUser.id);

      // Ensure they have the 'user' role
      await adminClient.from("user_roles").upsert(
        { user_id: existingUser.id, role: "user" },
        { onConflict: "user_id,role" }
      );

      return new Response(JSON.stringify({
        success: true,
        user_id: existingUser.id,
        message: "Existing user linked to company"
      }), {
        headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }

    // Create new user with auto-confirm
    const { data: newUser, error: createError } = await adminClient.auth.admin.createUser({
      email,
      password,
      email_confirm: true,
      user_metadata: { full_name: full_name || email },
    });

    if (createError) {
      return new Response(JSON.stringify({ error: createError.message }), {
        status: 400, headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
```

```
    }

    // Link profile to company (the handle_new_user trigger creates the profile)
    // Wait briefly for trigger to fire
    await new Promise(resolve =&gt; setTimeout(resolve, 500));

    await adminClient.from("profiles").update({
      company_id,
      full_name: full_name || null,
    }).eq("id", newUser.user!.id);

    // Assign 'user' role
    await adminClient.from("user_roles").insert({
      user_id: newUser.user!.id,
      role: "user",
    });

    return new Response(JSON.stringify({
      success: true,
      user_id: newUser.user!.id,
      message: "Client account created and linked to company"
    }), {
      headers: { ...corsHeaders, "Content-Type": "application/json" },
    });

  } catch (error) {
    return new Response(JSON.stringify({ error: error.message }), {
      status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" },
    });
  }
});
```

src/components/LanguageSelector.tsx

src/components/LanguageSelector.tsx

```
import { useLanguage } from "@/hooks/useLanguage";
import { Button } from "@/components/ui/button";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { Globe } from "lucide-react";

const LanguageSelector = () =&gt; {
  const { language, setLanguage, t } = useLanguage();

  return (
    &lt;DropdownMenu&gt;
      &lt;DropdownMenuTrigger asChild&gt;
        &lt;Button variant="ghost" size="sm" className="gap-2"&gt;
          &lt;Globe className="h-4 w-4" /&gt;
          &lt;span className="hidden sm:inline"&gt;
            {language === "en" ? "■■" : "■■"}
          &lt;/span&gt;
        &lt;/Button&gt;
      &lt;/DropdownMenuTrigger&gt;
      &lt;DropdownMenuContent align="end"&gt;
        &lt;DropdownMenuItem
          onClick={() =&gt; setLanguage("en")}
          className={language === "en" ? "bg-accent" : ""}
        &gt;
          &lt;span className="mr-2"&gt;■■&lt;/span&gt;
          {t("language.english")}
        &lt;/DropdownMenuItem&gt;
        &lt;DropdownMenuItem
          onClick={() =&gt; setLanguage("pt")}
          className={language === "pt" ? "bg-accent" : ""}
        &gt;
          &lt;span className="mr-2"&gt;■■&lt;/span&gt;
          {t("language.portuguese")}
        &lt;/DropdownMenuItem&gt;
      &lt;/DropdownMenuContent&gt;
    &lt;/DropdownMenu&gt;
  );
};

export default LanguageSelector;
```

src/components/NavLink.tsx

src/components/NavLink.tsx

```
import { NavLink as RouterNavLink, NavLinkProps } from "react-router-dom";
import { forwardRef } from "react";
import { cn } from "@/lib/utils";

interface NavLinkCompatProps extends Omit&lt;NavLinkProps, "className"&gt; {
  className?: string;
  activeClassName?: string;
  pendingClassName?: string;
}

const NavLink = forwardRef&lt;HTMLAnchorElement, NavLinkCompatProps&gt;(
  ({ className, activeClassName, pendingClassName, to, ...props }, ref) =&gt; {
    return (
      &lt;RouterNavLink
        ref={ref}
        to={to}
        className={({ isActive, isPending }) =&gt;
          cn(className, isActive &amp;&amp; activeClassName, isPending &amp;&amp; pendingClassName)
        }
        {...props}
      /&gt;
    );
  },
);

NavLink.displayName = "NavLink";

export { NavLink };
```